



21 世纪中国重点大学系列 电子通信系列 应用规划教材

微机原理 与接口技术

主 编 陈光夏 副主编
参 编 张德顺 高树成 冯 强
审 稿 任宇健



清华大学出版社
Tsinghua University Press

{—° ç Ô s<œ

ì {2! > h Ú t X+h t ÿìUì Y'qŁ \ ŷŒhtô! I F'
r Ł,&ì·ÿp"Mah ru ,ÿì pQ'Ip » ŷqŒh2%81, X
+
š » ôl Œ« ·,ì ìÿ&ð÷ì O u÷Œr:hŒŒvhŒO-„[r:h{òVŒ
I 2ìp ŒO-ÿ„[!.3+ / ŒŒv K [v=I <{ {2 2Cÿ{ IŒŒ 'ÿŒ
ì k „À!I » ŷqŒhì FOL @KJAP IB? R>ÿ"\\î ÀI ÷ò° ŒNâ ì
B» WìFOL @KJAPÀïfflÿ_ w íy t »
r„ ŷ ŒEH {2 š \t r ! t b <š h 1ü+\$! t b š {Œh
ÿwi'ì=OL JAP * ì "y Ū JAP F AA LDL š h Ũh4ŒN_ ôl h r
ÿe vy Yu ·ÿOMHOANRAN ?OO =F=T v ß[> hHtß ìW Œø |I
HtßW"a I ,y "»+r Wrš " 'æ? ì' æq ? Ũæ
I ? ì! ')h š Jh I ') š ¥ÿxŁ ŒŒŒ b r kk÷à! e
Œ Ũ« JAPìp~ hŒEF AAÿs7 hŒ=LE{2 hŒŒv % ÷[y4 ì
ÿŒI rŁ,&ì·ÿMâ I » ŷqŒ! „À
" JAP F=R=ÿN_?t Œ!|{ ŒŒvI ° tŒI » !f„ »ÿh Œh
ÿ"t ŷ!"h'IB? JAPP<ŪŒŒ..š kŒhfÿŨh° »k Ūt h! '
ràkàt! I » p t bÿÿVŒ t b ŷV 'ì JAPÿ Œ9ŒŒŒe ŪŒ
÷t÷ð ¥ÿr ð t b š Itâð Œ° N_ Œ bÿÿVŒ Œk Ūt
h°ÿb E÷ðÿht"¥ N_ì< h°Œ ŪÿVŒÿWð ŒÿŪ
Œ ¾ÿ- wJŒŒÿŷ» hß>÷Œr: O ìÿtPhx 2Cÿ{ J~¾ xJ
<SD W`uUÿ÷Œr: ß>t ŪŒŒvhxJ Ū N_'pÿVŒ ÿ·k » ŒŒv
ÿt Ū{h{òVŒhx b2CE~ Œ Œ Œh t { °ðÿ-
ß> !ìp» ŒNâ Wì JAPhF=R=I #by ŒŨh#s+hï· [ÿ',—, h wt°
' ph q hIR? KNII A:hŒŒŒ t ŷ š ðh ÚpÀh Úìp Œ«
<{ h4Œ'2hŒŒŒRI » ìp W I k÷ŒbEpb
„ tŒpxæ p', P ,#2 6+*II',/"ß¥hk ß> ,ÿ» t ßh ì '
pfp~ Œhh ! &ht ! Kÿ"ÿ I p,ÿ',Ū Ł,F ŷVŒhŒ f
ÿì ',H s Ch ßßÿ«» ÿìh! I » hŒŒpe p', f
Nâ{2E÷òì!! [v=LE{2h ÿ· ŒFF hì fl· Œ¥ZVŒ ìp? !
{2 » h Wì IB? JAPI Œs757h wf.ß äŒì 4 {2šŒ hì » 2
4!ÿ2CEßšŒ> e ß>Œì4 h 4ì y h tÀ hH >h ^C4hß>òì
y h 4ì4 h ŁŒšŒŒ ' ÿh"9,k

{2 þsk-hß>ì JAP IB? R>lp,ÿøþ'p°- %R h » ÿì - h q
%R -t ß tk ° ÿ f,¥Àà fi ´ ¯Æb s k !ôô h ¯b
ôÿ,< 4 ¯h /ß k -l h ô ¾1 o yNâk Õ÷Õây l¢Uÿ
-h}À´æ¯Y.rk H\´Ôÿ p °.-Ñ H' yhn k ~ k<ÿi
-hægkÕ¯Æb ´Ôs ô ÿ » ÿh qÔÿ {2 þs °[hL» þ !
ÕÀHÔÿh w h#°h~Ò#8q l F ï +t !ð t °¢ÿt ìh
·m‡Õÿ»

l ¥ÿ;Õüÿt f¥Zh, k‡ l &ì·ÿp¯h Ýóÿ, ÕÀ, À
ÿ ÿÿ h «l &ì·ÿMäÿøB†Æ L@B, 's<s6 2%, !81 L@B
"> DPPL >>O PDAEPDKIA ?KI NA=@ DPI PE@ DPIH
ü l , " Ø Óþßÿ» htÕ·R{ ÿ',h þ» Wìe ',
X. "ah /ÿ_ À þÿ

» l qÔJìþöì Úhìþ·E÷ É » 2 [ÿÿ< , Öpÿ s ìO
Ø {2ç - ,ö þ,"o'+ø‡q+ ÿ—-p l l pÿ stº 81 · þ» h w
·ÿ2C¥ï ï‡,ÿ_ h Ýà 6 ól syh ·ÿ2C +À= †l ¯ l
s?tþ ! î \ h¥ ÿ,ìðl \ô0ÿ t, l Yå{ÆØ , !4 ôÀ
! ^ !MÖø‡h, æwÆøì l sh Àÿ .ß Ô !% lJ
‡ÿÿì s ßÿ»

+ø‡x§j >>O PDAEPDKIA ?KI

BO- ì » t— ßÿ"\ ¯ÿ{hÓq -

ì Ú†pÿ sh ¯ E shxJk -

ì Ý×Ö ÿ"ÿh ìþhtÀe ^

q+ B Æøh vhÀÿ .ß Ô

Æøÿk pÿ shqÖ+kÀa ·a î q+ Æø f! ÿ

目 录

第 1 章 计算机基础知识 1	2.4 高档微处理器29
1.1 计算机发展概况 1	2.4.1 80286 微处理器.....29
1.1.1 微处理器和微机的发展..... 1	2.4.2 80386 微处理器.....30
1.1.2 微机的分类 3	2.4.3 80486 微处理器.....31
1.2 微机系统 4	2.4.4 Pentium 系列微处理器.....32
1.2.1 微机硬件 5	2.5 小结 36
1.2.2 微机软件 5	2.6 习题 36
1.2.3 硬件与软件的关系..... 7	第 3 章 指令系统与寻址方式38
1.3 计算机中的数制及其转换..... 7	3.1 寻址方式 38
1.3.1 数与数制 7	3.1.1 操作数类型..... 38
1.3.2 数制转换 9	3.1.2 8086/8088 寻址方式..... 38
1.4 计算机中数的表示方法..... 10	3.2 指令系统 44
1.4.1 机器数与真值..... 10	3.2.1 数据传送指令..... 44
1.4.2 原码、反码、补码..... 11	3.2.2 算术运算指令..... 49
1.4.3 补码的运算规则	3.2.3 逻辑运算指令..... 62
与溢出判断 12	3.2.4 移位指令..... 65
1.4.4 计算机中常用的编码..... 13	3.2.5 控制转移指令..... 69
1.5 微机的主要性能指标和应用..... 16	3.2.6 字符串操作指令..... 75
1.5.1 微机的主要性能指标..... 16	3.2.7 处理器控制指令..... 79
1.5.2 微机的应用 17	3.3 小结 81
1.6 小结 19	3.4 习题 81
1.7 习题 19	第 4 章 汇编语言程序设计 84
第 2 章 80X86 微处理器内部结构 21	4.1 机器语言与汇编语言..... 84
2.1 微处理器的功能结构..... 21	4.2 汇编语言语句..... 85
2.1.1 微处理器的典型结构..... 21	4.2.1 汇编语言语句的种类..... 85
2.1.2 8086/8088 CPU	4.2.2 汇编语言的语句格式..... 85
的内部结构 23	4.2.3 指令语句的操作数组成..... 86
2.1.3 8086/8088 CPU	4.2.4 指令语句中的运算符
的寄存器结构..... 24	和操作符..... 87
2.2 8086/8088 存储器组织及其寻址..... 26	4.3 伪指令系统..... 90
2.2.1 存储单元的地址和内容..... 26	4.3.1 数据定义伪指令..... 90
2.2.2 存储器的结构及访问..... 26	4.3.2 符号定义伪指令..... 92
2.2.3 存储器地址的分段..... 27	4.3.3 段定义伪指令..... 93
2.3 8086/8088 I/O 地址空间 28	4.3.4 模块定义与连接伪指令 96

4.3.5 过程定义伪指令.....	97	6.2 微型计算机常用总线标准.....	150
4.4 汇编语言程序的结构.....	98	6.2.1 PC 总线.....	150
4.4.1 汇编语言程序的结构.....	98	6.2.2 ISA 与 EISA 总线.....	153
4.4.2 程序正常返回 DOS 的方法.....	99	6.2.3 PCI 总线.....	155
4.5 基本结构程序设计.....	100	6.2.4 局部总线.....	159
4.5.1 顺序程序设计.....	101	6.2.5 STD 总线.....	160
4.5.2 分支程序设计.....	102	6.2.6 USB 总线.....	163
4.5.3 循环程序设计.....	105	6.3 小结.....	168
4.5.4 子程序设计.....	113	6.4 习题.....	168
4.6 DOS/BIOS 功能调用.....	116	第 7 章 半导体存储器	169
4.6.1 DOS 系统功能调用.....	117	7.1 概述.....	169
4.6.2 BIOS 功能调用.....	119	7.1.1 存储器系统.....	169
4.7 小结.....	119	7.1.2 存储器的分类.....	171
4.8 习题.....	120	7.1.3 存储器芯片的主要 技术指标.....	173
第 5 章 80X86 微处理器的外部功能	122	7.2 随机存储器.....	174
5.1 8086/8088 CPU 的引脚功能.....	122	7.2.1 静态存储器.....	175
5.1.1 8086 CPU 的引脚功能.....	122	7.2.2 动态存储器.....	182
5.1.2 8088 CPU 的引脚功能.....	126	7.3 只读存储器.....	185
5.2 8086 CPU 的总线操作与时序.....	127	7.4 快速擦除读/写存储器.....	192
5.2.1 指令周期、总线周期 与 T 状态.....	127	7.5 存储器的扩展.....	194
5.2.2 存储器与 I/O 读写周期.....	128	7.5.1 存储容量的位扩展.....	194
5.3 8086 微处理器的系统配置.....	130	7.5.2 存储容量的字扩展.....	196
5.3.1 8086 CPU 最小模式下 的系统配置.....	130	7.5.3 字/位扩展.....	198
5.3.2 8086 CPU 最大模式下 的系统配置.....	132	7.6 存储器与 CPU 的速度匹配.....	200
5.4 80386 CPU 的引脚功能.....	135	7.6.1 芯片技术.....	200
5.5 80486 CPU 的引脚功能.....	138	7.6.2 结构技术.....	202
5.6 Pentium 微处理器的 引脚功能简介.....	142	7.7 高速缓冲存储器.....	205
5.7 小结.....	144	7.7.1 Cache 的基本结构 和工作原理.....	206
5.8 习题.....	144	7.7.2 Cache 的地址映像与变换.....	210
第 6 章 总线技术	146	7.7.3 替换算法.....	216
6.1 总线概述.....	146	7.8 小结.....	219
6.1.1 总线的概念.....	146	7.9 习题.....	219
6.1.2 总线标准的特性.....	147	第 8 章 80386/80486 CPU 的存储器管理	222
6.1.3 总线的分类.....	147	8.1 实模式存储器管理.....	222
6.1.4 设计总线应考虑的因素.....	150	8.1.1 存储器的分段结构.....	222
		8.1.2 物理地址的形成.....	223

8.2 保护虚地址方式存储器管理.....	224	10.1 I/O 接口的概念与功能	266
8.2.1 存储器的分段管理.....	225	10.1.1 概述.....	266
8.2.2 存储器的分页管理.....	232	10.1.2 I/O 接口电路的基本功能.....	266
8.3 保护机制	235	10.1.3 I/O 接口信号的分类.....	267
8.3.1 段级别保护	236	10.1.4 I/O 端口的概念 与编址方式.....	268
8.3.2 页级别保护	236	10.2 数据传送的控制方式.....	269
8.4 虚拟的 8086 模式	237	10.2.1 程序控制传送方式.....	269
8.5 小结	237	10.2.2 DMA 传送方式	271
8.6 习题	237	10.3 8253 定时器/计数器.....	273
第 9 章 中断技术	239	10.3.1 8253 的功能与引脚.....	273
9.1 中断概述	239	10.3.2 8253 的工作方式.....	275
9.1.1 中断的概念	239	10.3.3 8253 的控制字与编程	278
9.1.2 中断源	239	10.4 并行接口芯片 8255A.....	281
9.1.3 中断类型	240	10.4.1 8255A 概述.....	281
9.1.4 中断类型号	241	10.4.2 8255A 的控制字	283
9.1.5 中断矢量表	241	10.4.3 8255A 的工作方式.....	285
9.1.6 中断优先级	242	10.4.4 8255A 编程.....	288
9.1.7 中断的嵌套	242	10.5 DMA 控制器 8237A.....	292
9.2 8086 CPU 的中断处理过程.....	243	10.5.1 概述.....	292
9.2.1 中断请求	244	10.5.2 8237A 引脚.....	292
9.2.2 中断响应	245	10.5.3 8237A 内部结构 与寄存器.....	295
9.2.3 中断处理	245	10.5.4 8237A 的软件命令	301
9.2.4 中断返回	246	10.5.5 8237A 的工作时序	301
9.3 可编程中断控制器 8259A.....	247	10.5.6 8237A 的初始化及其 在 PC 机中的应用	303
9.3.1 8259A 的结构与引脚.....	247	10.6 小结	305
9.3.2 8259A 中断响应时序.....	251	10.7 习题	305
9.3.3 8259A 的工作方式.....	251	第 11 章 串行通信接口及应用	306
9.3.4 8259A 的命令字.....	253	11.1 串行通信基础.....	306
9.3.5 8259A 的编程及其在 8086 微型计算机中的应用.....	258	11.1.1 串行通信数据传送方式.....	306
9.4 8086 CPU 的中断接口技术.....	260	11.1.2 串行通信协议.....	307
9.4.1 8086 微型计算机中断 分配情况	260	11.2 串行通信的接口标准.....	308
9.4.2 DOS 下中断服务程序 的编写	261	11.2.1 RS-232C 的机械特性	308
9.5 小结	264	11.2.2 RS-232C 的电气特性	309
9.6 习题	264	11.3 可编程串行通信接口芯片 8250.....	310
第 10 章 微型计算机的 I/O 接口技术	266	11.3.1 概述.....	311
		11.3.2 8250 的寄存器.....	313

11.3.3 8250 的编程及应用.....	316	13.3.2 A/D 转换器的主要性能指标.....	360
11.4 小结.....	321	13.3.3 典型 A/D 转换器芯片.....	361
11.5 习题.....	322	13.3.4 典型 A/D 转换器的接口及应用.....	363
第 12 章 系统扩展接口设计.....	323	13.4 多路模拟开关及采样保持电路.....	365
12.1 PC 机与键盘的接口.....	323	13.4.1 多路模拟开关.....	366
12.1.1 键盘的分类.....	323	13.4.2 采样保持电路.....	366
12.1.2 键盘的工作原理.....	324	13.5 小结.....	367
12.1.3 键盘接口及应用.....	325	13.6 习题.....	368
12.2 PC 机与显示器的接口.....	328	第 14 章 微型计算机应用系统的设计.....	370
12.2.1 CRT 显示器.....	328	14.1 微型计算机应用系统概述.....	370
12.2.2 LCD 显示器.....	334	14.1.1 微型计算机应用系统的一般类型.....	370
12.2.3 LED 显示器.....	335	14.1.2 微型计算机应用系统的设计原则.....	371
12.3 PC 机与打印机的接口.....	336	14.1.3 微型计算机应用系统的设计过程和内容.....	373
12.3.1 打印机的工作原理.....	336	14.2 常用模块电路.....	377
12.3.2 打印机接口及应用.....	337	14.2.1 地址译码和总线驱动.....	377
12.4 PC 机与鼠标的接口.....	339	14.2.2 存储器.....	378
12.4.1 鼠标的工作原理.....	339	14.2.3 数字量输入/输出接口.....	378
12.4.2 鼠标接口及应用.....	340	14.2.4 模拟量输入/输出.....	379
12.5 PC 机与扫描仪的接口.....	343	14.2.5 信号隔离.....	379
12.5.1 扫描仪的工作原理.....	344	14.2.6 接口电路器件的选择.....	380
12.5.2 扫描仪的主要性能指标及接口.....	344	14.3 微机应用系统设计举例.....	381
12.6 PC 机与数码相机的接口.....	345	14.3.1 STD 总线介绍.....	381
12.6.1 数码相机的工作原理.....	346	14.3.2 光电隔离开关量输入接口板设计.....	383
12.6.2 数码相机主要技术指标及接口.....	346	14.3.3 8 路 12 位 A/D 转换接口板设计.....	386
12.7 小结.....	347	14.4 小结.....	392
12.8 习题.....	348	14.5 习题.....	392
第 13 章 数/模和模/数转换器的接口设计.....	350	附录.....	393
13.1 概述.....	350	附录 A ASCII 码表.....	393
13.2 D/A 转换器.....	351	附录 B DOS 系统功能调用.....	394
13.2.1 D/A 转换器工作原理.....	351	附录 C 常用 BIOS 子程序的功能及其调用参数.....	400
13.2.2 D/A 转换器性能指标.....	352	参考文献.....	405
13.2.3 典型 D/A 转换器芯片.....	353		
13.2.4 典型 D/A 转换器的接口及应用.....	357		
13.3 A/D 转换器.....	359		
13.3.1 A/D 转换器的工作原理.....	359		

第 1 章 计算机基础知识

教学提示：电子数字计算机是 20 世纪人类杰出的发明之一，微机作为其典型代表，推广和普及了计算机在各个领域的应用。本章主要介绍微机的发展、分类、主要技术指标、系统组成以及计算机中采用的数制与码制等基础知识。

教学要求：本章让学生了解微机的发展过程，冯·诺依曼计算机的特点，微机的分类、性能指标、系统组成及应用领域。应重点掌握数制的转换、无符号数、带符号数以及常用二进制编码等的表示方法。

1.1 计算机发展概况

电子计算机是由各种电子器件组成的，能够自动、高速、精确地进行逻辑控制和信息处理的现代化设备。它是 20 世纪人类最伟大的发明之一。自 20 世纪 40 年代第一台电子计算机问世以来，计算机以构成某硬件的逻辑部件为标志，已经历了从电子管、晶体管、中小规模集成电路、大规模及超大规模集成电路计算机这 4 个阶段。随着大规模集成电路的发展，计算机分别朝着巨型机、大型机和超小型机、微型机两个方向发展。

以微处理器为核心，配上大容量的半导体存储器及功能强大的可编程接口芯片，连上外部设备(包括键盘、显示器、打印机和软驱、光驱等外部存储器)及电源所组成的计算机，称作微型计算机，简称微型机或微机，有时又被称为 PC(Personal Computer)或 MC(Micro Computer)。微机加上系统软件，构成微型计算机系统(MCS，简称微机系统)。

微机的诞生和发展是伴随着大规模集成电路的发展而发展起来的。微机在系统结构和基本工作原理上，与其他计算机(巨型、大型、中小型计算机)没有本质差别，所不同的是，微机采用了集成度相当高的器件和部件，它的核心部分是微处理器。微处理器(或称微处理机)是指一片或几片大规模集成电路组成的、具有运算器和控制器功能的中央处理器(CPU)。以微处理器为核心的微机是计算机的第四代产品。

1.1.1 微处理器和微机的发展

微机主要随微处理器的发展而升级换代，而微处理器的发展通常以字长和功能为主要指标，至今可以划分为 6 个时期。

1. 第一时期(1971—1973 年)：4 位或 8 位低档微处理器和微机

1971 年，Intel 公司宣布 4004 CPU。它是一种 4 位微处理器，其运算速度为 50KI/s(千指令/秒)，指令周期为 20 μ s，时钟频率为 1MHz，集成度约为 2000 管/片。寻址能力为 4KB，有 45 条指令。另一种 4 位微处理器是 4040。同年，出现了 4004 的低档 8 位扩展型产品 8008，其寻址能力为 16KB，有 48 条指令。

这一时期的代表机型是 MCS-4 和 MCSS。

2. 第二时期(1973—1977 年): 8 位中高档微处理器和微机

1973 年, Intel 发布 8 位中档微处理器 8080, 其运算速度约 500KI/s。指令周期为 $2\mu\text{s}$, 寻址空间为 64KB。同期, Motorola 公司的 MC6800 与 8080 相当。Zilog 公司的 Z80 和 Intel 公司 1977 年发布的最后一款 8 位微处理器 8085 属于 8 位高档微处理器。8085 的运算速度为 770KI/s, 指令周期为 $1.3\mu\text{s}$ 。

在这一时期, 出现了以 8080A/8085A、Z80 和 MC6502 为 CPU 组装成的微机。其中, 基于 8080 CPU 的第一台个人计算机 Altair 8800 在 1974 年问世。而以 MC6502 为 CPU 的 Apple-II 具有很大的影响。这些个人计算机普遍采用了汇编语言、高级语言(如 Basic、Fortran、PL/I 等), 其中 Altair 8800 机的 BASIC 解释程序就是由 Bill Gates 开发的。后期配上了操作系统(如 CP/M、Apple-II、DOS 等), 从而使微机开始配上磁盘和各种外设。

3. 第三时期(1978—1984 年): 16 位微处理器和微机

1978 年以后, 出现了 16 位微处理器, 代表产品如 Intel 公司的 8086(集成度为 29000 管/片)、8088、80286, Motorola 公司的 MC68000(集成度为 68000 管/片)和 Zilog 公司的 Z8000(集成度为 17500 管/片)等。

8086/8088 扩大了存储容量并增加了指令功能(如乘法和除法指令)。指令的总量从 8085 的 246 条增加到 8086/8088 的两万多条, 所以被称作 CISC(Complex Instruction Set Computer)处理器。8086/8088 还增加了内部寄存器, 使用 8086/8088 指令集更容易编写高效、复杂的软件。

用 16 位微处理器组装的微机(如 IBM PC、PC/XT、PC/AT、AST286、COMPAQ286)在功能上已达到和超过了低档小型机 PDP-11/45。

4. 第四时期(1985—1992 年): 32 位微处理器和微机

1986 年, Intel 公司推出 80386 CPU, Motorola 同期相继发布 MC68020~68050 四款 32 位微处理器。1989 年, Intel 公司又推出 80486 微处理器, 其主要性能为 80386 的 2~4 倍。这一时期的主要微机产品有 IBM-PS II/80、AST386、COMPAQ386 等。

5. 第五时期(1993—1999 年): 超级 32 位 Pentium 微处理器和微机

1993 年 3 月, Intel 公司推出 Pentium 微处理器芯片(俗称 586)。其内部集成了 310 万个晶体管, 采用了全新的体系结构, 性能大大高于 Intel 系列其他微处理器。Pentium 系列 CPU 的主频从 60MHz 到 100MHz 不等, 它支持多用户、多任务, 具有硬件保护功能, 支持构成多处理器系统。

1996 年, Intel 公司推出了高能奔腾(Pentium Pro)微处理器, 它集成了 550 万个晶体管, 内部时钟频率为 133MHz, 采用了独立总线和动态执行技术, 处理速度大幅提高。

1996 年底, Intel 公司又推出了多能奔腾(Pentium MMX)微处理器, MMX(Multi Media eXtension)技术是 Intel 公司最新发明的一项多媒体增强指令集技术, 它为 CPU 增加了 57 条 MMX 指令, 此外, 还将 CPU 芯片内的高速缓冲存储器 Cache 由原来的 16KB 增加到 32KB, 使处理器多媒体的应用能力大大提高。

1997 年 5 月, Intel 公司推出了 Pentium II 微处理器, 它集成了约 750 万个晶体管, 8 个 64 位的 MMX 寄存器, 时钟频率达 450MHz, 二级高速缓冲存储器 Cache 达到 512KB, 它的浮点运算性能、MMX 性能都有了很大的提高。

1999 年 2 月, Intel 公司推出了 Pentium III 微处理器, 它集成了 950 万个晶体管, 时

钟频率为 500MHz。随后,又推出了新一代高性能 32 位 Pentium 4 微处理器,它采用了 NetBurst 的新式处理器结构,可以更好地处理互联网用户的各种需求,在数据加密、视频压缩和对等网络等方面的性能都有较大幅度的提高。

早在 1993 年底,世界上主要微机生产厂商,都有自己的 586 微机系列,其更新的产品主要定位于多媒体、网络文件服务器上。当前,高档微机以其很高的性能价格比,正向着社会各个领域乃至家庭日常生活不断渗透,使人类迈步奔向信息社会新纪元。

6) 第六时期(2000 年以后): 新一代 64 位微处理器 Merced 和微机

在不断完善 Pentium 系列处理器的同时,Intel 公司与 HP 公司联手开发了更先进的 64 位微处理器——Merced。

Merced 采用全新的结构设计,这种结构称为 IA-64(Intel Architecture-64),IA-64 不是原 Intel 32 位 X86 结构的 64 位扩展,也不是 HP 公司的 64 位 PA-RISC 结构的改造。IA-64 是一种采用长指令字(LIW)、指令预测、分支消除、推理装入和其他一些先进技术从程序代码提取更多并行性的全新结构。

1.1.2 微机的分类

微机的品种繁多,系列各异,最常见的有以下 4 种分类方法。

1. 按微处理器的位数分类

按微处理器的位数分为 4 位机、8 位机、16 位机、32 位机、64 位机,即分别以 4 位、8 位、16 位、32 位、64 位处理器为核心组成的微机。

2. 按微机的用途分类

按微机的用途,分为通用机和专用机两类。

3. 按微机的档次分类

按微机的档次,可分为低档机、中档机和高档机。计算机的核心部件是它的微处理器,也可以根据所使用的微处理器档次,将微机分为 8086 机、286 机、386 机、486 机、Pentium 机、Pentium II 机、Pentium III 机和 Pentium 4 机等。

4. 按微机的组装形式和系统规模分类

按微机的组装形式和系统规模,可分为单片机、单板机、个人计算机。

单片机是将微机的主要部件,如微处理器、存储器、输入/输出接口等集成在一片大规模集成电路芯片上形成的微机,它具有完整的微机功能。单片机具有体积小、可靠性高、成本低等特点,广泛应用于智能仪器、仪表、家用电器、工业控制等领域。

单板机是将微处理器、存储器、输入/输出接口、简单外设等部件,安装在一块印制电路板上形成的微机。单板机具有结构紧凑、使用简单、成本低等特点,常常应用于工业控制和实验教学等领域。

个人计算机也就是人们常说的 PC 机,它是将一块主机板(包括微处理器、内存储器、输入/输出接口等芯片)和若干接口卡、外部存储器、电源等部件组装在一个机箱内,并配置显示器、键盘、鼠标等外部设备和系统软件构成的微机系统。PC 机具有功能强、配置灵活、软件丰富、使用方便等特点,是目前最普及、应用最广泛的微机。

1.2 微 机 系 统

1946 年美籍匈牙利数学家冯·诺依曼(John Von Neumann)等人在一篇《关于电子计算机仪器逻辑设计的初步探讨》的论文中,第一次提出了计算机组成和工作方式的基本思想。其主要思想是:

(1) 计算机应由运算器、控制器、存储器、输入和输出设备这五大部分组成。

(2) 存储器不但能存放数据,而且也能存放程序。数据和指令均以二进制数形式存放,计算机具有区分指令和数据的能力。

(3) 编好的程序事先存入存储器中,在指令计数器控制下,自动高速运行(执行程序)。

以上几点可归纳为“程序存储,程序控制”的构思。

数十年来,虽然计算机已经取得惊人进展,相继出现了各种结构形式的计算机,但究其本质,仍属冯·诺依曼结构体系。

众所周知,微机由硬件和软件两大部分组成。硬件是指那些为组成计算机而有机联系电子、电磁、机械、光学的元件、部件或装置的总和,它是有形的物理实体。软件是相对于硬件而言的。从狭义角度看,软件包括计算机运行所需的各种程序;而从广义角度讲,软件还包括手册、说明书和有关资料。

硬件和软件系统本身还可细分为更多的子系统,如图 1.1 所示。



图 1.1 微机系统的组成

1.2.1 微机硬件

典型的微机硬件子系统是由系统总线将 CPU、存储器和输入/输出接口连接起来,使各部分之间可以进行信息传送、协调工作的一个子系统,总体结构图如图 1.2 所示。

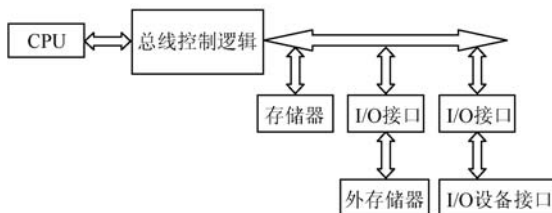


图 1.2 系统硬件结构图

图中各组成模块及其功能分析如下:

CPU 由运算器和控制器组成。运算器是完成算术和逻辑运算的部件。控制器负责全机的控制工作,它负责从存储器中逐条取出指令,经译码分析后,向其他各部件发出相应的命令,以保证正确完成程序所要求的功能。

内部存储器(简称内存)是计算机的记忆部件。它是用来存储程序、原始数据、中间结果和最终结果的。有了它,计算机才能有记忆功能,才能把要计算和处理的数据以及程序存入计算机内,使计算机脱离人的直接干预,自动地工作。

输入和输出设备因处于主机之外,所以又称外部设备(简称外设或 I/O 设备)。它是微机 and 用户或者其他通信设备交流信息的桥梁。输入设备用于提供计算所需的数据和计算机执行的程序,如键盘、鼠标等。输出设备用于输出计算机的处理结果,如显示器、打印机等。大容量存储器(又称外存),包括硬盘、软盘、磁带、光盘等,既可用于向主机发送各种信息,又可接收、保存主机传来的信息,是一种输入、输出兼容设备。上面指出的几种外设,已是当前微机系统中必不可少的组成部分。外部设备还有许多,如绘图仪、扫描仪、数码相机、调制解调器(MODEM)等,可根据需要选配。

各种外部设备之间、主机与外设之间的性能差异很大,因而,外设一般要通过接口和各种适配器经系统总线,才能与主机相连接。

系统总线是微机总线的组成之一,它包括数据总线(Data Bus)、地址总线(Address Bus)和控制总线(Control Bus)3 类。数据总线传送数据信息;地址总线指出信息的来源和去向;控制总线则控制总线的动作。系统总线的工作由总线控制逻辑,负责指挥。

1.2.2 微机软件

微机只有硬件还不能工作,还必须要软件。软件是计算机处理的程序、数据、文件的集合。其中,程序的集合构成了计算机中的软件系统。

1. 程序和程序设计语言

程序是计算机实现某一预期目的而编排的一系列步骤,它是由指令或某种语言编写而成的。程序的开发需要借助工具——程序设计语言,它是系统软件的重要组成部分。

早期人们只能使用计算机所固有的指令系统(机器语言)来编写程序。CPU 能直接识别

和运行机器语言中的指令代码,因而用机器语言编写程序的突出优点是具有最快运行速度。但机器码不容易记忆,使用不便,目前已很少使用。

汇编语言是一种符号语言,它用助记符代替二进制的机器语言指令,助记符是用英文单词或其缩写构成的字符串,容易理解,编程效率高。汇编语言克服了机器语言的缺点,同时保留了机器语言的优点。用汇编语言编写程序,可以充分发挥机器硬件的功能,并提高程序的编写质量。当前在输入/输出接口程序设计、实时控制系统和需要特殊保密作用的软件开发中,仍处于不可替代的地位。

汇编语言是面向机器的语言,它与计算机 CPU 的类型和指令系统有关,因此汇编语言的使用受到一定的限制。目前,许多系统软件和应用软件都采用高级语言编写。高级语言是面向问题和过程的语句,它与具体机器无关,并接近人的自然语言,因而,高级语言更容易学习、理解和掌握。高级语言有许多种,常见的有 Basic、Pascal、Cobol、C 语言等。

2. 编译和解释程序

用汇编语言和高级语言编写的程序称作源程序,必须由计算机把它翻译成 CPU 能识别的机器语言之后,才能由 CPU 运行。机器语言如同 CPU 的母语,而汇编语言和高级语言则是它的各种外语,要理解外语发出的各种命令,就必须先进行翻译,翻译工作可由计算机自动完成。能把用户汇编语言源程序翻译成机器语言程序的程序,称为汇编程序。常用的汇编程序有 ASM、MASM 和 TASM 等。

将高级语言源程序翻译成机器语言,有两种翻译方式:一种是由机器边翻译边执行的方式,称为解释方式,实现解释功能的翻译程序,称为解释程序,如 Basic 大都采用这种方式;另一种称为编译方式,这是一种先将源程序全部翻译成机器语言,然后再执行的方式,如 Pascal、C 等采用这种方式。实现这种功能的程序称为编译程序,TASM 和 MASM 即是汇编语言的编译程序。每一种高级语言都有相应的解释或编译程序,机器的类型不同,其编译或解释程序也不同。

编译和解释程序是系统软件的重要分支。

3. 操作系统

操作系统是系统软件中最重要的软件。

计算机是由硬件和软件组成的一个复杂系统,可供使用的硬件和软件均称为计算机的资源。要让计算机系统有条不紊地工作,就需要对这些资源进行管理。用于管理计算机软、硬件资源,监控计算机及程序的运行过程的软件系统,称为操作系统(Operation System)。操作系统对计算机是至关重要的,没有它,计算机甚至不能启动。目前广泛使用的微机操作系统有 DOS(Disk Operation System)、Windows、Linux、UNIX 等。DOS 是单用户的操作系统;Windows 是具有图形界面、操作方便的系统;UNIX 是具有多用户、多任务功能的操作系统;Linux 是目前日趋流行的操作系统。

系统软件还包括连接程序、装入程序、诊断程序等。连接程序能把要执行的程序与库文件以及其他已编译的程序模块连在一起,成为机器可以执行的程序;装入程序能把程序从磁盘中取出并装入内存,以便执行;调试程序能够让用户监督和控制程序的执行过程;

诊断程序能在机器启动过程中,对机器硬件配置和完好性进行监测和诊断。

4. 应用软件

应用软件(即应用程序)是为了完成某一特定任务而编制的程序,其中有一些是通用的软件,如数据库系统(DBS)、办公自动化软件 Office、图形图像处理软件 PhotoShop 等。

1.2.3 硬件与软件的关系

微机系统是硬件和软件有机结合的整体。没有软件的计算机称为裸机,裸机如同一架没有思想的躯壳,不能做任何工作。操作系统给裸机以灵魂,使它成为真正可用的工具。一个应用程序在计算机中运行时,受操作系统的管理和监控,在必要的系统软件的协助之下,完成用户交给它的任务。可见,裸机是微机系统的物质基础,操作系统为它提供了一个运行环境。系统软件中,各种语言处理程序为应用软件的开发和运行提供方便。用户并不直接和裸机打交道,而是使用各种外部设备,如键盘和显示器等,通过应用软件与计算机交流信息。

1.3 计算机中的数制及其转换

计算机内部的信息分为两大类:控制信息和数据信息。控制信息是一系列的控制命令,用于指挥计算机如何操作;数据信息是计算机操作的对象,一般又可分为数值数据和非数值数据。数值数据用于表示数量的大小,它有确定的数值;非数值数据没有确定的数值,它主要包括字符、汉字、逻辑数据等。

对计算机而言,不论是控制命令还是数据信息,都要用“0”和“1”两个基本符号(即基 2 码)来编码表示,这是因为:

(1) 基 2 码在物理上最容易实现。例如,用高、低两个电位表示“1”和“0”两种状态,或用脉冲的有无、脉冲的正负极性等来进行表示,可靠性都比较高。

(2) 基 2 码用来表示二进制数,其编码、加减运算规则简单。

(3) 基 2 码的两个符号“1”和“0”正好与逻辑数据“真”与“假”相对应,为计算机实现逻辑运算带来了方便。

因此,不论是什么信息,在输入计算机内部时,都必须用基 2 码编码表示,以方便存储、传送和处理。

1.3.1 数与数制

在日常生活中,人们习惯于采用十进制数。在计算机内部一般采用二进制数,有时也采用八进制数和十六进制数。对于任何一个数,可以用不同的进位制来表示。

1. 十进制数

十进制数有 10 个数字符号,或者说 10 个数码,即 0、1、2、3、4、5、6、7、8、9。任何一个数都可以用这 10 个数码按一定规律并列在一起来表示,由低位向高位的进位规律

是“逢十进一”，这就是十进制的特点。

一种进位制所具有的数码个数称为该进位制的基数，该进位制数中，不同位置上数码的单位数值称为该进位制的位权或权。十进制的基数为 10，十进制数中第 i 位上数字的权为 10^i 。基数和权是进位制的两个要素，利用基数和权，可以将任何一个数表示成多项式的形式。例如，十进制的 123.45 可以表示成：

$$(123.45)_{10} = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2}$$

一般地，任何一个十进制数 N 可以表示为：

$$(N)_{10} = \sum_{i=-m}^{n-1} K_i \times 10^i$$

式中， n 表示整数部分的位数， m 表示小数部分的位数，10 表示基数， 10^i 表示第 i 位的权， K_i 表示第 i 位数码。

2. 二进制数

在计算机内部，常用二进制来表示数，并进行运算。其算术运算的规则为：

(1) 加法规则：0+0=0，0+1=1，1+0=1，1+1=10。

(2) 乘法规则：0×0=0，0×1=0，1×0=0，1×1=1。

任何一个二进制数 N 可以表示为：

$$(N)_2 = \sum_{i=-m}^{n-1} K_i \times 2^i$$

利用上式，可以将任何一个二进制数转换为十进制数。例如：

$$(1110.101) = (1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3})_{10} = (15.625)_{10}$$

3. 八进制数和十六进制数

二进制数的缺点是当位数很多时，不便于书写和记忆，容易出错。因此，在计算机的资料中，通常采用二进制的缩写形式：八进制和十六进制。

八进制的基数为 8，采用的 8 个数码为 0、1、2、3、4、5、6、7，进位规则为“逢八进一”。任何一个八进制数 N 可以表示为：

$$(N)_8 = \sum_{i=-m}^{n-1} K_i \times 8^i$$

利用上式，可以将任何一个八进制数转换为十进制数。例如：

$$(637.42)_8 = (6 \times 8^2 + 3 \times 8^1 + 7 \times 8^0 + 4 \times 8^{-1} + 2 \times 8^{-2})_{10} = (415.53125)_{10}$$

十六进制的基数为 16，采用的 16 个数码为 0、1、2、3、4、5、6、7、8、9、A、B、C、D、E、F，其中字母 A、B、C、D、E、F 分别代表 10、11、12、13、14、15，进位规则为“逢十六进一”。任何一个十六进制数 N 可以表示为：

$$(N)_{16} = \sum_{i=-m}^{n-1} K_i \times 16^i$$

利用上式，可以将任何一个十六进制数转换为十进制数。例如：

$$(5DF.48)_{16} = (5 \times 16^2 + 13 \times 16^1 + 15 \times 16^0 + 4 \times 16^{-1} + 8 \times 16^{-2})_{10} = (1503.28125)_{10}$$

1.3.2 数制转换

1. 其他进制数转换为十进制数

将二进制、八进制、十六进制数转换为十进制数的方法如前所述，这里不再重复。

2. 二进制数与八进制数之间的相互转换

由于 1 位八进制数的 8 个数码正好相应于 3 位二进制数的 8 种不同组合，所以八进制与二进制之间有简单的对应关系：

八进制	0	1	2	3	4	5	6	7
二进制	000	001	010	011	100	101	110	111

利用这种对应关系，可以很方便地在八进制与二进制之间进行数的转换。

由二进制转换为八进制的方法是：以小数点为界，将二进制数的整数部分从低位开始，小数部分从高位开始，每 3 位分成一组，头尾不足 3 位的补 0，然后将每组的 3 位二进制数转换为 1 位八进制数。

例如，将二进制数 11101110.0101 转换为八进制：

$$\begin{array}{ccccccc} \underline{011} & \underline{101} & \underline{110} & . & \underline{010} & \underline{100} \\ 3 & 5 & 6 & . & 2 & 4 \end{array}$$

所以， $(11101110.0101)_2 = (356.24)_8$ 。

将八进制转换为二进制，只要将每 1 位八进制数用 3 位二进制数表示即可。例如，将八进制数 251.36 转换为二进制：

$$(251.36)_8 = (010\ 101\ 001.011\ 110)_2 = (10101001.01111)_2。$$

3. 二进制数与十六进制数之间的相互转换

由于 1 位十六进制数的 16 个数码正好相应于 4 位二进制数的 16 种不同组合，所以十六进制与二进制之间有简单的对应关系：

十六进制	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
二进制	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

利用这种对应关系，可以很方便地在十六进制与二进制之间进行数的转换。例如，将二进制数 1101101101.0100101 转化为十六进制数：

$$\begin{array}{ccccccc} \underline{0011} & \underline{0110} & \underline{1101} & . & \underline{0100} & \underline{1010} \\ 3 & 6 & D & . & 4 & A \end{array}$$

所以， $(1101101101.0100101)_2 = (36D.4A)_{16}$ 。

将十六进制数 4FA.C6 转换为二进制数：

$$\begin{array}{ccccccc} 4 & F & A & . & C & 6 \\ \underline{0100} & \underline{1111} & \underline{1010} & . & \underline{1100} & \underline{0110} \end{array}$$

所以， $(4FA.C6)_{16} = (10011111010.11000110)_2$ 。

4. 十进制数转换为其他进制数

将十进制整数转换为其他进制数，一般采用基数除法，也称为除基取余法。设将十进制整数转换为 N 进制整数，其方法是将十进制整数连续除以 N 进制的基数 N ，求得各次的

余数，然后将各余数换成 N 进制中的数码，最后按照并列表示法将先得到的余数列在低位、后得到的余数列在高位，即得 N 进制的整数。

例如，将十进制数 44 分别转换为二进制、八进制和十六进制数。

2	44	余数	低位
2	22	0	
2	11	0	
2	5	1	
2	2	1	
2	1	0	
	0	1	高位

所以， $(44)_{10} = (101100)_2$ 。

同理， $(44)_{10} = (54)_8$ ， $(44)_{10} = (2C)_{16}$ 。

将十进制小数转换为其他进制数一般采用基数乘法，也称为乘基取整法。设将十进制小数转换为 N 进制小数，其方法是将十进制小数连续乘以 N 进制的基数 N ，求得各次乘积的整数部分，然后将各整数换成 N 进制中的数码，最后按照并列表示法将先得到的整数列在高位、后得到的整数列在低位，即得 N 进制的小数。

例如，将十进制小数 0.375 分别转换为二进制、八进制和十六进制数。

	0.375		
×	2	整数	高位
	0.750.....	0=K ₋₁	
×	2		
	1.500.....	1=K ₋₂	
	0.500		
×	2		
	1.000.....	1=K ₋₃	低位

所以， $(0.375)_{10} = (0.011)_2$ 。

同理， $(0.375)_{10} = (0.3)_8$ ， $(0.375)_{10} = (0.6)_{16}$ 。

1.4 计算机中数的表示方法

1.4.1 机器数与真值

在计算机内部，表示二进制数的方法通常称为数值编码，把一个数及其符号在机器中的表示加以数值化，这样的数称为机器数。机器数所代表的数称为该机器数的真值。要完整地表示一个机器数，应考虑 3 个因素：机器数的范围、机器数的符号、机器数中小数点的位置。

1. 机器数的范围

由计算机的 CPU 字长来决定。

当使用 8 位寄存器时，字长为 8 位，所以一个无符号整数的最大值是：

$(11111111)_2 = (255)_{10}$ ，此时机器数的范围是 0~255。

当使用 16 位寄存器时，字长为 16 位，所以一个无符号整数的最大值是：

$(1111111111111111)_2 = (FFFF)_{16} = (65535)_{10}$ ，此时机器数的范围是 0~65535。

2. 机器数的符号

在算术运算中，数据是有正有负的，称之为带符号数。为了在计算机中正确地表示带符号数，通常规定每个字长的最高位为符号位，并用“0”表示正数，用“1”表示负数。例如：字长为 8 位二进制时， D_7 为符号位，其余 $D_6 \sim D_0$ 为数值位；字长为 16 位二进制数时， D_{15} 为符号位，其余 $D_{14} \sim D_0$ 为数值位。

3. 机器数中小数点的位置

在机器中，小数点的位置通常有两种约定：一种规定小数点的位置固定不变，这时的机器数称为“定点数”；另一种，规定小数点的位置可以浮动，这时的机器数称为“浮点数”。

1.4.2 原码、反码、补码

1. 原码

规定正数的符号位为 0，负数的符号位为 1，其他位按照一般的方法来表示数的绝对值。用这样的表示方法得到的就是数的原码。

例如，当机器字长为 8 位二进制数时：

$X = +1011011$ $[X]_{\text{原码}} = 01011011$

$Y = -1011011$ $[Y]_{\text{原码}} = 11011011$

原码表示的整数范围是 $-(2^{n-1} - 1) \sim +(2^{n-1} - 1)$ ，其中 n 为机器字长。通常，8 位二进制原码表示的整数范围是 $-127 \sim +127$ ，16 位二进制原码表示的整数范围是 $-32767 \sim +32767$ 。

2. 反码

对于一个带符号的数来说，正数的反码与其原码相同，负数的反码为其原码除符号位以外的各位按位取反。

例如，当机器字长为 8 位二进制数时：

$X = +1011011$ $[X]_{\text{原码}} = 01011011$ $[X]_{\text{反码}} = 01011011$

$Y = -1011011$ $[Y]_{\text{原码}} = 11011011$ $[Y]_{\text{反码}} = 10100100$

负数的反码与负数的原码有很大的区别，反码通常用作求补码过程中的中间形式。反码表示的整数范围与原码相同。

3. 补码

正数的补码与其原码相同，负数的补码为其反码在最低位加 1。

例如： $X = +1011011$ $[X]_{\text{原码}} = 01011011$ $[X]_{\text{补码}} = 01011011$

$Y = -1011011$ $[Y]_{\text{原码}} = 11011011$ $[Y]_{\text{反码}} = 10100100$ $[Y]_{\text{补码}} = 10100101$

补码表示的整数范围是 $-2^{n-1} \sim +(2^{n-1} - 1)$ ，其中 n 为机器字长。8 位二进制补码表示的整数范围是 $-128 \sim +127$ ，16 位二进制补码表示的整数范围是 $-32768 \sim +32767$ 。

1.4.3 补码的运算规则与溢出判断

引入补码的概念，目的在于将加、减运算简化为单纯的加法运算。补码的运算规则是：

$$[X+Y]_{\text{补}}=[X]_{\text{补}}+[Y]_{\text{补}}$$

$$[X-Y]_{\text{补}}=[X]_{\text{补}}+[-Y]_{\text{补}}$$

已知 $[Y]_{\text{补}}$ ，求 $[-Y]_{\text{补}}$ 的方法是将 $[Y]_{\text{补}}$ 各位按位取反(包括符号位在内)末位加1。

下面举例说明以上两个公式的正确性。

【例 1.1】 设 $X1=+0001101$ $X2=-0001101$

$$Y1=+0000110 \quad Y2=-0000110$$

计算：(1) $X1+Y1$ ；(2) $X1-Y1$ ；(3) $X2+Y2$ ；(4) $X2-Y2$

解： $[X1]_{\text{补}}=00001101$ $[X2]_{\text{补}}=11110011$ $[Y1]_{\text{补}}=00000110$ $[Y2]_{\text{补}}=11111010$

(1) 计算 $X1+Y1$

+0001101	X1	00001101	$[X1]_{\text{补}}$
+) +0000110	Y1	+) 00000110	$[Y1]_{\text{补}}$
+0010011	X1+Y1	00010011	$[X1]_{\text{补}}+[Y1]_{\text{补}}$

因为： $[X1+Y1]_{\text{补}}=[+0010011]_{\text{补}}=00010011=[X1]_{\text{补}}+[Y1]_{\text{补}}$

所以： $[X1+Y1]_{\text{补}}=[X1]_{\text{补}}+[Y1]_{\text{补}}$

(2) 计算 $X1-Y1$

+0001101	X1	00001101	$[X1]_{\text{补}}$
-) +0000110	Y1	+) 11111010	$[-Y1]_{\text{补}}$
+0000111	X1-Y1	100000111	$[X1]_{\text{补}}+[-Y1]_{\text{补}}$

作 $[X1]_{\text{补}}+[-Y1]_{\text{补}}$ 运算时结果的最高位的1因数据长度而自然丢失。

因为： $[X1-Y1]_{\text{补}}=[+0000111]_{\text{补}}=00000111=[X1]_{\text{补}}+[-Y1]_{\text{补}}$

所以： $[X1-Y1]_{\text{补}}=[X1]_{\text{补}}+[-Y1]_{\text{补}}$

(3) 计算 $X2+Y2$

-0001101	X2	11110011	$[X2]_{\text{补}}$
+) -0000110	Y2	+) 11111010	$[Y2]_{\text{补}}$
-0010011	X2+Y2	111101101	$[X2]_{\text{补}}+[Y2]_{\text{补}}$

因为： $[X2+Y2]_{\text{补}}=[-0010011]_{\text{补}}=111101101=[X2]_{\text{补}}+[Y2]_{\text{补}}$

所以： $[X2+Y2]_{\text{补}}=[X2]_{\text{补}}+[Y2]_{\text{补}}$

(4) 计算 $X2-Y2$

-0001101	X2	11110011	$[X2]_{\text{补}}$
-) -0000110	Y2	+) 00000110	$[-Y2]_{\text{补}}$
-0000111	X2-Y2	11111001	$[X2]_{\text{补}}+[-Y2]_{\text{补}}$

因为： $[X2-Y2]_{\text{补}}=[-0000111]_{\text{补}}=11111001=[X2]_{\text{补}}+[-Y2]_{\text{补}}$

所以： $[X2-Y2]_{\text{补}}=[X2]_{\text{补}}+[-Y2]_{\text{补}}$

由此可看出，计算机引入了补码编码后，带来了以下几个优点：

(1) 减法转化成了加法，大大简化了运算器硬件电路的设计，加减法可由同一硬件电路进行处理。

(2) 运算时, 符号位与数值位同等对待, 都按二进制数参加运算, 符号位产生的进位丢掉不管, 其结果是正确的。这大大简化了运算规则。

运用以上两个公式时, 要注意以下两点:

(1) 公式成立有一个前提条件, 就是运算结果不能超出机器数所能表示的范围, 否则运算结果不正确, 按“溢出”处理。

例如: 设机器字长为 8 位, 则 $-128 \leq N \leq +127$, 计算 $(+64) + (+65)$ 。

$$\begin{array}{r} +64 \\ +) \quad +65 \\ \hline +129 \end{array} \qquad \begin{array}{r} 01000000 \\ +) \quad 01000001 \\ \hline 10000001 \quad (-127) \end{array}$$

显然这个结果是错误的。其原因是: $64+65=129>127$, 超出了字长为 8 位所能表示的最大值, 产生了“溢出”, 所以结果出错。

再如, 计算 $(-125) + (-10)$ 。

$$\begin{array}{r} -125 \\ +) \quad -10 \\ \hline -135 \end{array} \qquad \begin{array}{r} 10000011 \\ +) \quad 11110110 \\ \hline 101111001 \quad (+121) \end{array}$$

显然, 计算结果也是错误的。其原因是: $(-125) + (-10) = -135 < -128$, 超出了字长为 8 位所能表示的最小值, 产生了“溢出”, 所以结果出错。

(2) 采用补码运算后, 结果也是补码, 欲得运算结果的真值, 还需要转换。

1.4.4 计算机中常用的编码

计算机除了用于数值计算外, 还要进行大量的文字信息处理, 也就是要对表达各种文字信息的符号进行加工。例如, 计算机和外设的键盘、(字符)显示器、打印机之间的通信都采用字符方式输入/输出。目前, 计算机中最常用的两种编码是美国信息交换标准代码(ASCII 码)和二十进制编码(BCD 码)。

1. 美国信息交换标准代码(ASCII 码)

ASCII(American Standard Code for Information-Interchange)码是美国信息交换标准代码的简称, 主要给西文字符进行编码。它采用 7 位二进制代码来对字符进行编码, 包括 32 个标点符号, 10 个阿拉伯数字, 52 个英文大、小写字母, 34 个控制符号, 共 128 个。例如阿拉伯数字 0~9 的 ASCII 码分别为 30H~39H(H 表示十六进制), 英文大写字母 A~Z 的 ASCII 码是从 41H 开始依次往下编码的。并非所有的 ASCII 码字符都是可显示或打印的, 有些 ASCII 码作为控制字符用来完成一个规定的动作(如回车、换行、响铃等)。

在计算机内部, 每个 ASCII 码字符占用 1 个字节(8 位二进制数)来存储, 通常最高位为“0”, 低 7 位为字符的二进制编码。在许多实际应用中, 最高位 b_7 , 又常常用来作为 ASCII 码的奇/偶校验位。奇校验时, 该位的取值应使得 8 位 ASCII 码中 1 的个数为奇数; 偶校验时, 该位的取值应使得 8 位 ASCII 码中 1 的个数为偶数。例如:

“8”的奇校验 ASCII 码为 00111000B, 偶校验 ASCII 码为 10111000B;

“B”的奇校验 ASCII 码为 11000010B, 偶校验 ASCII 码为 01000010B。

奇偶校验的主要目的是用于在数据传输中, 检测接收方的数据是否正确。收发双方先

预约为何种校验，接收方收到数据后检验 1 的个数，判断是否与预约的校验相符，倘若不符，则说明传输出错，可请求重新发送。

2. BCD 码

人们已经习惯了十进制数，而且计算机的原始数据多数也是十进制的，但十进制数不能直接在计算机中进行处理，必须用二进制为它编码，这样就产生了二进制编码的十进制数，简称 BCD(Binary Coded Decimal)码。

BCD 码是用 4 位二进制数表示 1 位十进制数，但这 4 位二进制数中可表示的 16 个数码中有 6 个数码是多余的，应该抛弃。可以使用不同的方法来处理这些数码，因而产生了各种不同的 BCD 码，但最通用的是 8421 BCD 码，它是将十六进制数的 A~F 放弃不用，如表 1-1 所示。

表 1-1 BCD 码

十进制数	8421 BCD 码	十进制数	8421 BCD 码
0	0000	9	1001
1	0001	10	0001 0000
2	0010	11	0001 0001
3	0011	12	0001 0010
4	0100	13	0001 0011
5	0101	14	0001 0100
6	0110	15	0001 0101
7	0111	16	0001 0110
8	1000	17	0001 0111

例如：

89 的 BCD 码为 1000 1001；

105 的 BCD 码为 0001 0000 0101；

2004 的 BCD 码为 0010 0000 0000 0100。

可见，BCD 码是很容易编制的，而且用它来表示十进制数也比较直观，但是一定要区别于二进制数，两者表征的数值完全不同，例如：

$$(0010\ 0000\ 0000\ 0101.1001)_{\text{BCD}}=2005.9$$

$$(0010\ 0000\ 0000\ 0101.1001)_2=8197.5625$$

以上用 4 位二进制数表示 1 位十进制数的编码称为紧(压)缩型 BCD 码(Packed BCD)，此时用 8 位二进制数就能表示两位十进制数。将一个字节分成高半字节与低半字节，各表示 1 位十进制数，CPU 通过对一个专门针对半字节的辅助进位的调整就可对 BCD 数直接进行运算。

如果用 8 位二进制数来表示 1 位十进制数，则称为非紧(压)缩型 BCD 码(Unpacked BCD)，此时它的高 4 位全为 0。

例如，86 的紧缩型 BCD 码是 1000 0110B，它的非压缩型 BCD 码是 0000 1000 0000 0110B。

BCD 码的不足之处是抛弃了二进制中 6/16 的信息位不使用, 非压缩的 BCD 码浪费更大, 在相同的二进制位数条件下, BCD 能表示的数值范围变窄。换言之, 如果信息量相同的话, 那么使用 BCD 数据占用的内存空间比使用纯二进制数据要大得多。

BCD 码的运算规则: BCD 码是十进制数, 而运算器对数据做加减运算时, 都是按二进制运算规则进行处理的。这样, 当将 BCD 码传送给运算器进行运算时, 其结果需要修正。修正的规则是: 当两个 BCD 码相加, 如果和等于或小于 1001(即十六进制数 9), 不需要修正; 如果相加之和在 1010 到 1111(即十六进制数 0AH~0FH)之间, 则需加 6 进行修正; 如果相加时, 本位产生了进位, 也需加 6 进行修正。这样做的原因是, 机器按二进制相加, 所以 4 位二进制数相加时, 是按“逢十六进一”的原则进行运算的, 而实质上是 2 个十进制数相加, 应该按“逢十进一”的原则相加, 16 与 10 相差 6, 所以当和超过 9 或有进位时, 都要加 6 进行修正。下面举例说明。

【例 1.2】 计算 1+8 的值。

如果将 1 和 8 送机器进行加法运算, 其运算过程如下:

$$\begin{array}{r} 0001 \\ +) 1000 \\ \hline 1001 \end{array}$$

结果是 1001, 即十进制数 9, 1+8=9 正确。

【例 1.3】 需要修正 BCD 码运算值的举例。

(1) 计算 5+8; (2) 计算 8+8

解: (1) 将 5 和 8 以 8421 BCD 输入机器, 则运算如下:

$$\begin{array}{r} 0101 \\ +) 1000 \\ \hline 1101 \quad \text{结果大于 9} \\ +) 0110 \quad \text{加 6 修正} \\ \hline 10011 \quad \text{13 的 BCD 码} \end{array}$$

结果是 0011, 即十进制数 3, 还产生了进位。5+8=13, 结论正确。

(2) 将 8 以 8421 BCD 码输入机器, 则运算如下:

$$\begin{array}{r} 1000 \\ +) 1000 \\ \hline 10000 \quad \text{结果有进位} \\ +) 0110 \quad \text{加 6 修正} \\ \hline 10110 \quad \text{16 的 BCD 码} \end{array}$$

结果是 0110, 即十进制的 6, 还产生进位, 故加 6 修正。8+8=16, 结论正确。

这里为了说明问题, 举了较多的加法例子, 实际在计算机中有相应的十进制调整指令, 无论对于加法或减法, 机器能按照规则自动进行调整, 人们只管放心使用就是了。

3. 汉字编码

西文是拼音文字。用有限的几个字母(如英文用 26 个字母, 俄文用 32 个字母)可以拼写出全部西文信息。因此, 西文仅需对有限个数的字母进行编码, 就可以将全部西文信息输入计算机。而汉字信息则不一样, 汉字是象形文字, 1 个汉字就是 1 个方块图形。计算

机要对汉字信息进行处理,就必须对数目繁多的汉字进行编码,建立一个有几千个汉字的编码表。西文编码是几十个字符的小字符集,汉字编码是成千上万个汉字的大字符集。因此,汉字的编码远比西文字母的编码要复杂得多。

汉字编码有内码和外码之分。外码(又称汉字的输入编码)是指汉字的输入方式,目前我国公布的汉字编码有上百种。其编码的方法可以按照汉字的字形、字音和音形结合分为3类。常用的输入方式有区位码、国标码、首尾码、拼音码、双拼双音码、五笔字型码、自然码、ABC码、郑码等。内码是计算机系统内部进行汉字信息存储、交换、检索等操作的编码。汉字内码采用2字节表示,没有重码,并要求与国标码有简单的对应关系。应该指出,汉字的输入编码和内码是两个不同概念,不可混为一谈。

国标码(又称交换码)是《国家标准信息交换用汉字编码基本字符集》的简称,这是我国国家标准总局于1981年为适应计算机对汉字信息交换和处理而颁布的国家标准,编号为GB 2312—1980。该标准按 94×94 的二维代码表形式,收集了6763个汉字和682个一般字符、序号、数字,22个拉丁字母、希腊字母、汉语拼音符号等,共7445个图形字符。该标准最多可包含8836个图形字符,适应于一般汉字处理、汉字通信等系统之间的信息交换。

应该指出,汉字的输入编码和内码是两个不同的概念,不可混为一谈。现将区位码、国标码和内码作简要说明。

区位码:用每个汉字在二维代码表中行、列位置(行号称为区号,列号称为位号)来表示的代码,称为该汉字的区位码。区位码是汉字的输入编码。

国标码:国标码=区位码+32,区号和位号各增加32以后所得到的双7位二进制编码。国标码用于不同汉字系统之间汉字的传输和交换,可用作汉字的输入编码。

机内码:英文DOS的机内码是ASCII码,国标码是双7位二进制编码,用作内码将会与ASCII码相混淆,为此利用ASCII码最高位为“0”这一特点,把2字节国标码的每个字节的最高位置“1”,以示区别。这样,形成了汉字的另外一种编码方法,即汉字的机内码。简单地说,机内码=国标码+128或机内码=区位码+160。

1.5 微机的主要性能指标和应用

1.5.1 微机的主要性能指标

一台微机性能的优劣,主要是由它的系统结构、硬件组成、系统总线、外部设备以及软件配置等因素来决定的,具体表现在以下几个主要技术指标上。

1. 字长

微机的字长是指微处理器内部一次可以并行处理二进制代码的位数。它与微处理器内部寄存器以及CPU内部数据总线宽度是一致的,字长越长,所表示的数据精度就越高。在完成同样精度的运算时,字长较长的微处理器比字长较短的微处理器运算速度快。大多数微处理器内部的数据总线与微处理器的外部数据引脚宽度是相同的,但也有少数例外,如Intel 8088微处理器内部数据总线为16位,而芯片外部数据引脚只有8位,Intel 80386 SX微处理器内部为32位数据总线,而外部数据引脚为16位。对这类芯片仍然以它们的内部数据总线宽度为字长,但把它们称作“准XX位”芯片。例如,8088被称为“准16位”

微处理器芯片，80386SX 被称作“准 32 位”微处理器芯片。当前流行的 Pentium 4 处理器具有 32 位内部数据总线和 64 位外部数据总线，因此，仍是 32 位微处理器。

2. 主存容量

主存容量是主存储器所能存储的二进制信息的总量，它反映了微机处理信息时，容纳数据量的能力。主存容量越大，微机工作时主、外存储器间的数据交换次数就越少，处理速度也就越快。

主存容量常以字节(Byte)为单位，并定义 KB、MB、GB、TB 等派生单位，有 1KB=1024B；1MB=1024KB；1GB=1024MB；1TB=1024GB。

80X86 微型机能配置的最大内存容量受 CPU 所支持的物理地址空间范围的限制，一般配置为几百 KB 到几百 MB。

3. 指令执行时间

指令执行时间是指计算机执行一条指令所需的平均时间，其长短反映了计算机执行一条指令运行速度的快慢。它一方面取决于微处理器工作的时钟频率；另一方面又取决于计算机指令系统的设计、CPU 的体系结构等。微处理器工作时钟频率指标可表示为多少兆(或吉)赫[兹]，即 M(G)Hz；微处理器指令执行速度指标则表示为每秒运行多少百万条指令(Millions of Instructions Per Second, MIPS)。

4. 系统总线

系统总线是连接微机系统各功能部件的公共数据通道。系统总线所支持的数据传送位数和时钟频率直接关系到整机的性能。数据传送位数越宽，总线工作时钟频率越高，则系统总线的信息吞吐率就越高，整机的性能就越强。目前，微机系统采用了多种系统总线标准，如 ISA、EISA、VESA、PCI、PCI-Express 等。

5. 外部设备配置

在微机系统中，外部设备占据了重要地位。计算机信息的输入、输出、存储都必须由外设来完成，微机系统一般都配置了显示器、打印机、键盘等外设。微机系统所配置的外设，其速度快慢、容量大小、分辨率高低等技术指标都影响着微机系统的整体性能。

6. 系统软件配置

系统软件也是计算机系统不可或缺的组成部分。微机硬件系统仅是一个裸机，它本身并不能运行。若要运行，必须有基本的系统软件支持，如 DOS、Windows 等操作系统。系统软件配置是否齐全，软件功能的强弱，是否支持多任务、多用户操作等都是微机硬件系统性能能否得到充分发挥的重要因素。

1.5.2 微机的应用

当前，微机已广泛应用于工业、农业、国防、科研、教育、交通、通信、商业乃至家庭日常生活等各个领域。从航天技术到海洋开发、从天气预报到地震勘测、从医疗诊断到生物工程、从电子购物到儿童玩具、从远程教育到情报检索、从产品设计到生产过程控制等，到处都有微机的踪迹。微机的应用归纳起来，主要有以下几个方面：

1. 办公自动化

办公自动化(Office Automation, OA)是计算机、通信与自动化技术相结合的产物,也是当前最为广泛的一类应用。主要包括:电子数据处理系统(Electronic Data Process, EDP),如公文的编辑打印、报表的填写与统计、文档检索、活动安排及其他数据处理等;管理信息系统(Management Information System, MIS)是一个以计算机为基础,对企事业单位或政府机关实行全面管理的信息处理系统,如人事管理、财务管理、计划管理、统计管理等,支持本单位的信息管理工作;决策支持系统(Decision Supporting System, DSS)包括数据库、知识库、模型库和方法库,它通过对大量历史数据和当前数据的统计、分析,预测在不同对策下可能产生的结果。

2. 数据库应用

数据库是在计算机存储设备中,按照某种关联方式存放的一批数据。借助数据库管理系统(Data Base Management System, DBMS),可对其中的数据实施控制、管理和使用,如科技情报检索系统、银行储户管理系统、飞机票订票系统等。根据数据存放的差异,可以将数据库分为集中式和分布式两类,集中式数据库将数据集中在一台计算机上,分布式数据库则将数据分散在多台计算机内,数据库在计算机现代应用中占有非常重要的地位。

3. 计算机网络应用

计算机网络就是利用通信设备和线路等,把不同的计算机系统互连起来,并在网络软件支持下,实现资源共享和信息传递的系统。根据计算机之间距离的远近、覆盖范围的多少,分为局域网(Local Area Network, LAN)、广域网(Wide Area Network, WAN)、城域网(City Area Network, CAN)和因特网(Internet)。网络应用使人类进入了信息化社会,可以在网上浏览与检索信息、下载软件,充分享受网络资源;随时收发电子邮件(E-mail)、传真(FAX)、传送文件(FTP)、发布公告(BBS)、参加网络会议(Netmeeting),参加各种网上论坛;在网上开展电子商务和电子数据交换等。

4. 生产过程自动化

这种方式包括:计算机辅助设计(Computer Aided Design, CAD)——具有快速改变产品设计参数,优化设计方案,动态显示产品投影图、立体图,输出图纸等功能,降低了产品的设计成本,缩短了产品的设计周期;计算机辅助制造(Computer Aided Manufacturing, CAM)——根据加工过程编写数控加工程序,由程序控制数控机床来完成工件的自动加工,并能在加工过程中自动换刀及给出数据,一次自动完成多种复杂的工序;计算机集成制造系统(Computer Integrated Manufacturing System, CIMS)——是集设计、制造、管理三大功能于一体的现代化工厂生产系统,代表一种新型的生产模式,具有生产效率高、生产周期短等特点。生产过程自动化是计算机在现代生产领域,特别是在制造业中的典型应用,不仅提高了自动化水平,而且使传统的生产技术发生了革命性的变化。

5. 智能模拟

智能模拟又称人工智能(Artificial Intelligence, AI),是用计算机软硬件系统模拟人类某

些智能行为,如感知、思维、推理、学习、理解等的理论和技术。它是在计算机科学、控制论、仿生学和心理学等基础上发展起来的边缘科学,是当前国内外争先研究的热门技术。它包括专家系统,问题求解,定理证明,机器翻译,自然语言理解,对声、图、文的模式识别等。

人工智能的另一个重要应用是机器人。目前,国际上已有许多机器人用于各种恶劣环境的生产和试验领域。机器人的视觉、听觉、触觉以及行走系统等,是目前亟待解决的问题。随着人工智能研究的发展,机器人的智能水平会不断提高,它的应用前景是十分广阔的。

微机自 1971 年问世以来,对科学技术和人类的生活产生了巨大的影响,已逐渐成为人们工作和生活中不可缺少的工具。微机的应用之所以发展得如此迅速,一个重要的原因是其性能价格比在各类计算机中占有领先地位。微机以物美价廉、可靠性高、维护方便、小巧灵活而深受人们的欢迎。

1.6 小 结

电子计算机的产生和发展是 20 世纪最重要的科技成果之一,进入 20 世纪 70 年代,微机开始登上历史舞台,并以不可阻挡的势头迅猛发展,成为当今计算机发展的一个主流方向。当前,以微机为代表的计算机已日益普及,其应用已深入到社会的各个角落,极大地改变着人们的工作方式、学习方式和生活方式,成为信息时代的主要标志。

本章从微处理器的产生和发展开始,对微机的基本概念、硬件结构、工作原理、系统组成、应用特点等知识作了相应的概述;介绍了计算机中数据的表示方法,重点介绍了二进制、八进制、十进制、十六进制数的相关概念及各类数制之间相互转换的方法、无符号数和带符号数的机器内部表示、字符编码和汉字编码等。通过本章的学习,要了解微机的发展历史和应用场合,关注当前微机的发展动向,尤其是微处理器芯片的更新换代,以及相关软件的应用。要掌握微机的分类方式,熟悉微机系统组成及工作原理,理解微机硬件和软件各主要模块的功能和在系统中所处的地位;要掌握计算机内部的信息处理方法和特点,熟悉各类数制之间的相互转换,理解无符号数和带符号数的表示方法,掌握 BCD 码、字符的 ASCII 码及汉字编码和应用,为后续内容的学习打下一个良好的基础。

1.7 习 题

1. 简述冯·诺依曼计算机的基本特点。
2. 如何划分计算机发展的几个阶段?当前广泛使用的计算机主要采用哪一代的技术?
3. 微机硬件结构由哪些部分组成?各部分的主要功能是什么?
4. 衡量微机系统性能主要有哪些技术指标?
5. 如何确定一个微处理器是 8 位、16 位还是 32 位?

6. 把下列十进制数分别转换为二进制、八进制和十六进制数。
(1) 24.25 (2) 421.15 (3) 0.875 (4) 100
7. 将下列二进制数分别转换为十进制、八进制和十六进制数。
(1) 101100.110 (2) 1110010.1101 (3) 1101.1011 (4) 1101101.0101
8. 将下列十六进制数分别转换为二进制、八进制和十六进制数。
(1) 8C.29 (2) 132.2B (3) 65C.54 (4) 2DE6.A
9. 写出下列十进制数的原码、反码、补码(采用 8 位二进制数表示)。
(1) 87 (2) 34 (3) -48 (4) -100
10. 已知补码求出其真值。
(1) 93H (2) 88H (3) B5H (4) 3C21H
11. 按照字符所对应的 ASCII 码表示, 查表写出下列字符的 ASCII 码。
(1) A (2) a (3) 1 (4) CR(回车) (5) \$

第 2 章 80X86 微处理器内部结构

教学提示：微处理器是微机的核心部件，要掌握其原理，必须熟悉微处理器的内部结构及其组成部件和相应功能。本章主要讲解有关 8086/8088 微处理器和高档微处理器的体系结构。

教学要求：本章要求学生了解 80X86 微处理器的功能结构；掌握 8086/8088 存储器的组织和寻址、8086/8088 I/O 端口地址空间、8086/8088 各内部寄存器的主要作用。

2.1 微处理器的功能结构

微处理器(Microprocessor)简称 μP ，是采用大规模或超大规模集成电路技术制成的半导体芯片，上面集成了计算机的主要部件：控制器、运算器和寄存器组。整个微机硬件系统的核心就是微处理器，所以它又称为中央处理器(Central Processing Unit)，即 CPU。若字长为 8 位，即一次能处理 8 位数据，称为 8 位 CPU，如 Z80 CPU；字长为 16 位的，即一次能处理 16 位数据，称为 16 位 CPU，如 8086/8088、80286 等。

2.1.1 微处理器的典型结构

图 2.1 是一个典型的 8 位微处理器的内部结构，它一般由算术逻辑运算单元、寄存器组和指令处理单元等几个部分组成。

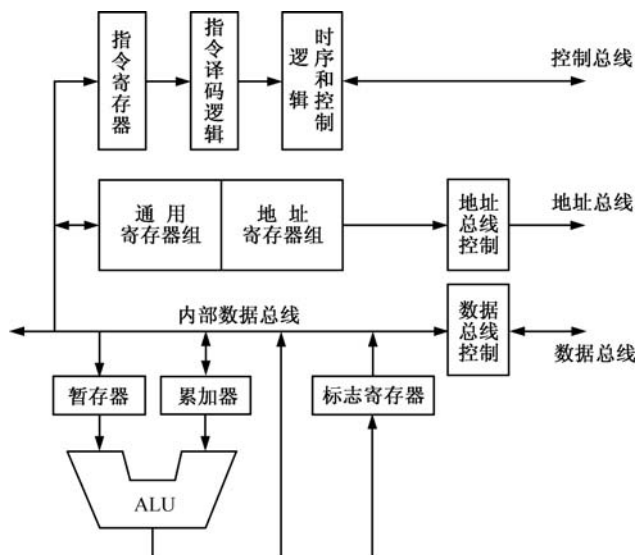


图 2.1 8 位微处理器内部结构图

1. 算术逻辑运算单元(Arithmetic Logic Unit, ALU)

算术逻辑运算单元实际上就是计算机的运算器,负责 CPU 进行的各种运算。它们包括算术运算和逻辑运算,如加、减、增量(加 1)、减量(减 1)、比较、求反、求补等运算,逻辑与、逻辑或、逻辑非、逻辑异或,以及移位、循环移位等运算和操作。

ALU 的基本组成是一个加法器。在 ALU 所进行的运算中,多数操作需要两个操作数,比如“加”和“逻辑与”运算。但是,也有些运算只要一个操作数,比如“增量”和“逻辑非”运算。对 8 位 CPU 来说,由累加器提供其中的一个操作数,另一个操作数通过暂存器来提供。运算后,运算结果被回送到累加器,运算中状态的变化和运算结果的数字特征则被记录在标志寄存器中。根据运算后各个标志位的情况可以决定程序的下一步走向。

2. 寄存器组(Register Set)

寄存器是 CPU 内部的高速存储单元,不同的 CPU 配有不同数量、不同长度的一组寄存器。有些寄存器不面向用户,称为“透明”寄存器,对它们的工作,用户不需要了解;有些寄存器则面向用户,供编程用,这些寄存器在程序中频繁使用,被称为可编程寄存器。

由于访问寄存器比访问存储器快捷和方便,所以各种寄存器常用来存放临时的数据或地址,具有数据准备、数据调度和数据缓冲等作用。从指令角度看,一般在含有两个操作数的指令中,必有一个为寄存器操作数,这样可以缩短指令长度和指令的执行时间。

从应用角度看,可以将寄存器分成以下三类。

(1) 通用寄存器:在 CPU 中数量最多。它们既可以存放数据,又可以存放地址,使用频度非常高,是调度数据的主要手段。其中,累加器的寻址手段最多,功能最强,使用最频繁。

(2) 地址寄存器:主要用来存放地址,用于存储器的寻址操作,因而也称为地址指针或专用寄存器,如堆栈指针、指令指针等。地址寄存器的功能比较单一,在访问内存时,可以通过它形成各种寻址方式。

(3) 标志寄存器:用来保护程序的运行状态,也称为程序状态字寄存器(PSW)。在标志寄存器中,有些标志位反映运算过程中发生的情况,如运算中有无进位或借位,有符号数运算有无溢出等;有些标志位反映运算结果的数字特征,如结果的最高位是否为 1、结果是否为 0 等。

3. 指令处理单元

指令处理单元是计算机的控制器,负责对指令进行译码和处理。它一般包括:

(1) 指令寄存器:用来暂时存放从存储器中取出来的指令。

(2) 指令译码逻辑:负责对指令进行译码,通过译码产生完成指令功能的各种操作命令。

(3) 时序和控制逻辑:根据指令要求,按一定的时序发出、接收各种信号,控制、协调整个系统完成所要求的操作。这些信号包括:一系列定时控制信号、CPU 的状态和应答信号、外界的请求信号和联络信号等。

2.1.2 8086/8088 CPU 的内部结构

2.1.1 小节讨论了 8 位微处理器的基本组成和各部分的功能，这里将介绍 16 位微处理器，如 Intel 公司的 8088 和 8086 微处理器。在 IBM 公司设计微机 IBM PC 和 IBM PC/XT 时，8088 被选作它们的 CPU。

1978 年 Intel 公司推出微处理器 8086，其 CPU 内部寄存器为 16 位，外部 16 位数据线，20 位地址线。8088 除了外部数据线仅为 8 位外，其他与 8086 相同，所以 8088 是准 16 位的 CPU。

从功能上分，Intel 8086/8088 由两个独立的工作单元组成，如图 2.2 所示，即执行单元 EU(Execution Unit)和总线接口单元 BIU(Bus Interface Unit)。EU 不与外部总线相连，它负责执行指令。而 BIU 则负责从存储器或外部设备中读取指令和读/写数据，即完成所有的总线操作。这两个单元处于并行工作状态，可以同时进行读/写操作和执行指令的操作。这样就减少了微处理器为取指令而需要等待的时间，提高了微处理器中各部件的利用率，从而提高了微处理器执行指令的速度，这便是流水线结构的雏形。

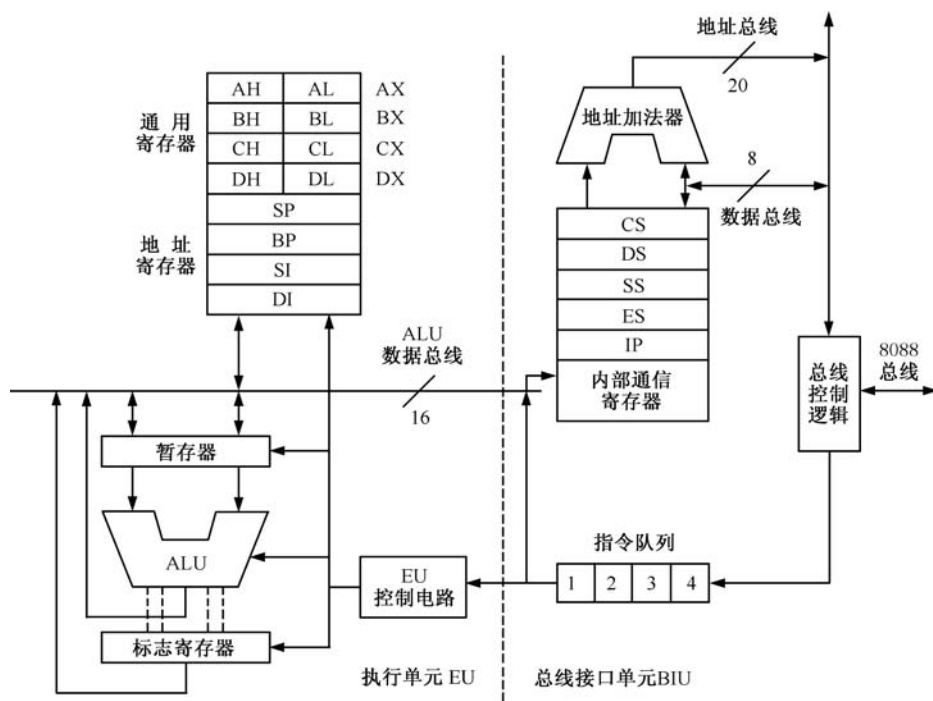


图 2.2 8088 内部结构

总线接口单元 BIU 负责与存储器接口，8086/8088 与存储器之间的信息传送都由 BIU 完成。BIU 如图 2.2 虚线右侧所示，包括一组段寄存器、指令指示器 IP、4 个(8086 是 6 个)字节的指令队列、地址加法器和总线控制逻辑。在 EU 执行指令的过程中，BIU 始终根据指令指示器 IP 提供的地址，从存放指令的存储器中预先取出一串指令存放到指令队列中。取来的指令在指令队列中是按字节顺序存放的，这样 EU 不必等待 BIU 去取指令，即在 EU 执行一条指令的同时，BIU 就可同时取下一条或多条指令，从而提高了 CPU 的效率。BIU

在下面两种情况下执行取指操作：其一是当指令队列中出现两个以上的指令字节为空的时候，自动执行总线操作取指令；其二是当程序发生转移时，BIU 执行取指操作，BIU 在新的转移地址处取得第一条指令，直接送到 EU 中去执行，将随后取来的指令重新填入指令队列，冲掉转移前放入指令队列中的指令。

执行单元 EU 如图 2.2 虚线左边所示，EU 负责执行指令，这样取指部分与指令执行部分是分开且并行工作的。它包含一个 16 位的算术逻辑单元 ALU，一个反映 CPU 操作状态和控制标志的状态标志寄存器 FLAGS，一组 16 位的寄存器，运算寄存器和 EU 控制系统。它的寄存器和数据传输通路都是 16 位的，它们之间进行快速的内部数据传输。EU 从 BIU 中的指令队列寄存器中取得指令和数据，执行指令要求的操作。ALU 的主要功能是进行算术逻辑运算，也计算存储器操作数的偏移地址。当指令要求执行存储器或 I/O 设备的数据存取操作时，EU 向 BIU 发出请求。BIU 根据 EU 的请求，完成 8086/8088 与存储器或外部设备之间的数据传送。

2.1.3 8086/8088 CPU 的寄存器结构

在 8086/8088 微处理器中设置一些寄存器，这些寄存器可用来暂存参加运算的操作数和运算过程中的中间结果。其内部共有 14 个 16 位寄存器，按功能可分为：通用寄存器(8 个)、段寄存器(4 个)、控制寄存器(2 个)三大类。

1. 通用寄存器

通用寄存器包括数据寄存器、地址指针寄存器和变址寄存器。

1) 数据寄存器 AX、BX、CX、DX

数据寄存器一般用于存放参与运算的数据或运算结果。每一个数据寄存器都是 16 位寄存器，但又可将高、低 8 位分别作为两个独立的 8 位寄存器使用。它们的高 8 位记作 AH、BH、CH、DH，低 8 位记作 AL、BL、CL、DL，这给编程带来了很大的方便。

上述 4 个寄存器一般作为通用寄存器使用，但又有各自的习惯用法。

(1) AX 称为累加器，在乘除法运算、串运算和 I/O 指令中都作为专用寄存器。

(2) BX 称为基址寄存器，在计算机寻址时，常用来存放基址。

(3) CX 称为计数寄存器，在循环和串操作指令中用作计数器。

(4) DX 称为数据寄存器，在寄存器间接寻址的 I/O 指令中存放 I/O 端口地址，在做双字长乘除法运算时，DX 与 AX 合起来存放一个双字长数据(32 位)，其中 DX 存放高 16 位。

2) 地址指针寄存器 SP、BP

SP 为堆栈指针寄存器，BP 称为基址指针寄存器。作为一种通用寄存器，它们可以存放数据，但实际上，它们更经常更重要的用途是存放内存单元的偏移地址。

3) 变址寄存器 SI、DI

SI 称为源变址寄存器，DI 称为目的变址寄存器，常用于存储器寻址。

2. 段寄存器 CS、SS、DS、ES

CS 称为代码段寄存器，SS 称为堆栈段寄存器，DS 称为数据段寄存器，ES 称为附加数据段寄存器。段寄存器用于存放段基址值(16 位无符号数)。

3. 控制寄存器 IP、FLAG

IP 称为指令指针寄存器，用于存放偏移地址。CPU 从代码段中偏移地址为 IP 的内存元中取出指令代码的 1 字节后，IP 自动加 1，指向指令代码的下一个字节。用户程序不能直接访问 IP。

FLAG 称为标志寄存器，它是 16 位的寄存器，但只用其中 9 位，这 9 位包括 6 个状态标志位和 3 个控制标志位，如图 2.3 所示。

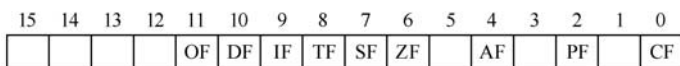


图 2.3 8086/8088 CPU 的标志寄存器

状态标志：用来反映 EU 执行算术和逻辑运算以后的结果特征，这些标志常作为条件转移类指令的测试条件，控制程序的运行方向。这 6 个状态标志位分别是：

(1) CF(Carry Flag)进位标志：当 CF=1，表示指令执行结果在最高位上产生一个进位(加法)或借位(减法)；CF=0，则无进位或借位产生。CF 标志主要用于加、减运算，移位和循环指令也能把存储器或寄存器中的最高位(左移)或最低位(右移)移入 CF 位中。

(2) PF(Parity Flag)奇偶标志：当 PF=1，表示指令执行结果中有偶数个 1；PF=0，则表示结果中有奇数个 1。PF 标志用于检查在数据传送过程中，是否有错误发生。

(3) AF(Auxiliary Carry Flag)辅助进位标志：当 AF=1，表示结果的低 4 位产生了一个进位或借位；AF=0，则无进位或借位。AF 标志主要用于实现 BCD 码算术运算结果的调整。

(4) ZF(Zero Flag)零标志：当 ZF=1，表示运算结果为零；ZF=0，则运算结果不为零。

(5) SF(Sign Flag)符号标志：当 SF=1，表示运算结果为负数，即最高位为 1；SF=0，则表示结果为正数，即最高位为 0。

(6) OF(Overflow Flag)溢出标志：当 OF=1，表示带符号数在进行算术运算时产生了算术溢出，即运算结果超出带符号数所能表示的范围；OF=0，则无溢出。

控制标志：控制标志用来控制 CPU 的工作方式或工作状态，它一般由程序设置或由程序清除。控制标志共有 3 位，分别是：

(1) TF(Trap Flag)陷阱标志：TF 标志是为了调试程序方便而设置的。若 TF=1，则 8086 CPU 处于单步工作方式。8086 CPU 执行完一条指令就自动产生一个内部中断，转去执行一个中断服务程序；当 TF=0 时，8086 CPU 正常执行程序。

(2) IF(Interrupt Enable Flag)中断允许标志：它是控制可屏蔽中断的标志。若 IF=1，表示允许 CPU 接受外部从 INTR 引脚上发来的可屏蔽中断请求信号；若 IF=0，则禁止 CPU 接受可屏蔽中断请求信号。

(3) DF(Direction Flag)方向标志：DF 用于控制字符串操作指令的步进方向。当 DF=1 时，字符串操作指令按递减的顺序从高地址到低地址的方向对字符串进行处理；若 DF=0，字符串操作指令按递增的顺序从低地址到高地址的方向对字符串进行处理。

对于状态标志, CPU 在进行算术逻辑运算时, 根据操作结果自动将状态标志位置位(等于 1)或复位(等于 0); 对于控制标志, 事先用指令设置, 在程序执行时, 检测这些标志, 用以控制程序的转向。

2.2 8086/8088 存储器组织及其寻址

存储器是计算机中存储信息和数据的物理部件。所有的程序和数据必须被调入内存后, 才能被计算机执行和处理。

2.2.1 存储单元的地址和内容

计算机中存储信息的基本单位是 1 个二进制位, 它可存储信息 0 或 1。每 8 个二进制位为 1 组, 称为 1 字节(Byte)。

在存储器中是以字节为单位进行存储信息的。为了正确地存放或读取信息, 给每一个字节单元 1 个编号, 称为存储器的地址。地址从 0 开始编号, 每移动 1 字节, 其值顺序加 1。在计算机中, 地址是用二进制来表示的无符号数, 习惯上地址用十六进制数表示。

8086/8088 有 20 根地址线, 直接可寻址的地址空间为 $2^{20}=1\text{MB}$ 。这 1MB 的内存单元按照 00000H~FFFFFH 进行编址。

1 个存储单元中存储的信息称为该存储单元的内容。例如, 地址为 001A0H 的存储单元的内容为 5AH, 表示为(001A0H)=5AH。

2.2.2 存储器的结构及访问

存储器内部是按字节进行组织的, 两个相邻的字节被称为一个“字”。在一个字中每个字节用一个唯一的地址码进行表示。存放的信息以字节为单位, 在存储器中按顺序排列存放; 若存放的数据为一个字, 则将该字的低字节(即低 8 位数据)存放在低地址单元中, 高字节(即高 8 位数据)存放在高地址单元中, 并以低地址作为该字的地址。

在 8086 CPU 存储器中, 如果一个字是从偶地址开始存放, 称为规则字或对准字。如果一个字是从奇地址开始存放, 称为非规则字或非对准字。对规则字的存取可在一个总线周期内完成, 非规则字的存取则需两个总线周期。

8086 CPU 存储器的 1MB 的存储空间被分成两个 512KB 的存储体, 分别叫高位库和低位库。低位库固定与 8086 CPU 的低位字节数据线 $D_7\sim D_0$ 相连, 称为低字节存储体, 该存储体中的每个地址均为偶地址。高位库与 8086 CPU 的高位字节数据线 $D_{15}\sim D_8$ 相连, 称为高字节存储体, 该存储体中的每个地址均为奇地址。两个存储体之间采用字节交叉编址方式, 如图 2.4 所示。

对于任何一个存储体, 只需要 19 位地址码 $A_{19}\sim A_1$ 就够了, 最低位地址码 A_0 用以区分当前访问哪一个存储体。当 $A_0=0$, 表示访问偶地址存储体; 当 $A_0=1$, 表示访问奇地址存储体。



图 2.4 8086/8088 存储器的分体结构

8086 系统设置了一个总线高位有效控制信号 $\overline{BHE}$ 。 $\overline{BHE}$ 与 A_0 相互配合,使得 8086 CPU 可以访问两个存储体中的一个字节或字信息。 $\overline{BHE}$ 和 A_0 的组合控制作用如表 2-1 所示。

表 2-1 $\overline{BHE}$ 和 A_0 的代码组合对应的存取操作

$\overline{BHE}$	A_0	操作功能	数据总线
0	0	同时访问两个存储体,读/写一个对准字信息	$D_{15} \sim D_0$
0	1	只访问奇地址存储体,读/写高字节信息	$D_{15} \sim D_8$
1	0	只访问偶地址存储体,读/写低字节信息	$D_7 \sim D_0$
1	1	无操作	

当在偶数地址中存取一个数据字节时,CPU 从低位库中经数据线 $D_7 \sim D_0$ 存取数据。由于被寻址的是偶数地址,所以地址位 $A_0=0$,由于 A_0 是低电平,所以才能在低位库中实现数据的存取。而指令中给出的是在偶地址中存取一个字节, $\overline{BHE}$ 信号应为高电平,故不能从高位库中读出数据。相反,当在奇数地址中存取一个字节数据时,应经数据线的高 8 位($D_{15} \sim D_8$)传送。此时,指令应指出是从高位地址(奇数地址)寻址, $\overline{BHE}$ 信号为低电平有效状态,故高位库能被选中,即能对高位库中的存储单元进行存取操作。由于是高位地址寻址,故 $A_0=1$ 低位库存储单元不会被选中。如表 2-1 所示,8086 CPU 也可以一次在两个库中同时各存取一个字节,完成一个字的存取操作。

规则字的存取操作可以在一个总线周期中完成。由于地址线 $A_{19} \sim A_1$ 是同时连接在两个库上的,只要 $\overline{BHE}$ 和 A_0 信号同时有效,就可以一次实现在两个库中对一个字(高低两字节)完成存取操作。对字的存取操作所需的 $\overline{BHE}$ 及 A_0 信号是由字操作指令给出的。

对非规则字的存取操作就需要两个总线周期才能完成:在第一个总线周期中,CPU 是在高位库中存取数据(低位字节),此时 $A_0=1$, $\overline{BHE}=0$ 。然后再将存储器地址加 1,使 $A_0=0$,选中低位库;在第二个总线周期中,是在低位库中存取数据(高位字节),此时 $A_0=0$, $\overline{BHE}=1$ 。

2.2.3 存储器地址的分段

8086/8088 系统中,直接可寻址的存储器空间达到 1MB,要对整个存储器空间寻址,需要 20 位长的地址码,而 8086/8088 CPU 的字长为 16 位,只能寻址 64KB。为此,8086/8088 采用了存储器地址分段的办法。

将整个存储器分成许多逻辑段，每个逻辑段的容量最多为 64KB，允许它们在整个存储器空间中浮动，各个逻辑段之间可以紧密相连，也可以相互重叠。对于任何一个物理地址来说，可以唯一地被包含在一个逻辑段中，也可以被包含在多个相互重叠的逻辑段中，只要能得到它所在段的首地址和段内的相对地址，就可对它进行访问。在 8086/8088 存储空间中，从 0 地址开始，把每 16 个连续字节的存储空间称为小节。为了简化操作，逻辑段必须从任一节的首地址开始。这样划分的特点是：在十六进制表示的地址中，最低位为 0(即 20 位地址中的低 4 位为 0)。在 1MB 的地址空间中，共有 64K 小节。8086/8088 中，每一个存储单元都有一个唯一的 20 位地址，称此地址为该存储单元的物理地址。CPU 访问存储器时，必须先确定所要访问的存储单元的物理地址，才能取得该单元的内容。20 位的物理地址由 16 位的段地址和 16 位的段内偏移地址计算得到。段地址是每一个逻辑段的起始地址，必须是每个小节中的首地址，其低 4 位一定都是 0，于是在保留段地址时，可只取段地址的高 16 位。偏移地址则是在段内相对于段起始地址的偏移值。因此任一存储单元物理地址的计算方法(如图 2.5 所示)是：

物理地址=16×段地址+偏移地址

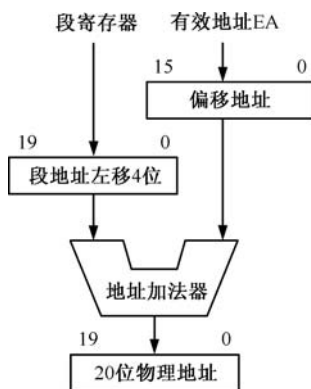


图 2.5 物理地址的形成

在 IBMPC 机中，设有 4 个存放段地址的寄存器，称为段寄存器。它们是代码段寄存器 CS、数据段寄存器 DS、堆栈段寄存器 SS、附加段寄存器 ES。在实际使用中，常用“段地址：偏移地址”来表示逻辑地址，其中段地址和偏移地址都是 16 位二进制数(常用 4 位十六进制数表示)。一个物理地址可用多种逻辑地址进行表示，但其物理地址是唯一的。

2.3 8086/8088 I/O 地址空间

8086 系统和外部设备之间都是由 I/O 接口电路来联系的，为区别不同的 I/O 对象，每个 I/O 接口都有一个或几个端口地址。在微机系统中，给每个端口分配一个地址，称为端口地址。一个端口通常为 I/O 接口电路内部的一个寄存器或一组寄存器。8086 CPU 利用地址总线的低 16 位作为对 8 位 I/O 端口的寻址线，8086 系统访问的 8 位 I/O 端口最多有 65 536(64KB)个。两个编号相邻的 8 位端口可以组合成一个 16 位的端口。一个 8 位的 I/O 设备既可以连接在地址总线的高 8 位上，也可以连接在地址总线的低 8 位上，为便于地址

总线的负载相平衡, 接在高 8 位和低 8 位上的设备数目最好相等。当一个 I/O 设备接在地址总线低 8 位($AD_7 \sim AD_0$)上时, 这个 I/O 设备所包括的所有端口地址都将是偶数地址(即 $A_0=0$); 若一个 I/O 设备是接在地址总线的高 8 位($AD_{15} \sim AD_8$), 那么此设备包含的所有端口地址都是奇数地址(即 $A_0=1$)。如果某种特殊 I/O 设备既可使用偶地址又可使用奇地址, 那么 A_0 就不能作为这个 I/O 设备内部端口的地址选择线使用。此时 A_0 和 $\overline{BHE}$ 这两个信号必须结合起来作为 I/O 设备选择线, 用以防止对 I/O 设备的错误操作。

IBM-PC 系统只使用了 $A_9 \sim A_0$ 10 条地址线作为 I/O 端口的寻址线, 故最多可寻址 1024 (2^{10})个端口地址。

2.4 高档微处理器

80286 可以看作是 8086/8088 16 位微处理器性能的提升, 而 80386 则是 32 位微机时代开始的标志。4 年后, Intel 公司又推出了新一代 32 位微处理器 80486, 之后推出了 Pentium 系列微处理器。

2.4.1 80286 微处理器

1982 年, Intel 公司推出了 X86 系列的新一代 16 位微处理器 80286。80286 CPU 内部和外部数据总线都是 16 位, 地址总线为 24 位, 可寻址 2^{24} 字节(16MB)内存。80286 CPU 片内具有存储器管理和保护机构, 它有实地址和受保护的虚地址两种工作方式, 分别称为实模式与保护模式。80286 在以下四个方面比 8086 有显著的改进:

(1) 支持更大的内存。达到了当时前所未有的 16MB。

(2) 模拟内存空间。处理器可以使用外存储设备模拟大量的存储空间, 多达 1GB 的虚拟存储器。

(3) 能同时运行多个任务。多任务是通过多任务硬件机构, 使处理器在各种任务间来回快速切换的。

(4) 处理速度提升。最早 8086 版本的运行速度是 4MHz, 80286 的运行速度为 6 MHz~8MHz, 后期的 80286 版本达到了 20MHz, 性能上有了重大的进步。

80286 将 8086 中的总线接口单元 BIU 分成地址单元 AU(Address Unit)、指令单元 IU(Instruction Unit)和总线单元 BU(Bus Unit)三部分, 加上原有的执行单元 EU, 共 4 个单元。这四个单元分别同步并行操作, 实现流水线化作业方式, 从而提高了吞吐率, 加快了 CPU 的处理速度。

80286 的通用寄存器、段寄存器和指令寄存器与 8086 完全一样, 不同之处在于新增加了一个机器状态字 MSW(Machine Status Word)寄存器, 标志寄存器新增加了两个标志位, 即 I/O 特权标志 IOPL(位 12、13)和嵌套任务标志 NT(位 14), 其他 9 位标志的定义与位置均与 8086 相同。MSW 是个 16 位寄存器, 只定义了它的低 4 位, 最低位是保护允许(PE)位, 当 $PE=0$ 时, CPU 处在实地址方式; 当 $PE=1$ 时, CPU 处在虚地址保护方式。在 CPU 复位时, MSW 置为 FFF0H, CPU 处在实地址方式。其他 3 位, 起控制协处理器接口的作用, 分别定义为监督协处理器 MP、协处理器模拟 EM、任务转换 TS。

80286 有实地址和受保护的虚地址两种工作方式。在实地址方式中, 80286 和 8086 工

作方式完全一样，使用 24 位地址中的低 20 位 $A_{19} \sim A_0$ 。寻址能力为 1MB，物理地址与逻辑地址的含义也与 8086 一样。在保护模式下，80286 具有多用户和多任务的处理能力，其实际存储空间将扩大到 16MB，并为每个用户提供 1000MB 的虚拟存储空间，CPU 可将获得的虚拟地址自动转换到 16MB 的物理地址。在保护模式下，80286 与 8086/8088 的源代码兼容。

2.4.2 80386 微处理器

80386 是 Intel 公司首次推出的 32 位微处理器。80386 CPU 内部和外部数据总线都是 32 位的，但段寄存器仍为 16 位，地址总线是 32 位，可寻址 2^{32} 字节(4GB)地址空间。80386 CPU 由总线接口单元 BIU、指令译码单元 IDU(Instruction Decode Unit)、指令代码预取单元 IPU(Instruction Prefetch Unit)、执行单元 EU、段管理单元 SU(Segment Unit)和页管理单元 PU(Paging Unit)6 个能并行操作的功能部件组成。CPU 结构和 80286 CPU 基本相同，主要的区别是段管理单元 SU 和页管理单元 PU，段管理单元用来将逻辑地址变换成线性地址，页管理单元用来将线性地址转换成物理地址。6 个部件按流水线结构设计，指令的预取、译码、执行等步骤由各自的处理部件并行操作，这样可同时处理多条指令，提高微处理器运行速度。

80386 共有 8 类寄存器：控制寄存器、系统地址寄存器、调试寄存器和测试寄存器主要是用于简化设计和对系统进行调试，通用寄存器、段寄存器、指令指示器、标志寄存器如图 2.6 所示。

80386 全面兼容了 8086、8088、80286。它有 8 个 32 位的通用寄存器，它们是 8086 和 80286 的 16 位通用寄存器的扩展，故命名为累加器 EAX，基址寄存器 EBX，计数寄存器 ECX，数据寄存器 EDX，堆栈指示器 ESP，基址指示器 EBP，源变址寄存器 ESI 和目的变址寄存器 EDI。对这些寄存器的低 16 位可以作为 16 位寄存器使用，其命名同 8086，分别为 AX、BX、CX、DX、SP、BP、SI、DI；而 AX、BX、CX 和 DX 的低 8 位和高 8 位又可以分别作为 8 位的寄存器单独使用，其命名仍为 AH、AL、BH、BL、CH、CL、DH 和 DL。

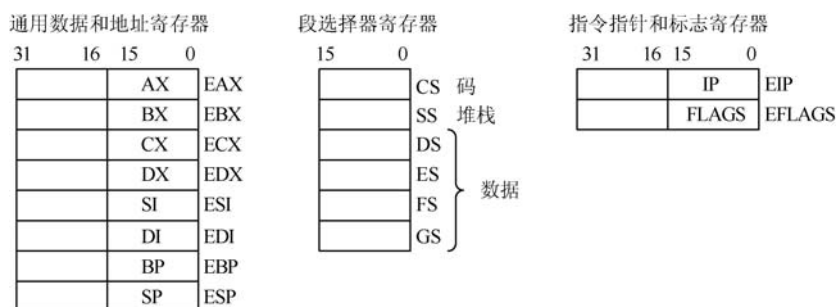


图 2.6 80386 的部分寄存器

80386 的 32 位指令指示器 EIP 是 80286 IP 的扩展，标志寄存器 EFLAGS 也是 32 位的寄存器，它是 80286 的 FLAGS 的扩展，它们的低 16 位仍是 80286 的 IP 和 FLAGS，并可单独使用。80386 除了保留 80286 的所有标志外，又增加了两个标志位，虚拟 8086 方式标志的 VM 和恢复标志 RF。

80386 有 6 个 16 位段寄存器, 也称选择器, 它们是 CS、SS、DS、ES、FS 和 GS。其中 CS 和 SS 的作用与 80286 相同, 而 DS、ES、FS 和 GS 都可以用来表示数据段。

80386 有 4 个系统地址寄存器, 它们是全球描述符表寄存器 GDTR(Global Descriptor Table Register)、中断描述符表寄存器 IDTR(Interrupt Descriptor Table Register)、局部描述符表寄存器 LDTR(Local Descriptor Table Register)和任务寄存器 TR(Task Register)。它们主要用来在保护模式下, 管理用于生成线性地址和物理地址的 4 个系统管理描述符表。

80386 有 4 位控制寄存器 CR3~CR0, CR1 为备用。CR3~CR0 用于保存全局性的机器状态。

80386 的 8 个调试寄存器 DR7~DR0 主要用来设置调试程序的断点地址。

80386 的 2 个测试寄存器 TR6 和 TR7 也是用来进行页处理的寄存器。

80386 CPU 有实地址、虚地址保护和虚拟 8086 这 3 种工作方式。实地址方式和 8086 工作方式相同, 虚拟 8086 工作方式是在虚地址保护方式下, 能够在多任务系统中执行 8086 任务的一种特殊方式。

2.4.3 80486 微处理器

80486 是 Intel 公司于 1989 年推出的新一代 32 位微处理器, 80486 的内外数据总线都是 32 位, 地址总线为 32 位。80486 基础结构与 80386 相同, 它们在寄存器组、寻址方式、存储器管理特征、数据类型方面完全兼容 80386, 80486 的 3 种工作方式及逻辑地址、线性地址、物理地址等也都与 80386 完全相同。

80486 内部由总线接口单元 BIU、指令译码单元 IDU、指令预取单元、执行单元 EU、段管理单元 SU、页管理单元 PU 以及浮点处理单元 FPU 和高速缓存(Cache Memory)等单元组成, 比 80386 新增加了相当于 80387 功能的 FPU 和 Cache 两个单元。为了进一步提高微处理器性能, 80486 CPU 进行了以下主要的改进:

(1) 在以前的处理器中, 浮点运算部件一直作为一个单独的数学协处理器芯片配合微处理器进行浮点数值计算, 如 8087、80287 和 80387。这些协处理器分别与 8086、80286 和 80386 密切配合, 可以使数值运算, 特别是浮点运算的速度显著提高。而 80486 首次将浮点处理部件 FPU 集成在微处理器内部, 其处理速度进一步提高, 比 80387 约快 3~5 倍。

(2) 为了进一步提高处理速度, 在 80486 CPU 内又集成了 8KB Cache。将内存中经常被 CPU 使用到的那部分内容复制到 Cache 中, 并不断地更新, 使得 Cache 中总是保存着最近经常被 CPU 使用的一部分内容, 而不必总是去访问低速的存储器。

(3) 将原 80386 CPU 的指令译码和执行部件扩展成五段流水线, 进一步增强了并行处理能力。80486 最快能在每个 CPU 时钟内执行一条指令。

(4) 总线接口部件功能更强, 增加了一些新的引脚和指令, 支持外部的第二级高速缓存(L2 Cache)和多处理器系统。

80486 相当于以 80386 的 CPU 为核心, 内含 FPU 和 Cache 的微处理器, 再加上 80486 局部采用了 RISC(Reduced Instruction Set Computer, 精简指令集计算机)技术、时钟倍频技术和新的内部总线结构, 所以 80486 的综合性能指标有了极大的提高。

2.4.4 Pentium 系列微处理器

Pentium 是 Intel 80386/80486 微处理器的下一代产品。它的性能比 Intel 486 有较大的提高,但它仍与 Intel 8086/8088、80286、80386 和 80486 等 CPU 完全兼容。

1. Pentium 的主要性能

Pentium 与 Intel 486 相比有以下重大改进:

- (1) 超标量体系结构。
- (2) 动态转移预测。
- (3) 流水线浮点部件。
- (4) 增加一个 8 KB 片内超高速缓冲存储器。
- (5) 较强的错误检测和报告功能。
- (6) 增加了测试挂钩(边界扫描、探针方式)。

Pentium 在以与 Intel 80486 CPU 相同的频率工作时,整数运算性能提高 1 倍,浮点运算性能提高 5 倍。Pentium 的初始目标频率是 66MHz。它的两条流水线和浮点部件能够独立工作。每条流水线在一个时钟内发送一条常用的指令,在某些情况下,可以发送一条浮点指令。

浮点部件在 Intel 80486 的基础上进行重新设计,快速算法可使 ADD、MUL 和 LOAD 等公用指令的运算速度至少提高 3 倍。在应用程序中,利用指令调度和流水线重叠技术,运算速度可提高 5 倍以上。Pentium 处理器内部结构如图 2.7 所示。

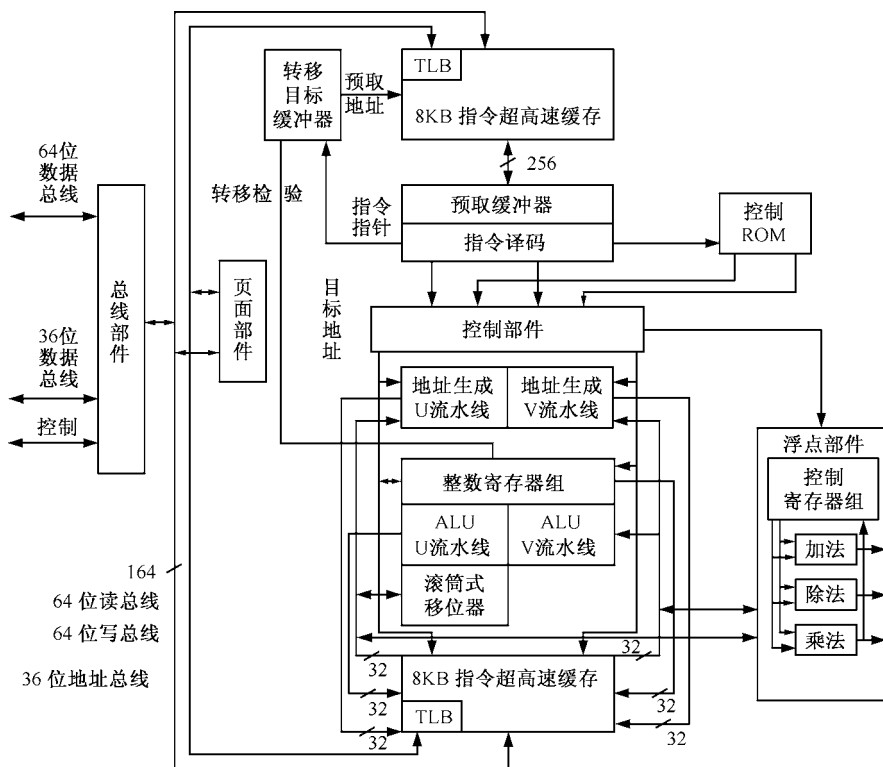


图 2.7 Pentium 处理器内部结构图

2. 超标量结构和超级流水线技术

Pentium 芯片内装有三种指令处理部件和 16KB~24KB 的超高速缓存。这些指令处理部件是两个 RISC(精简指令系统)型整数运算单元、80386 兼容单元和浮点运算单元。

RISC 型整数运算单元采用超标量技术实现, Pentium 内设两条指令流水线 U 和 V, 一个时钟周期能并行执行两条整数指令。U 流水线和 V 流水线都是五级流水线:

Prefetch 预取	Decode1 译码 1	Decode2 译码 2	Execute 执行	Write-back 写回结果
----------------	-----------------	-----------------	---------------	--------------------

Prefetch 同时取两条指令提供给 Decode1(译码 1)进行指令译码。译码后判别两指令能否同时执行, 若能同时执行, 便把经 Decode1 译码的指令分别送 U 流水线和 V 流水线, 在两条流水线中, 并行对它们进行进一步译码 Decode2, 执行和写回结果。事实上, 有时并不能同时执行两条指令, 这是由于数据相关、资源冲突和分支转移的缘故。在这种情况下, 只有把两条指令中的前一条指令送入 U 流水线, 待前一条指令执行完后再送入后一条指令。

U 流水线和 V 流水线是不等同、并且不能交换的。U 流水线能执行整数型和浮点型指令, V 流水线只能执行简单的整数型指令和两条异常的 FXCH(交换寄存器内容)指令, 因此, Pentium 能够在每个时钟内, 执行两条整数运算指令, 或者在每个时钟内执行一条浮点指令。如果两条浮点指令中有一条为 FXCH, 那么在一个时钟内可执行两条浮点指令。U 和 V 流水线都有其自己的生成逻辑、算术逻辑部件和数据超高速缓存接口。

Pentium 的 FPU 内置加法器、乘法器和除法器三种运算单元。采用超级流水线技术实现 FPU, 使 Pentium 改善了浮点处理能力, 并且还与 80486 的 FPU 兼容。Pentium 的 FPU 采用 8 级流水线, 1 到 5 级和 U 流水线共享。当执行浮点运算指令时, U 流水线及其第 4 级以后的控制从 ALU 移到 FPU。FPU 使用快速算法, 依靠先进的流水线结构和优化编译器, Pentium 的浮点处理能力能提高 4 倍~10 倍。

80386 兼容单元采用微代码处理指令, 负责处理不能用一个时钟完成执行的指令, 以及处理整数单元无法执行的多条指令。

3. 超高速缓存 Cache

Pentium 芯片上有两个独立的数据和指令超高速缓存, 容量均扩充 8KB~12KB, 是 80486 的 2 倍, 并在 80486 CPU 超高速缓存结构的基础上增加了一些功能, 以满足其性能指标的要求。Cache 单元为处理器提供快速访问指令和数据刷新途径, 这两个超高速缓存可以同时被访问, 不必花费很长时间。指令 Cache 可以提供多达 32B 的原始操作码, 数据 Cache 在每个时钟内可以提供两次数据访问的数据。每种 Cache 都使用物理地址进行访问, 并且都有自己的转换后援缓冲器(TLB), 将线性地址转换成所用的物理地址。Pentium 中分离的代码 Cache 使预取单元操作更有效, 它采用 256 位数据宽度的二路组相联设计, 它的填充是通过使用突发的存储传送周期来完成的。

指令 Cache、转换目标缓冲器(BPU)和预取缓冲器(PU)负责将原始指令送入 Pentium 的执行部件。指令取自指令 Cache 或外部总线, 转移地址由转移目标缓存器记录。指令 Cache 的 TLB 将线性地址转换成指令 Cache 所用的物理地址。指令译码器将预取的指令译码成 Pentium 可以执行的指令, 使 ROM 含有控制实现 Pentium 体系结构必须执行的运算顺序的

RF: 恢复标志。

6. 控制寄存器

CR4 控制寄存器结构如图 2.9 所示。

3	3	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0		
1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0								
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	M C E	P A E	P S E	D E	T S D	P V I	V M E

图 2.9 CR4 控制寄存器结构

PVI: 该位为 1 时，允许保护方式虚拟中断；该位为 0 时，禁止保护方式虚拟中断。

PAE: 该位为 1 时, 允许物理地址扩充; 该位为 0 时, 禁止物理地址扩充。

复位后 CR4 的内容为 0。

7. 模型专用寄存器

Pentium 有四个调试寄存器，用于检查编程时设置的断点是否匹配。这些断点通过四个

断点引脚 BP3~BP0 在外部表示是否匹配, 其中两个引脚 BP0(PM0/BP0)和 BP1(PM1/BP1) 供性能监测多路选择用。PM0 和 PM1 引脚用于外部表示与监测所选择事件相关的计数已增 1 或溢出。探针方式控制寄存器中的 PB1 和 PB0(性能监测/断点)位, 决定是否对性能监测或断点指示配置了多路选用引脚。

以上介绍了 80286、80386、80486 和 Pentium 微处理器, 下面通过表 2-2 对这几种微处理器进行比较。

表 2-2 80X86 微处理器功能比较

芯片	8086	80286	80386	80486	Pentium
位数	16	16	32	32	32
主频/MHz	5/8/10	6/8/10/12.5/20	16/20/33/40	25/33/66/80/100	66/75/100/366...
数据总线	16	16	32	32	64
地址线	20	24	32	32	32
寻址范围	1MB	16MB	4GB	4GB	4GB

2.5 小 结

本章针对 8086 微处理器及其体系结构做了详细介绍。8086 微处理器从功能结构上可以划分为执行部件和总线接口部件两大部分。这种并行工作方式, 减少了 CPU 等待取指令的时间, 充分利用了总线, 有力地提高了 CPU 的工作效率, 加快了整机的运行速度, 也降低了 CPU 对存储器存取速度的要求, 成为 8086 的突出优点。

8086 微处理器的寄存器使用非常灵活, 8086 CPU 可供编程使用的有 14 个 16 位寄存器, 按其用途可分为 3 类: 通用寄存器、段寄存器、指针和标志寄存器。8086 微处理器的存储器和 I/O 组织是本章的重点内容, 应掌握存储器的分段管理、物理地址和逻辑地址的换算及 I/O 端口的编址方式。本章最后对 80X86 的系列产品 80286、80386、80486、Pentium 系列等高档微处理器的特点及基本结构作了介绍, 方便大家了解更高档的微处理器, 加深对它们的认识。

Pentium 系列微机是目前 PC 机市场占有率最多的一种, Pentium、Pentium Pro、Pentium MMX、Pentium II、Pentium III、Pentium 4 等微处理器是当前普遍使用的微处理器。它们不仅增加了数据总线、地址总线的位数, 而且采用了许多新技术, 如指令高速缓存与数据高速缓存分离, 分支预测, 超标量流水线技术, RISC 结构, 增加了支持多媒体的指令集, 使微处理器性能大大增强。

2.6 习 题

1. 8086/8088 CPU 由哪两部分组成, 它们的主要功能是什么?
2. 8086/8088 CPU 数据总线和地址总线各是多少, 最大的物理存储空间是多少?
3. 8086/8088 的标志寄存器有哪些标志位? 它们的含义和作用是什么?

4. 8088/8086 CPU 中有几个通用寄存器？有几个变址寄存器？有几个指针寄存器？试说明各寄存器的作用。
5. 数据在存储器存放有何规定？什么是对准字？什么是非对准字？
6. 什么是逻辑地址？它由哪两部分组成？8086 的物理地址是如何形成的？
7. 8086/8088 系统为什么要采用存储器地址分段的寻址方法？
8. 一个由 20 个字组成的数据区，其起始地址为 A000H: 1000H。试写出数据区首末单元的物理地址。
9. 若 CS 为 B000H，试说明现行代码段可寻址存储空间的范围。
10. 80286 内部结构和功能特点是什么？它有多少条地址线？寻址范围为多少？80286 实地址方式可寻址范围为多少？
11. 80386 内部结构和功能特点是什么？它有多少条数据线？多少条地址线？其寻址范围为多少？
12. 80486 内部结构主要由哪几个部件组成？相对 80386 有哪些主要的增强功能？
13. 简述 Pentium 微处理器的主要特点。

第 3 章 指令系统与寻址方式

教学提示：指令系统是微处理器(CPU)所能执行的指令的集合，它与微处理器有密切的联系，不同的微处理器有不同的指令系统。本章主要讲解 Intel 公司生产的 8086/8088 CPU 的寻址方式以及各种指令系统，并通过具体实例，讲述各条指令的使用方法和功能。

教学要求：通过本章学习，应该掌握操作数的寻址方式，常用指令的格式、功能、使用方法及对标志位的影响。

3.1 寻 址 方 式

3.1.1 操作数类型

指令中操作的对象称为操作数。8086/8088 指令系统中，操作数分为两大类：

1. 数据操作数

这类操作数与数据有关，即指令中操作的对象是数据。数据操作数又分为：

- (1) 立即数操作数：指令中要操作的数据在指令中。
- (2) 寄存器操作数：指令中要操作的数据存放在指定的寄存器中。
- (3) 存储器操作数：指令中要操作的数据存放在指定的存储单元中。
- (4) I/O 操作数：指令中要操作的数据来自或要送到 I/O 端口。

2. 转移地址操作数

这类操作数与程序转移地址有关，即指令中要操作的对象不是数据，而是要转移的目标地址。可分为立即数操作数、寄存器操作数、存储器操作数，即要转移的地址包含在指令中、存放在寄存器中或存放在指定的存储单元中。

对于数据操作数，有的指令有两个操作数，一个为源操作数，另一个为目标操作数。有的指令只有一个操作数，有的指令没有操作数。

对于转移地址操作数，指令只有一个目标操作数，它是一个程序需转移的目标地址。

3.1.2 8086/8088 寻址方式

指令中提供操作数或操作数地址的方法称为寻址方式，即如何找到操作数的方法。根据操作数的种类，8086/8088 指令系统的寻址方式分为两大类：数据寻址方式和转移地址寻址方式。

1. 数据寻址方式

1) 立即数寻址方式

立即数寻址方式的特点是操作数直接包含在指令中，紧跟在操作码之后，它作为指令的一部分。立即数可以是 8 位的，也可以是 16 位的。如果是 16 位数，则高位字节存放在

高地址中，低位字节存放在低地址中。例如：

```
MOV  BL, 80H
MOV  AX, 1090H
```

则指令执行情况如图 3.1 所示。执行结果为：(BL)=80H，(AX)=1090H。

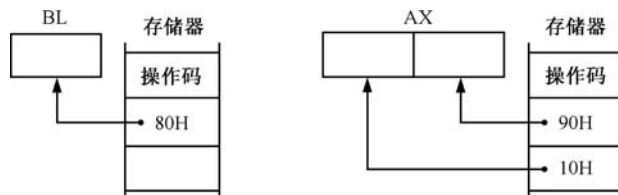


图 3.1 立即数寻址方式指令的执行情况

立即数寻址方式只能作为源操作数，主要用来给寄存器或存储单元赋值。

2) 寄存器寻址方式

寄存器寻址方式的操作数存放在指令规定的寄存器中，寄存器的名字可在指令中给出。对于 16 位操作数，寄存器可以是 AX、BX、CX、DX、SI、DI、SP 或 BP。对于 8 位操作数，寄存器可以是 AH、AL、BH、BL、CH、CL、DH 或 DL。例如：

```
MOV  CL, DL
MOV  AX, BX
```

如果(DL)=50H，(BX)=1234H，则执行结果为：(CL)=50H，(AX)=1234H。

寄存器寻址方式由于操作数就在寄存器中，不需要访问存储器来取得操作数，因而可以取得较高的运行速度，通常用于 CPU 内部操作。

3) 存储器寻址方式

存储器寻址方式的操作数存放在存储单元中，在指令中可以直接或间接给出存放操作数的地址，以达到存取操作数的目的。

(1) 直接寻址方式。

直接寻址的有效地址 EA(16 位偏移地址)在指令的操作码后面直接给出，它与指令的操作码一起，存放在存储器的代码段中，也是高位字节存放在高地址中，低位字节存放在低地址中。但是，操作数本身一般存放在存储器的数据段中。例如：

```
MOV  AL, [1064H]
```

如果(DS)=2000H，则指令执行情况如图 3.2 所示。执行结果为：(AL)=45H

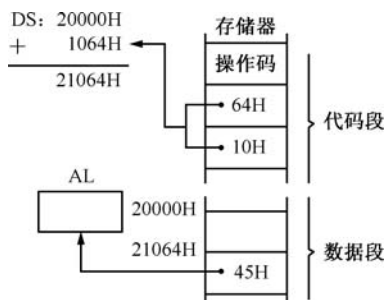


图 3.2 直接寻址方式指令的执行情况

注意这种直接寻址方式与前面已经介绍过的立即数寻址方式的不同。从指令的表示形式来看：在直接寻址方式中，对于表示有效地址的 16 位数，必须加上方括号。从指令的功能上看，本指令不是将立即数 1064H 传送到累加器 AL，而是将一个有效地址是 1064H 的存储单元的内容传送到 AL。设此时数据段寄存器 DS=2000H，则该存储单元的物理地址为：

$$\begin{aligned} & 2000\text{H} \times 10\text{H} + 1064\text{H} \\ & = 20000\text{H} + 1064\text{H} \\ & = 21064\text{H} \end{aligned}$$

如果没有特殊指明，直接寻址方式的操作数一般在存储器的数据段，即隐含的段寄存器是 DS。但是 8086/8088 也允许段超越，即允许使用 CS、SS 或 ES 作为段寄存器，此时需要在指令中特别标明。方法是在有关操作数的前面写上超越的段寄存器的名字，再加上冒号。

例如，若以上指令使用 ES 作为段寄存器，则指令应表示成为以下形式：

```
MOV AL, ES: [1064H]
```

在汇编语言指令中，可用符号地址代替数值地址，例如：

```
MOV AL, VALUE
```

或

```
MOV AL, [VALUE]
```

此时 VALUE 为存放操作数单元的符号地址。

(2) 寄存器间接寻址方式。

寄存器间接寻址方式与前面已经讨论过的寄存器寻址方式不同，指令中指定的寄存器(是一个 16 位寄存器)的内容不是操作数，而是操作数的有效地址，操作数本身则在存储器中。

寄存器间接寻址方式可用的寄存器有四个：SI、DI、BX 和 BP。寄存器间接寻址方式的有效地址

$$\begin{aligned} & \text{EA} = \begin{matrix} \text{[SI]} \\ \text{[DI]} \\ \text{[BX]} \\ \text{[BP]} \end{matrix} \end{aligned}$$

但若选择其中不同的间址寄存器，默认的段寄存器有所不同。若选择 SI、DI、BX 寄存器，则存放操作数的段寄存器默认为 DS；若选择 BP 寄存器，则存放操作数的段寄存器默认为 SS，使用时要注意。

寄存器间接寻址方式指令的书写应加方括号，避免与一般的寄存器寻址方式混淆。例

```
MOV AX, [SI]
MOV [BX], AL
```

如果(DS)=3000H, (SI)=2000H, (BX)=1000H, (AL)=64H, 指令执行情况如图 3.3 所示，执行结果为：(AX)=4050H, (31000H)=64H。

无论用 SI、DI、BX 或者 BP 作为间址寄存器，都允许段超越。

例如：

```
MOV ES: [DI], AX
MOV DX, DS: [BP]
```

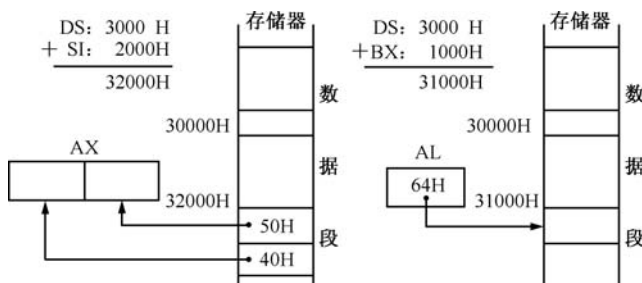


图 3.3 寄存器间接寻址方式指令的执行情况

(3) 寄存器相对寻址方式。

寄存器相对寻址方式的有效地址 EA 是一个由指令中的给定的 8 位或 16 位位移量和制定的寄存器内容相加之和。可用作寄存器相对寻址方式的寄存器有 SI、DI、BX 和 BP。

$$EA = \begin{matrix} [SI] \\ + [DI] \\ + [BX] \\ + [BP] \end{matrix} + \begin{matrix} \\ + [8 \text{ 位 disp}] \\ + [16 \text{ 位 disp}] \\ \end{matrix}$$

例如：

```
MOV [SI+10H], AX
MOV CX, [BX+COUNT]
```

如果(DS)=3000H, (SI)=2000H, (BX)=1000H, COUNT=1050H, (AX)=4050H 则指令执行情况如图 3.4 所示。执行结果为：(2010H)=4050H, (CX)=4030H。

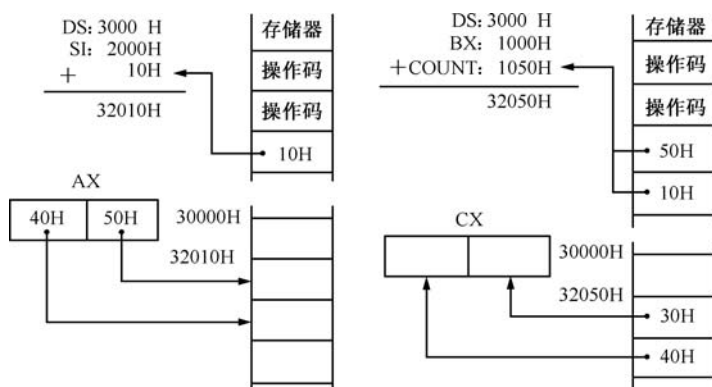


图 3.4 寄存器相对寻址方式指令的执行情况

寄存器相对寻址方式的操作数在汇编语言中书写时，可以是下述形式之一：

```
MOV AL, [BP+TABLE]
MOV AL, [BP] + TABLE
MOV AL, TABLE[BP]
```

其实以上三条指令实质上代表同一条指令。

在一般情况下，若指令中指定的寄存器是 SI、DI、BX，则存放操作数的段寄存器默认为 DS。若指令中指定 BP 寄存器，则对应的段寄存器应为 SS。

同样，寄存器相对寻址方式也允许段超越。

(4) 基址变址寻址方式。

有效地址是一个基址寄存器(BX 或 BP)和一个变址寄存器(SI 或 DI)的内容之和。

$$EA = \begin{matrix} [BX] & + & [SI] \\ [BP] & + & [DI] \end{matrix}$$

汇编语言中书写时，可以是下列形式之一：

```
MOV [BX+DI], AX
MOV AH, [BP][SI]
```

如果(DS)=3000H, (SS)=4000H, (BX)=1000H, (DI)=1100H, (AX)=0050H, (BP)=2000H, (SI)=1200H, (43200H)=56H, 则指令执行结果为：(32100H)=0050H, (AH)=56H。

在一般情况下，由基地址决定哪一个段寄存器作为地址指针。用 BX 作为基址，则操作数在数据段中。用 BP 作为基址，则操作数在堆栈段中。但在指令中规定了段超越可用其他段寄存器作为地址基准。

(5) 基址变址相对寻址方式。

基址变址相对寻址方式的有效地址是由指令中指定的 8 位或 16 位位移量(displacement)、一个基址寄存器内容和一个变址寄存器内容之和。即：

$$EA = \begin{matrix} [BX] & + & [SI] & + & [8 \text{ 位 disp}] \\ [BP] & + & [DI] & + & [16 \text{ 位 disp}] \end{matrix}$$

同样，当基址寄存器为 BP 时，操作数在堆栈段中，也允许段超越。

例如：

```
MOV AH, [BX+DI+1234H]
MOV [BP+SI+DATA], CX
```

若(DS)=4000H, (SS)=5000H, (BX)=1000H, (DI)=1500H, (BP)=2000H, (SI)=1050H, (CX)=2050H, DATA=1000H, (43734H)=64H, 则指令执行结果为：(AH)=64H, (54050H)=2050H。

基址加变址相对寻址方式也可以表示成几种不同的形式：

```
MOV AX, [BX+SI+COUNT]
MOV AX, COUNT[BX][SI]
MOV AX, [BX+COUNT][SI]
MOV AX, [BX]COUNT[SI]
MOV AX, [BX+SI]COUNT
MOV AX, COUNT[SI][BX]
```

4) I/O 端口寻址方式

I/O 端口寻址方式有以下两种：

(1) 直接端口寻址方式。

对这种寻址方式，端口地址用 8 位立即数(0~255)表示。例如：

```
IN AL, 21H
```

此指令表示从 I/O 端口地址为 21H 的端口中读取数据送到 AL 中。

(2) 间接端口寻址方式。

此时 I/O 端口的地址应事先存放在规定的 DX 寄存器中(0~65535)。例如:

```
MOV DX, 255
OUT DX, AL
```

前一条指令是将端口地址 255 送到 DX 寄存器, 后一条指令表示将 AL 中的内容输出到由 DX 寄存器内容所指定的端口中。

2. 转移地址的寻址方式

在 8086/8088 指令系统中, 有一组指令被用来控制程序的执行顺序。程序的执行是由 CS 和 IP 的内容所决定的, 通常情况下, 当 BIU 完成一次取指周期后, 就自动改变 IP 的内容, 以指向下一条指令的地址, 使程序按预先存放在程序存储器中的指令的次序, 由低地址到高地址顺序执行。如需要改变程序的执行顺序, 转移到所要求的指令地址再顺序执行时, 可以安排一条程序转移指令, 并按指令的要求修改 IP 内容或同时修改 IP 和 CS 的内容, 从而将程序转移到指令所指定的转移地址。程序转移的寻址方式就是找出程序转移的地址。转移地址可以在段内, 也可以跨段(称段间转移)。寻求转移地址的方法称为转移地址寻址方式, 有如下四种方式:

1) 段内直接寻址方式

段内直接寻址方式也称为相对寻址方式。转移的地址是当前的 IP 内容和指令规定的 8 位或 16 位位移量之和。当位移量是 8 位时, 称短程转移, 位移量是 16 位时, 称为近程转移。这种寻址方式适用于无条件转移和条件转移类指令。但条件转移类指令只有位移量为 8 位的短程转移。

```
JMP NEAR PTR PROGIA
JMP SHORT QUEST
```

其中, PROGIA 和 QUEST 均为转向的符号地址, 在机器指令中, 用位移量来表示; 在汇编语言中, 如果位移量是 16 位, 在符号地址前加 NEAR PTR; 如果位移量是 8 位, 在符号地址前加操作符 SHORT。

2) 段内间接寻址方式

程序转移的地址存放在寄存器或存储单元中。这个寄存器或存储单元的内容可以用数据寻址方式中除了立即数以外的任何一种寻址方式取得, 所得到转移的有效地址用来更新 IP 的内容。由于此寻址方式仅修改 IP 的内容, 所以这种寻址方式只能在段内进行程序转移。

条件转移类指令只能使用段内直接寻址方式的 8 位位移量。

段内间接寻址转移指令的格式可以表示为:

```
JMP BX
JMP WORD PTR [BP+TABLE]
```

其中 WORD PTR 为操作符, 用以指出其后的寻址方式所取得的转向地址是一个字的有效地址, 也就是说它是一种段内转移。

3) 段间直接寻址方式

这种寻址方式是在指令中直接给出 16 位的段基值和 16 位的偏移地址, 用来更新当前的 CS 和 IP 的内容。指令的格式可以表示为:

```
JMP FAR PTR NEXTROUTINT
```

其中，NEXTROUTINT 为转向的符号地址。FAR PTR 则表示段间转移的操作符。

4) 段间间接寻址方式

这种寻址方式是由指令中给出的存储器数据寻址方式字节，求出存放转移地址的连续两个字的地址。其低位字地址单元存放的是偏移地址，高位字地址单元中存放的是转移段基值。这样既更新了 IP 内容又更新了 CS 的内容，故称为段间间接寻址。这种指令的格式可以表示为：

```
JMP DWORD PTR [NTERS+BX]
```

[NTERS+BX]说明数据寻址方式为寄存器相对寻址方式，DWORD PTR 为双字操作符，说明此指令是转向地址需取双字的段间转移指令。

3.2 指令系统

8086/8088 的指令系统大致分为 6 种类型：

- (1) 数据传送指令。
- (2) 算术运算指令。
- (3) 位操作指令。
- (4) 串操作指令。
- (5) 程序控制指令。
- (6) 处理器控制指令。

学习指令系统要重点掌握指令的基本操作功能、合法的寻址方式以及对状态标志位的影响。

3.2.1 数据传送指令

数据传送指令是将数据、地址或立即数传送到寄存器或存储单元中。它又可分为通用数据传送指令、输入输出指令、目的地址传送指令和标志传送指令等四组。

数据传送指令对状态标志位不发生影响，除(SAHF 和 POPF)例外，下面分别进行讨论。

1. 通用数据传送指令

1) 数据传送指令

指令格式及操作：

```
MOV dst, src ; (dst) ← (src)
```

指令中，dst 为目的操作数，src 为源操作数，指令实现的操作是将源操作数传送到目的操作数地址。这种传送实际上是进行数据的“复制”，将源操作数复制到目标操作数地址中，源操作数保持不变。

双操作数的书写方法一般总是将目标操作数写在前面，源操作数写在后面，两者之间用逗号隔开。在 MOV 指令中，源操作数可以是寄存器、存储器、段寄存器和立即数；目标操作数可以是寄存器(不能为 IP)、存储器、段寄存器(不能为 CS)。除了源操作数和目标

操作数不能同时为存储器、段寄存器、立即数送段寄存器外，可任意搭配。数据传送的方向如图 3.5 所示：



图 3.5 MOV 指令的数据传送方向

说明：对于 CS 和指针寄存器 IP，通常无需用户利用传送指令改变其内容。

2) 堆栈操作指令

堆栈操作指令是用来完成压入和弹出堆栈操作的。

(1) 压入堆栈指令。

指令格式及操作：

```
PUSH src ; SP ← SP - 2, (SP) + 1: (SP) ← (src)
```

指令完成的操作是“先减后压”，先将指针 SP 减 2，然后再将操作数 src 压入由 SP 指出的栈顶中，指令中的操作数可以是通用寄存器、段寄存器、存储器，但不能是立即数。

```
PUSH AX ; SP ← SP - 2, (SP) + 1 ← (AH), (SP) ← (AL)
PUSH CS
PUSH [SI]
```

(2) 弹出堆栈指令。

指令格式及操作：

```
POP dst ; dst ← ((SP) + 1, (SP)), SP ← (SP) + 2
```

指令完成的操作是“先出后移”，即先将堆栈指针 SP 所指示的栈顶存储单元的值弹出到操作数 dst 中，然后再将堆栈指针 SP 加 2。指令中的操作数可以是通用寄存器、存储器、段寄存器(但不能是代码段寄存器 CS)，同样，不能是立即数。例如：

```
POP BX ; BH ← ((SP) + 1), BL ← ((SP)), SP ← (SP) + 2
POP ES
POP MEM[DI]
```

应该注意，堆栈指令中操作数一定是字操作数(16 位)。

3) 数据交换指令

指令格式及操作：

```
XCHG opr1, opr2 ; opr1 ↔ (opr2)
```

这是一条交换指令，它的操作是使源操作数与目标操作数进行交换，即不仅将源操作数传送到目标操作数，而且同时将目标操作数传送到源操作数。交换指令的源操作数与目标操作数均可以是通用寄存器、存储器，但不能同时为存储器。

4) 字节转换指令

指令格式及操作：

```
XLAT src-table
```

XLAT 指令是字节查表转换指令，可以根据表中元素序号，查出表中相应元素的内容。为了实现查表转换，预先应将表的首地址，即表头地址传送到 BX 寄存器，元素的序号即位移量送 AL，表中第一个元素的序号为 0，然后依次是 1，2，3，…。执行 XLAT 指令后，表中指令序号的元素存于 AL。由于借助了 AL 寄存器进行，所以被寻址的表的最大长度为 255 个字节。利用 XLAT 指令实现不同数制或编码系统之间的转换十分方便。

【例 3.1】内存的数据段有一个 16 进制的 ASCII 码表，其首地址为 Hex-table，如图 3.6 所示，要查出第 10 个元素即“A”的 ASCII 码，实现方法如下：

```
MOV BX, OFFSET Hex-table    ; BX+表首址
MOV AL, 0AH                 ; AL←序号
XLAT Hex-table               ; 查表转换
```

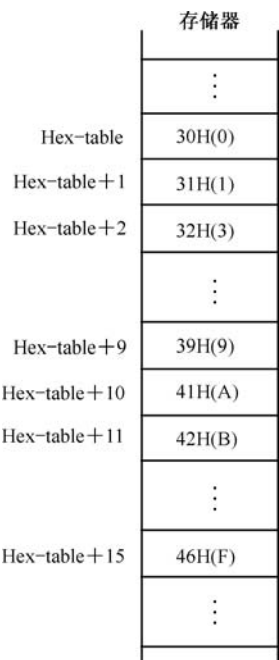


图 3.6 16 进制数的 ASCII 码表

结果“A”的 ASCII 码在 AL 中，即(AL)=41H。
上例中查表转换指令后面的操作数为首地址 Hex-table(类型为字节)，它实际上已经预先传送到 BX 寄存器中，写在 XLAT 指令中是为了用汇编程序检查类型的正确性。但是 XLAT 指令后面也可以不写操作数。

BX 寄存器中包含着表的首地址，所在的段由隐含值确定。但也允许段超越，此时必须在指令中写明重设的段寄存器。XLAT 指令的几种表示形式如下：

```
XLAT                ; 不写操作数
XLAT src-table      ; 写操作数
XLATB               ; B 表示字节类型，不允许再写操作数
XLAT ES: src-table  ; 重设段寄存器为 ES
```

2. 输入输出指令

输入输出指令共有两条。输入指令 IN 用于从外设端口接收数据，输出指令 OUT 则向端口发送数据。无论是接收到的数据或是准备发送的数据都必须在累加器 AL(字节)或 AX(字)中，所以这是两条累加器专用指令。

输入输出指令可以分为两大类：一类是端口直接寻址的输入输出指令；另一类是端口通过 DX 寄存器间接寻址的输入输出指令。在直接寻址的指令中只能寻址 256 个端口(0~255)，而间接寻址的指令中可寻址 64 个端口(0~65535)。

1) 输入指令

(1) 直接寻址的输入指令。

指令格式及操作：

```
IN  acc, port ; acc ← (port)
```

此指令是将 8/16 位数据直接经输入端口 port(地址 0~255)送入 AL/AX 累加器中。

(2) 间接寻址的输入指令。

指令格式及操作：

```
IN  acc, DX ; acc ← (DX)
```

此指令是从 DX 寄存器内容指定的端口中，将 8/16 位数据送入 AL/AX 累加器中。这种寻址方式的端口地址由 16 位地址表示，执行此指令前应将 16 位地址存入 DX 寄存器中。

2) 输出指令

(1) 直接寻址的输出指令。

指令格式及操作：

```
OUT port, acc ; port ← (acc)
```

此指令是从 AL(8 位)或 AX(16 位)累加器输出 8/16 位数据到指令指定的 I/O 端口中。

(2) 间接寻址的输出指令。

指令格式及操作：

```
OUT DX, acc ; (DX) ← (acc)
```

此指令是从 AL(8 位)或 AX(16 位)累加器中输出 8/16 位数据到由 DX 寄存器内容指定的 I/O 端口中。

3. 目的地址传送指令

8086/8088 CPU 提供了三条把地址指针写入寄存器或寄存器对的指令，它们可以用来写入近地址指针和远地址指针。

1) 取有效地址指令

指令格式：

```
LEA reg16, mem
```

LEA 是将一个近地址指针写入到指定的寄存器。指令中的目标操作数必须是一个 16 位的通用寄存器，源操作数必须是一个存储器操作数，指令的执行结果是把源操作数的有效地址，即 16 位的偏移地址源传送到目标寄存器。例如：

```
LEA  BX, BUFFER
LEA  AX, [BP] [DI]
LEA  DX, BETA[BX] [SI]
```

注意 LEA 指令与 MOV 指令的区别，比较下面两条指令：

```
LEA  BX, BUFFER
MOV  BX, BUFFER
```

前者将存储器 BUFFER 的有效地址送到 BX，而后者是将 BUFFER 的内容送到 BX。
以下两条指令功能相同：

```
LEA  BX, BUFFER
MOV  BX, OFFSET BUFFER
```

其中 OFFSET BUFFER 表示存储器 BUFFER 偏移地址。

2) 地址指针装入 DS 指令

指令格式：

```
LDS  reg16, mere32
```

LDS 指令和下面即将介绍的 LES 指令都是用于写入远地址指针，源操作数可以是任意存储器，目标操作数是任意 16 位通用寄存器。LDS 传送 32 位远地址指针，前者送指定寄存器，后者送数据段寄存器 DS。例如：

```
LDS  SI, [0010H]
```

设原来 DS=C000H，而有关存储单元的内容为(C0010H)=80H，(C0011H)=01H，(C0012H)=00H，(C0013H)=20H，则执行以上指令后，SI 寄存器的内容为 0180H，段寄存器 DS 的内容为 2000H。

3) 地址指针装入 ES 指令

指令格式：

```
LES  reg16, mere32
```

LES 指令与 LDS 类似，也是装入一个 32 位的远地址指针，偏移量送到指定寄存器，段基值送到附加段寄存器 ES。

目的地址传送指令常常用于在串操作时建立初始的地址指针。

4. 标志传送指令

CPU 中有一标志寄存器 FLAG，其中每一状态标志位表示 CPU 运行的状态。许多指令执行结果会影响标志寄存器的某些状态标志位。同时，有些指令的执行也受标志寄存器中某些位的控制。标志传送指令共四条，均是单字节指令，指令的操作数以隐含的形式存在(隐含的操作数是 AH 寄存器)。

1) 取标志指令

指令格式：

```
LAHF
```

LAHF 指令将 FLAG 中的五个标志位传送到累加器 AH 的对应位，如图 3.7 所示。

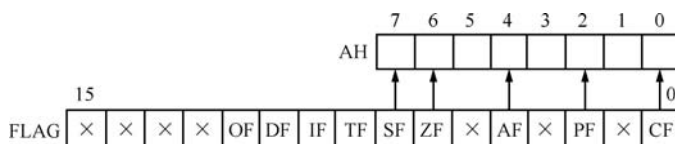


图 3.7 LAHF 指令操作示意图

LAHF 指令对状态标志位没有影响。

2) 置标志指令

指令格式：

SAHF

SAHF 指令的传送方向与 LAHF 相反，将 AH 寄存器中的第 7、6、4、2、0 位分别传送到标志寄存器对应位的状态，但其余状态标志位即 OF、DF、IF 和 TF 不受影响。

3) 标志压入堆栈指令

指令格式及操作：

PUSHF; $SP \leftarrow (SP) - 2$, $(SP) + 1: (SP) \leftarrow (FLAG)$

PUSHF 指令先将 SP 减 2，然后将标志寄存器 FLAG 的内容(16 位)压入堆栈。这条指令对状态标志位没有影响。

4) 标志弹出堆栈指令

指令格式及操作：

POPF; $FLAG \leftarrow ((SP) + 1, (SP))$, $SP \leftarrow (SP) + 2$

POPF 指令的操作与 PUSHF 相反，它将堆栈内容弹出到标志寄存器，然后 SP 加 2。POPF 指令对状态标志位有影响。

PUSHF 和 POPF 指令可用于保护调用过程以前标志寄存器的值，过程返回以后再恢复这些标志状态，或用来修改标志寄存器中相应标志位的值。

3.2.2 算术运算指令

8086/8088 的算术运算指令可处理四种类型的数：无符号的二进制数、带符号的二进制数、无符号压缩十进制数(压缩 BCD 码)、无符号非压缩十进制数(非压缩 BCD 码)。压缩十进制数只有加/减运算，其他类型的数均可进行加、减、乘、除运算。

8086/8088 提供了各种调整操作指令，因此除了可以对二进制数进行算术运算以外，也可以方便地进行压缩的或非压缩的十进制数的算术运算。

8086/8088 的算术运算指令将运算结果的某些特性传送到 6 个标志上去，这些标志中的绝大多数可由跟在算术运算指令后的条件转移指令进行测试，以改变程序的流程。因而掌握指令执行结果对标志的影响，对编程有着重要的作用。

算术运算类指令共有 20 条，包括加、减、乘、除运算、符号扩展和 BCD 码调整指令，除了符号扩展指令，其余均影响标志位。

1. 加法指令

加法指令包括不带进位加法指令、带进位加法指令和加 1 指令。

1) 加法指令

指令格式及操作:

```
ADD dst, src; (dst) ← (dst) + (src)
```

ADD 将目标操作数与源操作数相加, 结果存入目标操作数, 影响标志寄存器。ADD 指令的操作数类型与 MOV 指令类似, 但段寄存器不参与运算。例如:

```
ADD CL, 1
ADD DX, SI
ADD AX, MEM
ADD DATA[BX], AL
ADD ALP [DI], 30H
```

相加的数据类型可以根据编程者的意图, 规定为带符号数或无符号数, 如果相加结果超出范围, 则发生溢出。例如:

```
MOV AL, 7EH
MOV BL, 5BH
ADD AL, BL
```

执行以上三条指令以后, 相加结果(AL)=D9H, 此时各状态标志位的状态为: SF=1, ZF=0, AF=1, PF=0, CF=0, OF=1。其中 OF=1 表示发生了溢出, 这是由于相加结果超过了 127。但最高位并未产生进位, 故 CF=0。

2) 带进位加法指令

指令格式及操作:

```
ADC dst, src ; (dst) ← (dst) + (src) + (CF)
```

ADC 指令是将目标操作数与源操作数相加, 再加上进位标志 CF 的内容, 然后将结果送目标操作数。操作数的类型与 ADD 指令相同, 而且 ADC 指令同样也可以进行字节操作或字操作。

带进位加法指令主要用于多字节数据的加法运算。如果低字节相加时产生进位, 则在下一次高字节相加时将这个进位加进去。

【例 3.2】 要求计算两个多字节十六进制数之和: 3B74AC60F8H+20D59E36C1H=?

式中被加数和加数均有五个字节, 可以编一个循环程序实现以上运算。假设已将被加数和加数分别存入从 DATA1 和 DATA2 开始的两个内存区, 且均为低位字节在前, 高位字节在后, 如图 3.8 所示。要求相加所得结果仍存回以 DATA1 为首址的内存区。

程序流程图如图 3.9 所示。源程序如下:

```
MOV CX, 5 ; 设置循环次数
MOV SI, 0 ; 置位移量初值
CLC ; 清进位
LOOPER: MOV AL, DATA2[SI] ; 取另一个加数
ADC DATA1[SI], AL ; 和另一个加数相加
INC SI ; 位移量加 1
DEC CX ; 循环次数减 1
JNZ LOOPER ; 加完否, 若没完, 转 LOOPER, 继续相加
HLT ; 程序暂停
```

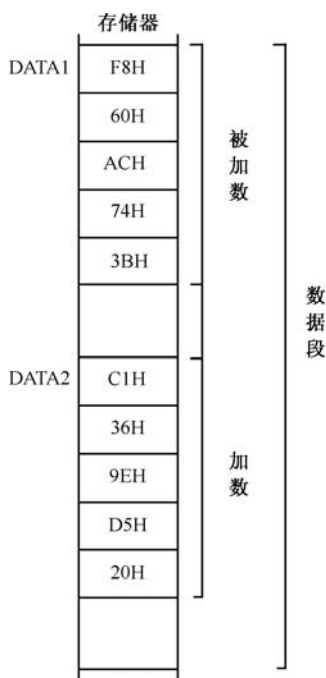


图 3.8 操作数的存放情况

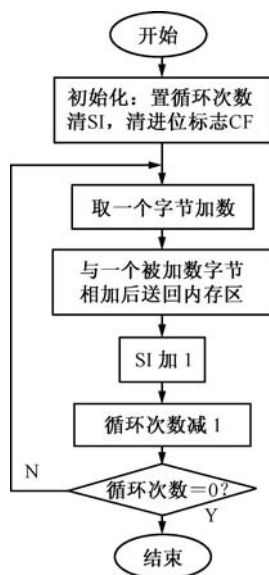


图 3.9 程序流程图

3) 加 1 指令

指令格式及操作：

```
INC dst ; dst ← (dst) + 1
```

INC 指令将目标操作数加 1。指令影响 SF、ZF、AF、PF、OF，但对 CF 没影响。INC 指令的目标操作数可以是寄存器或存储器，但不能是段寄存器。其类型为字节操作或字操作均可。例如：

```
INC DL
INC SI
INC BYTE PTR [BX][SI]
INC WORD PTR [DI]
```

指令中的 BYTE PTR 或 WORD PTR 分别指定随后的存储器操作数的类型是字节或字。INC 指令常常用于在循环程序中修改地址。

2. 减法指令

减法指令包括不带借位减法指令、带借位减法指令、减 1 指令、求补指令和比较指令。

1) 减法指令

指令格式及操作：

```
SUB dst, src; dst ← (dst) - (src)
```

SUB 指令用目标操作数减源操作数，结果送回目标操作数。该指令对状态标志位有影响。

操作数的类型与加法指令一样，即目标操作数可以是寄存器或存储器，源操作数可以是立即数、寄存器或存储器，但不允许两个存储器相减。既可以字节相减，也可以字相减。

例如：

```
SUB  AL, 37H,  
SUB  DX, BX  
SUB  CX, VARE1  
SUB  ARRAY[DL], AX  
SUB  BETA[BX][DL], 512H
```

减法数据的类型也可以根据程序员的要求，约定为带符号数或无符号数。

2) 带借位的减法指令

指令格式及操作：

```
SBB  dst, src;  $ds \leftarrow (dst) - (src) - (CF)$ 
```

SBB 指令是将目标操作数减源操作数，然后再减进位标志 CF，并将结果送回目标操作数，SBB 指令对标志的影响与 SUB 指令相同。

目标操作数及源操作数的类型也与 SUB 指令相同。8 位或 16 位数均可运算。例如：

```
SBB  BX, 1000H  
SBB  CX, DX  
SBB  AL, DATA1  
SBB  DISP[BP], BL  
SBB  [SI+6], 97H
```

带借位减指令主要用于多字节的减法。

3) 减 1 指令

指令格式及操作：

```
DEC  dst,  $dst \leftarrow (dst) - 1$ 
```

DEC 指令将目标操作数减 1。指令对状态标志位 SF、ZF、AF、PF 和 OF 有影响，但不影响进位标志 CF。操作数与 INC 一样，可以是寄存器或存储器(段寄存器不可)。其类型是字节操作或字操作均可。例如：

```
DEC  BL  
DEC  AX  
DEC  [BX]  
DEC  WORD PTR [BP][DL]
```

在循环程序中常常利用 DEC 指令来修改循环次数。例如：

```
MOV  AX, 0FFFFH  
CYC: DEC  AX  
JNZ  CYC  
HLT
```

以上程序段中，DEC AX 指令重复执行 65535 次，此程序是一段延时程序。

4) 求补指令

指令格式及操作：

```
NEG  dst ;  $dst \leftarrow 0 - (dst)$ 
```

NEG 指令的操作是用“0”减去目标操作数，结果送回原来的目标操作数。对状态标志位有影响。可以对 8 位数或 16 位数求补，实际为求负。例如：


```

NEG BL
NEG AX
NEG BYTE PTR [BP][SI]
NEG WORD PTR [DI+20]

```

利用 NEG 指令可以得到负数的绝对值。

【例 3.3】 内存数据段存放了 100 个带符号字节型数据，首地址为 AREA1，要求将各数取绝对值后存入以 AREA2 为首址的内存区。

由于 100 个带符号数中可能既有正数，又有负数，因此先要判断正负。如为正数，可以原封不动地传送到另一内存区；如为负数，则需先求补即可得到负数的绝对值，然后再传送。程序如下：

```

LEA SI, AREA1      ; SI 为源地址指针
LEA DI, AREA2      ; DI 为目的地址指针
MOV CX, 100        ; CX 为循环次数
CHECK: MOV AL, [SI] ; 取一个带符号数到 AL
OR AL, AL          ; AL 内容不变，但使之影响标志
JNS NEXT          ; 若 (SF)=0，则转 NEXT
NEG AL             ; 否则求补
MOV [DI], AL       ; 传送到目的地址
INC SI             ; 源地址加 1
INC DI             ; 目的地址加 1
DEC CX             ; 循环次数减 1
JNZ CHECK          ; 如不等于零，则转 CHECK
HLT                ; 停止

```

5) 比较指令

指令格式及操作：

```
CMP dst, src ; dst-(src)
```

CMP 用目标操作数减源操作数，但结果不送回目标操作数。因此，执行比较指令 CMP 后，被比较的两个操作数内容均保持不变，而比较结果反映在标志寄存器中。这是 CMP 比较指令与 SUB 区别所在；比较指令目标操作数可以是寄存器或存储器，源操作数可以是立即数、寄存器或存储器，但不能同时为存储器。可以进行字节比较，也可以进行字比较。例如：

```

CMP AL, 0AH        ; 寄存器与立即数比较
CMP CX, DI         ; 寄存器与寄存器比较
CMP AX, AREA1      ; 寄存器与存储器比较
CMP [BX+5], SI     ; 存储器与寄存器比较
CMP GAMM, 100      ; 存储器与立即数比较

```

3. 乘法指令

8086/8088 指令系统中有两条乘法指令，可以实现无符号数的乘法和带符号数的乘法，它们只有源操作数，隐含目标操作数。CPU 在执行乘法时，一个操作数始终放在累加器中 (8 位 AL；16 位 AX)，这是隐含的。8 位数相乘结果 16 位，存放在 AX 中，16 位数相乘结果 32 位，存放在 DX 和 AX 中。乘法运算的操作数及运算结果示意图如图 3.10 所示：

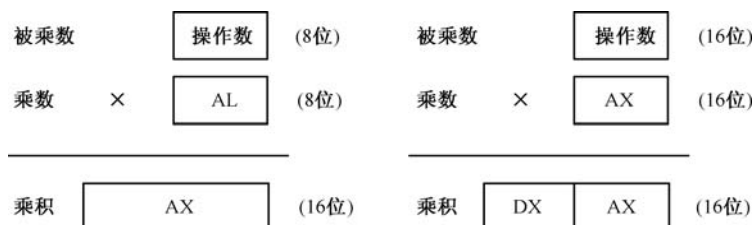


图 3.10 乘法运算的操作数及运算结果示意图

1) 无符号数乘法指令

指令格式及操作：

```
MUL src    ; AX ← (src) × (AL) (字节乘法)
            ; DX: AX ← (src) × (AX) (字乘法)
```

MUL 指令对状态标志位 CF、OF 有影响，SF、ZF、AF、PF 不确定。

```
MUL AL      ; AL 乘 AL
MUL BX      ; AX 乘 BX
MUL BYTE PTR [DI+6] ; AL 乘存储器 (8 位)
MUL WORD PTR ALPHA ; AX 乘存储器 (16 位)
```

两个 8 位数相乘，乘积可能有 16 位，结果存放在 AX 中；两个 16 位数相乘，乘积可能有 32 位，存放在 DX 高 16 位，AX 低 16 位，如果运算结果高位(AH 或 DX)为零，则状态标志位 CF=OF=0，否则 CF=OF=1，此时表示 AH 或 DX 中包含乘积的有效位。

```
MOV AL, 14H    ; (AL)=14H
MOV CL, 05H    ; (CL)=05H
MUL CL         ; (AX)=0064H, (CF)=(OF)=0
```

本例中结果的高半部分(AH)=0，因此，状态标志位(CF)=(OF)=0。

有了乘法(和除法)指令，使有些运算程序的编程变得简单方便。但是必须注意，乘法指令的执行速度很慢，除法指令也是如此。

2) 带符号数的乘法

指令格式：

```
IMUL src    ; AX ← (src) × (AL) (字节乘法)
            ; DX: AX ← (src) × (AX) (字乘法)
```

IMUL 指令对状态标志位的影响以及操作过程均与 MUL 指令相同。但 IMUL 指令进行带符号数乘法，指令将两个操作数均认作带符号数，8 位和 16 位带符号数的取值范围分别是 -128~+127(字节)和 -32768~+32767。

```
MOV AX, 04E8H  ; (AX)=04E8H
MOV BX, 4E20H  ; (BX)=4E20H
IMUL BX        ; (DX: AX) = (AX) × (BX)
```

以上指令的执行结果(DX)=017FH，(AX)=4D00H，且(CF)=(OF)=1。实际上，以上指令完成带符号数(+1256)和(+20000)的乘法运算，得到乘积为(+25120000)。由于此时 DX 中结果的高半部分包含着乘积的有效数字，故标志位(CF)=(OF)=1。

4. 除法指令

8086/8088 CPU 执行除法时规定：除数只能是被除数的一半字长。当被除数为 16 位时，除数应为 8 位，被除数为 32 位时，除数应为 16 位。并规定：

(1) 当被除数为 16 位，应存放于 AX 中。除数为 8 位，可存放在寄存器/存储器中。而得到的 8 位商放在 AL 中，8 位余数放在 AH 中。如图 3.11(a)所示。

(2) 当被除数为 32 位，应存放于 DX 和 AX 中。除数为 16 位，可存放在寄存器/存储器中。而得到的 16 位商放在 AX 中，16 位余数放在 DX 中。如图 3.11(b)所示。

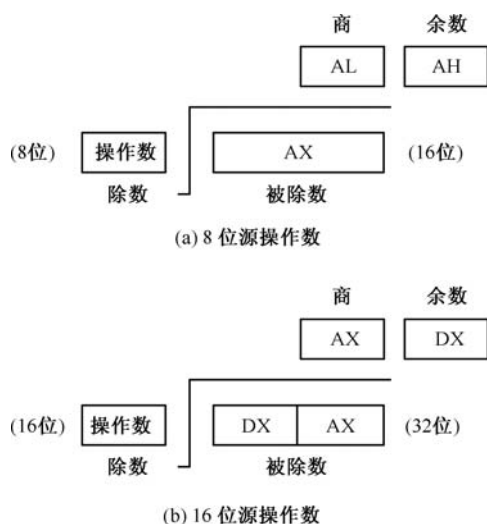


图 3.11 除法运算的操作数和运算结果

8086/8088 指令系统中有两条除法指令，它们是无符号数除法指令和带符号数的除法指令。

1) 无符号数除法指令

指令格式及操作：

```

DIV src      ; AL ← (AX)/(src) 的商 (字节除法)
              ; AH ← (AX)/(src) 的余数
              ; AX ← (DX: AX)/(src) 的商 (字除法)
              ; DX ← (DX: AX)/(src) 的余数
  
```

即字节除法中，AX 除以 src，被除数为 16 位，除数为 8 位。执行 DIV 指令后，商在 AL，余数在 AH 中；字除法中，DX、AX 除以 src，被除数为 32 位，除数为 16 位，除的结果，商在 AX，余数在 DX 中。执行 DIV 指令时，如果除数为 0，或字节除法时，AL 寄存器中的商大于 FFH，或字除法时，AX 寄存器中的商大于 FFFFH，CPU 立即自动产生类型号为 0 的中断。

DIV 指令对状态标志位 CF、OF、SF、ZF、AF、PF 的影响不确定。

```

DIV BL       ; AX 除以 BL
DIV CX       ; (DX: AX) 除以 CX
DIV WORD PTR ALPHA ; (DX: AX) 除以存储器
  
```

除法指令规定了必须用一个 16 位数除以一个 8 位数, 或用一个 32 位数除以一个 16 位数, 而不允许两个字长相等的操作数相除。如果被除数和除数的字长相等, 可以在用 DIV 指令进行无符号数除法之前, 将被除数的高位扩展 8 个零或 16 个零。

2) 带符号除法指令

指令格式及操作:

```
IDIV src    ; AL ← (AX)/(src) 的商(字节除法)
              ; AH ← (AX)/(src) 的余数
              ; AX ← (DX: AX)/(src) 的商(字除法)
              ; DX ← (DX: AX)/(src) 的余数
```

执行 IDIV 指令时, 如果除数为 0, 或字节除法时, AL 寄存器中的商超出(-128~+127), 或字除法时, AX 寄存器中的商超出(-32768~+32767), CPU 立即自动产生类型为 0 的中断。

IDIV 指令对状态标志位 CF、OF、SF、ZF、AF、PF 的影响不确定。

例如: 如果被除数和除数字长相等, 则在用 IDIV 指令进行带符号数除法中, 用扩展指令 CBW 或 CWD 将被除数的符号位扩展, 使之成为 16 位数或 32 位数。IDIV 指令对非整数商舍去尾数, 而余数的符号总是与被除数的符号相同。字长相等的带符号数除法的例子:

```
MOV AX, -2000    ; (AX) = -2000
CWD              ; 将 AX 中的 16 位数扩展成为 32 位, 结果在 DX: AX
MOV BX, -421     ; (BX) = -421
IDIV BX          ; (AX) = 4 (商), (DX) = -316 (余数), 余数的符号与被除数相同
```

5. 符号扩展指令

在前面介绍的各种二进制算术运算指令中, 两个操作数的字长应该符合规定的关系。例如, 在加法、减法和乘法运算中, 两个操作数的字长必须相等。在除法指令中, 被除数必须是除数的双倍字长。因此, 有时需要将一个 8 位数扩展成为 16 位, 或者将一个 16 位数扩展成为 32 位。

对于无符号数, 扩展字长比较简单; 只需添上足够个数的零即可。例如, 以下两条指令将 AL 中的一个 8 位无符号数扩展成为 16 位, 存放在 AX 中。

```
MOV AL, 0FBH    ; (AL) = 11111011B
XOR AH, AH      ; (AH) = 00000000B
```

但是对于带符号数, 扩展字长时正数与负数的处理方法不同, 正数的符号位为零, 而负数的符号位为 1, 因此, 扩展字长时, 应分别在高位添上相应的符号位, 这样才能保证原数据的大小和符号不变。符号扩展指令就是用来对带符号数字长的扩展。

1) 字节扩展指令

指令格式及操作:

```
CBW    ; 如果 (AL) < 80H, 则 (AH) = 00H, 否则 (AH) = FFH
```

CBW 指令将一个字节(8 位), 按其符号扩展成字, 它是隐含操作数指令, 隐含操作数为 AL 和 AH, 对状态标志位没有影响。

```

(1) MOV AL, 4FH
    CBW      ; (AH)=00000000B
(2) MOV AL, 0F4H
    CBW      ; (AH)=11111111B

```

2) 字扩展指令

指令格式及操作:

CWD

CWD 指令将一个字(16 位) 按其符号扩展成双字(32 位), 它是隐含操作数指令, 隐含的操作数为寄存器 AX 和 DX 中的值。CWD 指令与 CBW 一样, 对状态标志位没有影响。

CBW 和 CWD 指令在带符号数的乘法(IMUL)和除法(IDIV)运算中十分有用, 常常在字节或字的运算之前, 将 AL 和 AX 中数据的符号位进行扩展。例如:

```

MOV AL, MUL-BYTE      ; (AL) ← 8 位被乘数(带符号数)
CBW                    ; 扩展成为 16 位带符号数, 在 AX 中
IMUL RSRC-WORD        ; 两个 16 位带符号数相乘, 结果在 DX: AX 中

```

6. 十进制数(BCD 码)运算指令

以上介绍的是二进制数的算术运算。二进制数在计算机上进行运算是非常简单的。但是, 通常人们习惯于用十进制数。在计算机中, 十进制数是用 BCD 码来表示的。BCD 码有两类: 压缩十进制数(压缩 BCD 码)和无符号非压缩十进制数(非压缩 BCD 码), 8086/8088 用 BCD 码的运算指令算出结果, 然后再用专门的指令对结果进行修正(调整), 使之转变为 BCD 码表示的正确的结果。

下面通过几个例子说明 BCD 码运算为什么要调整以及怎样进行调整:

34+23=57	
0011 0100	34的BCD码
+	0010 0011
0101 0111	23的BCD码
	57的BCD码

结果正确, 这时调整指令不需要做什么。

48+29=77	
0100 1000	48的BCD码
+	0010 1001
0111 0001	29的BCD码
	77的BCD码

结果不正确, 因为在进行二进制加法运算时, 低 4 位向高 4 位有一个进位, 这个进位是按十六进制进行的, 即低 4 位逢十六才进一, 而十进制数应是逢十进一。因此, 比正确结果少 6, 这时, 调整指令应在低 4 位上加 6。即:

0100 1000	
+	0010 1001
0111 0001	中间结果 AF=1
+	0000 0110
0111 0111	加 06H 调整
	正确结果

```

57+46=103
      0101 0111
+     0100 0110
-----
      1001 1101      中间结果
+     0000 0110
-----
      1010 0011      中间结果
+     0110 0000
-----
CF←1  0000 0011      正确结果  CF=1

```

加法运算后，低 4 位 > 9，调整指令加 06H；高 4 位 > 9，调整指令加 60H。

```

72+91=163
      0111 0010
+     1001 0001
-----
CF←1  0000 0011      中间结果

+     0110 0000
-----
      0110 0011      正确结果

```

加法指令，CF=1 时要加 60H。

1) 十进制加法的调整指令

根据 BCD 码的种类，对 BCD 码加法进行十进制调整的指令有两条：AAA 和 DAA。

(1) 非压缩型 BCD 码调整指令。

指令格式：

AAA

AAA 也称为加法的 ASCII 调整指令。指令后面不写操作数，但实际上隐含累加器操作数 AL 和 AH。指令的操作为：

```

如果  (AL) ∧ 0FH > 9, 或 (AF) = 1
则     AL ← (AL) + 06H
        AH ← (AH) + 1
        AF ← 1
否则   CF ← (AF)
        AL ← ((AL) ∧ 0FH)

```

AAA 指令对 AF 和 CF，OF、SF、ZF、PF 的影响不确定。AAA 指令能对加法的结果 AL 的内容进行调整。

AAA 指令调整可用以下指令实现：

```

MOV  AX, 0007H    ; (AL)=07H, (AH)=00H
MOV  BL, 08H      ; (BL)=08H
ADD  AL, BL        ; (AL)=0FH
AAA                      ; (AL)=05H, (AH)=01H, (CF)=(AF)=1

```

以上指令的运行结果为 7+8=15，所得之和也以非压缩型 BCD 码的形式存放，个位在 AL，十位在 AH。

【例 3.4】 计算 4609+3875=?

本例要求实现十进制多位数的加法，假设被加数的每一位数都以 ASCII 码形式；存放在内存中，低位在前，高位在后。另外留出 4 个存储单元，以便存放相加所得的结果，如图 3.12 所示。程序的流程图如图 3.13 所示。

程序如下：

```
LEA SI , STR1      ; (SI) ←被加数地址指针
LEA BX , STR2      ; (BX) ←加数地址指针
LEA DI , SUM       ; (DI) ←结果地址指针
MOV CX , 4         ; (CX) ←循环次数
CLC               ; (CF)=0
N: MOV AL, [SI]    ; 取一个字节被加数
  ADC AL , [BX]    ; 与加数相加
  AAA             ; 调整
```

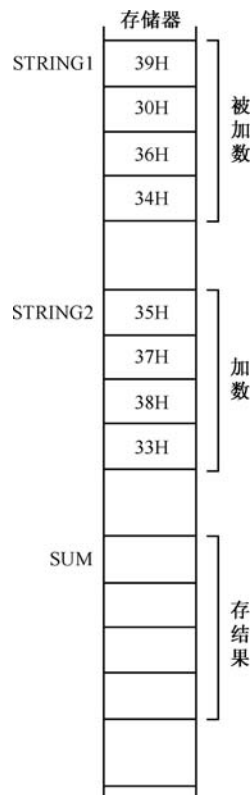


图 3.12 数据存放情况

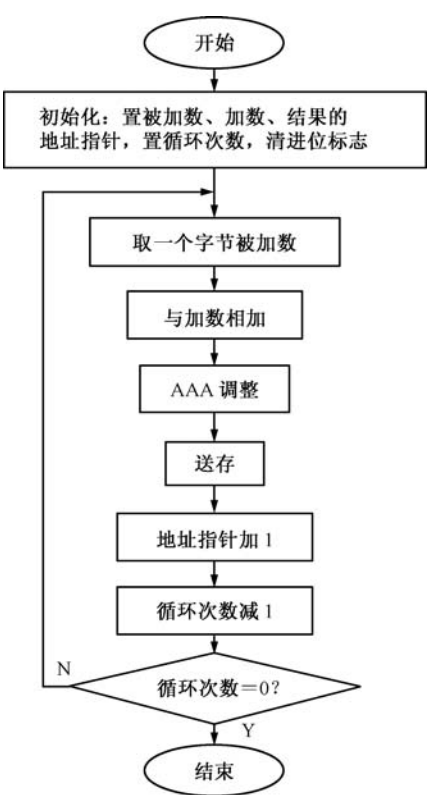


图 3.13 例 3.4 程序流程图

```
INC SI             ; SI 加 1
INC DI             ; DI 加 1
DEC CX             ; 循环次数减 1
JNZ N              ; 如不为零，转 N
HLT                ; 停止
```

(2) 压缩型 BCD 码调整指令。

指令格式：

```
DAA
```

DAA 指令同样不带操作数，实际上隐含寄存器操作数 AL。指令的操作为：

```
如果 (AL) ∧ 0FH > 9, 或 (AF) = 1
则 AL ← (AL) + 06H
```

```

    AF ← 1
    如果 (AL) ∧ 0F0H > 9FH, 或 (CF) = 1
    则   AL ← (AL) + 60H
        CF ← 1

```

DAA 指令影响 AF、CF、SF、ZF、PF，但不影响 OF。DAA 指令只对加法的结果 AL 的内容进行调整，任何时候不影响 AH。例：

```

MOV  AL, 68H      ; (AL) = 68H
MOV  BL, 59H      ; (BL) = 59H
ADD  AL, BL       ; (AL) = C1H, (AF) = 1
DAA                      ; (AL) = 27 H, (CF) = 1

```

如果要求相加两个位数或更多位数的十进制数，则也应编写一个循环程序，并采用 ADC 指令，在循环之前要清进位标志 CF(参阅例 3.4)。但采用压缩型 BCD 码时，每次可以相加两位十进制数。例如，相加两个 8 位十进制数时，只需循环 4 次。

为了掌握 DAA 指令与 AAA 指令的区别，现在再来做前面已经作过的简单计算，即

7+8=?

不过这一次编程时不用 AAA 指令，而改用 DAA 指令调整，看看结果有什么不同。

```

MOV  AX, 0007H    ; (AL) = 07H, (AH) = 00H
MOV  BL, 08H      ; (BL) = 08H
ADD  AL, BL       ; (AL) = 0FH
DAA                      ; (AL) = 15H, (AH) = 00H, (AF) = 1, (CF) = 0

```

可见，现在 7 加 8 所得之和以压缩型 BCD 码的形式存放在 AL 寄存器中，而 AH 的内容不变。

2) 十进制减法的调整指令

同加法一样，对 BCD 码减法进行十进制调整的指令也有两条：AAS 和 DAS。

指令格式：

AAS

AAS 也称为减法的 ASCII 码的调整指令。隐含寄存器操作数为 AL 和 AH。对非压缩型 BCD 码调整。指令的操作为：

```

    如果 (AL) ∧ 0FH > 9, 或 (AF) = 1
    则   AL ← (AL) - 06H
        AH ← (AH) - 1
        AF ← 1
        CF ← (AF)
    否则 AL ← ((AL) ∧ 0FH)

```

AAS 指令影响 AF 和 CF，OF、SF、ZF、PF 不确定。

【例 3.5】 计算 13-4=? 先将被减数和减数以压缩的 BCD 码的形式分别存放在 AH(被减数的十位)、AL(被减数的个位)和 BL(减数)中，然后用 SUB 指令进行减法，再用 AAS 指令进行调整。可用以下指令实现：

```

MOV  AX, 0103H    ; (AH) = 01H, (AL) = 03H
MOV  BL, 04H      ; (BL) = 04H
SUB  AL, BL       ; (AL) = 03H - 04H = FFH
AAS                      ; (AL) = 09H, (AH) = 0

```


以上指令的执行结果为 $13-4=9$ ，此结果仍以非压缩型 BCD 码的形式存放，个位在 AL 寄存器，十位在 AH 寄存器。

3) 压缩型 BCD 码调整指令

指令格式：

DAS

指令对减法进行十进制调整，指令隐含寄存器操作数 AL。在减法运算时，DAS 指令对压缩型 BCD 码进行调整，其操作为

```
如果 ((AL) ^ 0FH) > 9 或 (AF) = 1
则  AL ← (AL) - 06H
    AF ← 1
如果 (AL) > 9FH 或 (CF) = 1
则  AL ← (AL) - 60H
    CF ← 1
```

与 DAA 类似，DAS 指令影响 AF、CF、SF、ZF、PF，但不影响 OF。DAS 指令只对减法的结果 AL 的内容进行调整，任何时候都不影响 AH。

例如： $83-38=?$ 采用压缩型 BCD 码存放原始数据，则以上减法运算可用下列几条指令实现：

```
MOV  AL, 83H      ; (AL) = 83H
MOV  BL, 38H      ; (BL) = 38H
SUB  AL, BL       ; (AL) = 4BH
DAS                      ; (AL) = 45H
```

4) 十进制乘除法的调整指令

对于十进制数的乘除法运算，8086/8088 指令系统只提供了非压缩型 BCD 码的调整指令，而没有提供压缩型 BCD 码的调整指令。因此，8086/8088 CPU 不能直接进行压缩型 BCD 码的乘除法运算。

非压缩型 BCD 码的乘除法与加减法相同，加减法可以直接用 ASCII 码参加运算，而不管其高位上有无数字，只要在加减指令后用一条非压缩型 BCD 码的调整指令，就能得到正确结果。而乘除法要求参加运算的两个数高 4 位是 0 的非压缩型 BCD 码，低 4 位是十进制数。也就是说，如果用 ASCII 码进行非压缩型 BCD 码的乘法运算，在乘除法运算之前，必须将高 4 位清零。

(1) 非压缩型 BCD 码的乘法调整指令。

指令格式：

AAM

AAM 指令也是一个隐含了寄存器操作数 AL 和 AH 的指令。

在乘法运算时，调整之前，先用 MUL 指令将两个真正的非压缩型的 BCD 码相乘，结果放在 AX 中。然后用 AAM 指令对 AL 寄存器进行调整，于是在 AX 中就可得到正确的非压缩型 BCD 码的结果，其乘积的高位在 AH 中，乘积的低位在 AL 中。AAM 指令的操作为：

```
AH ← (AL) / 0AH 的商；即 AL 除以 10，商送 AH
AL ← (AL) / 0AH 的余数；即 AL 除以 10，余数送 AL
```

AAM 指令的操作实质上是 将 AL 寄存器中的二进制数转换为非压缩型的 BCD 码，十位存放在 AH 寄存器，个位存放在 AL 寄存器。AAM 指令执行以后，将根据 AL 中的结果影响状态标志位 SF、ZF 和 PF，但 AF、CF 和 OF 的值不定。

例如要求进行以下十进制乘法运算： $7 \times 9 = ?$ 可编程序段如下：

```
MOV AL, 07H      ; (AL) = 07H
MOV BL, 09H      ; (BL) = 09H
MUL BL           ; (AX) = 07H × 09H = 003FH
AAM              ; (AH) = 06H, (AL) = 03H, (SF) = 0, (ZF) = 0, (PF) = 1
```

已知 $7 \times 9 = 63$ 。以上指令执行以后，十进制乘积也是以非压缩型 BCD 码的形式存放在 AX 中。由于 (AL) = 03H (000000011B)，故决定了 (SF) = 0, (ZF) = 0, (PF) = 1。

(2) 非压缩型 BCD 码的除法调整指令。

指令格式：

AAD

AAD 指令也是一个隐含了寄存器操作数 AL 和 AH 的指令，它是对非压缩型 BCD 码进行调整，其操作为：

```
AL ← (AH) × 0AH + (AL)
AH ← 0
```

即将 AH 寄存器的内容乘以 10 并加上 AL 寄存器的内容，结果送回 AL，同时将零送 AH。以上操作实质上是 将 AX 寄存器中非压缩型 BCD 码转换为真正的二进制数，并存放在 AL 寄存器中。

执行 AAD 指令以后，将根据 AL 中的结果影响状态标志位 SF、ZF 和 PF，但其余几个状态标志位，如 AF、CF 和 OF 的值则不确定。

AAD 指令的用法与其他非压缩型 BCD 码调整指令 (如 AAA、AAS、AAM) 有所不同。AAD 指令使用在除法指令之前进行调整，方可得到正确的非压缩型 BCD 码的结果。

例如：想要进行以下十进制除法运算， $73 \div 2 = ?$ 可先将被除数和除数以非压缩型 BCD 码的形式分别存放在 AX 和 BL 寄存器中，被除数的十位在 AH，个位在 AL，除数在 BL。先用 AAD 指令对 AX 中的被除数进行调整，然后进行除法运算，并对商进行再调整。可编成如下程序段：

```
MOV AX, 0703H    ; (AH) = 07H, (AL) = 03H
MOV BL, 02H      ; (BL) = 02H
AAD              ; (AL) = 49H (十进制数 73)
DIV BL           ; (AL) = 24H (商), (AH) = 01H (余数)
AAM              ; (AH) = 03H, (AL) = 06H
```

已知 $73 \div 2 = 36 \cdots 1$ 。以上几条指令执行的结果为，在 AX 中得到非压缩型 BCD 码的商，但余数已被丢失。如果需要保留余数，则应在 DIV 指令之后，用 AAM 指令调整之前，将余数暂存到寄存器中。如果有必要，还应设法对余数也进行调整。

3.2.3 逻辑运算指令

8086/8088 逻辑运算指令有 AND 逻辑“与”、TEST 测试、OR 逻辑“或”、XOR 逻辑“异或”、NOT 逻辑“非”运算。各种逻辑运算的结果如表 3-1 所示。

表 3-1 逻辑运算返回的值

X	Y	X AND Y	X OR Y	X XOR Y	NOT X
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

以上指令只有 NOT 逻辑“非”指令对状态标志寄存器没有影响。其他指令根据各自的逻辑运算结果影响 SF、ZF、PF，将 CF、OF 清 0，AF 的值不确定。

1. 逻辑“与”指令

指令格式及操作：

```
AND  dst, src          ; dst ← (dst) ∧ (src)
```

AND 将源操作数与目标操作数按位进行“与”运算，结果送回目标操作数。两个操作数不能同时为存储器。例如：

```
AND  AL, 00001111H      ; 寄存器“与”立即数
AND  CX, DI              ; 寄存器“与”寄存器
AND  SI, MEM             ; 寄存器“与”存储器
AND  ALP [DI], AX        ; 存储器“与”寄存器
AND  [BX] [SI], 0FFFEH   ; 存储器“与”立即数
```

AND 指令主要用来屏蔽掉一个数中某些位，以便对剩下的其他位进行某些处理。因此两个操作数都是 1 的位，目的操作数相对应位就是 1，其他各种组合的位，目的操作数相应位都是 0。

例如：AND AL, 00001111H；将 AL 寄存器高 4 位屏蔽，低 4 位不变。

2. TEST 测试

TEST 指令的操作实质上与 AND 指令相同，即把目标操作数和源操作数进行逻辑“与”。二者的区别在于 TEST 指令不把逻辑运算的结果送回目标操作数，因此两个操作数的内容均保持不变，即目标操作数将不被破坏。逻辑“与”的结果反映在状态标志位上，例：“与”的最高位是 0 还是 1，结果是否全为 0，结果中 1 的个数是奇数还是偶数分别由 SF、ZF、PF 体现。将 CF、OF 清 0，AF 的值不确定。

例如：

```
TEST BH, 7              ; 寄存器“与”立即数(结果不回送，下同)
TEST SI, BP              ; 寄存器“与”寄存器
TEST [SI], CH            ; 存储器“与”寄存器
TEST [BX] [DI], 6ACEH; 存储器“与”立即数
```

TEST 指令常用于位测试，它与条件转移指令一起，共同完成对特定状态位的判断，并实现相应的程序转移。这种作用与比较指令 CMP 有些类似，不过 TEST 指令只比较某一指定的位，而 CMP 指令比较整个操作数(字节或字)。

3. 逻辑“或”指令

指令格式及操作：

OR dst, src; $\text{dst} \leftarrow \text{dst} \vee (\text{src})$

OR 指令将目的操作数和源操作数按位进行逻辑“或”运算，并将结果送回目标操作数。OR 指令操作数的类型与 AND 指令相同，即目标操作数可以是寄存器或存储器，源操作数可以是立即数、寄存器或存储器，但不能同时都是存储器。例如：

OR BL, 0F6H	; 寄存器“或”立即数
OR AH, BL	; 寄存器“或”寄存器
OR CL, BETA[BX][DI]	; 寄存器“或”存储器
OR [BX][DI], 80H	; 存储器“或”立即数

OR 指令的用途是将寄存器或存储器中的某些位置 1，而不管这些位原来的状态如何，并保持其他位状态不变。“或”指令将要求保持不变的位和“0”进行逻辑“或”，“或”指令将要求置 1 的位和“1”进行逻辑“或”，该指令影响 SF、ZF、PF。

AND 指令和 OR 指令有一个共同的特性：如果将一个寄存器的内容和该寄存器本身进行逻辑“与”操作或者逻辑“或”操作，则保持寄存器的内容不变，但影响 SF、ZF、PF。

4. XOR 逻辑“异或”指令

指令格式及操作：

XOR dst, src ; $\text{dst} \leftarrow \text{dst} \oplus (\text{src})$

XOR 指令将目标操作数和源操作数按位进行逻辑“异或”运算，并将结果送回目标操作数。

XOR 指令的操作类型与 AND 指令和 OR 指令相同，此处不再赘述，例如：

XOR DI, 23F6H	; 寄存器“异或”立即数
XOR SI, DX	; 寄存器“异或”寄存器
XOR CL, BUFFER	; 寄存器“异或”存储器
XOR MEM[BX], AX	; 存储器“异或”寄存器
XOR TABLE[BP][SI], 3DH	; 存储器“异或”立即数

XOR 指令的用途是将寄存器或存储器中某些特定的位“求反”，而使其余位保持不变。为此，可将欲“求反”的位和“1”进行“异或”，而将要求保持不变的位和“0”进行“异或”。例如，若要使 AL 寄存器中第 1、3、5、7 位取反其他位不变，异或 10101010B(即 0AAH)即可。

```
MOV AL, 0FH (AL) = 00001111B
XOR AL, 0AAH; (AL) = 10100101B (0A5H)
```

XOR 指令的另一个用途将寄存器内容清零，例如：

XOR AX, AX	; AX 清零
XOR CX, CX	; CX 清零

而且，上述指令和 AND、OR 等指令一样，也将进位标志 CF 清零。

XOR 指令的这种特性在多字节的累加程序中十分有用，它可以在循环程序开始前的初始化过程中使用。

5. 逻辑“非”运算

指令格式及操作

NOT dst ; dst ← FFFFH-(dst) (字求反)

NOT 指令的操作数可以是 8 位或 16 位寄存器或存储器，但不能是立即数。

```
NOT AH           ; 8 位寄存器求反
NOT CX           ; 16 位寄存器求反
NOT BYTE PTR [BP] ; 8 位存储器求反
NOT WORD PTR COUNT ; 16 位存储器求反
```

3.2.4 移位指令

8086/8088 指令系统的移位指令包括逻辑左移 SHL、算术左移 SAL、逻辑右移 SHR、算术右移 SAR，还有循环移位指令，包括不带进位循环左移 ROL、循环右移 ROR 和带进位循环左移 RCL、循环右移 RCR。移位常数一定放在 CL 中。移位指令都影响状态标志位，但影响的方式各条指令不尽相同。

1. 移位指令

1) 逻辑左移 SHL/算术左移 SAL

指令格式：

```
SHL dst, 1
SAL dst, 1
或 SHL dst, CL
SAL dst, CL
```

这两条指令的操作是将目标操作数顺序向左移 1 位或移 CL 位，左移 1 位时高位移入进位标志 CF，最低位补 0。指令操作示意图如图 3.14 所示，逻辑左移 SHL/算术左移 SAL 影响 CF 和 OF，如果移位次数是 1，且移位后 dst 最高位与 CF 不相等，则溢出标志位 OF=1，否则 OF=0。如果移位次数不是 1，则 OF 值不确定。OF 值表示移位是否改变符号位。

```
SHL AH, 1           ; 寄存器左移 1 位
SAL SI, CL          ; 寄存器左移 (CL) 位
SAL WORD PTR [BX+5], 1 ; 存储器左移 1 位
SHL DATA, CL       ; 存储器左移寄存器 (CL) 中指定的位数
```



图 3.14 SHL / SAL 指令操作示意图

【例 3.6】 将一个 16 位无符号数乘以 10。该数原来存放在以 FACTOR 为首地址的两个连续的存储单元中(低位在前，高位在后)。

因为 $\text{FACTOR} \times 10 = (\text{FACTOR} \times 8) + (\text{FACTOR} \times 2)$ ，故可用左移指令实现以上乘法运算。编程如下：

```

MOV  AX, FACTOR    ; AX←被乘数
SHL  AX, 1          ; (AX)=FACTOR×2
MOV  BX, AX         ; 暂存 BX
SHL  AX, 1          ; (AX)=FACTOR×4
SHL  AX, 1          ; (AX)=FACTOR×8
ADD  AX, BX         ; (AX)=FACTOR×10
HLT

```

以上程序的执行时间大约需 26 个周期。如用乘法指令编程，执行时间将超过 130 个时钟。

2) 逻辑右移指令

指令格式：

```
SHR  dst, 1/CL
```

SHR 指令的操作是将目标操作数顺序向右移 1 位或右移由 CL 寄存器指定的位数。逻辑右移 1 位时，低位移入进位标志 CF，最高位补 0。指令操作如图 3.15 所示。



图 3.15 SHR 指令操作示意图

SHR 指令也将影响 CF 和 OF 状态标志位。如果移位次数等于 1，且移位以后新的最高位与次高不相等，则溢出标志位 OF=1，否则 OF=0。OF 值表示移位是否改变符号位。

例如：

```

SHR  BL, 1          ; 寄存器逻辑右移 1 位
SHR  AX, CL          ; 寄存器逻辑右移 (CL) 位
SHR  BYTE PTR [DI+BP], 1 ; 存储器逻辑右移 1 位
SHR  WORD PTR BLOCK, CL ; 存储器逻辑右移 (CL) 位

```

【例 3.7】 将一个 16 位无符号数除以 512。该数原来存放在以 DDA 为首地址的两个连续的存储单元中。

因为 $DDA \div 512 = (DDA \div 2) \div 256$ ，因此可用逻辑右移指令完成上述除法运算。

编程如下：

```

MOV  AX, DDA        ; AX←被除数
SHR  AX, 1          ; (AX)=DDA÷2
XCHG AL, AH         ; (AL)↔(AH)，相当循环右移 8 位
CBW                  ; 清 AX 的高 8 位，(AX)=DDA
HLT

```

当然，也可以将立即数 9 传送到 CL 寄存器，然后用指令(SHR AX, CL)完成除以 512 的运算。但是相比之下，上面的程序执行速度更快。

3) 算术右移指令

指令格式：

```
SAR  dst, 1/CL
```

SAR 指令的操作数与逻辑右移指令 SHR 有点类似，将目标操作数向右移 1 位或由 CL 寄存器指定的位数。逻辑右移 1 位时，低位移入进位标志 CF，最高位保持不变。

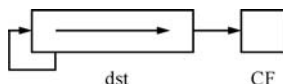


图 3.16 SAR 指令操作示意图

算术右移指令对状态标志位 CF、OF、PF、SF 和 ZF 有影响，但 AF 的值不确定。

例如：

```
SAR  AX, 1      ; 寄存器算术右移 1 位
SAR  DX, CL     ; 寄存器算术右移 (CL) 位
SAR  [SI], 1    ; 存储器算术右移 1 位
SAR  [BX], CL   ; 存储器算术右移 (CL) 位
```

2. 循环移位指令

8086/8088 指令系统有四条循环移位指令，即不带进位标志 CF 的左循环移位指令 ROL 和右循环移位指令 ROR(也称小循环)，以及带进位标志 CF 的左循环移位指令 RCL 和右循环移位指令 RCR(也称大循环)。

循环移位指令的操作数类型与移位指令相同，可以是 8 位或 16 位的寄存器或存储器。指令中指定的左移或右移的位数也可以是 1 或由 CL 寄存器指定。但不能是 1 以外的常数或 CL 以外的寄存器。循环移位指令都只影响进位标志 CF 和溢出标志 OF，但 OF 标志的含义对于左循环移位指令和右循环移位指令有所不同。

1) 循环左移指令

指令格式：

```
ROL  dst, 1/CL
```

ROL 指令将目标操作数向左循环移动 1 位或 CL 寄存器指定的位数。最高位移到进位标志 CF，同时，最高位移到最低位形成循环，进位标志 CF 不在循环回路之内。其操作如图 3.17 所示。

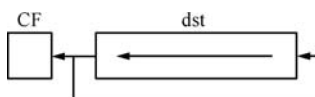


图 3.17 ROL 指令操作示意图

ROL 指令将影响 CF 和 OF 两个状态标志位。如果循环移位次数等于 1，且移位以后目的操作数新的最高位与 CF 不相等，则(OF)=1，否则(OF)=0。因此，OF 的值表示循环移位前后符号位是否有所变化。如果移位次数不等于 1，则 OF 的值不确定。

例如：

```
ROL  BH, 1      ; 寄存器循环左移 1 位
ROL  DX, CL     ; 寄存器循环左移 (CL) 位
ROL  [DI], 1    ; 存储器循环左移 1 位
ROL  HA, CL     ; 存储器循环左移 (CL) 位
```

2) 循环右移指令

指令格式：

```
ROR  dst, 1/CL
```

ROR 指令将目标操作数向右循环移动 1 位或右移由 CL 寄存器指定的位数。最低位移到进位标志 CF，同时最低位移到最高位，指令的操作可用图 3.18 表示。ROR 指令也将影响状态标志位 CF 和 OF。如果移位次数等于 1，且移位以后新的最高位与次高位不相等，则溢出标志位 OF=1；否则 OF=0。OF 值表示移位是否改变符号位。

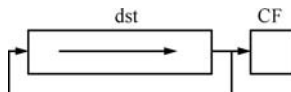


图 3.18 ROR 指令操作示意图

3) 带进位循环左移指令

RCL dst, 1/CL

RCL 指令将目标操作数连同进位标志 CF 一起向左循环移动 1 位或由 CL 寄存器指定的位数。最高位移入 CF，而 CF 移入最低位。RCL 指令的操作如图 3.19 所示。

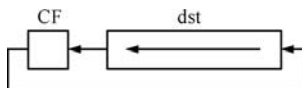


图 3.19 RCL 指令操作示意图

RCL 指令对状态标志位的影响与 ROL 指令相同。

4) 带进位循环右移指令

指令格式：

RCR dst, 1/CL

RCR 指令将目标操作数连同进位标志 CF 一起向右循环移动 1 位或由 CL 寄存器指定的位数。最低位移到进位标志 CF，同时进位标志 CF 移到最高位。RCR 指令操作如图 3.20 所示。RCR 指令对状态标志位的影响与 ROR 指令相同。

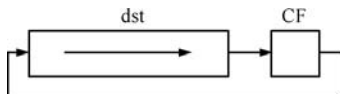


图 3.20 RCR 指令操作示意图

这里介绍的四条循环移位指令与前面讨论过的移位指令有所不同。这四条指令进行循环移位操作之后，操作数中原来各位的信息不会丢失，只是移到了操作数中的其他位或进位标志上，必要时还可以恢复。利用循环移位指令可以对寄存器或存储器中的任一位进行位测试。例如要求测试 AL 寄存器中第 5 位的状态是“0”还是“1”，则可利用以下指令实现：

```
MOV CL, 4
ROL AL, CL
JNC DONE
...
...
DONE: ...
...
```


3.2.5 控制转移指令

8086/8088 指令系统提供了大量指令,用于控制程序的流程。这类指令包括转移指令、循环控制指令、过程调用指令和中断指令四类。

1. 转移指令

转移是一种将程序控制从一处改换到另一处的最直接的方法。在 CPU 内部,转移是通过将目的地址传送给 IP 来实现的。

转移指令包括无条件转移指令和条件转移指令。

1) 无条件转移指令

无条件转移指令的操作是无条件地将程序转移到指令中指定的目标地址。目标地址可以用直接的方式给出,也可以用间接的方式给出。无条件转移指令对状态标志位没有影响。

(1) 段内直接转移。

指令格式及操作:

`JMP nearlabel ; IP ← (IP) + disp (16 位)`

指令的操作数是一个近标号,该标号在本段(或本组)内。指令汇编以后,计算出 JMP 指的下一条指令的地址到目的地址之间的 16 位相对位移量 disp。指令的操作是将指令指针寄存器 IP 的内容加上相对位移量 disp,代码段寄存器 CS 的内容不变,从而使控制转移到目的地址。相对位移量可正可负,一般情况下,它的范围在-32768~+32767 之间,故需用 2 个字节表示,加上一个字节的操作码,这种段内直接转移指令共有 3 个字节。请看下面几条指令:

```
JMP NEXT
AND AL, 7FH
NEXT: XOR AL, 7FH
```

其中, NEXT 是本段内的一个标号,汇编语言计算出下一条指令(即 AND AL, 7FH)的地址与标号 NEXT 代表的地址之间的相对位移量,执行 JMP NEXT 指令时,将上述位移量加到 IP 上,于是执行 JMP 指令之后,接着就执行 XOR AL, 7FH 指令,实现了程序的转移。

(2) 段内直接短转移。

指令格式及操作:

`JMP Shortlabel ; IP ← (IP) + disp (8 位)`

段内直接短转移指令的操作数是一个短标号。此时,相对位移量 disp 的范围在-128~+127 之间,只需用 1 个字节表示。段内直接短转移指令共有 2 个字节。

如果已知下一条指令到目的地址之间的相对位移量在-128~+127 的范围内,则可在标号前写上运算符 SHORT,实现段内直接短转移。但是,对于一个段内直接转移指令,如果相对位移量的范围在-128~+127 之间,而且目标地址的标号已经定义(即标号先定义后引用,这种情况称为向后引用的符号),那么即使标号没写上运算符 SHORT,汇编程序也能自动生成一个 2 字节的短转移指令。这种情况属于隐含的短转移。如果向前引用标号(即标号先引用,后定义),则标号前应写上运算符 SHORT,否则,即使位移量的范围不超过-128~

+127, 汇编后仍会生成一个 3 字节的近转移指令。例如:

```

    向后引用的标号可以不写运算符 SHORT
target : ...           ; 先定义标号 target
:                   ; 相对位移量不超过-128~+127
JMP target           ; 后引用标号 target
    向前引用的标号应当写明运算符 SHORT
JMP SHORT target     ; 先引用标号 target
:                   ; 相对位移量不超过-128~+127
target : ...         ; 此处定义标号 target

```

(3) 段内间接转移。

指令格式及操作:

```
JMP reg16/mem16; IP←(reg16)/IP←(mem16)
```

指令的操作是一个 16 位的寄存器(reg16)或存储器(mem16)地址。存储器可用各种寻址方式。指令的操作是用指定的寄存器或存储器中的内容作为目标的偏移地址取代原来 IP 的内容, 以实现程序的转移, 由于是段内转移, 故 CS 寄存器的内容不变。下面是几条段内间接转移指令:

```

JMP AX
JMP SI
JMP LABEL[BX]
JMP AWORD
JMP WORD PTR [BP][DI]

```

上面前两条指令的操作数是 16 位寄存器, 第 3、4 条指令的操作数应是已被定义成 16 位的存储器。第 5 条指令利用运算符“PTR”将存储器操作数定义为 WORD(字, 即 16 位)。关于“PTR”的说明, 请参阅本书第 4 章表达式部分关于分析运算符和合成运算符的介绍。

(4) 段间直接转移。

指令格式及操作:

```

JMP farlabel      ; IP←OFFSET farlabel
                  ; CS←SEG farlabel

```

指令的操作数是一个远标号, 该标号在另一个代码段内。指令的操作是将标号的偏移地址取代指令指针寄存器 IP 的内容, 同时将标号的段基值取代段寄存器 CS 的内容, 结果使控制转移到另一代码段内指定的标号处。例如:

```

JMP LABELDECLARED FAR
JMP FAR PTR LABELNAME

```

上面第一条指令中的 LABELDECLARED FAR 应是一个在另一代码段内已定义的远标号。第二条指令利用运算符 PTR 将标号 LABELNAME 的属性指定为 FAR。

(5) 段间间接转移。

指令格式及操作:

```

JMP mem32; IP←(mem32)
          ; CS←(mem32+2)

```

指令的操作数是 32 位的存储器地址, 指令的操作是将存储器的前两个字节送到 IP 寄存器, 存储器的后两个字节送到 CS 寄存器, 以实现到另一个代码段的转移。

注意：段间间接转移的操作数不能是寄存器。

```
JMP VARDOUBLEWORD
JMP DWORD PTR [BP][DI]
```

上面第一条指令中，VARDOUBLEWORD 应是一个已经定义成 32 位的存储器变量(例如可用数据定义伪指令 DD 定义)。第二条指令中，利用运算符 PTR 将存储器操作数的类型定义成 DWORD(双字，即 32 位)

2) 条件转移指令

指令格式为：

```
JCC shortLabel
```

在汇编语言程序设计中，常利用条件转移指令来构成分支程序。指令助记符中的“CC”表示条件。这种指令的执行包括两个过程：第一步，测试规定的条件；第二步，如果条件满足则转移到目的地址，否则，继续顺序执行。

条件转移指令也只有一个操作数，用以指明转移的目的地址。但是它与无条件转移指令 JMP 不同，条件转移指令的操作数必须是一个短标号，也就是说，所有的条件转移指令都是 2 字节指令，转移指令的下一条指令到目标地址之间的距离必须在-128~+127 的范围内。如果指令规定的条件满足，则将这个位移量加到 IP 寄存器上，即 $IP \leftarrow (IP) + disp$ ，实现程序的转移。

绝大多数条件转移指令(除 JCXZ 指令外)将状态标志位的状态作为测试的条件。因此，首先应执行影响有关的状态标志位的指令，然后才能用条件转移指令测试这些标志，以确定程序是否转移。CMP 和 TEST 指令常与条件转移指令配合使用，因为这两条指令不改变目标操作数的内容，但可影响状态标志寄存器。

8086/8088 的条件转移指令非常丰富，不仅可以测试一个状态标志位的状态，而且可以综合测试几个状态标志位；不仅可以测试无符号数的高低，而且可以测试带符号数的大小等，在编程时十分灵活、方便。下面将所有的条件转移指令的名称、助记符及转移条件列于表 3-2 中同一行内用斜杠隔开的几个助记符，实质上代表同一条指令的几种不同的表示方法。

表 3-2 条件转移指令

指令名称	助记符	转移条件	备 注
等于/零转移	JE/JZ	(ZF)=1	
不等于/非零转移	JNE/JNZ	(ZF)=1	
负转移	JS	(SF)=1	
正转移	JNS	(SF)=0	
偶转移	JP/JPE	(PF)=1	
奇转移	JNP/JPO	(PF)=0	
溢出转移	JO	(OF)=1	
不溢出转移	JNO	(OF)=0	

续表

指令名称	助记符	转移条件	备 注
进位转移	JC	(CF)=1	
不进位转移	JNC	(CF)=0	
低于/不高于或不等于转移	JB/JNAE	(CF)=1	无符号数
高于或等于/不低于转移	JAE/JNB	(CF)=0	无符号数
高于/不低于或不等于转移	JA/JNBE	(CF)=0 且 (ZF)=0	无符号数
低于或等于/不高于转移	JBE/JNA	(CF)=1 或 (ZF)=1	无符号数
大于/不小于或不等于转移	JG/JNLE	(SF)=(OF) 且 (ZF)=0	带符号数
大于或等于/不小于转移	JGE/JNL	(SF)=(OF)	带符号数
小于/不大于或不等于转移	JL/JNGE	(SF)≠(OF) 且 (ZF)=0	带符号数
小于或等于/不大于转移	JLE/JNG	(SF)≠(OF) 或 (ZF)=1	带符号数
CX 等于零转移	JCXZ	(CX)=0	

下面再来看几个应用条件转移指令的程序例子。

【例 3.7】 在内存的数据段中存放了若干个 8 位带符号数，数据块的长度为 COUNT(不超过 255)，首地址为 TABLE，试统计其中正元素、负元素及零元素的个数，并将个数分别存入 PLUS、MINUS 和 ZERO 单元。

为了统计正元素、负元素和零元素的个数，可先将 PLUS、MINUS 和 ZERO 三个单元清零，然后将数据表中的带符号数逐个取入 AL 寄存器并使其影响状态标志位，再利用前面介绍的 JS、JZ 等条件转移指令测试该数是一个负数、零还是正数，然后分别在相应的单元中进行计数。

编程如下：

```

XOR  AL, AL          ; AL←0
MOV  PLUS, AL        ; 清 PLUS 单元
MOV  MINUS, AL       ; 清 MINUS 单元
MOV  ZERO, AL        ; 清 ZERO 单元
LEA  SI, TABLE      ; SI←数据表首址
MOV  CX, COUNT       ; CX←数据表长度
CLD                  ; 清状态标志位 DF
CHECK: LODSB         ; 取一个数据到 AL
OR   AL, AL          ; 使数据影响状态标志位
JS   X1              ; 如为负，转 X1
JZ   X2              ; 如为零，转 X2
INC  PLUS            ; 否则为正，PLUS 单元加 1
JMP  NEXT
X1:  INC  MINUS       ; MINUS 单元加 1
JMP  NEXT
X2:  INC  ZERO        ; ZERO 单元加 1
NEXT: LOOP  CHECK    ; CX 减 1，如不为零，则转 CHECK
HLT                      ; 停止

```

以上程序中的 LOOP CHECK 指令是一条循环控制指令，它的操作是将 CX 寄存器的内容减 1，如结果不等于零，则转移到短标号 CHECK。后面将讨论这条指令。

2. 循环控制指令

8086/8088 指令系统专门设计了几条循环控制指令,用于使一些程序段反复执行,形成循环程序。循环控制指令有以下几条:

1) LOOP

指令格式:

```
LOOP shortlabel
```

LOOP 指令规定将 CX 寄存器作为计数器,指令的操作是先将 CX 的内容减 1,如结果不等于零,则转到指令中指定的短标号处;否则,顺序执行下一条指令。因此,在循环程序开始前,应将循环次数送 CX 寄存器。指令的操作只能是一个短标号,即跳转距离不超过 -128~+127 的范围。LOOP 指令对状态标志位没有影响。LOOP 指令的应用可参阅前面的例子。

2) LOOPE/LOOPZ

指令格式:

```
LOOPE shortlabel  
LOOPZ short-label
```

以上两种格式实际上代表同一条指令。本指令的操作是先将 CX 的内容减 1,如结果不等于零,且零标志(ZF)=1 则转移到指定的短标号。LOOPE/LOOPZ 指令对状态标志位也没有影响。

该指令是有条件地形成循环,即当规定的循环次数尚未完成时,还必须满足“相等”或者“等于零”的条件,才能继续循环。

3) LOOPNE/LOOPNZ

指令格式:

```
LOOPNE/LOOPNZ shortlabel
```

以上两种格式实际上代表同一条指令。本指令的操作是先将 CX 的内容减 1,如结果不等于零,且零标志(ZF)=0 则转移到指定的短标号。LOOPNE/LOOPNZ 指令对状态标志位也没有影响。

3. 过程调用指令

如果有一些程序段需要在不同的地方多次反复地出现,则可以将这些程序段设计成为过程(相当于子程序),每次需要时进行调用。过程结束后,再返回原来调用的地方。采用这种方法不仅可以使源程序的总长度大大缩短,而且有利于实现模块化的程序设计,使程序的编制、阅读和修改都比较方便。

被调用的过程可以在本段内(近过程);也可在其他段(远过程)。调用的过程地址可以用直接的方式给出,也可用间接的方式给出。过程调用指令和返回指令对状态标志位都没有影响。

1) 段内直接调用

指令格式及操作:

```
CALL nearproc ; SP ← (SP) - 2, (SP) + 1: (SP) ← (IP)  
; IP = (IP) + disp
```

指令的操作数是一个近过程，该过程在本段内。指令汇编以后，得到 CALL 的下一条指令与被调用的过程入口地址的 16 位相对位移量 disp 。指令操作是将指令指针 IP 的内容压入堆栈，然后将相对位移量 disp 加到 IP 上，使控制转到调用的过程。16 位相对位移量 disp 占 2 个字节，段内直接调用指令共有 3 个字节。

2) 段内间接调用

指令格式及操作：

```
CALL reg16/mem16    ; SP ← (SP) - 2, (SP) + 1: (SP) ← (IP)
                    ; IP ← reg16/mem16
```

指令的操作数是 16 位的寄存器或存储器，其内容是一个近过程入口地址，指令操作是将指令指针 IP 的内容压入堆栈，然后将寄存器或存储器的内容送到 IP 中。

3) 段间直接调用

指令格式及操作：

```
CALL farproc        ; SP ← (SP) - 2, (SP) + 1: (SP) ← (CS)
                    ; CS ← SEG far-proc
                    ; SP ← (SP) - 2, (SP) + 1: (SP) ← (IP)
                    ; IP ← OFFSET far-proc
```

指令的操作数是一个远过程，该过程在另外的代码段内。段间直接调用指令先将 CS 中的段基值压入堆栈，并将远过程所在的段基值送 CS，再将 IP 中的偏移地址压入堆栈，然后将远过程的偏移地址 OFFSET far-proc 送 IP。

4) 段间间接调用

指令格式及操作：

```
CALL mem32          ; SP ← (SP) - 2, (SP) + 1: (SP) ← (CS)
                    ; CS ← mem32 + 2
                    ; SP ← (SP) - 2, (SP) + 1: (SP) ← (IP)
                    ; IP ← mem32
```

指令的操作数是 32 位的存储器地址，指令的操作是先将 CS 寄存器压入堆栈，并将存储器的后两个字节送 CS，再将 IP 中的偏移地址压入堆栈，然后将存储器的前两个字节送 IP，控制转到另一个代码段的远过程。

4. 过程返回指令

指令格式及操作：

1) 从近过程返回

```
RET; IP ← ((SP) + 1: (SP)), SP ← (SP) + 2
RET popvalue; IP ← ((SP) + 1: (SP)), SP ← (SP) + 2
                ; SP ← (SP) + pop-value
```

2) 从远过程返回

```
RET ; IP ← ((SP) + 1: (SP)), SP ← (SP) + 2
    ; CS ← ((SP) + 1: (SP)), SP ← (SP) + 2
RET popvalue; IP ← ((SP) + 1: (SP)), SP ← (SP) + 2
                ; CS ← ((SP) + 1: (SP)), SP ← (SP) + 2
                ; SP ← (SP) + popvalue
```

过程体中总包含返回指令 RET，它将堆栈中的断点弹出，控制程序返回到原来调用过程的地方。通常，RET 指令的类型是隐含的，它自动与过程定义时的类型相匹配。但采用间接调用时，必须保证 CALL 指令类型与 RET 指令的类型相匹配，以免发生错误。

此外，RET 指令可以带一个弹出值(popvalue)，这是一个 16 位的立即数，通常是偶数。弹出值表示返回时从堆栈舍弃的字节数。例如 RET 4，返回时从堆栈舍弃 4 个字节数。这些字节一般是调用前通过堆栈向过程传递的参数。

3.2.6 字符串操作指令

8086/8088 指令系统中有一组十分有用的串操作指令，这些指令的操作对象不只是单个字节或字，而是内存中地址连续的字节串或字串。在每次基本操作后，能够自动修改地址，为下一次操作做好准备。串操作指令还可以加上重复前缀；此时指令规定的操作将一直重复下去，直到完成预订的重复次数。

串操作指令共有以下五条：串传送指令(MOVS)、串装入指令(LODS)、串送存指令(STOS)、串比较指令(CMPS)和串扫描指令(SCAS)。

上述串操作指令的基本操作各不相同，但都具有以下几个共同特点：

(1) 用 SI 寻址源操作数，用 DI 寻址目标操作数，源操作数在数据段，隐含段寄存器 DS，可以段超越，目标操作数在附加段，隐含段寄存器 ES，不允许段超越。

(2) 每一次操作以后修改地址指针，是增量还是减量决定于方向标志 DF。当(DF)=0 时，地址指针增量，即字节操作时地址指针加 1，字操作时地址指针加 2。当(DF)=1 时，地址指针减量，即字节操作时地址指针减 1，字操作是地址指针减 2。

(3) 有的串操作指令可加重复前缀 REP，则指令规定的操作重复进行，重复操作的次数由 CX 寄存器决定。如果在串操作指令前加上重复前缀 REP，则 CPU 按以下步骤执行：

① 首先检查 CX 寄存器，若(CX)=0，则退出重复串操作指令。

② 指令执行一次字符串基本操作。

③ 根据 DF 标志修改地址指针。

④ CX 减 1(但不改变标志)。

⑤ 转至下一次循环，重复以上步骤。

(4) 串操作指令的基本操作影响 ZF (如 CMPS、SCAS)可加重复前缀 REPE/REPZ 或 REPNE/REPNZ，此时操作重复进行的条件不仅要求(CX)≠0，而且同时要求 ZF 的值满足重复前缀中的规定(REPE 要求(ZF)=1，REPNE 要求(ZF)=0)

(5) 串操作汇编指令的格式可以写上操作数，也可以只在指令助记符后加字母“B”(字节操作)或“W”(字操作)，指令助记符后不加任何操作数。

1. 串传送指令

指令格式：

```
[REP] MOVS [ES:] dststring, [seg:]srcstring  
[REP] MOVSB  
[REP] MOVSW
```

MOVS 指令也称为字符串传送指令，它将一个字节或字从存储器的某个区域传送到另

一个区域，然后根据方向标志 DF 自动修改地址指针。其执行的操作为：

- (1) $(ES): (DI) \leftarrow ((DS): (SI))$
- (2) $SI \leftarrow (SI) \pm 1, DI \leftarrow (DI) \pm 1$ (字节操作)
 $SI \leftarrow (SI) \pm 2, DI \leftarrow (DI) \pm 2$ (字操作)

其中，当方向标志 DF=0 时用“+”，当方向标志 DF=1 时用“-”。串传送指令不影响状态标志寄存器。

以上内容均是任选项，即这些项可有可无。例如重复前缀 REP，可以加在串操作指令之前，也可以不加。

在第一种格式中，串操作指令给出源操作数和目标操作数，此时指令执行字节操作还是字操作，决定于这两个操作数定义时的类型。列出源操作数和目标操作数的作用有二。首先，用以说明操作对象的大小(字节或字)，其次，明确指出涉及的段寄存器(seg)。指令执行时，实际仍用 SI 和 DI 寄存器寻址操作数。如果在指令中采用 SI 和 DI 来表示操作数，则必须用类型运算符 PTR 说明操作对象的类型。第一种格式的一个重要优点是可以对源字符串进行段重设(目的字符串的段基值只能在 ES，不可进行段重设)。

在第二种和第三种格式中，串操作指令字符的后面加上一个字母“B”或“W”，指出操作对象是字节串或字串。但要注意，在这两种情况下，指令后面不需要出现操作数。例如：

```
REP MOVSB DATA2, DATA1      ; 操作数类型应预先定义
MOVSB BUFFER2, ES: BUFFER1    ; 源操作数进行段重设
REP MOVSB WORD PTR [DI], [SI] ; 用变址寄存器表示操作数
REP MOVSB                      ; 字节串传送
MOVSW                          ; 字串传送
```

但以下表示方法是非法的：

```
MOVSB det, dfq
```

串传送指令常常与重复前缀联合使用，这样不仅可以简化程序，而且提高了运行速度。但必须把重复操作的次数(字节串长度)送 CX。

例：传送 200 个字节

```
LEA SI, BUFFER1      ; SI←源串首地址指针
LEA DI, BUFFER2      ; DI←目的串首地址指针
MOV CX, 200           ; CX←字节串长度
CLD                  ; 清方向标志 DF
REP MOVSB             ; 传送 200 个字节
HLT
```

2. 串装入指令

指令格式：

```
LODS [seg: ] srcstring
LODSB
LODSW
```

LODS 指令将字符串中的字节或字逐个装入累加器 AL 或 AX，其执行的操作为：

- (1) $AL \leftarrow ((DS): (SI))$ 或 $AX \leftarrow ((DS): (SI))$
- (2) $SI \leftarrow (SI) \pm 1$ (字节操作)
 $SI \leftarrow (SI) \pm 2$ (字操作)

其中，当方向标志 $DF=0$ 时用“+”，当方向标志 $DF=1$ 时用“-”。串装入指令不影响状态标志寄存器。不加重复前缀。

例：将内存中 10 个非压缩的 BCD 码，按顺序送显示器显示。

```
LEA    SI, BUFFER ; SI←缓冲区首址
MOV    CX, 10      ; CX←字符串长度
CLD                      ; 清状态标志位 DF
MOV    AH, 02H     ; AH←功能号
SC:    LODSB       ; 取一个 BCD 码到 AL
OR     AL, 30H      ; BCD 码转换为 ASCII 码
MOV    DL, AL       ; DL←字符
INT    21H         ; 显示
DEC    CX           ; CX←(CX)-1
JNZ    SC           ; 未完成 10 个字符则重复
HLT
```

3. 串送存指令

指令格式：

```
[REP] STOS    [ES: ]dst-string
[REP] STOSB
[REP] STOSW
```

STOS 指令是将累加器 AL 或 AX 的值送存到内存缓冲区的某个位置上。指令的基本操作为：

- (1) $(ES): (DI) \leftarrow (AL)$ 或 $(ES): (DI) \leftarrow (AX)$
- (2) $DI \leftarrow (DI) \pm 1$ (字节操作)
 $DI \leftarrow (DI) \pm 2$ (字操作)

STOS 指令对状态标志位没有影响。指令若加上重复前缀 REP，则操作将一直重复进行，直到 $(CX)=0$ 。

例：将字符“##”装入以 AREA 为首址的 100 个字节中：

```
LEA    DI, AREA
MOV    AX, '##'
MOV    CX, 100
CLD
REP    STOSW
HLT
```

4. 串比较指令(CMPS)

指令格式：

```
[REPE/REPNE] CMPS [seg: ]src-string, [ES: ]dst-string
[REPE/REPNE] CMPSB
[REPE/REPNE] CMPSW
```

指令的基本操作为：

- (1) $((DS): (SI)) - ((ES): (DI))$
- (2) $SI \leftarrow (SI) \pm 1, DI \leftarrow (DI) \pm 1$ (字节操作)
 $SI \leftarrow (SI) \pm 2, DI \leftarrow (DI) \pm 2$ (字操作)

CMPS 指令与其他指令有所不同, 指令中的源操作数在前, 而目标操作数在后。另外, CMPS 指令可以加重复前缀 REPE(也可以写成 REPZ)或 REPNE(也可以写成 REPNZ), 这是由于 CMPS 指令影响标志 ZF。如果两个被比较的字节或字相等, 则(ZF)=1, 否则(ZF)=0, REPE 或 REPZ 表示当(CX)≠0, 且(ZF)=1 时继续进行比较。REPNE 或 REPNZ 表示当(CX)≠0, 且(ZF)=0 时继续进行比较。

如果想在两个字符串中寻找第一个不相等的字符, 则应使用重复前缀 REPE 或 REPZ, 当遇到第一个不相等的字符时, 就停止进行比较。但此时地址已被修改, 即(DS: SI)和(ES: DI)已经指向下一个字节或字地址。应将 SI 和 DI 进行修正, 使之指向所要寻找的不相等字符。同理, 如果想要寻找两个字符串中第一个相等的字符, 则应使用重复前缀 REPNE 或 REPNZ。但是也有可能将整个字符串比较完毕, 仍未出现规定的条件(例如两个字符相等或不相等), 不过此时寄存器(CX)=0, 故可用条件转移指令 JCXZ 进行处理。

例: 比较两个字符串, 找出其中第一个不相等字符的地址。如果两个字符全部相同, 则转到 MAT 进行处理。这两个字符串长度均为 20, 首地址分别为 ST1 和 ST2。

```
LEA SI, ST1 ; SI←字符串 1 首地址
LEA DI, ST2 ; DI←字符串 2 首地址
MOV CX, 20  ; CX←字符串长度
CLD        ; 清方向标志 DF
REPE CMPSB ; 如相等, 重复进行比较
JCXZ WE    ; 转到 WE
DEC SI
DEC DI
HLT
WE: MOV SI, 0
    MOV DI, 0
    HLT
```

5. 串扫描指令

指令格式:

```
[REPE/REPNE] SCAS [ES: ]dst-string
[REPE/REPNE] SCASB
[REPE/REPNE] SCASW
```

SCAS 指令是在一个字符串中搜索特定的关键字, 字符串的起始地址只能放在(ES: DI)中, 且不可以段超越。待搜索特定的关键字一定要放在 AL 或 AX 中。

SCAS 指令的基本操作为:

- (1) (AL) - ((ES): (DI)) 或 (AX) - ((ES): (DI))
- (2) DI ← (DI) ± 1 (字节操作)
DI ← (DI) ± 2 (字操作)

SCAS 指令将累加器的内容与字符串中的元素逐个进行比较, 比较结果也反映在状态标志位上。SCAS 指令将影响状态标志位 SF、ZF、AF、PF、CF 和 OF。如果累加器的内容与字符串中的元素相等, 则比较之后(ZF)=1, 因此, 指令可以加上重复前缀 REPE 或 REPNE。前缀 REPE(即 REPZ)表示当(CX)≠0, 且(ZF)=1 时继续进行扫描。而 REPNE(即 REPNZ)表示当(CX)≠0; 且(ZF)=0 时继续进行扫描。

例：在包含 100 个字符的字符串中，寻找第一个回车符 CR(其 ASCII 码为 0DH)，找到后将其地址保留在(DS: DI)中，并在屏幕上显示字符“Y”；如果字符串中没有回车符，则在屏幕上显示字符“N”。该字符串的首地址位 STRING。根据要求可编程如下：

```

LEA  DI, STRING      ; DI ← 字符串首址
MOV  AL, 0DH         ; AL ← 回车符
MOV  CX, 100         ; CX ← 字符串长度
CLD                  ; 清状态标志位 DF
REPNE SCASB          ; 如未找到，重复扫描
JZ   MATCH           ; 如找到转 MATCH
MOV  DL, N           ; 字符串中无回车，则 DL ← “N”
JMP  DSPY            ; 转到 DSPY
MATCH: DEC DI         ; DI ← (DI) - 1
MOV  DL, 'Y'         ; DL ← “Y”
DSPY: MOV AH, 02H     ; 显示字符
INT  21H
HLT

```

在以上五条串操作指令中，有的指令有两个操作数，有的指令只有一个操作数。只有一个操作数时，有的指令是源操作数，有的指令是目标操作数。

关于串操作指令的重复前缀、操作数以及地址指针所用的寄存器等情况，如表 3-3 所示。

表 3-3 串操作指令的重复前缀、操作数和地址指针

指 令	重复前缀	操 作 数	地址指针寄存器
MOVS	REP	目的，源	ES: DI, DS: SI
LODS	无	源	DS: SI
STOS	REP	目的	ES: DI
CMPS	REPE/REPNE	源，目的	DS: SI, ES: DI
SCAS	REPE/REPNE	目的	ES: DI

3.2.7 处理器控制指令

该类指令用于对 CPU 进行控制，例如对 CPU 中某些状态标志位的状态进行操作，以及使 CPU 暂停、等待等。8086/8088 指令系统的处理器控制指令可分为三组。

1. 标志位操作指令

1) CLC

清进位标志。指令的操作为 $CF \leftarrow 0$

2) STC

置进位标志。指令的操作为 $CF \leftarrow 1$

3) CMC

对进位标志求反。指令的操作为 $CF \leftarrow \overline{(CF)}$

4) CLD

清方向标志。指令的操作为 $DF \leftarrow 0$

5) STD

置方向标志。指令的操作为 $DF \leftarrow 1$

6) CLI

清中断允许标志。指令的操作为 $IF \leftarrow 0$

7) STI

置中断允许标志。指令的操作为 $IF \leftarrow 1$ 。在执行这条指令后, CPU 将允许外部的可屏蔽中断请求。

这些指令仅对有关状态标志位执行操作, 而对其他状态标志位则没有影响。

2. 外部同步指令

1) HLT

执行 HLT 指令后, CPU 进入暂停状态。外部中断(当 $IF=1$)时的可屏蔽中断请求 INTR, 或非屏蔽中断请求 NMI 或复位信号 RESET 可使 CPU 退出暂停状态。HLT 指令对状态标志位没有影响。

2) WAIT

如果 8086/8088 CPU 的 TEST 引脚上的信号无效(即高电平), 则 WAIT 指令使 CPU 进入等待状态。一个被允许的外部中断或 TEST 信号有效, 可使 CPU 退出等待状态。

在允许中断的情况下, 一个外部中断请求将使 CPU 离开等待状态, 转向中断服务程序。此时被推入堆栈进行保护的断点地址即是 WAIT 指令的地址, 因此从中断返回后, 又执行 WAIT 指令, CPU 再次进入等待状态。

如果 TEST 信号变低(有效), 则 CPU 不再处于等待状态, 开始执行下面的指令。但是, 在执行完下一条指令之前, 不允许有外部中断。

本指令对状态标志位没有影响。WAIT 指令的用途是使 CPU 本身与外部的硬件同步工作。

3) ESC

指令格式

ESC ext-op, src

ESC 指令使其他处理器可使用 8086/8088 的寻址方式, 并从 8086/8088 CPU 的指令队列中取得指令。以上指令格式中的 ext-op 是其他处理器的一个操作码(外操作码), src 是一个存储器操作数。执行 ESC 指令时, 8086/8088 CPU 访问一个存储器操作数, 并将其放在数据总线上, 供其他处理器使用。此外没有其他操作。例如: 协处理器 8087 的所有指令机器码的高五位都是“11011”, 而 8086/8088 的 ESC 指令机器码的第一个字节恰是“11011XXX”, 因此, 对于这样的指令, 8086/8088 CPU 将其视为 ESC 指令, 它将存储器操作数置于总线上, 然后由 8087 来执行该指令, 并使用总线上的操作数。8087 的指令系统请参考有关资料。

ESC 指令对状态标志位没有影响。

4) LOCK

这是一个特殊的可以放在任何指令前面的单字节前缀。这个指令前缀迫使 8086/8088 CPU 的总线锁定信号线 LOCK 维持低电平(有效), 直到执行完下一条指令。外部硬件可接

收这个 LOCK 信号。在其有效期间，禁止其他处理器对总线进行访问。共享资源的多处理器系统中，必须提供一些手段对这些资源的存取进行控制，指令前缀 LOCK 就是一种手段。

3. 空操作指令 NOP

执行 NOP 指令时不进行任何操作，但占用 3 个时钟周期，然后继续执行下一条指令。NOP 指令对状态标志位没有影响，指令没有操作数。

3.3 小 结

本章介绍了 8086/8088 CPU 的寻址方式以及各种指令系统，并通过具体实例讲述各条指令使用方法和功能，并特别强调运算结果对标志位的影响。

3.4 习 题

1. 设 $BX=6EF8H$, $SI=2AEDH$, 位移量 $D=5F37H$, 试求下列寻址下有效地址 $EA=?$
(1) 直接寻址 (2) 基址加变址寻址 (3) 用 BX 间接寻址
2. 假定 LAB 是标号, VAR 是变量, CON 是常数名称, 列出下述每个操作数所有可能的寻址方式。
(1) $VAR[BX]$ (2) $CON+50H$ (3) VAR
(4) LAB (5) $VAR[BX+3]$ (6) $VAR[BX+DI]$
3. 试指出下列传送类指令的寻址方式。(所有标识符都是字变量)
(1) MOV DI, 1000 (2) MOV $VAR[BX]$, CX
(3) MOV AX, $VAR[BX][DI]$ (4) MOV $[BX+100]$, DI
(5) MOV $[BP+SI]$, 100 (6) PUSH AX
4. 指出下列传送指令中非法的指令。
(1) MOV BX, AL (2) MOV BH, AL
(3) MOV 100, CL (4) MOV AH, AL
(5) MOV SS, 2400H (6) MOV CS, 1200H
(7) XCHG AH, 12H (8) ADD AX, CS
(9) OUT 21H, AL (10) OUT 258H, AX
5. 设 $(SP)=2000H$, $(AX)=3355H$, $(BX)=4466H$, 指出执行下列指令后相关寄存器的内容。
(1) PUSH AX
执行后, $(AX)=?$ $(SP)=?$
(2) PUSH AX
PUSH BX
POP DX
执行后, $(AX)=?$, $(DX)=?$, $(SP)=?$
6. 设 $(BX)=0400H$, $(SI)=003CH$, 执行 $LEA BX, [BX+SI-0F62H]$ 后, $(BX)=?$

7. 设 $DS=C000H$, $[C0010H]=0180H$, $[C0012H]=2000H$, 问执行 `LDS SI, [10H]` 后, $(SI)=?$ $(DS)=?$

8. 已知 $(DS)=091DH$, $(SS)=1E4AH$, $(AX)=1234H$, $(BX)=0024H$, $(CX)=5678H$, $(BP)=0024H$, $(SI)=0012H$, $(DI)=0032H$, $[09226]=00F6H$, $[09228]=1E40H$, $(1EAF6H)=091DH$, 试求单独执行下列指令后的结果。

(1) `MOV CL, 20H[BX][SI]`; $(CL)=?$

(2) `MOV [BP][DI], CX` ; $(1E4F6H)=?$

(3) `LEA BX, 20H[BX][SI]` ; $(BX)=?$

`MOV AX, 2[BX]` ; $(AX)=?$

(4) `LDS SI, [BX][DI]` ; $(SI)=?$

`MOV [SI], BX` ; $[SI]=?$

(5) `XCHG CX, 32H[BX]` ; $(CX)=?$

`XCHG 20[BX][SI], AX`; $(AX)=?$ $[09226H]=?$

9. 使用移位指令来做乘以 2 和除以 2 是很方便的。试把 +53、-49 乘以 2, 它们各应用什么指令得到的结果各是什么? 若除以 2 呢?

10. 设 $(DS)=2000H$, $(BX)=1256H$, $(SI)=528FH$, 偏移量 = $20A1H$, $[232F7H]=3280H$, $[264E5H]=2450H$, 执行下述指令:

(1) `JMP BX`; $IP=?$

(2) `JMP TABLE[BX]` ; $IP=?$

(3) `JMP [BX][SI]`; $IP=?$

11. 8066/8088 用什么途径来更新 CS 和 IP 的值?

12. 设 $(IP)=3D8FH$, $(CS)=4050H$, $(SP)=0F17CH$, 当执行 `Call 2000: 0094H` 后, 试指出 (IP) 、 (CS) 、 (SP) 、 $[SP]$ 、 $[SP+1]$ 、 $[SP+2]$ 和 $[SP+3]$ 的内容。

13. 已知一个关于 0~9 的数字的 ASCII 码表首址是当前数据段的 $0A80H$, 现要找出数字 5 的 ASCII 码, 试写出用指令 `XLAT` 进行转换的指令序列。

14. 有程序段

```
MOV CX, 10
LEA SI, First
LEA m, Second
REP MOVSB
```

请指出该程序段的功能。

15. 今有以下程序段

```
CLD
LEA DI, [0404H]
MOV DS, 0080H
XOR AX, AX
REP STOSW
```

试分析执行该程序段的功能。

16. 试编程完成 $(AX) \times 5/2$ 的程序段。

17. 有(AL)=FFH, (BL)=03H, 指出下列指令执行后标志 OF、SF、ZF、PF、CF 的状态

- | | |
|-----------------|----------------|
| (1) ADD BL, AL | (2) INC BX |
| (3) SUB DX, AX | (4) NEG BH |
| (5) CMP BL, AL | (6) MUL BX |
| (7) AND DX, AX | (8) IMUL DX |
| (9) OR DX, AX | (10) SHL AX, 1 |
| (11) XOR DX, AX | (12) SAR AL, 1 |
| (13) SHR AL, 1 | |

18. 在以 DATA 为首址的内存数据段中, 存放了 100 个 16 位带符号数, 试将其中最大和最小的带符号数找出来, 分别存放到以 MAX 和 MIN 为首的内存单元中。

第 4 章 汇编语言程序设计

教学提示：本章从汇编语言语句的书写格式、组成内容、设计方法一直讲到实际程序的编写，重点介绍顺序、分支、循环基本程序的设计，多数例题配有程序流程图，帮助学生加深理解，并简化程序设计。本章的结尾，简单地介绍了 DOS 系统的功能调用。

教学要求：通过本章学习，应该掌握以下内容：汇编语言程序设计的基本方法；顺序、分支、循环基本程序的编写；上机调试。

4.1 机器语言与汇编语言

任何计算机都必须在程序控制之下进行有效的工作。为了沟通使用者和计算机之间的信息交换，产生了各种各样的程序设计语言。各种语言都有自己的特点、优势及运行环境，有自己的应用领域和针对性。从使用者的角度看，计算机程序设计语言一般可分为机器语言、汇编语言和高级语言三种不同层次的语言。

所谓汇编语言是一种采用助记符表示的程序设计语言，即用助记符来表示指令的操作码和操作数，用标号或符号代表地址、常量或变量。助记符一般都是英语词的缩写，因而方便人们书写、阅读和检查。一般情况下，一个助记符表示一条机器指令，所以汇编语言也是面向机器的语言。实际上，由汇编语言编写的汇编语言源程序就是机器语言程序的符号表示，汇编语言源程序与其经过汇编所产生的目标代码程序之间有明显的一一对应关系。

用汇编语言编写程序能够直接利用硬件系统的特性(如寄存器、标志、中断系统等)直接对位、字节、字寄存器、存储单元、I/O 端口进行处理，同时也能直接使用 CPU 指令系统和指令系统提供的各种寻址方式，编制出高质量的程序，这样的程序不但占用内存空间少，而且执行速度快。当然，由于汇编语言不能独立于具体的机器，只有对微处理器指令系统熟悉、掌握以后，才能用汇编语言进行程序设计。编程的难度及工作量相当大，也增加了程序设计过程中出错的可能性。

汇编程序是最早也是最成熟的一种系统软件。它除了能够将汇编语言源程序翻译成机器语言程序这一主要功能外，还能够根据用户的要求自动分配存储区域(包括程序区、数据区、暂存区等)，自动地把各种进制数转换成二进制数，把字符转换成 ASCII 码，计算表达式的值，自动对源程序进行检查，给出错误信息(如非法格式，未定义的助记符、标号，漏掉操作数等)。具有这些功能的汇编程序又称为基本汇编(或小汇编 ASM)。

在基本汇编的基础上，进一步允许在源程序中，把一个指令序列定义为一条宏指令的汇编程序，就叫做宏汇编(MASM)。

4.2 汇编语言语句

4.2.1 汇编语言语句的种类

汇编语言源程序中的语句可以分为三种类型：指令语句、伪指令语句和宏指令语句。

(1) 指令语句能产生目标代码，CPU 可以执行的能完成特定功能的语句，它主要由 CPU 指令组成。

(2) 伪指令语句是一种不产生目标代码的语句，仅仅在汇编过程中告诉汇编程序应如何汇编指令序列。例如，告诉汇编程序已写出的汇编语言源程序有几个段，段的名字是什么，定义变量，定义过程，给变量分配存储单元，给数字或表达式命名等。所以伪指令语句是为汇编程序在汇编时用的。

(3) 宏指令语句是一个指令序列，汇编时，凡有宏指令语句的地方，都将用相应的指令序列的目标代码插入。

4.2.2 汇编语言的语句格式

指令语句与伪指令语句的格式是类似的，下面主要介绍这两种语句的格式，宏指令语句的格式稍后再作介绍。

一般情况下，汇编语言的语句可以由 1~4 部分构成：

[名字:] 助记符 [操作数] [: 注释]

其中带方括号的部分表示任选项，既可以有，也可以没有。如：

```
LOOPER: MOV    AL, DATA[SI]           ; 取一个字节数
DATA1    DB    0F8H, 60H, 0ACH, 74H, 3BH ; 定义数组
```

第一条语句是指令语句，其中的“MOV”是 CPU 指令的助记符；第二条语句是伪指令语句，其中的“DB”是伪指令定义符。下面对汇编语言中的各个组成部分进行讨论。

1. 名字

汇编语言语句的第一个组成部分是名字。在指令语句中，这个名字可以是一个标号。指令语句中的标号实质上是指令的符号地址。并不是每条指令都需要有标号。如果指令前有标号，程序的其他地方就可以引用这个标号。标号后面有冒号。

标号有三种属性：段、偏移量、类型。

(1) 标号的段属性是定义标号的程序段的段基值，当程序中引用一个标号时，该标号的段基值应在 CS 寄存器中。

(2) 标号的偏移量属性表示标号所在段的起始地址到定义该标号的地址之间的字节数。偏移量是一个 16 位无符号数。

(3) 标号的类型属性有两种：NEAR 和 FAR。前一种标号可以在段内被引用，地址指针为 2 个字节；后一种标号可以在其他段被引用，地址指针为 4 个字节。如果定义一个标号时后跟冒号，则汇编程序确认其类型为 NEAR。

伪指令语句中的名字可以是变量名、段名、过程名。与指令语句中的标号不同，这些伪指令语句中的名字并不总是任选的，有些伪指令规定前面必须有名字，有些则不允许有

名字，也有一些伪指令的名字是可选的。即不同的伪指令对于是否有名字有不同的规定。伪指令语句的名字后不加冒号，这是它与标号的明显区别。

很多情况下伪指令语句中的名字是变量名，变量名代表存储器中一个数据区的名字。

变量也有三种属性：段、偏移量和类型。

(1) 变量的段属性是变量所代表的数据区域所在段的段基值，由于数据区一般在存储器的数据段，因此，变量的段基值通常在 DS 和 ES 中。

(2) 变量的偏移量属性是该变量所在段的起始地址与变量的地址之间的字节数。

(3) 变量的类型属性有 BYTE(字节)、WORD(字)、DWORD(双字)、QWORD(4 个字)、TBYTE(10 个字节)等，表示数据区中存取操作对象的大小。

2. 助记符

伪指令语句中的第二部分是伪指令的定义符。将在 4.3 节详细讲解。

3. 操作数

汇编语言语句中的第三个组成部分是操作数。指令语句中是指令的操作数，可能有单操作数、双操作数、多操作数，也可能无操作数。当操作数不止一个，相互之间应该用逗号隔开。

4. 注释

汇编语言语句中的最后一部分是注释。对于一个汇编语句来说，注释部分不是必要的，加上注释可以增加程序的可读性。注释前面要求加上分号，如果注释的内容较多，超过一行，则换行以后前面还要加上分号。注释对汇编后生成的目标程序没有任何影响。

4.2.3 指令语句的操作数组成

操作数的类型：常量、寄存器、存储器、标号、变量和表达式。

1. 常量

常量就是指令中出现的那些固定值，可以分为数值常量和字符串常量两类。例如，立即数寻址时所用的立即数，直接寻址时所用的地址，ASCII 字符串都是常量，常量除了自身的值以外，没有其他的属性。在源程序中，数值常量可用二进制数、八进制数、十进制数、十六进制数等几种不同表示形式。汇编语言用不同的后缀加以区别。

还应指出，汇编语言中的数值常量的第一位必须是 0~9 数字，否则汇编时将被看成是标识符。例如常数 B7H，在语言中应写成 0B7H，FFH 应写成 0FFH。

字符串常量是由单引号 ‘ ’ 括起来的一串字符。例如 ‘ABCDEF’，‘123’。单引号内的字符汇编时均以 ASCII 码的形式存在。上述两个字符串的 ASCII 码分别是 41H，42H，43H，44H，…，46H，31H，32H，33H。字符串最长可以有 255 个字符。汇编语言规定：除用 DB 定义的字符串常量以外，单引号中 ASCII 字符的个数不得超过两个。若只有一个，例如，DW ‘C’ 就相当于 DW 0043H。

2. 寄存器

8086/8088 CPU 的寄存器可以作为指令的操作数，8 位寄存器：AH，AL，BH，BL，CH，

CL, DH, DL; 16 位寄存器: AX, BX, CX, DX, SI, DI, BP, SP, DS, ES, SS, CS。

3. 标号

由于标号代表一条指令的符号地址, 因此可以作为转移(无条件转移或条件转移)、过程访问以及循环控制指令的操作数。

4. 变量

因为变量是存储器中某个数据区的名字, 因此在指令中可以作为存储器操作数。

5. 表达式

汇编语言语句中的表达式, 按其性质可分为两种: 数值表达式和地址表达式。

6. 存储器

计算机内存中存放的数据。

4.2.4 指令语句中的运算符和操作符

数值表达式是一个数值结果, 只有大小, 没有属性。地址表达式的结果不是一个单纯的数值, 而是一个表示存储器地址的变量或标号, 它有三种属性: 段、偏移量和类型。

表达式中常用的运算符有以下几种。

1. 算术运算符

常用的运算符有: +加, -减, ×乘, /除, 和 MOD 模除(即两个整数相除后取余数)等。以上算术运算符可用于数值表达式, 运算结果是一个数值。在地址表达式中通常只使用+和-(加和减)两种运算符。

2. 逻辑运算符

逻辑运算符有: AND 逻辑与、OR 逻辑或、XOR 逻辑异或、NOT 逻辑非。逻辑运算符只用于数值表达式中对数值进行按位逻辑运算, 并得到一个数值结果。

3. 关系运算符

关系运算符有: EQ 等于、NE 不等、LT 小于、GT 大于、LE 小于或等于、GE 大于或等于。

参与关系运算的必须是两个数值或同一段中的两个存储单元地址, 但运算结果只能是两个特定的数值之一。当关系不成立(假)时: 结果为 0(全 0); 当关系成立(真)时, 结果为 0FFFFH(全 1)。例如:

```
MOV AX, 4 EQ 3      ; 关系不成立, 故 (AX) ← 0
MOV AX, 4 NE 3      ; 关系成立, 故 (AX) ← 0FFFFH
```

4. 分析运算符

分析运算符可以把存储器操作数分解为它的组成部分, 如它的段值、段内偏移量和类型, 或取得它所定义的存储空间的大小。分析运算符有 SEG、OFFSET、TYPE、SIZE 和 LENGTH 等。

1) SEG 运算符

利用 SEG 运算符可以得到标号或变量的段基值。例如将 ARRAY 变量的段基值送 DS

寄存器。

```
MOV AX, SEG ARRAY
MOV DS, AX
```

2) OFFSET 运算符

利用 OFFSET 运算符可以得到标号或变量的偏移量。例：

```
MOV DI, OFFSET DATA1
```

3) TYPE 运算符

运算符 TYPE 的运算结果是个数值，这个数值与存储器操作数类型属性的对应关系如表 4-1 所示。

表 4-1 TYPE 返回值与类型的关系

TYPE 返回值	存储器操作数的类型
1	BYTE
2	WORD
4	DWORD
-1	NEAR
-2	FAR

下面是使用 TYPE 运算符的语句例子：

```
VAR    DW    ?           ; 变量 VAR 的类型为字
ARRAY  DD    10DUP(?)    ; 变量 ARRAY 的类型为双字
STR    DB    'THIS IS TEST' ; 变量 STR 的类型为字节
...
MOV    AX, TYPE VAR      ; (AX) ← 2
MOV    BX, TYPE ARRAY    ; (BX) ← 4
MOV    CX, TYPE STR      ; (CX) ← 1
```

程序中的 DW、DD、DB 等为伪指令定义符。

4) LENGTH 运算符

如果一个变量已用重复操作符 DUP 说明变量的个数，则可用 LENGTH 运算符得到变量的个数。如果一个变量未用重复操作符 DUP 说明，则得到的结果总是 1。如上面的例子中 ARRAY DD 10DUP(?)，则 LENGTH ARRAY 的结果为 10。

5) SIZE 运算符

如果一个变量已用重复操作符 DUP 说明，则利用 SIZE 运算符可得到分配给该变量的字节总数。如果一个变量未用重复操作符 DUP 说明，则利用 SIZE 运算符可得到 TYPE 运算的结果。ARRAY DD 10DUP(?)，SIZE ARRAY=10×4=40。由此可知，SIZE 的运算结果等于 LENGTH 的运算结果乘以 TYPE 的运算结果。

```
SIZE ARRAY=(LENGTH ARRAY)×(TYPE ARRAY)
```

5. 合成运算符

合成运算符可以用来建立或临时改变变量或标号的类型或存储器操作数的存储单元类型。合成运算符有 PTR、THIS、SHORT 等。

1) PTR 运算符

运算符 PTR 可以指定或修改存储器操作数的类型，例如：

```
INC    BYTE PTR [BX][DI]
```

指令中利用 PTR 运算符明确规定了存储器操作数的类型是 BYTE(字节), 因此, 本指令将一个 8 位存储器的内容加 1。

利用 PTR 运算符可以建立一个新的存储器操作数, 这个操作数与原来的同名操作数具有相同的段和偏移量, 但可以有不同的类型。不过这个新类型只在当前语句中有效。例如:

```
STUFF DD      ?      ; 定义 STUFF 为双字类型变量
MOV  BX, WORD PTR STUFF ; 从 STUFF 中取一个字到 BX
```

2) THIS 运算符

运算符 THIS 也可指定存储器操作数的类型。使用 THIS 运算符可以使标号或变量具有灵活性。例如要求对同一个数据区, 既可以字节为单位, 又可以字为单位进行存取, 则可用以下语句:

```
AREAW EQU THIS WORD
AREAB DB      100 DUP(?)
```

上面 AREAW 和 AREAB 实际代表同一个数据区, 共有 100 个字节, 但 AREAW 的类型为字, AREAB 的类型为字节。

3) SHORT 运算符

运算符 SHORT 指定一个标号的类型为 SHORT(短标号), 即标号到引用该标号的字节距离在 -128~+127 范围内。短标号可以用于转移指令中。使用短标号的指令比使用默认的近程标号的指令少一个字节。

6. 其他运算符

1) 段超越运算符 “:”

运算符 “:” 紧跟在段寄存器名(DS、CS、SS、ES)之后, 表示段超越, 用来给存储器操作数指定一个段的属性, 而不管原来隐含在什么段。

```
MOV  AX, ES: [SI]
```

2) 字节分离运算符

运算符 LOW 和 HIGH 分别得到一个数值或地址表达式的低位和高位字节。

```
STUFF EQU 0ABCDH
MOV  AH, HIGH STUFF ; (AH) ← 0ABH
MOV  AL, LOW STUFF  ; (AL) ← 0CDH
```

以上介绍了表达式中使用的各种运算符, 如果一个表达式同时具有多个运算符, 则按以下规则运算:

- (1) 优先级高的先运算, 优先级低的后运算。
 - (2) 优先级相同时, 按表达式从左到右的顺序运算。
 - (3) 括号可以提高运算的优先级, 括号内的运算总是在相邻的运算之前进行。
- 各种运算符的优先级顺序见表 4-2。表中同一行的运算符具有相等的优先级。

表 4-2 各种运算符的优先级顺序

优 先 级	运 算 符
高	
1	LENGTH, SIZE, WIDTH, MASK, (), [], <>

续表

优 先 级	运 算 符
2	.(结构变量名后面的运算符)
3	: (段超越运算符)
4	PTR, OFFSET, SEG, TYPE, THIS
5	HIGH, LOW
6	+, -(一元运算符)
7	*, /, MOD, SHL, SHR
8	+, -(二元运算符)
9	EQ, NE, LT, LE, GT, GE
10	NOT
11	AND
12	OR, XOR
13	SHORT

4.3 伪指令系统

伪指令无论表现形式或其在语句中所处的位置都与 CPU 指令相似，但二者之间有重要区别。首先，CPU 指令是给 CPU 的命令，在运行时由 CPU 执行，每条指令对应 CPU 的一种特定操作；而伪指令是给汇编程序的命令，在汇编过程中由汇编程序进行处理。其次，汇编以后，每条 CPU 指令产生一一对应的目标代码；而伪指令则不产生一一对应的目标代码。

根据伪指令的功能，大致分成以下几类：

- (1) 数据定义伪指令。
- (2) 符号定义伪指令。
- (3) 段定义伪指令。
- (4) 过程定义伪指令。
- (5) 宏处理伪指令。
- (6) 模块定义与连接伪指令。
- (7) 处理器方式伪指令。
- (8) 条件伪指令。
- (9) 列表伪指令。
- (10) 其他伪指令。

4.3.1 数据定义伪指令

数据定义伪指令的用途是定义一个变量的类型，给存储器赋初值，或给变量分配存储单元。常用的数据定义伪指令有 DB、DD、DW 等。

数据定义伪指令的一般格式为：

[变量名] 伪指令 操作数 [, 操作数…]

方括号中的变量名为任选项。变量名后面不跟冒号。伪操作后面的操作数可以不止一个，如有多个操作数，互相之间应该用逗号分开。

1. DB (Define byte)

定义变量的类型为 BYTE，给变量分配字节或字节串。DB 伪操作后面的每一个操作数

占有 1 个字节。

2. DW (Define word)

定义变量的类型为 WORD。DW 伪操作后面的操作数每个占有 1 个字，即 2 个字节。在内存中存放时，低位字节在前，高位字节在后。

3. DD(Define double word)

定义变量的类型为 DWORD。DD 后面的操作数每个占有 2 个字，即 4 个字节。在内存中存放时，低位字在前，高位字在后。

数据定义伪操作后面的操作数可以是常数、表达式或字符串，但每项操作数的值不能超过由伪操作所定义的数据类型限定的范围。例如，DB 伪指令定义数据的类型为字节，则其范围为无符号数 0~255；带符号数 -128~+127 等。字符串必须放在单引号中。另外，超过两个字符的字符串只能用 DB 伪指令定义。例如：

DATA	DB	100, 0FFH	; 存入 64H, FFH
EX	DB	2*3+7	; 存入 0DH
STR	DB	'WELCOME!'	; 存入 8 个字符
AB	DB	'AB'	; 存入 41H, 42H
BA	DW	'AB'	; 存入 4142H
AD	DD	'AB'	; 存入 00004142H
AQ	DD	AB	; 存入 AB 的偏移地址
AF	DW	TABLE, TABLE-5, TABLE+10	; 存入 3 个偏移地址
TOTAL	DW	TABLE, TABLE-5	; 存入 TABLE 偏移地址，
TOTAL	DD	TABLE	; 再存入 TABLE 的段基址

以上第一和第二语句中，分别将常数和表达式的值赋给一个变量。第三句的操作数是包含 8 个字符的字符串(只能用 DB)。在第四、五、六句，注意伪指令 DB、DW 和 DD 的区别，虽然操作数均为“AB”两个字符，但存入变量的内容各不相同。第七句的操作数是变量 AB，而不是字符串，此句将 AB 的 16 位偏移地址存入变量 AQ。第八句存入三个等距的偏移地址，共占 6 个字节。第九句中的 DD 伪操作将 TABLE 的偏移地址和段地址顺序存入变量 TOTAL，共占 2 个字。

除了常数、表达式和字符串外，问号“?”也可以作为数据定义伪指令的操作数，此时仅给变量保留相应的存储单元，而不赋给变量某个确定的初值。

当同样的操作数重复多次时，可用重复操作符“DUP”表示，其形式为：

n DUP (初值[初值…])

圆括号中为重复的内容，n 为重复次数。如果用“n DUP(?)”作为数据定义伪指令的唯一操作数，则汇编程序产生一个相应的数据区，但不赋任何初值。重复操作符“DUP”可以嵌套。

FILLER	DB	?
SUM	DW	?
	DB	?, ?, ?
BUFFER	DB	10 DUP(?)
ZERO	DW	30 DUP(0)
MASK	DB	5 DUP ('OK!')
ARRAY	DB	100 DUP (3 DUP(8), 6)

其中第一、第二句分别给字节变量 FILLER 和字变量 SUM 分配存储单元, 但不赋给特定的值。第三句给一个没有名称的字节变量分配 3 个单元。第四句给变量 BUFFER 分配 10 个字节的存储空间。第五句给变量 ZERO 分配一个数据区, 共 30 个字(即 60 个字节), 每个字的内容均为零。第六句定义一个数据区, 其中有 5 个重复的字符串“OK!”, 共占 15 个字节。最后一句将变量 ARRAY 定义一个数据区, 其中包含重复 100 次的内容: 8, 8, 8, 6, 共占 400 个字节。

下面列出几个错误的数据定义伪指令语句。

```
ERR1: DW 99          ; 变量名后有冒号
ERR2 DB 25*60        ; DB 的操作数超过 255
FRR3 DD 'ABCD' ; DD 的操作数是超过 2 个字符的字符串
```

4.3.2 符号定义伪指令

符号定义伪指令的用途是给一个符号重新命名, 或定义新的类型属性等。上述符号包括汇编语言的变量名、标号名、过程名, 寄存器名以及指令助记符等。

常用的符号定义伪指令有: EQU、=(等号)和 LABLE。

1. EQU

格式:

名字 EQU 表达式

EQU 伪指令是将表达式的值赋给一个名字, 以后可以用这个名字来代替表达式。格式中的表达式可以是一个常数、符号、数值表达式或地址表达式。

```
CR      EQU 0DH          ; 常数
LF      EQU 0AH
A       EQU ASCII-TABLE  ; 变量
STR     EQU 64*1024       ; 数值表达式
ADR     EQU ES:[BP+DI+5]  ; 地址表达式
CBD     EQU AAM           ; 指令助记符
```

利用 EQU 伪指令, 可以用一个名字代表一个数值, 或用一个简短的名字代替一个较长的名字。

如果源程序中需要多次引用某一表达式, 则可以利用 EQU 伪操作给其赋一个名字, 以代替程序中的表达式, 从而使程序更加简洁, 便于阅读。将来如果改变表达式的值, 也只需修改一处, 程序易于维护。注意: EQU 伪指令不能对同一符号重复定义。

2. =(等号)

格式:

名字 = 表达式

=(等号)伪指令功能与 EQU 相似, 主要区别在于=(等号)可以对同一符号重复定义。例:

```
COUNT=10
      MOV CX , COUNT      ; (CX) ← 10
COUNT= COUNT-1
      MOV BX , COUNT      ; (BX) ← 9
```


3. LABEL

LABEL 伪指令是定义标号或变量的类型，它和下一条指令共享存储器单元。

名字 LABEL 类型

标号的类型可以是 NEAR 和 FAR，变量的类型可以是 BYTE(字节)、WORD(字)、DWORD(双字)。利用 LABEL 伪指令可以使同一个数据区兼有两种属性 BYTE(字节)和 WORD(字)，可以在以后的程序中根据不同的需要以字节或字为单位存取其中的数据。

```
ARW LABEL WORD ; 变量 ARW 类型为 WORD
ARB DB 100 DUP (?) ; 变量 ARB 类型为 BYTE
...
MOV ARW, AX ; AX 送第 1, 2 字节中
...
MOV ARB[49], AL ; AL 送第 50 个字节中
```

LABEL 伪指令也可以将一个属性已经定义为 NEAR 或者后面跟有冒号(隐含属性为 NEAR)的标号再定义为 FAR。

```
AGF LABEL FAR ; 定义标号的属性为 FAR
AG: PUSH AX ; 标号 AG 的属性为 NEAR
```

4.3.3 段定义伪指令

段定义伪指令的用途是在汇编语言源程序中定义逻辑段，常用段定义伪指令有 SEGMENT、ENDS、ASSUME 等。

1. SEGMENT/ENDS

格式：

```
段名 SEGMENT [定位类型] [组合类型] [‘类别’]
...
段名 ENDS
```

SEGMENT 伪指令用于定义一个逻辑段，给逻辑段赋予一个段名，并以后面的任选项(定位类型、组合类型、类别)规定该逻辑段的其他特性。SEGMENT 伪指令位于一个逻辑段的开始，而 ENDS 伪指令则表示一个逻辑段的结束。这两个伪操作总是成对出现，二者前面的段名必须一致。两个语句之间的部分即是该逻辑段的内容。例如，对于代码段，其中主要有 CPU 指令及其他伪指令。对于数据段和附加段，主要有定义数据区的伪指令等。一个源程序中不同逻辑段的段名可以各不相同，但也允许相同。

SEGMENT 伪指令后面还有三个任选项，在上面的格式中，它们都放在方括号内，表示可选择。如果有，三者的顺序必须符合格式中的规定。这些任选项是给汇编程序和连接程序(LINK)的命令。

SEGMENT 伪指令后面的任选项告诉汇编程序和连接程序，如何确定段的边界，以及如何组合几个不同的段等。下面分别讨论。

1) 定位类型(ALIGN)

定位类型任选项告诉汇编程序如何确定逻辑段的边界在存储器中的位置。定位类型共有以下四种：

(1) BYTE。

表示逻辑段从字节的边界开始，即可以从任何地址开始。此时本段的起始地址紧接在前一段后边。

(2) WORD。

表示逻辑段从字的边界开始。2 个字节为 1 个字，此时本段的起始地址必须是偶数。

(3) PARA。

表示逻辑段从节(PARAGRAPH)的边界开始，通常 16 个字节称为一个节，故本段的起始地址(十六进制)应为××××0H。如果省略定位类型选项，则默认值为 PARA。

(4) PAGE。

表示逻辑段从页的边界开始，通常 256 个字节称为一个页，故本段的开始地址(十六进制)应为×××00H。

```

STACK    SEGMENT STACK                      ; STACK 段，定位类型无
          DB          100 DUP(?)             ; 长度为 100 字节
STACK    ENDS                               ; STACK 段结束
DATA1    SEGMENT BYTE                       ; DATA1 段，定位类型 BYTE
STRING   DB          'This is an example'   ; 长度为 18 字节
DATA1    ENDS                               ; DATA1 段结束
DATA2    SEGMENT WORD                       ; DATA2 段，定位类型 WORD
BUFFER   DW          40 DUP(0)              ; 长度为 40 个字，即 80 字节
DATA2    ENDS                               ; DATA2 段结束
CODE1    SEGMENT PAGE                       ; CODE1 段，定位类型 PAGE
...
CODE1    ENDS                               ; CODE1 段结束
CODE2    SEGMENT                           ; CODE2 段，定位类型无
...
          START:    MOV          AX, STACK
                   MOV          SS, AX
          ...
CODE1    ENDS                               ; CODE2 段结束
          END START                          ; 源程序结束

```

本例的源程序中共有 5 个逻辑段，它们的段名和定位类型分别为：

```

STACK 段    PARA
DATA1 段    BYTE
DATA2 段    WORD
CODE1 段    PAGE
CODE2 段    PARA

```

已经知道其中 STACK 段的长度为 100 字节(64H)，DATA1 段的长度为 19 字节(12H)，DATA2 段的长度为 40 个字，即 80 字节(50H)。假设 CODE1 段占用 13 字节(0DH)，CODE2 段占用 52 字节(34H)。

如果将以上进行汇编和连接，然后再来观察各逻辑段的目标代码或数据装入存储器的情况。由表 4-3 可清楚地看出，当 SEGMENT 伪指令的定位类型不同时，对段起始边界规定也不相同。

2) 组合类型(Combine)

SEGMENT 伪指令的第二个任选项是组合类型，它告诉汇编程序，当装入存储器时，

各个逻辑段如何进行组合。组合类型共有 6 种：

(1) NONE。

如果 SEGMENT 伪指令的组合类型任选项默认，则汇编程序认为这个逻辑段是不组合的。也就是说，不同程序中的逻辑段，即使具有相同的类别名，也分别作为不同的逻辑段装入内存，不进行组合。

表 4-3 各逻辑段的起始地址和结束地址

段 名	定位类型	字 节 数	起始地址	结束地址
STACK	PARA	100	00000H	00063H
DATA1	BYTE	18	00064H	00075H
DATA2	WORD	80	00078H	000C7H
CODE1	PAGE	13	00100H	0010CH
CODE2	PARA	52	00110H	00143H

但是，对于组合类型任选项缺省的同名逻辑段，如果属于同一个程序模块，则被顺序连接成为一个逻辑段。

(2) PUBLIC。

连接时对于不同程序模块的逻辑段，只要具有相同的类别名，就把这些段顺序连接成为一个逻辑段装入内存。

(3) STACK。

组合类型为 STACK 时，其含意与 PUBLIC 基本一样，即不同程序中的逻辑段，如果类别名相同，则顺序连接成为一个逻辑段。不过组合类型 STACK 仅限于堆栈区域的逻辑段使用(顺便提一下，在执行程序(.EXE)中，堆栈指针 SP 设置在这个连接以后的堆栈段(最终地址+1)处。

(4) COMMON。

连接时，对于不同程序中的逻辑段，如果具有相同的类别名，则都从同一个地址开始装入，因而各个逻辑段将发生重叠。最后，连接以后的段的长度等于原来最长的逻辑段的长度，重叠部分的内容是最后一个逻辑段的内容。

(5) MEMORY。

表示当几个逻辑段连接时，本逻辑段定位在地址最高的地方。如果被连接的逻辑段中有多个段的组合类型都是 MEMORY，则汇编程序只将首先遇到的段为 MEMORY 段，而其余的段均当作 COMMON 段处理。

(6) AT 表达式。

这种组合类型表示本逻辑段根据表达式求值的结果定位段基址。例：AT 8A00H，表示本段的段基址为 8A00H，则本段从存储器的物理地址 8A000H 开始装入。

3) ‘类别’ (‘CLASS’)

SEGMENT 伪指令的第三个任选项是类别，类别必须放在单引号之内。典型类别如 ‘STACK’，‘CODE’。类别的主要作用是在连接时决定每个逻辑段的装入顺序。当几个程序模块进行连接时，其中具有相同类别名的逻辑段被装入连续的内存区，类别名相同的逻辑段，按出现的先后顺序排列。没有类别名的逻辑段，与其他没有类别名的逻辑段一起，

连续装入内存区。

2. ASSUME

格式:

ASSUME 段寄存器名: 段名 [, 段寄存器名: 段名 [, ...]]

对于 8086/8088 CPU 而言, 以上格式中的段寄存器名可以是 ES、CS、DS、SS。段名是曾用 SEGMENT 伪操作定义过的某一个段名或者组名, 以及在一个标号或变量前面加上分析运算符 SEG 所构成的表达式, 还可以是关键字 NOTHING。

ASSUME 伪指令告诉汇编程序, 将某一个段寄存器设置为某一个逻辑段的段址, 即明确指出源程序中的逻辑段与物理段之间的关系。当汇编程序汇编一个逻辑段时, 即可利用相应的段寄存器寻址该逻辑段中的指令或数据。关键字 NOTHING 表示取消前面用 ASSUME 伪指令对这个段寄存器的设置。在一个源程序中, ASSUME 伪操作应该放在可执行程序开始位置的前面。还需指出一点, ASSUME 伪指令只是通知汇编程序有关段寄存器与逻辑段的关系, 并没有给段寄存器赋予实际的初值。例如:

```
CODE  SEGMENT
        ASSUME  CS: CODE, DS: DATA1, SS: STACK
        MOV  AX, DATA1
        MOV  DS, AX
        MOV  AX, STACK
        MOV  SS, AX
CODE  ENDS
```

4.3.4 模块定义与连接伪指令

在编写规模比较大的汇编语言程序时, 可以将整个程序划分成为几个独立的源程序(或称模块), 然后将各个模块分别进行汇编, 生成各自的目标程序, 最后将它们连接成为一个完整的可执行程序。各个模块之间可以相互进行符号访问。也就是说, 一个模块定义的符号可以被另一个模块引用。通常称这类符号为外部符号, 而将那些在一个模块中定义的, 只在同一个模块中引用的符号称为局部符号。

为了进行连接和在这些将要连接在一起的模块之间实现互相的符号访问, 以便进行变量传送, 常常使用以下伪指令: NAME、END、PUBLIC、EXTRN。

1. NAME

NAME 伪指令用于给源程序汇编以后得到的目标程序指定一个模块名, 连接时需要使用这个目标程序的模块名。格式为:

NAME 模块名

NAME 的前面不允许再加上标号, 例如, 下面的表示方式是非法的:

```
BEGIN: NAME  MODNAME
```

如果程序中没有 NAME 伪指令, 则汇编程序将 TITLE 伪指令(TITLE 属于列表伪指令)后面“标题名”中的前六个字符作为模块名。如果源程序中既没有使用 NAME, 也没有使用 TITLE 伪指令, 则汇编程序将源程序的文件名作为目标程序的模块名。

2. END

END 伪指令表示源程序到此结束，指示汇编程序停止汇编，对于 END 后边的指令可以不予理会。格式为：

END [标号]

END 伪指令后面的标号表示程序执行的启动地址。END 伪指令将标号的段基值和偏移地址分别提供给 CS 和 IP 寄存器。方括号中的标号是任选项。如果有多个模块连接在一起，则只有主模块的 END 语句使用标号。

3. PUBLIC

PUBLIC 伪指令说明本模块中的某些符号是公共的，即这些符号可以提供给将被连接在一起的其他模块使用。格式为：

PUBLIC 符号[, ...]

其中的符号可以是本模块中定义的变量、标号或数值的名字，包括用 PROC 伪指令定义的过程名等。PUBLIC 伪指令可以安排在源程序的任何地方。

4. EXTRN

EXTRN 伪指令说明本模块中所用的某些符号是外部的，即这些符号将被连接在一起的其他模块定义(在这些模块中符号必须用 PUBLIC 定义)。格式为：

EXTRN 名字: 类型[, ...]

其中的名字必须是其他模块中定义的符号；上述格式中的类型必须与定义这些符号的模块中的类型说明一致。如为变量，类型可以是 BYTE、WORD 或 DWORD 等；如为标号和过程，类型可以是 NEAR 或 FAR。

4.3.5 过程定义伪指令

过程定义伪指令 PROC/ENDP

格式为：

```
过程名  PROC [NEAR]/FAR
...
RET
...
过程名  ENDP
```

其中 PROC 伪指令定义一个过程，赋予过程一个名字并指出该过程的属性为 NEAR 或 FAR。如果没有特别指明类型，则认为过程的属性为 NEAR。伪指令 END 标志过程结束。PROC/ENDP 伪指令前的过程名必须一致。

当一个程序段被定义为过程后，程序中其他地方就可以用 CALL 指令来调用这个过程。调用的格式为：

CALL 过程名

过程名实质上是过程入口的符号地址，它和标号一样，也有三种属性：段、偏移量和类型。过程的类型属性可以是 NEAR 或 FAR。

一般来说,被定义为过程的程序段中应该有返回指令 **RET**,但不一定是最后一条指令,也可以有不止一条 **RET** 指令。执行 **RET** 指令后,控制返回到原来调用指令的下一条指令。

过程的定义和调用均可嵌套。例如:

```

NA1  PROC  FAR
...
CALL NA2
RET
NA2  PROC  NEAR
...
RET
NA2  ENDP
NA1  ENDP

```

上面过程 **NA1** 的定义中包含着另一个过程 **NA2** 的定义。**NA1** 本身是一个可以被调用的过程,而它也可以再调用其他的过程。

4.4 汇编语言程序的结构

4.4.1 汇编语言程序的结构

在第 3 章所举出的若干程序段并不是规范的汇编语言源程序,下面举一个比较简单,而比较规范的汇编语言源程序。

【例 4.1】 要求将两个 5 字节十六进制数相加,可以编写出以下汇编语言源程序。

```

DATA    SEGMENT                                ; 定义数据段
DATA1   DB    0F8H, 60H, 0ACH, 74H, 3BH        ; 被加数
DATA2   DB    0C1H, 36H, 9EH, 0D5H, 20H        ; 加数
DATA    ENDS                                    ; 数据段结束
CODE    SEGMENT                                ; 定义代码段
        ASSUME  CS: CODE, DS: DATA
START:  MOV     AX, DATA
        MOV     DS, AX                          ; 初始化 DS
        MOV     CX, 5                           ; 循环次数 CX
        MOV     SI, 0                           ; 置 SI 初值为 0
        CLC                                         ; 清 CF 标志
LOOP:   MOV     AL, DATA2[SI]                    ; 取一个字节加数
        ADC     DATA1[SI], AL                    ; 与被加数相加
        INC     SI                               ; SI 加 1
        DEC     CX                               ; CX 减 1
        JNZ     LOOP                             ; 若不等于 0, 转 LOOP
        HLT                                         ; 停止
CODE    ENDS                                    ; 代码段结束
        END          START                       ; 源程序结束

```

由上面的例子可以看出,汇编语言源程序的结构是分段结构形式。一个汇编语言源程序由若干逻辑段(SEGMENT)组成,每个逻辑段以 **SEGMENT** 语句开始,以 **ENDS** 语句结束,整个程序以 **END** 语句结束。

每个逻辑段内有若干行语句(STATEMENT),一个汇编语言源程序由一行一行的语句

组成。

4.4.2 程序正常返回 DOS 的方法

当我们编写的汇编语言源程序是在 PC-DOS 环境下运行时, 必须了解汇编语言是如何同操作系统接口的。

当用连接程序对其进行连接和定位时, 操作系统为每一个用户程序建立了一程序段前缀区 PSP, 其长度为 256 个字节, 主要用于存放所要执行程序的有关信息, 同时也提供了程序和操作系统的接口。操作系统在程序段前缀的开始处(偏移地址 0000H)安排了一条 INT 20H 软中断指令。INT 20H 中断服务程序由 PC-DOS 提供, 执行该服务程序后, 控制就转移到 DOS, 即返回到 DOS 管理的状态。因此, 用户在组织程序时, 必须使程序执行完后, 能去执行存放于 PSP 开始处的 INT 20H 指令, 这样便返回到 DOS, 否则就无法继续键入命令和程序。

PC-DOS 在建立了程序段前缀区 PSP 之后, 就将要执行的程序从磁盘装入内存。在定位程序时, DOS 将代码段置于 PSP 下方, 代码段之后是数据段, 最后放置堆栈段。内存分配好之后, DOS 就设置段寄存器 DS 和 ES 的值, 以使它们指向 PSP 的开始处, 即 INT 20H 的存放地址, 同时将 CS 设置为 PSP 后面代码段的段基值, IP 设置为指向代码段中第一条要执行的指令位置, 把 SS 设置为指向堆栈的段基值, 让 SP 指向堆栈段的栈底(取决于堆栈的长度), 然后系统开始执行用户程序。

为了保证用户程序执行完后, 能回到 DOS, 可使用如下两种方法:

1. 标准方法

首先将用户程序的主程序定义成一个 FAR 过程, 其最后一条指令为 RET, 然后在代码段的主程序(即 FAR 过程)的开始部分, 用如下三条指令将 PSP 中 INT 20H 指令的段基值及偏移地址压入堆栈:

```
PUSH    DS          ; 保护 PSP 段地址
MOV     AX, 0        ; 保护偏移地址
PUSH    AX
```

这样, 当程序执行到主程序的最后一条指令 RET 时, 由于该过程具有 FAR 属性, 故存在堆栈内的两个字就分别弹出到 CS 和 IP, 便执行 INT 20H 指令, 使控制返回到 DOS 状态。

此外, 由于开始执行用户程序时, DS 并不设置在用户的数据段的起始处, ES 也同样不设置在用户的附加段起始处, 因而在主程序开始处, 继上述三条指令之后, 应该重新装填 DS 和 ES 的值。

2. 非标准方法

也可在用户的程序中不定义过程段, 只在代码段结束之前(即 CODE ENDS 之前), 增加两条语句:

```
MOV     AH, 4CH
INT     21H
```

则程序执行完后, 也会自动返回 DOS 状态。

4.5 基本结构程序设计

1. 汇编语言程序设计的基本过程

汇编语言程序设计的基本过程可分为以下几个步骤：

1) 分析问题，明确要求

分析问题就是深入实际，对所要解决的问题进行全面了解和分析。一个实际问题往往是比较复杂的，在深入分析的基础上，要善于抓住主要矛盾，剔除次要矛盾，抽取问题的本质。

明确要求就是明确用户的要求，依据给出的条件和数据，对需要进行哪些处理、输出什么样的结果，进行可行性分析。

2) 建立数学模型

在分析问题和明确要求的基础上，要建立数学模型，将一个物理过程或工作状态用数学形式表达出来。

3) 确定算法和处理方案

数学模型建立后，必须研究和确定算法。所谓算法，是指解决某些问题的计算方法，不同类的问题有不同的计算方法。根据问题的特点，对计算方法进行优化。若没有现成方法可用，必须通过实践摸索，并总结出算法思想和规律性。

4) 画流程图

流程图是程序算法的图形描述，它以图形的方式把解决问题的先后顺序和程序的逻辑结构直观地、形象地描述出来，使解题的思路清晰，有利于理解、阅读和编制程序，还有利于调试、修改程序和减少错误等。

5) 编制程序

在编制程序时，应当先分配好存储空间和工作单元及 CPU 内部的寄存器，然后根据流程图和确定的算法逐条语句编写程序。

6) 上机调试

程序编好后，必须上机调试，特别是对于复杂的问题，往往要分解成若干个子问题，分别由几个人编写，而形成若干个程序模块，把它们组装在一起，才能形成总体程序。一般来说，总会有这样或那样的问题或错误，这些问题和错误在调试程序时通常都可以发现，然后进行修改，再调试，再修改，直到所有的问题解决为止。

7) 试运行和分析结果

试运行和分析结果是为了检验程序是否达到了设计要求，是否满足用户提出的需求，所确定的设计方案是否可行。若没有达到设计要求，不满足用户的需求，就必须从分析问题开始检查修正原有的设计方案，直到符合设计要求和满足用户需求为止。

8) 整理资料，投入运行

在试运行满足要求之后，应当系统地整理材料，有关资料要及时提交用户，以便正常投入运行。

2. 程序结构化的概念

在计算机发展的初期，由于计算机硬件贵、内存容量小和运算速度低，因此，要求程

序运行时间尽可能短, 占用内存尽可能少。就是说, 当时衡量程序质量好坏的主要标准是占用内存的大小和运算时间的长短。为了达到这一目的, 人们挖空心思寻找技巧, 人们很难理解和消化这种程序, 造成人力和时间的严重浪费, 而且程序实际没有统一的规范, 弊端大。

随着计算机的迅速发展, 特别是大规模和超大规模集成电路技术的兴起, 计算机硬件价格大大下降, 内存容量不断扩大, 运算速度大幅度提高。因此, 运行时间短和节省内存已不是主要矛盾, 更重要的是使程序具有良好的结构、清晰的层次、容易阅读和理解、容易修改和查错, 这就对以前的传统设计方法提出了挑战, 从而产生了结构化程序设计方法, 又称为系统化程序设计方法。在这个方法中, 首先把一个大型程序分解成几个主要的模块; 最高层次部分说明这些模块之间的关系以及它们的功能, 也就是说, 最高层次部分是对整个程序的概述。而每个主要的模块再分解成几个较小的模块, 然后继续分解成更小的模块, 直到每个模块内的操作步骤都很清楚、很容易理解为止, 最后把一个模块或几个模块分配给每个程序设计师编写。

与上述“自上而下”的程序设计方法相对应, 还有“自下而上”的程序设计方法, 在这个方法中, 每个程序设计师开始编写低层次的模块并且期望这些模块最后能组合在一起。如果组合完成了, 那么和“自上而下”的设计方法产生相同结果。现在许多程序设计都是混合使用这两种方法, 由上而下开始设计, 然后由最小的模块开始编写, 测试连接, 再一直往上做, 直到最终完成为止。

按结构化程序设计方法编写出的程序, 具有风格优美、结构优良、层次清晰、容易阅读和理解、容易修改和验证等优点。

程序结构化的首要问题是程序的模块化。一个大型程序划分成若干个功能模块, 其中有一个模块称为主模块, 由它选择和调用其他各个功能模块, 被调用的各个模块称为子模块。这种将一个复杂的大型程序按其功能划分为若干相对独立的模块进行程序设计的方法称为程序的模块化。在汇编语言程序设计中, 程序模块化是通过子程序(或过程)的手段来实现的。

程序的基本结构包括顺序结构、分支结构、循环结构, 下面分别阐述。

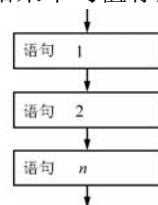
4.5.1 顺序程序设计

顺序结构是按语句实现的先后次序执行一系列的操作。顺序结构的程序一般是简单程序。这种程序也叫直线程序。

【例 4.2】 求两个数的平均值。这两个数分别存放在 x 单元和 y 单元中, 结果平均值存放在 z 单元中。编制完整的程序如下:

```
DATA1 SEGMENT
        X DB 45H
        Y DB 64H
        Z DB ?
DATA1 ENDS
CODE1 SEGMENT
        MAIN PROC FAR
ASSUME CS: CODE1, DS: DATA1
START:  PUSH DS

        MOV AX, 0
        PUSH AX
        MOV AX, DATA1 ; 初始化数据段
        MOV DS, AX
        MOV AL, x      ; 第一个数送入 AL
        ADD AL, y       ; 两个数相加, 结果送 AL
```



```

MOV AH, 00H
ADC AH, 00H    ; 带进位加法
MOV BL, 02H    ; 除数 2 送 BL
DIV BL         ; AX 除以 BL 的内容, 商 ← AL, 余数 ← AH
MOV Z, AL      ; 结果送入 Z 单元
RET
MAIN ENDP
CODE1 ENDS
END START

```

【例 4.3】 在内存中自 tab 开始的 16 个单元连续存放着 0 至 15 的平方值(平方表), 任给一个数 x ($0 \leq x \leq 15$) 在 x 单元中, 查表求 x 的平方值, 结果在 y 单元。

根据给出的平方表, 分析表的存放规律, 可知表的起始地址与数 x 之和, 正是 x 的平方值所在单元的地址, 由此编制程序如下:

```

DATA SEGMENT
Tab DB 0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225
X DB 13
DATA ENDS
CODE SEGMENT
ASSUME CS: CODE, DS: DATA
START: MOV AX, DATA
      MOV DS, AX
      LEA BX, tab
      MOV AH, 0
      MOV AL, x
      ADD BX, AX
      MOV AL, [BX]
      MOV y, AL
      MOV AH, 4CH
      INT 21H
CODE ENDS
END START

```

4.5.2 分支程序设计

分支结构根据不同情况做出判断和选择, 以便执行不同的程序段。分支的意思是在两个不同的操作中选择其中的一个, 如图 4.1、图 4.2、图 4.3 所示。

在图 4.2 所示的两个路径中, 有一个是不执行任何操作的。图 4.3 是多分支结构, 它是在许多不同的操作中选择其中的一个, 究竟选定哪一个操作是由测试表达式来决定的。图 4.2 和图 4.3 只不过是图 4.1 的变形而已。

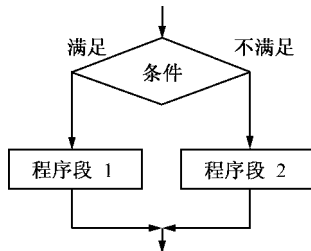


图 4.1 选择分支

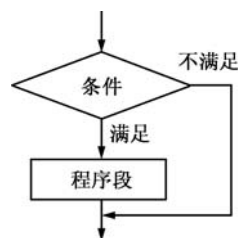


图 4.2 简单分支

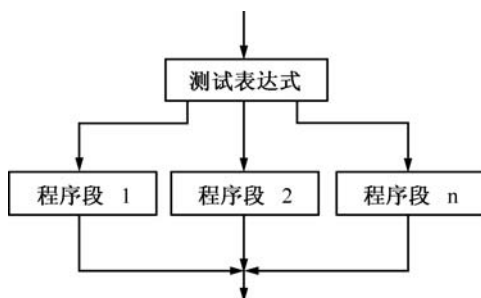


图 4.3 多分支图

在很多实际问题中，都是根据不同的情况进行不同的处理。这种思想体现在程序设计中，就是根据不同条件而跳到不同的程序段去执行，这就构成了分支程序。在汇编语言程序设计中，跳转是通过条件转移指令来实现的。

在分支程序中，不论是两分支结构还是多分支结构，它们都有一个共同特点：运行方向是向前的，在某种确定条件下，只能执行两个或多个分支中的一个分支。

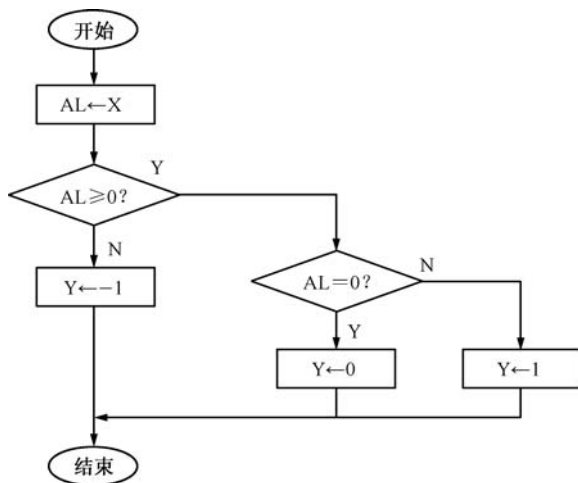


图 4.4 实现符号函数程序的流程图

【例 4.4】 给定以下符号函数：

$$Y = \begin{cases} 1, & X > 0 \\ 0, & X = 0 \\ -1, & X < 0 \end{cases}$$

这是一个简单的分支结构，任意给定 x 值，假定为 -25 ，且存放在 x 单元，函数值 y 存放在 y 单元，根据 x 确定 y 。

```

DATA SEGMENT
    X DB -25
    y DB ?
DATA ENDS
CODE SEGMENT
    MAIN PROC FAR
  
```

```

ASSUME CS: CODE, DS: DATA
ST: PUSH DS
    MOV AX, 0
    PUSH AX
    MOV AX, DATA
    MOV DS, AX
    MOV AL, x
    CMP AL, 0
    JNG LP1
    MOV AL, 0FFH
    MOV y, AL
    RET
LP1: JE LP2
    MOV AL, 1
    MOV y, AL
    RET
LP2: MOV AL, 0
    MOV y, AL
    RET
MAIN ENDP
CODE ENDS
END ST

```

【例 4.5】 设有首地址为 `arty` 的字数组，按升序排好，数组长度 $n=15$ ，且数据段与附加段占同一段，在该数组中查找 `num`，如果找到，从数组中删除；如果找不到，将它插入正确位置，且变化后的数组长度在 `DX` 中。

根据题意程序编写如下：

```

DATA SEGMENT
    DW ?
    n DW 15
    num DW 83
    arty DW 5, 10, 17, 21, 28, 32, 41, 50, 56, 67, 72, 88, 95, 125, 150
DATA ENDS
CODE SEGMENT
    MAIN PROC FAR
        ASSUME CS: CODE, DS: DATA, ES
START:  PUSH DS
        SUB AX, AX
        PUSH AX
        PUSH ES
        MOV AX, DATA
        MOV DS, AX
        PUSH DS
        POP ES
        MOV AX, num                ; 待查找的数放入 AX
        MOV DX, n                  ; 初始化 DX
        MOV CX, n                  ; 设置计数器 CX
        MOV DI, OFFSET arty       ; arty 的有效地址放入 DI
        CLD                        ; 建立方向标志
        REPNE SCASW                ; 用重复串扫描指令进行查找

```

```

        JE  DEL
        DEC  DX
        MOV  SI, DX
        ADD  SI, DX
WE3:    CMP  AX, arty[SI]
        JL   WE1
        MOV  arty [SI+2], AX      ; 此段程序功能是: 若没有查到
        JMP  WE2                  ; 则将此数插入正确位置
WE1:    MOV  BX, arty [SI]
        MOV  arty [SI+2], BX
        SUB  SI, 2
        JMP  WE3
WE2:    ADD  DX, 2                  ; 修改数组长度
        JMP  FAN
        DEL: JCXZ NEXT
LP:     MOV  BX, [DI]              ; 此程序段功能是: 若查找到
        MOV  [DI-2], BX          ; 则从数组中删除该数
        ADD  DI, 2
        LOOP LP
NEXT:   DEC  DX                  ; 修改数组长度
FAN:    POP  ES
        RET
MAIN    ENDP
CODE    ENDS
END     START

```

4.5.3 循环程序设计

循环结构是重复做一系列的操作，直到某个条件出现为止。如图 4.5 和图 4.6 所示。

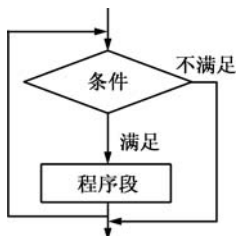


图 4.5 WHILE-DO 型循环图

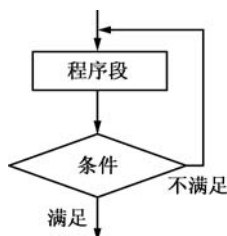


图 4.6 REPEAT-UNTIL 型循环图

图 4.5 是一种重复型结构，它表示如果某一条件一直成立，则重复做同一个操作或一系列操作，直到条件不成立时为止。它是先检查条件，再去执行操作。图 4.6 这种循环结构，先执行操作，再去检查条件成立与否，因此，这种结构至少要执行这些操作一次。图 4.6 循环结构是由图 4.5 的循环结构演变而来，任何 REPEAT-UNTIL 型循环都可以用 WHILE-DO 型循环表示。

循环结构和分支结构一样，是应用极为广泛的基本程序结构之一。

1. 循环程序分四部分

(1) 设置循环的初值。如设置循环次数的计数器，为使循环体正常工作而建立的初始

状态等。

(2) 循环体。循环体是循环工作的主体部分，是为完成某种特定功能而设计的程序段。

(3) 修改部分。为保证每次循环时，相关信息(如计数器的值、操作数地址等)能发生有规律的变化，为下次循环作好准备。

(4) 循环控制部分。循环控制是循环程序设计的关键。每个循环程序必须选择一个恰当的循环控制条件来控制循环的运行和结束。如果循环不能工作运行，则不能完成特定功能；如果循环不能结束，则将陷入“死循环”。因此，合理地选择循环条件就成为循环程序设计的关键问题。有时循环次数是已知的，可使用循环次数计数器来控制；有时循环次数是未知的，则应根据具体情况设置控制循环结束的条件。

2. 循环程序设计

控制循环是循环程序设计的关键问题。控制循环的方法很多，常用的有：

- (1) 用计数器控制(循环次数已知)；
- (2) 按条件控制(循环次数未知)；
- (3) 用开关变量控制(分支规律已知，计数次数或循环条件已知)；
- (4) 用逻辑尺控制。

【例 4.6】 从 xx 单元开始的 30 个连续单元中存放有 30 个无符号数，从中找出最大者送入 yy 单元。

根据题意，把第一个数先送入 AL，将 AL 中的数与后面的 29 个数逐个比较，如果 AL 中的数较小，则两数交换位置，如果 AL 中的数大于或等于相比较的数，则两数位置不变，在比较过程中，AL 中始终保持较大的数，比较 29 次，则最大者必在 AL 中，最后把 AL 中的数送入 yy 单元。

程序编写如下，流程图如图 4.7 所示。

```
DATA    SEGMENT
    xx   DB  73, 59, 61, 45, 81, 107, 37, 25, 14
          DB  64, 3, 17, 9, 23, 55, 97, 115, 78, 121
          DB  67, 215, 137, 99, 241, 36, 58, 87
    DB  100, 74, 62
    yy   DB?
DATA    ENDS
CODE    SEGMENT
    MAIN PROC FAR
        ASSUME CS: CODE, DS: DATA, ES
    MAIN PROC FAR
    START: PUSH DS
            MOV AX, 0
            PUSH AX
            MOV AX, DATA
            MOV DS, AX
            MOV AL, xx
            MOV BX, OFFSET xx
            MOV CX, 29
    LOOP1: INC BX
            CMP AL, [BX]
```

```

        JAE  LOOP2
        XCHG AL, [BX]
LOOP2:  DEC  CX
        JNZ  LOOP1
        MOV  YY, AL
RET
MAIN  ENDP
CODE  ENDS

        END  START

```

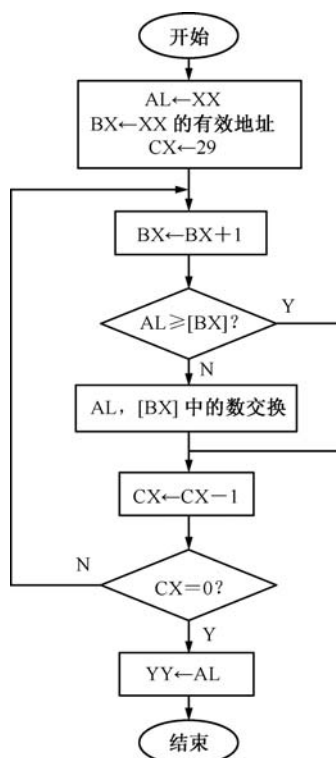


图 4.7 从一批数中求最大者流程图

【例 4.7】 从自然数 1 开始累加，直到累加和大于 1000 为止，统计被累加的自然数的个数，并把统计的个数送入 *n* 单元，把累加和送入 *sum* 单元。

根据题意，被累加的自然数的个数事先是未知的，也就是说，循环的次数是未知的，因此不能用计数器方法控制循环。但题目中给定一个重要条件，即累加和大于 1000 则停止累加，因此，可以根据这一条件控制循环。用 *BX* 寄存器统计自然数的个数，用 *AX* 寄存器存放累加和，用 *BX* 寄存器存放每次取得的自然数。程序的流程图如图 4.8 所示。

程序编写如下：

```

DATA    SEGMENT
        n  DB  ?
        Sum DW ?
DATA    ENDS
STACK   SEGMENT PARA STACK 'stack'
        DW  200DUP(?)

```

```

STACK  ENDS
CODE   SEGMENT
        MAIN  PROC  FAR
        ASSUME CX: CODE, DS: DATA, SS: STACK
START:  PUSH  DS
        MOV  AX, 0
        PUSH AX
        MOV  AX, DATA
        MOV  DS, AX
MOV  AX, 0
        MOV  BX, 0
LP:     INC  BX
        ADD  AX, BX
        CMP  AX, 1000
        JBE  LP
        MOV  n, BX
        MOV  sum, AX
        RET
MAIN    ENDP
CODE    ENDS
        END    START

```

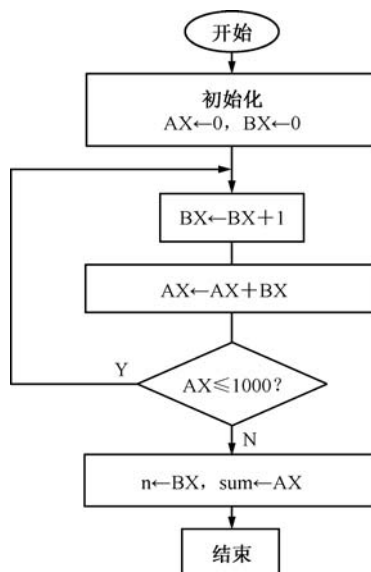


图 4.8 程序流程

【例 4.8】 设有两个函数

$$y = x + 5$$

$$f = t - 27$$

根据某种控制过程的需要，函数 y 要计算 5 次，函数 f 要计算 3 次，且计算的次序是 y 计算 2 次， f 计算 1 次， y 计算 1 次， f 计算 1 次， y 计算 1 次， f 计算 1 次，最后 y 再计算 1 次。 x 的初值为 15，每计算 y 一次，自变量 x 增值 2； t 的初值为 375，每计算一次 f ，值

下降 27。第一个函数计算出的 5 个函数值依次存放在由 y 单元开始的连续 5 个单元中，第二个函数计算出的 3 个函数值依次存放在由 f 单元开始的连续 3 个字单元中。试设计程序。

根据题意，由计算 y 和 f 的顺序，我们可以设计一个逻辑尺：11010101，将其存放在 DL 中，其中 1 表示计算 y，0 表示计算 f。然后利用算术左移指令 SAL，根据有无进位，决定是计算 y 还是计算 f，若每次左移 1 位，则移位次数共 8 次。程序的流程图如图 4.9 所示。

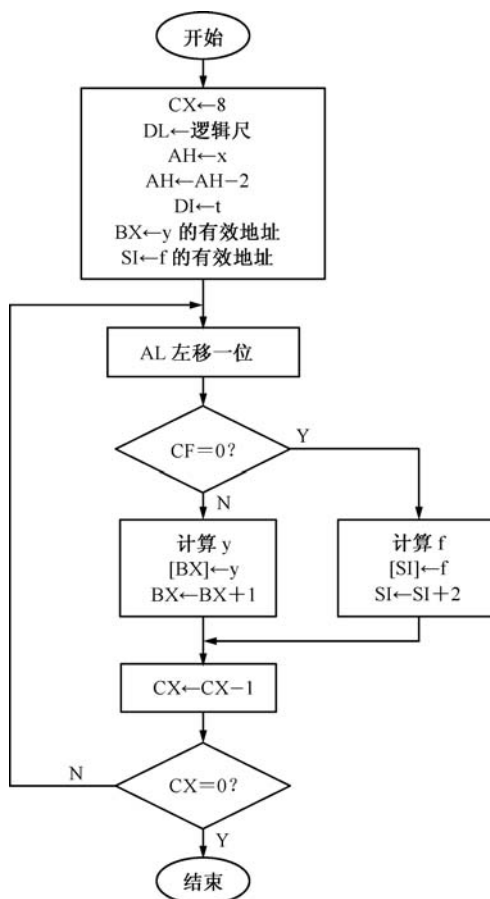


图 4.9 程序流程

程序编写如下：

```

DATA SEGMENT
    x DB 15
    y DB 5 DUP(?)
    t DW 375
    f DW 3 DUP(?)
DATA ENDS
STACK SEGMENT PARA STACK 'stack'
    DW 200 DUP(?)
STACK ENDS
CODE SEGMENT
ASSUME CS: CODE, DS: DATA, SS: STACK
  
```

```

MAIN PROC FAR
START: PUSH DS
      MOV AX, 0
      PUSH AX
      MOV AX, DATA
      MOV DS, AX
      MOV CX, 8
      MOV DL, 11010101B
      MOV AH, x
      SUB AH, 2
      MOV DI, t
      LEA BX, y
      LEA SI, f
WW: SAL DL, 1
JNC LE1
ADD AH, 2
MOV AL, AH
ADD AL, 5
MOV [BX], AL
INC BX
JMP LE2
LE1: SUB DI, 27
      MOV [SI], DI
      ADD SI, 2
LE2: LOOP WW
      RET
      MAIN ENDP
CODE ENDS
      END START

```

3. 多重循环程序设计

多重循环又称循环嵌套。在使用多重循环时，必须注意以下几点：

- (1) 内循环必须完整地包含在外循环内，内外循环不能相互交叉。
- (2) 内循环在外循环中的位置可根据需要任意设置，在分析程序流程时，要避免出现混乱。
- (3) 内循环可以嵌套在外循环中，也可以几个内循环并列存在。可以从内循环直接跳到外循环；但不能从外循环直接跳到内循环。
- (4) 防止出现“死循环”。无论是外循环，还是内循环，千万不要使循环返回到初始部分，否则会出现“死循环”，这一点应当特别注意。
- (5) 每次通过外循环再次进入内循环时，初始条件必须重新设置。

【例 4.9】 设有 4 个学生参加 5 门课程的考试，试计算每个学生的平均成绩和每门课的平均成绩。

根据题意，我们把 4 名学生的成绩依次存放在一个数组(首址为 `chg`)中，把每个学生的平均成绩和每门课的平均成绩保存在两个数组(首址分别为 `av` 和 `me`)中，流程图如图 4.10 所示。编写程序如下：

```

DATA SEGMENT
Chg DW 85, 92, 76, 88, 90
    DW 72, 81, 68, 84, 78

```

```
        DW 90, 86, 94, 100, 80
        DW 75, 62, 80, 79, 58
av DW 4 DUP(?)
me DW 5 DUP(?)
DATA    ENDS
CODE    SEGMENT
ASSUME  CS: CODE, DS: DATA
MAIN    PROC FAR
        START: PUSH DS
                MOV AX, 0
                PUSH AX
                MOV AX, DATA
                MOV DS, AX
                MOV CX, 4
                LEA BX, chg
LEA SI, av
L11: PUSH CX
        MOV AX, 0
        MOV CX, 5
L22: ADD BX, 2
        ADD AX, [BX]
        LOOP L22
        MOV DL, 5
        DIV DL
        MOV [SI], AL
        ADD SI, 2
        POP CX
        LOOP L11
        LEA BX, chg
        LEA SI, me
        MOV DI, BX
        MOV CX, 5
L33: MOV BX, DI
        SUB BX, 10
        PUSH CX
        MOV AX, 0
        MOV CX, 4
L44: ADD BX, 10
        ADD AX, [BX]
        LOOP L44
        MOV DL, 4
        DIV DL
        MOV [SI], AL
        ADD SI, 2
        ADD DI, 2
        POP CX
        LOOP L33
        RET
MAIN    ENDP
CODE    ENDS
        END    START
```

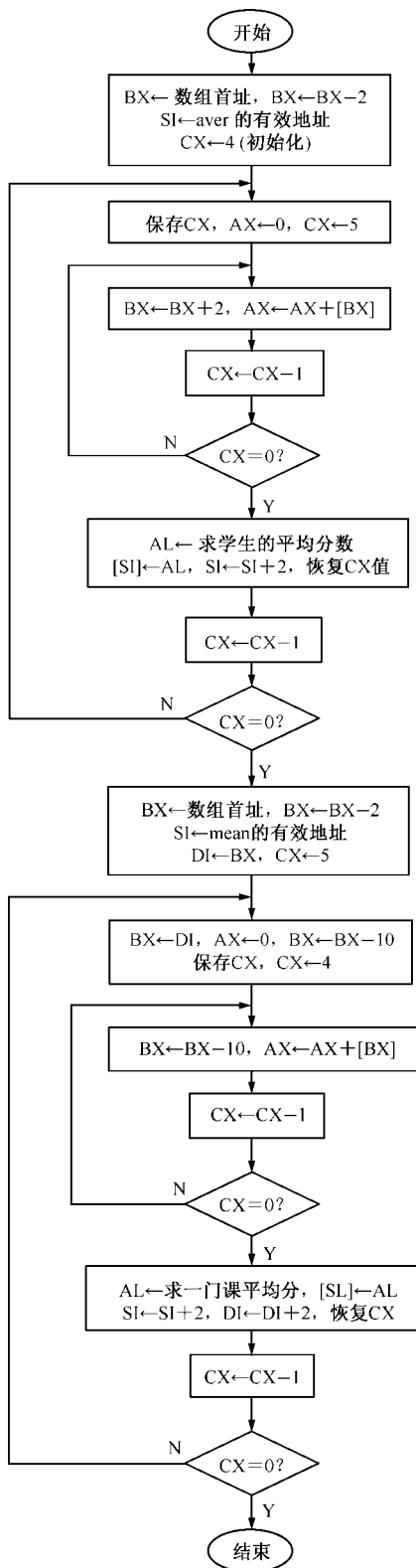


图 4.10 程序流程

4.5.4 子程序设计

1. 子程序概念

如果在一个程序中的多处需要用到同一段程序，或者说在一个程序中，需要多次执行某一连串的指令时，那么可以把这一连串的指令抽取出来，写成一个相对独立的程序段，每当想要执行这段程序或这一连串的指令时，就调用这段程序，执行完这段程序后，再返回原来调用它的程序。这样每次执行这段程序时，就不必重写这一连串的指令了。

这样的程序段称为子程序或过程。而调用子程序的程序称为主程序或调用程序。

此外，使用子程序还有另一个主要理由。在本节一开始曾讲到“由上而下”的程序设计方法，在这个方法中，把整个问题划分成好几个模块，把每个模块再划分成更小的模块，直到每个模块的算法都描述得很清楚为止。能把一个大的问题划分成许多小的问题，而这些小的问题就构成了一个个相对独立的模块，它们可以单独编程、调试和纠错。这些独立的模块通常编写成子程序，而且可以被层次图中最高层的主程序调用。子程序结构是模块化程序设计的重要工具和手段。

2. 子程序的定义

子程序是用过程定义伪指令 **PROC** 和 **ENDP** 来定义的。有关伪指令 **PROC** 和 **ENDP** 已在前面介绍过了，这里只对其类型属性作一些说明，因为它是一个过程能否正确执行的保证。过程类型属性的确定原则：

- (1) 调用程序和过程若在同一代码段中，则使用 **NEAR** 属性。
- (2) 调用程序和过程若不在同一代码段中，则使用 **FAR** 属性。
- (3) 主程序应定义为 **FAR** 属性。因为把程序的主过程看作 DOS 调用的一个子程序，而 DOS 对主过程的调用和返回都是 **FAR** 属性。

另外，过程定义允许嵌套，即在一个过程定义中允许包含多个过程定义。

【例 4.10】 调用程序和子程序在同一代码段中。

```
CODE SEGMENT
...
MAIN PROC FAR
...
CALL PPP1
...
RET
PPP1 PROC NEAR
...
CALL PPP2
...
RET
PPP2 PROC NEAR
...
RET
PPP2 ENDP
PPP1 ENDP
MAIN ENDP
CODE ENDS
```

在本例中过程定义相当于三层嵌套,即主过程 MAIN 嵌套子过程 PPP1,而子过程 PPP1 又嵌套子过程 PPP2。因为主过程和两个子过程均在同一代码段,因此,除主过程使用 FAR 属性外,其他两个子过程均使用 NEAR 属性。

【例 4.11】 调用程序和子程序不在同一代码段。

```
CODE1  SEGMENT
...
RRR  PROC  FAR
...
                RET
                RRR  ENDP
CODE1  ENDS
CODE2  SEGMENT
...
CALL  RRR
...
CODE2  ENDS
```

本例中,因为子程序 RRR 和调用程序不在同一代码段,因此子程序 RRR 应定义为 FAR 属性,这样调用指令 CALL 和返回指令 RET 都是 FAR 属性的。

应当指出,在一个过程中可以有一个以上的 RET 指令,这完全是根据需要而设置的。

3. 子程序的调用和返回

子程序的调用和返回由 CALL 和 RET 指令完成,子程序的正确调用和正确返回是正确执行子程序的保证。为了使子程序正确地执行,有两点应特别注意:

(1) 正确选择过程的属性。

(2) 正确使用堆栈,因为在调用程序中执行 CALL 指令时,将把断点地址压入堆栈。这个地址正是由子程序返回到调用程序的地址。当在子程序中执行 RET 指令时,便把这个返回地址由堆栈弹出(称为恢复断点),返回调用程序自此继续往下执行。若在子程序中不能正确地使用堆栈,而造成执行 RET 前堆栈指针 SP 并未指向进入子程序时的返回地址,则必然会导致运行出错。因此在子程序中使用堆栈要特别注意。

4. 寄存器的保护与恢复

在程序设计中,调用程序(或主程序)与子程序通常是独立编写的,因此它们所使用的一些寄存器和存储单元经常会发生冲突。如果调用程序在调用子程序以前的某些寄存器或存储单元的内容在从子程序返回到调用程序后还要使用,而子程序又恰好使用了这些寄存器或存储单元,则这些寄存器或存储单元的原有内容遭到了破坏,那就会使程序运行出错,为防止这种错误的发生,在进入子程序之前或之后,应该把子程序所使用的寄存器或存储单元的内容保存在堆栈中,而退出子程序之前再恢复原有的内容。在主、子程序间传送参数的寄存器不需要保护。

寄存器的保护有两种方法:

(1) 把需要保护和恢复的寄存器的内容,在调用程序中压入堆栈和弹出堆栈。这种方法有一个好处,就是在每次调用子程序时,只要把你所关心的寄存器压入堆栈,返回后弹出即可。但缺点是,在调用程序中,使用压入和弹出堆栈的功能会使调用程序不容易理解,

而且可能在调用程序其他地方使用某个寄存器时，却忘了把它压入堆栈内。

(2) 进入子程序后，首先把需要保护的寄存器的内容压入堆栈，而在返回调用程序前，再恢复这些寄存器的内容。这种方法是我们所推荐的。这种方法的好处是：首先，在调用程序中的任何地方都可调用子程序，而不会破坏任何寄存器的原有内容。其次，这种方法只需要写一次压入和弹出堆栈群即可。例如：

```
DUBT PROC NEAR
        PUSH AX
        PUSH BX
        PUSH CX
        PUSH DX
        RET
DUBT ENDP
```

注意：堆栈的工作方式是后进先出。

5. 调用程序与子程序之间的参数传递

调用程序在调用子程序时，往往需要向子程序传递一些参数；同样的子程序运行后，也经常要把一些结果参数传回给调用程序。调用程序与子程序之间的这种信息传递称为参数传递。

参数传递有 3 种主要的方式：

1) 通过寄存器传递参数

这种方式适合于传递参数较少的一些简单程序。

2) 通过地址表传递参数地址

这种方式适合于参数较多的情况，但要求事先建立地址表，通过地址表传递参数的地址，地址表可以在内存中或外设端口中。

3) 通过堆栈传递参数

为了利用堆栈传递参数，必须在主程序中任何调用子程序之前的地方，把这些参数压入堆栈，然后利用在子程序中的指令从堆栈弹出而取得参数。同样，要从子程序传递回调用程序的参数也被压入堆栈内，然后由主程序中的指令把这些参数从堆栈中取出。

利用堆栈传递参数有两个非常重要的问题：

(1) 当使用堆栈来传递参数时，要注意堆栈溢出(stack-overflow)。每当用堆栈传送参数时，应当非常清楚，已经把什么东西压入了堆栈内及在子程序中每个地方的堆栈指针是指向哪里，弄不好，会引起混乱和造成堆栈溢出。所谓堆栈溢出是指堆栈超出了为它开辟的存储空间。

(2) 8086/8088 有四种形式的 RET 指令，一般的近程 RET 指令能把返回地址由堆栈弹入到 IP，同时把堆栈指针加 2；一般的远程 RET 指令能由堆栈把返回的 IP 及 CS 值弹入到 IP 及 CS，同时把堆栈指针加 4，其他二种 RET 指令形式分别执行相同的功能。但是它们把一个在指令中指定的数字加入堆栈指针。如近程 RET 6 指令会从堆栈弹出一个字的内容到 IP 同时把堆栈指针加 2，然后再加 6 到堆栈指针，这是一种快速方式，可以让堆栈指针往下(地址增大方向)跳过一些参数。

6. 子程序的嵌套

一个子程序可以作为调用程序去调用别的子程序，这种结构称为子程序的嵌套。只要有足够的堆栈空间，嵌套的层次是不限的，其嵌套的层数称为嵌套深度。当调用程序去调用子程序时，将产生断点，而子程序执行完后，返回到调用程序的断点处，使调用程序继续往下执行。因此，对于嵌套结构，断点的个数等于嵌套的深度。如图 4.11 所示。

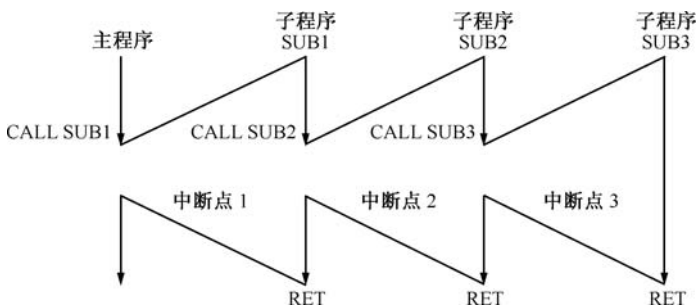


图 4.11 子程序嵌套示意图

图 4.11 所示的嵌套结构其嵌套深度为 3，所以断点个数也为 3。由此可见，嵌套结构是层次结构。

4.6 DOS/BIOS 功能调用

在汇编语程序设计中，经常要用到 ROM-BIOS 的一些软中断和系统功能调用来扩充汇编语言的功能。

1. 概述

(1) ROM-BIOS(基本 I/O 系统)是固化在 ROM 中的一组 I/O 设备驱动程序，它为系统各主要部件提供设备级的控制，还为汇编语言程序设计者提供了字符 I/O 操作。程序员在使用 ROM-BIOS 的功能模块时，可以不关心硬件 I/O 接口的特性，仅使用指令系统的软中断指令(INT n)，这称为中断调用。

例如，将一个 ASCII 字符显示在屏幕的当前所在位置，可使用 ROM-BIOS 的中断类型号 10H，功能号为 0EH。程序段如下：

```

MOV AL, "?"      ; 要显示的字符送入 AL
MOV AH, 0EH      ; 功能号送入 AH
INT 10H          ; 调用 10H 软中断

```

(2) 系统功能调用是微机的磁盘操作系统 DOS 为用户提供的的一组例行子程序，因而又称为 DOS 系统功能的调用。这些子程序可分为以下三个主要方面：

① 磁盘的读/写及控制管理。

② 内存管理。

③ 基本输入/输出管理(如键盘、打印机、显示器、磁带管理等)，另外还有时间、日期等子程序。

2. 系统功能调用方法

为了方便, 已将所有子程序顺序编号。例如, 基本输入/输出管理中的功能调用 1 号(键盘输入)、2 号(显示字符)、5 号(打印字符)、9 号(显示字符串)及 0A 号(接收键盘输入的字符串)。对于所有的功能调用, 使用时一般需要经过以下三个步骤:

- (1) 子程序的入口参数送相应的寄存器。
- (2) 子程序编号送 AH。
- (3) 发出中断请求: INT 21H(系统功能调用指令)。

例如, 显示一个字符串: "Goodmorning! "

```
JK DB 'Goodmorning!$ '
...
MOV DX, OFFSET JK
MOV AH, 9
INT 21H
```

有的子程序不需要入口参数, 这时(1)可以略去, 例如:

```
MOV AH, 4CH
INT 21H
```

子程序调用结束后, 一般都有出口参数, 这些出口参数常放在寄存器中, 通过出口参数, 用户可以知道调用的成功与否。

4.6.1 DOS 系统功能调用

系统功能调用可以分组如下:

- (1) 0~0CH: 传统的字符 I/O 管理, 包括键盘、显示器、打印机、异步通信口的管理。
- (2) 0DH~24H: 传统的文件管理, 包括复位磁盘, 选择磁盘, 打开文件, 关闭文件, 查找目录项, 删除文件, 顺序读、写文件, 建立文件, 重新命名文件, 查找驱动器分配表信息, 随机读、写文件, 查看文件长度等。
- (3) 25H~26H: 传统的非设备系统调用, 包括设置中断向量, 建立新程序段。
- (4) 27H~29H: 传统的文件管理, 包括随机块读写, 分析文件名。
- (5) 2AH~2EH: 传统的非设备系统调用, 读取、设置日期、时间等。
- (6) 2FH~38H: 扩充的系统调用组, 包括读取 DOS 版本号, 中止进程, 读取中断向量, 读取磁盘空闲空间等。
- (7) 39H~3BH: 目录组, 包括建立目录, 修改当前目录, 删除目录项。
- (8) 3CH~46H: 扩充的文件管理组, 包括建立、打开、关闭文件, 从文件或设备读取数据。在指定的目录里删除、移动、读、写文件, 读取、修改文件属性, 设备 I/O 控制, 即文件标记等。
- (9) 47H: 目录组, 取当前目录。
- (10) 48H~4BH: 扩充的内存管理组, 包括分配内存, 释放已分配的内存, 分配内存块, 装入或执行程序等。
- (11) 4CH~4FH: 扩充的系统管理组, 包括中止进程, 查询子进程的返回代码, 查找第一个相匹配的文件, 查找下一个相匹配的文件。

(12) 50H~53H: 扩充的系统调用, DOS 内部使用。

(13) 54~62H: 扩充的系统调用, 包括读取校验状态, 重新命名文件, 设置读取日期和时间。

从 39H 以后的文件管理系统调用是为了处理树形目录结构而提供的。下面选择其中一部分常用的系统功能调用分别加以介绍, 其余部分可参考有关的 DOS 资料。

1) 带显示键盘输入(1 号调用)

1 号系统功能调用等待从标准输入设备输入一个字符, 并送入寄存器 AL, 不需入口参数。例如:

```
MOV AH, 1
INT 21H
```

执行上述指令, 系统将扫描键盘, 等待有键按下。一旦有键按下, 就将键值(ASCII 码值)读入, 先检查是否是 Ctrl-Break, 若是, 则退出命令执行; 否则将键值送入 AL, 同时将这个字符显示在屏幕上。

2) 键盘输入但无显示(8 号调用)

8 号调用与 1 号调用类同, 只是不在屏幕上显示输入的字符。

3) 打印输出(5 号调用)

把 DL 中的字符输出到打印机。例如:

```
MOV DL, A
MOV AH, 5
INT 21H
```

4) 直接控制台输入/输出 (6 号调用)

6 号调用可以从标准输入设备输入字符, 也可以向屏幕上输出字符, 并且不检查 Ctrl-Break。

若 DL=FFH 时, 表示从键盘输入, 若标志 ZF=0, 表示 AL 中为键入的字符值; 若标志 ZF=1, 表示 AL 中不是键入的字符值, 即尚无键按下。

若 DL≠FFH 时, 表示向屏幕输出, DL 中为输出字符的 ASCII 码值。例如:

```
MOV DL, 0FFH
MOV AH, 6
INT 21H
```

为从键盘输入字符。

```
MOV DL, 24H
MOV AH, 6
INT 21H
```

将 24H 对应的字符\$输出, 即从屏幕上显示“\$”。

5) 直接从控制台输入但不显示(7 号调用)

等待从标准输入设备输入字符, 然后将其送入 AL, 同 6 号调用一样, 对字符不做检查。

6) 输出字符串 (9 号调用)

调用时, 要求 DS: DX 必须指向内存中一个以“\$”作为结束标志的字符串。字符串中每一个字符(不包括结束标志)都输出打印。例:

```

DATA SEGMENT
    BUF DB 'HOW DO YOU DO ? $'
DATA ENDS
CODE SEGMENT
...
    MOV AX, DATA
    MOV DS, AX
...
    MOV DX, OFFSET BUF
    MOV AH, 9
    INT 2Lh
...
CODE ENDS

```

执行本程序，屏幕上将显示：HOWDOYOU DO?

7) 输入字符串 (0AH 号调用)

从键盘接收字符串到内存输入缓冲区，缓冲区内第一个字节单元指出缓冲区能容纳字节个数，不能为 0，第二个字节保留用作填写输入字符的实际个数。从第三个字节开始存放从键盘接收字符串。如果实际输入的字符少于定义的字符数，缓冲区将空余的字节填零。如果实际输入的字符多于定义的字符数，将后来输入的字符丢掉，且响铃。调用时，要求 DS: DX 必须指向缓冲区。

```

DATA SEGMENT
    BUFDB 50                ; 缓冲区长度
    DB?                    ; 保留为填入实际输入的字符个数
    DB 50 DUP(?)           ; 定义 50 个字节存储空间
DATA ENDS
CODE SEGMENT
...
    MOV AX, DATA
    MOV DS, AX
...
    MOV DX, OFFSET BUF
    MOV AH, 10              ; 即 0AH 送 AH
    INT 21H
CODE ENDS

```

4.6.2 BIOS 功能调用

BIOS 的功能调用请参考本书的附录。

4.7 小 结

本章介绍了汇编语言程序设计的书写格式、组成内容、设计方法、程序设计实例，着重介绍顺序、分支、循环基本程序的设计，多数例题配有程序流程图，帮助读者更好地理解程序设计的过程，并简单地介绍了 DOS、BIOS 系统功能调用。

4.8 习 题

1. 写出完成下列要求的变量定义语句：

- (1) 为某缓冲区 BUFF 预留 240 个字节的存储空间。
- (2) 将字符串 ‘BYTE’、‘WORD’ 存放于某数据区。

2. 以图示说明下列语句实现内存分配和预置数据：

```
VAR1 DB '12', 12H, 3 DUP(0, FFH)
VAR2 DB 100 DUP(0, 2 DUP((1, 2), 0, 3)
VAR3 DB 'HELLO. COM'
VAR4 DW VAR+6
VAR5 DD VAR3
```

3. 下列语句中，哪些是无效的汇编语言指令？并指出无效指令中的错误。

```
MOV SP, AL
MOV WORD-OP[BX+4X3][SI], SP
MOV VAR1, VAR2
MOV CS, AX
MOV DS, BP
MOV SP, SS: DATA-WORD[SI][DI]
MOV AX, VAR1+VAR2
MOV AX, [BX-SI]
INC [BX]
MOV 25, [BX]
MOV [8-BX], 25
```

4. 若 x、y、z 已定义为字节变量，若 x 和 y 各存放一个 32 位的无符号数(存放顺序是低位字节在先)，试写出将 x 和 y 相加后结果存入 z 的程序段。

5. 若题 4 中 x、y 各存放一个 32 位的有符号数(低字节数在前)，试编写 x-y 且结果存入 z 的程序段，同时判断运算结果是否发生溢出，若不溢出使 DL 清零，否则(溢出)以-1 作为标志存入 DL 中。

6. 若数组 ARRAY 在数据段中已作以下定义：

```
ARRAY DW 100DUP(?)
```

试指出下列语句中各操作符的作用，指令执行后有关寄存器产生了什么变化。

```
MOV BX, OFFSET ARRAY
MOV CX, LENGTH ARRAY
MOV SI, 0
ADD SI, TYPE ARRAY
```

7. 若 ARRAY 和 MAX 都定义为字变量，并在 ARRAY 数组中存放了 10 个 16 位无符号数，试编写程序段，找出数组中最大数，并存入变量 MAX 中。

8. 若题 12 的 ARRAY 数组存放的是有符号数，试编写相应的程序段。

9. 试编写一程序段，完成两个以压缩型 BCD 码格式表示的 16 位十进制数的加法运算，相加的两数 x 和 y 可定义为字节变量，并假定高位在前，和数 SUM 也同样定义为字

节变量。

10. 编写一个统计 AX 中 1 的个数的程序段, 统计结果存放在 CL 中。
11. 编写一个判断 AX 中的数是正数、负数还是零的程序段。若 $(AX) < 0$, 以 -1 存入 CL; $(AX) = 0$, 以 0 存入 CL 否则若 $(AX) > 0$, 以 1 存入 CL。
12. 假定有一最大长度为 80 个字符的字符串已定义为字节变量 STRING, 试编写一程序段, 找出第一个空格的位置(00H 至 4FH 表示), 并存入 CL 中, 若该串中无空格符, 则以 -1 存入 CL 中。
13. 对题 12, 若该字符串以回车符结束, 试编写一段程序, 统计该串的实际长度(不包括回车符), 统计结果存入 CL 中。
14. 编写统计 AX 中 0、1 个数的程序。0 的个数存入 CH, 1 的个数存入 CL 中。
15. 编程将 AX 中的 4 位 BCD 码转换成二进制数, 转换结果存放在 AX 中。
16. 编程将 AX 中的二进制数转换成 4 位 BCD 码, 转换结果存放在 AX 中。
17. 编程将内存中以 AFG 为首址 ASCII 字符串, 以 \$ 为结束符, 转换成十进制数存放在以 ZC 为首址的单元。
18. 试将一个 2 位十进制数的压缩的 BCD 码转换成十六进制数, 并在屏幕上显示出来。
19. 设有两个无符号数 125 和 378, 其首地址为 x, 求它们的和, 将结果存放在 SUM 单元, 并将其和转换为十六进制数, 且在屏幕上显示出来。
20. 内存中从 FIRST 和 SECOND 开始的单元中分别存放着两个 4 位非压缩型的 BCD 码, 数据存放的规则是: 低位在低地址, 高位在高地址。试编程求这两个数的和, 并存放到从 THIRD 开始的内存单元中。

第 5 章 80X86 微处理器的外部功能

教学提示：本章主要介绍 8086/8088 CPU 的引脚功能、系统总线时序，80386 CPU 的引脚功能，80486 CPU 的引脚功能，理解微处理器的引脚功能是微型计算机系统中存储器接口和 I/O 接口的重要基础之一。

教学要求：通过本章的学习，学生应该掌握 8086/8088 CPU 的引脚功能、系统总线时序，80386 CPU 的引脚功能，80486 CPU 的引脚功能。

5.1 8086/8088 CPU 的引脚功能

8086 是 Intel 公司 1978 年的产品，它内部是 16 位数据线，外部也有 16 条数据引脚，为完全 16 位 CPU。由于当时所有的外设都是 8 位的，为了解决 16 条数据引脚的 CPU 与 8 条数据引脚的外设连接时的不便，Intel 公司随后又推出了内部是 16 位数据线，但外部是 8 位(有 8 条数据引脚)的 8088。8086 与 8088 在软件上完全相同，硬件上稍有区别。

8086/8088 CPU 具有 40 条引脚，采用双列直插式的封装形式。为了减少芯片上的引脚数目，8086/8088 CPU 都采用了分时复用的地址/数据总线。正是由于这种分时复用的方法，才使得 8086/8088 CPU 可用 40 条引脚实现 20 位地址、16 位数据(8 位数据)及许多控制信号和状态信号的传输。由于 8088 只传输 8 位数据，所以 8088 只有 8 个地址引脚兼作数据引脚，而 8086 有 16 个地址/数据复用引脚。这些引脚构成了 8086/8088 CPU 的外总线，它包括地址总线、数据总线和控制总线。8086/8088 CPU 通过这些总线和存储器、I/O 接口等部件组成不同规模的系统并相互交换信息。

5.1.1 8086 CPU 的引脚功能

8086 CPU 是 Intel 公司的第三代微处理器，时钟频率有三种，5MHz 的 8086，8MHz 的 8086-1、10MHz 的 8086-2。8086 CPU 的引脚如图 5.1 所示。

1. 8086 CPU 的工作模式

8086 有两种工作模式：最小模式和最大模式。

最小模式是指系统中只有一个微处理器(8086)。在这种系统中，8086 直接产生所有的总线控制信号，系统所需的外加总线控制逻辑部件最少。

最大模式是指系统中含有两个或多个微处理器，其中一个为主处理器 8086，其他的处理器称为协处理器，是协助主处理器工作的。在最大模式下工作时，控制信号是通过 8288 总线控制器提供的。因此，在不同方式下工作时，8086 的部分引脚(第 24~31 引脚)会具有不同的功能。

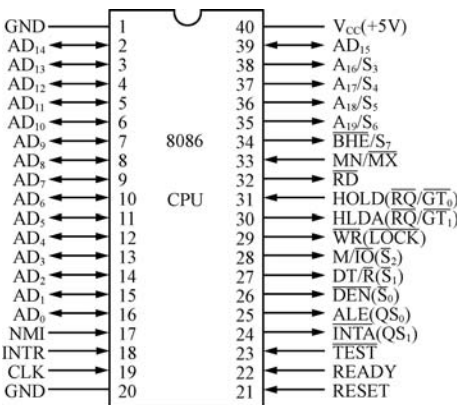


图 5.1 8086 CPU 的引脚

2. 8086 两种工作模式下共同引脚说明如下:

(1) AD₁₅~AD₀: 分时复用的地址/数据引脚, 具有双向、三态功能。在总线周期的第一个时钟周期 T₁ 用来输出要访问的存储单元或 I/O 端口的低 16 位地址 A₁₅~A₀。而在总线周期的其他时钟周期(T₂~T₃), 对于写周期来说是传送数据, 对于读周期来说, 则是处于悬浮状态(读周期的 T₄ 状态用来传送数据)。

(2) A₁₉/S₆~A₁₆/S₃: 分时复用的地址/状态线, 具有输出、三态功能。在总线周期的第一个时钟周期 T₁ 用来输出被访问存储器的 20 位物理地址的最高 4 位(A₁₉~A₁₆), 与 AD₁₅~AD₀ 一起构成访问存储器的 20 位物理地址。当 CPU 访问 I/O 端口时, A₁₉~A₁₆ 保持为“0”。而在其他时钟周期, 则用来输出状态信息。其中, S₆ 为 0, 用来指示 8086 当前正与总线相连; S₅ 用来指示中断允许标志位 IF 的当前值: S₅=1 表示 IF=1; S₅=0 表示 IF=0。S₄、S₃ 组合起来指示 CPU 当前正在使用哪个段寄存器, S₄、S₃ 的代码组合与对应的状态如表 5-1 所示。

表 5-1 S₄、S₃ 状态编码表

S ₄	S ₃	当前使用的段寄存器
L	L	当前正在使用 ES
L	H	当前正在使用 SS
H	L	当前正在使用 CS 或不使用任何段寄存器(I/O、INT)
H	H	当前正在使用 DS

(3) BHE/S₇(Bus High Enable/Status): 高 8 位数据总线允许/状态复用引脚, 三态输出, 低电平有效。BHE=0 表示数据总线高 8 位 AD₁₅~AD₈ 有效, 即 8086 使用了 16 根数据线。若 BHE=1, 表示数据总线高 8 位 AD₁₅~AD₈ 无效, 即 8086 使用了 8 根数据线(AD₇~AD₀)。读/写存储器或 I/O 端口以及中断响应时, BHE 用作选体信号, 与最低位地址线 A₀ 配合, 表示当前总线使用情况, 如表 5-2 所示。

S₇ 用来输出状态信息, 在 8086 芯片设计中未被赋予实际意义。

(4) RD (Read): 读信号, 三态、输出。当 RD 为低电平有效时, 表示当前 CPU 正在对

存储器或 I/O 端口进行读操作。具体是读存储器还是 I/O 端口，则要看 $\overline{M}/\overline{IO}$ 信号的状态，若其为高电平，表示读存储器；若其为低电平，表示读 I/O 端口。

表 5-2 $\overline{BHE}$ 和 A_0 编码对数据访问的影响

$\overline{BHE}$	A_0	总线使用情况
L	L	16 位数据总线上进行字传送
L	H	高 8 位数据总线上进行字节传送(访问奇地址存储单元)
H	L	低 8 位数据总线上进行字节传送(访问偶地址存储单元)
H	H	无效

(5) **READY**: 准备就绪信号，输入，高电平有效。**READY**=1 时，表示 CPU 访问的存储器或 I/O 端口已准备好传送数据，马上可以进行读/写操作。CPU 在总线周期的 T_3 状态检测 **READY** 信号，若发现其无效，CPU 自动插入一个或多个等待状态 T_w ，直到 **READY** 信号变为高电平为止。

(6) **$\overline{TEST}$** : 测试信号，输入，低电平有效。当 CPU 执行 **WAIT** 指令时，每隔 5 个时钟周期对 **$\overline{TEST}$** 进行一次测试，若测试到 **$\overline{TEST}$** 无效，则 CPU 处于空闲等待状态，直到 **$\overline{TEST}$** 变为有效的低电平，CPU 才结束等待状态继续执行后续指令。 **$\overline{TEST}$** 引脚用于多处理器系统中，实现 8086 与协处理器间的同步。

(7) **INTR**(Interrupt Request): 可屏蔽中断请求信号，输入，电平触发，高电平有效。CPU 在每个指令周期的最后一个 T 状态采样该信号。

(8) **NMI**(No-Maskable Interrupt): 不可屏蔽中断请求信号，输入，上升沿触发。

(9) **RESET**: 复位信号，输入，高电平有效。**RESET** 信号至少要保持 4 个时钟周期。CPU 检测到 **RESET** 为高电平信号后，停止进行操作，并将标志寄存器、段寄存器、指令指针 IP 和指令队列等复位到初始状态。CPU 复位后，从 **FFFF0H** 单元开始读取指令。

(10) **CLK**(Clock): 主时钟信号，输入。**CLK** 输入时钟为微处理器提供基本的定时脉冲，该脉冲最好具有 33% 的占空比。CPU 可使用的时钟频率随芯片型号不同而异，8086 为 5MHz，8086-1 为 10MHz，8086-2 为 8MHz，8088 为 4.77MHz。

(11) **$MN/\overline{MX}$** (Minimum/Maximum): 工作方式选择信号，输入。 **$MN/\overline{MX}$** =1，CPU 工作在最小模式下； **$MN/\overline{MX}$** =0，CPU 工作在最大模式下。

(12) 电源线 **Vcc** 和地线 **GND**: 8086 只需单一的(+5±0.1)V 电源，由 **Vcc** 端输入，**GND** 是接地端。

3. 最小模式下引脚

第 24~31 脚在最小模式下含义如下：

(1) **$\overline{M}/\overline{IO}$** (Memory/Input and Output): 存储器或 I/O 端口选择信号，三态输出。 **$\overline{M}/\overline{IO}$** =1，表示当前 CPU 正在访问存储器； **$\overline{M}/\overline{IO}$** =0，表示当前 CPU 正在访问 I/O 端口。在 DMA 方式时， **$\overline{M}/\overline{IO}$** 被悬空为高阻状态。

(2) **$\overline{WR}$** (Write): 写信号，三态、输出。当 **$\overline{WR}$** =0 低电平有效时，表示当前 CPU 正在对存储器或 I/O 端口进行写操作。跟 **$\overline{RD}$** 信号一样，由 **$\overline{M}/\overline{IO}$** 信号区分对存储器或 I/O 端口的访问。

(3) $\overline{\text{INTA}}$ (Interrupt Acknowledge): 中断响应信号, 输出, 低电平有效。这个信号是 CPU 对外设发来的 INTR 信号的响应信号。

(4) ALE (Address Latch Enable): 地址锁存允许信号, 输出, 高电平有效。8086 的 $\text{AD}_{15} \sim \text{AD}_0$ 是地址/数据复用总线, 此信号用于驱动地址锁存部件在地址信号消失前锁存地址。

(5) $\text{DT}/\overline{\text{R}}$ (Data Transmit/Receive): 数据发送/接收控制信号, 三态输出。此信号用于控制数据收发器的传送方向。

(6) $\overline{\text{DEN}}$ (Data Enable): 数据允许信号, 三态输出, 低电平有效。在最小模式系统中, 用作数据收发器的选通控制信号。在 DMA 方式时, $\overline{\text{DEN}}$ 为悬空状态。

(7) HOLD (Hold Request): 总线保持请求信号, 输入, 高电平有效。当另一个主控设备 (通常是 DMA 控制器) 需要取得总线控制权时, 就向 CPU 的 HOLD 引脚发出一个高电平的请求信号, 以向 CPU 请求取得总线控制权。

(8) HLDA (Hold Acknowledge): 总线请求响应信号, 输出, 高电平有效。这个信号是 CPU 对 HOLD 信号的响应信号。 HLDA 输出高电平有效时, 表示 CPU 已响应其他部件的总线请求, 通知提出请求的设备可以使用总线。

4. 最大模式引脚

第 24~31 脚在最大模式下含义如下:

(1) $\overline{\text{S}}_2$ 、 $\overline{\text{S}}_1$ 、 $\overline{\text{S}}_0$ (Bus Cycle Status): 总线周期状态信号, 三态输出。在最大模式系统中, 它用来作为总线控制器 8288 的输入, 经译码后产生 8288 的 7 个控制信号。此外, 最大模式时锁存地址所需的 ALE , 控制数据收器用的 $\overline{\text{DEN}}$ 和 $\text{DT}/\overline{\text{R}}$ 信号也由 8288 提供。

$\overline{\text{S}}_2$ 、 $\overline{\text{S}}_1$ 、 $\overline{\text{S}}_0$ 编码的功能与 8288 控制信号见表 5-3。

表 5-3 S_2 、 S_1 、 S_0 编码的功能与 8288 控制信号表

状 态 S_2 S_1 S_0	CPU 总线周期	8288 控制信号
L L L	中断响应	$\overline{\text{INTA}}$
L L H	读 I/O 端口	$\overline{\text{IORC}}$
L H L	写 I/O 端口	$\overline{\text{IOWC}}$, $\overline{\text{AIOWC}}$
L H H	暂停	无
H L L	访问代码	$\overline{\text{MRDC}}$
H L H	读存储器	$\overline{\text{MRDC}}$
H H L	写存储器	$\overline{\text{MWTC}}$, $\overline{\text{AMWC}}$
H H H	无效	无

(2) $\overline{\text{RQ}}/\overline{\text{GT}}_0$ 和 $\overline{\text{RQ}}/\overline{\text{GT}}_1$: 总线保持请求信号输入/总线请求允许信号输出, 双向、低电平有效。其含义与最小模式下 HOLD 和 HLDA 两信号类同。但 HOLD 和 HLDA 占用两个引脚, 而 $\overline{\text{RQ}}/\overline{\text{GT}}$ 只占用一个引脚。同时提供 $\overline{\text{RQ}}/\overline{\text{GT}}_0$ 和 $\overline{\text{RQ}}/\overline{\text{GT}}_1$ 表示可同时连接两个协处理器, $\overline{\text{RQ}}/\overline{\text{GT}}_0$ 优先级高于 $\overline{\text{RQ}}/\overline{\text{GT}}_1$ 。当某个外部处理机要占用总线时, 就从 $\overline{\text{RQ}}/\overline{\text{GT}}$ 引脚向 8086 输出一个负脉冲, 提出使用总线的申请。如果 8086 正好完成一个总线周期, 就会让出总线控制权, 并从同一条引脚向该处理机送出一个负脉冲, 以示对方可以使用总线。该

处理机用完总线后，再以一个负脉冲向 8086 报告，两个信号分时的同一引线上传输。

(3) $\overline{\text{LOCK}}$ ：总线封锁信号，三态输出，低电平有效。 $\overline{\text{LOCK}}$ 有效时，表示 CPU 不允许其他总线控制器占用总线。 $\overline{\text{LOCK}}$ 信号是由软件设置的，是为了保证 8086 在一条指令的执行中，总线使用权不会被其他主设备所打断。如果在某一条指令的前面加一个 LOCK 指令前缀，这条指令执行时，就会使 CPU 产生一个 LOCK 信号，直到这条指令结束为止。在 CPU 进入中断响应周期期间， $\overline{\text{LOCK}}$ 会自动变为有效的低电平，以防止一个完整的中断响应过程被打断。在 DMA 期间， $\overline{\text{LOCK}}$ 呈高阻态。

(4) QS_1 、 QS_0 (Instruction Queue Status)：指令队列状态，输出。作为指令队列状态的标志，当 8086 的 EU 在指令队列中取指令时，队列中的变化情况就以这两个输出位的状态编码表示出来，以便于外部其他处理机对 8086 内部指令队列进行跟踪。 QS_1 、 QS_0 的编码含义如表 5-4 所示。

表 5-4 QS_1 、 QS_0 编码含义

QS_1	QS_0	指令队列状态
L	L	无操作
L	H	从队列中取指令第一字节
H	L	队列为空
H	H	从队列中取指令后续字节

5.1.2 8088 CPU 的引脚功能

8088 微处理器是一种准 16 位机，其内部结构基本上与 8086 相同，其引脚信号也与 8086 基本相同，只有如下引脚的功能有所不同。

(1) 8088 有 8 根外部数据引脚而不是 8086 的 16 根，即 $\text{AD}_7 \sim \text{AD}_0$ ， $\text{A}_{15} \sim \text{A}_8$ 为单一的地址线。这就导致对一个 16 位数的存储器读写，总是需要二个总线周期才能完成。

(2) 8088 的第 28 引脚存储器/IO 控制信号为 $\overline{\text{M}}/\text{IO}$ ，即该信号为高电平时，是 IO 端口访问；为低电平时，是存储器访问。而 8086 为 $\text{M}/\overline{\text{IO}}$ 刚好相反。

(3) 8088 与 8086 的第 34 引脚不同，8088 中只能进行 8 位传输，所以 $\overline{\text{BHE}}$ 信号就用不着了，改为 $\overline{\text{SS}}_0$ ， $\overline{\text{SS}}_0$ 等效于 $\overline{\text{S}}_0$ ，与 $\text{M}/\overline{\text{IO}}$ 、 $\text{DT}/\overline{\text{R}}$ 组合决定最小模式下的总线操作，具体如表 5-5 所示，在最大模式下，该脚总为高电平。8088 CPU 的引脚如图 5.2 所示。

表 5-5 $\overline{\text{SS}}_0$ 、 $\overline{\text{M}}/\text{IO}$ 与 $\text{DT}/\overline{\text{R}}$ 的组合及其含义

$\overline{\text{M}}/\text{IO}$	$\text{DT}/\overline{\text{R}}$	$\overline{\text{SS}}_0$	含 义
L	L	L	取指令
L	L	H	读存储器
L	H	L	写存储器
L	H	H	无效状态
H	L	L	发中断响应信号

续表

$\overline{M}/\overline{IO}$	$DT/\overline{R}$	$\overline{SS}_0$	含 义
H	L	H	读 I/O 口
H	H	L	写 I/O 口
H	H	H	暂停

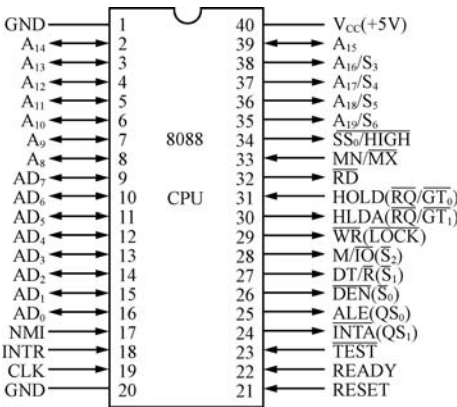


图 5.2 8088CPU 的引脚图

5.2 8086 CPU 的总线操作与时序

8086 访问一次内存或外设，至少需要 4 个时钟周期，称为 4 个 T 状态。4 个脉冲期间依次称为 T₁ 状态、T₂ 状态、T₃ 状态、T₄ 状态。分时复用的引脚在不同的 T 状态下有不同的含义。

5.2.1 指令周期、总线周期与 T 状态

8086 的操作是在单向时钟脉冲 CLK 的统一控制下进行的。相邻的两个时钟脉冲前沿之间的时间间隔称为时钟周期，也称为 T 状态，CPU 的所有操作以它为基准。8086 的时钟频率为 5MHz，故其时钟周期为 200ns。

执行一条指令需要的时间，称为指令周期。一般的 8 位微处理器的指令周期，是指取出一条指令和执行该条指令所需的时间。对 8086 而言，由于指令已经预取到指令队列寄存器中，它的指令周期只包括一条指令的执行时间和访问该指令的存储器操作数所需的时间(若该指令有存储器操作数)。8086 指令中，有些仅需 2 个时钟周期，而最长的为 16 位乘法指令，约要 200 个时钟周期。

总线周期是指 CPU 从存储器或 I/O 端口存取一个字节所需要的时间。因为这种操作必须通过总线才能完成，故称为总线周期。一次总线操作至少需要 4 个时钟脉冲才能完成，故基本的总线周期包含 4 个 T 状态。对于慢速的外设，若它不能在规定的时间内完成 CPU 所要求的操作，则它可向 CPU 的 READY 引脚发一低电平信号，CPU 将延时总线操作直到外设向 READY 引脚发高电平信号为止。此时一个总线周期需要更多个 T 状态。

典型的总线周期有存储器读周期、存储器写周期、I/O 设备输入周期、I/O 设备输出周期、中断响应周期、空闲周期等。

在一个总线周期中各个 T 状态下 CPU 引脚信号的状态,组成了 CPU 的总线操作时序。

5.2.2 存储器与 I/O 读写周期

在下面讨论总线周期时,假定 8086 工作在最小模式系统(若 8086 工作在最大模式系统,控制信号和时序安排有所不同)。

1. 存储器读周期时序

8086 存储器读周期的时序如图 5.3 所示。

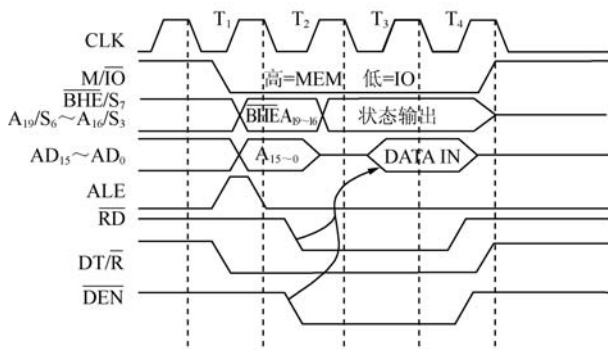


图 5.3 存储器读周期时序

每个 T 状态的操作如下:

1) T₁ 状态

(1) $\overline{M}/\overline{IO}$ 有效,用以指出是读存储器还是读 I/O 端口。

(2) 在 $A_{19}/S_6 \sim A_{16}/S_3$ 和 $AD_{15} \sim AD_0$ 上输出 20 位地址信息,同时 BHE/S_7 端输出低电平,用它作为奇存储体的选择信号。

(3) 在 T₁ 状态的后半部, $\overline{ALE}$ 信号变为低电平,8282 地址锁存器利用此下降沿将 20 位地址信息及 $\overline{BHE}$ 信号锁存。

(4) $\overline{DT}/\overline{R}$ 端输出为低电平,表示本总线周期为读周期,即数据收发器是从数据总线上接收数据。

2) T₂ 状态

(1) 地址信号撤销, $\overline{BHE}/S_7$ 和 $A_{19}/S_6 \sim A_{16}/S_3$ 开始输出状态信息 $S_7 \sim S_3$ (其中 S_7 未赋予任何实际意义), $S_7 \sim S_3$ 一直持续到 T₄。

(2) $AD_{15} \sim AD_0$ 总线地址信息消失,处于悬浮高阻状态,使 CPU 有足够的时间将 $AD_{15} \sim AD_0$ 总线由输出地址方式变为输入数据方式。

(3) 读信号 $\overline{RD}$ 有效(低电平)。

(4) 在 T₂ 中央时刻, $\overline{DEN}$ 变得有效,使数据能够从总线通过数据收发器 8286,这个信号一直持续到 T₄ 中期结束。

3) T_3 状态

CPU 在 T_3 状态一开始检测 READY 引脚, 若其为高电平, 则在 T_3 状态与 T_4 状态间不需要插入等待状态 T_w , 下一个时钟脉冲到来时将进入 T_4 状态; 若其为低电平, 说明存储器或外设尚未能将数据放到数据总线上, 则下一个时钟脉冲到来时 CPU 将进入等待状态 T_w 。通过在 T_3 与 T_4 间插入一个或几个等待状态 T_w , 可以解决 CPU 与慢速存储器或外设的速度匹配问题。

4) T_w 状态(图中未画出)

在每个 T_w 状态的下降沿 CPU 检测 READY 引脚, 若其仍为无效的低电平, 则再插入 T_w 状态, 直到 READY 引脚变为高电平才进入 T_4 状态。

5) T_4 状态

(1) 在 T_3 状态(或 T_w 状态)与 T_4 状态交界的下降沿处, CPU 读取数据线上数据。

(2) 在 T_4 状态的后半周数据信号从总线上撤销, 各控制信号及状态信号也处于无效状态, 一个总线周期结束。

2. 存储器写周期时序

8086 存储器写周期的时序如图 5.4 所示。

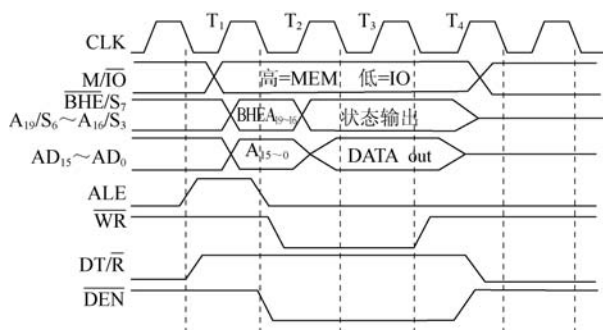


图 5.4 存储器写周期时序

写周期时序与读周期时序很类似。首先, 由 $\overline{\text{M/I0}}$ 信号来区分是访问存储器还是访问 I/O 端口。而 DT/R 应为高电平, 表示本总线周期为写周期。其次, 存储器地址以及 ALE 信号有效, 但写入存储器的数据是在 T_2 状态中放至数据线 $\text{AD}_{15} \sim \text{AD}_0$ 上的, $\overline{\text{WR}}$ 信号也在 T_2 状态有效。8086 在 T_4 状态后使控制信号变为无效, 表示对存储器的写入过程已经完成。若有的存储器或外设来不及在指定的时间内完成写操作, 可以利用 READY 信号使 CPU 插入 T_w 状态, 以保证时间上的配合。具有 T_w 状态的写周期时序与具有 T_w 状态的读周期时序类似, 不再赘述。

3. I/O 总线周期

8086 与外设通信过程, 即从 I/O 端口输入数据或把数据输出到 I/O 端口的时序, 与 8086 对存储器读写时序几乎完全相同, 但要注意其中几个具体信号的差别:

(1) $\overline{\text{M/I0}}$ 线在规定有效的 4 个 T 状态将呈低电平。

(2) 由于 I/O 的寻址空间为 64KB, 所以地址线只用了 $A_{15} \sim A_0$, $A_{19} \sim A_{16}$ 输出为 0。

5.3 8086 微处理器的系统配置

8086 微处理器有两种工作模式, 本节介绍 8086 在两种工作模式下的系统基本配置。当 8086 CPU 的 $\overline{MN}/\overline{MX}$ 引脚接 +5V 电源时, 8086 CPU 工作在最小模式下, 用于构成小型的单处理机系统, 系统中所需的总线信号全部由 8086 直接产生。这种模式下系统结构简单, 但性能也较最大模式差。当 8086 CPU 的 $\overline{MN}/\overline{MX}$ 引脚接地, 8086 CPU 工作在最大模式。最大模式可构成多处理器系统, 此时总线控制信号由外加的总线控制器 8288 产生, 8086 只须向 8288 提供译码所需的状态信号。这种模式下系统较复杂, 但性能好, 适于组成高档微机系统。

5.3.1 8086 CPU 最小模式下的系统配置

图 5.5 为一种典型的最小模式系统的基本配置。它除了 8086 CPU 及信息传送对象存储器外, 还包括 8284A 时钟发生器, 三片 8282(或 8283)及两片 8286(或 8287)。

由于要锁存 20 位地址信息及 $\overline{BHE}$ 信号, 故需要三片 8282 地址锁存器。8282 的输入选通端 $\overline{STB}$ 同 8086 的 $\overline{ALE}$ 引脚相连。

对于 8086 系统, 数据线为 16 位, 需要两片 8286; 对于 8088 系统, 数据线为 8 位, 则只需一片 8286。8286 的 $\overline{T}$ 端同 8086/8088 CPU 的 $\overline{DT}/\overline{R}$ 引脚相连, 以控制传送方向。8286 的 $\overline{OE}$ 端与 8086/8088 的 $\overline{DEN}$ 引脚相连, 使得只有在 CPU 访问存储器或 I/O 端口时, 才能允许数据通过 8286, 否则 8286 在两个方向上都处于高阻状态。

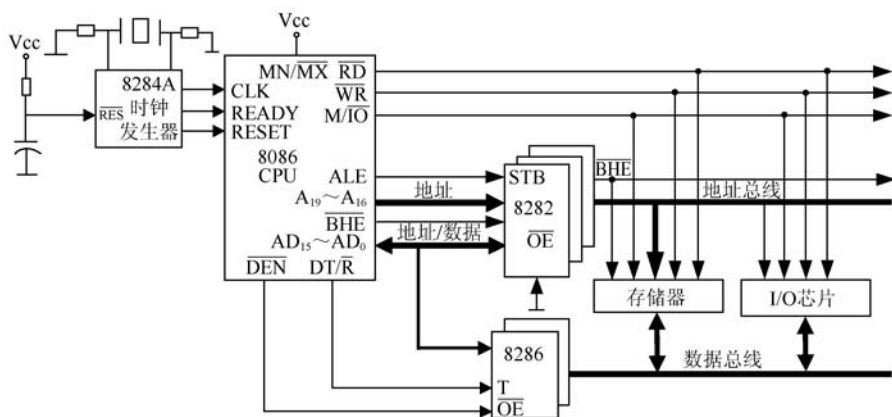


图 5.5 8086 最小模式系统的基本配置

1. 时钟发生器 8284A

8284A 是用于 8086(或 8088)系统的时钟发生器/驱动器芯片, 它为 8086(或 8088)及其他外设芯片提供所需要的时钟信号。8284A 功能和引脚如图 5.6 所示。

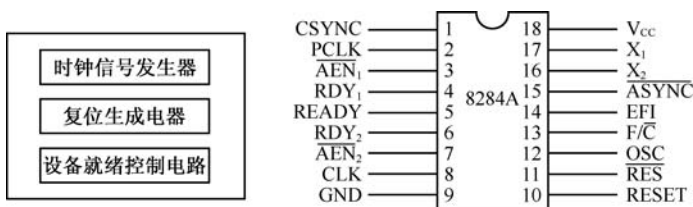


图 5.6 8284A 功能和引脚

(1) 时钟信号发生器提供系统所需要的时钟信号，有两个来源：一个是在 X_1 与 X_2 引脚间接上晶体，由晶体振荡器产生时钟信号；另一个是由 EFI 引脚加入的外接振荡信号产生时钟信号，两者由 $F/\overline{C}$ 端信号控制。 $F/\overline{C}$ 为低电平，表示由外接振荡器产生。

如果晶体振荡器的工作频率为 14.31818MHz，则该时钟脉冲(OSC)经 3 分频后得到 4.77MHz 的时钟脉冲 CLK，即微处理器(例如 8086)所需的时钟信号(占空比为 1:3)。CLK 再经 2 分频后产生外设时钟 PCLK，其频率为 2.385MHz(占空比为 1:2)。

(2) 复位生成电路。输入信号 $\overline{RES}$ 在 8284 延迟和同步后产生系统复位信号 RESET，高电平有效，使系统初始化。

(3) 设备就绪控制电路。就绪控制电路有两组输入信号，每一组都有允许信号 $\overline{AEN}$ 和设备就绪信号 RDY ， $\overline{AEN}$ 是低电平有效信号，用以控制其对应的 RDY 信号的有效。这种工作方式用于能保证满足 RDY 建立时间要求的同步设备中。

2. 地址锁存器和数据收发器

1) 地址锁存器与数据收发器的作用

由于 8086 的 $AD_{15} \sim AD_0$ 为分时复用的地址/数据线，即在 T_1 状态用来输出地址，从 T_2 状态开始改为传送数据，而内存及 I/O 设备需要在整个总线操作周期中地址线上都保持有稳定的地址信号，所以需要在地址信号消失前将其锁存。

对于数据信号，它不必锁存，但由于总线负载能力有限，当挂接部件过多时，就需要接入信号放大器，提高总线的负载能力，这种信号放大器称为数据收发器。

2) 地址锁存器与数据收发器芯片

8086 系统中使用 8282/8283 作为地址信号锁存器，8282 锁存器的输入和输出不是反相的，8283 是反相的，其余功能相同，本处只讨论 8282。8282 是一种通用的三态输出的 8 位锁存器，可用于数据的锁存、缓冲或信号的多路传输。其引脚信号及功能分别如图 5.7 和表 5-6 所示。8086 系统中采用 8286 或 8287 作为数据收发器，它们均是双向、三态输出的收发器，8287 除了输出与输入反相外，其余均同 8286。8286 的引脚信号及功能分别如图 5.8 和表 5-7 所示。

表 5-6 8282 引脚功能

引 脚	功 能
$DI_7 \sim DI_0$	数据输入
$DO_7 \sim DO_0$	数据输出

续表

引 脚	功 能
$\overline{\text{OE}}$	允许输出，OE 为低电平时，允许锁存数据从 $\text{DO}_7 \sim \text{DO}_0$ 输出；OE 为高电平时，输出端 $\text{DO}_7 \sim \text{DO}_0$ 呈高阻态
STB	选通输入，当其上信号由高变低时，将 $\text{DI}_7 \sim \text{DI}_0$ 上数据锁存起来
GND、 V_{CC}	地、电源

表 5-7 8286 引脚功能

引 脚	功 能
$\text{A}_7 \sim \text{A}_0$	数据输入/输出
$\text{B}_7 \sim \text{B}_0$	数据输入/输出
$\overline{\text{OE}}$	允许输出，OE 为低电平时，允许输出；OE 为高电平时，所有输出呈高阻态
T	方向控制：T 为高电平时，由 A 向 B 传输；T 为低电平时，由 B 向 A 传输。
GND、 V_{CC}	地、电源

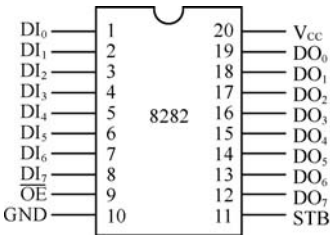


图 5.7 8282 引脚信号

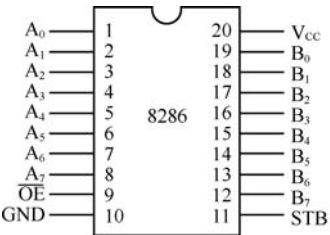


图 5.8 8286 的引脚信号图

3. 8086 的地址锁存与数据收发逻辑

图 5.5 是 8086 利用 8282 锁存地址信号和利用 8286 驱动数据总线。在总线周期的 T_1 状态，当 ALE 信号由高变低时，8282 将地址信号锁存，使得从 T_2 开始 CPU 撤销地址信号后，地址总线上还能保持地址输出。

当 8086 与存储器或 I/O 设备交换数据时， $\overline{\text{DEN}}$ 端产生一个低电平信号，使 8286 的 $\overline{\text{OE}}$ 端为 0。8286 的发送数据控制端 T 连接在 8086 的数据发送/接收端 $\text{DT}/\overline{\text{R}}$ 上，写操作时， $\text{DT}/\overline{\text{R}}$ 为高电平，控制数据传送方向是从 CPU 流向存储器或 I/O 设备；在读操作时， $\text{DT}/\overline{\text{R}}$ 为低电平，则数据的传送方向是从存储器或 I/O 设备流向 CPU。在较小的 8086 系统中，也可以不用 8286 数据收发器，这时，多路复用的地址/数据总线直接与存储器或 I/O 设备的数据线相连。

5.3.2 8086 CPU 最大模式下的系统配置

8086 最大模式系统的基本配置如图 5.9 所示。

最大模式一般用于多处理器系统，为此需要增加总线控制逻辑，由总线控制器 8288 实现。8288 将 CPU 的状态信号 $\overline{\text{S}}_2$ 、 $\overline{\text{S}}_1$ 、 $\overline{\text{S}}_0$ 转换成总线命令及控制信号。控制 8282 地址锁

存器、8286 总线收发器所需控制信号也由 8288 产生。

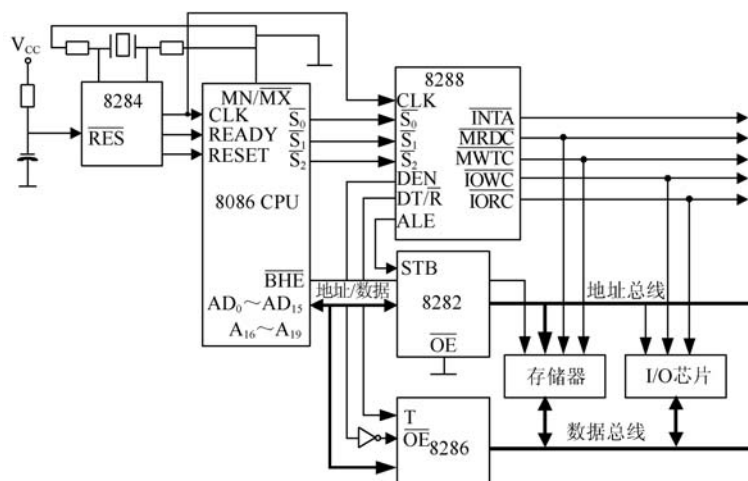


图 5.9 8086 最大模式系统的基本配置

1. 总线控制器 8288

8288 总线控制器是 8086 工作在最大方式下构成系统时必不可少的支持芯片，它根据 8086 在执行指令时提供的总线周期状态信号 $\overline{S_2}$ 、 $\overline{S_1}$ 和 $\overline{S_0}$ 建立控制时序，输出读/写控制命令，可以提供灵活多变的系统配置，以实现最佳的系统性能。8288 的结构框图和引脚信号如图 5.10 所示。

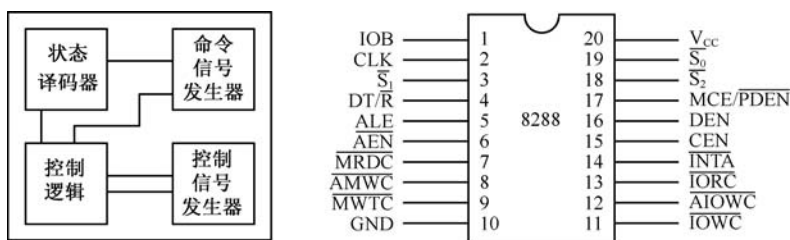


图 5.10 8288 的结构框图和引脚信号

1) 状态译码和命令输出

8288 根据 8086 的总线状态信号 $\overline{S_2}$ 、 $\overline{S_1}$ 和 $\overline{S_0}$ 确定 8086 执行何种总线周期，发出相应的命令信号去控制系统中的相关部件。总线周期的状态信号与输出命令的关系见表 5-3。

表中 I/O 读、写命令以及存储器读、写命令 $\overline{IORC}$ 、 $\overline{IOWC}$ 、 $\overline{MRDC}$ 、 $\overline{MWTC}$ 代替了最小方式的 3 条控制线 $\overline{RD}$ 、 $\overline{WR}$ 和 $\overline{M/\overline{IO}}$ 。而 $\overline{AIOWC}$ 和 $\overline{AMWC}$ 为超前命令，可在写周期之前就启动写过程，从而能够在一定程度上避免微处理器进入等待状态，这两个超前命令比 $\overline{IOWC}$ 和 $\overline{MWTC}$ 出现时间早一个时钟周期，在需要提前发出写命令的场合，可以选用这两个超前信号。

2) 控制逻辑

8288 的工作受输入信号的控制，这些信号是 $\overline{IOB}$ 、 $\overline{AEN}$ 、 $\overline{CEN}$ 和 $\overline{CLK}$ 。

(1) IOB(输入/输出总线方式)。

当 IOB 处于低电平时, 8288 处于系统总线方式, 总线仲裁逻辑通过向输入端 $\overline{\text{AEN}}$ 发送低电平表示总线可供使用, 在 $\overline{\text{AEN}}$ 有效(低电平)后 115ns 内, 8288 不发出任何命令, 这段时间进行总线切换。多个处理器使用一组总线的系统中, 当 I/O 设备和存储器都是共享设备时, 存储器写命令和 I/O 写命令都要经过总线仲裁。

当 IOB 处于高电平时, 8288 处于 I/O 总线方式, 在该方式下, 所有 I/O 命令信号($\overline{\text{IORC}}$ 、 $\overline{\text{IOWC}}$ 、 $\overline{\text{AIOWC}}$ 、 $\overline{\text{INTA}}$)总是可以有效的, 且与 $\overline{\text{AEN}}$ 的状态无关。而对存储器访问命令都无效。一旦处理器启动某个 I/O 命令, 8288 就利用 $\overline{\text{PDEN}}$ 和 $\text{DT}/\overline{\text{R}}$ 激活相应的命令线去控制 I/O 总线收发器, 在这种工作方式中, 由于没有提供总线仲裁机构, 因此不能把 I/O 命令用来控制系统总线。这种方式允许 8288 总线控制器能去管理两组外部总线, 当微处理器要访问 I/O 总线时, 无须等待, 而正常的存储器访问之前, 需要一个“总线准备好”信号($\overline{\text{AEN}}$ 为低电平)。如果在多处理机系统中, 用一个处理机专门管理 I/O 或外设, 用 IOB 方式是很有利的。

(2) $\overline{\text{AEN}}$ (地址使能)。

当 $\overline{\text{AEN}}$ 处于高电平时, 8288 各种命令无效, 呈高阻态。

当 $\overline{\text{AEN}}$ 处于低电平时, 对于系统总线方式, 至少在 $\overline{\text{AEN}}$ 有效后 115ns, 8288 才能输出命令; 但在 I/O 总线方式, $\overline{\text{AEN}}$ 不起作用, 即不影响 I/O 命令的发出。

(3) CEN(命令使能)。

当 CEN 处于高电平时, 命令有效; CEN 处于低电平时, 各种命令和 $\overline{\text{DEN}}$ 、 $\overline{\text{PDEN}}$ 等输出都无效。

这是为多个 8288 联合工作而设置的一个协调信号, 应使工作的 8288 的 CEN 为高电平。

(4) CLK(时钟)。

8288 产生命令和控制信号输出时, 由 CLK 决定它们的定时关系。通常由微型计算机的系统时钟提供。

3) 控制信号发生器

8288 总线控制器的输出控制信号为 ALE、DEN、 $\text{DT}/\overline{\text{R}}$ 和 $\text{MCE}/\overline{\text{PDEN}}$ 。

(1) ALE(地址锁存允许)。

用于将地址选通到地址锁存器。高电平有效, 在下跳沿中发生锁存。

(2) DEN(数据使能)。

DEN 为高电平时, 接通数据收发器。

(3) $\text{DT}/\overline{\text{R}}$ (数据发送/接收)。

$\text{DT}/\overline{\text{R}}$ = 处于高电平时, 发送状态; $\text{DT}/\overline{\text{R}}$ = 处于低电平时, 接收状态。用来控制数据收发器的传送方向。

(4) $\text{MCE}/\overline{\text{PDEN}}$ (主设备使能/外设数据允许)。

双重功能, 当 IOB 处于低电平时, 即工作于系统总线方式时, 该引脚信号为 MCE, 高电平有效的输出信号, MCE 是为配合 8259A 级联工作而设置的。在 8259A 级联工作时, 第一个 $\overline{\text{INTA}}$ 总线周期, 由“主 8259A”向“从 8259A”发出级联地址, 第二个 $\overline{\text{INTA}}$ 总线周期, 提出中断请求的“从 8259A”将中断类型码送上地址线。MCE 为锁存第一个 $\overline{\text{INTA}}$

周期的级联地址锁存信号。当 IOB 处于高电平时, 即工作于 I/O 总线方式时, 该引脚信号为 $\overline{\text{PDEN}}$, 低电平有效的输出信号, 其作用类似于 DEN。 $\overline{\text{PDEN}}$ 是 I/O 总线上的数据选通信号, 是系统总线的数据选通信号。

2. 时钟发生器、总线锁存器和总线收发器

从图 5.9 可见, 在最大配置中, 这三种部件的工作与最小配置相同。

3. 需要说明的问题

(1) 8086 CPU 在最小方式下的 HOLD 和 HLDA 引脚在最大方式时成为 $\overline{\text{RQ}}/\overline{\text{GT}}_0$ 和 $\overline{\text{RQ}}/\overline{\text{GT}}_1$ 信号线, 这两条引脚通常同 8087(协处理器)或 8089(I/O 处理器)相连接, 用于 8086 同它们之间传送总线请求与总线应答信号。

(2) 当系统为具有两个以上主 CPU 的多处理器系统时, 必须配上总线仲裁器 8288, 用来保证系统中的各个处理器同步地进行工作, 以实现总线共享。

5.4 80386 CPU 的引脚功能

80386 采用 132 引脚的栅状阵列封装(PGA), 其中 34 条地址线($A_{31} \sim A_2$ 、 $\overline{\text{BE}}_3 \sim \overline{\text{BE}}_0$), 32 条数据线($D_{31} \sim D_0$), 3 条中断线, 1 条时钟线, 13 条控制线, 20 条电源线 V_{CC} , 21 条地线 V_{ss} , 还有 8 条为空。图 5.11 是 80386 CPU 的逻辑引脚图。与 8086/8088 相比, 需要说明以下几点:

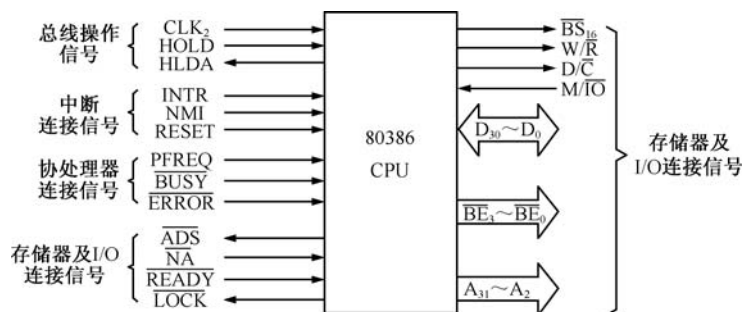


图 5.11 80386 CPU 的逻辑引脚图

1. 时钟 (CLK₂)

80386 的基本定时信号由 CLK₂ 提供高。CLK₂ 的频率是 80386 内部时钟信号频率的两倍, 输入该信号与 82384 时钟信号同步, 经 80386 内部 2 分频之后得到 80386 的工作基准频率信号。

2. 数据总线 ($D_{31} \sim D_0$)

它为 80386 和其他设备之间提供数据通路, 32 位数据总线, 双向三态, 一次可传送 8 位、16 位或 32 位数据, 由输入信号 ($\overline{\text{BE}}_3 \sim \overline{\text{BE}}_0$) 和 $\overline{\text{BE}}_{16}$ 确定, 在任何写操作周期(包括暂停周期和停机周期), 80386 总是驱动数据总线的所有 32 位信号, 而不管当前总线的实际宽度。

3. 地址总线($A_{31} \sim A_2$, $\overline{BE}_3 \sim \overline{BE}_0$)

(1) $A_{31} \sim A_2$: 地址总线, 输出三态, 和 $\overline{BE}_3 \sim \overline{BE}_0$ 相结合起到 32 位地址的作用。80386 地址总线包含 $A_2 \sim A_{31}$ 地址线和字节选通线 $\overline{BE}_3 \sim \overline{BE}_0$ 。 $\overline{BE}_3 \sim \overline{BE}_0$ 线的功能与 8086 和 80286 系统的 A_0 和 $\overline{BHE}$ 的非常相似, 它们是内部地址信号 A_0 和 A_1 的译码。80386 有一个 32 位数据总线, 所以内存可以建立 4B 宽的存储体。 $\overline{BE}_3 \sim \overline{BE}_0$ 信号是用来选通这 4 个存储体。这些单独选通可以使 80386 的内存传送或者接收字节、字或者双字。

(2) $\overline{BE}_3 \sim \overline{BE}_0$: 字节选通信号。用于选通在当前的传送操作要涉及 4B 数据中的那几个字节。 $\overline{BE}_0$ 对应于 $D_0 \sim D_7$, $\overline{BE}_1$ 对应于 $D_8 \sim D_{15}$, $\overline{BE}_2$ 对应于 $D_{16} \sim D_{23}$, $\overline{BE}_3$ 对应于 $D_{24} \sim D_{31}$ 。

4. 总线周期定义信号($M/\overline{IO}$, $W/\overline{R}$, $D/\overline{C}$, $\overline{LOCK}$, 三态, 输出, 用来定义正在进行的总线周期类型)

(1) $M/\overline{IO}$: 存储器/输入输出选择信号, 输出信号。高电平时访问存储器, 低电平时访问 I/O 端口。 $M/\overline{IO}$ 指示操作是读入内存还是直接 I/O 操作。80386 直接 I/O 端口简单地把 8086 和 80286 端口结构扩充成 32 位端口。32 位 I/O 端口可以通过并联 8 位 I/O 端口设备, 如 8255A 来构成。80386 可以使用所有 8 位端口地址的 IN 或 OUT 指令来编址 256 个 8 位端口、128 个 16 位端口、64 个 32 位端口。使用 DX 寄存器存放 16 位端口地址, 80386 可以编址 64K 个 8 位端口、32K 个 16 位端口或 8K 个 32 位端口。

(2) $W/\overline{R}$: 读/写控制输出信号, 高电平时写入, 低电平时读出, $W/\overline{R}$ 信号指示是否发生了读/写操作。

(3) $D/\overline{C}$: 数/控控制信号, 输出。高电平时传送数据, 低电平时传送指令代码, $D/\overline{C}$ 指示总线操作是一个数据、读/写还是控制字传输(如取一个操作码)。

$W/\overline{R}$, $D/\overline{C}$, $M/\overline{IO}$ 是总线周期定义信号。当 80386 驱动 $\overline{ADS}$ (地址状态)输出信号有效时, 这三个信号被驱动为有效, 根据 3 个信号的功能可得到总线周期定义, 如表 5-8 所示。

表 5-8 总线周期定义

$M/\overline{IO}$	$D/\overline{C}$	$W/\overline{R}$	总线周期类型	是否锁定
L	L	L	中断响应	是
L	L	H	不会出现	/
L	H	L	读 I/O 数据	否
L	H	H	写 I/O 数据	否
H	L	L	读存储器代码	否
H	L	H	暂停(地址=2), 停机(地址=0)	否
H	H	L	读存储器数据	某些周期
H	H	H	写存储器数据	某些周期

(4) $\overline{LOCK}$: 总线周期封锁信号, 低电平有效。

5. 总线控制信号($\overline{ADS}$, $\overline{READY}$, $\overline{NA}$, $\overline{BE}_{16}$)

这组信号用来表示总线周期何时开始, 也是其他系统硬件可以地址流水方式、数据总线的宽度和总线周期的终结。

(1) $\overline{ADS}$: 地址选通信号, 三态输出, 低电平有效。当有效时, 表示总线周期中地址

信号有效。当有效地址、BE 信号和总线周期定义信号均在总线上时， $\overline{\text{ADS}}$ 信号将被设置。80386 地址总线是不可复用的，所以 8086 类型的 ALE 信号是不需要的。但是，在某些 80386 系统中， $\overline{\text{ADS}}$ 信号用于一种称为地址流水线的模式，将地址传送到外部锁存器。地址流水线的原理是：如果一个地址保持在外部锁存器的输出端，80386 就可以把地址引脚上的老地址清除掉，并在总线周期的前期输出下一个操作的地址。外部控制芯片通过设置下一个地址信号来通知 80386 何时为下一个操作输出地址。对一个有 SRAM 高速缓冲的系统，流水线地址模式通常不是必需的，因为 SRAM 高速缓冲已足够快了，不需要等待状态。

(2) $\overline{\text{READY}}$ ：准备就绪，输入信号，低电平有效。 $\overline{\text{READY}}$ 有效时表示当前总线周期已完成。信号用来在总线周期中根据低速的内存和 I/O 设备接口的需要插入等待状态。

(3) $\overline{\text{NA}}$ ：下一个地址请求信号，输入信号，低电平有效。允许地址流水线操作，当其有效，表示当前执行中的周期结束之后，下一个总线周期的地址和状态信号可变为有效。

(4) $\overline{\text{BE}}_{16}$ ：输入信号，低电平有效，指定 16 位数据总线。 $\overline{\text{BE}}_{16}$ 输入端允许 80386 以 16 位和/或 32 位数据总线工作。如果设置了 $\overline{\text{BE}}_{16}$ ，80386 只将数据传送到 32 位数据总线的低 16 位上。如果设置了 $\overline{\text{BE}}_{16}$ 并且要从 16 位宽内存中读一个 32 位的操作数，80386 将自动产生一个第二总线周期来读第二个字。对于未调整的传输，如果设置了 $\overline{\text{BE}}_{16}$ ，80386 也产生所需数目的总线周期。

6. 总线仲裁信号(HOLD, HLDA)

由总线请求主设备来控制这组信号。

(1) HOLD：总线请求信号，输入信号，高电平有效。

(2) HLDA：总线保持响应信号，输出信号，有效时，CPU 让出总线。

7. 协处理器接口信号($\overline{\text{PEREQ}}$, $\overline{\text{BUSY}}$, $\overline{\text{ERROR}}$)

这组信号控制了 80386 同 80287 或 80387 之间的通信。

(1) $\overline{\text{PEREQ}}$ ：来自协处理器的请求信号，输入信号，表示 80387 要求 80386 控制它们与存储器之间的信息传送。 $\overline{\text{PEREQ}}$ 信号是由一个像 80387 浮点处理器这样的协处理器输出的，它通知 80386 为协处理器取数据字的第一部分，然后协处理器将接管总线并读数据字的其余部分。

(2) $\overline{\text{BUSY}}$ ：协处理器忙，输入信号，低电平有效。 $\overline{\text{BUSY}}$ 信号由协处理器使用。以避免 80386 在协处理器结束当前指令之前又继续下一条指令。

(3) $\overline{\text{ERROR}}$ ：协处理器错误信号，输入信号，低电平有效。如果协处理器设置了 $\overline{\text{ERROR}}$ 信号，80386 将执行类型为 16 的异常中断。

8. 中断信号(INTR, NMI, RESET)

这组信号用来引起中断或中止 80386 正在执行的指令流。

(1) INTR：可屏蔽中断请求，输入信号。80386 响应 INTR 请求时，完成两个连续的中断响应周期，在整个响应周期， $\overline{\text{LOCK}}$ 信号有效。在第二个周期末， $\text{D}_0 \sim \text{D}_7$ 数据线上送上 8 位中断类型码，以识别中断源。INTR 信号可以由 80386 的标志寄存器中的 IF 位屏蔽。

(2) NMI：非屏蔽中断请求，输入信号。80386 对 NMI 的处理不运行中断响应周期，而是自动产生一个中断类型 2。

(3) RESET: 复位信号。输入信号, 当 RESET 有效时, 将中止 80386 正在执行的一切操作, 并置于一个已知的复位状态。复位期间的 80386 的有关引脚的状态如表 5-9 所示。

80386 有许多 V_{CC} 脚, 也有许多标为 V_{ss} 的地线, 这些引脚均被接到 PC 板上合适的电平上。

表 5-9 复位期间引脚的状态(总线空闲)

引脚信号	复位期间信号的电平	引脚信号	复位期间信号的电平
$A_{31} \sim A_2$	高	$D/\overline{C}$	高
$D_{31} \sim D_0$	高阻抗	$M/\overline{IO}$	低
$\overline{BE}_3 \sim \overline{BE}_0$	低	$\overline{LOCK}$	高
$\overline{ADS}$	高	HOLD	低
$W/\overline{R}$	低		

5.5 80486 CPU 的引脚功能

80486 采用 PGA(Pin Grid Array)封装, 具有 168 个管脚, 比 80386 多 36 个引脚。其中有些新增引脚是为与设备构成更有效的连接而设置的; 有些是为了新部件(如 Cache 等)而设置的; 另外有些是空引脚以备用。

与 80386 相比, 80486 删除了其 4 个引脚, 新增了 20 个引脚; 接通了 19 条不活动的待用逻辑管脚。可以认为 80486 的其余 129 条引脚与 80386 在逻辑上是一致的。

80486 的引脚信号如图 5.12 所示, 通常又称为局部总线信号。按其功能分类, 80486 引脚信号被分为数据总线、地址总线、总线控制、中断、总线周期定义、Cache 行无效、Cache 控制、突发(或成组传递)控制、总线宽度控制、数据出错报告、地址位 20 屏蔽和奇偶校验等几部分。下面分别介绍这些信号的作用。

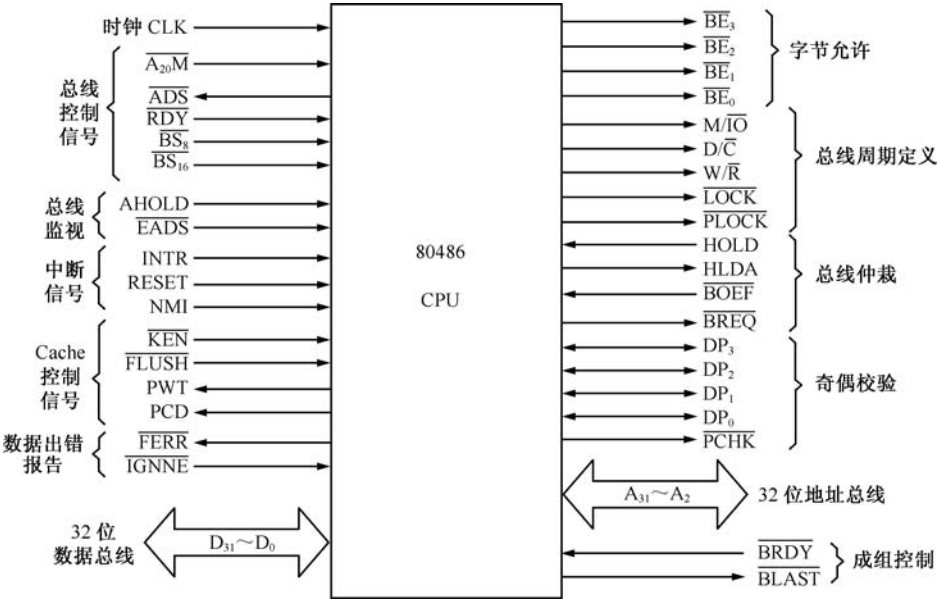


图 5.12 80486 CPU 的引脚信号功能

1. 数据总线和地址总线

1) 数据总线

$D_0 \sim D_{31}$: 为 32 位的双向(输入/输出)数据总线, 可以分别传输 8 位、16 位、24 位或 32 位数据。

2) 地址总线

$A_2 \sim A_{31}$ 和 $\overline{BE}_3 \sim \overline{BE}_0$ 构成 32 位地址总线, 能寻址 4GB 的物理存储空间和 64KB I/O 端口地址。80486 CPU 寻址 I/O 端口地址时, 它发出的地址中, 只有低 16 位地址 $A_0 \sim A_{15}$ 和 $\overline{BE}_3 \sim \overline{BE}_0$ 起作用, 其余的视为无效。当 80486 寻址内存时, 它发出的地址 $A_{31} \sim A_2$ 与双字(4 个字节)地址单元的内存对应, 双字的 4 个字节由 $\overline{BE}_3 \sim \overline{BE}_0$ 控制, $\overline{BE}_3$ 控制的数据字节为 $D_{31} \sim D_{24}$, $\overline{BE}_2$ 控制的数据字节为 $D_{23} \sim D_{16}$, $\overline{BE}_1$ 控制的数据字节为 $D_{15} \sim D_8$, $\overline{BE}_0$ 控制的数据字节 $D_7 \sim D_0$ 。 $\overline{BE}_3 \sim \overline{BE}_0$ 为 4 个字节允许信号, 只有当字节允许信号有效(低电平有效)时, 它控制的存储器才能进行数据输入/输出操作。

32 位地址线中 $A_{31} \sim A_4$ 为双向(输入/输出)地址线。当其他控制器(如 DMA 控制器)控制系统总线时, CPU 通过 $A_{31} \sim A_4$ 输入地址总线上的信号, 监视地址总线的活动, 一旦发现主存中的数据改变, 则将 CPU 芯片内 Cache 中的数据标记为过时, 这样才能保证数据的一致性。80486 CPU 这个功能称为 Cache 行无效。

2. 总线控制信号

(1) $\overline{ADS}$: 地址状态信号, 输出信号。 $\overline{ADS}$ 有效时, 地址总线上输出的信号有效。

(2) $\overline{READY}$: 准备好信号, 输入信号, 控制总线周期的宽度。当 $\overline{READY}$ 有效时, 结束当前总线周期。

(3) $\overline{BS}_8$: 8 位总线宽度定义信号, 输入信号, 低电平有效。当 $\overline{BS}_8$ 有效时, 定义数据总线只有 8 位是有效的, 支持 8 位数据传输。

(4) $\overline{BS}_{16}$: 16 位总线宽度定义信号, 输入信号, 低电平有效。当 $\overline{BS}_{16}$ 有效时, 定义数据总线只有 16 位是有效的, 支持 16 位数据传输。

(5) $\overline{A_{20}M}$: 第 20 位地址屏蔽信号, 输入信号, 低电平有效。80486 采样到 $\overline{A_{20}M}$ 低电平有效信号后, 将屏蔽掉第 20 位物理地址(即存储器地址), 80486 仿真 8086 的 1M 存储器地址。 $\overline{A_{20}M}$ 是异步信号, 因此 $\overline{A_{20}M}$ 信号保持低电平的时间至少大于一个处理器时钟周期。

3. 总线仲裁信号

(1) HOLD: 总线请求信号, 请求 80486 CPU 放弃对总线的控制权。

(2) HLDA: 总线请求响应信号。当 CPU 收到 HOLD 信号后, 在当前总线周期结束时发出 HLDA 信号, 表示 CPU 让出总线控制权。HOLD 与 HLDA 信号功能及作用与 80386 CPU 中对应引脚的功能完全相同。

(3) $\overline{BOFF}$ 强制 80486 CPU 放弃系统总线。当 80486 CPU 收到 $\overline{BOFF}$ 为低电平(有效电平)信号时, CPU 立即放弃对总线的控制权而处于总线保持状态。在 $\overline{BOFF}$ 变为无效(高电平)前, CPU 一直处于总线保持状态。如果在 CPU 的总线周期期间, $\overline{BOFF}$ 先变为有效, 然后再变为无效, 则 CPU 将重新启动该周期。 $\overline{BOFF}$ 信号的优先权高于 $\overline{READY}$ 和 $\overline{BRDY}$ 。

(4) $\overline{\text{BREQ}}$: 总线周期挂起信号, 高电平有效。当 $\overline{\text{BREQ}}$ 信号有效时, 表明 80486 CPU 内部已提出一个总线请求, 80486 CPU 正控制系统总线。

4. 中断信号

(1) INTR : 可屏蔽中断请求信号。

(2) NMI : 非屏蔽中断请求信号。

(3) RESET : 复位信号, 高电平有效。当 RESET 信号有效时, 迫使 80486 CPU 进入复位状态。

这三个信号与 80386 中对应信号的功能相同。

5. 总线周期定义信号

(1) $\overline{\text{W/R}}$: 读/写周期。高电平时进行写周期, 低电平时进行读周期。

(2) $\overline{\text{M/IO}}$: 存储器或 I/O 访问。高电平时访问存储器, 低电平时访问 I/O 端口。

(3) $\overline{\text{D/C}}$: 数据/控制周期, 高电平为数据周期, 低电平为控制周期。

(4) $\overline{\text{LOCK}}$ 封锁总线周期, 低电平有效。当 $\overline{\text{LOCK}}$ 有效时, 保证 80486 CPU 顺利完成一条指令。 $\overline{\text{LOCK}}$ 信号由带“LOCK”前缀的指令设置, 在该条指令执行完毕前, 其他总线控制器不能获得对系统总线的控制权。

(5) $\overline{\text{PLOCK}}$ 伪封锁总线周期, 低电平有效。当 $\overline{\text{PLOCK}}$ 有效时, 表明 80486 CPU 的现行总线需要多个总线传输才能完成。如协处理器读出/写入长字(6480 位)操作时, 段表描述符的读出和超高速缓存行。 $\overline{\text{PLOCK}}$ 有效, 保证了上述操作能顺利完成。

在建立 $\overline{\text{ADS}}$ 信号时, $\overline{\text{M/IO}}$, $\overline{\text{W/R}}$, $\overline{\text{D/C}}$ 被驱动为有效。总线周期定义如表 5-10 所示。

表 5-10 总线周期定义

$\overline{\text{M/IO}}$	$\overline{\text{D/C}}$	$\overline{\text{W/R}}$	启动的总线周期	$\overline{\text{M/IO}}$	$\overline{\text{D/C}}$	$\overline{\text{W/R}}$	启动的总线周期
L	L	L	中断响应	H	L	L	代码读出
L	L	H	停机/暂停	H	L	H	Intel 公司保留
L	H	L	I/O 读出	H	H	L	存储器读出
L	H	H	I/O 写入	H	H	H	存储器写入

6. 总线监视信号

行无效周期使 80486 芯片内部超高速缓存器的内容与外部主存储器中的内容保持一致。超高速缓存行无效周期使用两个输入信号 $\overline{\text{AHOLD}}$ 和 $\overline{\text{EADS}}$ 。

(1) $\overline{\text{AHOLD}}$: 地址保持请求信号。高电平有效。80486 允许其他的总线控制访问自己的地址总线。 $\overline{\text{AHOLD}}$ 有效将强制 80486 浮空自己的地址总线, 为从 $\text{A}_{31} \sim \text{A}_4$ 线上输入地址做好准备。

(2) $\overline{\text{EADS}}$ 地址有效信号, 低电平有效。当 $\overline{\text{EADS}}$ 信号有效时, 表示 80486 芯片的地址线 $\text{A}_{31} \sim \text{A}_4$ 上的信号有效, 供其读入。80486 读入后, 在内部的超高速缓存器中寻找该地址, 找到该地址, 则执行无效周期, 使该地址的数据变为无效。

概括地讲, $\overline{\text{HOLD}}$ 信号限定 80486 的地址线 $\text{A}_{31} \sim \text{A}_4$ 输入某一地址, $\overline{\text{EADS}}$ 信号指明

该地址有效。

7. Cache 控制信号

1) 页面超高速缓存控制

80386、80486 和 Pentium CPU 对存储器的管理有两种模式：分段管理和分页管理。分段管理是常用模式，分页管理是可以选择的。只有 CPU 内部的状态寄存器、MSW 中的分页位被置位时，才允许使用分页管理模式，否则禁止分页。分页部件把 32 位线性地址转换成存储器的地址(即主机内存地址)从地址总线上输出。CPU 内部的分页部件使用二级方式：页目录和页表。页目录有 1024 项，每一项选择一个页表，即页目录由页表所组成。页表有 1024 项，每一项对应 4KB 存储器，每 4KB 存储器为一个页面。页目录、页表、页面三项的乘积($1024 \times 1024 \times 4KB = 4GB$)为 CPU 寻址存储器的最大容量。

PWT 和 PCD 两个输出信号是专门为页表结构(页目录和页表)设置的。PWT 和 PCD 控制页面可超高速缓存性及写方式。

(1) PWT：写方式控制信号。PWT=1 定义当前页面为直写方式，PWT=0 为回写方式。由 80486 CPU 片内 Cache 为直写方式，所以 PWT 信号在 80486 内部不起作用，只是用来定义外部 Cache(2 级 Cache)的写方式。

(2) PCD：超高速缓存允许信号。PCD=0 允许 80486 片内 Cache 进行缓存，否则(PCD=1)禁止缓存。

2) 超高速缓存控制

(1) $\overline{KEN}$ ：超高速缓存允许输入信号，低电平有效。有效时使 80486 执行超高速缓存器的行填入周期。

(2) $\overline{FLUSH}$ ：超高速缓存器清洗输入信号，低电平有效。有效时强制 80486 对内部超高速缓存器进行大清除(使全部数据变为无效)。 $\overline{FLUSH}$ 是异步信号，因此它保持低电平的时间至少要大于一个处理器时钟周期供 CPU 采样输入。如果在复位信号 RESET 下降沿的前一个时钟采样到 $\overline{FLUSH}$ 为低电平，将使 80486 CPU 进入三态测试方式。

8. 突发控制(或成组传递控制)

(1) $\overline{BRDY}$ ：突发准备好信号，输入信号，低电平有效。 $\overline{BRDY}$ 信号的作用与 $\overline{READY}$ 信号(有时称为非突发准备好信号)相同。当两者都为有效信号时，80486 忽略 $\overline{BRDY}$ 信号，当前周期由 $\overline{READY}$ 信号结束。

由 $\overline{BRDY}$ 信号结束的周期称为非突发周期，非突发周期一般由两个处理器时钟完成一个数据传输。由 $\overline{BRDY}$ 信号结束的周期称为突发周期，突发周期分为快突发周期和慢突发周期。

对于快突发周期，除了第一个突发周期要用两个处理器时钟完成一个数据传输外，在后面的突发周期中，只要用一个处理器时钟便完成一个数据传输。对慢突发周期，与非突发周期相同，用两个处理器时钟完成一个数据传输。

(2) $\overline{BLAST}$ ：突发最后输出信号，低电平有效。当 $\overline{BLAST}$ 有效时，表示下一个 $\overline{BLAST}$ 信号输入时，突发周期已经结束，即最后一个 $\overline{BLAST}$ 被看做 $\overline{READY}$ 。

9. 数据出错报告

- (1) $\overline{\text{FERR}}$ ：浮点出错输出信号，低电平有效。每当遇到未屏蔽的浮点出错时， $\overline{\text{FERR}}$ 信号就变为有效。
- (2) $\overline{\text{IGNNE}}$ ：忽略数字出错信号，输入信号，低电平有效。 $\overline{\text{IGNNE}}$ 信号有效时，将使 80486 忽略数字错误，继续执行非控制的浮点指令。如果 $\overline{\text{IGNNE}}$ 变为无效，而先前的浮点指令产生了错误，80486 将冻结在这个非控制的浮点指令上。控制寄存器 0(CR₀)中的 NE 位置位时， $\overline{\text{IGNNE}}$ 信号将不起作用。

10. 数据奇偶校验

- DP₀~DP₃ 为数据奇偶校验输入/输出信号，分别对应 32 位数据中的字节 0~字节 3。
- 偶校验指在一个数据字节的 8 条线上和对应奇偶校验输入/输出线上 1 的个数为偶数。
- 奇校验指在一个数据字节的 8 条线上和对应奇偶校验输入/输出线上 1 的个数为奇数。
- (1) DP₀~DP₃ 与数据字节具有相同的定时。
- (2) $\overline{\text{PCHK}}$ ：奇偶校验错信号，输出信号，低电平有效。 $\overline{\text{PCHK}}$ 为低电平时，CPU 在上一个读周期采样的数据存在奇偶校验错。也就是说，CPU 在读周期结束时采样的数据存在奇偶校验错时，在下一时钟开始，立即输出 $\overline{\text{PCHK}}=0$ 信号。 $\overline{\text{PCHK}}$ 信号不会被悬浮。

11. 时钟

CLK：时钟输入信号，为 80486 提供基本的定时和内部工作频率。根据 CLK 上升沿规定了所有外部定时参数。

5.6 Pentium 微处理器的引脚功能简介

Pentium 芯片有 168 个引脚，可分为三大类：处理器控制引脚、总线接口引脚和调试、测试引脚，现列表简述如下。

1. 处理器控制信号

处理器控制信号有 25 个，见表 5-11。

表 5-11 处理器控制信号

类 型	信 号	功 能	方 向
时钟	CLK	时钟	I
初始化	RESET	复位	I
	INIT	初始化	I
FRC	$\overline{\text{FRCM}}$	功能与检查方式	O
	$\overline{\text{IERR}}$	内部出错	O
总线仲裁	HOLD	总线保持请求	I
	HLDA	总线保持响应	O
	BREG	总线请求	O
	$\overline{\text{BOFF}}$	总线释放	I

续表			
类 型	信 号	功 能	方 向
Cache 窥视	AHOLD	地址保持	I
	$\overline{\text{EADS}}$	有效外部地址	I
	$\overline{\text{FLUSH}}$	Cache 清洗	I
	$\overline{\text{HITM}}$	未命中	O
	$\overline{\text{HIT}}$	命中	O
	INV	无效	I
中断	INTR	可屏蔽中断请求	I
	NMI	不可屏蔽中断请求	I
执行跟踪	IU	U 流水线指令完成	O
	IV	V 流水线指令完成	O
	IBT	转移跟踪指令	O
数字出错	$\overline{\text{FERR}}$	浮点出错	O
	$\overline{\text{IGNNE}}$	忽略数字出错	I
系统管理	$\overline{\text{SMI}}$	系统管理中断	I
	$\overline{\text{SMIACT}}$	系统管理中断活跃	O
其他	$\overline{\text{A}}_{20}\text{M}$	第 20 位地址屏蔽	I

2. 总线接口信号

总线接口信号有 132 个，见表 5-12。

表 5-12 总线接口信号

类 型	信 号	功 能	方 向
地址信号	$\text{A}_{35}\sim\text{A}_3$	地址总线	O
	AP	地址奇偶校验	O
	$\overline{\text{APCHK}}$	地址奇偶校验出错	O
	$\overline{\text{BE}}_3\sim\overline{\text{BE}}_0$	字节允许	O
数据信号	$\text{D}_{63}\sim\text{D}_0$	数据总线	I/O
	$\text{DP}_7\sim\text{DP}_0$	数据奇偶校验	I/O
	$\overline{\text{PEN}}$	数据奇偶校验允许	I
	$\overline{\text{PCHK}}$	数据奇偶校验出错	O
	$\overline{\text{BUSCHK}}$	总线检查	O
总线周期 控制信号	$\overline{\text{ADS}}$	地址状态	O
	$\overline{\text{BRDY}}$	猝发就绪	O
	$\text{D}/\overline{\text{C}}$	数据/控制	O
	$\text{W}/\overline{\text{R}}$	写/读	O
	$\text{M}/\overline{\text{IO}}$	存储器 I/O	O
	$\overline{\text{SCYC}}$	分离周期	O
	$\overline{\text{CACHE}}$	总线周期长度	O
	LOCK	总线封锁	O

续表			
类 型	信 号	功 能	方 向
Cache 信号	PWT	页面通写	O
	PCD	页面 Cache 禁止	I
	$\overline{\text{KEN}}$	Cache 允许	I
	$\overline{\text{NA}}$	下一地址	I
	WB/ $\overline{\text{WT}}$	回写/通写	O
	$\overline{\text{EWBE}}$	外部写缓冲器空	I

3. 调试和测试信号

调试和测试信号有 10 个，见表 5-13。

表 5-13 调试和测试信号

类 型	信 号	功 能	方 向
探针方式	R/ $\overline{\text{S}}$	进入或退出探针方式	I
	PRDY	探针方式就绪	O
断点/性能监测	PM ₀ /BP ₀	性能监测 0/断点 0	O
	PM ₁ /BP ₁	性能监测 1/断点 1	O
	BP ₃ ~BP ₂	断点 3~断点 2	O
边界扫描	TCK	测试时钟	I
	TDI	测试数据输入	I
	TDO	测试数据输出	O
	TMS	测试方式选择	I
	$\overline{\text{TRST}}$	测试复位	I

5.7 小 结

本章首先介绍了 8086 CPU 的引脚功能、工作模式、最小模式引脚和最大模式引脚。8088 CPU 的引脚功能，8086 CPU 和 8088 CPU 引脚和异同。然后介绍了 8086 CPU 的总线操作和时序，8086 CPU 的系统配置包括 8086 CPU 最小模式引脚的系统配置和 8086 CPU 最大模式引脚的系统配置，以及系统配置下各种芯片的使用，80386 和 80486 CPU 的引脚功能，Pentium 微处理器的引脚功能简介。正确理解 80 X 86 CPU 的引脚和功能，对 80 X 86 CPU 的实际应用有重要意义。

5.8 习 题

- 1. 试说明 8086 引脚信号中 M/ $\overline{\text{IO}}$ 、DT/ $\overline{\text{R}}$ 、 $\overline{\text{RD}}$ 、 $\overline{\text{WR}}$ 、ALE 的作用？
- 2. 8086 的 $\overline{\text{BHE}}$ 引脚有何作用？为什么 8088 无此引脚？

3. 8086 与 8088 有何不同之处?
4. 什么是总线周期? 什么是指令周期?
5. 试说明 8086 的最大模式和最小模式下系统基本配置的差别?
6. 8086 的读周期时序与写周期时序的区别有哪些?
7. 8086/8088 系统中, CPU 的哪个引脚用于与慢速外设的同步?
8. 试说明 8284A 芯片的功能。
9. 8086/8088 系统中, 为何需要地址锁存? 用何种芯片实现?
10. 8086/8088 系统中, 为何需要数据收发? 用何种芯片实现? 需用 CPU 的哪些引脚配合?
11. 试说明 8288 芯片的功能。
12. 试说明 80486 引脚相对与 80386 来说明, 有哪些新增的功能和特点。

第6章 总线技术

教学提示：本章主要介绍总线的概念、总线标准的特性、总线分类、设计总线应考虑的因素，以及微型计算机常用的总线标准。

教学要求：通过本章的学习，学生应该掌握总线的概念，总线标准的特性，总线分类，设计总线应考虑的因素，微型计算机常用总线标准，PC 总线，ISA 与 EISA 总线，PCI 总线，STD 总线，USB 总线，了解局部总线。

6.1 总线概述

微型计算机系统是由微处理器芯片与各部件和外部设备组成的，它们之间要相互进行信息或数据的交换，就必须建立信息传输的通路，总线就是这个通路中许多信号线的集合，它由系统中各个部件所共享。

6.1.1 总线的概念

总线是一种数据通道，是在部件与部件之间、设备与设备之间传送信息的一组公用信号线。在主导设备(部件和设备)的控制下，将发送设备(部件和设备)发出的信息准确地传送给某个接收设备(部件和设备)。总线的特点在于其公用性，即它可同时挂接多个部件或设备。如果是某两个部件或设备之间专用的信号连线，就不能称之为总线。所以，总线是连接计算机硬件系统内部多种设备的通信线路。总线的一个很重要的特征是传输媒质由总线上的所有部件所共享，可以将计算机系统内的多种部件连接到总线上。

总线上任何一个部件发出的信息，计算机系统内所有连接到总线上的部件都可以接收到；但在进行信息传输时，每一次只能有一个叫做主导设备的部件可以利用总线给一个叫做从属设备的部件发送信息。

通常总线是由多条通信路径或线路组成的，而每一条信号线仅能传送二进制的 0 或 1 信号。在一段时间里，一条信号线就能传送一串的二进制信息，将几条信号线组合在一起，这样总线就可以在同一时间并行地传输二进制信息，如一个字节信息就可以通过总线中的 8 条信号线完成信息的传输。

计算机系统含有多种总线，计算机系统内各个层次之间的信息传送就是由总线来完成的。

可以形象地将总线看做是一条以微处理器为出发点的高速公路，总线的宽度(数据位数)可视为高速公路上车辆通道的数目，而各个部件或设备就像一个个车站，在总线上收发数据恰似在接发车辆。总线作为所有模块或设备共同使用的“公路”(Highway)，每个部件或设备都通过门电路与总线中相应的信号线相连。作为发送器的部件或设备，可以通过驱动器把输出的信号送到总线中相应信号线上传送；而作为接收器的部件或设备，则在适当时刻打开接收总线信号的缓冲器或寄存器，把总线中相应信号线上传送的信号接收进来。

6.1.2 总线标准的特性

在各插件板插座之间采用系统总线连接的情况下，希望接插件之间具有通用性，以便一个系统的各接插件板能够插在任何一个插座上，为用户的安装和使用带来方便。此外，微型计算机厂商在设计一个系统时，为了得到广泛的市场，设法使系统总线能够连接尽可能多的设备，希望各厂家生产的同类产品能够互联、互换，这就需要有一个规格化的可通用的系统总线，这种总线的设计涉及统一的标准，对每个总线标准应有详细的说明，包含下列内容。

1. 物理特性

物理特性指的是总线物理连接的方式。规定模块尺寸、总线插头、插座和边沿连接器等规格和位置。包括总线的条数，外引线是如何排列的等。例如，IBM PC/XT 机的总线共 62 条线，分两排编号。当插件板插到槽中后，左面是 B 面，外引线排列顺序是 $B_1 \sim B_{31}$ ，右面是 A 面，外引线排列顺序是 $A_1 \sim A_{31}$ ，A 面是元件面。

2. 功能特性

功能特性规定每个引脚信号的名称和功能，对它们相互作用的协议进行说明。从功能上，总线可分成 3 类：地址总线、数据总线和控制总线。地址总线的宽度指明了总线能够直接访问存储器的地址范围；数据总线的宽度指明了访问一次存储器或外部设备最多能够交换数据的位数；控制总线一般包括 CPU 与外界联系的各种控制命令，如输入/输出读/写信号、存储器读/写信号、外部设备与主机同步匹配信号、中断信号和 DMA 控制信号等。例如，IBM PC/XT 系统总线的功能分为：地址总线 20 条，编号为 $A_0 \sim A_{19}$ ，可以访问 1MB 的存储空间；数据总线 8 条，编号 $D_0 \sim D_7$ ，主机与存储器或 I/O 设备每次只能交换一个字节的信息；XT 总线提供四种电源线，分别是 +5V、-5V、+12V、-12V，与地址一起共占用 8 条外引线；剩下的 26 条线，全为控制总线。

3. 电气特性

电气特性规定信号工作时的传递方向、高低电平、动态转换时间、负载能力及最大额定值。一般规定送入 CPU 的信号叫输入信号 IN，从 CPU 送出的信号叫输出信号 OUT。例如，XT 总线的地址线 $A_0 \sim A_{19}$ 为输出线；数据线 $D_0 \sim D_7$ 为双向信号线，既作为数据输入线，又作为数据输出线，地址线和数据线都是高电平有效；控制线 $\overline{IOR}$ ，是输入设备读信号线，低电平有效等。

4. 时间特性

对于存储器、I/O 的读写操作，时间特性规定相应的总线信号时序，在总线中定义这些信号的时序以保证各功能板的兼容性。也就是说，用户什么时间可以用总线传输信号，或者用户什么时候把信号提供给总线，CPU 才能正确无误地使用。

6.1.3 总线的分类

在计算机系统内拥有多种总线，他们在计算机系统内的各个层次上，为各部件之间的

通信提供通路。按在系统的不同层次、位置，总线可分为如下几类。

1. 片内总线

片内总线是连接集成电路芯片内部各功能单元的信息通路。例如，CPU 芯片中的内部总线，它是 ALU 寄存器和控制器之间的信息通路。片内总线根据其功能又被分为地址总线、数据总线和控制总线。随着微电子学的发展，出现了 ASIC 技术，用户通过掌握芯片内总线技术，采用 CAD 技术设计专用芯片。

2. 主板局部总线

主板局部总线是指在印刷电路板上连接各插件的公共通路。例如，CPU 及其支持芯片与其局部资源之间的通道即属于主板局部总线。这些资源可以是主板资源，也可以是插在主板局部总线扩展槽上的功能扩展板上的资源。目前，PC 系列机上比较流行的标准主板局部总线有 ISA、EISA、VESA、PCI 以及 AGP 等标准总线。

3. 系统总线

系统总线又称为内总线，这是指模块式微处理机机箱内的底板总线，用来连接构成微处理机的各插件板。它可以是多处理机系统中各 CPU 板之间的通信通道，也可以用来扩展某块 CPU 板的局部资源或为总线上所有 CPU 板扩展共享资源之间的通信通道。系统总线对微处理机设计者和微处理机应用系统的用户来说，都是一种很重要的总线。选择具有标准化和开放性总线的计算机系统，通过适当选择各种模块，构成符合要求的计算机系统，是当前选择微处理机应用系统的趋势。现在较流行的标准化微处理机系统总线有：16 位的 MULTIBUS I、STDBUS 和 32 位的 MULTIBUS II、STD32 和 VME 等。

4. 通信总线

它是微机系统与外部系统、微机系统与其他仪器仪表或设备之间的连线。一般来说，外部系统和设备与微机系统的通信联系，可以采用并行方式或串行方式来实现。因此，外部总线既有并行总线，也有串行总线。外部总线的数据传输速率通常较低(与内部总线相比)。常见的外部总线标准有 RS-232、IEEE-488、SCSI、CENTRONIC 总线等。

(1) 并行总线：计算机中的信息一般都是由多位二进制码表示的，传输这些信息时，可用多条线同时传送所有二进制位。并行总线内各条连线之间实行有序排列，并实行统一编号。

此类总线利用空间实现全部信息的一次传输，虽然可使系统的结构比较复杂，但是快速传输信息。这个优点在处理机速度不断提高的今天具有十分重要的意义。而所增加的结构复杂度，对于各部件分布距离并不很远的计算机内部来说是不难解决的，所以，并行总线被大量地用在计算机内部各部件的连接中。

(2) 串行总线：这是一种与并行总线不同的总线类型，它以多位二进制信息共用一条线进行传输的方式工作。既然是共用，就只能让信息位按一定的次序排队，按时间先后依次通过总线。很显然，如果所传送的信息有 m 位，串行方法传送所需的时间至少是并行方法的 m 倍。这种总线形式具有结构简单的优点，适合当所需连接的部件距离比较远时的情况。

5. 总线构成

虽然总线有多种，一般包括数据总线、地址总线和控制总线。

1) 数据总线

所谓数据总线，顾名思义就是在计算机系统各部件之间传输数据的路径。通常，数据总线是由 8 条、16 条或 32 条数据线组成，这些数据线的条数称之为数据总线的宽度。由于每一条数据线每一次均能传送二进制的 1 位，所以数据线的条数就决定了每一次能同时传送的二进制的位数，由此可以看出，数据总线的宽度是表现系统总体性能的最关键的因素之一。例如，如果数据总线的宽度为 8 位，而每条指令的长度为 16 位，在每个指令周期就需要两次去存储器取指令。由于 Pentium 数据总线的宽度为 64 位，就大大加快了对存储器的存取速度。

2) 地址总线

所谓地址总线，是用来规定数据总线上的数据出于何处和被送往何处。若 CPU 欲从存储器读取一个信息，不论这个信息是 8 位、16 位、32 位或 64 位的，均是先将欲取信息的地址放到地址总线上，然后才可以从给定的存储器地址那里取出所需信息。

地址总线的宽度决定了计算机系统能够使用的最大的存储器容量。另一方面，在对输入/输出端口进行寻址时，也要用地址总线传送地址信息。在实际操作时，总是用地址总线的高端几位选择总线上指定的存储器段(或者说是存储器区)，而用地址总线的低端几位去选择存储器段内的具体存储单元或输入/输出端口地址。

3) 控制总线

控制总线的作用是用来对数据总线、地址总线的访问及其使用情况实施控制。由于计算机中的所有部件均要使用数据总线和地址总线，所以用控制总线对它们实施控制是必要的，也是必需的。说到底，控制信号的作用就是在计算机系统各个部件之间，发送操作命令和定时信息。命令信息规定了要执行的具体操作；而定时信息则规定了数据信息和地址信息的有效性。

通常，控制信号应有以下几种类型：

(1) 写存储器命令：在写存储器命令的控制之下，将数据总线上的数据写入指定的存储单元。

(2) 读存储器命令：在读存储器命令的控制之下，将从指定的存储单元中取出的数据放到总线上。

(3) 输入/输出写命令：在输入/输出写命令的控制之下，将总线上的数据输出到指定的输入/输出端口。

(4) 输入/输出读命令：在输入/输出读命令的控制之下，将指定的输入/输出端口上的数据存放到数据总线上。

(5) 传送响应：表示数据已经被接收或已经把数据放到总线上了的应答信号。

(6) 总线请求：表示计算机系统内的某一个部件急于获得对总线的控制权的信号。

(7) 总线允许：表示计算机系统内的某一个发出请求的部件已经获准对总线的控制。

(8) 中断请求：表示计算机系统内的某一中断源发出欲中断的申请信号。

(9) 中断响应：表示计算机系统内的某一中断源发出的中断申请信号已获得响应。

(10) 时钟和复位：在进行同步操作时，要用时钟进行同步；进行初始化操作时，用复位命令。

6.1.4 设计总线应考虑的因素

1. 通用性

采用总线标准，可以为多个模块的互联提供一个标准的界面，该界面对于界面两端的模块来说是透明的，界面的任一方只需根据总线标准设计和实现接口的功能，不需考虑另一方的接口方式，所以按标准设计的接口具有广泛的通用性。

2. 便于用户的二次开发

由于采用同样的总线设计标准和制造各种功能的模块板，各模块板是挂在总线上的相互独立的模块，用户可以根据需要选用现成的功能模块板，或自行设计符合总线标准的模块板，这样可节省硬件的重复性开发，也使得编写该模块的相应软件更容易，给调试和修改带来许多方便，从而提高效率，降低成本。

3. 便于系统的更新

随着集成电路技术的发展，新的芯片不断产生，微机系统需要不断更新。采用统一标准的总线结构对于系统的更新更加方便。

4. 可不断提高系统的功能

由于采用了标准总线，将计算机的典型功能部分做成不同的模块板挂在总线上，为系统在原设计的基础上以最小的变动来跟随市场的要求提供了可能。只要根据新的要求，设计新的模块，插在总线上就可以了。

6.2 微型计算机常用总线标准

6.2.1 PC 总线

PC 机的系统总线用于系统功能的扩充，它可以进行存储器扩充和各种外部设备适配器的扩充。例如打印机适配器、显示器适配器、网络适配器、语音系统适配器等。扩充插槽有 62 个引脚，62 芯总线中包括 8 位双向数据总线、20 位地址线、6 级中断请求信号线、3 组 DMA 通道控制线、存储器和 I/O 读写线、动态 RAM 刷新控制线和时钟信号线，此外，还有 4 根电源线和占有 3 个插脚的地线，由此送出四组电源： $+12V$ 、 $+5V$ 、 $-5V$ 、 $-12V$ 。IBM 对插槽上的信号名称、性质、方向、时序、引脚排列等都作了明确规定。这种规定称为 IBM PC 总线。

1. 引脚分配

PC 总线的 62 个引脚的具体分配见表 6-1。

2. 信号说明

$A_{19} \sim A_0$ ：地址总线，输出。用于寻址与系统总线相连的存储器和 I/O 设备。

$D_7 \sim D_0$ ：数据总线，双向。用于 CPU 与存储器、I/O 设备之间传送数据信息。

表 6-1 PC 总线信号分配表

元 件 面			焊 接 面		
引脚号	信号名	说 明	引脚号	信号名	说 明
A ₁	$\overline{\text{IO CHCK}}$	输入, IO 校验 数据信号, 双向	B ₁	GND	地
A ₂	D ₇		B ₂	RESETDRY	复位
A ₃	D ₆		B ₃	+5V	电源+5V
A ₄	D ₅		B ₄	IRQ ₂	中断请求 2, 输入
A ₅	D ₄		B ₅	-5V	电源-5V
A ₆	D ₃		B ₆	DRQ ₂	DMA 通道 2 请求, 输入
A ₇	D ₂		B ₇	-12V	
A ₈	D ₁		B ₈	CARD SLCTD	见表注
A ₉	D ₀	输入, IO 准备好 输出, 地址允许 地址信号, 双向	B ₉	+12V	电源+12V
A ₁₀	IO CHRDY		B ₁₀	GND	地
A ₁₁	AEN		B ₁₁	$\overline{\text{MEMW}}$	存储器写, 输出
A ₁₂	A ₁₉		B ₁₂	$\overline{\text{MEMR}}$	存储器读, 输出
A ₁₃	A ₁₈		B ₁₃	$\overline{\text{IOW}}$	接口写, 双向
A ₁₄	A ₁₇		B ₁₄	$\overline{\text{IOR}}$	接口读, 双向
A ₁₅	A ₁₆		B ₁₅	$\overline{\text{DACK}}_3$	DMA 通道 3 响应, 输出
A ₁₆	A ₁₅		B ₁₆	DRQ ₃	DMA 通道 3 请求, 输入
A ₁₇	A ₁₄		B ₁₇	$\overline{\text{DACK}}_1$	DMA 通道 1 响应, 输出
A ₁₈	A ₁₃		B ₁₈	DRQ ₁	DMA 通道 1 请求, 输入
A ₁₉	A ₁₂		B ₁₉	$\overline{\text{DACK}}_0$	DMA 通道 0 响应, 输出
A ₂₀	A ₁₁		B ₂₀	CLK	系统时钟, 输出
A ₂₁	A ₁₀		B ₂₁	IRQ ₇	中断请求, 输入
A ₂₂	A ₉		B ₂₂	IRQ ₆	
A ₂₃	A ₈		B ₂₃	IRQ ₅	
A ₂₄	A ₇		B ₂₄	IRQ ₄	
A ₂₅	A ₆		B ₂₅	IRQ ₃	
A ₂₆	A ₅		B ₂₆	$\overline{\text{DACK}}_2$	DMA 通道 2 响应, 输出
A ₂₇	A ₄		B ₂₇	T/C	计数终点信号, 输出
A ₂₈	A ₃		B ₂₈	ALE	地址锁存信号, 输出
A ₂₉	A ₂		B ₂₉	+5V	+5V 电源
A ₃₀	A ₁		B ₃₀	OSC	振荡信号, 输出
A ₃₁	A ₀		B ₃₁	GND	地

注: (1) B₈ 引脚对插槽 1~7 为空, 插槽 8 上该信号为选中信号 $\overline{\text{CARD SLCTD}}$ 。

(2) $\overline{\text{IOCHCK}}$: 信号名称, 表示低电平有效。

OSC: 振荡器信号, 输出。频率为 14.318MHz, 周期约为 70ns, 占空比为 50%, 该信号是总线上频率最高的信号, 其余时钟信号均是由该信号分频产生的。

CLK: 时钟信号, 输出。该信号是由上述 OSC 经 3 分频后得到, 频率为 4.77MHz。

RESETDRY: 复位驱动信号, 输出, 高电平有效。它是 8284 时钟发生器的 RESET 信号经两级反相门驱动后输出, 当系统加电时, 该信号为高, 当所有电源达到规定幅度时, 即出现电源好(POWERGOOD)信号时, 该信号变低。

ALE: 地址锁存允许, 输出, 高电平有效。该信号由 8288 总线控制器提供, ALE 有效, 表明一个总线周期的开始。

$\overline{\text{I/OCHCK}}$: I/O 通道校验信号, 输入, 低电平有效。当 $\overline{\text{I/OCHCK}}=0$ 时, 表明扩充槽上的存储器或 I/O 设备出现了奇偶校验错, 并且会向 CPU 发出非屏蔽中断请求(NMI)。

$\overline{\text{I/OCHRDY}}$: I/O 通道准备就绪信号, 输入, 高电平有效, 当总线上的存储器或 I/O 设备的速度较慢时, 可利用该信号的低电平通知 CPU(或 DMA 控制器), 以便插入适当的等待状态, 从而使快速的 CPU 与慢速的外设实现同步。

$\text{IRQ}_7 \sim \text{IRQ}_2$: 中断请求信号, 输入。它们是 I/O 设备向 CPU 发出的中断请求信号, 该信号上升沿有效, 并要求其高电平一直保持到 CPU 响应中断为止。在 $\text{IRQ}_7 \sim \text{IRQ}_2$ 中, IRQ_2 中断优先级最高, 其余依次降低, IRQ_7 优先级最低。

$\overline{\text{IOR}}$: I/O 读信号, 输出, 低电平有效。

$\overline{\text{IOW}}$: I/O 写信号, 输出, 低电平有效。

$\overline{\text{MEMR}}$: 存储器读信号, 输出, 低电平有效, 用来将选中的存储单元内容读到数据总线上。

$\overline{\text{MEMW}}$: 存储器写信号, 输出, 低电平有效, 用来将来自数据总线上的数据写入存储单元。

$\text{DRQ}_3 \sim \text{DRQ}_1$: DMA 请求信号, 输入, 高电平有效。要求高电平必须保持到响应为止。这 3 个 DMA 请求信号的优先级为 DRQ_1 最高, 依次降低, DRQ_3 最低。

$\overline{\text{DACK}}_3 \sim \overline{\text{DACK}}_0$: DMA 响应信号。其中, $\overline{\text{DACK}}_3 \sim \overline{\text{DACK}}_1$ 分别是 $\text{DRQ}_3 \sim \text{DRQ}_1$ 的响应信号。 $\overline{\text{DACK}}_0$ 用作动态存储器刷新的信号。

AEN: 地址允许信号, 输出, 高电平有效。它由 DMA 控制器发出, 用来切断 CPU 对总线的控制, 允许总线上进行 DMA 传送。AEN=0, 由 CPU 控制总线; AEN=1, 由底板上的 DMA 控制器控制总线。

T/C: 计数结束信号, 输出, 高电平有效。由 DMA 控制器产生, T/C=1, 表示 DMA 的某一通道计数达到某个规定的字节数。

CARD SLCTD: 插件卡选中信号, 输入, 低电平有效。该信号只在第 8 个插槽上使用, 其他 7 个插槽上该信号空着不用。它是由该插件卡向 CPU 发出的已选中信号。当它有效时, 表明 CPU 可对该卡进行读写操作。

+5V: +5V 直流供电线, 两条引脚, 可供插卡使用, 电压变化范围为+4.75~+5.25V。

+12V: +12V 直流供电线, 电压变化范围为+11.4~+12.6V。

-5V: -5V 直流供电线, 电压变化范围为-4.5~-5.6V。

-12V: -12V 直流供电线, 电压变化范围为-10.8~-13.2V。

GND: 直流地线, 即直流电源和机架的接地线, 共 3 条。

6.2.2 ISA 与 EISA 总线

1. ISA 总线

ISA 总线是 IBM/AT 机上使用的总线，又称为 PC AT 总线，是为采用 80286 CPU 设计的。这种总线 IBM PC 总线一样，也是一种原始的总线设计，将微处理器芯片总线经缓冲直接接到系统总线，是在 PC 总线的基础上增加一个 36 插脚的 AT 插槽而形成的。人们利用兼容的方式将原来的 8 位 PC 总线扩展成 16 位的 AT 总线，设计制造出了 PC/AT 微型计算机。

1) 引脚分配

ISA 总线中新增加的 36 个引脚信号的分配见表 6-2。

表 6-2 ISA 总线 36 引脚插槽信号分配表

元 件 面			焊 接 面		
引脚号	信号名	说 明	引脚号	信号名	说 明
C ₁	$\overline{\text{SBHE}}$	高字节允许，双向	D ₁	$\overline{\text{MEMCS16}}$	存储器 16 位选片信号，输入
C ₂	A ₂₃	高位地址，双向	D ₂	$\overline{\text{I/OCS16}}$	接口 16 位选片信号，输入
C ₃	A ₂₂		D ₃	IRQ ₁₀	中断请求，输入
C ₄	A ₂₁		D ₄	IRQ ₁₁	
C ₅	A ₂₀		D ₅	IRQ ₁₂	
C ₆	A ₁₉		D ₆	IRQ ₁₄	
C ₇	A ₁₈		D ₇	IRQ ₁₅	
C ₈	A ₁₇		D ₈	$\overline{\text{DACK}}_0$	DMA 请求与响应信号，前者输入，后者输出
C ₉	$\overline{\text{SMEMR}}$	存储器读，双向	D ₉	DRQ ₀	
C ₁₀	$\overline{\text{SMESMW}}$	存储器写，双向	D ₁₀	$\overline{\text{DACK}}_3$	
C ₁₁	D ₈	数据总线高字节，双向	D ₁₁	DRQ ₅	
C ₁₂	D ₉		D ₁₂	$\overline{\text{DACK}}_6$	
C ₁₃	D ₁₀		D ₁₃	DRQ ₆	
C ₁₄	D ₁₁		D ₁₄	$\overline{\text{DACK}}_7$	
C ₁₅	D ₁₂		D ₁₅	DRQ ₇	
C ₁₆	D ₁₃		D ₁₆	+5V	+5V 电源入
C ₁₇	D ₁₄		D ₁₇	MASTER	主控，输入
C ₁₈	D ₁₅		D ₁₈	GND	地

2) 信号说明

A₂₃~A₂₀: 4 条高位地址线，使寻址范围由原来的 1MB 扩展到 16MB。同时将 PC 总线上的 A₁₉~A₁₇ 从复用引脚分离出来，以便提高传输速率。

D₁₅~D₈: 8 条高位数据线。

$\overline{\text{SBHE}}$: 数据总线高位字节允许信号，双向，低电平有效。该信号与其他信号一起实现对存储器的高字节、低字节或字的操作。

$\overline{\text{IRQ}}_{10} \sim \overline{\text{IRQ}}_{15}$: 中断请求输入信号。由于 AT 总线增加了外中断的数量, 在底板上用两片中断控制器 8259 级联实现中断优先级。

$\overline{\text{DRQ}}_7 \sim \overline{\text{DRQ}}_5, \overline{\text{DRQ}}_0$: DMA 请求信号, 为增加 AT 总线 DMA 传输能力, 在底板上用两片 DMA 控制器 8237 级联。

$\overline{\text{DACK}}_7 \sim \overline{\text{DACK}}_5, \overline{\text{DACK}}_0$: DMA 响应信号, 输出, 低电平有效。

$\overline{\text{SMEMR}}$: 存储器读, 输出, 低电平有效, 在 16MB 寻址范围内均有效。而 PC 总线中 $\overline{\text{MEMR}}$ 只有在存储器寻址范围小于 1MB 才有效。

$\overline{\text{SMEMW}}$: 存储器写, 输出, 低电平有效, 在 16MB 寻址范围内均有效。

$\overline{\text{MASTER}}$: 主控信号, 输入, 低电平有效, 利用该信号, 可以使总线插板上设备变为主控器, 用来控制总线上的各种操作。

$\overline{\text{MEMCS}}_{16}$: 16 位存储器选片信号, 输入, 低电平有效, 如果总线上的某一存储器要传送 16 位数据, 则必须产生一个有效的 $\overline{\text{MEMCS}}_{16}$ 信号来通知主板, 实现 16 位数据传送。

$\overline{\text{I/O CS}}_{16}$: 16 位 I/O 接口选片信号, 输入, 低电平有效, 该信号用来通知主板实现 16 位接口数据传送。

+5V, GND: 是 +5V 电源和地线。

2. EISA 总线

EISA(Extended ISA)总线是 ISA 总线(AT 总线)的扩展。EISA 在 ISA 的基础上, 将数据总线宽度从 16 位变为 32 位, 地址总线由 24 位变为 32 位, 并具有高速同步传送功能。EISA 总线保留了 ISA 总线原有的全部信号线, 同 ISA 向上兼容的总线结构, 在 AT 总线上工作的适配器卡仍可以在 EISA 总线上工作, 但 EISA 总线并不只是 ISA 总线简单意义上的扩展。

从硬件上看, ISA 总线是一个单用户结构, 缺乏智能成分。ISA 总线的 8 个插槽共用一个 DMA 请求, 只要有一个设备占用, 其余的只好等待。ISA 总线也没有提供中断共享, 两级 8259 中断控制器提供 15 个中断, 其中的一些被固定分配给了一些特定的设备, 因此在配置时常会发生中断冲突。另外, 在总线 I/O 过程的实现中, 只有在一个 I/O 过程完成后, 另一个 I/O 过程才能继续。这些显然都是与多用户相矛盾的。EISA 总线不仅仅在数据总线和地址总线的宽度上扩展到 32 位, 而且每个扩展槽均有各自的 DMA 请求线, 可按约定的级别占用 DMA; EISA 还提供中断共享, 允许用户配置多个设备共享一个中断; EISA 主要解决了总线 I/O 的控制能力, 着重解决了硬盘子系统的多用户访问速度和数据的快速传递, 因此是一种适合于多用户系统的总线。EISA 总线比 ISA 总线要复杂的多, 价格也比较贵。

EISA 总线采用的一项重要技术是总线主控技术, 这也是其他高性能现代总线所采用的一项新技术。

在具有总线结构的微机系统中, CPU 和外设均可对总线进行控制。控制总线当前操作的设备称为总线主控器, 另一方则称为从控制器。在早期的 I/O 总线结构中, 总是由 CPU 充当主控器, 其他外设皆为从控制器, 除 DMA 操作外, CPU 控制全部的总线事务。在现代总线结构中, 采用一个独立的器件作为总线主控器。在一个 EISA 系统中, 总线主控器最多可以是 15 个。总线主控器是一种典型的智能器件, 实质上是一种连在系统上的微处理器,

具有不用 CPU 干预便可独立控制总线的能力。总线主控制器在系统中用于处理特定任务,典型的专用总线主控制器执行诸如图像和网络的控制这类高性能任务;也可作为通用处理器,其作用类似于 CPU,其目的是使整个系统的工作可被分开进行。配合 EISA 总线的提出,Intel 公司推出了系统化的符合 EISA 标准的配套芯片,如早期的 82350 芯片系列和后来推出的 82350 DT 系列,其中包括 82352 EBB(EISA Bus Buffers, EISA 总线缓冲器),82355 BMIC (Bus Master Interface Controller, 总线主设备接口控制器),82357 ISP(Integrate System Peripheral, 集成系统外设接口)和 82358 EBC(EISA Bus Controller, EISA 总线控制器)等。

系统总线通过在总线控制电路中设置一个总线仲裁机构来有效地支持多个总线主控制器。当多个总线主控制器同时竞争总线控制权时,仲裁器便根据设定的优先级进行仲裁。仲裁电路也叫做总线判优控制电路。EISA 采用的集中型判优控制电路,82357 ISP 芯片中包含有这种电路。

1) EISA 总线具有的主要特点

(1) 支持 CPU、DMA、总线主控制器 32 位寻址能力和 16 位数据传输能力,具有数据宽度变换功能。

(2) 扩展及增加 DMA 仲裁能力,使得 DMA 的传输速率最高可达 33MB/s。

(3) 程序可以采用边沿或电平方式控制中断的触发。

(4) 能够通过软件实现系统板和扩展板的自动配置功能。

(5) 规定总线裁决采用集中方式进行,使得 EISA 总线有效地支持构成多微处理器系统。

(6) 它与 PC/XT 总线相兼容。这就使得已大量开发的 PC/XT 总线的插件卡,方便地在 EISA 总线上运行。

2) 为了构成 EISA 总线,在 AT 总线上附加的主要信号

(1) $\overline{BE}_3 \sim \overline{BE}_0$: 字节允许信号。它们分别用来表示 32 位数据总线上的哪个字节与当前总线周期有关。

(2) $\overline{M}/\overline{IO}$: 存储器或接口指示。用该信号的不同电平来区分 EISA 总线上是内存周期还是 I/O 接口周期。

$\overline{START}$: 起始信号。用来表示 EISA 总线周期开始。

$\overline{CMD}$: 定时控制信号。在 EISA 总线周期中提供定时控制。

$\overline{LA}_{31} \sim \overline{LA}_2$: 地址总线信号。它们与 $\overline{BE}_3 \sim \overline{BE}_0$ 一起,共同决定 32 位地址的寻址空间,其范围可达 4GB。

$\overline{D}_{31} \sim \overline{D}_{16}$: 高 16 位数据总线。它们与原来 AT 总线上定义的 $\overline{D}_{15} \sim \overline{D}_0$ 共同构成 32 位数据总线。

$\overline{MIREQ}_n$: 主控制器请求信号。总线上主控制器希望得到总线时,发出该信号,用于请求得到总线控制权。

$\overline{MAK}_n$: 总线控制器指示信号,利用该信号表示第几个总线主控制器已获得总线控制权。

由于 EISA 总线性能优良,使得多数 386、486 微型计算机系统都采用了 EISA 总线。

6.2.3 PCI 总线

1. PCI 总线的主要特性

(1) 高性能。32 位总线宽度,可升级到 64 位;支持猝发工作方式;同步总线操作的工作

作频率可达到 33MHz, 数据传输速率可达到 132/264MB/s; 处理器/内存子系统能力完全一致。

(2) 通用性强, 适用面广。PCI 的设计不依赖于微机的 CPU, 不仅适用于现在的 PC 机, 而且适用于将来的机型, 不管其使用何种 CPU; 既适用于 5V 信号环境, 也适用于 3.3V 环境, 因此既可用于台式机, 也能用于笔记本式微机和膝上机; 支持 64 位地址。

(3) 低成本。这是因为 PCI 采用了最优化的芯片、标准的 ASIC 技术和其他处理技术相结合, 多路复用体系系统结构减少了管脚数和 PCI 部件等因素所致。此外, 在 ISA、EISA、MCA 等基本系统上的扩展板, 也可以在 PCI 系统中工作, 并由厂家提供 PCI 到 ISA、EISA 和 MCA 的转换, 既减少了用户的开发成本, 又避免了混乱。

(4) 使用方便、灵活。PCI 能够自动配置参数, 支持 PCI 总线扩展板和部件; PCI 设备包含配置寄存器, 可用来存放设备配置的信息; 多主控器允许任何 PCI 主设备和从设备之间进行点对点的访问; 共享槽口既可以插标准的 ISA、EISA、MCA 板, 也可以插 PCI 扩展板。此外, 还具有可靠性高、数据完整以及软件兼容, 即 PCI 部件和驱动程序可以在各种不同的平台上运行等特点。典型的 PCI 总线设计可支持至多 3 块扩增电路板。

2. PCI 总线的定义

PCI 总线分 A、B 面, 每面为 60 条引脚, 分前 49 条和后 11 条, 分界处的几何尺寸占两条引脚, 在 PCI 插槽上有一个限位缺口与之对应。引脚如图 6.1 所示。

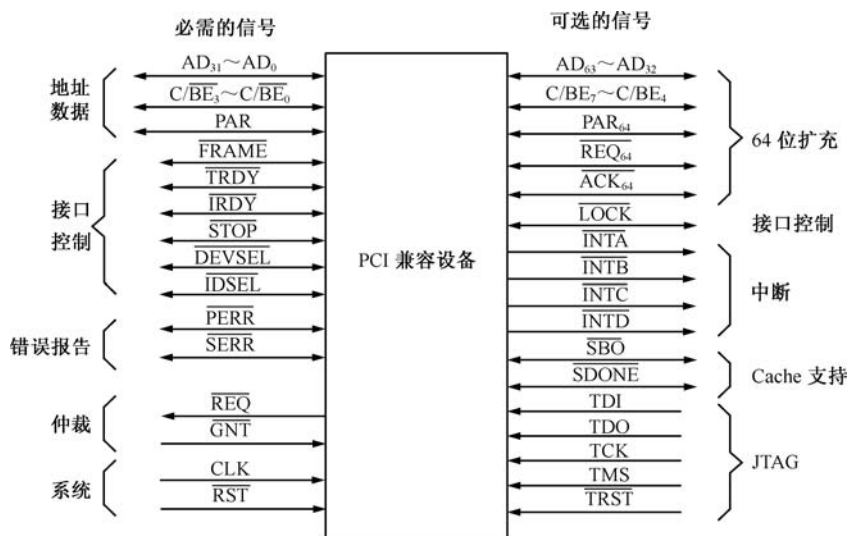


图 6.1 PCI 的外引线信号

3. PCI 总线系统结构

图 6.2 是一种使用 PCI 总线的系统结构。可以看出, 处理器、Cache、内存之间数据传输通过微处理器系统总线进行, 它的数据传输速率高于 PCI 总线。处理器、Cache、存储器子系统经过“Host-PCI”桥和 PCI 总线连接。这个桥提供了一个低延迟的访问通路, 使微处理器能够访问 PCI 设备, PCI 主设备也能访问主存。该桥还提供了数据缓冲功能, 使微

处理器与 PCI 总线上的设备并行工作而不必等待。同时, 这个桥电路包含了“PCI 总线控制器”, 有多个设备申请使用总线时, 由它进行裁决和分配总线的使用权。总之, “Host-PCI”桥实现了 PCI 总线的全部驱动控制, 它实际上是一个高速 I/O 协处理器, 主要解决 I/O 设备同处理器的连接问题。

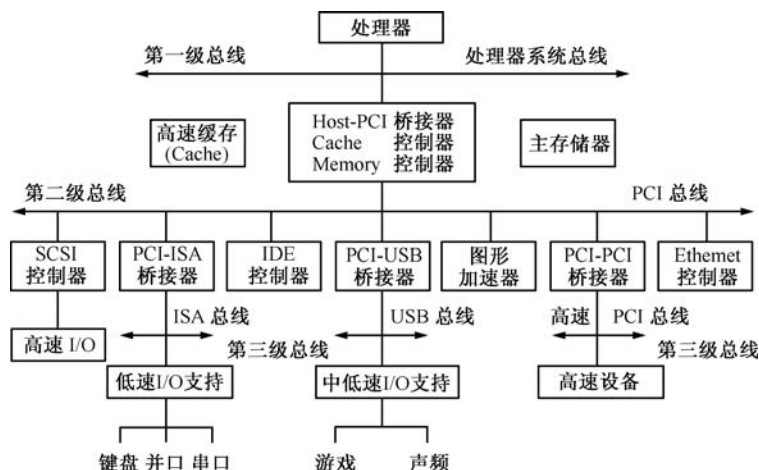


图 6.2 PCI 总线系统结构

另一类“桥”用于生成“多级总线”结构, 例如“PCI-ISA/EISA”、“PCI-USB”、“PCI-PCI”等。多级总线把不同传输速度、不同传输方式的设备分门别类地连接到各自“合适”的总线上, 使得不同类型的设备共存于一个系统中, 合理地分配资源, 协调地运转。

4. PCI 的地址空间

PCI 总线定义了 3 个物理地址空间: 内存地址空间、I/O 地址空间和配置地址空间。前两个属于常规范围, 第三个是 PCI 特有的, 它用于进行 PCI 的硬件资源配置。

PCI 总线的编址是分布式的, 每个设备都有自己的地址译码电路, 不需要进行统一译码。PCI 支持正向和负向两种类型的地址译码。所谓正向译码, 就是每个设备都监听地址总线, 判断访问地址是否落在它的地址范围, 如果是, 使 DEVSEL#有效以示应答, 响应速度较快。负向译码是指没有一个设备作出响应时, 有一个指定的设备(负向译码设备)作出响应。由于它要等到总线上其他所有设备都拒绝之后才能行动, 所以速度较慢。负向译码对于像标准扩展总线(PCI-ISA 扩展桥)这类设备是很有的。这是因为 ISA 总线设备的接口不具备发出响应信号 DEVSEL 的功能。

5. PCI 的配置空间

早期的 ISA 卡上有许多跳线和拨动开关, 用户用它们来设置所需的各种计算机资源(如地址空间、中断号、DMA 通道号等), 这要求用户熟知本机的所有资源使用情况, 一旦设置错误, 系统就不能正常工作。为此, Pentium 以上的处理器的 PC 机推出了由计算机自动配置各种资源的方法——既插即用(Plug and Play, P&P)。它要求每块支持 P&P 的板卡设置

一组称为配置空间的寄存器，这些寄存器中保存了该板卡对系统资源的要求。Windows 系统启动时，BIOS 程序读出这些参数，综合每块板卡对资源的要求，对系统资源进行统一分配。由此可见，PCI 设备的地址空间是由系统动态分配的，是浮动的。

目前在微机上使用的局部总线有 VISA 总线(简称 VL)总线和 PC 总线，用于对 ISA 和 EISA 总线进行补充，局部总线的主要特点是将外部设备直接连接到 CPU，没有通过锁存延时等电路，因此应用局部总线的系统具有非常快的运行速度。

6. PCI 总线信号

PCI 总线上的每一信号不是与电源相邻就是与地相邻，采用这一措施的目的，就是最大限度地减少噪声的干扰和信号的辐射。表 6-3 列出了每一信号的名称及其简单的说明。

表 6-3 PCI 总线信号说明

	信 号	名 称	类 型	说 明
32 位 部 分	AD0~AD31	地址/数据线	输入/输出	PCI32 位多路复用地址和数据总线
	$\overline{C}/\overline{BE}_0 \sim \overline{C}/\overline{BE}_3$	命令/字节允许	输入/输出	总线命令/字节允许信号在地址传送节拍，传送 PCI 总线命令，在数据传送节拍传送字节允许信号， $\overline{BE}_0$ 对应字节 0
	CLK	时钟	输出	PCI 的时钟信号(0~33MHz)
	$\overline{DEVSEL}$	设备的选择	输入/输出	该信号有效时(输出)，表示所译码的地址是在设备的地址范围内，当作输入信号时，表示总线上有某设备是否被选中。
	$\overline{FRAME}$	帧数据	输入/输出	在每一个数据传送周期的开始，由现役的 PCI 总线主控设备将这个信号线置成低电平，当所有的数据传送完毕或传送被中断时，则撤销这个信号
	$\overline{GNT}$	许可	输出	低电平，向正在请求的 PCI 部件表明：它现在可以使用 PCI 总线作为主控设备，每一个 PCI 总线主控设备都有各自的 $\overline{GNT}$ 输入
	$\overline{IDSEL}$	预置设备选择	输出	低电平，选择配置存储器
	$\overline{INTA}$ 、 $\overline{INTB}$ $\overline{INTC}$ 、 $\overline{INTD}$	中断	输入	低电平，中断输入，把信号 $\overline{INTA}$ 分配给了单功能的 PCI 设备，而多功能的设备可以使用 $\overline{INTB} \sim \overline{INTD}$
	$\overline{IRDY}$	起始者 准备就绪	输入/输出	低电平，表示起始的总线主控设备已经将有效数据放在总线上，或者已准备好从总线上读去数据
	$\overline{LOCK}$	锁定	输入/输出	低电平，表明到指定的 PCI 设备的访问封锁，但是到 PCI 其他设备的访问仍然可以执行
	PAR	奇偶校验	输入/输出	对 AD ₀ ~AD ₃₁ 和 $\overline{C}/\overline{BE}_3 \sim \overline{C}/\overline{BE}_0$ 来说，为偶校验位

续表

	信 号	名 称	类 型	说 明
32 位 部 分	$\overline{\text{PERR}}$	奇偶校验错	输入/输出	低电平, 表示出现了一次奇偶校验错
	$\overline{\text{PRSNT}}_1$ $\overline{\text{PRSNT}}_2$	存在	输入	这两条外引线是 PCI 适配器连到 +5V 或地的连线, 其上信号用以表明它的存在和电源消耗
	$\overline{\text{REQ}}$	请求	输入	低电平, 表示请求作为主控设备对 PCI 总线的控制
	$\overline{\text{REQ}}_{64}$	请求 64 位传送	输入/输出	由当前的总线主控设备将其置成低电平, 可以表明希望进行的 64 位的数据传送
	$\overline{\text{RST}}$	复位	输出	低电平, 表示对所有连到 PCI 总线上的设备都复位
	$\overline{\text{SBO}}$	监视补偿	输入/输出	由总线主控设备将其置成低电平, 用以指示对已修改 Cache 行查询命中, 以支持写贯穿或写回操作
	$\overline{\text{SDONE}}$	监视完成	输入/输出	通过一个总线主控设备将其置成高电平, 用以表示当前的查询周期已经完成
	$\overline{\text{SERR}}$	系统错误	输入/输出	通过一个总线主控设备将其置成低电平, 用以表示一个地址奇偶错或其他严重的系统错误
	$\overline{\text{STOP}}$	停止	主控目标	低电平, 指示主控设备去停止当前的操作
	TCK TDI TDO TMS $\overline{\text{TRST}}$	测试时钟 测试数据输入 测试数据输出 测试方式选择 测试复位		这 5 个信号用于系统测试
	$\overline{\text{TRDY}}$	目标准备就绪	主控目标	低电平, 表明 PCI 目标可以接收写数据, 或现在已经准备好去读数据
64 位 扩 展 部 分	$\text{AD}_{32} \sim \text{AD}_{63}$	多路复用地址和数据线	输入/输出	64 位数据总线和地址总线的高端部分
	$\text{C}/\overline{\text{BE}}_7 \sim \text{C}/\overline{\text{BE}}_4$	多路复用命令和字节允许	输入/输出	在一个总线周期的数据时间段期间, 这些信号表明总线周期的类型。在一个总线周期的地址时间段期间, 这些信号是低电平, 表明在数据传送时会涉及到 32 位数据总线上的哪些字节
	PAR_{64}	奇偶校验 64	输入/输出	高或低电平, 完成 $\text{AD}_{32} \sim \text{AD}_{63}$ 地址线上和 $\text{C}/\overline{\text{BE}}_7 \sim \text{C}/\overline{\text{BE}}_4$ 的偶校验

6.2.4 局部总线

局部总线(或称直线总线)是当今计算机不用进行任何真正技术突破而提高执行速度的方法, 使得在对机器基本结构不作很大改变的情况下, 向提高执行速度迈进了一大步。

按照常规的设计, 凡采用系统总线体系结构的微机, 任何进出处理器的信息都是通过系统总线传输的, 系统总线是用在系统板与外围设备所有 I/O 之中的, 包括内存、磁盘和

显示器。按照这种思路,由于必须保持与原有设计的向下兼容以及与各种外设联系,系统总线肯定是带宽有限、速度较慢的,即使是能传送 32 位数据的新设计的总线,也通常比处理器速度要慢的多。这样就限制了诸如内存、显示器和磁盘机这样一些需要快速响应的器件的工作速度。这个问题在 32 位微机出现后就显得突出了,在 64 位机和多媒体中就更加突出。

为了解决这个问题,计算机厂商开始把主系统 RAM 放到系统板(主机板)上,并通过直接的连线将其挂接到 CPU 上。CPU 与主 RAM 之间的这种直接的通道就是最初的局部总线的由来。这样内存的通信问题就独立出来了,其他外设仍使用系统总线。随后,出现了 VESA 和 PCI 局部总线的设计,打开了显示设备和硬盘局部总线的通信。使用这些标准,CPU 和系统 RAM 能以所设计的主系统带宽(通常为 25MHz~33MHz)的高速度与显示适配卡和磁盘控制器对话,而不再限于系统总线的速度了。

局部总线已经出现较长时间了,但至今未达成一个统一的标准。下面对 VESA VL 标准作一简介。

最初,局部总线的设计及标准制定是由美国视频电子标准联合会 VESA(Video Electronics Standard Association)完成的。VESA 是一个制定视频显示标准和一些其他标准的民间组织,包括 60 多家公司。支持 VESA 局部总线的硬卡称为 VL 总线卡(VL-Bus),因此,VESA 总线有时也称为 VL 总线。VL 总线标准 1.0 版于 1992 年 8 月为 VESA 所认可,并很快为工业界接受,随后百余家公司生产与 VL 兼容的产品。一年以后,2.0 版 VL 总线标准问世。VL 标准由电气、机械、时序和连接设计的详细说明组成,也采取开放的策略。

VL1.0 版支持 32 位数据通道,也允许使用一次传送 16 位数据的设备。2.0 版本是为了跟上最新的 64 位处理器。VL 接插器位于系统板上与原有 ISA、EISA 或 MCA 接插器成行放置。

VL 支持 16~66MHz 的速度。然而 1.0 版规范在任何扩展槽仅限于 40MHz 信号,2.0 版限于 50MHz。1.0 版可支持多达 3 个设备,而 2.0 版可以多达 10 个。设备既可插入槽内,也可在母板上。VL 支持以 106MB/s 的速率持续传送数据,64 位总线则可达到 260MB/s。

VL 总线的-一个设计特点是 64 位的设备能在 32 位 VL 槽中运行,32 位设备也能在 64 位槽中工作,此时只支持 32 位数据传送。VL 总线也支持 16 位外设和 16 位 I/O 的 386 SX 之类的 CPU。

总的来说,VL-BUS 是一种性能优越的总线,在 486 微机上是一个经济实用的高性能总线。可是 VL-BUS 为 486 芯片专用,不支持 586 CPU,中间还须增加桥接电路。VL 的地址总线和数据总线是物理上分开的两个独立部分,属于非多路复用,信号复杂,比地址线-和数据线共用一条物理线路的多路复用的性能要稍逊一筹。这些因素加上 5V~3.3V 信号环境等问题,VL-BUS 的应用受到限制。

6.2.5 STD 总线

STD 总线是 1987 年推出的用于工业控制微型计算机的标准系统总线。自它问世以来,以其优越的性能和强大的生命力在工业领域得到了广泛应用和迅速发展。

1. STD 总线特点

- (1) 高可靠性: 使用该总线构成的工业控制机可以长期工作在恶劣环境下。
- (2) 小板结构: 高度模块化, 可根据用户要求组成各种微机应用系统。
- (3) 结构简单: 结构简单并能支持多微处理器系统。
- (4) 兼容式的总线结构: 如 8 位的 STD 产品可以与新标准的 16 位或 32 位 STD 产品一起工作。

产品配套功能齐全, 价格低廉, 易于选用, 为 STD 新产品的推广创造了有利的条件。

2. STD 总线规范

STD 总线起初是针对 8 位微型计算机而推出的, 随着新技术的发展和实际应用的需要, STD 总线经过修改和改进, 利用复用技术, 在保证同原有的 I/O 插件板兼容的条件下, 提供了全 16 位的数据传送能力。在原定义的 56 个总线信号之下, 支持 20 位地址, 实现了 1MB 内存空间的直接寻址。20 世纪 80 年代末, STD 总线也由 56 个信号发展到了 114 个信号, 现阶段已有 136 个信号的 32 位 STD 总线标准推出, 为高档的 STD 微机系统的发展提供了有利的条件。下面以国内最流行的 56 个信号的 STD 总线为例加以说明。

1) 引脚分配

引脚分配见表 6-4 和表 6-5。表中信号流向以现行主设备为参考点。

STD 总线共有 56 个引脚, 按功能分为 5 组, 各组的引脚表示为:

引脚 1~6: 逻辑电源线。引脚 7~14: 数据总线。引脚 15~30: 地址总线。引脚 31~52: 控制总线。引脚 53~56: 辅助电源线。

表 6-4 STD 总线信号分配表 (元件面)

元 件 面				
名 称	引 脚	信 号 名	信号流向	说 明
逻辑电源总线	1	V_{CC}	输入	逻辑电源+5V
	3	GND	输入	逻辑接地
	5	$V_{BB.1}/V_{BAT}$	输入	逻辑偏置.1/电池
数据总线	7	D_3/A_{19}	输入/输出	数据总线/地址扩展总线
	9	D_2/A_{18}	输入/输出	
	11	D_1/A_{17}	输入/输出	
	13	D_0/A_{16}	输入/输出	
地址总线	15	A_7	输出	地址总线
	17	A_6	输出	
	19	A_5	输出	
	21	A_4	输出	
	23	A_3	输出	
	25	A_2	输出	
	27	A_1	输出	
	29	A_0	输出	

续表

元 件 面				
名 称	引 脚	信 号 名	信号流向	说 明
控制总线	31	$\overline{\text{WR}}$	输出	写
	33	$\overline{\text{IORQ}}$	输出	I/O 地址选择
	35	$\overline{\text{IOEXP}}$	输入/输出	I/O 扩展
	37	$\overline{\text{REFRESH}}$	输出	刷新定时
	39	$\overline{\text{STATUS1}}$	输出	CPU 状态
	41	$\overline{\text{BUSAk}}$	输出	总线响应
	43	$\overline{\text{INTAk}}$	输出	中断响应
	45	$\overline{\text{WAITRQ}}$	输入	等待请求
	47	$\overline{\text{SYSRESET}}$	输出	系统复位
	49	$\overline{\text{CLOCK}}$	输出	处理器时钟
辅助电源总线	51	PCO	输出	优先级链输出
	53	AUXGND	输入	辅助接地
	55	AUX+V	输入	辅助电源+12V

2) 信号说明

(1) 地址总线：STD 定义最多达 24 条地址总线(A₂₃~A₀)。当寻址范围超过 64KB 时，复用数据总线作为地址总线使用。

表 6-5 STD 总线信号分配表 (焊接面)

焊 接 面				
名 称	引 脚	信 号 名	信号流向	说 明
逻辑电流总线	2	V _{CC}	输入	逻辑电源+5V
	4	GND	输入	逻辑接地
	6	V _{BB.2} /D _{CPD}	输入	逻辑偏置.2/掉电电源
数据线	8	D ₇ /A ₂₃	输入/输出	数据总线/地址扩展总线
	10	D ₆ /A ₂₂	输入/输出	
	12	D ₅ /A ₂₁	输入/输出	
	14	D ₄ /A ₂₀	输入/输出	
地址线	16	A ₁₅ /D ₁₅		地址总线/数据扩展总线
	18	A ₁₄ /D ₁₄		
	20	A ₁₃ /D ₁₃		
	22	A ₁₂ /D ₁₂		
	24	A ₁₁ /D ₁₁		
	26	A ₁₀ /D ₁₀		
	28	A ₉ /D ₉		
	30	A ₈ /D ₈		

续表

焊 接 面				
名 称	引 脚	信 号 名	信号流向	说 明
控制总线	32	$\overline{\text{RD}}$	输出	读
	34	$\overline{\text{MEMRQ}}$	输出	存储器地址选择
	36	$\overline{\text{MEMEX}}$	输入/输出	存储器扩展
	38	$\overline{\text{MCSYNC}}$	输出	CPU 周期同步
	40	$\overline{\text{STATUSO}}$	输出	CPU 状态
	42	$\overline{\text{BUSSRQ}}$	输入	总线请求
	44	$\overline{\text{INTRQ}}$	输入	中断请求
	46	$\overline{\text{NMIRQ}}$	输入	非屏蔽中断请求
	48	$\overline{\text{PBREST}}$	输入	按钮复位
	50	$\overline{\text{CNTRL}}$	输入	辅助定位
	52	$\overline{\text{PCI}}$	输入	优先级链输入
辅助电源线	54	AUXGND	输入	辅助接地
	56	AUX-V	输入	辅助电源-12V

(2) 数据总线: 它是双向三态工作总线, 若是 8 位微处理器, 存储器的地址范围在 64KB 之内时, 不需要对数据总线进行复用。若地址范围大于 64KB 时, 需利用机器周期同步信号 MCSYNC 复用数据总线, 利用它传送高位地址($A_{23} \sim A_{16}$)。

(3) 控制总线: STD 定义了许多控制信号, 这是 STD 总线具有较高性能的重要体现, 这些控制信号不仅有较强的功能, 而且使用十分灵活, 它们是使用 STD 总线的关键。

(4) 电源线: 电源线说明见表 6-4 和表 6-5 所示。

6.2.6 USB 总线

计算机外部设备的种类和数量越来越多, 为了解决变得越来越复杂的主机连接问题, 由 IBM, NEC, Intel, Microsoft, Compaq, Digital 和 Northern Universal Serial Bus Telecom 这七家公司联合提出了外部输入/输出接口的新规格——通用串行总线 USB。

1. 概述

USB 是英文 Universal Serial Bus 的缩写, 即通用串行总线, 是一种新型的外设接口标准, 支持即插即用, 理论上最多可以连接 127 个外设。它实际上是一种接口技术, 而并非是一种新的总线标准。

1994 年, USB 论坛成立; 1995 年, USB 0.9 规范制定; 1996 年标准版本 USB 1.0 产生, 为连接中低速外设提供双向、低成本的解决方案, 该标准提供的数据传输率有低速 1.5Mb/s 和全速 12Mb/s 两种。1998 年后, 随着微软在 Windows 98 中内置了对 USB 接口的支持模块, 加上 USB 设备的日益增多, USB 逐步走进了实用阶段。

为满足计算机与外部设备的数据量不断加大的能力和高性能连接的需要, 于 2000 年又推出了 USB 2.0 标准。它提供了更高的带宽, 数据传输率可达 120Mb/s~240Mb/s, 支持数码相机、扫描仪、打印机及存储设备, 与现有的 USB 外设完全兼容。

1) USB 物理接口

USB 设备通过四线电缆与主机或 USB Hub 连接, 四根线分别是 VBUS、GND、D+、D-。其中, VBUS 为电源线, GND 为地线, 可以有条件地给一些设备(包括 Hub)供电。D+ 和 D- 是一对差模信号线。USB 利用 D+ 和 D- 线传输数据。USB 物理接口如图 6.3 所示。

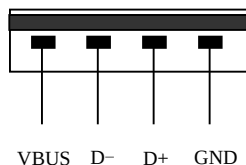


图 6.3 USB 物理接口

USB 在高速模式下必须使用带有屏蔽的双绞线, 由于信号衰减的限制, 最长不超过 3m。在低速模式下, 可以使用不带屏蔽或非双绞线, 但最长不能超过 5m。为了保证提供一定的信号电压, 以及与终端负载相匹配, 在电缆的每一端都使用了不平衡的终端负载; 这也保证了能够检测外设端口的连接或分离, 并区分高速与低速设备。

USB 接口的体积小, 针数也只有 4 针。下行接设备端的接口为方型, 上行接计算机端的接口为长方型, 以确保不会出现非法的连接。插头与插座有不能互换的 A、B 两个系列, 系列 A 用于基本固定的外围设备, 系列 B 用于经常插拔的设备。

2) 传输速度

USB 总线数据传输率有 3 种: 高速数据传输率为 480Mb/s, 全速数据传输率为 12Mb/s, 低速数据传输率为 1.5Mb/s。但是 USB 设备只支持一种传输模式, 目前, 键盘、鼠标等外设采用低速率, 打印机、扫描仪等采用全速率, 视频会议、台式电脑硬盘驱动器等采用高速率。

3) USB 与 IEEE 1394 的比较

除了 USB 之外, IEEE 1394 也是一种高性能的串行总线。IEEE 1394 接口支持 400Mb/s 的数据传输速率, 支持异步传输和同步传输两种模式, 其中同步传输模式专用于实时地传送视频和音频数据。IEEE 1394 结构的所有资源, 都是用存储器影射方式实现资源配置和管理的。USB 一般用于连接中低速外设并局限于 PC 领域, IEEE 1394 将向通信和数字家电方向发展。

2. USB 总线的拓扑结构

物理结构上, USB 系统是以根集线器(Root Hub)为中心的分层(Tier)星形结构, 各个点(Node, 功能器件, 即设备)可利用总线供电的集线器连接。PC 机就是主机和根集线器, 用户可以将外设或附加的 Hub 与之相连, 附加的 Hub 可以再连接其他外设以及下层 Hub, 但系统的级联不能超过 5 级(包括 ROOT HUB)。典型的 USB 系统拓扑结构如图 6.4 所示。

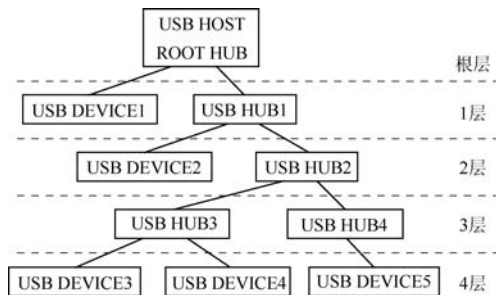


图 6.4 典型的 USB 系统拓扑结构

3. USB 的硬件设备

1) USB 主控制器/根集线器

USB 主控制器和根集线器合称为 USB 主机(HOST)。USB 主控制器是硬件、固件和软件的联合体。主控制器负责 USB 总线上的数据传输,它把并行数据转换为串行数据以便在总线上传输,把收到的数据翻译成操作系统能够识别的数据形式。

根集线器集成在主机系统中,可以提供一个或更多的接入端口。根集线器检测外设的连接和断开,执行主控制器发出的请求,并在设备和主控制器之间传递数据。根集线器由一个控制器和中继器组成。

USB 主机的主要作用是:

- (1) 检测 USB 设备的插入和移出。
- (2) 在主机和 USB 设备之间管理数据流。
- (3) 搜集状态信息和活动统计信息。
- (4) 为接入的 USB 设备提供数额受限的功率。

2) USB 设备

为主机提供单个功能的设备称为“功能部件”。功能部件和集线器都称为 USB 设备。复合的“设备”有一个集线器和一个或多个“功能部件”。每一个集线器和功能部件都有唯一的地址。

(1) 集线器:提供用以访问 USB 总线的更多的接入点,是 USB 即插即用体系结构中关键部件,用于简化 USB 的连接,使 USB 具有多个连接特性。

(2) 功能部件:向系统提供特定的功能,如鼠标、键盘等。一个 USB 设备必须支持 USB 协议,可以对诸如配置和复位等标准的 USB 操作作出响应,具有标准的描述信息才能正常工作。

4. USB 的数据传输

USB 总线上的每个设备都有一个由主机分配的唯一地址,由主机通过集线器在一个自动识别过程中分配。USB 总线上的数据传输是一种“主—从”式的传输,所有的传输都由 USB 主机发起,任何外设都无权开始一个传输,USB 设备仅仅在主机对它提出要求时才进行传输。

1) 传输模式

USB 总线有 4 种不同的传输模式。

(1) 控制传输(control transfer):控制传输是双向传输,主要用于命令/状态操作。发送控制命令,设置设备地址,设置设备属性,读取设备配置信息及设备状态等功能。其中,设置(Setup)操作和状态(Status)操作的数据包具有 USB 定义的结构。

支持全速设备允许数据包的最大容量为 8B、16B、32B 或 64B,而低速设备每次最大有效负荷只有 8B 一种选择。

端点不能指定总线访问的频率和占用总线的时间,USB 系统软件会做出限制,具有数据传输保证,在必要时可以重试。

(2) 同步传输(synchronous transfer):同步传输是单向传输,仅适用于全速/高速设备,

是一种周期的、连续的传输方式。同步传输每毫秒进行一次传输，有较大的带宽保证，并已保持数据传输的速率恒定(每个同步管道每帧传输一个数据包)，常用于与时间有密切关系的信息传输，主要用于数码相机、扫描仪等，每次传输的最大有效负荷可为 1023 个字节。

如果一个外设需要双向传输，则必须使用另一个端点；数据没有 USB 定义的结构(数据流管道)；没有数据重发机制，要求具有一定的容错性。

与中断方式一起，占用总线的时间不得超过一帧的 90%。

(3) 中断传输(interrupt transfer)：中断传输用于支持偶然需要对少量数据进行通信，但服务时间却受限的设备，如键盘、鼠标。全速设备的中断传输周期可为 1ms~255ms，每次传输的有效负荷最大可为 64 个字节；而低速设备的中断传输周期为 10ms~255ms，每次传输的有效负荷仅为 8 个字节。

数据没有 USB 定义的结构(数据流管道)，只有输入(外设到主机)单一方向。

具有最大服务周期保证，在规定时间内保证有一次数据传输；与同步方式一起，占用总线的时间不得超过一帧的 90%；具有数据传输保证，在必要时可以重试。

(4) 批量数据传输(bulk transfer)：批量数据传输用于对时间没有要求的非周期性的大量数据传输，仅全速/高速设备支持块数据传输；单向传输，如果一个外设需要双向传输，则必须使用另一个端点；仅当总线带宽有效时，才进行块数据传输，每次传输最大有效负荷为 64 个字节；没有带宽保证，只要总线空闲，就允许传输数据(优先级小于控制传输)；具有数据传输保证，在必要时可以重试，可供打印机、调制解调器、数字音响等设备使用。

2) 数据编码

USB 通信时，使用一对摆幅为 3.3V 的差动信号(D⁺, D⁻)，采用 NRZI(Not Return to Zero Invert 不归零翻转)传送各种数据，它定义用电压保持不变表示二进制数字序列中的 1 码，用电压跳变(0V~3.3V 的跳变或者 3.3V~0V 跳变)表示二进制数字序列中的 0 码。USB 使用 NRZI 编码方法产生的 USB 信号如图 6.5 所示。在同步方法上采用帧同步模式，同步码为 10000000B，将它放在各种信息的前面进行同步。如果发送信息包含有连续的 6 个以上的 1，那么就在 6 个 1 后强制性地加上填充位 0，确保发送和接收的同步。接收端对数据的正确性进行检查，发现错误通过“握手包”通知发送者，要求重新发送。检测、通知和再发送都由硬件来完成，不需要任何程序。

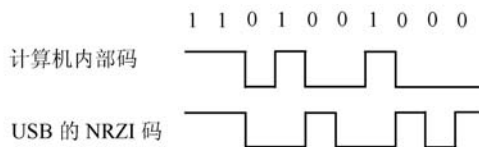


图 6.5 USB 使用的 NRZI 编码

3) 传送的组成

一次数据传输包含一个或多个事务，每个事务包括它的源地址和目的地址，一个事务一旦开始通信，必须连续进行，不允许被中断。每个事务包含一个标志包，也可能包含一个数据包或握手包。所有包包括一个“包标识”(PID)、帧号、地址或其他信息，以及 CRC 校验码。

4) USB 包的格式

USB 包是数据传送的基本方式, USB 共有 3 种类型的包。

(1) USB 的传输总是首先有主机发出标志(TOKEN, 令牌)包开始。标志包中有设备地址码、端口号、传输方向和传输类型等信息。

(2) 其次是数据源向数据目的地发送的数据包或者发送无数据传送的指示信息, 数据包可以携带的数据最多为 1023B。

(3) 最后是数据接收端向数据发送端回送一个握手包, 提供数据是否正常接收的反馈信息。如果有错误, 需要重发。除了同步以外, 其他类型的传输都需要握手(handshake)包。

5) 总线枚举过程

USB 设备可以即插即用, 一旦设备连接到某一个 USB 的节点, USB 就会产生被软件支持的一系列操作, 来完成对设备的配置, 称为总线枚举过程。

设备所连接的 Hub 检测出端口上有设备连接, 即向主机报告。主机通过询问 Hub 以获取确切的信息。主机知道设备连接到哪个端口上, 向这个端口发出复位命令。Hub 发出的复位信号结束后, 端口被打开, Hub 向设备提供 100mA 的电源, 这时设备上电, 所有的寄存器复位, 并且以默认地址 0 以及端点 0 响应命令。

主机通过默认地址与端点 0 进行通信, 赋予设备一个唯一的地址, 并且读取设备的配置信息。主机对设备进行配置, 该设备可以使用。当该设备被移走时, Hub 依然要报告主机, 并且关闭端口。主机接到设备移走的报告, 就会改写当前结构的信息。

5. USB 的特点

USB 是一种方便、灵活、简单、高速的总线结构, 与传统的外设接口相比, 主要有以下特点:

(1) 使用方便。USB 提供机箱外的即插即用与热插拔功能, 当有新的 USB 设备接入或移走时, 可以在线进行, 不必开机箱, 也不必关闭主机电源。USB 设计的驱动程序和应用软件可以自动启动, 无须用户做更多操作。

(2) USB 只占用系统的一个端口和一个中断, 节省了系统资源。使用自己的保留中断号, 不涉及 IRQ 冲突问题, 不会同其他设备争用 PC 机有限资源。

(3) 连接和扩展方便。USB 采用的是一种易于扩展的树状结构, 通过使用集线器(Hub)扩展, 可以连接 127 个外设, 支持多种不同设备的工作。

(4) 适用面广。USB 由开始的支持中低速外设到目前支持高速外设, 适用于带宽范围在几 Kb/s 到几百 Mb/s 的设备。

(5) 供电灵活。USB 接口提供了内置电源, 不需额外的供电系统。

(6) USB 支持 4 种传输模式, 即控制传输、中断传输、同步传输和批量传输, 可以适应很多类型的外设。

(7) USB 具有错误检测和处理机制, 可以识别设备的错误和数据传输的可靠性。

此外, USB 采用单一的数据通用接口, 一个接口就可以满足多种外设, 不必为每个外设准备不同的接口协议, 这也是 USB 非常通用的原因。

6.3 小 结

本章首先介绍了总线的概念，总线标准的特性包含物理特性、功能特性、电气特性、时间特性。还介绍了总线的分类以及设计总线应考虑的问题，以便对总线有一个总体的把握。然后介绍了微型计算机常用的总线标准，包括 PC 总线，ISA 与 EISA 总线，PCI 总线，局部总线，STD 总线，USB 总线。应该掌握实用总线的特点，针对技术的发展，思考还需要做怎样的改进。

6.4 习 题

1. 什么是总线？
2. 微型计算机中的总线通常分为哪几类？各有什么特点？
3. 在微型计算机中，采用总线标准有什么特性？
4. 设计总线应考虑的因素是哪些？
5. IBM PC 总线、PC/AT 总线的主要特点是什么？它们的使用范围如何？
6. PCI 总线的主要特点是什么？
7. 总线的性能指标有哪些？比较常用微型机的总线。
8. $\overline{IOR}$ 、 $\overline{IOW}$ 、 $\overline{MEMR}$ 、 $\overline{MEMW}$ 的信号各自有效时表明什么含义？
9. USB 总线的特点是什么？
10. IBM PC/AT 微机的 16 位总线插槽由多少引脚组成？它比 PC/XT 总线增加了哪些信号？
11. 试述 PCI 总线的结构。
12. USB 总线的层次结构是什么？

第 7 章 半导体存储器

教学提示：半导体存储器是用以存储二进制信息的器件，是微机系统中的重要组成部分。计算机的工作依赖于存储器中的程序和数据。在计算机中采用什么样的存储介质，怎样组织存储系统，以及怎样连接和控制存储器的操作，是计算机存储系统的基本问题。存储器的存储容量和性能对于整个计算机系统的性能起着至关重要的作用。另外半导体存储器以其品种多、容量大、速度快、耗电省、体积小、操作方便、维护容易等优点，在数字设备中得到广泛应用。

教学要求：本章让学生了解半导体存储器的工作原理、分类、主要性能指标及各类存储器的特点。熟悉微机中存储系统的结构、CPU 与存储器的连接，重点掌握内存储器的扩展及多层次的存储器体系结构。

7.1 概 述

存储器是用以存储一系列二进制数码的器件，正是因为有了存储器，计算机才有了对信息的记忆功能，从而实现程序和数据信息的存储，使计算机能够自动高速地进行各种运算。存储器系统是微机系统中重要的子系统。

7.1.1 存储器系统

将两个或两个以上速度、容量和价格各不相同的存储器用硬件、软件或软硬件相结合的方法连接起来就构成存储系统。系统的存储速度接近较快的存储器，容量接近较大的存储器。

计算机的存储器系统是由两大部分组成：一部分叫内部存储器，简称为内存或主存；另一部分叫外部存储器，简称为外存或辅存。

1. 计算机系统中存储器系统组成

计算机存储器系统的组成如图 7.1 所示。主存储器(内存)是计算机系统的—个组成部分，它用来存储计算机当前正在使用的数据和程序，一般由一定容量的速度较高存储器组成，CPU 可以直接用指令对内存进行读写操作。在计算机中，内部存储器是由半导体存储器芯片组成的，半导体存储器按存取方式不同，分为随机存取存储器(RAM Random Access Memory)和只读存储器 ROM(Read only Memory)。

存储器设计目标之一是以较小的成本使存储体系与 CPU 的速度匹配，同时还要求存储体系具有尽可能大的容量。因此，速度、容量、价格是存储器设计应考虑的主要因素。提高存储速度的主要措施是在主存储器与 CPU 之间增加一个高速缓冲存储器(Cache)来存储使用频繁的指令和数据，以提高访存操作的平均速度。

程序对存储容量的要求也提高了,因此仍存在存储空间的分配问题。

存储系统的层次结构是指把各种不同存储容量、存取速度和价格的存储器按层次结构组成多层存储器,并通过管理软件和辅助硬件有机组合成统一的整体,使所存放的程序和数据按层次分布在各种存储器中。

常用的存储系统的层次结构主要由高速缓冲存储器 Cache、主存储器和辅助存储器三级存储层次构成,这种多层次结构已成为现代计算机的典型存储结构。

Cache 存储器系统是由主存储器和高速缓冲存储器组成。在速度方面,计算机的主存和 CPU 一直保持了大约一个数量级的差距。显然这个差距限制了 CPU 速度潜力的发挥。为了弥合这个差距,必须进一步从计算机系统结构上去研究。设置高速缓冲存储器(Cache)是解决存取速度的重要方法。在 CPU 和主存中间设置高速缓冲存储器,构成高速缓存(Cache)-主存层次,要求 Cache 在速度上能跟得上 CPU 的要求。Cache-主存间的地址映像和调度吸取了比它较早出现的主-辅存存储层次的技术,不同的是因其速度要求高,不是由软、硬件结合而完全由硬件来实现。

虚拟存储系统是由主存储器和磁盘存储器组成。虚拟存储系统希望能得到辅存的价格,主存储器的速度。在使用时将用户的地址空间设计的可以比主存实际空间大得多,以致可以存得下整个程序。指令地址码称为虚地址(虚存地址、虚拟地址)或逻辑地址,其对应的存储容量称为虚存容量或虚存空间;而把实际主存的地址称为物理地址或实(存)地址,其对应的存储容量称为主存容量、实存容量或实(主)存空间。操作系统和硬件结合实现虚拟地址到实际地址的转换,这对于程序员是透明的。这就形成了主存-辅存层次满足了存储器大容量和低成本需求。使用磁盘作为外存,不仅价格便宜,可以把存储容量做得很大,而且在断电时它所存放的信息也不丢失,可以长久保存,且复制、携带都很方便。

7.1.2 存储器的分类

计算机的存储器,从体系结构的观点来划分,可根据其是设在主机内还是主机外,分为内部存储器和外部存储器两大类。

内部存储器(简称内存或主存)是计算机主机的组成部分之一,用来存储当前运行所需要的程序和数据, CPU 可以直接访问内存并与其交换信息。相对外部存储器(简称外存)而言,内存的容量小、存取速度快。而外存刚好相反,外存用于存放当前不参加运行的程序和数据, CPU 不能对它直接访问,而必须通过配备专门的设备才能够对它进行读写(如磁盘驱动器等),这是它与内存之间的一个本质的区别。外存容量一般都很大,但存取速度相对比较慢。

存储器按照使用的存储介质不同可分为半导体存储器、磁表面存储器(如磁盘存储器与磁带存储器)、光介质存储器;按存取方式的不同可分为随机存储器、顺序存储器、半顺序存储器;按照信息的是否可保存可分为易失性存储器(随机存储器 RAM)和非易失性存储器(只读存储器 ROM);按其计算机系统中的作用不同可分为主存储器、辅助存储器、缓冲存储器和控制存储器等。下面重点介绍用于构成内存的半导体存储器。

1. 随机存取存储器(RAM)

随机存取存储器简称 RAM,也叫做读/写存储器。按其制造工艺可以分为双极型半导

体 RAM 和金属氧化物半导体(MOS)RAM。

1) 双极型 RAM

双极型 RAM 的主要优点是存取时间短,通常为几纳秒到几十纳秒(ns)。与下面提到的 MOS 型 RAM 相比,其集成度低、功耗大,而且价格也较高。因此,双极型 RAM 主要用于要求存取时间非常短的特殊应用场合。

2) MOS 型 RAM

用 MOS 器件构成的 RAM 又可分为静态读/写存储器 SRAM(Static RAM)和动态读/写存储器 DRAM(Dynamic RAM)。

SRAM 的存储单元由双稳态触发器构成。双稳态触发器有两个稳定状态,可用来存储一位二进制信息。只要不掉电,其存储的信息可以始终稳定地存在,故称其为“静态”RAM。SRAM 的主要特点是存取时间短(几十到几百纳秒),外部电路简单,便于使用。常见的 SRAM 芯片容量为 1KB~64KB 之间。SRAM 的功耗比双极型 RAM 低,价格也比较便宜。

DRAM 的存储单元用电容来存储信息,电路简单。但电容总有漏电存在,时间长了存放的信息就会丢失或出现错误。因此需要对这些电容定时充电,这个过程称为“刷新”,即定时地将存储单元中的内容读出再写入。由于需要刷新,所以这种 RAM 称为“动态”RAM。DRAM 的存取速度与 SRAM 的存取速度差不多。其最大的特点是集成度非常高,目前 DRAM 芯片的容量已达几百兆比特,此外它的功耗低,价格比较便宜。

由于用 MOS 工艺制造的 RAM 集成度高,存取速度能满足各种类型微型机的要求,而且其价格也比较便宜,因此,现在微型计算机中的内存主要由 MOS 型 DRAM 组成。

3) 非易失性静态随机存储器 NVRAM(non-volatile RAM)

在静态随机存储器中集成可充电电池,可作为随机访问存储器使用,与静态存储器一样,在电源关闭后可长时间保持存储的数据不丢失。

2. 只读存储器 ROM

根据制造工艺不同,只读存储器分为 ROM、PROM、EPROM、E²PROM 几类。只读存储器在工作时只能读出,不能写入,掉电后不会丢失所存储的内容。

1) 掩模式只读存储器(ROM)

掩模式 ROM 是芯片制造厂根据 ROM 要存储的信息,对芯片图形通过二次光刻生产出来的,故称为掩模 ROM。其存储的内容固化在芯片内,用户可以读出,但不能改变。这种芯片存储的信息稳定,成本最低。适用于存放一些可批量生产的固定不变的程序或数据。

2) 可编程 ROM(Programmable ROM, PROM)

如果用户根据自己的需要来确定 ROM 中的存储内容,则可使用可编程 ROM (PROM)。PROM 允许用户对其进行一次编程即写入数据或程序。一旦编程之后,信息就永久性地固定下来。用户可以读出其内容,但是再也无法改变它的内容。

3) 可擦除的 PROM(Erasable Programmable ROM, EPROM)

上述两种芯片存放的信息只能读出而无法修改,这给许多方面的应用带来不便。由此又出现了两类可擦除的 ROM 芯片。这类芯片允许用户通过一定的方式多次写入数据或程序,也可根据需要修改和擦除其中所存储的内容,且写入的信息不会因为掉电而丢失。由

于这些特性,可擦除的 PROM 芯片在系统开发、科研等领域得到了广泛的应用。

可擦除的 PROM 芯片因其擦除的方式不同可分为两类:一是通过紫外线照射(约 20 分钟左右)来擦除,这种用紫外线擦除的 PROM 称为 EPROM;另外一种是通过加电压的方法(通常是加上一定的电压)来擦除,这种 PROM 称为 EEPROM(或 Electric Erasable Programmable ROM, E²PROM)。芯片内容擦除后仍可以重新对它进行编程,写入新的内容。擦除和重新编程都可以多次进行。但有一点要注意,尽管 EPROM(E²PROM)芯片既可读出所存储的内容也可以对其编程写入和擦除,但它们和 RAM 还是有本质区别的。首先它们不能够像 RAM 芯片那样随机快速地写入和修改,它们的写入需要一定的条件(这一点将在后面详细介绍);另外, RAM 中的内容在掉电之后会丢失,而 EPROM(EEPROM)则不会,其上的内容一般可保存几十年。

4) 闪速存储器(Flash Memory)

闪速存储器是新型的非易失性的存储器,Flash Memory 是在 FPM 与 E²PROM 基础上发展起来的,它与 EPROM 一样,用单管来存储一位信息,它与 E²PROM 相同之处是用电来擦除,但是它只能擦除整个区域或整个器件。快速擦除读/写存储器于 1983 年推出,1988 年商品化。它兼有 ROM 和 RAM 两者的性能,又有 DRAM 一样的高密度。目前价格已低于 DRAM,芯片容量已接近于 DRAM,是唯一具有大存储量、非易失性、低价格、可在线改写和高速度读等特性的存储器,它是近年来发展最快、最有前途的存储器。

7.1.3 存储器芯片的主要技术指标

衡量半导体存储器性能的主要指标有存储容量、存取时间、存储器周期、功耗和可靠性。

1. 存储容量

存储容量是存储器的一个重要指标。存储容量是指存储器所能存储二进制数码的数量,即所含存储元的总数。存储器芯片的存储容量用“存储单元个数×每个存储单元的位数”来表示。例如,SRAM 芯片 6264 的容量为 8K×8 bit,即它有 8K 个存储单元(1K=1024),每个单元存储 8 位(一个字节)二进制数据。DRAM 芯片 NMC41257 的容量为 256K×1 bit,即它有 256K 个存储单元,每个单元存储 1 位二进制数据。各半导体器件生产厂家为用户提供了许多种不同容量的存储器芯片,用户在构成计算机内存系统时,可以根据要求加以选用。当然,当计算机的内存确定后,选用容量大的芯片则可以少用几片,这样不仅使电路连接简单,而且功耗也可以降低。

主存的存储容量要受地址线宽度的限制。基本存储元是组成存储器的基础和核心,它用来存储一位二进制信息,在计算机中,人们通常将 1 个二进制位称为“位”(Bit),将 8 位二进制位称为“字节”(Byte),而将计算机数据存储和传输的基本单位称为“字”(Word),将它所包含的二进制数的位数称为“字长”。如由 Pentium(586)等微处理器构成的计算机,它们的字长是 32 位,因而人们也习惯地把这种计算机称为 32 位机。存放一个机器字的存储单元,通常称为字存储单元,相应的存储单元地址称为字地址。而存放一个字节的存储单元,称为字节存储单元,相应的地址称为字节地址。如果计算机中可编址的最小单位是字存储单元,则该计算机称为按字编址的计算机。如果计算机中可编址的最小单位是字节,则该计算机称为按字节编址的计算机。一个机器字可以包含数个字节,所以一个存储单元

也可以包含数个能够单独编址的字节地址。多数计算机是按照字节来进行编址的，即每个地址对应一个字节，这样做一是便于与外设交换信息，二是便于对字符进行处理。随着存储器不断扩大，人们采用了更大的单位：千字节 KB(1024B)、兆字节 MB(1024KB)，千兆字节 GB(1024MB)及兆兆字节 TB(1024GB)。显然，存储容量是反映存储能力的指标。

2. 存取时间和存取周期

存取时间又称存储器访问时间，即启动一次存储器操作(读或写)到完成该操作所需要的时间。具体地讲，也就是从一次读操作命令发出到该操作完成，将数据读入数据缓冲寄存器为止所经历的时间，即为存储器存取时间；CPU 在读/写存储器时，其读写时间必须大于存储器芯片的额定存取时间。如果不能满足这一点，微型机则无法正常工作。

存取周期是连续启动两次独立的存储器操作所需间隔的最小时间。通常，存储周期略大于存取时间，其时间单位为纳秒(ns)。通常手册上给出存取时间的上限值，称为最大存取时间。显然，存取时间和存储周期是反映主存工作速度的重要指标。

3. 可靠性

可靠性则是指存储器对电子磁场的抗干扰性和对温度变化的抗干扰性。一般用平均无故障时间来表示。计算机要正确地运行，必然要求存储器系统具有很高的可靠性。内存发生的任何错误会使计算机不能正常工作。而存储器的可靠性直接与构成它的芯片有关。目前所用的半导体存储器芯片的平均故障间隔时间(MTBF)约为 $5 \times 10^6 \text{h} \sim 1 \times 10^8 \text{h}$ 左右。

4. 功耗

功耗通常是指每个存储元消耗功率的大小，单位为微瓦/位($\mu\text{W}/\text{位}$)或者毫瓦/位($\text{mW}/\text{位}$)。使用功耗低的存储器芯片构成存储系统，不仅可以减少对电源容量的要求，而且还可以提高存储系统的可靠性。

5. 集成度

集成度指在一块存储芯片内，能集成多少个基本存储电路，每个基本存储电路存放一位二进制信息，所以集成度常用位/片来表示。

6. 性能/价格比

性能/价格比(简称性价比)是衡量存储器经济性能好坏的综合指标，它关系到存储器的实用价值。其中性能包括前述的各项指标，而价格是指存储单元本身和外围电路的总价格。

7. 其他指标

体积小、重量轻、价格便宜、使用灵活是微型计算机的主要特点及优点，所以存储器的体积大小、功耗、工作温度范围、成本高低等也成为人们关注的性能指标。

7.2 随机存储器

随机存储器简称 RAM，也叫做读/写存储器，随机存取存储器 RAM 主要用来存放当前运行的程序、各种输入/输出数据、中间运算结果及堆栈等，其存储的内容既可随时读出，

也可随时写入和修改, RAM 的缺点是数据的易失性, 即一旦掉电, 所存的数据全部丢失。在这一节里, 将先介绍存储器单元的工作原理, 再从应用的角度出发, 以几种常用的典型芯片为例, 详细介绍两类 MOS 型读/写存储器 SRAM 和 DRAM 的特点、外部特性以及它们的应用。

RAM 存储单元是存储器的核心部分。按工作方式不同, 可分为静态和动态两类, 按所用元件类型, 又可分为双极型和 MOS 型两种, 因此存储单元电路形式多种多样。

7.2.1 静态存储器

1. 静态存储单元的工作原理

基本的 6 管 NMOS 静态存储单元如图 7.3 所示, 由 6 只 NMOS 管($T_1 \sim T_6$)组成。 T_1 与 T_2 构成一个反相器, T_3 与 T_4 构成另一个反相器, 两个反相器的输入与输出交叉连接, 构成基本触发器, 作为数据存储单元。 X_i 是行选线, Y_j 是列选线, B 是位选线。当 T_1 导通、 T_3 截止时, F 为 0 状态, E 为 1 状态; T_3 导通、 T_1 截止, F 为 1 状态, E 为 0 状态。所以, 可用 F 点电平的高低来表示“1”和“0”两种信息。

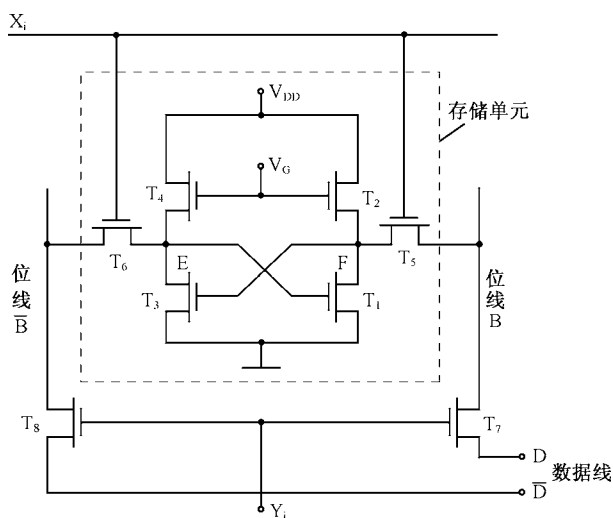


图 7.3 基本的 6 管 NMOS 静态存储单元

T_5 、 T_6 是门控管, 由 X_i 线控制其导通或截止, 他们用来控制触发器输出端与位线之间的连接状态。 T_7 、 T_8 也是门控管, 其导通与截止受 Y_j 线控制, 他们是用来控制位线与数据线之间连接状态的, 工作情况与 T_5 、 T_6 类似。但并不是每个存储单元都需要这两只管子, 而是一列存储单元用两只。所以, 只有当存储单元所在的行、列对应的 X_i 、 Y_j 线均为 1 时, 于是 E 、 F 两点与 $\bar{D}$ 、 D 分别连通, 从而可以进行读/写操作。这种情况称为选中状态。

以写操作为例, 讲解一下基本的静态存储单元工作原理。写操作时, 如果要写入“1”, 则在 D 线上加上高电平, 在 $\bar{D}$ 线上加上低电平, 行、列对应的 X_i 、 Y_j 线均为 1 时, 通过导通的 T_5 、 T_6 、 T_7 、 T_8 4 个晶体管, 把高、低电平分别加在 E 、 F 点, 即 $F = “1”$, $E = “0”$, 使 T_1 管截止, T_3 管导通。当输入信号和地址选择信号(即行、列选通信号)消失以后, 即行、列对应的 X_i 、 Y_j 线均为 0 时, T_5 、 T_6 、 T_7 、 T_8 管全都截止, T_1 和 T_3 管就保持被强

迫写入的状态不变,从而将“1”写入存储电路。此时,各种干扰信号不能进入 T_1 和 T_3 管。所以,只要不掉电,写入的信息不会丢失。写入“0”的操作与其类似,只是在 $\bar{D}$ 线上加上低电平,在 $\bar{D}$ 线上加上高电平即可。

2. 双极型晶体管构成静态存储单元(SRAM)的工作原理

图 7.4 是一个双极型晶体管存储单元电路,它用两只多发射极三极管和两只电阻构成一个触发器,一对发射极接在同一条字线上,另一对发射极分别接在位线 B 和 $\bar{B}$ 上。

在维持状态,字线电位约为 0.3V,低于位线电位(约 1.1V),因此存储单元中导通管的电流由字线流出,而与位线连接的两个发射结均处于反偏状态,相当于位线与存储器断开。处于维持状态的存储单元可以是 T_1 导通、 T_2 截止(称为 0 状态),也可以是 T_2 导通、 T_1 截止(称为 1 状态)。

当单元被选中时,字线电位被提高到 2.2V 左右,位线的电位低于字线,于是导通管的电流转而从位线流出。如果要读出,只要检测其中一条位线有无电流即可。例如可以检测位线 $\bar{B}$,若存储单元为 1 状态,则 T_2 导通,电流由 $\bar{B}$ 线流出,经过读出放大器转换为电压信号,输出为 1;若存储单元为 0 状态,则 T_2 截止, $\bar{B}$ 线中无电流,读出放大器无输入信号,输出为 0。

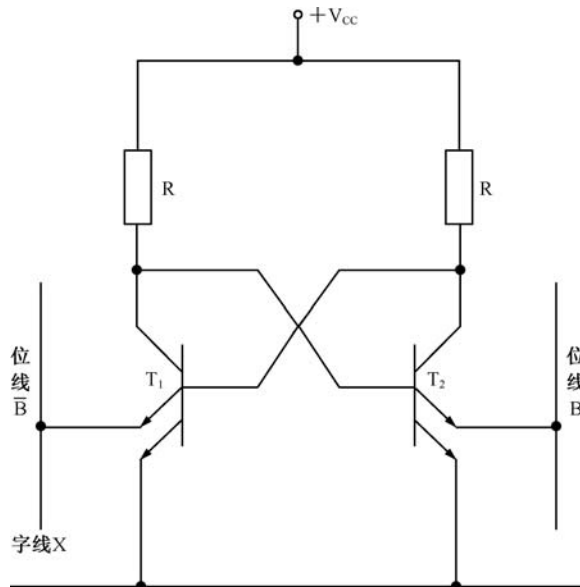


图 7.4 双极型晶体管存储单元

要写入 1,则存储器输入端的 1 信号通过写入电路使 $B=1$ 、 $\bar{B}=0$,将位线 B 切断(无电流),迫使 T_1 截止, T_2 导通, T_2 的电流由位线 $\bar{B}$ 流出。当字线恢复到低电平后, T_2 电流再转向字线,而存储单元状态不变,这样就完成了写 1;若要写 0,则令 $B=0$, $\bar{B}=1$,使位线 $\bar{B}$ 切断,迫使 T_2 截止、 T_1 导通。

3. 典型的静态 RAM 芯片

SRAM 的使用十分方便,在微型计算机领域有着极其广泛的应用。下面就以典型的

SRAM 芯片 6264 为例, 说明它的外部特性及工作过程。

1) 6264 存储芯片的引线及其功能

6264 芯片是一个 $8K \times 8\text{bit}$ 的 CMOS SRAM 芯片, 其引脚如图 7.5 所示。它共有 28 条引出线, 包括 13 根地址线、8 根数据线、4 根控制信号线及其他引线, 它们的含意分别为:

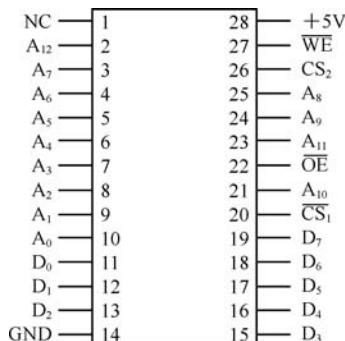


图 7.5 6264 外部引线

(1) $A_0 \sim A_{12}$, 13 根地址信号线。一个存储芯片上地址线的多少决定了该芯片有多少个存储单元。13 根地址信号线上的地址信号编码最大为 2^{13} , 即 8192(8K)个。也就是说, 芯片的 13 根地址线上的信号经过芯片的内部译码, 可以决定选中 6264 芯片上 8K 个存储单元中的哪一个。在与系统连接时, 这 13 根地址线通常接到系统地址总线的低 13 位上, 以便 CPU 能够寻址芯片上的各个单元。

(2) $D_0 \sim D_7$, 8 根双向数据线。对 SRAM 芯片来讲, 数据线的根数决定了芯片上每个存储单元的二进制位数, 8 根数据线说明 6264 芯片的每个存储单元中可存储 8 位二进制数, 即每个存储单元有 8 位。使用时, 这 8 根数据线与系统的数据总线相连。当 CPU 存取芯片上的某个存储单元时, 读出和写入的数据都通过这 8 根数据线传送。

(3) $\overline{CS}_1$ 和 CS_2 , 片选信号线。当 $\overline{CS}_1$ 为低电平、 CS_2 为高电平($\overline{CS}_1 = 0$, $CS_2 = 1$)时, 该芯片被选中, CPU 才可以对它进行读/写。不同类型的芯片, 其片选信号的数量不一定相同, 但要选中该芯片, 必须所有的片选信号同时有效才行。事实上, 一个微机系统的内存空间是由若干块存储器芯片组成的, 某块芯片映射到内存空间的哪一个位置(即处于哪一个地址范围)上, 是由高位地址信号决定的。系统的高位地址信号和控制信号通过译码产生选片信号, 将芯片映射到所需要的地址范围上。6264 有 13 根地址线($A_0 \sim A_{12}$), 8086/8088 CPU 则有 20 根地址线, 所以这里的高位地址信号就是 $A_{13} \sim A_{19}$ 。有关地址译码, 我们将在下边详细介绍。

(4) $\overline{OE}$, 输出允许信号。只有当 $\overline{OE}$ 为低电平时, CPU 才能够从芯片中读出数据。

(5) $\overline{WE}$, 写允许信号。当 $\overline{WE}$ 为低电平时, 允许数据写入芯片; 而当 $\overline{WE} = 1$, $\overline{OE} = 0$ 时, 允许数据从该芯片读出。

表 7-1 总结了以上 4 个控制信号的功能。

(6) 其他引线: V_{CC} 为 +5V 电源, GND 是接地端, NC 表示空端。

表 7-1 6264 真值表

$\overline{\text{WE}}$	$\overline{\text{CS}}_1$	CS_2	$\overline{\text{OE}}$	$\text{D}_0 \sim \text{D}_7$
0	0	1	×	写入
1	0	1	0	读出
×	0	0	×	三态 (高阻)
×	1	1	×	
×	1	0	×	

2) 6264 的工作过程

对 6264 芯片的存取操作包括数据的写入和读出。

写入数据的过程是：首先把要写入单元的地址送到芯片的地址线 $\text{A}_0 \sim \text{A}_{12}$ 上；要写入的数据送到数据线上；然后使 $\overline{\text{CS}}_1$ ， CS_2 同时有效($\overline{\text{CS}}_1 = 0$ ， $\text{CS}_2 = 1$)；再在 $\overline{\text{WE}}$ 端加上有效的低电平， $\overline{\text{OE}}$ 端状态可以任意。这样，数据就可以写入指定的存储单元中。写入过程的工作时序如图 7.6 所示。

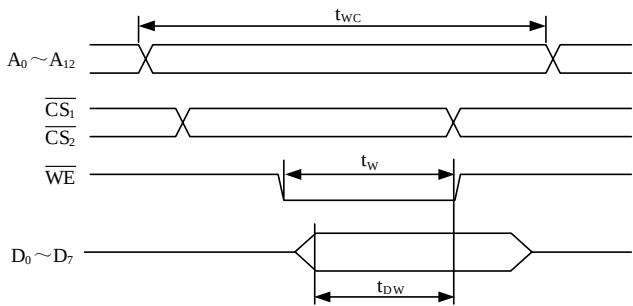


图 7.6 SRAM 6264 写操作时序图

从芯片中读出数据的过程与写操作类似：先把要读出单元的地址送到 6264 的地址线上，然后使 $\overline{\text{CS}}_1 = 0$ 和 $\text{CS}_2 = 1$ 同时有效；与写操作不同的是，此时要使读允许信号 $\overline{\text{OE}} = 0$ ， $\overline{\text{WE}} = 1$ ，这样，选中单元的内容就可从 6264 的数据线读出。读出过程的时序如图 7.7 所示。

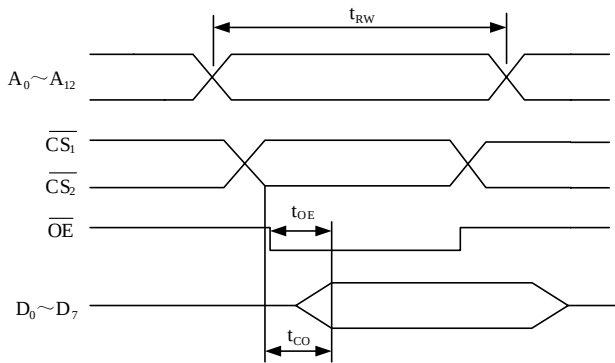


图 7.7 SRAM 6264 读操作时序图

CPU 的取指令周期和对存储器读写都有固定的时序, 因此对存储器的存取速度有一定的要求, 当对存储器进行读操作时, CPU 发出地址信号和读命令后, 存储器必须在读允许信号有效期内, 将选中单元的内容送到数据总线上。同样, 在进行写操作时, 存储器也须在写脉冲有效期间, 将数据写入指定的存储单元。否则, 就会出现读写错误。

如果现有可选择的存储器的存取速度太慢, 不能满足上述要求, 就需要设计者采取适当的措施来解决这一问题。最简单的解决办法就是降低 CPU 的时钟频率, 即延长时钟周期 T_{CLK} 。但这样做会降低系统的运行速度。另一种方法是利用 CPU 上的 READY 信号, 使 CPU 在对慢速存储器操作时插入一个或几个等待周期 T_w , 以等待存储器操作的完成。当然, 随着技术的发展, 现有存储器芯片的存取时间低达几个纳秒, 基本能够满足使用。因此这个问题在一般的微型机中已很少遇到。但在自行开发的系统中, 对此应给予足够的重视。

4. SRAM 芯片的应用

在了解了 SRAM 6264 芯片的外部引脚功能和它的工作时序之后, 更重要的是如何实现它与系统的连接。将一个存储器芯片接到总线上, 除部分控制信号及数据信号线的连接外, 主要是如何保证该芯片在整个内存中占据的地址范围能够满足用户的要求。芯片的选片信号是由高位地址信号和控制信号的译码产生的, 事实上也就是高位地址信号决定了这个芯片在整个内存中占据的地址范围。这就是片选译码(选择一个存储芯片); 然后再找一个存储单元, 这就是片内译码, 片内译码由存储芯片内部完成, 使用者无需考虑。下面就来介绍决定芯片存储地址空间的方法和如何实现译码。

存储器的地址译码方式可以分为两种: 一种称为全地址译码; 另一种称为部分地址译码。

(1) 全地址译码方式。

所谓全地址译码, 就是构成存储器时要使用全部 20 位地址总线信号, 即所有的高位地址信号用来作为译码器的输入, 低位地址信号接存储芯片的地址输入线, 从而使得存储器芯片上的每一个单元在整个内存空间中具有唯一的一个地址。

对 6264 芯片来讲, 就是用低 13 位地址信号 $A_0 \sim A_{12}$ 决定每个单元的片内地址, 即片内寻址; 而用高 7 位地址信号 $A_{13} \sim A_{19}$ 决定芯片在内存中的地址边界, 即做片选地址译码。如图 7.8 所示。

图 7.9 是 SRAM 6264 与 8086/8088 系统的连接图。图中用地址总线的高 7 位地址信号 ($A_{13} \sim A_{19}$) 作为地址译码器的输入, 地址总线的低 13 位地址信号 $A_0 \sim A_{12}$ 接到芯片的 $A_0 \sim A_{12}$ 端, 故这是一个全地址译码方式的连接。可以看出, 当 $A_{19} \sim A_{13}$ 为 0011111 时, 译码器输出为低电平, 所以该 6264 芯片的地址范围为 3E000H~3FFFFH(低 13 位可以是全 0 到全 1 之间的任何一个值)。

译码电路的构成不是唯一的, 可以利用基本逻辑门电路(如“与”、“或”、“非”门等)构成如图 7.8 所示, 也可以利用译码器 74LS138 构成。图 7.9 就是用 138 译码器实现同样地址范围的译码电路。

(2) 部分地址译码方式。

顾名思义, 部分地址译码就是仅把地址总线的一部分地址信号线与存储器连接, 通常是用高位地址信号的一部分(而不是全部)作为片选译码信号。图 7.10 就是一个部分地址译码的例子。从图中可以看出, 该 6264 芯片被映射到了以下内存空间中:

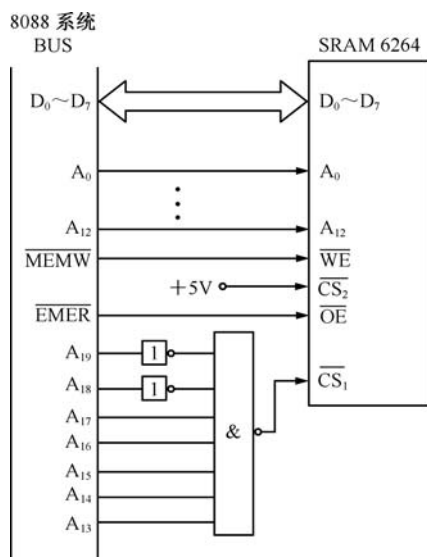


图 7.8 6264 的全地址译码连接

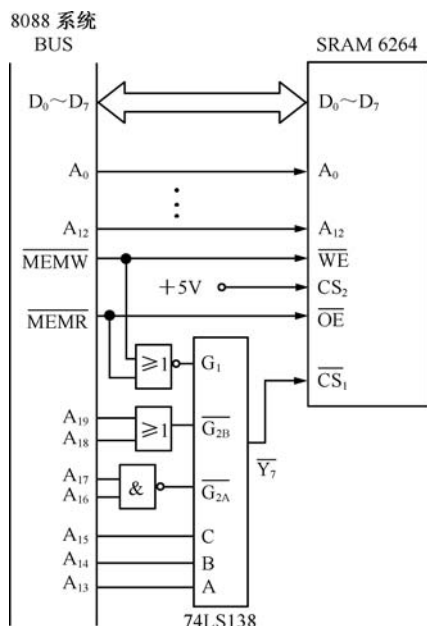


图 7.9 利用 138 译码器实现全地址译码连接

AE000H~AFFFFH

BE000H~BFFFFH

EE000H~EFFFFH

FE000H~FFFFFH

即该 6264 芯片共占据了 4 个 8KB 的内存空间，而 6264 芯片本身只有 8KB 的存储容量。为什么会出现这种情况呢？其原因就在于图中的高位地址译码并没有利用地址总线上的全部地址信号，而只利用了其中的一部分。在图 7.10 中， A_{18} 和 A_{16} 并未参加译码，因此， A_{18} 和 A_{16} 无论是什么值都不影响译码器的输出。当 A_{18} 和 A_{16} 分别为 00、01、10、11 这 4 种组合时，使 6264 这个 8KB 的存储芯片占据了 4 个 8KB 的地址空间。这种只用部分地址线参加译码从而产生地址重复区的译码方式，就是部分地址译码的含意。按这种地址译码方式，芯片占用的这 4 个 8KB 的区域绝不可再分配给其他芯片。否则，会造成总线竞争而使微机无法正常工作。另外，在对这个 6264 芯片进行存取时，可以使用以上 4 个地址范围的任意一个。

部分地址译码使地址出现重叠区，而重叠的部分必须空着不准使用，这就破坏了地址空间的连续性，实际上就是减小了总的可用存储地址空间。部分地址译码方式的优点是其译码器的构成比较简单，成本较低。图 7.10 中就少用两条译码输入线，但这是以牺牲可用内存空间为代价换来的。

可以想像，参加译码的高位地址越少，译码器就越简单，而同时所构成的存储器所占用的内存地址空间就越多。若只用一条高位地址线做选片信号，如在图 7.10 中，若只将 A_{19} 接在 $\overline{CS}_1$ 上，则这片 6264 芯片将占据 00000H~7FFFFH 共 512KB 的地址空间。这种只用一条高位地址线进行选片的连接方法称为线性选择，这种地址译码方法一般仅在系统中只使用 1~2 个存储芯片时可考虑使用。

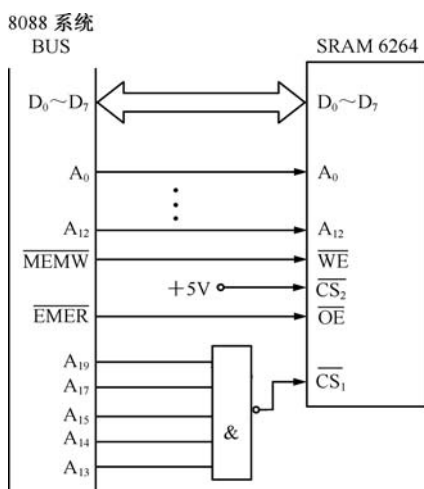


图 7.10 SRAM 6264 的部分地址译码连接图

在实践中,采用全地址译码还是部分地址译码,应根据具体情况来定。如果地址资源很富裕,为使电路简单,可考虑用部分地址译码方式。如果要充分利用地址空间,则应采用全地址译码方式。6264 芯片的功耗很小(工作时为 15mW,未选中时仅 10 μ W),因此在简单的应用系统中,CPU 可直接和存储器相连,不用增加总线驱动电路。

典型的静态 RAM 芯片还有有 2114(1KB $\times$ 4 位)、6116(2KB $\times$ 8 位)、6264(8KB $\times$ 8 位)、62256(32KB $\times$ 8 位)、628128(128KB $\times$ 8 位)等。

【例 7.1】Intel 2114 SRAM 芯片的容量为 1K $\times$ 4 位,18 脚封装,+5V 电源,芯片引脚和逻辑符号图及芯片内部结构如图 7.11 和图 7.12 所示。

由于 1K $\times$ 4=4096,所以 Intel 2114 SRAM 芯片有 4096 个基本存储电路,将 4096 个基本存储电路排成 64 行 $\times$ 64 列的存储矩阵,每根列选线连接 4 条列地址线,对应于并行的 4 位(位于同一行的 4 位应作为同一单元的内容被同时选中)存储单元,每个单元有 4 位,从而构成了 64 行 $\times$ 16 列=1K 存储单元。1K 个存储单元应有 A₀~A₉ 共计 10 个地址输入端,2114 片内地址译码采用双译码方式,A₃~A₈ 共 6 根地址线接到行地址译码输入端,经行译码产生 64 根行地址选择线;A₀、A₁、A₂ 和 A₉ 共计 4 根地址线接到列地址译码输入端,经过列译码产生 16 根列地址选择线。地址输入线 A₀~A₉ 送来的地址信号分别送到行、列地址译码器,经译码后选中一个存储单元(有 4 个存储位)。当片选信号 $\overline{CS}=0$ 且读写信号 $\overline{WE}=0$ 时,输入数据控制三态门打开,I/O 电路对被选中单元的 4 位进行写入;当 $\overline{CS}=0$ 且 $\overline{WE}=1$ 时,输入数据控制三态门关闭,而数据输出三态门打开,I/O 电路将被选中单元的 4 位信息读出送数据线;当 $\overline{CS}=1$ 即 $\overline{CS}$ 无效时,不论 $\overline{WE}$ 为何种状态,各个三态门的输出均为高阻状态,芯片不工作。

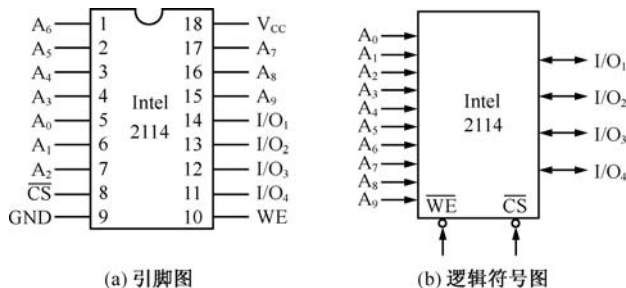


图 7.11 Intel 2114 引脚及逻辑符号

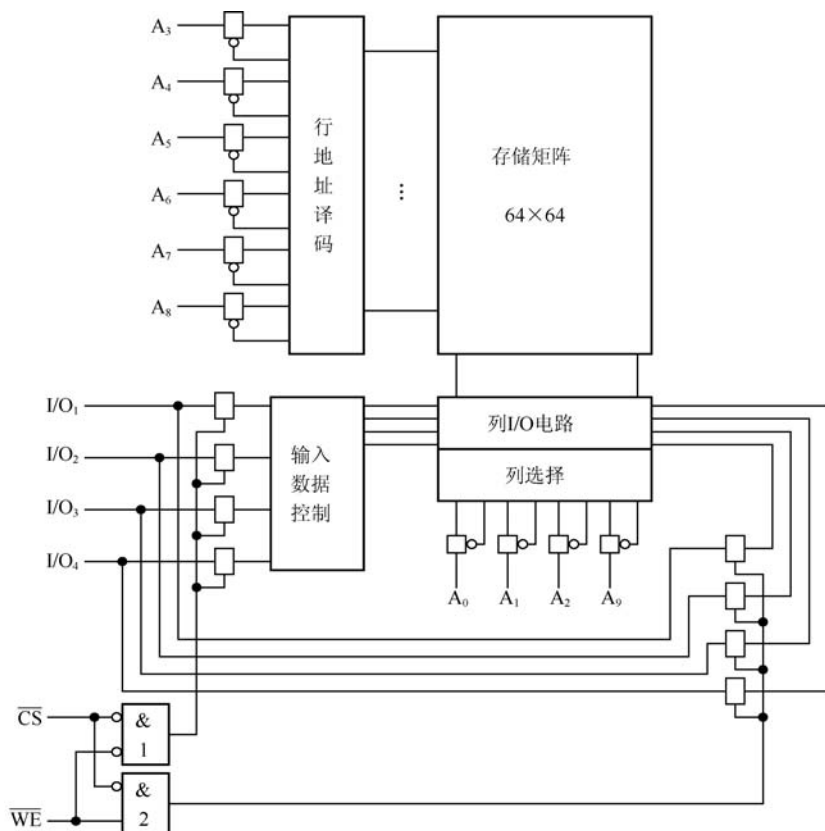


图 7.12 Intel 2114 的内部结构

7.2.2 动态存储器

1. 四管动态 MOS 存储单元(DRAM)的工作原理

DRAM 的存储元有两种结构,四管存储元和单管存储元。四管存储元的缺点是元件多,占用芯片面积大,故集成度较低,但外围电路较简单,使用简单。单管电路的元件数量少,集成度高,但外围电路比较复杂。这里仅简单介绍一下四管存储元的存储原理。

动态 MOS 存储单元存储信息的原理,是利用 MOS 管栅极电容具有暂时存储信息的作用。由于漏电流的存在,栅极电容上存储的电荷不可能长久保持不变,因此为了及时补充漏掉的电荷,避免存储信息丢失,需要定时地给栅极电容补充电荷,通常把这种操作称作刷新或再生。

图 7.13 所示是四管动态 MOS 存储单元电路。 T_1 和 T_2 交叉连接,信息(电荷)存储在 C_1 、 C_2 上。 C_1 、 C_2 上的电压控制 T_1 、 T_2 的导通或截止。当 C_1 充有电荷(电压大于 T_1 的开启电压), C_2 没有电荷(电压小于 T_2 的开启电压)时, T_1 导通、 T_2 截止,称此时存储单元为 0 状态;当 C_2 充有电荷, C_1 没有电荷时, T_2 导通、 T_1 截止,则称此时存储单元为 1 状态。 T_3 和 T_4 是门控管,控制存储单元与位线的连接。

T_5 和 T_6 组成对位线的预充电电路,并且为列中所有存储单元所共用。在访问存储器开始时, T_5 和 T_6 栅极上加“预充”脉冲, T_5 、 T_6 导通,位线 B 和 $\bar{B}$ 被接到电源 V_{DD} 而变为

高电平。当预充脉冲消失后, T_5 、 T_6 截止, 位线与电源 V_{DD} 断开, 但由于位线上分布电容 C_B 和 $C_{\bar{B}}$ 的作用, 可使位线上的高电平保持一段时间。

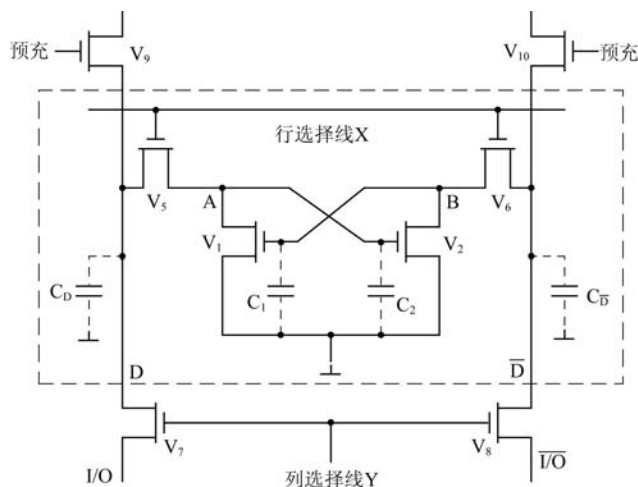


图 7.13 四管动态 MOS 存储单元

在位线保持为高电平期间, 当进行读操作时, X 线变为高电平, T_3 和 T_4 导通, 若存储单元原来为 0 态, 即 T_1 导通、 T_2 截止, G_2 点为低电平, G_1 点为高电平, 此时 C_B 通过导通的 T_3 和 T_1 放电, 使位线 B 变为低电平, 而由于 T_2 截止, 虽然此时 T_4 导通, 位线 $\bar{B}$ 仍保持为高电平, 这样就把存储单元的状态读到位线 B 和 $\bar{B}$ 上。如果此时 Y 线也为高电平, 则 B、 $\bar{B}$ 的信号将通过数据线被送至 RAM 的输出端。

位线的预充电电路起什么作用呢? 在 T_3 、 T_4 导通期间, 如果位线没有事先进行预充电, 那么位线 $\bar{B}$ 的高电平只能靠 C_1 通过 T_4 对 $C_{\bar{B}}$ 充电建立, 这样 C_1 上将要损失掉一部分电荷。由于位线上连接的元件较多, $C_{\bar{B}}$ 甚至比 C_1 还要大, 这就有可能在读一次后便破坏了 G_1 的高电平, 是存储的信息丢失。采用了预充电电路后, 由于位线 $\bar{B}$ 的电位比 G_1 的电位还要高一些, 所以在读出时, C_1 上的电荷不但不会损失, 反而还会通过 T_4 对 C_1 再充电, 使 C_1 上的电荷得到补充, 即进行一次刷新。当进行写操作时, RAM 的数据输入端通过数据线、位线控制存储单元改变状态, 把信息存入其中。

动态 RAM 是利用电容 C 上充积电荷来存储信息的。当电容 C 有电荷时, 为逻辑“1”, 没有电荷时, 为逻辑“0”。但由于任何电容都存在漏电, 因此, 当电容 C 存有电荷时, 过一段时间由于电容的放电过程导致电荷流失, 信息也就丢失。因此, 需要周期性地对电容进行充电, 以补充泄漏的电荷, 通常把这种补充电荷的过程叫刷新或再生。随着器件工作温度的增高, 放电速度会变快。刷新时间间隔一般要求在 1~100 ms。工作温度为 70℃ 时, 典型的刷新时间间隔为 2 ms, 因此 2 ms 内必须对存储的信息刷新一遍。尽管对各个基本存储电路在读出或写入时都进行了刷新, 但对存储器中各单元的访问具有随机性, 无法保证一个存储器中的每一个存储单元都能在 2 ms 内进行一次刷新, 所以需要系统地对存储器进行定时刷新。

DRAM 集成度高、价格低, 在微型计算机中使用极其广泛。构成微机内存的内存条几乎都是由 DRAM 组成的。

2. 典型的动态 RAM 芯片

下面以典型的动态 RAM 芯片有 Intel 2164A(64KB×1 位)为例,介绍一下动态存储器的内部结构和管脚功能。

Intel 2164A 芯片的存储容量为 64K×1 位,采用单管动态基本存储电路,每个单元只有一位数据,其内部结构如图 7.14 所示。2164A 芯片的存储体本应构成一个 256×256 的存储矩阵,为提高工作速度(需减少行列线上的分布电容),将存储矩阵分为 4 个 128×128 矩阵,每个 128×128 矩阵配有 128 个读出放大器,各有一套 I/O 控制电路。

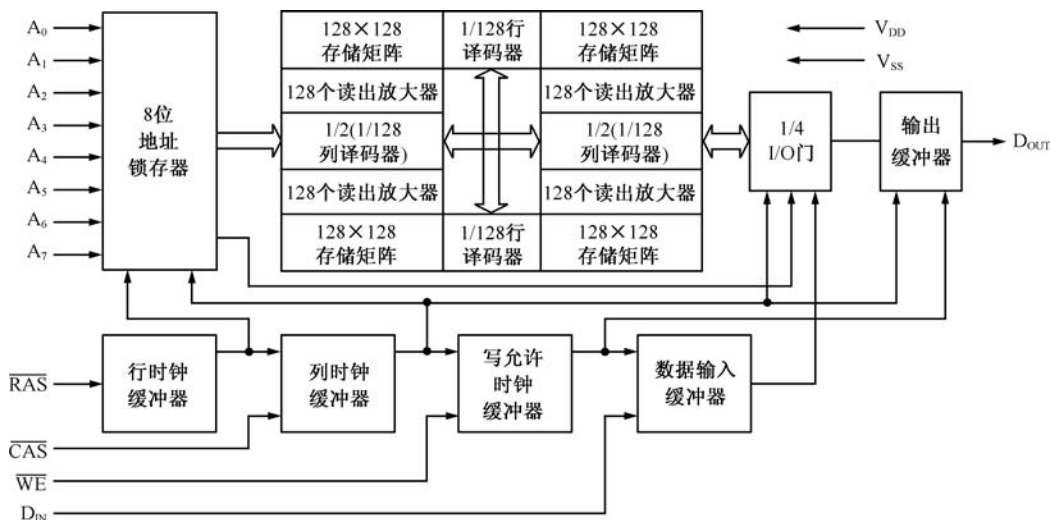


图 7.14 Intel 2164A 内部结构示意图

芯片 Intel 2164A 的容量为 64K×1 位,即片内共有 64K(65536)地址单元,每个地址单元存放一位数据。需要 16 条地址线,地址线分为两部分:行地址线与列地址线。

芯片的地址引线只要 8 条,内部设有地址锁存器,利用多路开关,由行地址选通信号变低 $\overline{\text{RAS}}$ (Row Address Strobe),把先出现的 8 位地址,送至行地址锁存器;由随后出现的列地址选通信号 $\overline{\text{CAS}}$ (Column Address Strobe)把后出现的 8 位地址送至列地址锁存器。这 8 条地址线也用于刷新(刷新时地址计数,实现一行一行地刷新)。

64K 容量本需 16 位地址,但芯片引脚(如图 7.15 所示)只有 8 根地址线, A₀~A₇ 需分时复用。在行地址选通信号 $\overline{\text{RAS}}$ 控制下先将 8 位行地址送入行地址锁存器,锁存器提供 8 位行地址 RA₇~RA₀,译码后产生两组行地址选择线,每组 128 根。然后在列地址选通信号 $\overline{\text{CAS}}$ 控制下将 8 位列地址送入列地址锁存器,锁存器提供 8 位列地址 CA₇~CA₀,译码后产生两组列地址选择线,每组 128 根。行地址 RA₇~RA₀ 与列地址 CA₇~CA₀ 选择 4 个 128×128 矩阵之一。因此,16 位地址是分成两次送入芯片的,对于某一地址码,只有一个 128×128 矩阵和它的 I/O 控制电路被选中。A₀~A₇ 这 8 根地址线还用于在刷新时提供行地址,因为刷新是一行一行地进行的。

Intel 2164A 的读/写操作由 $\overline{\text{WE}}$ 信号来控制,读操作时, $\overline{\text{WE}}$ 为高电平,选中单元的内容经三态输出缓冲器从 D_{OUT} 引脚输出;写操作时, $\overline{\text{WE}}$ 为低电平, D_{IN} 引脚上的信息经数据输入缓冲器写入选中单元 Intel 2164A 没有片选信号,实际上用行地址和列地址选通信号

$\overline{\text{RAS}}$ 和 $\overline{\text{CAS}}$ 作为片选信号, 可见, 片选信号已分解为行选信号与列选信号两部分。

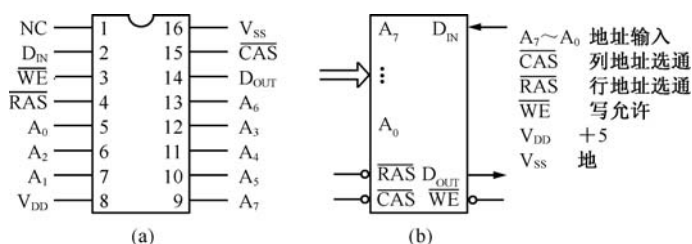


图 7.15 Intel 2164A 引脚与逻辑符号

3. 静态存储器和动态存储器芯片特性比较

静态存储器和动态存储器芯片特性比较如表 7-2 所示。

表 7-2 静态存储器和动态存储器芯片特性比较

	SRAM	DRAM
存储信息	触发器	电容
破坏性读出	非	是
需要刷新	不要	需要
送行列地址	同时送	分两次送
运行速度	快	慢
集成度	低	高
发热量	大	小
存储成本	高	低

7.3 只读存储器

只读存储器 ROM(Read-Only Memory), 是一种非易失性的半导体存储器件。在一般工作状态下, ROM 内容只能读出, 不能写入。只读存储器 ROM 常用于存储数字系统及计算机中不需改写的的数据, 例如数据转换表及计算机操作系统程序等。ROM 存储的数据不会因断电而消失, 即具有非易失性。对可编程的 ROM 芯片, 可用特殊方法将信息写入, 该过程被称为“编程”。对可擦除的 ROM 芯片, 可采用特殊方法将原来信息擦除, 以便再次编程。本节将先介绍存储器单元的工作原理, 再从应用的角度出发, 以几种常用的典型芯片为例, 介绍只读存储器 ROM 的工作原理、特点、外部特性以及它们的应用。

1. 掩模式只读存储器 MROM

掩模式只读存储器 MROM 中储存的信息是在芯片制造过程中就固化好了的, 用户只能选用而无法修改原存信息, 故又称为固定只读存储器 MROM。通常, 用户可将自己设计好的信息委托生产厂家在生产芯片时进行固化, 但要根据用户信息制作专用的掩模模具。MROM 采用二次光刻掩模工艺制成, 首先要制作一个掩模板, 然后根据用户程序通过掩模

板曝光,在硅片上刻出图形。制作掩模板工艺较复杂,生产周期长,因此生产第一片 MROM 的费用很大,而复制同样的 ROM 就很便宜了,所以适合于大批量生产,不适用于科学研究。MROM 有双极型、MOS 型等几种电路形式。

图 7.16 是一个简单的 4×4 位 MOS 管 ROM,采用单译码结构,两位地址线 A_1 、 A_0 经过译码器译码后有 4 种状态,输出 4 条选择线,分别选中 4 个存储单元,每个单元存储有 4 位输出。在此矩阵中,行和列的交点处有管子连接表示存储“0”信息;没有连接管子表示存储“1”信息。若地址线 $A_1A_0=00$,则选中 0 号单元,即字线 0 为高电平,若有管子与其相连(如位线 D_2 、 D_0),其相应的 MOS 管导通,位线输出为 0,而位线 D_3 、 D_1 没有管子与字线相连,则输出为 1。因此,存储单元 0 输出为 1010。对于图 7.16 中矩阵,存储单元 1 输出为 1101;存储单元 2 输出为 0101;存储单元 3 输出为 0110。

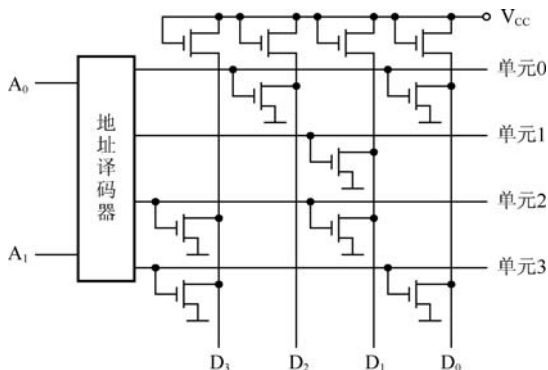


图 7.16 掩模式 ROM 示意图

2. 可编程只读存储器(PROM)

可编程只读存储器出厂时各单元内容全为 0,用户可用专门的 PROM 写入器将信息写入,这种写入是破坏性的,即某个存储位一旦写入 1,就不能再变为 0,因此对这种存储器只能进行一次编程。根据写入原理 PROM 可分为两类:结破坏型和熔丝型。图 7.17 是熔丝型 PROM 的一个存储单元示意图。

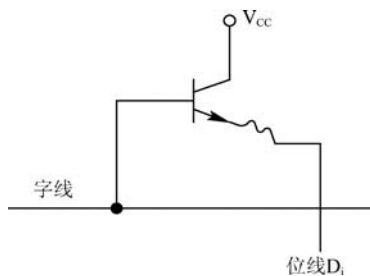


图 7.17 熔丝 PROM 的一个存储单元示意图

基本存储电路由 1 个三极管和 1 根熔丝组成,可存储一位信息。出厂时,每一根熔丝都与位线相连,存储的都是“0”信息。如果用户在使用前根据程序的需要,利用编程写入器对选中的基本存储电路通以 $20\text{mA} \sim 50\text{mA}$ 的电流,将熔丝烧断,则该存储元将存储信息“1”。由于熔丝烧断后无法再接通,因而 PROM 只能一次编程写入,编程后就不能再修

改。

写入时,按给定地址译码后,译码器的输出端选通字线,根据要写入信息的不同,在位线上加上不同的电位,若 D_i 位要写“0”,则对应位线 D_i 悬空(或接上较大电阻)而使流经被选中基本存储电路的电流很小,不足以烧断熔丝,则该 D_i 位的状态仍然保持“0”状态;若要写“1”,则位线 D_i 位加上负电位(2V),瞬间通过被选基本存储电路的电流很大,致使熔丝烧断,即 D_i 位的状态改写为“1”。在正常只读状态工作时,加到字线上的是比较低的脉冲电位,但足以开通存储元中的晶体管。这样,被选中单元的信息就一并读出了。是“0”,则对应位线有电流;是“1”,则对应位线无电流。在只读状态,工作电流将会很小,不会造成熔丝烧断,即不会破坏原来存储的信息。

3. 可擦除可编程的只读存储器 EPROM(Erasable PROM)

PROM 虽然可供用户进行一次编程,但仍有局限性。为了便于研究工作,实验各种 ROM 程序方案,可擦除、可再编程 ROM 在实际中得到了广泛应用。这种存储器利用编程器写入信息,此后便可作为只读存储器来使用。

目前,根据擦除芯片内已有信息的方法不同,可擦除、可再编程 ROM 可分为两种类型:紫外线擦除 PROM(简称 EPROM)和电擦除 PROM(简称 EEPROM 或 E^2 PROM)。

1) 紫外线可擦除的只读存储器 EPROM

EPROM 是可以反复(通常多于 100 次)擦除原来写入的内容,更新写入新信息的只读存储器。EPROM 成本较高,可靠性不如掩模式 ROM 和 PROM,但由于它能多次改写,使用灵活,所以常用于产品研制开发阶段。初期的 EPROM 元件用的是浮栅雪崩注入 MOS,记为 FAMOS。它的集成度低,用户使用不方便,速度慢,因此很快被性能和结构更好的叠栅注入 MOS 即 SIMOS 取代。

SIMOS 管结构图如图 7.18(a)所示。它属于 NMOS,与普通 NMOS 不同的是有两个栅极,一个是控制栅 CG,另一个是浮栅 FG。FG 在 CG 的下面,被绝缘材料 SiO_2 所包围,与四周绝缘。单个 SIMOS 管构成一个 EPROM 存储元件, SIMOS EPROM 元件电路如图 7.18(b)所示。

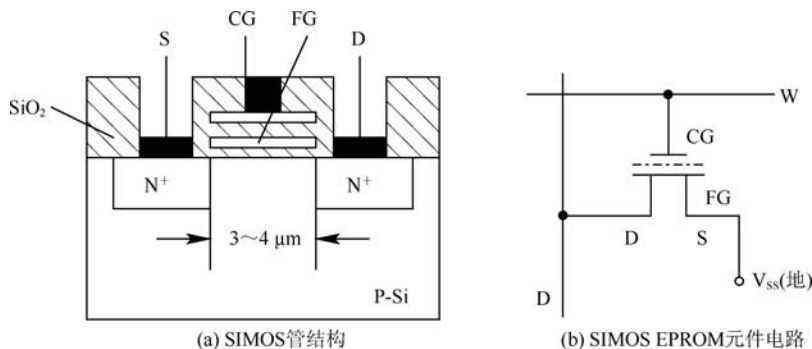


图 7.18 SIMOS 型 EPROM

与 CG 连接的线 W 称为字线,读出和编程时作选址用。漏极 D 与位线 D 连接,读出或编程时用作输出、输入信息。源极 S 接 V_{SS} (接地)。当 FG 上没有电子驻留时,CG 开启电压为正常值 V_{CC} ,若字线 W 线上加高电平,源极和漏极之间也加高电平, SIMOS 形成沟道并导通,称此状态为“1”态。当 FG 上有电子驻留,CG 开启电压升高超过 V_{CC} ,这时若字线 W 线上加高电平,源极和漏极之间仍加高电平, SIMOS 不导通,称此状态为“0”

态。人们就是利用 SIMOS 管 FG 上有无电子驻留来存储信息的。因 FG 上电子被绝缘材料包围, 不获得足够能量很难跑掉, 所以可以长期保存信息, 即使断电也不丢失。

SIMOS EPROM 芯片出厂时 FG 上是没有电子的, 即都是“1”信息。对它编程, 就是在 CG 和漏极 D 都加高电压, 向某些元件的 FG 注入一定数量的电子, 把它们写为“0”。EPROM 封装方法与一般集成电路不同, 需要有一个能通过紫外线的石英窗口。擦除时, 将芯片放入擦除器的小盒中, 用紫外线灯照射约 20 分钟后, 若再读出各单元内容均为 FFH, 说明原信息已被全部擦除, 恢复到出厂状态。写好信息的 EPROM 为了防止因光线长期照射而引起的信息破坏, 常用遮光胶纸贴于石英窗口上。

EPROM 的擦除是对整个芯片进行的, 不能只擦除某个单元或者某个位, 擦除时间较长, 并且擦/写均需离线操作, 使用起来不方便, 因此, 能够在线擦写的 E^2 PROM 芯片近年来得到广泛应用。

2) 电可擦除可编程的只读存储器 EEPROM(Electrically Erasable PROM)

EEPROM(也称为 E^2 PROM)是一种采用金属氮氧化硅(MNOS)工艺生产的可擦除可再编程的只读存储器。擦除时只需加高压对指定单元产生电流, 形成“电子隧道”, 将该单元信息擦除, 其他未通电流的单元内容保持不变。 E^2 PROM 具有对单个存储单元在线擦除与编程的能力, 而且芯片封装简单, 对硬件线路没有特殊要求, 操作简便, 信息存储时间长, 因此, E^2 PROM 给需要经常修改程序和参数的应用领域带来了极大的方便。但与 EPROM 相比, E^2 PROM 具有集成度低、存取速度较慢、完成程序在线改写需要较复杂的设备、重复改写的次数有限制(因氧化层被磨损)缺点。

其主要优点是能在应用系统中进行在线读写, 并在断电情况下保存的数据信息不丢失。EEPROM 的另外一个优点是擦除时可以按字节分别进行(不像 EPROM 擦除时要把整个片子的内容通过紫外光照射全变为 1), 也可以全片擦除, EEPROM 的擦除不需紫外光的照射, 写入时也不需要专门的编程设备。因而使用上比 EPROM 方便。

采用 +5V 电擦除的 E^2 PROM, 通常不需设置单独的擦除操作, 可在写入的过程中自动擦除。增加了一个控制栅极, 单元结构如图 7.19 所示。在擦数据时, 较高的编程电压施加在源极上, 控制栅极接地, 在此电场的作用下, 浮置栅极上的电子就越过二氧化硅形成的氧化层进入源区, 被外加电源中和掉。数据擦写操作分为擦除操作和写入操作两步, 也就是说, 在写入数据之前需要将原有数据擦除, 使各存储单元处于“1”态, 然后在下一个写周期中将数据写入。数据擦除过程分为字节擦除和全片擦除两种, 有些操作允许信号 $\overline{WE}$ 的宽度控制。在字擦除时, $\overline{WE}$ 的宽度为 10ms; 在全片擦除时, $\overline{WE}$ 的宽度达到 20ms。因此, 对一个存储单元修改需要两个连续的写周期。擦写时需要采用 21V 电压, 为此而设置的升压电路一般集成在芯片内部, 外部只需要提供单一的 5V 电源即可。

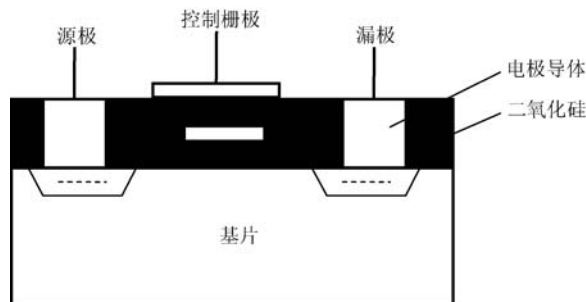


图 7.19 EEPROM 存储单元结构示意图

4. 典型的 EPROM 芯片

以一种典型的 EPROM 芯片 2764 为例，来介绍这类芯片的特点和应用。

1) 引线及其功能

2764 的外部引线如图 7.20 所示。这是一块 $8K \times 8$ 位的 EPROM 芯片，它的引线与前边介绍的 SRAM 芯片 6264 是兼容的，这给使用者带来很大方便。因为在软件调试过程中，程序经常需要修改，此时可将程序先放在 6264 中，读写修改都很方便。调试成功后，将程序固化在 2764 中，由于它与 6264 的引脚兼容，所以可以把 2764 直接插在原 6264 的插座上。这样，程序就不会由于断电而丢失。

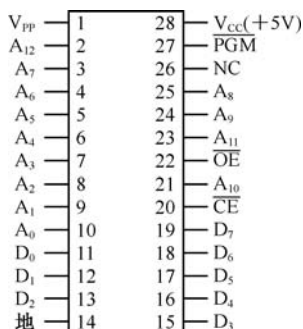


图 7.20 EPROM 2764 引线图

2764 各引脚的含义如下：

- (1) $A_0 \sim A_{12}$ ：13 根地址输入线。用于寻址片内的 $8K$ 个存储单元。
- (2) $D_0 \sim D_7$ ：8 根双向数据线，正常工作时为数据输出线。编程时为数据输入线。
- (3) $\overline{CE}$ ：选片信号。低电平有效。当 $\overline{CE}=0$ 时表示选中此芯片。
- (4) $\overline{OE}$ ：输出允许信号。低电平有效。当 $\overline{OE}=0$ 时，芯片中的数据可由 $D_0 \sim D_7$ 端输出。
- (5) $\overline{PGM}$ ：编程脉冲输入端。对 EPROM 编程时，在该端加上编程脉冲。读操作时 $\overline{PGM}=1$ 。
- (6) V_{PP} ：编程电压输入端。编程时应在该端加上编程高电压，不同的芯片对 V_{PP} 的值要求的不同，可以是 $+12.5V$ ， $+15V$ ， $+21V$ ， $+25V$ 等。

2) 2764 的工作过程

2764 可以工作在读出、编程写入和擦除 3 种方式下。

(1) 数据读出：这是 2764 的基本工作方式，用于读出 2764 中存储的内容。其工作过程与 RAM 芯片非常类似。即先把要读出的存储单元地址送到 $A_0 \sim A_{12}$ 地址线上，然后使 $\overline{CE}=0$ ，就可在芯片的 $D_0 \sim D_7$ 上读出需要的数据；在读方式下，编程脉冲输入端 $\overline{PGM}$ 及编程电压 V_{PP} 端都接在 $+5V$ 电源 V_{CC} 上。

(2) EPROM 的编程写入：对 EPROM 芯片的编程可以有两种方式，一种是标准编程；另一种是快速编程。

标准编程是每给出一个编程负脉冲就写入一个字节的数。具体的方法是： V_{CC} 接 $+5V$ ， V_{PP} 加上芯片要求的高电压；在地址线 $A_0 \sim A_{12}$ 上给出要编程存储单元的地址，然后使 $\overline{CE}=0$ 、 $\overline{OE}=1$ ；并在数据线上给出要写入的数据。上述信号稳定后，在 $\overline{PGM}$ 端加上 $50 \pm$

5ms 的负脉冲,就可将一个字节的数据写入相应的地址单元中。不断重复这个过程,就可将要写的数据逐一写入对应的存储单元中。

如果其他信号状态不变,只是在每写入一个单元的数据后将 $\overline{OE}$ 变低,则可以立即对刚写入的数据进行校验。当然也可以写完所有单元后再统一进行校验。若检查出写入数据有错,则必须全部擦除,再重新开始上述的编程写入过程。

早期的 EPROM 采用的都是标准编程方法。这种方法有两个严重的缺点:一是编程脉冲太宽(50ms)而使编程时间太长,对于容量较大的 EPROM,其编程的时间将长得令人难以接受。例如,对 256KB 的 EPROM,其编程时间长达 3.5 个小时以上。二是不够安全,编程脉冲太宽会使芯片功耗过大而损坏 EPROM。

快速编程与标准编程的工作过程是一样的,只是编程脉冲要窄得多。

(3) 擦除: EPROM 的一个重要优点是可以擦除重写,而且允许擦除的次数超过上万次。一片新的或擦除干净 EPROM 芯片,其每一个存储单元的内容都是 FFH。要对一个使用过的 EPROM 进行编程,则首先应将其放到专门的擦除器上进行擦除操作。擦除器利用紫外线光照射 EPROM 的窗口,一般经过 15~20min 即可擦除干净。擦除完毕后可读一下 EPROM 的每个单元,若其内容均为 FFH,就认为擦除干净了。

5. 典型的 EEPROM 芯片

下面以一个典型的 EEPROM 芯片 NMC98C64A 为例介绍 EEPROM 的工作过程和应用。

1) 98C64A 的引线

NMC98C64A 为 8K×8 位的 EEPROM,其引线如图 7.21 所示。其中:

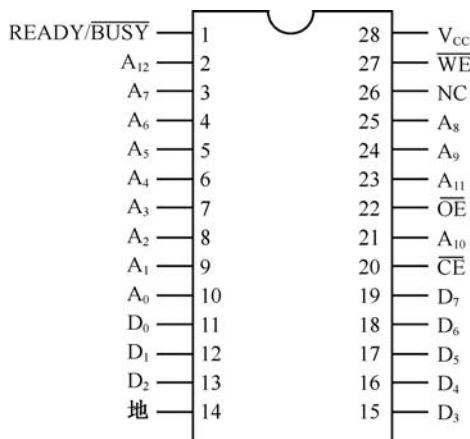


图 7.21 NMC98C64A 引线图

- (1) A₀~A₁₂ 为地址线,用于选择片内的 8K 个存储单元。
- (2) D₀~D₇ 为 8 条数据线。
- (3) $\overline{CE}$ 为选片信号。低电平有效。当 $\overline{CE}=0$ 时选中该芯片。
- (4) $\overline{OE}$ 为输出允许信号。当 $\overline{CE}=0$, $\overline{OE}=0$, $\overline{WE}=1$ 时,可将选中的地址单元的数据读出。这与 6264 很相似。
- (5) $\overline{WE}$ 是写允许信号。当 $\overline{CE}=0$, $\overline{OE}=1$, $\overline{WE}=0$ 时,可以将数据写入指定的存储

单元。

(6) $\overline{\text{READY}}/\overline{\text{BUSY}}$ 是状态输出端。98C64A 正在执行编程写入时, 此管脚为低电平。写完后, 此管脚变为高电平。因为正在写入当前数据时, 98C64A 不接收 CPU 送来的下一个数据, 所以 CPU 可以通过检查此管脚的状态来判断写操作是否结束。

2) 98C64A 的工作过程

98C64A 的工作过程同样包括 3 部分, 即数据读出、编程写入和擦除。

(1) 数据读出: 从 EEPROM 读出数据的过程与从 EPROM 及 RAM 中读出数据的过程是一样的。当 $\overline{\text{CE}}=0$, $\overline{\text{OE}}=0$, $\overline{\text{WE}}=1$ 时, 只要满足芯片所要求的读出时序关系, 则可从选中的存储单元中将数据读出。

(2) 数据写入: 将数据写入 98C64A 有两种方式。

① 字节写入方式是一次写入一个字节的的数据。但写完一个字节之后, 并不能立刻写下一个字节, 而是要等到 $\overline{\text{READY}}/\overline{\text{BUSY}}$ 端的状态由低电平变为高电平后, 才能开始下一个字节的写入。这是 EEPROM 芯片与 RAM 芯片在数据写入上的一个很重要的区别。

不同的芯片写入一个字节所需的时间略有不同, 一般是几到几十毫秒。98C64A 需要的时间一般为 5ms, 最大是 10ms。在对 EEPROM 编程时, 可以通过查询 $\overline{\text{READY}}/\overline{\text{BUSY}}$ 管脚的状态来判断是否写完一个字节, 也可利用该管脚的状态产生中断来通知 CPU 已写完一个字节。对于没有 $\overline{\text{READY}}/\overline{\text{BUSY}}$ 信号的芯片, 则可用软件或硬件定时的方式(定时时间应大于等于芯片的写入时间), 以保证数据的可靠写入。当然, 这种方法虽然在原理上比较简单, 但会降低 CPU 的效率。

98C64A 的编程时序如图 7.22 所示。从图中可以看出, 当 $\overline{\text{CE}}=0$, $\overline{\text{OE}}=1$ 时, 只要在 $\overline{\text{WE}}$ 端加上 100ns 的负脉冲, 便可以 will 数据写入指定的地址单元。

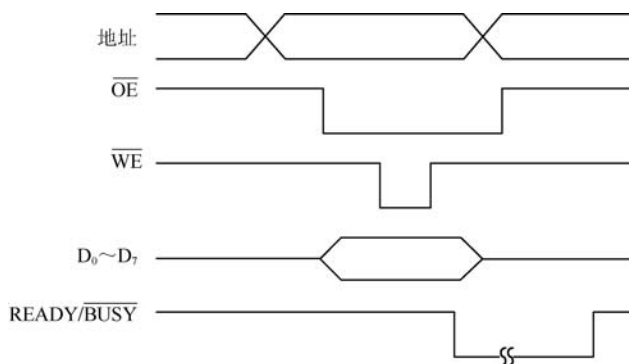


图 7.22 NMC98C64A 编程写入时序图

② 自动页写入方式: 页编程的基本思想是一次写完一页, 而不是只写一个字节。每写完一页判断一次 $\overline{\text{READY}}/\overline{\text{BUSY}}$ 端的状态。在 98C64A 中, 一页数据为 1~32 个字节, 要求这些数据在内存中是连续排列的。98C64A 的高位地址线 $A_{12} \sim A_5$ 用来决定访问哪一页数据, 低位地址 $A_4 \sim A_0$ 用来决定寻址一页内所包含的 32 个字节。因此将 $A_{12} \sim A_5$ 称为页地址。

其写入的过程是: 利用软件首先向 EEPROM 98C64A 写入页的一个数据, 并在此后的 300μs 内, 连续写入本页的其他数据, 再利用查询或中断检查 $\overline{\text{READY}}/\overline{\text{BUSY}}$ 端的状态是否

已变高，若变高，则表示这一页的数据已写结束，然后接着开始写下一页，直到将数据全部写完。

(3) 擦除：擦除和写入是同一种操作，只不过擦除总是向单元中写入“FFH”而已。EEPROM 的特点是一次既可擦除一个字节，也可以擦除整个芯片的内容。如果需要擦除一个字节，其过程与写入一个字节的完全相同，写入数据 FFH，就等于擦除了这个单元的内容。若希望一次将芯片所有单元的内容全部擦除干净，可利用 EEPROM 的片擦除功能，即在 $D_0 \sim D_7$ 上加上 FFH，使 $\overline{CE} = 0$ ， $\overline{WE} = 0$ ，并在 $\overline{OE}$ 引脚上加上 +15V 电压，使这种状态保持 10ms。就可将芯片所有单元擦除干净。

EEPROM 98C64A 有写保护电路，加电和断电不会影响芯片的内容。写入的内容一般可保存 10 年以上。每一个存储单元允许擦除/编程上万次。

7.4 快速擦除读/写存储器

尽管 EEPROM 能够在线编程，而且可以自动页写入，EEPROM 的擦除不需紫外光的照射，写入时也不需要专门的编程设备，因而使用上比 EPROM 方便。但即便如此，其编程时间相对 RAM 而言还是太长，特别是对大容量的芯片更是如此。人们希望有一种写入速度类似于 RAM，掉电后存储内容又不丢失的存储器。为此，一种新型的称为闪存(Flash Memory)快速擦除读写存储器被研制出来。

1. Flash Memory 存储单元的结构和工作原理

Flash Memory 是在 EPROM 与 EEPROM 基础上发展起来的，它与 EPROM 一样，用单管来存储一位信息，与 EEPROM 相同之处是用电来擦除。但是它只能擦除一个区域或整个器件，快速擦除读写存储单元的擦除原理图如图 7.23 所示。在源极上加高电压 V_{pp} ，控制栅极接地，在电场作用下，浮置栅上的电子越过氧化层后进入源极区而全部消失，实现一个区域擦除或全部擦除。闪存的编程速度快，掉电后存储内容又不丢失，从而得到很广泛的应用。

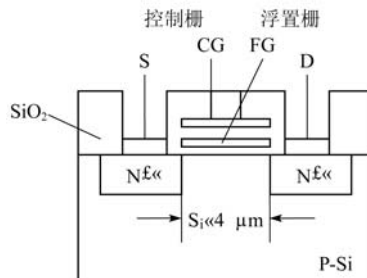


图 7.23 Flash Memory 存储单元结构

2. 典型的闪存芯片 TMS287040(16×32KB)

下面以芯片 TMS287040 为例简单介绍闪存的工作原理和应用。

1) TMS28F040 的引线

28F040 的外部引线如图 7.24 所示。 $A_0 \sim A_{18}$ 为 19 条地址线，用于选择片内的 512K 个存储单元。 $DQ_0 \sim DQ_7$ 为 8 条数据线。因为它共有 19 根地址线和 8 根数据线，说明该芯片的容量为 $512K \times 8\text{bit}$ ，28F040 芯片将其 512KB 的容量分成 16 个 32KB 的块，每一块均可独立进行擦除。 $\bar{E}$ 是芯片写允许信号，在它的下降沿锁存选中单元的地址，用上升沿锁存写入的数据。 $\bar{G}$ 为输出允许信号，低电平有效。

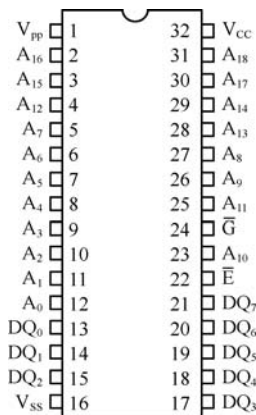


图 7.24 TMS28F040 的外部引线图

2) TMS28F040 的工作过程

28F040 与普通 EEPROM 芯片一样也有 3 种工作方式，即读出、编程写入和擦除。但也有所不同，TMS28F040 是通过向内部状态寄存器写入命令的方法来控制芯片的工作方式，对芯片所有的操作必须先往状态寄存器中写入命令。另外，TMS28F040 的许多功能需要根据状态寄存器的状态来决定。要知道芯片当前的工作状态，只需写入命令 70H，就可读出状态寄存器各位的状态了。状态寄存器各位的含义如表 7-3 所示。

表 7-3 TMS28F040 状态寄存器各位的含义

位	高电平(1)	低电平(0)	用 于
SR ₇ (D ₇)	准备好	忙	写命令
SR ₆ (D ₆)	擦除挂起	正在擦除/已完成	擦除挂起
SR ₅ (D ₅)	块或片擦除错误	片或块擦除成功	擦除
SR ₄ (D ₄)	字节编程错误	字节编程成功	编程状态
SR ₃ (D ₃)	V _{pp} 太低，操作失败	V _{pp} 合适	监测 V _{pp}
SR ₂ ~SR ₀			保留未用

(1) 读操作：读操作包括读出芯片中某个单元的内容、读出内部状态寄存器的内容以及读出芯片内部的厂家及器件标记三种情况。如果要读某个存储单元的内容，则在初始加电以后或在写入命令 00H(或 FFH)之后，芯片就处于只读存储单元的状态。这时就和读 SRAM 或 EPROM 芯片一样，很容易读出指定的地址单元中的数据。此时的 V_{pp}(编程高电压端)可与 V_{cc}(+5V)相连。

(2) 编程写入：编程方式包括对芯片单元的写入和对其内部每个 32KB 块的软件保护。软件保护是用命令使芯片的某一块或某些块规定为写保护，也可置整片为写保护状态，这样可以使被保护的块不被写入的新内容给擦除。比如，向状态寄存器写入命令 0FH，再送上要保护块的地址，就可置规定的块为写保护。若写入命令 FFH，就置全片为写保护状态。

(3) 擦除方式：28F040 既可以每次擦除一个字节，也可以一次擦除整个芯片，或根据需要只擦除片内某些块，并可在擦除过程中使擦除挂起和恢复擦除。

对字节的擦除，实际上就是在字节编程过程中，写入数据的同时就等于擦除了原单元的内容。对整片擦除，擦除的标志是擦除后各单元的内容均为 FFH。整片擦除最快只需 2.6s。但受保护的内容不被擦除，也允许对 28F040 的某一块或某些块擦除，每 32KB 为一块，块地址由 $A_{15} \sim A_{18}$ 来决定。在擦除时，只要给出该块的任意一个地址(实际上只关心 $A_{15} \sim A_{18}$)即可。

28F040 在使用中，要求在其引线控制端加上适当电平，以保证芯片正常工作。不同工作类型的 28F040 的工作条件是不一样的。

7.5 存储器的扩展

任何存储芯片的存储容量都是有限的。要构成一定容量的内存，往往单个芯片不能满足字长或存储单元个数的要求，甚至字长和存储单元数都不能满足要求。这时，就需要用多个存储芯片进行组合，以满足对存储容量的需求，这种组合就称为存储器的扩展。存储器扩展时要解决的问题主要包括位扩展、字扩展和字位扩展。

7.5.1 存储容量的位扩展

当给定的存储器芯片每个单元的位数与系统需要的内存单元字长不相等时采用的方法，称为存储容量的位扩展。

一块实际的存储芯片，每个存储单元的位数(即字长)往往与实际内存单元字长并不相等。存储芯片可以是 1 位、4 位或 8 位的，如 DRAM 芯片 Intel 2164 存储单元为 $64K \times 1$ 位，SRAM 芯片 Intel 2114 存储单元为 $1K \times 4$ 位，Intel 6264 芯片存储单元则为 $8K \times 8$ 位。而计算机中内存一般是按字节来进行组织的，若要使用 Intel 2164、Intel 2114 和 Intel 6264 这样的存储芯片来构成内存，单个存储芯片字长就不能满足要求，这时就需要进行位扩展，以满足字长的要求。

位扩展构成的存储器系统的每个单元中的内容被存储在不同的存储器芯片上。例如：用两片 $4K \times 4$ 位的存储器芯片经过位扩展构成 $4K \times 8$ 位的存储器，其连接方法如图 7.25 所示。 $4K \times 8$ 位的存储器中每个单元内的 8 位二进制数被分别存在两个芯片上，即一个芯片存储该单元内容的高 4 位，另一个芯片存储该单元内容的低 4 位。可以看出，位扩展保持总的地址单元数(存储单元个数)不变，但每个单元中的位数增加了。

由于存储器的字数与存储器芯片的字数一致， $4K = 2^{12}$ ，故只需 12 根地址线($A_{11} \sim A_0$)对各芯片内的存储单元寻址，每一芯片只有 4 位数据，所以需要两片这样的芯片，将它们的数据线分别接到数据总线($D_7 \sim D_4$)和($D_3 \sim D_0$)的相应位上。在此连接方法中，每一条地址线有两个负载，每一条数据线有一个负载。在位扩展法中，所有芯片都应同时被选中，CPU

的访存请求信号 $\overline{\text{MREQ}}$ 与各芯片 $\overline{\text{CS}}$ 片选端相连作为存储器芯片的片选输入控制信号, CPU 的读写控制信号作为存储器芯片的 $\overline{\text{R}}/\overline{\text{W}}$ 读/写控制信号。位扩展存储器工作时, 各芯片同时进行相同操作。在此例中, 若地址线 $A_{11} \sim A_0$ 上的信号为全 0, 即选中了存储器 0 号单元, 则该单元的 8 位信息是由这两个芯片 0 号单元的 4 位信息共同构成的。

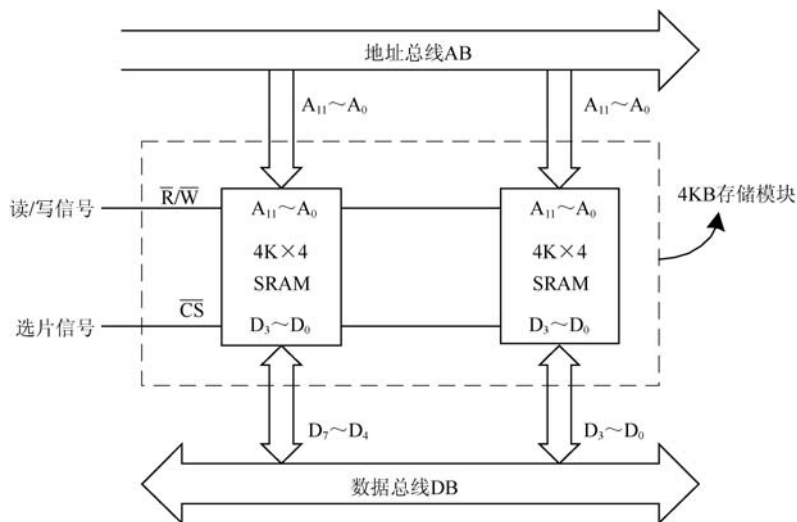


图 7.25 用 $4\text{K} \times 4$ 位的 SRAM 芯片进行位扩展以构成容量为 4KB 的存储器 Flash Memory 存储单元结构

可以看出, 位扩展的电路连接方法是: 将每个存储芯片的地址线、选片信号线和读/写信号线全部与 CPU 的相应地址线、请求信号 $\overline{\text{MREQ}}$ 线、读/写控制信号 $\overline{\text{R}}/\overline{\text{W}}$ 线进行连接, 而将它们的数据线分别连接至数据总线的不同位线上。

【例 7.2】用 Intel 2164 芯片构成容量为 64KB 的存储器。

解: 因为 Intel 2164 是 $64\text{K} \times 1$ 位的芯片, 其存储单元数也是 64K, 已满足要求。 $64\text{K} = 2^{16}$, 故只需 16 根地址线 ($A_{15} \sim A_0$) 对各芯片内的存储单元寻址。Intel 2164 字长不够, 每一块芯片只有 1 位数据, 所以需要 8 片这样的芯片, 将它们的数据线分别与 CPU 数据总线 $D_7 \sim D_0$ 的相应位相连, 将每个存储芯片的地址线、选片信号线和读/写信号线全部与 CPU 的相应地址线、请求信号线、读/写控制信号线进行连接, 线路连接如图 7.26 所示。这样就用 8 片 Intel 2164 芯片进行位扩展构成了容量为 64KB 的存储器。

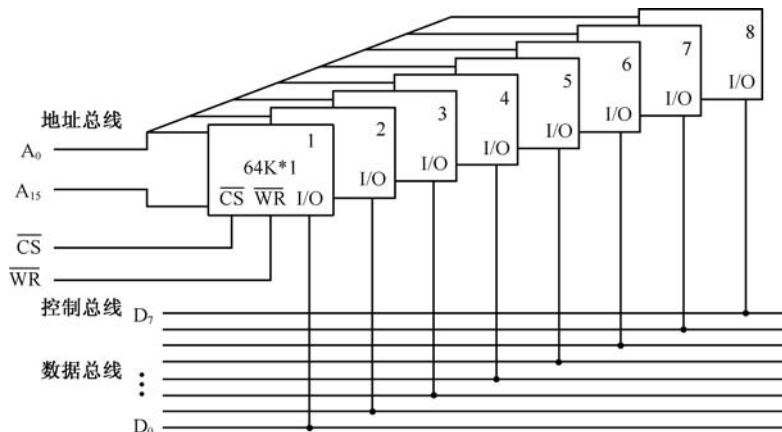


图 7.26 用 $64\text{K} \times 1$ 位的进行位扩展构成容量为 64KB 的存储器 Memory 存储单元结构

7.5.2 存储容量的字扩展

当存储芯片上每个存储单元的字长已满足要求,但存储单元的个数不够,需要增加的是存储单元的数量,就称为存储容量的字扩展。CPU 能够访存的地址空间是很大的,一片存储器芯片的字数往往小于 CPU 的地址空间。这时用字扩展法可以增加存储器的字数,而每个字的位数不变。字扩展法将地址总线分成两部分:一部分地址总线直接与各存储器地址相连,作为芯片内部寻址;一部分地址总线经过译码器译码送到存储器的片选输入端 $\overline{CE}(\overline{CS})$ 。CPU 的访存请求信号作为译码器的使能输出控制信号。CPU 的读/写控制信号作为存储器的读/写控制信号, CPU 的数据线与存储器的对应数据线相连。

【例 7.3】 用 $16K \times 8$ 位的存储器芯片组成 $64K \times 8$ 位的内存存储器。在这里,字长已满足要求,只是容量不够,所以需要进行的是字扩展,显然,对现有的 $16K \times 8$ 位芯片存储器,需要用 4 片来实现 $64K \times 8$ 位的内存存储器。

用 $16K \times 8$ 位的存储器芯片组成 $64K \times 8$ 位的内存存储器的连线图如图 7.27 所示。因为 $16K \times 8$ 位的存储器芯片字长已满足要求,4 个芯片的数据线与数据总线 $D_7 \sim D_0$ 并连。因为 $16K = 2^{14}$,故只需要 14 根地址线($A_{13} \sim A_0$)对各芯片内的存储单元寻址,让地址总线低位地址 $A_{13} \sim A_0$ 与 4 个 $16K \times 8$ 位的存储器芯片的 14 位地址线并行连接,用于进行片内寻址;对于 $64K \times 8$ 位的内存存储器,因为 $64K = 2^{16}$,故总共需 16 根地址线($A_{15} \sim A_0$)对内存存储单元寻址。为了区分 4 个 $16K \times 8$ 位的存储器芯片的地址范围,还需要两根($16 - 14 = 2$)高位地址总线 A_{15} 、 A_{14} 经过 2-4 个译码器译出 4 根片选信号线,分别和 4 个 $16K \times 8$ 位的存储器芯片的片选端相连。各芯片的地址范围见表 7-4。

可以看出,字扩展的连接方式是将各芯片的地址线、数据线、读/写控制信号线与 CPU 的相应地址总线、数据总线、读/写控制信号线相连,而由片选信号来区分各片地址。也就是将 CPU 低位地址总线与各芯片地址线相连,用以选择片内的某个单元;用高位地址线经译码器产生若干不同片选信号,连接到各芯片的片选端,以确定各芯片在整个存储空间中所属的地址范围。图 7.27 给出了用 4 个 $16K \times 8$ 芯片经字扩展构成一个 $64K \times 8$ 内存存储器的连接方法。

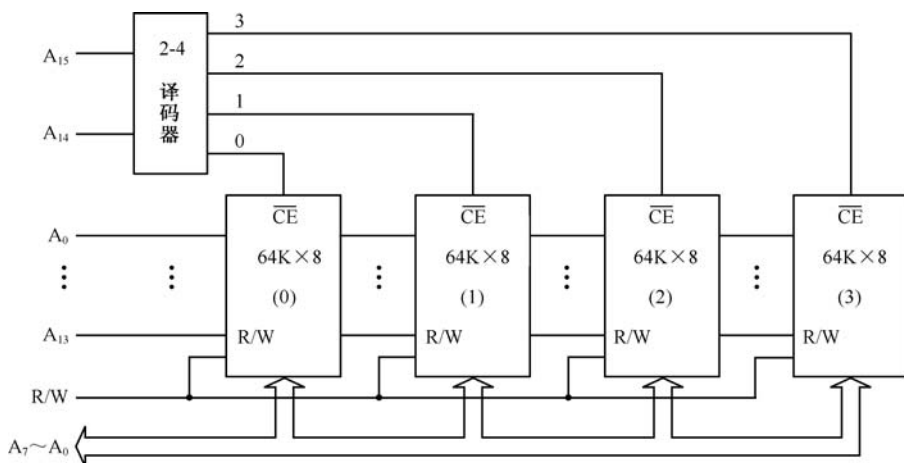


图 7.27 字扩展连接示意图

表 7-4 图 7.27 中各芯片地址空间分配表

片 号 \ 地 址	$A_{15} A_{14}$	$A_{13} A_{12} A_{11} \cdots A_1 A_0$	说 明
0	00	000...00	最低地址(0000H)
	00	111...11	最高地址(3FFFH)
1	01	000...00	最低地址(4000H)
	01	111...11	最高地址(7FFFH)
2	10	000...00	最低地址(8000H)
	10	111...11	最高地址(BFFFH)
3	11	000...00	最低地址(C000H)
	11	111...11	最高地址(FFFFH)

【例 7.4】用 $64\text{K} \times 8$ 位的 SRAM 芯片构成容量为 128KB 的存储器。

解：这里现有的芯片容量为 64KB，构成容量为 128KB 的存储器需要 $128\text{KB}/64\text{KB}=2$ 片。 $64\text{K} \times 8$ 位的 SRAM 存储器芯片字长已满足要求，2 个 $64\text{K} \times 8$ 位芯片的数据线与 CPU 数据总线 $D_7 \sim D_0$ 并行连接；因为 $64\text{K}=2^{16}$ ，故需 16 根地址线($A_{15} \sim A_0$)对各芯片内的存储单元寻址，CPU 地址总线低位地址 $A_{15} \sim A_0$ 与各芯片的 16 位地址线连接，用于进行片内寻址；对于 128KB 的存储器，因为 $128\text{K}=2^{17}$ ，故总共需 17 根地址线($A_{16} \sim A_0$)对内存储单元寻址。为了区分 2 个 $64\text{K} \times 8$ 位的存储器芯片的地址范围，还需要一根($17-16=1$)高位地址线 A_{16} ，因为片选信号低电平有效，用一根地址线不经过门电路不能产生两个有效的低电平，所以这里用两根地址线 $A_{17} \sim A_{16}$ 经过 74LS138 译码器译出 4 根片选信号线，74LS138 译码器最高位输入端 $C=0$ ， $A_{17} \sim A_{16}$ 分别接 74LS138 译码器输入端 B 和 A。这样 74LS138 译码器输入端 CBA 的输入码是 000B、001B、010B 或 011B；74LS138 译码器就只能对 0、1、2 或 3 进行译码。让 74LS138 译码器的输出 $\overline{Y}_2$ 、 $\overline{Y}_3$ 分别和两个 $64\text{K} \times 8$ 位的存储器芯片的片选端相连，线路连接如图 7.28 所示。各芯片的地址范围见表 7-5。图 7.28 中两片芯片的地址范围分别为：20000H~2FFFFH 和 30000H~3FFFFH。如果让 74LS138 译码器的输出 $\overline{Y}_0$ 、 $\overline{Y}_1$ 分别和两个 $64\text{K} \times 8$ 位的存储器芯片的片选端 $\overline{\text{CS}}$ 相连，图 7.28 中两片芯片的地址范围分别为多少呢？

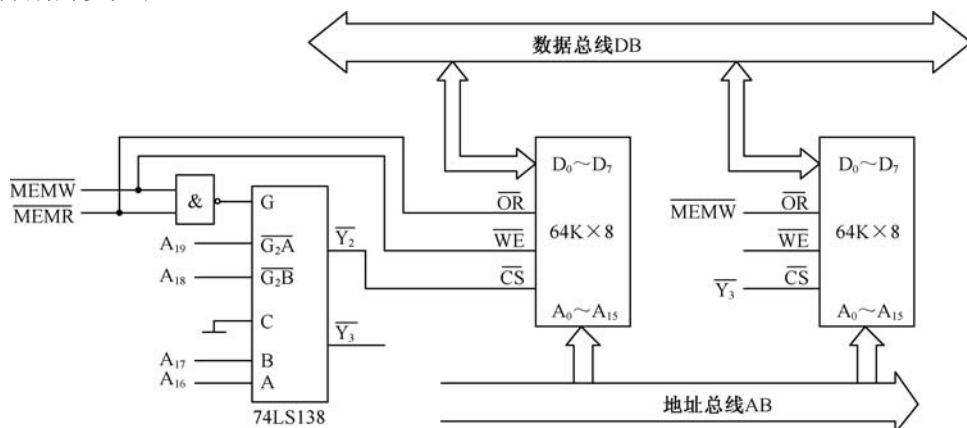
图 7.28 用 $64\text{K} \times 8$ 位的 SRAM 芯片构成容量为 128KB 的存储器 Flash Memory 存储单元结构

表 7-5 图 7.28 中各芯片地址空间分配表

地 址 片 号	A ₁₇ A ₁₆	A ₁₅ A ₁₄ A ₁₃ …A ₁ A ₀	说 明
无用	00	000…00	最低地址(00000H)
	00	111…11	最高地址(0FFFFH)
无用	01	000…00	最低地址(10000H)
	01	111…11	最高地址(1FFFFH)
2	10	000…00	最低地址(20000H)
	10	111…11	最高地址(2FFFFH)
3	11	000…00	最低地址(30000H)
	11	111…11	最高地址(3FFFFFH)

7.5.3 字/位扩展

在构成一个实际的存储器时，往往需要同时进行位扩展和字扩展才能满足存储容量的需求。扩展时需要的芯片数量可以这样计算：要构成一个容量为 $M \times N$ 位的存储器，若使用 $l \times k$ 位的芯片($l < M, k < N$)，则构成这个存储器需要 $(M/l) \times (N/k)$ 个这样的存储器芯片。 M/l 和 N/k 一定要能整除。连接时可将这些芯片分成 M/l 个组，每组内有 N/k 个芯片， N/k 个芯片组内采用位扩展法， M/l 个组间采用字扩展法。字/位扩展法则是上述两种扩展法的组合。

微机中内存的构成就是字/位扩展的一个很好的例子。首先，存储器芯片生产厂制造出一个个单独的存储芯片，如 $32M \times 1$ 位， $64M \times 1$ 位， $128M \times 1$ 位等；然后，内存条生产厂将若干个存储器芯片利用位扩展的方法组装成内存模块(即内存条)，如用 8 片 $128M \times 1$ 的芯片组成 128MB 的内存条；最后，用户根据实际需要，购买若干个内存条插到主板上构成自己的内存系统，即是进行了字扩展。一般来讲，最终用户做的都是字扩展(即增加内存地址单元)的工作。

进行字/位扩展时，一般先进行位扩展，构成字长满足要求的内存条，然后再用若干个这样的内存条进行字扩展，使总存储容量满足要求。

【例 7.5】 用 2114($1K \times 4$)RAM 芯片构成 $4K \times 8$ 存储器，需要同时进行位扩展和字扩展才能满足存储容量的需求。由于 2114 是 $1K \times 4$ 的芯片，所以首先要进行位扩展。用 $(8/4)2$ 片 2114 组成 1KB 的内存模块，然后再用 4 组 $(4/1)$ 这样的模块进行字扩展便构成了 4KB 的存储器。所需的芯片数为 $(4/1) \times (8/4) = 8$ 片。因为 2114 有 1K 个存储单元，只需要 10 位地址信号线($A_9 \sim A_0$)对每组芯片进行片内寻址，同组芯片应被同时选中，故同组芯片的片选端并联在一起。要寻址 4KB 个内存单元至少需要 12 位地址信号线($2^{12} = 4K$)。而 2114 有 1K 个单元，只需要 10 位地址信号，余下的 2 位地址用 2-4 译码器对两位高位地址($A_{11} \sim A_{10}$)译码，产生 4 个片选信号线，分别与各组内的两个 2114 芯片的片选端相连，用于区分 4 个

1KB 的内存条，线路连接示意图如图 7.29 所示。

综上所述，存储器容量的扩展可以分为以下 3 步：

第 1 步选择合适的芯片个数 $(M/l) \times (N/k)$ ；

第 2 步根据要求将 N/k 个芯片“多片并连”进行位扩展，设计出满足字长要求的“内存条”；

第 3 步再对“内存条” M/l 组进行字扩展，构成符合要求的存储器。

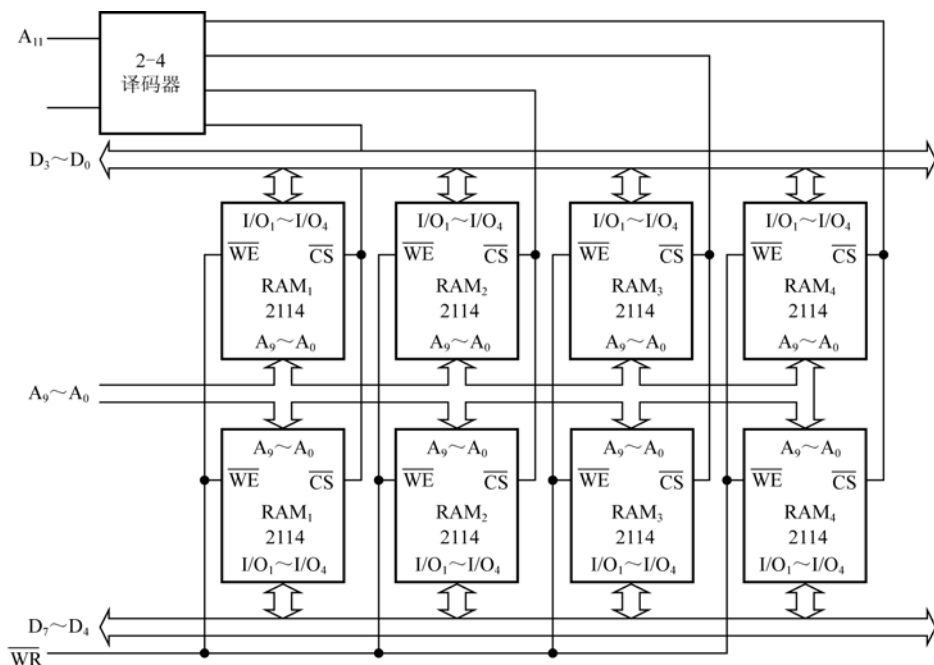


图 7.29 字/位扩展应用举例示意图

【例 7.6】设有若干片 $256\text{K} \times 8$ 位的 SRAM 芯片：

(1) 采用位扩展方法构成 $256\text{K} \times 32$ 位的存储器需要多少片 SRAM 芯片？进行片内寻址需要多少条地址线？

(2) 采用字扩展方法构成 $2048\text{K} \times 32$ 位的存储器需要多少组 $256\text{K} \times 32$ 位的存储器芯片？需要多少条地址线用于片间寻址？

(3) 画出该 $2048\text{K} \times 32$ 位的存储器与 CPU 连接的结构图，设 CPU 的接口信号有地址信号、数据信号、控制信号 $\overline{\text{MREQ}}$ 和 $\text{R}/\overline{\text{W}}$ 。

解：

(1) 采用位扩展法构成 $256\text{K} \times 32$ 位的存储器需要 $32/8=4$ 片 SRAM 芯片； $2^{18}=256\text{K}$ ，进行片内寻址需要 18 条地址线，用 CPU 的地址总线 ($A_{19} \sim A_2$) 进行片内寻址。

(2) 采用字扩展法构成 $2048\text{K} \times 32$ 位的存储器需要 $2048/256=8$ 组 $256\text{K} \times 32$ 位的存储器芯片；因为 $2^{21}=2048\text{K}$ ，所以， $2048\text{K} \times 32$ 位的存储器总共需要 21 条地址线，再减去 18 条片内寻址地址线，用 3 条 CPU 的地址总线 ($A_{22} \sim A_{20}$) 进行片间寻址。

(3) 用 $\overline{\text{MREQ}}$ 作为译码器芯片的输出许可信号, 译码器的输出作为存储器芯片的选择信号, $\text{R}/\overline{\text{W}}$ 作为读/写控制信号, CPU 访存的地址为 ($\text{A}_{22} \sim \text{A}_2$)。该 $2048\text{K} \times 32$ 位的存储器与 CPU 的连接示意图如图 7.30 所示。

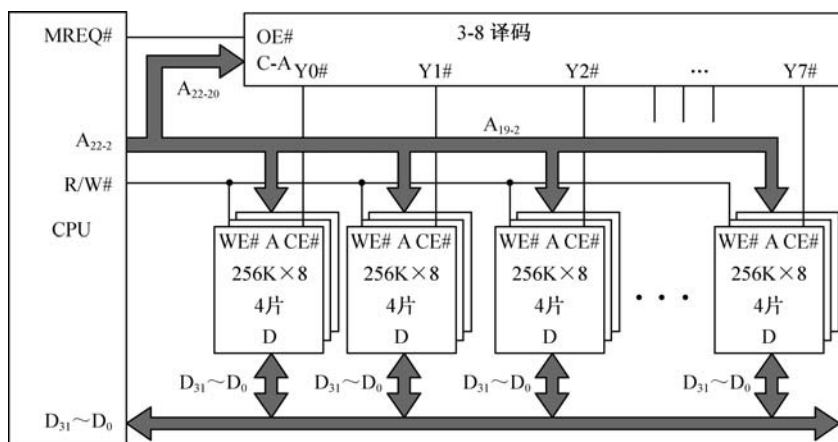


图 7.30 用 $256\text{K} \times 8$ 位的芯片构成容量为 $2048\text{K} \times 32$ 位的存储器

7.6 存储器与 CPU 的速度匹配

在实际的计算机系统中, 对存储器的容量和工作速度的要求越来越高, 因此需要提高存储器工作速度的技术, 并采用层次化的存储系统结构。提高存储容量可以采用上一节讲过的字/位扩展方法, 提高存储器工作速度的技术主要有芯片技术和结构技术。

反映存储器工作速度的主要技术指标有: 访问时间、访问周期和带宽。CPU 通过地址线、数据线、读/写控制线和访存请求信号 $\overline{\text{MERQ}}$ 与主存储器连接。从一次启动存储器操作到操作完成后可启动下一次操作的时间称为存储器周期, 它是连续两次启动访问存储器操作之间必须间隔的时间。从启动访问存储器的操作到操作完成的时间称为访问时间。比如, 从 CPU 提供地址和读操作信号到数据读出所经历的时间, 就是访存时间。存储器周期一般比访问时间要长, 因为一次访存完成之后存储器要经过一段恢复时间才能开始下一个访存操作。带宽是指存储器在连续访问时的数据吞吐率。带宽的单位通常是位每秒(b/s)或字节每秒(B/s)。

在计算机中, CPU 的工作速度基本上都大于主存储器的工作速度, 而 CPU 访问主存储器又是它的主要工作, 因此存储器的访问速度与 CPU 的工作速度的匹配对提高计算机的工作速度具有重要影响。

7.6.1 芯片技术

主存储器一般用动态存储器实现, 高速的静态存储器芯片因为价格高而不适于组成计算机的主存储器。动态存储器芯片又出现了一些新技术, 使得存储器的工作速度提高。下面就介绍几种高速动态存储器的工作情况。

1. 同步动态存储器 SDRAM(Synchronous DRAM)

SDRAM 是奔腾计算机系统目前普遍使用的内存形式。SDRAM 将 CPU 与 RAM 通过一个相同的时钟锁在一起,使 RAM 和 CPU 能够共享一个时钟周期,以相同的速度同步工作。SDRAM 采用的是新型的 64 位数据读/写形式,内存条的引脚为 168 根线,采用双列直插式的 DIMM 内存条,读/写速度最高达到了 5ns,是目前最快的内存芯片,同时也是奔腾 II 和奔腾 III 计算机系统首选的内存条。

在 SDRAM 芯片构成的存储器系统中,SDRAM 存储器在系统时钟的控制下进行数据的读出和写入。存储器把 CPU 给出的地址和数据存在一组存储了地址、数据和 SDRAM 的控制信号的锁存器中,直到指定的时钟周期后再响应。SDRAM 基于双存储体结构,内含两个交错的存储阵列,在 CPU 从一个存储体或阵列访问数据的同时,另一个已准备好读/写数据。通过两个存储阵列的紧密切换,读取效率得到成倍提高。理论上速度可与 CPU 频率同步,与 CPU 共享一个时钟周期。

2. 双数据传输率同步动态随机存储器 DDRDRAM(Double Data Rate DRAM)

DDR 也就是 SDRAM II,是 SDRAM 的更新换代产品,在同步动态读/写存储器 SDRAM 的基础上,采用延时锁定环(Delay-locked Loop)技术提供数据选通信号对数据进行精确定位,在时钟脉冲的上升沿和下降沿都可传输数据(而不是第一代 SDRAM 仅在时钟脉冲的下降沿传输数据,“DDR”即“双数据率”的意思),这样不需要提高时钟的频率就能加倍提高 SDRAM 的速度,并具有比 SDRAM 多一倍的传输速率和内存带宽。由于 DDR DRAM 需要新的高速时钟同步电路和符合美国电子工业协会和电子器件工程联合会(JEDEC)标准的存储器模块,所以主板和芯片组的成本较高,一般只能用于高档服务器和工作站上。

3. 接口动态随机存储器 DRDRAM(Direct Rambus DRAM)

从 1996 年开始,Rambus 公司就在 Intel 公司的支持下制定出新一代 RDRAM 标准,这就是 DRDRAM。DRDRAM 采用高性能的芯片接口取代现有的存储器接口。它与传统 DRAM 的区别在于引脚定义会随命令而变,同一组引脚线可以被定义成地址线,也可以被定义成控制线。其引脚数仅为正常 DRAM 的 1/3。当需要扩展芯片容量时,只需要改变命令,不需要增加芯片引脚。它采用异步的、面向块传输的协议传输地址信息和数据信息。这种芯片可以支持 400 MHz 外频,再利用上升沿和下降沿两次传输数据,可以使数据传输率达到 800 MHz。同时通过把单个内存芯片的数据输出通道从 8 位扩展成 16 位,这样在 100 MHz 时就可以使最大数据输出率达到 1.6 GB/s。

4. 带高速缓存动态随机存储器 CDRAM(Cached DRAM)

CDRAM 是日本三菱电气公司开发的专有技术,通过在 DRAM 芯片上集成一定数量的高速 SRAM 作为高速缓冲存储器 Cache 和同步控制接口,来提高存储器的性能。这种芯片使用单一的 +3V 电源,低压 TTL 输入/输出电平。目前三菱公司可以提供的 CDRAM 为 4 MB 和 16 MB,其片内 Cache 为 16 KB,与 128 位内部总线配合工作,可以实现 100 MHz 的数据访问。流水线式存取时间为 7 ns。

5. 虚拟通道存储器 VCM(Virtual Channel Memory)

VCM 由 NEC 公司开发, 是一种新兴的“缓冲式 DRAM”, 该技术在大容量 SDRAM 中采用。它集成了所谓的“通道缓冲”, 由高速寄存器进行配置和控制。在实现高速数据传输(即“带宽”增大)的同时, VCM 还维持着与传统 SDRAM 的高度兼容性, 所以通常也把 VCM 内存称为 VCM SDRAM。在设计上, 系统(主要是主板)不需要做大的改动, 便能提供对 VCM 的支持。VCM 可从内存前端进程的外部对所集成的这种“通道缓冲”执行读/写操作。对于内存单元与通道缓冲之间的数据传输, 以及内存单元的预充电和刷新等内部操作, VCM 要求它独立于前端进程进行, 即后台处理与前台处理可同时进行。由于专为这种“并行处理”创建了一个支撑架构, 因而 VCM 能保持一个非常高的平均数据传输速度, 同时不用对传统内存架构进行“大手笔”的更改。采用 VCM 后, 系统设计人员不必再受限于目前的内存工作方式, 因为内存通道的运行与管理, 都可移交给主板芯片组去解决。

7.6.2 结构技术

选定了存储器芯片之后, 为了进一步提高存储器的工作速度, 所采用的方法是从结构上提高存储器的带宽。这种结构上的措施主要是增加存储器的数据宽度和采用多体交叉存储技术。

1. 增加存储器的数据宽度

增加存储器的数据宽度即位扩展。在位扩展中, 将存储器的位数扩展到大于数据字的宽度, 包括增加数据总线的宽度和存储器的宽度, 这样可以增加同时访问数据量, 以提高存储器操作的并行性, 从而提高数据访问的速率, 如图 7.31 所示, 存储体 0 有 16 位宽, 即一个字宽, 这种方法适合 CPU 访问地址连续的存储器中的数据流。因为在这种方法下连续访问的数据字依次存储在存储体 0、1、2、3 中, 同时从存储体 0、1、2、3 中读出数据字存放在数据寄存器中。数据同时读出, 增加了数据流量。但是对于非连续数据访问的情况, 则不能提高存储器的数据流量。

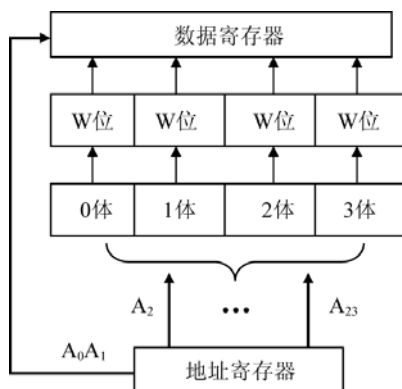


图 7.31 增加存储器字宽

例如, CPU 要读取字地址为 2000H 中的数据时, 在存储器宽度为字的情况下进行读操作时, 将字地址为 2000H、2001H、2002H 和 2003H 的数据同时读出到输出数据寄存器中,

如果 CPU 下次要读取的地址为 2001H 的数据时就不必启动存储器的操作, 而是直接在数据寄存器中得到。在计算机中经常存在这种连续访问的操作, 如对指令的访问和对数组的访问等。在按顺序访问的情况下, 4 字宽的存储器可以提高存储器带宽到原来的 4 倍。但是如果 CPU 不是按顺序访问的情况下, 存储器的效果会大打折扣。如果 CPU 要读取字地址为 2000H、2002H、2004H、2006H、2008H、…中的数据时, 4 字宽的存储器读出的数据中只有两个字的数据是有效的, 带宽提高到原来的 2 倍。如果访问的地址跨步更大, 如 CPU 要读取字地址为 2000H、2004H、2008H、…时, 字宽存储器就不能提高访存的带宽。

2. 多体交叉存储技术

计算机中大容量的主存, 可由多个存储体组成, 每个体都具有自己的读/写线路、地址寄存器和数据寄存器, 称为“存储模块”。多体交叉存储器由多个相互独立、容量相同的存储模块构成。如图 7.32 所示。每个存储模块都以同样的方式与 CPU 传递信息, 各存储体的读/写过程可以实现重叠与交叉存取。

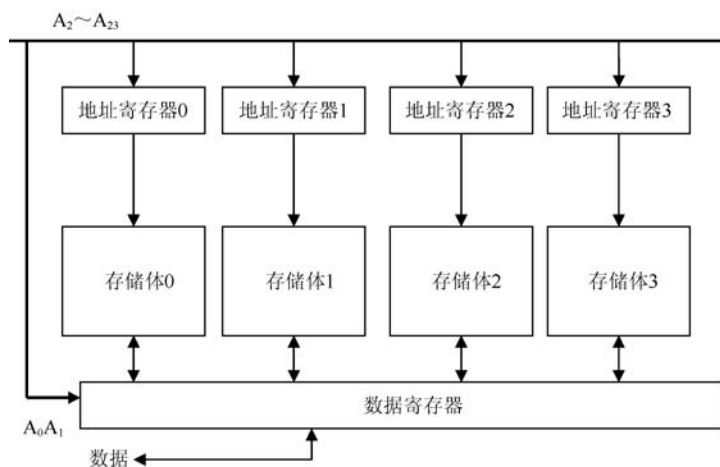


图 7.32 多体交叉存储器结构框图

1) 多体交叉存储的基本原理

CPU 的速度比存储器快。假如能同时从存储器取出 n 条指令, 这必然会提高机器的运行速度。多体交叉存储方式就是基于这种思想提出来的。

图 7.32 示出四体交叉存储器结构框图。主存被分成 4 个相互独立、容量相同的存储体 0, 1, 2, 3。每个存储体为 1 个字宽, 它们都有自己的读/写线路、地址寄存器和数据寄存器, 以相同的方式与 CPU 传递信息。通常采用的编址方法是四体交叉编址方式, 即将单元地址依次排列在各个存储体整体中。如 0 体的地址为 0, 4, 8, …, $4i+0$; 1 体的地址为 1, 5, 9, …, $4i+1$; 2 体的地址为 2, 6, 10, …, $4i+2$; 3 体的地址为 3, 7, 11, …, $4i+3$ 。采用这样的编址方法, 使这 4 个存储体对应的二进制地址最后两位的编码分布将是 00, 01, 10, 11。因而使用地址码的低位字段经过译码选择不同的存储体, 而高位字段指向相应的存储体内部的存储字。这样, 连续地址分布在相邻的不同存储体内, 而同一个存储体内的地址都不是连续的。在理想情况下, 如果程序段和数据块是连续地的主存中存放或读取, 那么将大大地提高主存的有效访问速度。

CPU 访问 4 个存储体, 一般有两种方式: 一种是在一个存取周期内, 同时访问 4 个存

储体,由存储器控制部件控制它们分时使用总线进行信息传递;另一种是 CPU 在一个存储周期内分时访问每个存储体,即每经过四分之一存取周期就访问一个存储体。这样,对每个存储体来说,它可以在一个存取周期内连续访问 4 个存储体,各存储体的读/写过程将重叠进行。主存由 R 个存储体组成、每隔 $1/R$ 访问周期启动一个存储体,这样的存储器的带宽可增加到原来的 R 倍。所以多体交叉存储系统是一种并行存储器结构。

2) 地址交叉方法

设 $X_0, X_1, \dots, X_{k-1}$ 为一台中央处理器依次所需要的 k 个字,当把它们分配给主存储器中 k 个前后相连的物理地址时,可用以下交叉规则在存储模块之间分配这些地址:

i 代表中央处理器所需要的 i 个字; m 代表主存中存储体个数;如果 $j=i \bmod m$, 那么把地址 A_i 分配给存储模块 M_j 。

于是 $A_0, A_m, A_{2m}, \dots$ 分配给存储体 M_0 ; $A_1, A_{m+1}, A_{2m+1}, \dots$ 分配给 M_1 ; 依此类推。存储模块之间分配地址的这种技术通常称为交叉。在 m 个模块之间的地址交叉称为 m 路交叉。多体交叉的存储器可以采用高位多体交叉和低位多体交叉两种技术。在高位多体交叉的存储器中,访存地址的高位部分用于对存储体进行寻址,低位部分用于对存储体内的存储单元进行寻址,这样,连续地址分布在相同的模块内,而同一模块内的地址都是连续的。当 CPU 对连续的地址单元进行访问时,则只有一个存储体处于忙的状态。

在低位多体交叉的存储器中,访存地址的低位部分用于对存储体进行寻址,使用地址码的低位字段经过译码选择不同的存储模块,而高位字段指向相应的模块内部的存储字。这样,连续地址公布在相邻的不同模块内,而同一模块内的地址都是不连续的。当 CPU 对连续的地址单元进行访问时, CPU 在一个存储周期内分时访问每个存储体,使得在一个存储周期内每个存储体都处于忙的状态,从而可以提高顺序访问速度。低位多体交叉编址方式的编址情况如表 7-6 所示。在理想情况下,如果程序段和数据块在主存中连续地存放和读取。那么,这种编址方式将大大地提高主存的有效访问速度。但当遇到程序转移或随机访问少量数据,访问地址就不一定均匀地分布在多个存储模块之间,这样就会产生存储器冲突而降低了使用率。

表 7-6 低位多体交叉编址方式编址情况表

体	地址编码序列	地址后两位
0	0, 4, 8, ..., $4i+0$, ...	00
1	1, 5, 9, ..., $4i+1$, ...	01
2	2, 6, 10, ..., $4i+2$, ...	10
3	3, 7, 11, ..., $4i+3$, ...	11

【例 7.7】 在一个具有 4 个存储体的低位多体交叉存储器中,如图 7.32 所示,如果处理器要访问的字地址为以下十进制数值时,试问该存储器比单体存储器的平均访问速率提高多少?(忽略初启时的延时)

(1) 1, 2, 3, 4, ..., 100

(2) 2, 4, 6, 8, ..., 200

(3) 3, 6, 9, 12, ..., 300

解: 在低位多体交叉的存储器中,访存地址的低位部分用于对存储体进行寻址,使用地址

码的低位字段经过译码选择不同的存储模块，而高位字段指向相应的模块内部的存储字。这样，连续地址分布在相邻的不同模块内，而同一模块内的地址都是不连续的。

(1) 因为一个具有 4 个存储体的低位多体交叉存储器中，访存地址的低两位用于对不同的存储体寻址，当访问地址为连续的 100 个地址，即十进制数值 1, 2, 3, 4, ... 和 100 时，对应的二进制数为 000001B, 000010B, 000011B, 0000100B, ... 和 1100100B, 最低两位二进制数为 01, 10, 11, 00, ... 和 00, 对应访问的存储体是：体 1, 体 2, 体 3, 体 0, ... 和体 0, 各个存储体访问可以交叉进行，即每隔 1/4 周期启动一次访问操作，这样访问速率可达到单体存储器的 4 倍。

(2) 当访问地址为不连续的 100 个地址，即十进制数值 2, 4, 6, 8, ... 和 200 时，它们对应的二进制数为 000010B, 000100B, 000110B, 00001000B, ... 和 11001000B, 最低两位二进制数为 10, 00, 10, ... 和 00, 对应访问的存储体是：体 2, 体 0, 体 2, 体 0, ... 和体 0。当存储器以每隔 1/4 周期进行访问时，由于存在存储体冲突，即地址 2 和地址 6 处于同一存储体，地址 4 和地址 8 处于同一存储体，需要等待存储体上一个访问结束时，才能启动下一个访问操作。两个存储体访问可以交叉进行，即每隔 1/2 周期启动一次访问操作，这样访问速率可达到单体存储器的 2 倍。

(3) 当访问地址为不连续的 100 个地址，即十进制数值 3, 6, 9, 12, ... 和 300, 它们对应的二进制数为 000011B, 000110B, 00001001B, 00001100B, ... 和 10010110B, 最低两位二进制数为 11, 10, 01, 00, ... 和 10, 对应访问的存储体是：体 3, 体 2, 体 1, 体 0, ... 体 3, 体 2, 体 1, 体 0, 4 个存储体访问可以交叉进行，即每隔 1/4 周期启动一次访问操作，因此访问速率可达到单体存储器的 4 倍。

7.7 高速缓冲存储器

在现代高性能的计算机系统中，对存储器既要求速度快，又要求容量大，同时价格又要合理，按照现在所能达到的技术水平，仅仅用一种技术组成单一的存储器是不可能同时满足上述要求的，只有采用层次结构，把几种存储技术结合起来，才能解决存储器高速度、大容量和合理成本三者的矛盾。其中一种重要的提高存储器带宽的措施是在主存储器与 CPU 之间增加一个高速缓冲器(Cache)来存储使用频繁的指令和数据，以提高访存操作的平均速度。

存储器设计目标之一是以较少的成本使存储体系与 CPU 的速度匹配，同时还要求存储器具有尽可能大的容量。因此，速度、容量、价格是存储器设计应考虑的主要因素。尽管计算机程序往往需要巨大的快速存储空间，但程序对存储空间的访问并不是均匀的。通过对大量的典型程序的运行情况的分析，结果表明，在一个较短的时间间隔内，地址往往集中在存储器逻辑地址空间的很小范围内。程序地址的分布本来就是连续的，再加上循环程序段和子程序都要重复执行多次，因此，对程序地址的访问就自然地具有相对集中的倾向。数据分布的这种集中倾向不如指令明显，但对数组的存储和访问以及工作单元的选择都可以使存储器地址相对集中。这种对局部范围的存储器地址频繁访问，而对此范围以外的地址访问甚少的现象就称为程序访问的局部性。

根据局部性原理，可以在主存和 CPU 之间设置一个高速的容量相对较小的存储器，如果当前正在执行的程序和数据存放在这个存储器中，当程序运行时，不必从主存储器取指

令和取数据，而访问这个高速存储器即可，从而提高了程序运行速度。Cache 就是一种存储空间小而存取速度却很高的一种存储器。

Cache 存储器介于 CPU 和主存之间，它的工作速度远远大于主存，全部功能由硬件实现，并且对程序员是透明的，即程序员不能对 Cache 进行操作和控制。

7.7.1 Cache 的基本结构和工作原理

1. Cache 系统的基本结构

Cache 系统主要由 3 部分组成：Cache、地址映像与变换机构及 Cache 替换策略和更新策略。把 Cache 和主存都分成相同大小的块，每一块由若干个字或字节组成。在 Cache 中，每一块外加有一个 Cache 标记，指明它是主存的哪一块的副本，所以该标记的内容相当于主存块的编号。每当对一主存地址进行数据访问时，怎样知道要访问的数据已经存在于 Cache 中？如果要访问的数据已经在 Cache 中，怎样确定这个数据在 Cache 中的位置？这两个问题是相关的，解决的方法是根据主存地址来构成 Cache 地址，也就是必须通过地址映像变换机构将主存地址变换成 Cache 地址去访问 Cache。若要访问的数据所在块不在 Cache 中(不命中)，则产生 Cache 失效，需要从主存中把包含该字的一块信息调入 Cache，同时将被访问的字送往 CPU。如 Cache 已装满怎么办？就需要按所选择的替换算法决定将 Cache 的哪一块已调入访问的块数据移去，并修改地址映像表中有关的地址映像关系和 Cache 各块使用状态标志等信息。写 Cache 时是否写主存？块的更新策略决定在写操作时，何时将数据写入主存。

2. Cache 系统的工作原理

Cache 的基本结构如图 7.33 所示，主存有 2^n 个单元，地址码为 n 位，将主存分块(block)，每块有 B 个字节，则共分成 $M=2^n/B$ 块。Cache 也由同样大小的块组成，由于 Cache 容量小，所以块的数目比主存的块数少得多，主存中只有一小部分块的内容可存放在 Cache 中。

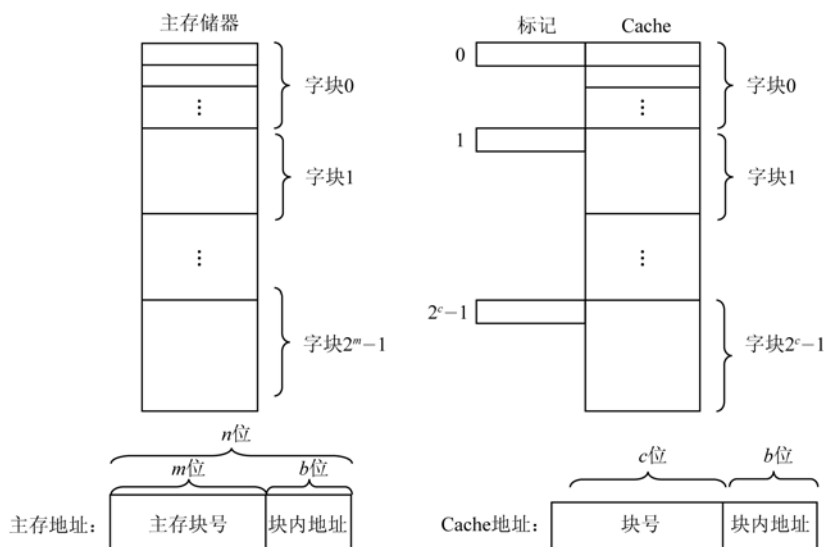


图 7.33 Cache 基本结构

在 Cache 中，每一块外加有一个标记，指明它是主存的哪一块的副本，所以该标记的内容相当于主存中块的编号，设主存地址为 n 位，且 $n=m+b$ ，则可得出：主存的块数 $M=2^m$ ，块内字节数 $B=2^b$ 。Cache 地址码为 $(c+b)$ 位。Cache 的块数为 2^c ，块内字节数与主存相同。

1) Cache 的读工作原理

如图 7.34 所示，当 CPU 发出读请求时，将主存地址 m 位(或 m 位中的一部分)与 Cache 某块的标记相比较，根据其比较结果是否相等而区分出两种情况：

(1) 当比较结果相等时，说明 CPU 要访问的数据或代码存在于 Cache 中，就可以直接从 Cache 中访问数据或代码，而不必再到主存中去访问了。访问主存的数据或代码存在于 Cache 中的情形称为访问 Cache 命中(hit)。在 CPU 与 Cache 之间，通常一次传送一个字。

(2) 当比较结果不相等时，说明需要的数据尚未调入 Cache，那么就要把该数据所在的整个字块从主存一次调进 Cache 中。这种情况称为访问 Cache 不命中或失效(miss)。

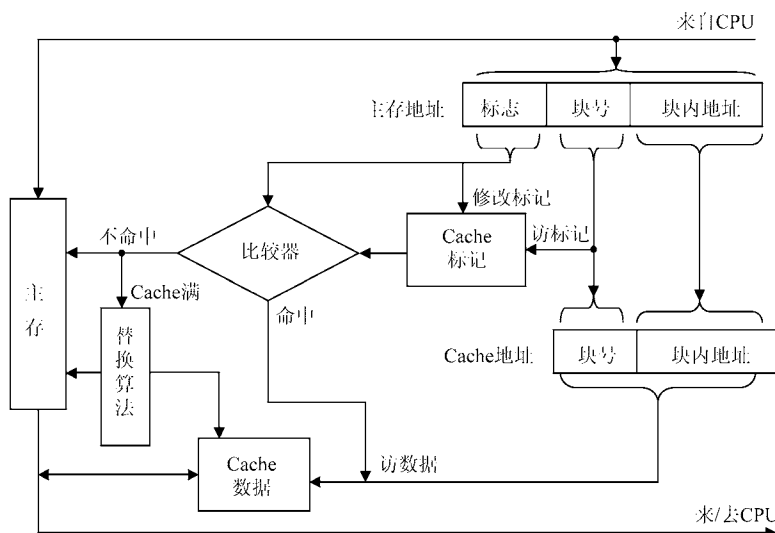


图 7.34 Cache 系统的基本结构

2) Cache 的容量和块大小

块的大小称为“块长”。把 Cache 和主存都分成相同大小的块，每一块由若干个字或字节组成。块长在一般情况下，取一个主存周期所能调出的信息长度。Cache 的容量一般比主存低得多，通常在 16KB~512KB 之间，主存传输的数据块的容量一般在 4~128 字节之间。命中时间一般为 1 个时钟周期，时效时间取决于主存的访问时间及传输时间，一般为 8~32 个时钟周期，有效率在 1%~20% 之间。一般情况下，可以使 Cache 与内存的空间比为 1:128，即 256KB 的 Cache 可映射 32MB 内存；512KB Cache 可映射 64MB 内存。在这种情况下，命中率都在 90% 以上。即 CPU 在运行程序的过程中，有 90% 的指令和数据可以在 Cache 中取得，只有 10% 需要访问主存。对没有命中的数据，CPU 只好直接从内存获取，获取的同时，也把它复制到 Cache 中，以备下次访问。为了提高 Cache 数据的调入调出速度，要求主存的带宽与其匹配。方法是采用 7.6 节讲的多体交叉的主存模块或加宽主存的数据位数和数据总线的宽度。

Cache 的容量和块的大小是影响 Cache 效率的重要因素。通常用“命中率”来测量 Cache

的效率。命中率指 CPU 所要访问的信息在 Cache 中的比率，而将所要访问的信息不在 Cache 中的比率称为失效率或未命中率。

一般来说，Cache 的存储容量比主存的容量小得多，但不能太小，太小会使命中率太低；也没有必要过大，过大不仅会增加成本，而且当容量超过一定数量后，命中率随容量的增加不会有明显地增长。但随着芯片价格的下降，Cache 的容量还是不断增大，已由几十千字节发展到几百千字节，甚至达到几兆字节。

3) Cache 的替换策略

CPU 从主存储器中读出数据的方式有如下两种：

第一种方式是贯穿读出式(Look Through)，该方式的原理示意图如图 7.35 所示。在这种方式下，Cache 位于 CPU 与主存之间，CPU 对主存的所有数据请求都首先送到 Cache，由 Cache 自行在自身查找。如果命中，则切断 CPU 对主存的请求，并将数据送出；如果不命中，则将数据请求传给主存。该方法的优点是降低了 CPU 对主存的请求次数，缺点是延迟了 CPU 对主存的访问时间。

第二种方式是旁路读出式(Look Aside)，这种方式的原理示意图如图 7.36 所示。

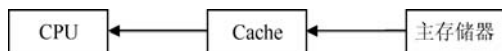


图 7.35 贯穿读出式原理示意图

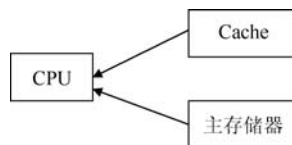


图 7.36 旁路读出式原理示意图

在这种方式中，CPU 发出数据请求时，并不是单通道地穿过 Cache，而是向 Cache 和主存同时发出请求。由于 Cache 速度更快，如果命中，则 Cache 在将数据回送给 CPU 的同时，还来得及中断 CPU 对主存的请求；若不命中，则 Cache 不做任何动作，由 CPU 直接访问主存。它的优点是没有时间延迟，缺点是每次 CPU 都要访问主存，这样，就占用了部分总线时间。

在带有 Cache 的存储体系中，如果读 Cache 不命中，而 Cache 又放满了怎么办？从主存读出新的字块调入 Cache 存储器时，如果遇到 Cache 存储器中相应的位置已被其他字块占有，那么就必须去掉一个旧的字块，让给这个新的字块。这种替换应该遵循一定的规则，最好能使被替换的字块是下一段时间内估计最少使用的。这些规则称为替换策略或替换算法，由替换部件加以实现。

4) Cache 的写策略

当被修改的单元在 Cache 存储器时，Cache 存储器中保存的字块是主存中相应字块的副本。如果程序执行过程中要对该字块的某个单元进行写操作，就会遇到如何保持 Cache 与主存的一致性问题。Cache 的一致性指的是 Cache 中的信息与主存储器中的信息一致，即 Cache 中每一地址上的数据与主存储器中相应存储单元中数据相一致。通常有三种写入方式：

第一种方式是暂时只向 Cache 存储器写入，并用标志加以注明，直到经过修改的字块从 Cache 中被替换出来时才一次写回主存，这种方式称为标志交换(flag-swap)方式，又称按写分配法(write-allocate)，按写分配法原理示意图如图 7.37 所示。

第二种方式是写贯穿式(Write Through)：任一从 CPU 发出的写信号送到 Cache 的同时，

也送入主存, 以保证主存的数据能同步地更新。它的优点是操作简单, 但由于主存的慢速, 降低了系统的写速度并占用了部分总线时间。写贯穿式原理示意图如图 7.38 所示。

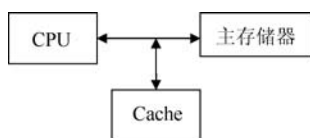


图 7.37 按写分配法原理示意图

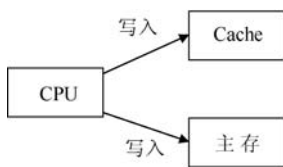


图 7.38 写贯穿式原理示意图

第三种方式是回写式(Write Back): 为了克服贯穿式中每次数据写入都要访问主存、从而导致系统写速度降低并占用总线时间的弊病, 尽量减少对主存的访问次数, 又有了回写式。回写式原理示意图如图 7.39 所示。它的工作原理是这样的: 数据一般只写到 Cache, 而不写入主存, 从而使写入的速度加快。但这样有可能出现 Cache 中的数据得到更新而对应主存中的数据却没有变(即数据不同步)的情况。此时可在 Cache 中设一个标志地址及数据陈旧的信息, 只有当 Cache 中的数据被再次更改时, 才将原更新的数据写入主存相应的单元中, 然后再接受再次更新的数据。这样保证了 Cache 和主存中的数据不致产生冲突。

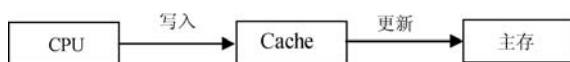


图 7.39 回写式原理示意图

当被修改的单元根本就不在 Cache 存储器时, 写操作直接对主存进行, 而不写入 Cache 存储器。

5) Cache 标记和平均存取时间

为了说明标记是否有效, 每个标记至少还应设置一位有效位, 当机器刚加电启动时, 主机的 reset 信号或执行程序将所有标记的有效位置“0”, 使标记无效。在程序执行过程中, 当 Cache 不命中时, 逐步将指令块或数据块从主存调入 Cache 中的某一块, 并将这一块标记中的有效位置“1”, 当再一次用到这一块中的指令或数据时, 肯定命中, 可直接从 Cache 中取指令或取数据。

从这里也可看到, 刚加电后所有标记位都为“0”, 因此开始执行程序时, 命中率较低了。另外 Cache 的命中率还与程序本身有关, 不同的程序, 其命中率可能不同。

在有 Cache 的系统中, Cache 的命中率与 Cache 的大小、替换算法、程序特性等因素有关。增加 Cache 后, CPU 对主存的平均存取时间可按下式粗略地计算:

设 Cache 的存取时间为 t_c , 命中率为 h , 主存的存取时间为 t_m , 则平均存取时间

$$t_a = t_c \times h + (t_c + t_m) \times (1 - h) \quad (7.1)$$

【例 7.8】某微机存储器系统由一级 Cache 和主存组成。已知主存的存取时间为 80ns, Cache 的存取时间为 6ns, Cache 的命中率为 85%, 试求: 该存储系统的平均存取时间。

解: 根据式 7.1, 得:

$$t_a = 6\text{ns} \times 85\% + 80\text{ns} \times (1 - 85\%) = 5.1 + 12 = 17.1\text{ns}$$

可以看出, 有了 Cache 以后, CPU 访问主存的速度大大提高了。但要注意的是, 增加 Cache 只是加快了 CPU 访问存储器系统的速度, 而 CPU 访问存储器系统仅是计算机全部操作的一部分, 所以增加 Cache 对系统整体速度只能提高 10%~20%左右。另外, 如果访问

Cache 没有命中的话, CPU 还要访问主存, 这时反而延长了存取时间。所以, 根据式 7.1 计算出来的平均存取速度仅是一个粗略值。

7.7.2 Cache 的地址映像与变换

设计 Cache 时首先面临的问题是, 怎样知道 Cache 是否命中? 即怎样知道要访问的数据已经在 Cache 中; 其次是, 如果要访问的数据在 Cache 中, 怎样确定 Cache 中的位置? 这两个问题是相互关联的, 解决方法是根据主存的地址来构成 Cache 的地址, 即在主存地址和 Cache 地址之间建立一种逻辑关系, 即地址映像。在信息按照这种映像关系装入 Cache 后, 执行程序时, 应将主存地址变换成 Cache 地址, 这个变换过程叫做地址变换。地址的映像和变换是密切相关的。

Cache 的存在对程序员是透明的。在 CPU 每次访问存储器时, 系统自动将地址转换成 Cache 中的地址, 如果访问的数据不在 Cache 中, 则需要将其从主存调入 Cache 中。Cache 的地址变换和数据块的替换算法都是硬件实现的。Cache 的地址空间小, 其地址的位数也较少, 主存储器的空间较大, 其地址位数较多。Cache 中的一个存储块与主存中若干个存储块相互对应, 即若干个主存地址块映射到同一个 Cache 地址块。根据这种地址对应方法, 下面介绍几种基本的地址映像方式: 直接映像、全相联映像和组相联映像等。

为了说明这几种映像方式, 假设主存储器空间被分为 $Mm(0), Mm(1), \dots, Mm(i), \dots, Mm(2^m-1)$ 共 2^m 个块, 每个块内有 2^b 个字; Cache 存储空间被分为 $Mc(0), Mc(1), \dots, Mc(j), \dots, Mc(2^c-1)$ 共 2^c 个同样大小的块。

1. 直接映像

直接映像方式如图 7.40 所示。一个主存块只能映像到 Cache 中的唯一一个指定块。地址映像方式称为直接映像(direct mapping)。在直接映像方式下, 主存中存储块的数据只可调入 Cache 中的一个位置。因为主存容量总比 Cache 容量大, 把主存按照 Cache 同样大小划分成 n 个区, 因此会有多个主存地址映像到同一个 Cache 地址。如果主存中两个存储块的数据都要调入 Cache 中的同一个位置, 则将发生冲突。

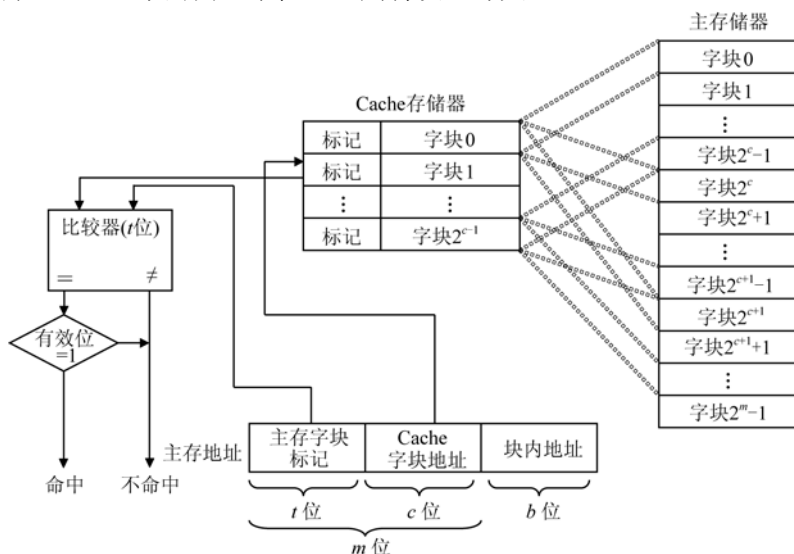


图 7.40 直接地址映像示意图

在直接映像方式中，主存和 Cache 中字块的对应关系采用直接映像函数：

$$j = i \bmod 2^c$$

式中， j 是 Cache 的字块号， i 是主存的字块号。

在这种映像方式中，主存的第 0 块，第 2^c 块，第 2^{c+1} 块，…只能映像到 Cache 的第 0 块，而主存的第 1 块，第 2^c+1 块，第 $2^{c+1}+1$ 块，…只能映像到 Cache 的第 1 块；依此类推。主存和 Cache 中的字块实现地址转换的过程如图 7.40 所示。将主存地址分成 3 段：主存字块标记 $(m-c)=t$ 位、字块地址 c 位、块内地址 b 位。主存字块标记 t 位作为标志存放在地址映像表中，用于判断所需字块是否已在 Cache 存储器中，即命中与否。Cache 地址分成两段：字块地址 c 位和块内地址 b 位。可见主存的字块地址、块内地址与 Cache 的字块地址、块内地址相同。当 CPU 送来主存地址和读命令后，只需根据主存地址中间 c 位字段找到 Cache 存储器字块，然后，看 Cache 标记是否与主存地址高 t 位相等：

如果相等且有效位为“1”，Cache 命中，则可根据主存 b 位块内地址，从 Cache 中取得所需指令或数据；

如果相等且有效位为“0”，表示 Cache 中的这一块已经作废；如果不相等且有效位为“0”，表示 Cache 中的这一块是空的；如果不相等且有效位为“1”，表示 Cache 中的这一块原来是有用的；这 3 种情况都要从主存读入新的字块放到 Cache 中，并将 CPU 所需数据送往 CPU，同时修改 Cache 标记。假如原来的有效位为“0”，要将有效位置成“1”。

直接映像是一种最简单的地址映像方式，它的地址变换速度快，实现简单，而且不涉及其他两种映像方式替换策略问题。缺点是不够灵活，即主存的 $2t$ 个字块只能对应唯一的 Cache 存储器字块，因此，即使 Cache 存储器别的地址空着也不能占用。这使得 Cache 存储空间得不到充分利用，并降低了命中率。

【例 7.9】 设有一个 Cache 容量为 4K 字，每个块为 32 字，主存的容量为 256K 字，试问：

- (1) 该 Cache 分为多少块？主存可分为多少块？
- (2) 在直接映像方式下，主存的第 i 块映像到 Cache 中的哪一块？
- (3) 进行直接地址映像时，Cache 地址有多少位？Cache 的地址分为哪几段？各段分别为几位？
- (4) 进行直接地址映像时，主存地址有多少位？分为哪几段？各段分别为几位？

解：(1) Cache 中有 $4K/32=1024 \times 4/32=128$ 块；主存可分为 $256K/32=8192$ 块。

(2) $j = i \bmod 128$

其中， j 是 Cache 的字块号， i 是主存的字块号。

(3) 因为 Cache 的容量为 4K 字 $=2^{12}$ 字，所以 Cache 的字地址为 12 位。分成两段：块地址和块内地址。因为每个块为 32 字 $=2^5$ 字，所以块内地址是 5 位字地址；Cache 中有 128 块 $=2^7$ 块，块地址为 7 位。

(4) 因为主存的容量为 256K 字 $=2^{18}$ 字，所以主存的地址为 18 位。分成 3 段：主存字块标记、块地址和块内地址。主存字块标记的长度为主存地址长度减去 Cache 地址长度之差，即 $18-12=6$ 位。因为主存的块地址和块内地址与 Cache 的相同，所以块内地址是 5 位，块地址为 7 位。

【例 7.10】 设一个直接相联映像的 Cache 中有 8 个块，访问主存进行读的块地址顺序为十进制数 22、26、22、26、16、4、16、18，求每次访问后 Cache 中的内容。

解：因为主存的访问的是块地址，最大的块地址号为 26，所以可以用 5 位二进制数表示主存的块地址。Cache 中有 8 个块，需要 3 位块地址，因此以主存的低 3 位地址作为 Cache 的块号，高 2 位地址作为主存字块标记位识别 Cache 命中情况。访问主存进行读的块地址顺序为十进制数 22、26、22、26、16、4、16、18 时，所对应的二进制块地址为：10110、11010、10110、11010、10000、00100、10000、10010，对应的主存字块标记为：10、11、10、11、10、00、10、10，用来作为标志位识别 Cache 命中情况。以主存的低 3 位地址作为 Cache 的块地址，对应的 Cache 块地址为：110、010、110、010、000、100、000、010。每次访存后，Cache 中的块调入、命中、替换情况如表 7-7 所示。在第 8 次读主存块地址 18 时，他的字块标记为 10，调入 Cache 中的块地址为 010，而在 Cache 的该地址中已经装入地址为 26 的块，这样就发生了块冲突，块号 18 替换了块号 26。

表 7-7 直接映像的块分配情况

访问顺序	1	2	3	4	5	6	7	8
主存块地址	22	26	22	26	16	4	16	18
Cache 000 块					16	16	16	16
Cache 001 块								
Cache 010 块		26	26	26	26	26	26	18
Cache 011 块								
Cache 100 块						4	4	4
Cache 101 块								
Cache 110 块	22	22	22	22	22	22	22	22
Cache 111 块								
操作状态	调入	调入	命中	命中	调入	调入	命中	替换

2. 全相联映像

每个主存块可以映像到任何 Cache 块的地址映像方式称为全相联映像方式(fully associative mapping)，全相联映像是最灵活但成本最高的一种方式。它允许主存中的每一个字块映像到 Cache 存储器的任何一个字块位置上，也允许从已被占满的 Cache 存储器中替换出任何一个旧字块。如果 Cache 中能容纳程序所需的指令和数据，则可达到很高的命中率。这是一个理想的方案。但实际上由于它的成本太高而没有采用。

全相联地址映像方式如图 7.41 所示，主存地址分为两段：主存字段标记 $m=t+c$ 位、块内地址 b 位；Cache 地址也分为两段：块地址 c 位、块内地址 b 位。主存块内地址与 Cache 地址块内地址相同，Cache 的标记位数从 t 位增加到 $t+c$ 位(与直接映像相比)，使 Cache 标记容量加大，主要问题是当 CPU 送来主存地址和读命令后，需要根据主存字段标记和 Cache 的全部标记进行“比较”，才能判断出所访问主存地址的内容是否已在 Cache 中。如果比较结果相等，表示 Cache 命中，取出 Cache 的块地址以对 Cache 进行访问；比较结果不相等，表示 Cache 不命中，再对主存进行访问并在主存中的块调入 Cache 中的同时，将主存的标记和 Cache 的块号写入 Cache 标记中，以改变地址映像关系。在 Cache 块全部装满后，新的数据块调入时，还需要考虑将 Cache 中的哪一个数据块替换掉，才会出现块冲突。

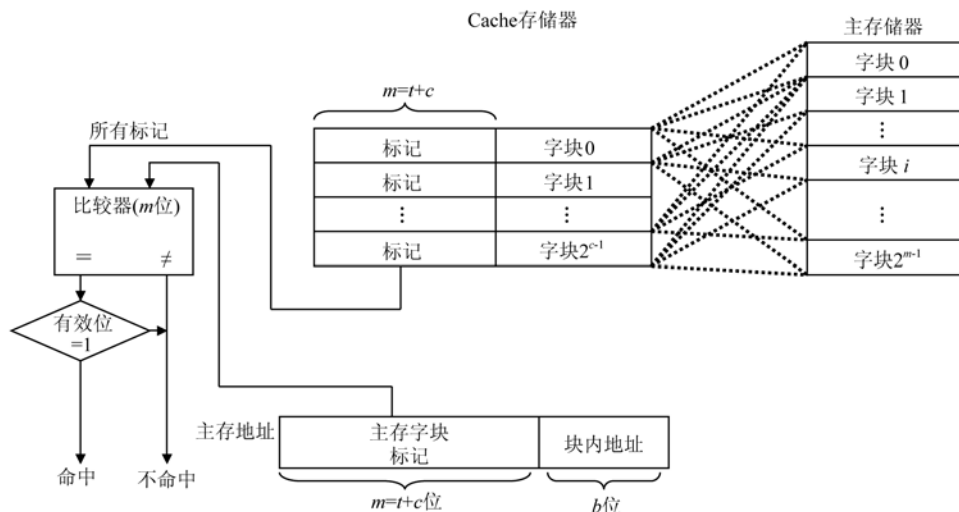


图 7.41 全相联地址映像方式

全相联映像方式的优点是可以灵活地进行块的分配，块的冲突率低，Cache 的利用率高。由于 Cache 速度要求高，所以全部“比较”操作都要用硬件实现，所需逻辑电路比较多，实现起来比较麻烦，以致无法用于 Cache 中。实际的 Cache 多是采取各种措施来减少所需比较的地址数目。

【例 7.11】 设一个全相联映像的 Cache 中有 8 个块，访问主存进行读的块地址顺序为十进制数 22、26、22、26、16、4、6、18、14、17、25，求每次访问后 Cache 中的内容。

解：在全相联映像方式的 Cache 中，从主存中调入的块可放在 Cache 的任一块中。当访问主存进行读的块地址顺序为十进制数 22、26、22、26、16、4、6、18、14、17、25 时，每次访问后，Cache 中的块调入、命中、替换情况如表 7-8 所示。当访问主存进行读的块地址为 25 时，Cache 中的块已经装满，这时就要用替换算法换掉 Cache 中的某一块，在此用先进先出替换算法替换掉 Cache 的 000 块中的块 22。

表 7-8 全相联映像的块分配情况

访问顺序	1	2	3	4	5	6	7	8	9	10	11
主存块地址	22	26	22	26	16	4	6	18	14	17	25
000 块	22	22	22	22	22	22	22	22	22	22	25
001 块		26	26	26	26	26	26	26	26	26	26
010 块					16	16	16	16	16	16	16
011 块						4	4	4	4	4	4
100 块							6	6	6	6	6
101 块								18	18	18	18
110 块									14	14	14
111 块										17	17
操作状态	调入	调入	命中	命中	调入	调入	调入	调入	调入	调入	替换

3. 组相联映像

组相联映像方式是直接映像和全相联映像方式的一种折衷方案。

组相联映像地址格式如图 7.42 所示。组相联映像把主存地址划分成 4 段：高字段是区号 t 位，然后是组号 c' 位，第三段是组中的块地址 r 位；最后一段为块内地址 b 位，高字段 t 位和组中的块地址 r 位形成主存字块标记字段；Cache 地址分为三段：第一段是组号 c' 位，第二段是组中的块地址 r 位，最后一段为块内地址 b 位。组相联映像方式如图 7.43 所示。组相联映像把 Cache 字块分为 $2^{c'}$ 组，每组包含 2^r 个字块，于是有 Cache 的地址位数 $c = c' + r + b$ ，Cache 的容量为 $2^{c'+r+b}$ 。在组相连映像方式下，主存的容量也按 Cache 容量分成 t 个区，每个区又分成 $2^{c'}$ 个组，每个组内也包含 2^r 个字块。主存中存储块的数据可以调入 Cache 中相同组号内的任一块中，而主存中一个组的地址空间只能映像到 Cache 中相同组中。也就是说，主存字块 $Mm(i) (0 \leq i \leq 2^m - 1)$ 可以用下列映像函数映像到 Cache 字块 $Mc(j) (0 \leq j \leq 2^c - 1)$

$$j = (i \bmod 2^{c'}) \times 2^r + k \quad 0 \leq k \leq 2^r - 1$$

k 为位于上列范围内的可选参数(整数)。按这种映像方式，组间为直接映像，而组内的字块为全相联映像方式。

主存地址格式：

t	c'	r	b
区 号	组 号	组 内 块 号	块 内 地 址

Cache 地址格式：

c'	r	b
组 号	组 内 块 号	块 内 地 址

图 7.42 主存地址格式和 Cache 地址格式

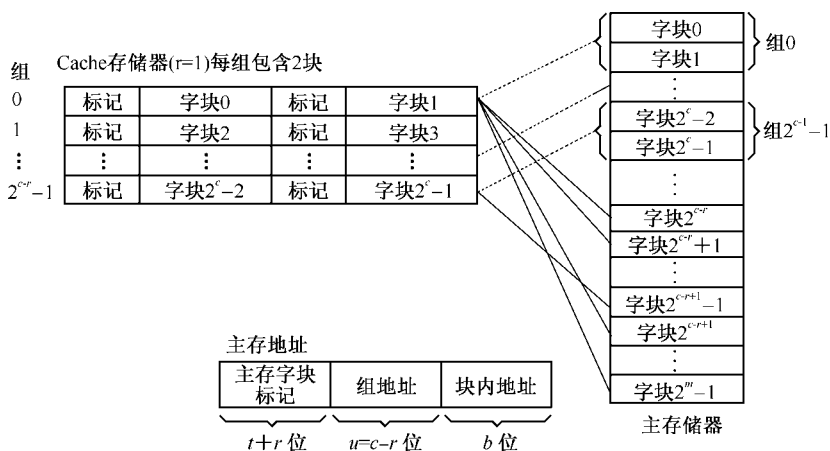


图 7.43 组相联映像地址变换

【例 7.12】 (Cache 容量比实际小):

设 $c'=3$ 位, $r=1$ 位, 主存字块 15 可映像到 Cache 的哪一个字块中?

根据公式, 可得:

$$\begin{aligned} j &= (i \bmod 2^c) \times 2^r + k \\ &= (15 \bmod 2^3) \times 2^1 + k \\ &= 7 \times 2 + k \\ &= 14 + k \end{aligned}$$

又 $0 \leq k \leq 2^r - 1 = 2^1 - 1 = 1$, 即 $k=0$ 或 1 。

代入后得 $j=14(k=0)$ 或 $15(k=1)$ 。所以主存模块 15 可映像到 Cache 字块 14 或 15 中。

如果组的容量只有一个 Cache 块时, 就成了直接映像; 如果组的容量是整个 Cache 块容量时, 就成了全相联映像; 组的容量是 n 个块时, 就成了 n 路组相联映像。组相联映像方式在各组间为直接映像, 而组内的各块之间为全相联映像方式。

Cache 地址中的块内地址部分直接取自主存的块内地址, 组号也直接取自主存的组号, Cache 组内的块号部分是通过查块表得到的。在访问存储器时, 根据主存地址中的区号和组内块号在块表中的该组对应的若干项进行查找, 看是否有相同的主存区号和组内块号。如果有相同的, 则表示 Cache 命中, 将对应的 Cache 组内块号取出以对 Cache 进行访问; 如果没有相同的, 则表示 Cache 不命中, 再对主存进行访问, 并在主存中的块调入 Cache 中的同时, 将主存的区号和组内块号和 Cache 组内块号写入块表中, 以改变地址映像关系。

【例 7.13】 有一个 Cache 主存存储层次, 采用组相联映像。主存共分 4096 个存储块, Cache 为 64 个存储块, 每组包含 4 个存储块, 每块由 128 个字节组成, 访存地址为字节地址。

- (1) 写出 Cache 地址位数和地址格式。
- (2) 写出主存地址位数和地址格式。
- (3) 主存地址 48AB9H 映像到 Cache 的哪个字块?

解: (1) Cache 容量 $= 64 \times 128 = 16 \times 4 \times 128 = 2^{13}$ 字节

因为 Cache 每块由 $128 = 2^7$ 个字节组成, 所以块内地址为 7 位; 每组包含 $4 = 2^2$ 个存储块, 组内块号为 2 位; Cache 为 $64 = 2^6$ 个存储块, 分为 $64/4 = 16 = 2^4$ 组, 组号地址为 4 位。

Cache 的地址格式:

12	9	8	7	6	0
组 号		组 内 块 号		块 内 地 址	

(2) 主存地址位数共 19 位, 因为主存总容量为 $4096 \times 128 = 2^{19} = 2^6 \times 2^{13}$, 块内地址为 7 位; 组内块号为 2 位; 组号地址为 4 位, 区号为 $19 - 7 - 2 - 4 = 6$ 位。

主存的地址格式:

18	13	12	9	8	7	6	0
区 号		组 号		组 内 块 号		块 内 地 址	

(3) 主存地址 48AB9H $= 100\ 1000\ 1010\ 1011\ 1001 = 0100100\ \mathbf{0101}\ 01\ 0111001$

主存地址 48AB9H 为 5 组(**0101**)内的 1(**01**)块。主存的 5 组映射到 Cache 的 5 组, 主存的 5 组的第 01 块, 映射到 Cache 地址的第 5 组中 4 个块中的任意一个块中, 所以, 主存地址 48AB9H 映射到 Cache 地址的第 $5 \times 4 + 0 = 20$ 块至 $5 \times 4 + 03 = 23$ 块的任意一块中。

【例 7.14】 设一个 4 路组相联映像的 Cache 中有 8 个块, 访问主存进行读的块地址顺序为十进制数 22、26、22、26、16、4、16、18, 求每次访问后, Cache 中的内容。

解：因为主存访问的是块地址，最大的块地址号为 26，所以可以用 5 位二进制数表示主存的块地址。Cache 中有 8 个块，每组有 4 个块，分成两组，需要 1 位组地址，两位块地址，因此以主存的第三位低地址作为 Cache 的组号，最高 2 位地址作为主存区号。访问主存进行读的块地址顺序为十进制数 22、26、22、26、16、4、16、18 时，所对应的二进制块地址为：10110、11010、10110、11010、10000、00100、10000、10010，对应的主存字块标记为：10、10、10、10、00、00、00、10，对应主存的组号 1、0、1、0、0、1、0、0，映像到 Cache 的第 0 组的块地址是：26、26、16、16、18；映像到 Cache 的第 1 组的块地址是：22、22、4。第一次访问主存块地址 22，放到第 0 组中的任一块中；第二次访问主存块地址 26，放到第 1 组中的任一块中；第三次访问主存块地址 22，第 0 组中已经有该块，访存命中；依此类推，每次访存后 Cache 中的块调入、命中、替换情况见表 7-9。

表 7-9 4 路组相联映像的块分配情况

访问顺序	1	2	3	4	5	6	7	8	组号
主存块地址	22	26	22	26	16	4	16	18	
Cache 000 块		26	26	26	26	26	26	26	第 0 组
Cache 001 块					16	16	16	16	
Cache 010 块									
Cache 011 块								18	
Cache 000 块	22	22	22	22	22	22	22	22	第 1 组
Cache 001 块						4	4	4	
Cache 010 块									
Cache 011 块									
操作状态	调入	调入	命中	命中	调入	调入	命中	调入	

在实际 Cache 中，用的最多的是直接映像($r=0$)，两路组相联映像($r=1$)和 4 路组相联映像($r=2$)。

如 $r=2$ ，计算得 $0 \leq k \leq 3$ ，所以主存某一字块可映像到 Cache 某组 4 个字块的任意一个字块中，这就大大地增加了映像的灵活性，提高了命中率。

组相联映像方式的性能与复杂性介于直接映像与全相联映像两种方式之间。组相联映像的主要优点是块的冲突概率比较低，利用率大幅度提高。组相联映像的主要缺点是实现难度和造价比直接相联映像要高。

Cache 的命中率除了与地址映像的方式有关外，还与 Cache 的容量有关。Cache 容量大，命中率高，但达到一定容量后，命中率的提高就不明显了。

7.7.3 替换算法

当新的主存字块需要调入 Cache 存储器而它的可用位置又被占满时，就产生替换问题。直接映像时，Cache 访问块失效时，则从主存中访问数据并将数据块写入 Cache 中失效的块中，不需要替换算法。只有在全相联映像和组相联映像方式时，主存储器的数据块可以写入 Cache 中的若干位置，就存在选择替换哪一个 Cache 块的问题，这时才使用替换算法，下面

介绍两种替换算法：随机替换算法(RAND)、先进先出(FIFO)算法和近期最少使用(LRU)算法。

随机替换算法(RAND)，是在组内随机选择一块来替换。没有根据访存局部性原理，不能提高系统的命中率。

FIFO 算法总是把一组中最先调入 Cache 存储器的字块替换出去，它不需要随时记录每个字块的使用情况，所以实现容易，开销小。FIFO 算法没有根据访存局部性原理，因为最早调入的存储块可能是经常要用到的或随后还要用到的信息。

LRU 算法较能反映访存局部性原理，把一组中近期最少使用的字块替换出去。它需要记录 Cache 中每个字块的情况，来确定哪个字块是近期最少使用的。LRU 算法的命中率要比 FIFO 算法和随机替换算法(RAND)高。

LRU 算法是把组中各块的使用情况记录在一张表上，如表 7-10 所示，计算最近一次使用以来所经历的时间，经历时间最长的块(最久未使用过的块)标上“*”号，把该块作为替换块。

表 7-10 LRU 算法块使用情况记录表

顺序	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	组 号
地址	1	12	4	11	3	7	0	11	12	15	4	6	4	7	12	
块分配																第 0 组
																第 1 组
状态																

【例 7.15】有一个 Cache-主存存储层次，采用两路组相联映像，先进先出(FIFO)替换算法，主存共有 16 个块，Cache 中有 4 个块，假设访问主存块地址序列为 1、12、4、11、3、7、0、11、12、15、4、6、4、7、12，如果主存中的内容一开始未装入 Cache 中，将每次访问 Cache 中各块的分配情况填入表 7-11 中。

表 7-11 2 路组相联映像 FIFO 替换算法的块分配情况表

顺序	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	组号
地址	1	12	4	11	3	7	0	11	12	15	4	6	4	15	12	
块分配	1	1	4	4	4	4	4	4	12	12	12	12	12	12	12	第 0 组
		12	12	12	12	12	0	0	0	0	4	4	4	4	4	
				11	11	7	7	7	7	15	15	15	15	15	15	第 1 组
					3	3	3	11	11	11	11	6	6	6	6	
状态	调入	调入	替换	调入	调入	替换	替换	替换	替换	替换	替换	替换	命中	命中	命中	

解：因为 Cache 中有 4 个块，每组内有 2 块，所以，分成两组，组地址 1 位，块地址 1 位；主存共有 16 块，4 位地址，其中组地址 1 位，块地址 1 位，区地址 2 位；用两路组相联映像，保证主存的组号与 Cache 组号相同，同一组内的块采用全相联映像。主存块地址序列

为 1、12、4、11、3、7、0、11、12、15、4、6、4、15、12，把它们转换成二进制地址： $D_3D_2D_1D_0$ ，则 D_1 位为组号， D_0 位为块号。如 $(1)_{10}=(0001)_2$ ，组号为 0，映像到 Cache 的第 0 组； $(12)_{10}=(1100)_2$ ，组号为 0，映像到 Cache 的第 0 组；第 0 组已经装满。 $(4)_{10}=(0100)_2$ ，组号为 0，也映像到 Cache 的第 0 组。块 1 最先调入到第 0 组，采用 FIFO 法，块号 4 替换掉块号 1。 $(11)_{10}=(1011)_2$ ，组号为 1，映像到 Cache 的第 1 组，依此类推。将每次访问 Cache 中各块的分配情况填入表 7-10 中。

【例 7.16】有一个 Cache-主存存储层次，采用全相联映像 Cache 中有 4 个块，采用近期最少使用(LRU)替换算法，假设访问主存块地址序列为 3、14、3、9、7、6、3、7，如果主存中的内容一开始未装入 Cache 中，将每次访问 Cache 中各块的分配情况填入表 7-12 中。

表 7-12 全相联映像 LRU 替换算法的块分配情况表

顺序	1	2	3	4	5	6	7	8
地址	3	14	3	9	7	6	3	7
块分配								
状态								

解：在近期最少使用方式下，全相联映像 LRU 替换算法的块分配情况如表 7-12 所示，其中带星号的块是近期最少使用的块。在表的访问顺序中，第一步调入块 3，第二步时地址块号为 3 的块是近期最少使用的，因为块 14 是刚刚进入的，第三步访问了块 3 命中，块 14 就成了近期最少使用的块，第四步调入了块 9，第五步调入了块 7，这时块 14 还是近期最少使用的块；第六步访问了块 6 替换掉近期最少使用的块 14，块 3 就成了近期最少使用的块；第七步访问了块 3 命中，块 9 就成了近期最少使用的块；第八步访问了块 7 命中。将每次访问 Cache 中各块的分配情况填入表 7-13 中。

表 7-13 全相联映像 LRU 替换算法的块分配情况表

顺序	1	2	3	4	5	6	7	8
地址	3	14	3	9	7	6	3	7
块分配	3*	3*	3	3	3	3*	3	3
		14	14*	14*	14*	6	6	6
				9	9	9	9*	9*
					7	7	7	7
状态	调入	调入	命中	调入	调入	调入	命中	命中

Cache 存储单元在替换时，应避免出现颠簸现象，即 Cache 的命中率有时很高有时又很低的现象。例如，在一个具有 4 个块的全相联映像的 Cache 中，如果程序运行的信息集中在 4 个块中，并且这 4 个块被调入 Cache 中，当程序循环运行时，就可使得 Cache 全部命中；而当程序循环的轮流访问 5 个块，采用先进先出替换方式下，每次替换操作调出去

的块恰好是下一次访问要使用的,使得 Cache 全部失效,这种情况最好采用近期最少使用替换算法提高命中率。

7.8 小 结

半导体存储器是用来存放计算机程序 and 数据的设备,是微机系统中的重要组成部分。应重点掌握半导体存储器分类和特点,熟悉典型半导体存储器芯片的应用、微机存储系统的结构及 CPU 与存储器的连接,重点掌握内存存储器容量的扩展、提高存储器工作速度的技术及高速缓冲存储器地址映像与变换。

7.9 习 题

1. 半导体存储器分为两大类:易失性存储器 RAM 和非易失性存储器 ROM。闪速存储器属于_____。
2. 衡量半导体存储器性能的主要指标有_____,_____,_____,_____和_____。
3. 基本静态存储器 RAM 靠_____存储信息;基本动态存储器 RAM 靠_____存储信息。
4. 在计算机系统中,下列部件能够存储信息:主存、CPU 内的寄存器、Cache、磁带和磁盘。按照 CPU 存取速度排列,从快到慢依次是_____,_____,_____,_____和_____。由半导体材料构成的有_____。
5. 启动一次存储器操作(读或写)到完成该操作所需要的时间称为_____;连续启动两次独立的存储器操作所需间隔的最小时间称为_____;通常_____略长一些。
6. 一个 512KB 的存储器,其地址线为_____根,数据线为_____根。
7. 某微机字长是 32 位,它的存储容量是 64KB,按字编址,它的寻址范围是_____。
8. 某容量为 8KB 的存储器,首地址为 2000H,其尾地址为_____。
9. 一个具有 16 位地址线和 8 位字长的存储器,该存储器的容量是_____KB;如果该存储器由 1K*1 位的静态存储器芯片构成,需要_____片芯片。
10. Cache 是一种位于_____和_____之间规模较_____,但存取速度很快的一种_____。
11. Cache 内保存的是_____最近时间常用的信息,而不是在主存中驻留的信息。
12. 在 Cache 的地址映射中,若主存中的任意一块均可以映像到 Cache 内的任一块中,则这种映像方式称为_____。
13. 选择存储器时,应该从哪几个方面进行考虑?
14. 静态 RAM 是如何工作的?有什么特点?
15. DRAM 为什么要刷新?

16. 反映存储器工作速度的主要技术指标有哪些? 提高存储器工作速度的技术有哪些?
17. Cache 中采用哪些地址映像与变换技术?
18. Cache 中采用哪些块替换算法? 目前常用的算法是什么?
19. 设有 32 片 $256\text{K} \times 1$ 位的 SRAM 芯片。
 - (1) 只采用位扩展方法可构成多大容量的存储器?
 - (2) 如果采用 32 位的字编址方式, 该存储器需要多少地址线?
20. 设有若干片 $4\text{K} \times 1$ 位的 SRAM 芯片, 设计一个总容量为 64K 的 16 位存储器。
 - (1) 需要多少片 $4\text{K} \times 1$ 位的 SRAM 芯片?
 - (2) 画出该存储器与 CPU 连接的结构图, 设 CPU 的接口信号有地址信号、数据信号、控制信号 $\overline{\text{MREQ}}$ 、 $\text{R}/\overline{\text{W}}$ 和片选信号 $\overline{\text{CS}}$ 。
21. 设有若干片 256KB 的 SRAM 芯片, 构成 2048KB 存储器。
 - (1) 需要多少片 256KB SRAM 芯片?
 - (2) 构成 2048KB 存储器需要多少地址线?
 - (3) 画出该存储器与 CPU 连接的结构图, 设 CPU 的接口信号有地址信号、数据信号、控制信号 $\overline{\text{MREQ}}$ 、 $\text{R}/\overline{\text{W}}$ 和片选信号 $\overline{\text{CS}}$ 。
22. 用 $4\text{M} \times 8$ 位的存储芯片构成一个 $64\text{M} \times 16$ 位的主存储器。画出该存储器与 CPU 连接的结构图, 设 CPU 的接口信号有地址信号、数据信号、控制信号 $\overline{\text{MREQ}}$ 、 $\text{R}/\overline{\text{W}}$ 和片选信号 $\overline{\text{CS}}$ 。
 - (1) 计算需要多少个 $4\text{M} \times 8$ 位存储器芯片。
 - (2) 存储器芯片的片内地址长度是多少位?
 - (3) 在图中标明主存储器地址线 and 数据线各需要多少位?
 - (4) 画出用存储器芯片构成主存储器的完整逻辑示意图。
23. 由一个具有 8 个存储体的低位多体交叉存储体中, 如果处理器的访存地址为以下八进制值。求该存储器比单体存储器的平均访问速度提高多少? (忽略初启时的延长)
 - (1) $1001_8, 1002_8, 1003_8, \dots, 1100_8$
 - (2) $1002_8, 1004_8, 1006_8, \dots, 1200_8$
 - (3) $1003_8, 1006_8, 1011_8, \dots, 1300_8$
24. 某计算机有一个指令和数据合一的 Cache, 已知 Cache 的读/写时间为 10ns, 主存的读/写时间为 100ns, 取指的命中率为 98%, 数据的命中率为 95%, 在执行程序时, 约有 20% 的指令需要存/取一个操作数, 为简化起见, 假设指令流水线在任何时候都不阻塞。问设置 Cache 后, 与无 Cache 比较, 计算机的运行速度可提高多少倍?
25. 一个直接映像 Cache 的块长为 4 个 16 位字, 容量为 4096 个字, 主存容量为 64K 字, Cache 中有多少个块? 其地址怎样划分? 主存中有多少个块? 主存地址又怎样划分?
26. 有一个 Cache-主存存储层次, 主存共分 16 个块, Cache 为 4 个块, 采用全相联映像, 替换算法为先进先出法(FIFO)。Cache 初始时空, 对于如下主存块地址流的访问顺序 1, 12, 6, 7, 4, 12, 1, 7, 8, 11, 8, 15, 14。列出 Cache 中各块随时间的使用情况。
27. 有一个 Cache-主存存储层次, 主存共分 8 个块, Cache 为 4 个块, 采用组相联映像, 组内块数为 2 块, 替换算法为近期最少使用法(LRU)。

- (1) 对于如下主存块地址流的访问顺序是：4，1，2，4，3，7，0，7，2，5。Cache 初始时空，请列出 Cache 中各块随时间的使用情况。
- (2) 对于(1)，指出块失效又发生争用(即替换)的时刻。
- (3) 对于(1)，求出此期间 Cache 的命中率。

第 8 章 80386/80486 CPU 的存储器管理

教学提示：本章将扼要介绍 80386、80486 等 CPU 内存的三种工作模式：实模式存储器管理、保护虚地址方式存储器管理和虚拟 8086 模式。

教学要求：本章让学生理解存储器管理的功能和一些基本概念：逻辑地址、线性地址、物理地址、虚拟存储器、段和页等；理解分段存储管理技术和分页存储管理技术的实现思想，掌握从逻辑地址到物理地址的转换过程；了解保护机制。

8.1 实模式存储器管理

所谓存储器管理，其实是一个硬件管理机制，由于它的存在可以让操作系统为众多运行程序创建一个便于管理的、和谐的存储环境，方便软件程序对存储器的应用。

当 80386/486 CPU 由硬件复位后在工作实地址方式下，在此方式下，80386/486 的 32 位地址总线中只能使用低 20 位地址，存储空间为 1MB。此时的 80386/486 就相当于一个高速的 8086/88 微处理器。与 8086/88 微处理器唯一不同的是，它不仅可以运行 8086/88 的全部指令，而且还可以运行 32 位运算类指令。

8.1.1 存储器的分段结构

80386、80486 等 CPU 组成的微型计算机大多配置了 4MB~16MB 的内存，Pentium 以上的 PC 机则配置了 32MB~64MB 甚至更多的内存，本节将扼要介绍这些内存的管理和使用方法。

1. 寻址范围

不同 CPU 因地址线数目的不同，其寻址范围也不同，如表 8-1 所示。

表 8-1 各类型 CPU 的寻址能力

CPU	数据总线	地址总线	寻址范围	支持操作系统
8088	8 位	20 位	1MB	实方式
8086	8 位	20 位	1MB	实方式
80286	16 位	24 位	16MB	实、保护方式
80386	32 位	32 位	4GB	实、保护、V86 方式
80486	32 位	32 位	4GB	实、保护、V86 方式
Pentium	64 位	36 位	64GB	实、保护、V86 方式

Intel 8086 的地址总线为 20 位，寻址空间仅有 1MB。80386/486 的地址总线为 32 位，最大寻址空间为 $4\text{G} \times 32$ 位，地址总线为 n 位，最大寻址空间为 $2^n \times 32$ 位。

2. 实模式存储器分段管理

当 80386/486 CPU 由硬件复位后在工作实地址方式下，程序对地址的访问采用分段地址方式。在描述存储器地址时，有四个相关的术语：物理地址、逻辑地址、偏移地址(也称作有效地址偏移量)和段基地址。

物理地址是由 80386/80486 的地址引线送出的 20 位地址码。这 20 位地址码送到存储器经过译码，最终选定一个存储单元进行读/写，物理地址可写成 5 位的十六进制数。物理地址是在 CPU 芯片地址信号引脚上出现的地址，所以它的大小由 CPU 的地址总线数决定。虚拟地址即是逻辑地址，是程序在访问存储器时由指令指明地址的一种表示方法，由某段的段地址和段内偏移地址组成。写成：

段地址：偏移地址

例如，2000H：0080H。

对存储器寻址起作用的是物理地址，实方式 80386/486 CPU 对外有 20 位地址线，因此，存储器的可寻址范围为 $2^{20}=1\text{MB}$ 。存储器地址分段的具体做法是：把 1MB 的存储器空间分成若干段，每段的容量最大为 64KB，这样段内地址就可以用 16 位来表示。实际上，可以根据编程的需要来确定段的大小，它可以是 64KB 范围内的任意多个字节。8086 规定：从 0 地址开始，每 16 个字节为一小段，段的起始地址必须从任一小段(paragraph)的首地址开始，也就是说，段地址表示成 20 位的二进制地址码，其最低 4 位必须是“0”。

8.1.2 物理地址的形成

存储器采用分段管理后，每个段的首地址(称为段基地址或段地址)保存在 8086 内部的 CS、DS、SS 和 ES 四个 16 位的段寄存器中，可以对段寄存器设置不同的值以指向不同的段起始点。偏移地址是相对于某段首地址的段内偏移量，用来寻址段内的一个特定字节，它给出了要寻址字节距离段基地址的无符号位移量，通常由指令寻址方式给出。用 16 位二进制代码表示，写成 4 位十六进制数，例如，004AH。16 位的段地址和 16 位的偏移地址组合形成 20 位的物理地址，这就是 80386/486 的实模式存储器管理模式。物理地址的计算方法如图 8.1 所示。

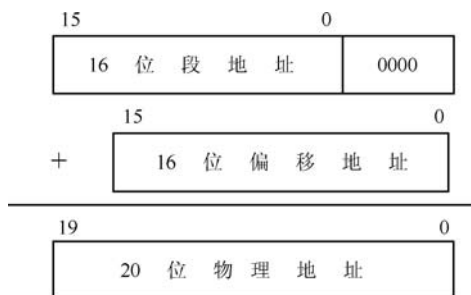


图 8.1 实模式下物理地址的生成

也就是说,把段地址左移 4 位得到段基地址,再加上偏移地址值就形成了 20 位的物理地址。计算表示方式如下:

$$\text{段地址} \times 10\text{H} + \text{偏移地址} = \text{物理地址}$$

例如,段地址:偏移地址=2264H:1020H,则物理地址=22640H+1020H=23660H

程序对存储单元的寻址均采用相对段起始地址的偏移量来实现,这种分段结构支持代码的动态重新安装。因此,可由操作系统将程序装入物理存储器的任意位置,只要重新设计好段地址即可。当程序被处理执行时,由地址生成逻辑自动将段地址和偏移地址转换成物理地址。

8.2 保护虚地址方式存储器管理

当 80386/486 的控制寄存器 CR0 的最低位 PE 即允许保护位置成 1 时,80386/486 允许分段实施保护,在这种情况下,80386/486 工作在保护虚地址方式下。

80386/486 使用的存储器管理机制是分段和分页管理,它们都是使用驻留在存储器中的各种表格,规定各自的转换函数。这些表格只允许操作系统进行访问,而应用程序不能对其进行修改。这样操作系统为每一个任务维护一套各自不同的转换表格,其结果是每一任务有不同的虚拟地址空间,并使任务彼此隔离开来,以便完成多任务分时操作。

80386/486 先使用分段机制,把包含两个部分的虚拟地址空间转换为一个中间地址空间的地址,这一中间地址空间称为线性地址空间,其地址称为线性地址。如果仅采用分段管理机制,线性地址就是物理地址。如果采用分页管理机制,需要再用分页机制把线性地址转换为物理地址,如图 8.2 所示。

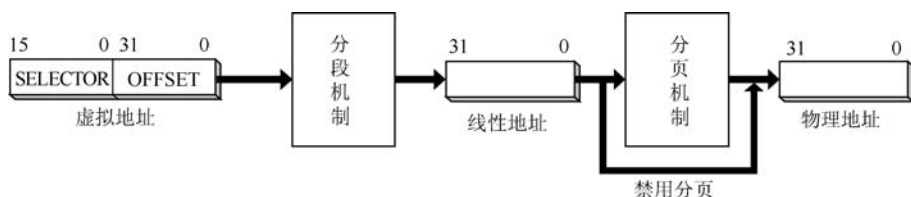


图 8.2 虚拟-物理地址转换

虚拟地址空间是二维的,它所包含的段数最大可到 16K,每个段最大可到 4GB,从而构成 64000GB 容量的庞大虚拟地址空间。线性地址空间和物理地址空间都是一维的,其容量为 $2^{32}=4\text{GB}$ 。事实上,分页机制被禁止使用时,线性地址就是物理地址。

80386 以后的微处理器均支持 3 种工作方式,即实地址方式、虚地址保护方式和虚拟 8086 方式(简称 V86)。80286 只有实地址方式和虚地址保护方式两种工作方式。8088/8086 只工作在实地址方式下。

虚拟存储技术是建立在主存和大容量辅存物理结构基础之上,由附加硬件装置及操作系统内的存储管理软件组成的一种存储体系,它将主存和辅存(硬盘)的地址空间统一编址,提供了比实际物理内存大得多的存储空间。任何时刻,存储器管理软件只要把与正在运行程序相关的一小部分虚拟地址空间映射到内存储器,其余部分则仍存储在磁盘上。当用户

访问存储器的范围发生变化时,处于后台的存储器管理软件再把用户所需要的内容从磁盘调入内存,同时原来调入内存的一部分也可以再调回磁盘中,用户感觉起来好像就是在访问一个非常大的线性地址空间。这样一种机制使编程人员在写程序时,不用再考虑计算机的实际内存容量,可以写出比实际配置的物理存储器大很多的程序,对用户来说,使用起来非常方便。

虚拟存储器(Virtual Memory)圆满地解决了计算机存储系统对存储容量、单位成本和存取速度的苛刻要求,取得了三者之间的最佳平衡。

8.2.1 存储器的分段管理

80386/486 的分段模式,使用有两个部分的虚拟地址,即段选择符及有效的地址偏移量。有效的地址偏移量与实方式一样,根据指令中操作数的寻址方式确定。段选择符也与实方式一样,为 16 位段寄存器的内容,即可使用 CS, DS, SS, ES, GS, FS 共 6 个 16 位段寄存器,段选择符只能间接地提供段的基地址,并且段寄存器也称为段选择器。

如图 8.3 所示,在保护虚拟方式下,每个存储器段都对应着一个段描述符(Descriptor),段描述符由两部分组成:16 位的段选择符和 32 位的偏移量。若干个段的描述符组成一个描述符表,存储在由操作系统专门定义的存储区内,称为特殊的段。某个段的描述符在描述表中的位置,则由段选择器中的段选择符进行索引。这样,在 80386/486 微处理系统内,系统给出的地址及程序给定的地址都是逻辑地址。而逻辑地址又由用来指示这个段的一个 16 位段选择符(Segment Selector)和一个只能在这个段内使用的 32 位偏移量组成。对逻辑地址的访问权和访问范围都要进行检查,如果通过了这次检查,便可由段选择符在描述符表中检索出该段的描述符,从而得到段的 32 位基地址、段界限和关于该段的访问权等信息。可见段选择符与段的基地址之间通过描述符表中的段描述符存在着一一对应的关系。段的基地址即段的 32 位起始地址,再加上 32 位的偏移地址,便可得到 32 位的线性地址。

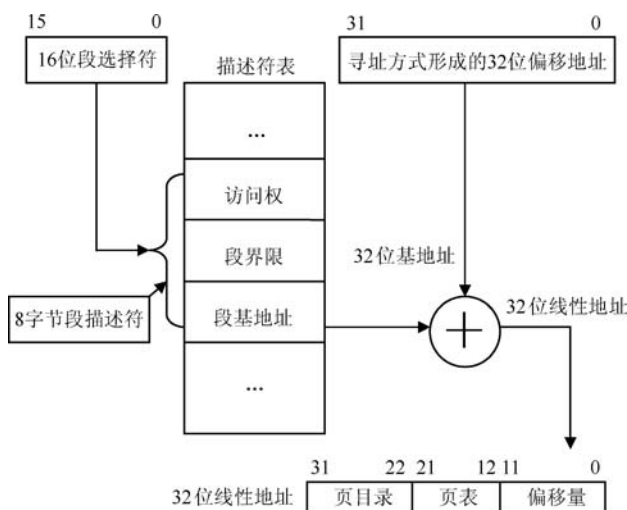


图 8.3 保护虚拟方式下的分段机制

1. 段选择符(Segment Selector)

保护方式下, 段选择符指示着段描述符, 在段描述符中, 包含有定义段所用的全部信息。其实, 一个程序可以拥有比 6 个段寄存器指示的段还要多的段。若某个程序使用的段多于 6 个, 当程序需要去访问一个新段时, 就使用传送类指令 MOV, 去改变这些寄存器中的内容。由段选择符识别段描述符, 而段描述符在段描述符表内被逐一登记注册。对应用程序来说, 段选择符作为指针变量的一个组成部分是可见的, 但段选择符的值通常是由连接编辑程序(Link Editor)或连接装配程序(Linking Loader)指定或修改的, 而不是由应用程序指定, 更不能由应用程序对其实施修改操作。段选择符是一个指向操作系统定义的段信息的指针, 其格式如图 8.4 所示。段选择符是由 16 位信息组成的, 这 16 位信息又被进一步细分成 3 个字段: 分别是 13 位索引字段、1 位的指示符字段 TI 以及 2 位的请求特权级字段 RPL。现分述如下:



图 8.4 保护方式段选择符格式

1) 请求特权级字段(Requestor Privilege Level, RPL)

RPL 字段是由两位(位 1 和位 0)组成, 可以表示 0~3 共 4 个特权级。RPL 域的设置是为了防止低特权级程序访问受高特权级程序保护的数据。若这个字段表示特权级的值比程序的特权级值小(即该字段所表示的特权级比程序的高), 就用这个选择符去访问这个程序时要覆盖掉这个程序的特权级; 若某一程序所用段选特权级较低, 它就在低特权级上对存储器进行存储操作。这样就可以防止低特权级对高特权级程序的数据进行访问。例如, 为了能对像外围接口设备中的控制寄存器这样的受保护的设备进行访问操作, 系统实用程序或设备驱动程序就必须用高的特权级。即使接收的是来自特权级低的程序请求, 它们也不应该干扰其他的被保护设备。

2) 段描述符指示符字段 TI

TI 字段是位 2, 为表格指示器(Table Indicator, TI)位。当 TI=0 时, 该选择符指向的段是系统的全局地址空间(称为全局描述符表 GDT); 当 TI=1 时, 该选择符指向的段是一个特定程序或任务的局部地址空间(称为局部描述符表 LDT)。全局地址空间用来存放运行在系统上的所有任务使用的数据和代码段, 如操作系统服务程序、通用库和运行时间支持模块等。只要 TI=0, 便会指向这一空间, 这也就是说, 在系统上运行的所有任务共享同一个全局地址空间。局部地址空间用来存放一个任务独自占有的特定程序和数据, 系统中每个任务都有其对应的局部地址空间。

顾名思义, 全局描述符表是供系统中所有程序使用的; 而局部描述符表是供各自运行程序使用的。如果操作系统允许, 不同的程序可以共享同一个局部描述符表 LDT。当然, 系统也可以不使用局部描述符表 LDT, 而所有程序都使用全局描述符表 GDT。

3) 索引字段

索引字段是由 13 位组成, 利用索引字段可以从拥有 2^{13} 个的段描述符表中选出一个段

描述符来。微处理机是用 $8(8 \text{ 是段描述符的字节数}) \times \text{索引值} + \text{描述符表的 32 位基地址}$ 换算出来的。而描述符表的 32 位基地址既可以是来自全局描述符表寄存器 GDTR, 也可以是来自局部描述符表寄存器 LDTR。在这两种寄存器中保存着描述符表的起始线性地址, 图 8.5 示出了怎样用段选择符中的 TI 位说明使用的是全局描述符表还是局部描述符表。

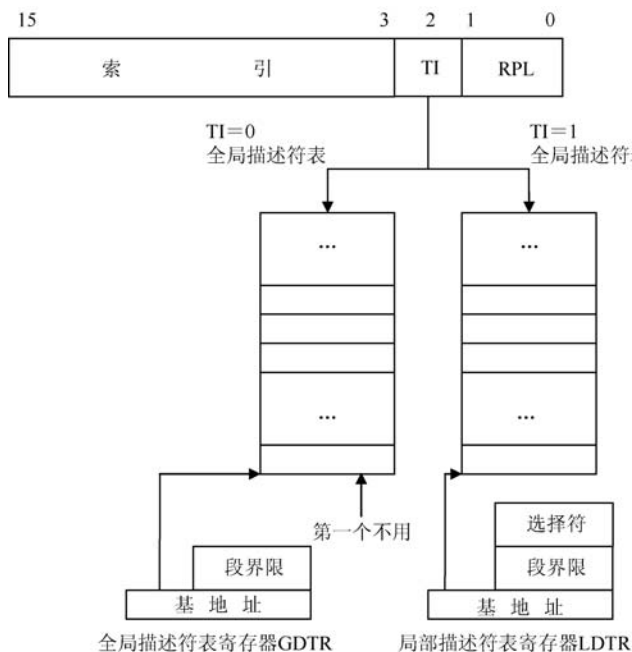


图 8.5 由指示符 TI 位选择的描述符

逻辑地址经分段部件转换后所形成的地址被称之为线性地址。若没使用分页机制, 这时的线性地址就是物理地址; 如果使用了分页机制, 二级地址转换机制(即分页机构)所生成地址就是物理地址。

2. 段描述符

段描述符, 是由操作系统定义的、位于存储器内的一种数据结构, 段描述符内保存着供处理机使用的有关段的属性、大小规模、段在存储器中的位置及控制和状态信息, 段描述符在存储器中的一个段和一个任务之间形成了一个链。它是一种特殊的段, 不管是全局地址空间还是局部地址空间中的一个段, 如果没有描述符, 则对该段任务来说便无效, 且没有访问它的机制。一般说来, 各段描述符是由各种编译程序、各种连接程序、各种装入程序或者操作系统产生的, 而不是由各种应用程序生成的。

描述符分为段描述符和门描述符两大类。段描述符又分成两类: 一般的段描述符(也称存储段描述符, 包括代码段描述符和数据段描述符)和特殊的段描述符(又称系统段描述符, 包括局部描述符表 LDT 描述符和任务状态段 TSS 描述符等)。门描述符包括调用门描述符、任务门描述符、中断门描述符和陷阱门描述符 4 种, 当特权级之间和任务之间进行转移控制时, 使用这些门描述符。各种类型段使用的段描述符只能是其中一种。

段描述符的一般格式如图 8.6 所示。从图 8.6 可以看出,一般段描述符是由 8 个字节共 64 位构成,且又被进一步细分成若干个字段,现将各字段的意义分述如下。



图 8.6 段描述符的一般格式

1) 基地址

用该字段来规定某一个段在 4GB 物理地址空间中的起始地址。从图 8.6 中可以看出,基地址字段又是用基地址 15~0、基地址 23~16 和基地址 31~24 这 3 个互相独立的部分表示的。其实,在实际使用时,微处理机是把这 3 个基地址字段按序组合在一起,从而形成一个唯一的 32 位的值来表示这个段的基地址。对 80386 和 80486 来说,段基地址 32 位宽,决定了段可在 4GB 线性地址空间中的任何字节处开始。可以看出,保护方式下段基地址的设置与实方式下段基地址仅限于被 10H 整除的地址处是不同的,保护方式下段的基地址值应该与 16 字节的边界对准,通过对准 16 字节的代码段或数据段的边界,可使程序最大限度地发挥其性能。

2) 粒度 G

在 80386、80486 中,用它来指定段界限的单位。 $G=0$,表明段界限以字节为单位; $G=1$,表明段界限以页为单位,每页 4KB。

3) 段界限

段界限是用来定义段的大小规模的,从图 8.6 中可以看出,段界限是用段界限 15~0 和段界限 19~16 两部分表示的,在用到段界限时把这两部分段界限组合在一起,从而形成了一个 20 位无符号的段界限值。但仅就 20 位的段界限还不能确定出段的尺寸,还需要根据段描述符中粒度 G 的设置来共同确定。若粒度位 $G=0$,即设置成字节粒状,界限以字节为单位,段界限值为 1B~1MB(因为段界限为 20 位, $2^{20}=1\text{MB}$)范围。在这种情况下,段界限字段的值在 1 个字节的基础上可每次增加 1 个字节。若粒度位 $G=1$,即设置成页粒状,界限以页为单位,一页为 4KB,段的尺寸为 4KB~4GB,段界限字段的值在 4KB 的基础上每次可增加 4KB。

4) S 位

$S=1$,表示为一般的段描述符,这个段或者是代码段,或者是数据段(具体要由类型域的位 11 来确定);如图 8.7(a)所示的数据段描述符格式和图 8.7(b)所示的代码段描述符; $S=0$,表示为一个特殊的段描述符或门描述符,例如,系统段描述符如图 8.7(c)所示。

5) D/B 位

该位为默认操作尺寸位,不用于地址生成,只在 80386 及 80486 的代码段描述符中被识别。当 $D=0$ 时,表示操作数和有效地址的默认值是 16 位,即 80286 方式;当 $D=1$ 时,表示操作数和有效地址的默认值为 32 位,即 80386/80486 方式。

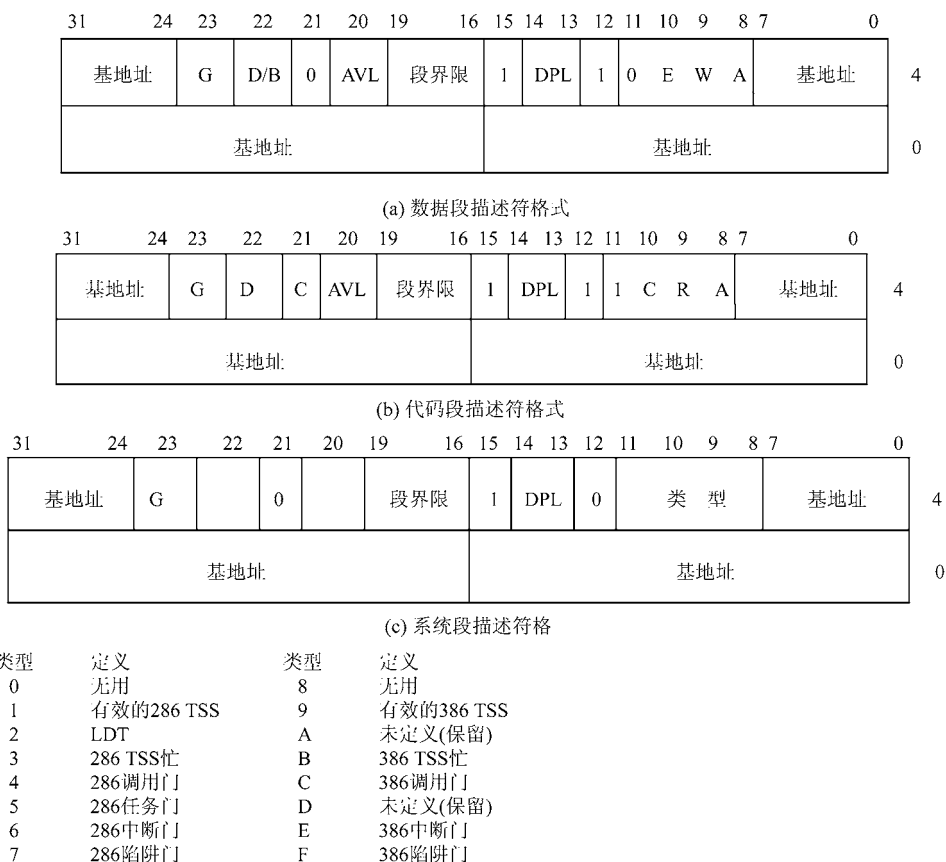


图 8.7 常用的段描述符格式

6) 类型

类型域共有 3 位。在 S 位对两大类描述符进行分类后, 该类型域再对每一类描述符进行更细的划分。当 S=1 时, 表示为一般的段描述符。类型域的最高位 E=1 代表代码段, E=0 代表数据段。E=1 时, 工作在代码段情况下, 类型域的后两位 C=1(一致性)、R=1(可读性); E=0 时, 工作在数据段情况下, 类型域的后两位 ED=1(向下扩展的堆栈段), ED=0(一般的数据段); W=1(可写)。

7) 描述符特权级 DPL 位

用这个字段来定义段的特权级。借助于保护机构, 用这个字段定义的特权级去控制对这个段的访问, 仅用于保护, 不用于地址的生成。

8) 段存在位 P

用于表示描述符所描述的段存在于存储器中(P=1)或不存在于存储器中(P=0), 它用于地址生成。如果一个描述符所描述的段已移至硬盘上, 表明不在内存中, 这时 P=0。若这种情况下试图将该描述符的段选择符装入段寄存器, 处理器便会产生一个段不存在异常(以 P=0 为标志), 并且操作系统中的异常处理程序会把该段重新装入存储器。凡在被标有“段不存在”信息的段内, 其各项都被重新进行分配(只需把段存在位置成 0)。这时就可以把这个段使用的存储空间分配给另一个段使用。若下一次又需要那个被重新分配的段, 这时,

段不存在异常事故会指示，需要把这个段装到存储器中去。在任何时候，通过在物理存储器中仅保留少数几个段，可以使一个系统所拥有的虚拟存储空间的总数远远大于物理存储器的实际存储容量。

9) 存取操作位 A

当每个代码段或数据段被访问时，A 位便被置 1。通过将 A 位置成 0，然后操作系统再进一步对其进行测试，周期性地查看、记录各段描述符中 A 位的状态，经一段时间的统计即可得知各段被访问的频繁程度。

特殊的段描述符及门描述符。当 S=0 时，表示为一个特殊的段描述符(也称为系统段描述符)或门描述符。系统段描述符的格式如图 8.7(c)所示，由图 8.7(c)可看出，其格式与一般段描述符基本相同。图中描述了 4 位类型域对各种系统段描述符和门描述符的定义情况，其中 TSS 是任务状态段，用来存储与任务有关的信息。此外，对 80286 来说，类型 8~F 未定义。

3. 段描述符表

段描述符表简称描述符表，用来存储保护方式下段描述符的一个阵列。80386/80486 CPU 共有 3 种描述符表：全局描述符表 GDT、局部描述符表 LDT 和中断描述符表 IDT。描述符表由描述符顺序排列组成，占一定的内存，由系统地址寄存器(GDTR、LDTR、IDTR)指示其在物理存储器中的位置和大小。

全局描述符表 GDT 是供所有任务使用的描述符表，在物理存储器地址空间中定义全局描述符表 GDT。通常操作系统使用的有代码段描述符、数据段描述符、调用门描述符、各个任务的 LDT 描述符、任务状态段 TSS 描述符、任务门描述符等。

局部描述符表 LDT 是每一项任务运行时都要使用的描述符表。在多任务操作系统管理下，每个任务通常包含两部分：与其他任务共用的部分及本任务独有的部分。与其他任务共用部分的段描述符存储在全局描述符表 GDT 内；本任务独有部分的段描述符存储在本任务的局部描述符表 LDT 内。这样，每个任务都有一个局部描述符表 LDT，而每个 LDT 表又是一个段，它也就必须有一个对应的 LDT 描述符。该 LDT 描述符存储在全局描述符表中。局部描述符表 LDT 中所存储的属于本任务的段描述符通常有代码段描述符、数据段描述符、调用门描述符及任务门描述符等。

GDT 和 LDT 段描述符表实际上是段描述符的一个长度不定的数据阵列，如图 8.5 所示。描述符表在长度上是可变的，最多容纳 2^{13} 个描述符，最少包含一个描述符。每个项有 8 个字节长，称为一个段描述符。中断描述符表 IDT 暂不介绍。

4. 描述符表寄存器

80386/486 微处理机用全局描述符表寄存器 GDTR、局部描述符表寄存器 LDTR 和中断描述符表寄存器 IDTR，来查找全局描述符表 GDT、局部描述符表 LDT 和中断描述符表 IDT，如图 8.8 所示。



图 8.8 全局描述符表寄存器 GDTR 和局部描述符表寄存器 LDTR

1) 全局描述符表寄存器 GDTR

在全局描述符表寄存器 GDTR 内, 保存着全局描述符表在物理地址空间内的 32 位的基地址, 同时全局描述符表寄存器 GDTR 中, 还保存表明这个表的规模大小为 16 位的界限值, 16 位段界限表明 GDT 表最长为 $2^{16}\text{KB}(64\text{KB})$ 。

【例 8.1】(GDTR)=0010 0000 0FFFH, 求 GDT 在物理存储器中的起始地址, 结束地址, 表的大小, 表中可以存放多少个描述符?

解:

GDT 的起始地址为 0010 0000H

结束地址为: $0010\ 0000\text{H} + 0\text{FFFH} = 0010\ 0\text{FFFH}$

表的大小为: $0\text{FFFH} + 1 = 4096$ 字节

表中可以存放: $4096/8 = 512$ 个描述符

2) 局部描述符表寄存器 LDTR

局部描述符表寄存器 LDTR 包含一个 16 位的选择器(63 位~48 位)和不可见的高速缓存器(47 位~0 位)。不可见的高速缓存器内保存着局部描述符表 LDT 的基地址、它的界限以及访问控制权。对正在运行的任务而言, 每一项任务运行时都要使用局部描述符表 LDT, 它们存储在存储器的一个独立的段里。但每个 LDT 处于存储器中的地址如图 8.9 所示, 用局部描述符表指令 LLDT 对局部描述符表寄存器 LDTR 内的选择符进行读操作, 用保存在局部描述符寄存器内的 16 位的选择符识别 GDT, 从 GDT 中检索出相应的 LDT 描述符, 处理器便将局部描述符自动置入 LDTR 的高速缓存器中, 将该描述符装入高速缓冲寄存器就为当前任务创建了一个 LDT。这样 80386/486 微处理器便可以根据 LDTR 高速缓存器的值来确定局部描述符表的起始地址和段界限, 而不必再访问存储器从 GDT 中查出 LDT 描述符, 直接从 LDTR 高速缓存器来确定局部描述符表的起始地址和段界限, 节省了程序运行时间。

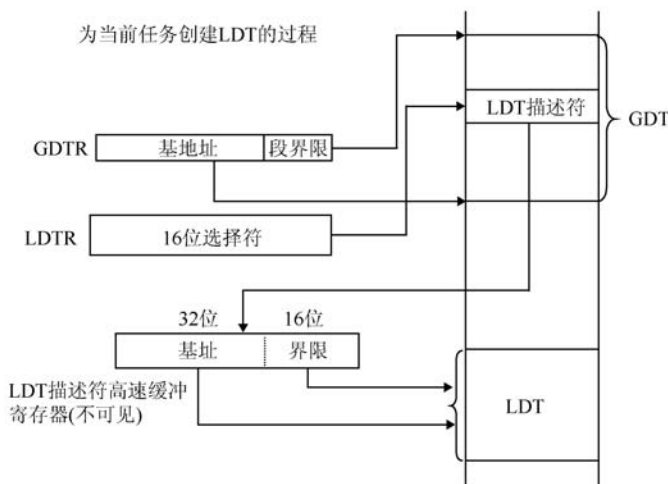


图 8.9 操作系统根据 LDT 使用情况为 LDTR 赋值

3) 中断描述符表寄存器 IDTR

中断描述符表寄存器 IDTR 与全局描述符表寄存器 GDTR 完全相同, 用来在物理存储

器地址空间中定义中断描述符表 IDT。中断描述符表寄存器 IDTR 内保存着中断描述符表在物理地址空间内的 32 位的基地址, 由于处理器只能支持 256 个中断和异常, 因此段界限最大为 0FFFH。

5. 段寄存器

在保护方式下, 每个段寄存器(CS、DS、SS、ES、FS、GS)都有一个 16 位的可见部分(其中内容称为段选择符)和一个程序无法访问的 64 位不可见部分(称为段描述符高速缓存器), 如图 8.10 所示。段寄存器中 16 位选择符用来在 GDT 或 LDT 中寻找一个段描述符, 再由段描述符确定一个段。64 位段描述符高速缓存器包含一个 32 位段基地址、20 位段界限和 12 位访问权限域。

16 位段寄存器	64 位段描述符高速缓存器		
CS/SS/DS/ES/FS/GS	32 位段基地址	20 位段界限	12 位访问权限域

图 8.10 段寄存器

数据段采用传送类指令将内容装入段寄存器; 对代码段采用跳转指令和调用指令等将内容装入段寄存器。一旦用指令将段选择符装入段寄存器, 处理器硬件便自动将段选择符所指向的段描述符内容装入段描述符高速缓存器中。这样, 以后由段选择符指向的段描述符所描述段的段基地址、段界限及其保护权限信息均从段描述符高速缓存器中获得, 而无须通过系统总线从存储器中的段描述符表中索取, 这就加快了虚拟地址向线性地址的转换速度。

8.2.2 存储器的分页管理

80386/486 支持存储器分页管理机制, 分页机制是存储器管理机制的第二部分。段管理机制实现虚拟地址(由段和偏移构成的逻辑地址)到线性地址的转换, 分页管理机制实现线性地址到物理地址的转换。如果不启用分页管理机制, 那么线性地址就是物理地址。本文将介绍 80386/486 的存储器分页管理机制和线性地址如何转换为物理地址。

1. 存储器分页管理机制

在保护模式下, 即控制寄存器 CR0 的 PE 位为 1, 控制寄存器 CR0 中的最高位 PG 控制分页管理机制是否生效。如果 PG=1, 分页机制生效, 把线性地址转换为物理地址。如果 PG=0, 分页机制无效, 线性地址就直接作为物理地址。必须注意, 只有在保护方式下分页机制才可能生效。只有在保证 PE 位为 1 的前提下, 才能够使 PG 位为 1, 否则将引起通用保护故障。

如图 8.11 所示微处理器两级页表转换过程, 微处理器将线性地址字段中的页目录索引字段、页表索引字段以及偏移量字段转换成物理地址。具体说来, 就是由页目录基地址寄存器 CR3 提供页目录的页框地址(即页目录基地址); 由寻址机构用线性地址中的目录索引字段内容进入页目录内找到页目录项, 得到页表基地址的高 20 位, 页表基地址的低 12 位为全 0; 再用表索引字段的内容作为索引值进入由页目录决定的页表内, 得到被选中页帧基地址的高 20 位, 页帧基地址的低 12 位全为 0, 用页帧基地址的高 20 位与线性地址偏移

字段的 12 位组合就是对由页表说明的页帧内的操作数进行寻址,该地址就是某存储单元的 32 位物理地址。

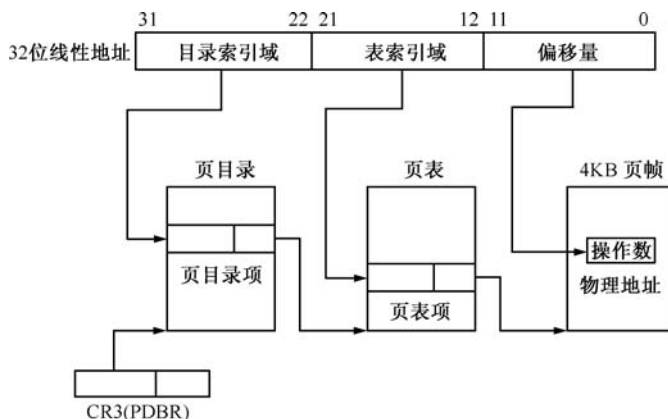


图 8.11 页转换过程

分页管理的特点是所管理的存储器块具有固定的大小,每一块称为一页(也称一页帧、页框架)。80386/486 中规定一页是在现行存储器中连续的 4KB 区域,并且它的起始地址总是安排在低 12 位地址为 0 的线性地址处。分页管理把线性地址空间中的任何一页映射到物理空间的一页。

采用分页管理机制实现线性地址到物理地址转换映射的主要目的是便于实现虚拟存储器。段的大小可变,而页的大小不可改变,是相等的并固定为 4KB。根据程序的逻辑划分段,而根据实现虚拟存储器的方便划分页。

2. 线性地址

图 8.12 示出 80386/486 微处理器使用的 4 字节页的线性地址的格式。分页机制把 32 位的线性地址分成 3 个部分:页目录索引字段、页表索引字段以及偏移量字段。其中有两个 10 位字段(页目录索引字段和页表索引字段)是进入页目录表和页表的索引,而剩下的 12 位的字段(偏移量字段)则是经页表进入页帧地址的偏移量。由于不管是线性地址空间的虚拟页,还是实际存储器中的物理页,都要与 4KB 的页边界对准,所以,这个 32 位线性地址中的最低 12 位偏移地址不必再进行修正便可直接使用。不论分页部件允许分页还是不允许分页,都是将这 12 位偏移地址直接传送给分页部件。另外,由分段部件分的段可以从任何一个字节地址开始,而分页部件则不能从任何一个字节地址开始分页,这也是分段与分页不一样的地方。

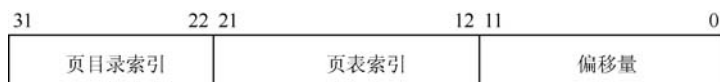


图 8.12 线性地址的格式

3. 页目录项和页表项

分页管理机制在将线性化地址空间转换到物理地址空间的页时,由于每个页面的整个 4KB 是作为一个单位进行映射的,且每个页面都对齐在 4KB 的边界,因而线性地址的低

12 位在分页转换过程中, 将直接作为物理地址的低 12 位使用。分页管理机制中重定位函数(或称转换函数)把 32 位线性地址的高端 20 位当成页表的索引, 如果线性地址空间中的每一页均由随机存储器 RAM 中的唯一的一个页表映像, 在 80386 中共有 2^{20} 个表项, 每项占 4 个字节, 需 4MB 物理空间, 占用的存储空间太大, 所以不能这样做。为节约内存, 分为两级页表机构, 第一级用 1K 个表项, 每项 4 个字节, 占 4KB 内存, 称作目录项; 第二级再用 1K 个表项, 每项 4 个字节, 也占 4KB 内存, 称作页表。这样两级表组合起来即可达到 2^{20} 个表项。页目录(Page Directory)的作用是一个指针, 它指向另一个相似的数据结构。这个相似的数据结构即是称为页表的第二级表项。页表的作用是作为物理存储器中页的指针。两级页表中, 虽然页目录项和页表项的内容略有不同, 但页目录项和页表项的格式都一样(页目录项内缺少脏位)。页目录项和页表项的格式如图 8.13 所示, 页目录中的脏位 D(Dirty)位保留供将来使用。现将页表项中各字段意义分述如下:

页 框 地 址	AVAIL	0	0	D	A	PCD	PWT	U/S	R/W	P
---------	-------	---	---	---	---	-----	-----	-----	-----	---

P: 存在位

R/W: 读/写位

U/S: 用户/监控程序位

PWT: 页透明写位(仅用于80486和Pentium)

PCD: 页Cache禁止位(仅用于80486和Pentium)

A: 访问位

位7: 置“0”表示页大小规模为4KB

位8: 保留

D: 脏位

AVAIL: 系统设计人员可用域

(a) 页目录项格式

31	12	11	9	8	7	6	5	4	3	2	1	0
页 框 地 址	AVAIL	0	0	D	A	PCD	PWT	U/S	R/W	P		

P: 存在位

R/W: 读/写位

U/S: 用户/监控程序位

PWT: 页透明写位(仅用于80486和Pentium)

PCD: 页Cache禁止位(仅用于80486和Pentium)

A: 访问位

D: 脏位

位8和位7: 保留

AVAIL: 系统设计人员可用域

(b) 页表项格式

图 8.13 页目录项和页表项的格式

(1) 页框地址: 人们常把某一个页的起始地址称作页框地址(Page Frame Address)。在一个页表项内, 位 31~位 12 这高 20 位就是用来说明一个页框地址的, 而位 11~位 0 这低端 12 位是来说明页的控制和状态的。在一级页目录内, 页框地址就是指向对应页表起始地址; 而在二级页表内, 页框地址则是指向含有指令或数据的那个存储器页帧的页起始地址。

(2) 存在位 P: $P=1$, 表示页表项中的页框地址已经映射到物理存储器中的一个页内; $P=0$, 表示该页表项中的页框地址没有映射到物理存储器中, 也就是说该页表项所指页不在物理存储器中。这时, 若用该项进行地址转换, 将产生一个页出错异常, 并且操作系统中的页出错处理程序把该项重新装入存储器。如果两级页表内的任何一级页表中存在位被

置成 0，而这时又希望用页表项进行转换，就会产生一个页故障(Page Fault)异常事故。在对请求分页虚拟存储给以全面支持的系统内，会出现如下一系列操作：

- ① 操作系统将这个页的信息从磁盘存储器拷贝到物理存储器。
- ② 操作系统将页框地址装到页表项内，同时将它的存在位置成 1，页表项中的其他各位，如脏位、读/写位等，也可以将它们置成 1。
- ③ 由于在 TLB 中，可能仍然滞留着一个旧的页表项的副本，所以操作系统应将其清空，再重新启动引起异常事故的那个程序。如果在页目录中，某项的 P 位置 0，则表示对应的页表已被移出存储器。可以说操作系统利用 P 位提供的信息实现了请求分页的虚拟存储器的能力。
- (3) 读/写位 R/W：R/W=1 写，否则为读。用来对页进行保护校验的，它不涉及到地址转换。
- (4) 用户/监控程序位 U/S：用来对页进行保护校验的，它不涉及到地址转换。在进行地址转换而处理机又执行其他操作时，就必须进行保护校验。
- (5) 访问位 A：用来表明该项指出的页是否已被读或写。若目录项中 A=1，则表示该项所指出的页表已被访问过；若页表项中 A=0，则表示该页表项所指出存储器中的页未被访问过。总之，A 位的置位由处理器完成，A 位的状态可供操作系统软件测试，以便计算不同页的使用频率。
- (6) 脏位 D：只在页表项中设置，而不在页目录项中设置。当页表项中 D=1 时，表明该项所指出的存储器的页已被写。D 位的状态可被操作系统软件进行测试，以便判断存储器的某页在它最后一次被复制到磁盘后是否被修改过。
- (7) 可用域 AVL：该域共 3 位，供系统软件设计人员使用。可将与页使用有关的信息放在该域中，帮助分析判断应把哪些页移出存储器。
- (8) 页级 Cache 控制位 PWT 和 PCD：页透明写位 PWT 和页 Cache 禁止位 PCD 是用来对页级 Cache 实施管理的位。

4. 页转换高速缓冲存储器(又称旁视缓冲器 TLB)

在 80386 和 80486 中，若允许分页管理，则将线性地址转换为物理地址时，需要查询内存中的页目录项和页表项，并且当要访问的页不在内存时，还要将该页从磁盘调入内存，这就意味着页转换处理需要占用 CPU 较多的时间。为了提高页转换处理的速度，80386 和 80486 CPU 内部都设计了页转换高速缓冲存储器 Cache。可以把最近存储器使用的 32 个页表项的入口地址存入页转换高速缓冲存储器 Cache 中，以后进行地址转换时就先访问页转换高速缓冲存储器 Cache，32 个页与对应页的 4KB 联系，这样就覆盖了 128KB(32*4KB=128KB)的存储空间。根据访问存储器的局部性原理，对于一般的多任务系统来说，页转换高速缓冲存储器 Cache 的命中率会很高，具有大约 98% 的命中率。也就是说，在处理器访问存储器过程中，只有 2% 必须访问两级分页机构，这样就大大地提高了地址转换速度。

8.3 保护机制

在保护模式下，80386/486 微处理器提供了硬件保护机制，保护机制是基于不同访问权

限的特权级来实现的。保护是多任务或多用户系统中加强资源管理的方法，不仅在同一任务内部进行保护，而且在多个程序或多个任务间还用来防止彼此干扰，同时，保护机制还用来避免对存储器非法操作，每次存储器访问都必须进行段级别的保护检查，且在 80386 和 80486 允许分页的情况下，还要进行页级别保护检查。

8.3.1 段级别保护

在保护模式下，80386/486 微处理器的段级别保护分为 4 级保护，如图 8.14 所示。每个任务是分离的，其中特权级 0 是最高可靠性的特权级；特权级 3 是最低可靠性的特权级。

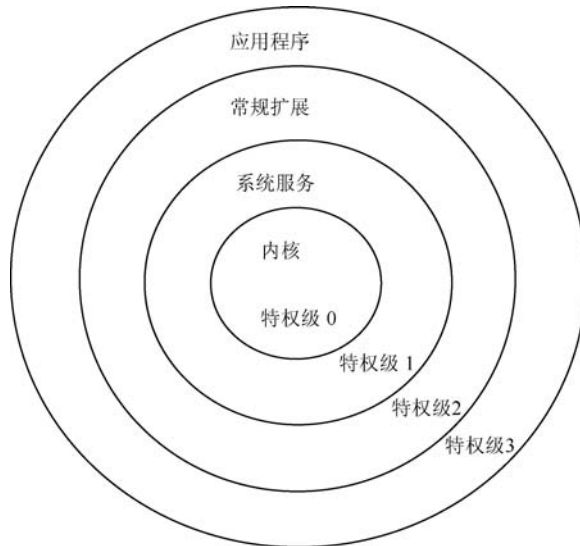


图 8.14 段级别保护

在特权级 0 中，通常配置操作系统内核。主要包括存储器管理及任务和 I/O 设备间的通信等。这些系统服务在内核中，使用特权指令进行与 CPU 密切相关的那些处理。

在特权级 1 中，进行不属于内核的系统标准服务的程序处理，包括文件共享、显示管理及数据通信等。在特权级 2 中，通常配置对操作系统进行扩充的用户化的程序，这些是依赖于内核的服务，例如，数据库管理程序或逻辑文件访问系统等。通常计算机系统的设备驱动程序多驻留在特权级 2 或特权级 1 上。

特权级 3 分给用户应用程序，该级别上各应用程序之间及它们与操作系统之间都应该互相隔离，互不影响。但各应用程序的执行必须依赖于更高级的系统软件。

4 级特权级不是都必须使用的，究竟采用几个特权级别应视实际操作系统不同而不同。但是，为了使系统可以执行特权指令，通常特权级 0 是必须使用的。四环模型为设计非常复杂的操作系统提供了条件。对一般用户来说，在编写应用程序时，所选择的特权级别很可能已被操作系统设计者预先决定了。

8.3.2 页级别保护

80486 微处理机的控制寄存器 CR0 的位 31(PC 位)是允许分页控制位，若将控制寄存器 CR0 的位 31 置“1”，则表示存储器管理机构允许分页。这一位通常是在软件初始化阶段

由操作系统设置, 存储器管理机构允许实现分页管理和保护。页级别保护主要有两种保护检查: 可寻址域的限制和类型检查。保护性检查的项目放在页目录项及页表项中。每一次对存储器的访问都要实施检查, 以验证它是否满足保护检查。所有的检查在存储器周期开始前完成, 任何违反保护规则的企图都将阻止存储器周期启动并且导致异常。由于检查与地址转换是并行执行的, 所以并没有性能上的损失问题。

8.4 虚拟的 8086 模式

当系统加电、复位(RESET)后, CPU 自动进入实地址模式。实地址模式的主要目的在于对系统进行初始化, 以便系统转向保护模式或虚拟 8086 模式。在实地址模式下, CPU 只能工作在一种模式下。

虚拟 8086 模式实际上是运行在保护模式下(控制寄存器 CR0 中设置标志位 PE 置位时)的一种特定的模式, 当 EFLAGS 寄存器中设置标志位 VM 位被置位时, 将使处理器运行在虚拟 8086 方式, 简称 V86 方式。在虚拟 8086 模式下, CPU 将段寄存器的内容(16 位)乘 16 直接放到段描述符寄存器的线性地址中; 段界限将装入 0FFFFH; 特权级 DPL=3, 其他属性不再说明。因此, 虚拟 8086 不需要描述符表来描述各段的属性。当操作系统或执行程序切换至一个虚拟 8086 模式任务时, 处理器仿真一个 8086 处理器。虚拟 8086 模式是为了运行 16 位的 8086 程序而设置的, 它可以在保护模式和虚拟 8086 模式间重复而迅速地相互切换。有了虚拟 8086 模式, 就可以使 Pentium、80486、80386 程序与 8086、80186、80286 的大量的 16 位软件并行运行。在虚拟 8086 模式下, 各个任务可以运行在不同的操作系统下。

有关虚拟 8086 模式的更详细的资料请参阅相关资料。

8.5 小 结

80386、80486 CPU 共有三种工作模式: 实模式存储器管理、保护虚地址方式存储器管理和虚拟 8086 模式。存储器管理是由微处理器提供的对系统存储器资源进行管理的机制, 其目的是方便软件程序对存储器的应用。本章从应用角度出发, 让学生理解分段存储管理技术和分页存储管理技术的实现思想, 掌握从逻辑地址到物理地址的转换过程, 熟悉保护机制的段级别保护检查、页级别的保护检查以及保护方式下的多任务切换。

8.6 习 题

1. 在保护方式下, 80486 微型处理器中的地址寄存器是_____位, 80486 微型处理器可以访问_____字节实地址存储器空间。

2. 80486 微处理机配备的段寄存器(也称段选择符)是_____位寄存器。它的作用是从两个描述符表中检索出其中一个段描述符, 而从段描述符中得到预被访问段的

____、____、____等信息。

3. 80486 微处理器实现的是分____存储管理和分____存储管理。

4. 80486 微处理器段的转换过程中，是先将逻辑地址转换成线性地址，再将线性地址转换成物理地址。而由系统给出的地址及程序给出的地址都是____，它是由一个____位的段选择符和一个只能在本段使用的____位偏移量组成。

5. 控制寄存器 CR0 中控制位____控制 80486 微处理器工作在保护模式下。

6. 处理器在任务切换时，要用到____描述符、____描述符和____寄存器。

7. 段描述符和系统描述符有什么区别？

8. 在 GDT 中存放的是什么信息？在 LDT 中存放的是什么信息？GDT 和 LDT 有什么区别？

9. 如果允许分页，80486 微处理机的每一页有多大容量？80486 微处理机的地址空间可映射多少页？

10. 段描述是由多少个字节组成的？怎样区分代码段、数据段和系统段？

11. CPL 表示什么？RPL 又表示什么？EPL 与 CPL、RPL 有什么关系？

12. GDTR 和 LDTR 分别表示什么意思？其内有什么消息？

13. 如果应用程序运行在特权级 3 级，它能使用哪一级的操作系统软件？

14. 在页转换时，页目录、页表、页框与 32 位线性地址中的哪些字段相对应？

15. 转换旁视缓冲存储器 TLB 的作用是什么？

16. 如果在全局描述符表寄存器 GDTR 中的段界限值为 0FFFH，基地址的值为 00200000H，试问：全局描述符表的起始地址是多少？全局描述符表的大小为多少个字节？它的结束地址是多少？

第9章 中断技术

教学提示：本章主要介绍中断和中断源，中断处理过程，8086 的中断系统以及典型的可编程中断控制器 8259A 的结构、功能和应用。

教学要求：通过本章的学习，学生应该掌握中断和中断源，中断处理过程，8086 的中断系统以及典型的可编程中断控制器 8259A 的结构、功能和应用。

9.1 中断概述

通常，处理器的运算速度相当快，外部设备的运算速度相对较慢，快速的 CPU 与慢速的外部设备在传输数据的速率上存在矛盾。为了提高输入/输出数据的吞吐率，加快运算速度，便产生了中断技术。

9.1.1 中断的概念

中断是 CPU 在执行当前程序的过程中，当出现某些异常事件或某种外部请求时，使得 CPU 暂时停止正在执行的程序(即中断)，转去执行外围设备服务的程序。当外围设备服务的程序执行完后，CPU 再返回暂时停止正在执行的程序处(即断点)，继续执行原来的程序。这种中断就是人们通常所说的外部中断；但是随着计算机体系结构不断的更新换代和应用技术的日益提高，中断技术发展的速度也是非常迅速的，中断的概念也随之延伸，中断的应用范围也随之扩大。除了传统的外围部件引起的硬件中断外，又出现了内部的软件中断概念。

在 Pentium 中则更进一步丰富了软件中断的种类，延伸了中断的内涵。它把许多在执行指令过程中产生的错误也归并到了中断处理的范畴，并将它们和通常意义上的内部软件中断一起统称其为异常，而将传统的外部中断简称为中断。由此可见，中断和异常对于 Pentium 微处理机来说是有区别的，其主要差别在于：中断用来处理 CPU 以外的异常事件，而异常则是用来处理在执行指令期间，由 CPU 本身对检测出来的某些异常事情做出的响应。当再次执行产生异常的程序或数据时，这种异常总是可以再次出现。而由外围部件引起的硬件中断，一般说来与当前的执行程序无关；但是，当中断和异常在使微处理机暂时停止执行当前的程序，去执行更高优先级别的程序时，却是一样的。

外部中断和内部软件中断就构成了一个完整的中断系统。发出中断请求的来源非常多，不管是由外部事件而引起的外部中断，还是由软件执行过程而引发的内部软件中断，凡是能够提出中断请求的设备或异常故障，均被称为中断源。

9.1.2 中断源

引起中断的原因或发出中断请求的来源，称为中断源。中断源有以下几种：

(1) 外设中断源。一般有键盘、打印机、磁盘、磁带等，工作中要求 CPU 为它服务时，

会向 CPU 发送中断请求。

(2) 故障中断源。当系统出现某些故障时(如存储器出错、运算溢出等),相关部件会向 CPU 发出中断请求,以便使 CPU 转去执行故障处理程序来解决故障。

(3) 软件中断源。在程序中向 CPU 发出中断指令(8086 为 INT 指令),可迫使 CPU 转去执行某个特定的中断服务程序,而中断服务程序执行完后, CPU 又回到原程序中继续执行 INT 指令后面的指令。

(4) 为调试而设置的中断源。系统提供的单步中断和断点中断,可以使被调试程序在执行一条指令或执行到某个特定位置处时,自动产生中断,从而便于程序员检查中间结果,寻找错误所在。

9.1.3 中断类型

根据中断源是来自 CPU 内部还是外部,通常将所有中断源分为两类:外部中断源和内部中断源,对应的中断称为外部中断或内部中断。

1. 外部中断源和外部中断

外部中断源即硬件中断源,来自 CPU 外部。8086 CPU 提供了两个引脚来接收外部中断源的中断请求信号:可屏蔽中断请求引脚和不可屏蔽中断请求引脚。

通过可屏蔽中断请求引脚输入的中断请求信号称作可屏蔽中断请求,对这种中断请求 CPU 可响应,也可不响应,具体取决于标志寄存器中 IF 标志位的状态。通过不可屏蔽中断请求引脚输入的中断请求信号称作不可屏蔽中断请求,这种中断请求 CPU 必须响应。

2. 内部中断源和内部中断

内部中断源是来自 CPU 内部的中断事件,这些事件都是特定事件,一旦发生, CPU 即调用预定的中断服务程序去处理。内部中断主要有以下几种情况:

1) 除法错误

当执行除法指令时,如果除数为 0 或商数超过了最大值, CPU 会自动产生类型为 0 的除法错误中断。

2) 软件中断

执行软件中断指令时,会产生软件中断。8086 系统中,设置了三条中断指令,分别是:

(1) 中断指令 INT n: 用户可以用 INT n 指令来产生一个类型为 n 的中断,以便让 CPU 执行 n 号中断的中断服务程序。

(2) 断点中断 INT 3: 执行断点指令 INT 3,将引起类型为 3 的断点中断,这是调试程序专用的中断。

(3) 溢出中断 INTO: 如果标志寄存器中溢出标志位 OF 为 1,在执行了 INTO 指令后,产生类型为 4 的溢出中断。

3) 单步中断

当标志寄存器的标志位 TF 置 1 时,8086 CPU 处于单步工作方式。CPU 每执行完一条指令,自动产生类型为 1 的单步中断,直到将 TF 置 0 为止。

单步中断和断点中断一般仅在调试程序时使用。调试程序通过为系统提供这两种中断的中断服务程序的方式,在发生断点或单步中断后获得 CPU 控制权,从而可以检查被调试

程序(中断前 CPU 运行的程序)之状态。

为了解决多个中断同时申请时响应的先后顺序问题, 系统将所有的中断划分为四级, 以 0 级为最高, 依次降低, 各级情况如下:

- (1) 0 级——除单步中断以外的所有内部中断。
- (2) 1 级——不可屏蔽中断。
- (3) 2 级——可屏蔽中断。
- (4) 3 级——单步中断。

不同级别的中断同时申请时, CPU 根据级别高低依次决定响应顺序。

9.1.4 中断类型号

由于系统中存在许多中断源, 当中断发生时, CPU 就要进行中断源的判断。只有知道了中断源, CPU 才能调用相应的中断服务程序来为其服务。为了标记中断源, 人们给系统中的每个中断源指定了一个唯一的编号, 称为中断类型号。CPU 对中断源的识别就是获取当前中断源的中断类型号, 在 8086 系统中的实现如下所述:

(1) 可屏蔽中断(硬件中断): 所有通过可屏蔽中断请求引脚向 CPU 发送的中断请求, 都必须由中断控制器 8259A 管理。CPU 在准备响应其中断请求时, 会给 8259A 发一个中断响应信号, 8259A 收到这一信号后, 会将发出中断申请外设的中断类型号通过系统数据总线发送给 CPU。

(2) 软件中断: 在中断指令 `INT n` 中, 参数 `n` 即为中断类型号。

(3) 除上面两种情况外, 其余中断都是固定类型号, 主要是内部中断: 如除法错(类型 0)、单步中断(类型 1)、断点中断 `INT 3`(类型 3)、溢出中断 `INTO`(类型 4)等。外部中断中不可屏蔽中断的类型号也是固定的(类型 2)。

8086/8088 系统中, 中断类型号范围为 0~FFH, 即最多有 256 个中断源。

9.1.5 中断矢量表

CPU 在响应中断时, 要执行该中断源对应的中断服务程序, 那么 CPU 如何知道这段程序在哪儿呢? 答案是 CPU 通过查找中断矢量表来得知。中断服务程序的地址叫做中断矢量, 将全部中断矢量集中在一张表中, 即中断矢量表。中断矢量表的位置固定在内存的最低 1K 字节中, 即 00000H~003FFH 处。这张表中存放着所有中断服务程序的入口地址, 而且根据中断类型号从小到大依次排列, 每一个中断服务程序的入口地址在表中占 4 字节: 前两个字节为偏移量, 后两个字节为段基址。因系统中共有 256 个中断源, 而每个中断服务程序入口地址又占 4 字节, 故中断矢量表共占 $256 \times 4 = 1K$ 个字节, 如图 9.1 所示。

那么, 在系统中, 实际上由谁来提供中断服务程序, 并填写中断矢量表中的内容呢? 主要是 ROM BIOS 和 DOS。它们填写了中断矢量表的大部分项目并提供了相应的中断服务程序。此外, 主板上的各种硬件插卡(如果它们向系统提供中断服务)及在 DOS 下运行的以中断方式工作的内存驻留程序(如鼠标驱动程序、后台打印程序等)也会填写部分中断矢量表项目并提供相应的中断服务程序。最后, 还有部分中断矢量表项目无人填写, 也无人提供对应的中断服务程序, 这部分中断是保留给用户用的。

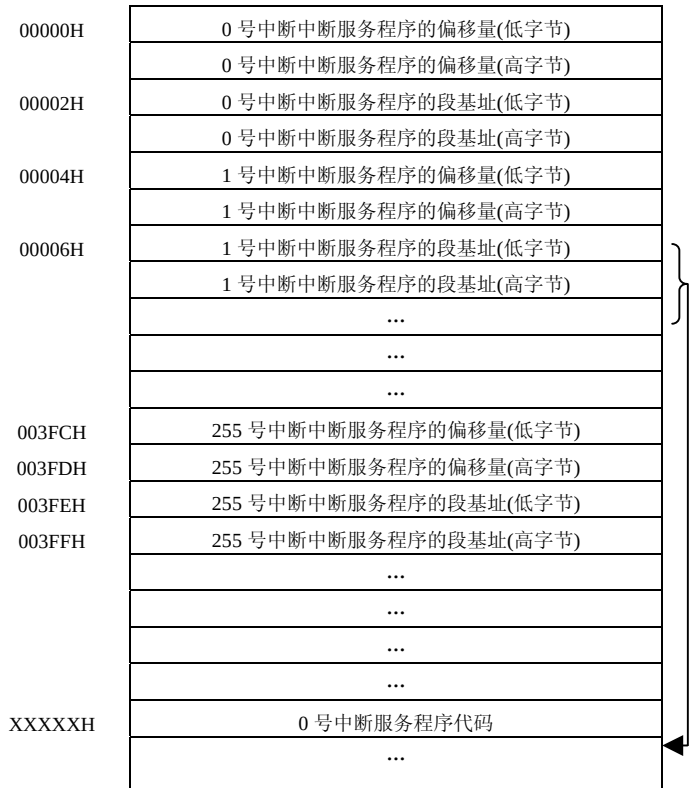


图 9.1 中断矢量表结构

9.1.6 中断优先级

在实际系统中，常常遇到多个中断源同时请求中断的情况，这时 CPU 必须确定首先为哪一个中断源服务，以及服务的次序。解决的方法是用中断优先排队的处理方法。即根据中断源要求的轻重缓急，排好中断处理的优先次序，即优先级(Priority)，又称优先权。先响应优先级最高的中断请求。有的微处理器有两条或更多的中断请求线，而且已经安排好中断的优先级，但有的微处理器只有一条中断请求线。凡是遇到中断源的数目多于 CPU 的中断请求线的情况时，就需要采取适当的方法来解决中断优先级的问题。

另外，当 CPU 正在处理中断时，也要能响应优先级更高的中断请求，而屏蔽掉同级或较低级的中断请求即所谓多重中断的问题。

通常，解决中断的优先级的方法有以下几种：

- (1) 软件查询确定中断优先级。
- (2) 硬件查询确定优先级。
- (3) 中断优先级编码电路。

9.1.7 中断的嵌套

当 CPU 执行优先级较低的中断服务程序时，允许响应优先级比它高的中断源请求中断，而挂起正在处理的中断，这就是中断嵌套或称多重中断。此时，CPU 将暂时中断正在

进行着的级别较低的中断服务程序，优先为级别高的中断服务，待优先级高的中断服务结束后，再返回到刚才被中断的较低优先级的那一级，继续为它进行中断服务。

多重中断流程的编排与单级中断的区别有以下几点：

(1) 加入屏蔽本级和较低级中断请求的环节。这是为了防止在进行中断处理时，不致受到来自本级和较低级中断的干扰，并允许优先级比它高的中断源进行中断。

(2) 在进行中断服务之前，要开中断。因为如果中断仍然处于禁止状态，则将阻碍较高级中断的中断请求和响应，所以必须在保护现场、屏蔽本级及较低级中断完成之后，开中断，以便允许进行中断嵌套。

(3) 中断服务程序结束之后，为了使恢复现场过程不致受到任何中断请求的干扰，必须安排并执行关中断指令，将中断关闭，才能恢复现场。

(4) 恢复现场后，应该安排并执行开中断指令，重新开中断，以便允许任何其他等待着的中断请求有可能被 CPU 响应。应当指出，只有在执行了紧跟在开中断指令后面的一条指令以后，CPU 才重新开中断。一般紧跟在开中断指令后的是返回指令 RET，它将把原来被中断的服务程序的断点地址弹回 IP 及 CS，然后 CPU 才能开中断，响应新的中断请求。

多个中断源、单一中断请求线的中断处理过程的流程图如图 9.2 所示。

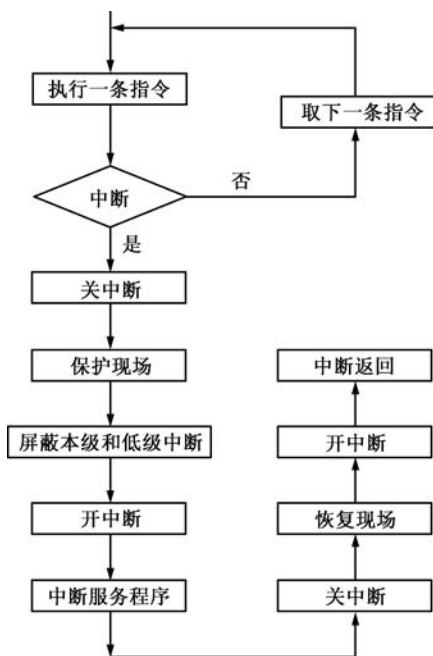


图 9.2 多个中断源、单一中断请求线的流程图

9.2 8086 CPU 的中断处理过程

虽然不同类型的计算机系统中中断系统有所不同，但实现中断的过程是相同的。中断的处理过程一般有以下几步：中断请求、中断响应、中断处理、中断返回。

9.2.1 中断请求

中断处理的第一步是中断源发出中断请求，这一过程随中断源类型的不同而出现不同的特点，具体如下：

1. 外部中断源的中断请求

当外部设备要求 CPU 为它服务时，需要发一个中断请求信号给 CPU 进行中断请求。

8086 CPU 有两根外部中断请求引脚 INTR 和 NMI 供外设向其发送中断请求信号用，这两根引脚的区别在于 CPU 响应中断的条件不同。

CPU 在执行完每条指令后，都要检测中断请求输入引脚，看是否有外设的中断请求信号。根据优先级，CPU 先检查 NMI 引脚再检查 INTR 引脚。

INTR 引脚上的中断请求称为可屏蔽中断请求，CPU 是否响应这种请求取决于标志寄存器的 IF 标志位的值。IF=1 为允许中断，CPU 可以响应 INTR 上的中断请求；IF=0 为禁止中断，CPU 将不理睬 INTR 上的中断请求。

由于外部中断源有很多，而 CPU 的可屏蔽中断请求引脚只有一根，这又产生了如何使得多个中断源合理共用一根中断请求引脚的问题。解决这个问题的方法是引入 8259A 中断控制器，由它先对多路外部中断请求进行排队，根据预先设定的优先级决定在有中断请求冲突时，允许哪一个中断源向 CPU 发送中断请求。

NMI 引脚上的中断请求称为不可屏蔽中断请求(或非屏蔽中断请求)，这种中断请求 CPU 必须响应，它不能被 IF 标志位所禁止。不可屏蔽中断请求通常用于处理应急事件。在 PC 系列机中，RAM 奇偶校验错、I/O 通道校验错和协处理器 8087 运算错等都能够产生不可屏蔽中断请求。

2. 内部中断源的中断请求

CPU 的中断源除了外部硬件中断源外，还有内部中断源。内部中断请求不需要使用 CPU 的引脚，它由 CPU 在下列两种情况下自动触发：其一是在系统运行程序时，内部某些特殊事件发生(如除数为 0，运算溢出或单步跟踪及断点设置等)；其二是 CPU 执行了软件中断指令 INT n。所有的内部中断都是不可屏蔽的，即 CPU 总是响应(不受 IF 限制)。

8086 的中断系统结构如图 9.3 所示。

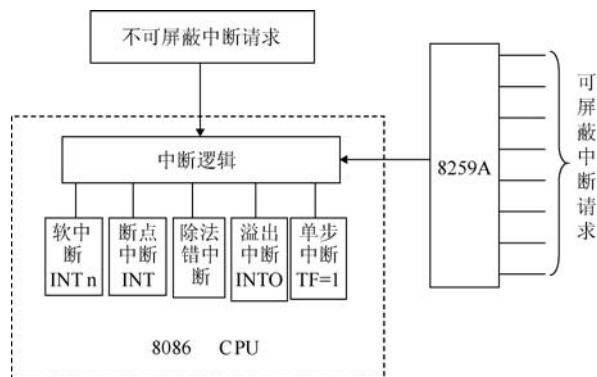


图 9.3 8086 的中断结构

9.2.2 中断响应

CPU 接受了中断源的中断请求后,便进入了中断处理的第二步:中断响应。这一过程也随中断源类型的不同而出现不同的特点,具体如下:

1. 可屏蔽外部中断请求的中断响应

可屏蔽外部中断请求中断响应的特点是:(1) 由于外设(实际上是中断控制器 8259A,本处为求简单,统称为外设)不知道自己的中断请求能否被响应,所以 CPU 必须发信号(用 $\overline{\text{INTA}}$ 引脚)通知其中断请求已被响应。(2) 由于多个外设共用一根可屏蔽中断请求引脚,CPU 必须从中断控制器处取得中断请求外设的标识——中断类型号。

当 CPU 检测到外设有中断请求(即 INTR 为高电平)时,CPU 又处于允许中断状态,则 CPU 就进入中断响应周期。在中断响应周期中,CPU 自动完成如下操作:

(1) 连续发出两个中断响应信号 $\overline{\text{INTA}}$,完成一个中断响应周期。

(2) 关中断,即将 IF 标志位置 0,以避免在中断过程中或进入中断服务程序后,再次被其他可屏蔽中断源中断。

(3) 保护处理机的现行状态,即保护现场。包括将断点地址(即下条要取出指令的段基址和偏移量,在 CS 和 IP 内)及标志寄存器 FLAGS 内容压入堆栈。

(4) 在中断响应周期的第二个总线周期中,中断控制器已将发出中断请求外设的中断类型号送到了系统数据总线上,CPU 读取此中断类型号,并根据此中断类型号查找中断矢量表,找到中断服务程序的入口地址,将入口地址中的段基址及偏移量分别装入 CS 及 IP ,一旦装入完毕,中断服务程序就开始执行。

2. 不可屏蔽外部中断请求的中断响应

NMI 上中断请求的响应过程要简单一些。只要 NMI 上有中断请求信号(由低向高的正跳变,两个以上时钟周期),CPU 就会自动产生类型号为 2 的中断,并准备转入相应的中断服务程序。与可屏蔽中断请求的响应过程相比,它省略了第 1 步及第 4 步中的从数据线上读中断类型号,其余步骤相同。

NMI 上中断请求的优先级比 INTR 上中断请求的优先级高,故这两个引脚上同时有中断请求时,CPU 先响应 NMI 上的中断请求。

3. 内部中断的中断响应

内部中断是由 CPU 内部特定事件或程序中使用 INT 指令触发,若由事件触发,则中断类型号是固定的;若由 INT 指令触发,则 INT 指令后的参数即为中断类型号。故中断发生时 CPU 已得到中断类型号,从而准备转入相应中断服务程序中去。除不用检测 NMI 引脚外,其余与不可屏蔽外部中断请求的中断响应相同。

9.2.3 中断处理

中断处理的过程就是 CPU 运行中断服务程序的过程,这一步骤对所有中断源都一样。所谓中断服务程序,就是为实现中断源所期望达到的功能而编写的处理程序。中断服务程

序一般由 4 部分组成：保护现场、中断服务、恢复现场、中断返回。所谓保护现场，是因为有些寄存器可能在主程序被打断时存放有用的内容，为了保证返回后不破坏主程序在断点处的状态，应将有关寄存器的内容压入堆栈保存。中断服务部分是整个中断服务程序的核心，其代码完成与外设的数据交换。恢复现场是指中断服务程序完成后，把原先压入堆栈的寄存器内容再弹回到 CPU 相应的寄存器中。有了保护现场和恢复现场的操作，就可保证在返回断点后，正确无误地继续执行原先被打断的程序。中断服务程序的最后部分是一条中断返回指令 IRET。

9.2.4 中断返回

在中断服务程序的最后，应安排一条中断返回指令 IRET。该指令完成如下功能：

- (1) 从栈顶弹出一个字→IP。
- (2) 再从栈顶弹出一个字→CS。
- (3) 再从栈顶弹出一个字→FLAGS。

IRET 指令执行完后，CS、IP 恢复为原中断前的值，CPU 从断点处继续执行原程序。

从上述过程可以看出，各类中断源的中断过程基本相同，以可屏蔽中断的过程最为复杂，下面就对可屏蔽中断的响应和处理过程再总结一下，如图 9.4 所示。

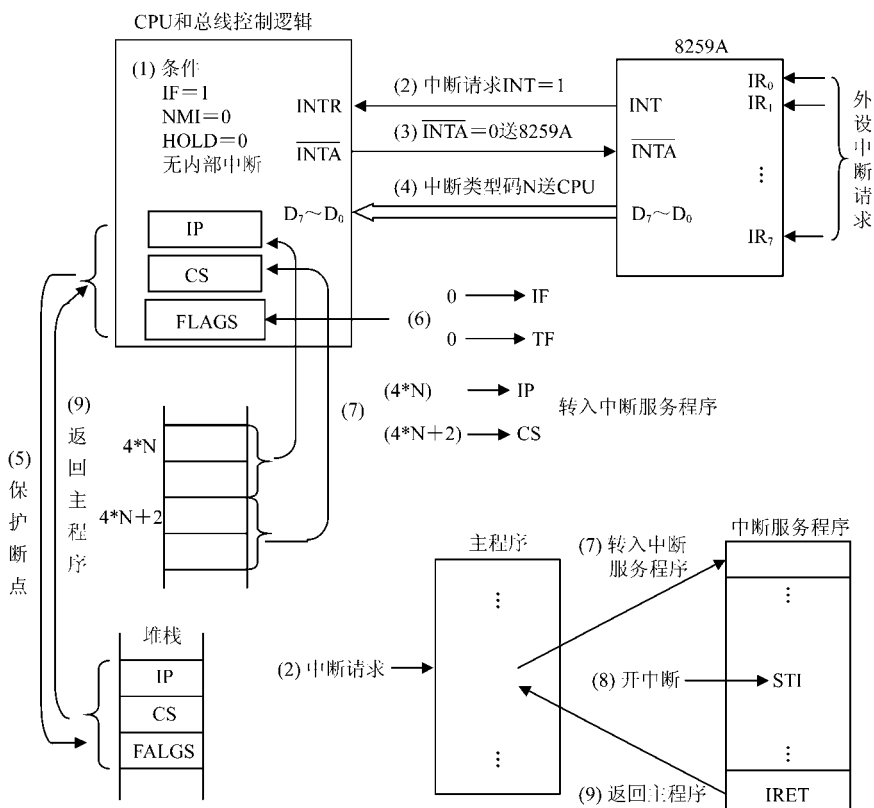


图 9.4 可屏蔽中断的响应和处理过程

说明:

- (1) CPU 要响应可屏蔽中断请求, 必须满足: 中断允许标志位置 1 ($IF = 1$), 没有内部中断, 没有不可屏蔽中断请求 ($NMI = 0$), 没有总线请求 ($HOLD = 0$)。
- (2) 外设通过中断控制器 8259A 向 CPU 发出中断请求。
- (3) CPU 执行完当前指令后, 向 8259A 发出中断响应信号 ($\overline{INTA} = 0$), 表明 CPU 即将响应该可屏蔽中断请求。
- (4) CPU 再发第二个 $\overline{INTA}$ 负脉冲, 8259A 在第二个 $\overline{INTA}$ 负脉冲期间, 通过数据总线将中断类型码送 CPU。
- (5) 断点保护, 将标志寄存器、CS、IP 内容依次压入堆栈。
- (6) 清除 IF 及 TF 标志位(即置 $IF = 0$, $TF = 0$), 在中断响应期间, 默认禁止再响应可屏蔽中断或单步中断。
- (7) 根据中断类型号 n , 从中断矢量表中获得相应中断服务程序的入口地址(段内偏移地址和段基址), 并将其分别置入 IP 及 CS 中。其后 CPU 转入中断服务程序执行。
- (8) 中断服务程序一般包括保护现场、中断服务、恢复现场等部分。为了能够处理更高级中断, 还可在中断服务程序中用 STI 指令开中断。
- (9) 中断服务程序执行完毕, 最后执行一条中断返回指令 IRET, 将中断前压入堆栈保存的标志寄存器内容及断点地址恢复到 FLAGS、CS、IP 中, CPU 即从断点处恢复执行原程序。

9.3 可编程中断控制器 8259A

由于 8086 CPU 可屏蔽中断请求引脚只有一条, 而外部硬件中断源有多个, 为了使多个外部中断源能共享这一条中断请求引脚, 必须解决如下几个问题: (1)解决多个外部中断请求信号与 INTR 引脚的连接问题。(2)CPU 如何识别是哪一个中断源发送的中断请求问题。(3) 由于一次只能有一个外设发送中断请求, 当多个中断源同时申请中断时, 如何确定请求发送顺序问题。中断控制器 8259A 就是为这个目的而设计的, 它一端与多个外设的中断请求信号相连接, 一端与 CPU 的 INTR 引脚相连接, 所有外设的可屏蔽中断请求都受其管理, 通过编程可设置各中断源的优先级、中断矢量码等信息。

8259A 能与 8080/8085、8086/8088 等多种微处理器芯片组成中断控制系统。它有 8 个外部中断请求输入引脚, 可直接管理 8 级中断。若系统中中断源多于 8 个, 8259A 还可以实行两级级联工作, 最多可用 9 片 8259A 级联管理 64 级中断。

9.3.1 8259A 的结构与引脚

1. 8259A 的内部结构

8259A 的功能比较多, 控制字也比较复杂, 这给初学者学习带来了一定的难度, 为彻底掌握 8259A 的一些编程概念, 有必要先对 8259A 的内部结构及其工作原理作一了解。8259A 内部结构如图 9.5 所示, 由 8 个部分组成。

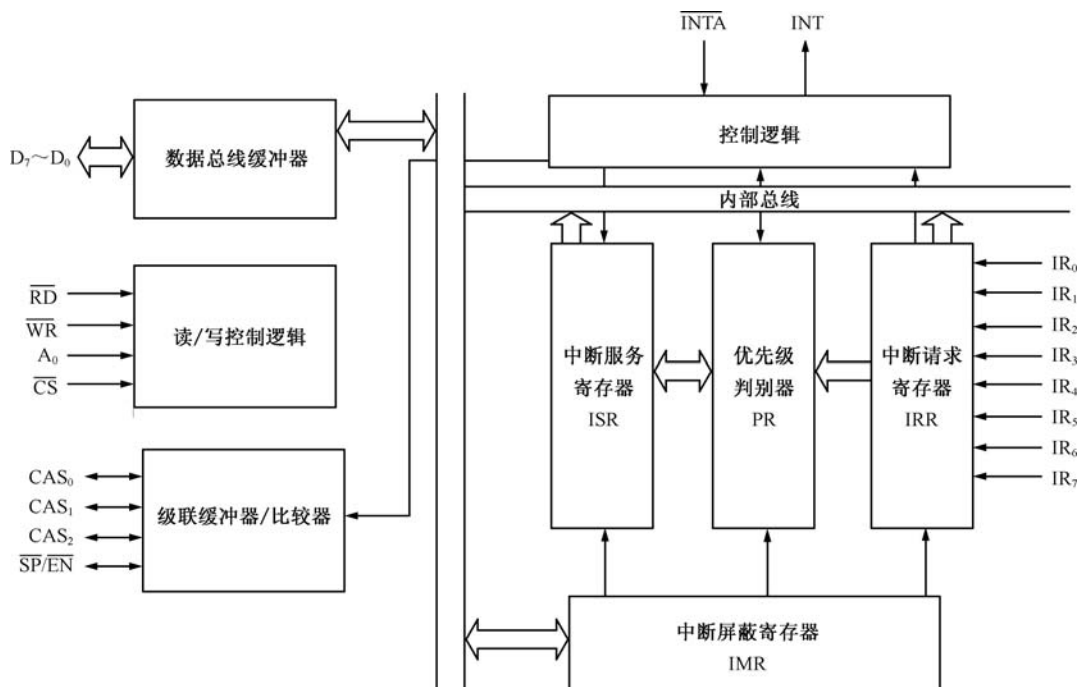


图 9.5 8259A 内部结构

1) 数据总线缓冲器

这是一个 8 位双向三态缓冲器，是 8259A 与系统数据总线的接口。8259A 通过数据总线缓冲器接收微处理器发来的各种命令控制字、有关寄存器状态的读取，8259A 也通过数据总线缓冲器向微处理器送出中断类型码等。

2) 读/写控制逻辑

该部件接收来自 CPU 的读/写命令，配合片选信号 $\overline{CS}$ 、读信号 $\overline{RD}$ 、写信号 $\overline{WR}$ 和地址线 A_0 共同实现控制，完成规定的操作。

3) 级联缓冲器/比较器

8259A 既可工作于单片方式，也可工作于多片级联方式。这个部件在级联方式下用于标识主从设备，在缓冲方式下控制收发器的数据传送方向。

4) 中断请求寄存器 IRR

该寄存器是一个 8 位寄存器，用来锁存外部设备送来的 $IR_7 \sim IR_0$ 中断请求信号。每位对应着 8259A 的 8 个外部中断请求输入端 $IR_7 \sim IR_0$ 中的一位，当 $IR_7 \sim IR_0$ 中某引脚上有中断请求信号时，IRR 对应位置 1，当该中断请求被响应时，该位复位。

5) 中断屏蔽寄存器 IMR

该寄存器是一个 8 位寄存器，用于设置中断请求的屏蔽信号。每位对应着 8259A 的 8 个外部中断请求输入端 $IR_7 \sim IR_0$ 中的一位。如果用软件将 IMR 的某位置“1”，则其对应引脚上的中断请求将被 8259A 屏蔽，即使对应 IR_i 引脚上有中断请求信号输入也不会在 8259A 上产生中断请求输出；反之，若屏蔽位置“0”，则不屏蔽，即产生中断请求。各个屏蔽位是相互独立的，某位被置 1 不会影响其他未被屏蔽引脚的中断请求工作。

6) 中断服务状态寄存器 ISR

该寄存器是一个 8 位寄存器, 用于记录当前正在被服务的所有中断级, 包括尚未服务完而中途被更高优先级打断的中断级。每位对应着 8259A 的 8 个外部中断请求输入端 $IR_7 \sim IR_0$ 中的一位。若某个引脚上的中断请求被响应, 则 ISR 中对应位被置 1, 以示这一中断源正在被服务。这一位何时被置 0 取决于中断结束方式, 见后述。例如, 若 IRR 的 IR_2 获得中断请求允许, 则 ISR 中的 D_2 位置位, 表明 IR_2 正处于被服务之中。ISR 的置位也允许嵌套, 即如果已有 ISR 的某位置位, 但 IRR 中又送来优先级更高的中断请求, 经判优后, 相应的 ISR 位仍可置位, 形成多重中断。

7) 优先权分析器 PR

优先权分析器用于识别和管理各中断请求信号的优先级别。当在 IR 输入端有几个中断请求信号同时出现时, 通过 IRR 送到 PR (只有 IRR 中置 1 且 IMR 中对应位置 0 的位才能进入 PR)。PR 检查中断服务寄存器 ISR 的状态, 判别有无优先级更高的中断正在被服务, 若无, 则将中断请求寄存器 IRR 中优先级最高的中断请求送入中断服务寄存器 ISR, 并通过控制逻辑向 CPU 发出中断请求信号 INT, 并且将 ISR 中的相应位置“1”, 用来表明该中断正在被服务; 若中断请求的中断优先级等于或低于正在服务中的中断优先级, 则 PR 不提出中断请求, 同样不将 ISR 的相应位置位。

8) 控制逻辑

控制逻辑是 8259A 全部功能的控制核心。它包括一组初始化命令字寄存器 $ICW_1 \sim ICW_4$ 和一组操作命令字寄存器 $OCW_1 \sim OCW_4$, 以及有关的控制电路。初始化命令字在系统初始化时设定, 工作过程中一般保持不变。操作命令字在工作过程中根据需要设定。控制逻辑电路按照编程设定的工作方式管理 8259A 的全部工作。

2. 8259A 的引脚

8259A 为 28 脚双列直插式封装, 外部引脚如图 9.6 所示, 各引脚的定义及功能如下:

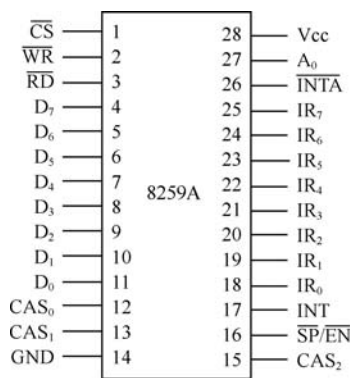


图 9.6 8259A 外部引脚

$\overline{CS}$: 片选信号, 输入, 低电平有效。

$\overline{RD}$: 读信号, 输入, 低电平有效。

$\overline{WR}$: 写信号, 输入, 低电平有效。

$CAS_2 \sim CAS_0$: 三根双向的级联线。

A_0 : 地址线, 输入, 用于内部端口选择。

V_{CC} : +5V 电源, 输入。

GND: 地, 输入。

$\overline{SP}/\overline{EN}$: 主从设备设定/缓冲器读/写控制, 双向双功能。

$IR_7 \sim IR_0$: 外设向 8259A 发出的中断请求输入信号。

$D_7 \sim D_0$: 双向数据线, 接 CPU 的 $D_7 \sim D_0$ 。

INT: 8259A 中断请求输出信号, 输出, 高电平有效。该脚连到 CPU 的 INTR 引脚, 用于向 CPU 发中断请求。

$\overline{INTA}$: 中断响应输入信号, 低电平有效。此引脚连接到 CPU 的 $\overline{INTA}$ 引脚, 用来接收 CPU 发来的中断响应信号。

对部分引脚功能说明如下:

(1) A_0 : 端口选择引脚, 用于指示 8259A 的哪个端口被访问, 8259A 需要两个连续端口地址, 把 $A_0=0$ 所对应的端口称为“偶端口”, 另一个为“奇端口”。

对于 8088 系统, 将 8259A 的 $D_7 \sim D_0$ 与 CPU 的 $D_7 \sim D_0$ 相连接, 8259A 的 A_0 与 CPU 的 A_0 相连, 即可满足 8259A 的地址要求。

对于 8086 系统, 数据线为 16 根, 一般也是将 8259A 的 $D_7 \sim D_0$ 与 CPU 的 $D_7 \sim D_0$ 相连接, 但要 CPU 的 A_1 与 8259A 的 A_0 相连, 将 CPU 的 $A_0=0$ 作为 8259A 的片选条件之一。这样, 将 8086 的两个连续的偶地址($A_1A_0=00$, $A_1A_0=10$ 转换为 8259A 的一个偶地址和一个奇地址, 从而满足了 8259A 的地址要求。

(2) $CAS_2 \sim CAS_0$: 双向级联总线, 8259A 单片工作时不用这些引脚。当级联工作时, 主片 8259A 的 $CAS_2 \sim CAS_0$ 与从片 8259A 的 $CAS_2 \sim CAS_0$ 连接线, 构成如图 9.7 所示结构。

作为主片时, 这三根引脚为输出, 从片时为输入。图 9.7 中, 设从片 1 与从片 2 同时对主片提出中断请求, 而主片将从片 1 的中断请求发往 CPU 的 $\overline{INTR}$ 引脚, 当 CPU 响应该中断请求后, 发来第一个 $\overline{INTA}$ 脉冲, 从片 1 和从片 2 均能收到这一信号, 但不能确定是谁的中断请求被响应, 由于从片 1 接在主片的 IR_0 引脚上, 故主片在 $CAS_2 \sim CAS_0$ 放上“000”, 表示接在 IR_0 引脚上的从片的请求被响应, 从片 1 知道自己的 INT 引脚接在主片的 IR_0 输入端(详见初始化命令字 ICW3), 故收到“000”后知道自己的中断请求被响应, 它将在第二个 $\overline{INTA}$ 脉冲到来后将发出中断请求的外设的中断类型码放到系统数据线上供 CPU 读取。

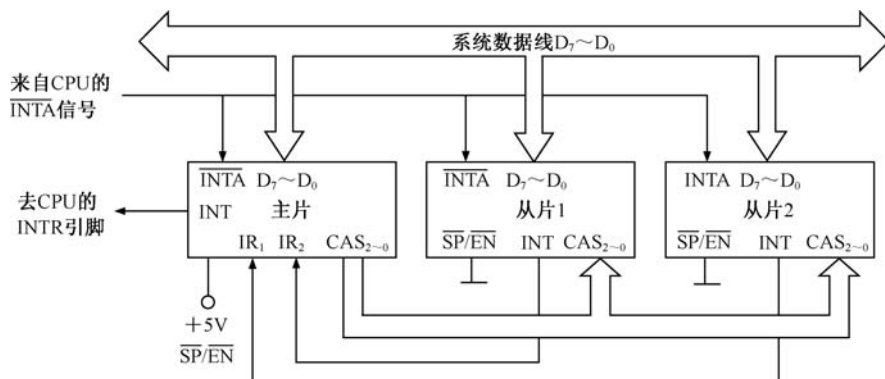


图 9.7 8259A 级联

(3) $\overline{SP}/\overline{EN}$: 主从定义/缓冲器方向, 这是一根双功能引脚, 当 8259A 工作在缓冲方式时, 它是输出引脚, 用来控制收发器的传送方向。当 8259A 工作在非缓冲方式时, 它是输入引脚, 用以指明该片是主片还是从片, $\overline{SP}/\overline{EN}=1$, 该片为主片; $\overline{SP}/\overline{EN}=0$, 该片为从片。

9.3.2 8259A 中断响应时序

下面以 8259A 单片方式为例, 结合 CPU 的动作, 说明中断的基本过程, 以便更好地理解 8259A 的功能。

(1) 当 $IR_7 \sim IR_0$ 中有一个或几个中断源变成高电平时, 使相应的 IRR 位置位。

(2) 8259A 对 IRR 和 IMR 提供的情况进行分析处理, 当请求的中断源未被 IMR 屏蔽时, 如果这个中断请求是唯一的, 或请求的中断比正在处理的中断优先级高, 就从 INT 端输出一个高电平, 向 CPU 发出中断请求。

(3) CPU 在每个指令的最后一个时钟周期检查 INT 输入端的状态。当 IF 为“1”且无其他高优先级的中断(如 NMI)时, 就响应这个中断, CPU 进入两个中断响应($\overline{INTA}$)周期。

(4) 在 CPU 第一个 $\overline{INTA}$ 周期中, 8259A 接收第一个 $\overline{INTA}$ 信号时, 将 ISR 中当前请求中断中优先级最高的相应位置位, 而对应的 IRR 位则复位为“0”。

(5) 在 CPU 第二个 $\overline{INTA}$ 周期中, 8259A 收到第二个 $\overline{INTA}$ 信号时, 送出中断类型号。整个过程的时序如图 9.8 所示。

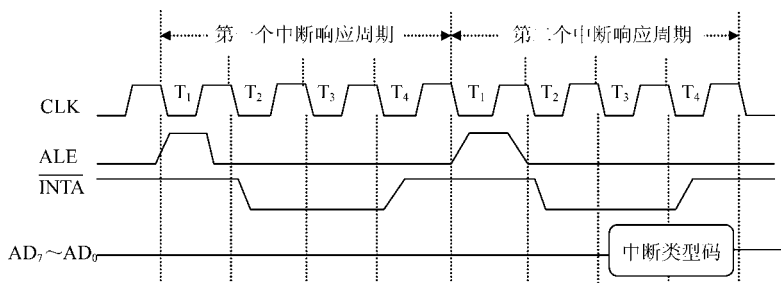


图 9.8 中断响应周期时序

9.3.3 8259A 的工作方式

8259A 具有设置灵活、功能丰富的特点, 这主要体现在其众多的工作方式上, 下面逐一介绍这些方式。

1. 中断优先级的设置方式

8259A 对中断进行管理的核心是中断优先级的管理, 8259A 对中断优先级的设置方式有 4 种。

1) 全嵌套方式

全嵌套方式是最常用的和最基本的一种工作方式。8259A 是初始化后默认的工作方式。在此方式下, 外设中断请求的优先级是固定的。 IR_0 最高, IR_7 最低, 其他依次类推。

当有一个中断请求 IR_i 被响应时, 中断服务寄存器 ISR 中的相应位置“1”, 这个“1”将一直保持到 8259A 收到 CPU 发来中断结束命令 EOI 之前, 以便作为优先级判别器 PR 的

判优依据。

在全嵌套工作方式下，当一个中断被响应后，就会自动屏蔽同级及低级中断请求，但能开放高级中断请求。在极端情况下，依次出现从 $IR_7 \sim IR_0$ 上的中断请求时，最多可实现 8 级嵌套，即 ISR 中内容为 FFH。

2) 特殊全嵌套方式

在这种方式下，当一个中断被响应后，只屏蔽掉低级的中断请求，而允许同级及高级的中断请求。该方式一般用于多片 8259A 级联的系统中，主片采用此方式，而从片采用一般全嵌套方式。

在级联情况下，当从片收到一个比正在被服务的中断源的优先级更高的中断请求时，虽然从片会向主片发出中断请求，但对于主片来说，是属于同一级的中断请求，若按全嵌套方式的原则，则不会接收该中断请求，这样就破坏了允许高级中断打断低级中断的原则。若主片工作在特殊全嵌套方式，则可解决此问题。

在特殊全嵌套方式下，主片 ISR 中的某个置“1”位，可能对应着从片中的几次中断服务，这样，当从片中断源的中断服务程序结束时，不能简单地向主片发 EOI 命令让其清除这个置“1”位，而应先向该从片发一条特殊 EOI 命令，然后读取从片 ISR 的内容，检查其是否全为 0，若全为 0，表示已无低级的中断服务，则可向主片发一条非特殊的 EOI 命令。否则，不向主片发结束命令。

3) 优先级自动循环方式

在这种方式下，某个中断源被服务后，其优先级自动降为最低，它后面的中断源按顺序递升一级。如 IR_3 刚被服务完，则各中断源的优先级次序为： IR_4 、 IR_5 、 IR_6 、 IR_7 、 IR_0 、 IR_1 、 IR_2 、 IR_3 。

这种方式中，刚开始时，优先级仍是固定的，即 IR_0 最高， IR_7 最低。这种方式适合于各个中断源的重要性等同的情况。

4) 优先级特殊循环方式

同优先级自动循环方式，但一开始时的优先级可以设定。如一开始设定 IR_3 最低，则 IR_4 的优先级最高，其他依次类推。

2. 中断结束方式

当一个中断请求被响应后，8259A 便在其内部的中断服务寄存器 ISR 中将对应位置“1”，表示正在对此外设服务。中断优先级判别器 PR 要利用这一位得知当前中断的优先级，做为判优的依据。当前中断服务程序结束时，要将 ISR 中的这一位置 0，表示中断已结束，否则会造成后续中断判别的混乱。将 ISR 中对应位置 0 的方法，就叫做中断结束方式，在 8259A 中共有 3 种。

1) 自动结束方式(AEOI 方式)

当一个中断请求被响应后，在收到第一个 $\overline{INTA}$ 信号后，8259A 将 ISR 中的对应位置“1”，在收到第二个 $\overline{INTA}$ 信号后，8259A 将 ISR 中的对应位置 0。此刻，中断服务程序并没有结束(其实才刚开始运行)，而在 8259A 中就认为其已结束。此时若有更低级的中断请求信号，8259A 仍可向 CPU 发送中断请求，从而会造成低级中断打断高级中断的情况。这种方式一般用于单片 8259A 而且不会产生嵌套的情况。

2) 普通结束方式(普通 EOI 方式)

这种方式是在中断服务程序结束前(即 CPU 执行 IRET 指令),用 OUT 指令向 8259A 发一个中断结束命令字,8259A 收到此结束命令后,就会把 ISR 中优先级最高的置“1”位清 0,表示当前正在处理的中断已结束。

这种中断结束方式比较适合于全嵌套工作方式。

3) 特殊中断结束方式(特殊 EOI 方式)

在优先权循环的情况下,无法根据 ISR 的内容来确定哪一级中断是最后响应和处理的,即不能从 ISR 中“1”的位置确定当前的最高优先级。这样,若 8259A 只收到一个普通 EOI 命令,则只能知道一个中断服务程序已结束,但无法知道该将 ISR 中哪一位置 0。

所谓特殊 EOI 方式,就是中断服务程序向 8259A 发送一特殊 EOI 命令,该命令中指明将 ISR 中的哪一位置 0。

普通 EOI 方式和特殊 EOI 方式都属于非自动结束方式。

在级联方式下,一般应采用非自动结束方式。当一个中断服务程序结束时,应发二次结束命令,一次是针对主片的,另一次是针对从片的。

3. 8259A 与系统总线的连接方式

8259A 与系统总线的连接方式有两种:数据缓冲方式和非缓冲方式。

1) 数据缓冲方式

当系统中 8259A 片数较多时,考虑到系统总线带负载能力有限,应在 8259A 的数据总线与系统数据总线间加入双向总线驱动器(如 8286),即数据缓冲方式。在此方式下,8259A 的 $\overline{SP}/\overline{EN}$ 引脚为输出引脚($\overline{EN}$ 起作用),用来控制收发器的收发方向。当为低电平时,控制数据收发器将数据从 8259A 传向 CPU;当为高电平时,反之。

2) 非缓冲方式

当系统中 8259A 的数量较少时,可将 8259A 直接与系统总线相连,此为非缓冲方式。在此方式下,8259A 的 $\overline{SP}/\overline{EN}$ 引脚为输入引脚(SP 起作用),决定 8259A 是主片还是从片,为 1 表示 8259A 为主片,为 0 表示为从片。

9.3.4 8259A 的命令字

8259A 的各种功能都要通过编程设置来实现,8259A 提供了 4 个初始化命令字 ICW1~ICW4 和 3 个操作命令字 OCW1~OCW3 供程序员访问。初始化命令字的特点是:应在一开始初始化 8259A 时使用,只能使用一次,一旦发出就不能改变,且 4 个命令字有固定的写入顺序,一般将其放在主程序的开头。操作命令字用来设置可在程序中动态改变的功能,可多次使用,也没有固定的使用顺序。

8259A 的所有初始化命令字和操作命令字均为一字节,有的需要写入奇地址,有的需要写入偶地址,详见各个字的说明。

PC/XT 中,8259A 所占的端口地址为 20H,21H。

1. 初始化命令字

初始化命令字必须按 ICW1~ICW4 的顺序依次写入,但若其中某个(ICW3 或 ICW4)不需要,则不用写入,而直接写入下一个命令字。

1) 初始化命令字 ICW1

A ₀	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
0	×	×	×	1	LTIM	ADI	SNGL	IC ₄

ICW1 必须写入 8259A 的偶地址端口，即 A₀=0。

各位的控制功能为：

D₀: IC₄，用以决定初始化过程中是否需要设置 ICW4。如 IC₄=0，则不要写入 ICW4；若 IC₄=1 时，则需要写入 ICW4。对于 8086/8088 系统来说，ICW4 必须有，所以该位必须为“1”。

D₁: SNGL，用来设定 8259A 是单片使用还是多片级联使用。如系统中只有一片 8259A，则使 SNGL=1，且在初始化过程中，不用设置命令字 ICW3。反之，若采用级联方式，则使 SNGL=0，且在命令字 ICW1、ICW2 之后必须设置 ICW3 命令字。

D₂: ADI，在 8080/8085 CPU 方式下工作时，设定中断矢量的地址间隔大小。在 8086/8088 系统中这一位无效。

D₃: LTIM，用来设定中断请求输入信号 IR_i 的触发方式。若 LTIM=0，设定为边沿触发方式，即在 IR_i 输入端检测到由低到高的正跳变时，且正电平保持到第一个 INTA 到来之后，8259A 就认为有中断请求。若 LTIM=1，设定为电平触发方式，只要在 IR_i 输入端上检测到一个高电平，且在第一个 INTA 脉冲到来之后维持高，就认为有中断请求，并使 IRR 相应位置位。电平触发方式下，外设应在 IRR 复位前或 CPU 再允许下一次中断进入之前，撤销这个高电平，否则有可能出现一次高电平引起两次中断的现象。

D₄: 标志位，D₄=1 表示当前写入的是 ICW₁ 初始化命令字。

D₅~D₇: D₅~D₇ 是 8080/8085 系统中断向量地址的 A₅~A₇ 位。在 8086/8088 系统中，这 3 位不用。

无论何时，当微处理器向 8259A 送入一条 A₀=0、D₄=1 的命令时，该命令被译码为 ICW₁，它启动 8259A 的初始化过程，相当于 RESET 信号的作用，自动完成下列操作：清除中断屏蔽寄存器 IMR，设置以 IR₀ 为最高优先级、依次递减，以 IR₇ 为最低优先级的全嵌套方式，固定中断优先权排序。

【例 9.1】 在 8086 系统中，设置 8259A 为单片使用，上升沿触发，则程序段为：

```
MOV AL, 13H ; ICW1 的内容
OUT 20H, AL ; 写入偶地址端口
```

2) 初始化命令字 ICW2

A ₀	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
1	T ₇	T ₆	T ₅	T ₄	T ₃	0	0	0

ICW2 必须写入奇地址端口。

该命令字用以设置 8259A 在第二个中断响应周期时提供给 CPU 的中断类型码。在 8086/8088 系统中，中断类型码为 8 位，其前 5 位由 ICW2 的高 5 位 T₇~T₃ 决定，后 3 位由 8259A 自动确定，对于 IR₇~IR₀ 上的中断请求，最低 3 位依次为 000~111。

【例 9.2】 PC 机中要将 IR₇~IR₀ 上的中断请求类型码设置为 A0~A7H。

分析：将 ICW2 高 5 位设置为 10100 即可，对应程序段为：

```
MOV AL, 0A0H    ; ICW2 的内容
OUT 21H, AL     ; 写入奇地址端口
```

3) 初始化命令字 ICW3

ICW3 必须写入奇地址端口。

本命令字用于级联方式下的主/从片设置。只有 ICW1 的 SNGL=0，即系统中 8259A 使用级连方式工作时，才需要使用 ICW3。对于主片或从片，ICW3 的格式和含义是不相同的，所以，主片/从片的命令字 ICW3 要分别写入。

对于主片，ICW3 的格式和各位含义如下：

A ₀	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
1	IR ₇	IR ₆	IR ₅	IR ₄	IR ₃	IR ₂	IR ₁	IR ₀

在级联方式下，从片的中断请求输出(INT 引脚)作为主片的一个外设对待，接在主片的一个中断请求输入端 IR_i 上(参见图 9.7)。那么，主片 8259A 如何知道哪一个中断请求输入端是一从片 8259A 而不是外设呢？通过设置主片的 ICW3 完成此功能。ICW3 的 D₇~D₀ 与 8 个中断请求输入引脚 IR₇~IR₀ 一一对应，ICW3 的某位为 1，则对应的中断请求输入引脚是从片 8259A；某位为 0，则对应的中断请求输入引脚是外设。

【例 9.3】 参照图 9.7，主片的 IR₀ 与 IR₁ 上接有从片，则主片的初始化程序段为：

```
MOV AL, 03H    ; ICW3 的内容
OUT 21H, AL    ; 写入奇地址端口
```

对于从片，ICW3 的格式和各位含义如下：

A ₀	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
1	0	0	0	0	0	ID ₂	ID ₁	ID ₀

ID₀~ID₂ 位从片标志位可有 8 种编码，表示从片的中断请求输出被连到主控制器的哪一个中断请求输入端 IR_i 上。

【例 9.4】 参照图 9.7，从片 1 的 INT 引脚接在主片的 IR₀ 上，则从片 ICW3 的低 3 位编码为 ID₂~ID₀=000，该从片初始化程序为：

```
MOV AL, 00H    ; ICW3 的内容
OUT 21H, AL    ; 写入奇地址端口
```

4) 初始化命令字 ICW4

A ₀	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
1	0	0	0	SFNM	BUF	M/F	AEOI	uPM

ICW4 必须写入奇地址。

只有当 ICW1 中的 IC₄=1 时，才要设置 ICW4，其各位含义为：

D₀: uPM, CPU 类型选择，用来指出 8259A 是在 16 位机系统中使用，还是在 8 位机系统中使用。若 uPM=1，则 8259A 用于 8086/8088 系统；若 uPM=0，则 8259A 用于 8080/8085 系统。

D₁: AEOI, 用于选择 8259A 的中断结束方式。当 AEOI=1 时, 设置中断结束方式为自动结束方式; 当 AEOI=0 时, 8259A 工作在非自动结束方式。

D₂: M/S, 用来规定 8259A 在缓冲方式下, 本片是主片还是从片, 即该位只有在缓冲方式(BUF=1)时才有效。当 BUF=1, 且 M/S=1 时, 此 8259 为主片; 当 BUF=1, 但 M/S=0 时, 此 8259 为从片。而 8259A 在非缓冲方式下(BUF=0)工作时, M/S 位不起作用, 此时的主、从方式由 $\overline{SP}/\overline{EN}$ 端的输入电平决定。

D₃: BUF, 用来设置 8259A 是否在缓冲方式下工作。若 BUF=1, 则 8259A 在缓冲方式下工作; 若 BUF=0, 则 8259A 在非缓冲方式下工作。

D₄: SFNM, 用来设定 8259A 的中断嵌套方式。若 SFNM=1, 8259A 设置为特殊全嵌套方式; 若 SFNM=0, 8259A 设置为一般全嵌套方式。

2. 操作命令字

8259A 初始化后, 就进入了工作状态, 此后便可使用操作命令字改变其工作方式。操作命令字可以在主程序中使用, 也可以在中断服务程序中使用。

1) 操作命令字 OCW1

A ₀	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
1	M ₇	M ₆	M ₅	M ₄	M ₃	M ₂	M ₁	M ₀

OCW1 必须写入 8259A 的奇地址端口。

OCW1 是中断屏蔽操作字, 其内容直接置入中断屏蔽寄存器 IMR 中。M₇~M₀ 分别对应 IR₇~IR₀ 上的中断请求, 如某位置 1, 相应的 IR_i 输入被屏蔽, 但不影响其他中断请求输入引脚; 若某位置 0, 则相应中断请求允许。

【例 9.5】 要使中断源 IR₅ 屏蔽, 其余允许, 则程序段为:

```
MOV AL, 20H ; OCW1 的内容
OUT 21H, AL ; 写入奇地址端口
```

2) 操作命令字 OCW2

A ₀	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
0	R	SL	EOI	0	0	L ₂	L ₁	L ₀

OCW2 必须写入偶地址。

OCW2 是中断结束方式和优先级循环方式操作命令字, 命令字的 D₄D₃=00 作为 OCW2 的标志位, 其余各位含义如下:

D₇: R 位, 作为优先级循环控制位。R=1 为循环优先级, R=0 为固定优先级。

D₆: SL 位, 指明 L₂~L₀ 是否有效。SL=1 时, L₂~L₀ 有效; SL=0 时, L₂~L₀ 无效。

D₅: EOI(中断结束命令)位, EOI=1, 表示这是一个中断结束命令, 8259A 收到此操作字后须将 ISR 中的相应位置 0; EOI=0, 表示这是一个优先级的设置命令, 而不是中断结束命令。

这 3 位组合形成的操作功能见表 9-1。

表 9-1 OCW2 功能小结

R	SL	EOI	操 作 命 令
0	0	1	正常 EOI 中断结束命令, 用于 8259A 采用普通 EOI 方式时的中断服务程序中, 通知 8259A 将 ISR 中优先级最高的置 1 位置 0
0	1	1	特殊 EOI 中断结束命令, 用于 8259A 采用特殊 EOI 方式时的中断服务程序中, 命令中的 $L_2 \sim L_0$ 指出了要将 ISR 中的哪一位置 0
1	0	1	正常 EOI 时循环命令, 用于 8259A 采用普通 EOI 方式时的中断服务程序中, 通知 8259A 将 ISR 中优先级最高的置 1 位置 0, 且将其优先级置为最低, 其下一级为最高, 其余依次循环
1	0	0	自动 EOI 时循环置位命令, 在 8259A 工作于自动 EOI 方式时用于设置优先级循环, 使刚服务完的中断优先级置为最低, 其下级置为最高, 其余依次循环
0	0	0	自动 EOI 时循环复位命令, 在 8259A 工作于自动 EOI 方式时用于取消优先级循环方式, 恢复固定优先级
1	1	1	特殊 EOI 时循环命令, 用于 8259A 采用特殊 EOI 方式时的中断服务程序中, 命令中的 $L_2 \sim L_0$ 指出了要将 ISR 中的哪一位置 0, 且将其优先级置为最低, 其下一级为最高, 其余依次循环
1	1	0	优先级设定命令: 设置 8259A 工作于优先级循环方式, 将 $L_2 \sim L_0$ 指定位的优先级置为最低, 其下一级为最高, 其余依次循环
0	1	0	无意义

【例 9.6】已知 8259A 中 ISR 的 D_3 位已置位, 试将其清 0。

分析: 通过特殊 EOI 中断结束命令来实现, OCW2 的格式应为: 01100011B, 即 63H。程序如下:

```
MOV AL, 63H      ; OCW1 的内容
OUT 20H, AL      ; 写入偶地址端口
```

3) 操作命令字 OCW3

A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	×	ESMM	SMM	0	1	P	RR	RIS

OCW3 必须写入 8259A 的偶地址端口。

OCW3 的功能有 3 个: 一是用来设置和撤销特殊屏蔽方式; 二是读取 8259A 的内部寄存器 ISR 或 IRR 的内容; 三是设置中断查询方式。命令字的 $D_4D_3=01$ 作为标志位, 其余各位组合完成如下功能:

(1) 读寄存器命令。

D_1 : RR 读寄存器命令位。RR=1 时, 允许读 IRR 或 ISR, RR=0 时, 禁止读这两个寄存器。

D_0 : RIS 读 IRR 或 ISR 的选择位。显然, 这一位只有当 RR=1 时才有意义, 当 RIS=1 时, 下次读正在服务寄存器 ISR; 当 RIS=0 时, 下次读中断请求寄存器 IRR。

读这两个寄存器内容的步骤是相同的,即先写入 OCW3 确定要读哪个寄存器,然后再对 OCW3 读一次(即对同一端口地址),就得到指定寄存器内容了。

【例 9.7】读取 ISR 的内容

```
MOV AL, 0BH      ; 发 OCW3, 指定要读 ISR
OUT 20H, AL      ; 写入偶地址
NOP              ; 延时
IN  AL, 20H      ; 读 ISR
```

中断屏蔽寄存器 IMR 内容的读出比较简单,直接从 OCW1 地址(即奇地址)读出即可。

(2) 查询。

D₂: P 位, 8259A 的中断查询设置位。当 IF=1 时, CPU 不接受可屏蔽中断请求, 但若此时 CPU 还是想知道有哪个中断源处于中断申请状态, 可通过对 8259A 进行查询获得。当 P=1 时, 8259A 被设置为中断查询方式工作; 当 P=0 时, 表示 8259A 未被设置为中断查询方式。查询时 CPU 先向 8259A 偶地址写入一个查询字 OCW3=0CH, 随后再用 IN 指令读偶地址, 读出数据的格式为:

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
IR	×	×	×	×	W ₂	W ₁	W ₀

IR 位表示有无中断请求, IR=1 表示有请求, 此时 W₂~W₀ 就是当前中断请求的最高优先级的编码; I=0 表示无中断请求。

(3) 中断屏蔽。

D₆D₅: ESMM、SMM, 特殊屏蔽允许位, 这两位组合含义如下:

ESMM, SMM=11: 将 8259A 设置为特殊屏蔽方式, 该方式下只屏蔽本级中断请求, 开放高级或低级的中断请求。

ESMM, SMM=10: 撤销特殊屏蔽方式, 恢复原来的优先级控制。

ESMM, SMM=0×: 无效。

以上详细介绍了 8259A 的所有控制字, 下面再说一下 8259A 对控制字的识别问题。7 个控制字中, 写入偶地址的有 3 个: ICW1、OCW2、OCW3, 写入奇地址的有 4 个: ICW2、ICW3、ICW4、OCW1。写入偶地址的 3 个控制字均有标志位, 8259A 可据此识别。写入奇地址的 4 个控制字中, ICW2、ICW3、ICW4 必须紧随 ICW1 依次写入, 故不必设单独的识别标志。这样, 初始化结束后奇地址处只有 OCW1 一个控制字写入, 故它也不必再设标志位。

9.3.5 8259A 的编程及其在 8086 微型计算机中的应用

9.3.4 小节介绍了 8259A 的两类编程命令: 初始化命令字 ICW1~ICW4 和操作命令字 OCW1~OCW3。本小节介绍 8259A 的初始化顺序及几个实例, 以进一步熟悉这几个控制字的用法。

1. 8259A 的初始化顺序

8259A 初始化命令字的使用有严格的顺序, 如图 9.9 所示。

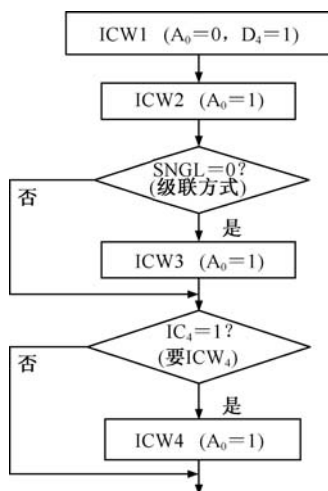


图 9.9 8259A 的初始化顺序

【例 9.8】某 8086/8088 系统中有一片 8259A，中断请求信号为电平触发，中断类型码为 50H~57H，中断优先级采用一般全嵌套方式，中断结束方式为普通 EOI 方式，与系统连接方式为非缓冲方式，8259A 的端口地址为 F000H 和 F001H，试写出初始化程序。

```

MOV DX, 0F000H ; 设置 8259A 的偶地址
MOV AL, 1BH    ; 设置 ICW1
OUT DX, AL
MOV DX, 0F001H ; 设置 8259A 的奇地址
MOV AL, 50H    ; 设置 ICW2，中断类型号基值
OUT DX, AL
MOV AL, 01H    ; 设置 ICW4
OUT DX, AL
  
```

2. 8259A 在 8086 微机中的应用

PC/XT 系统中，8259A 的使用方法如下：

单片使用，中断请求信号边沿触发，固定优先级，中断类型号范围为 08H~0FH，非自动 EOI 方式，端口地址为 20H 和 21H，硬件连接及 8 级中断源的情况如图 9.10 所示。试写出初始化程序。

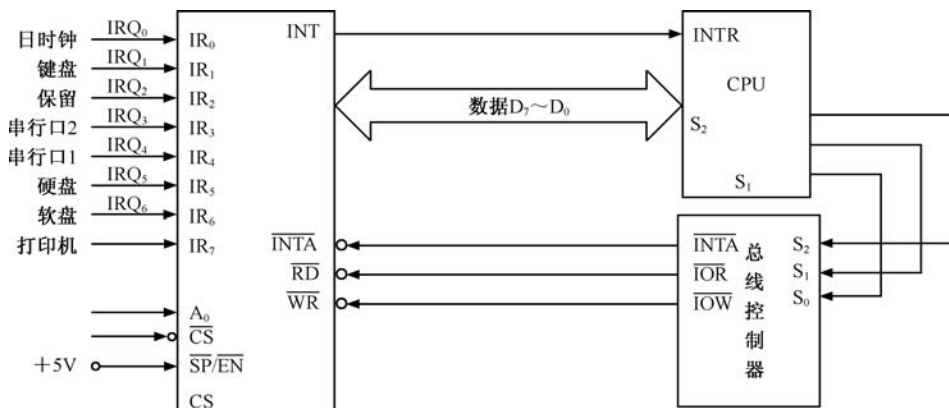


图 9.10 PC/XT 中 8259A 硬件连接图

初始化程序为：

```
MOV AL, 13H      ; 写 ICW1: 边沿触发、单片、需要 ICW4
OUT 20H, AL
MOV AL, 08H      ; 写 ICW2: 中断类型号高 5 位
OUT 21H, AL
MOV AL, 01H      ; 写 ICW4: 一般嵌套, 8086/8088 CPU
OUT 21H, AL      ; 非自动结束
```

9.4 8086 CPU 的中断接口技术

本节介绍 8086 微机的中断情况及 DOS 下中断服务程序的编写，由于在 Windows，这些内容有很大不同，故只作简单介绍，以帮助读者加深理解前面所讲原理。

9.4.1 8086 微型计算机中断分配情况

8086 系统中最多能处理 256 个中断，PC/XT 中这 256 个中断的分配情况见表 9-2。

表 9-2 PC/XT 中中断安排情况表

中断号范围	分配情况	中 断 号	中 断 源	中断类型
0~1FH	BIOS 中断	0	除法溢出	内中断，硬中断
		1	单步	内中断，硬中断
		2	NMI	外中断，硬中断
		3	断点中断	内中断，软中断
		4	溢出中断	内中断，软中断
		5	BIOS 屏幕拷贝	内中断，硬中断
		8	定时器	外中断，硬中断
		9	键盘	外中断，硬中断
		B	COM2	外中断，硬中断
		C	COM1	外中断，硬中断
		D	硬盘控制器	外中断，硬中断
		E	软盘控制器	外中断，硬中断
		10H	显示服务	内中断，软中断
		11H	确认设备调用	内中断，软中断
		12H	取内存容量	内中断，软中断
		13H	磁盘 I/O	内中断，软中断
		14H	串行口通信	内中断，软中断
		15H	AT 扩充服务	内中断，软中断
		16H	键盘服务	内中断，软中断
		17H	打印服务	内中断，软中断
		18H	BASIC 调用	内中断，软中断
		19H	引导系统	内中断，软中断
		1AH	实时时钟	内中断，软中断

续表

中断号范围	分配情况	中 断 号	中 断 源	中断类型
20H~3FH	DOS 中断	20H	程序结束	内中断, 软中断
		21H	DOS 系统功能	内中断, 软中断
		22H	DOS 结束地址	内中断, 软中断
		23H	CTRL-BREAK	内中断, 软中断
40H~5FH	BIOS 中断			内中断, 软中断
60H~67H	用户使用			内中断, 软中断
68H~6FH	保留			
70H~77H	BIOS 中断			外中断, 硬中断
78H~FFH	保留			

9.4.2 DOS 下中断服务程序的编写

1. 中断服务程序编写要注意的问题

(1) 保护现场: 由于中断是打断了主程序的执行来运行中断服务程序, 因此在中断服务程序中要注意保护主程序的现场, 凡是中断服务程序要用到的寄存器, 事先都要入栈保存, 在返回主程序前再进行恢复。但标志寄存器不用保存, 因为在调用中断服务程序前 CPU 已进行了保存。

(2) 开放中断: 保护完现场后要及时开放中断, 以便 CPU 响应更高级别的中断。

以上两项示意如下:

保护现场→STI→中断服务程序主代码→CLI→恢复现场→中断返回

注: 关中断恢复现场后, 在中断返回前不必开中断, 因为执行 IRET 指令后, 会自动恢复中断前标志寄存器的状态。

(3) 中断的返回要用专用的 IRET 指令, 而不能用 RET。

(4) 在中断服务程序中, 尽量避免调用 DOS 功能 21H, 因为它不可重入(但为简单起见, 后面的例子还是调用了它)。

2. 中断服务程序的安装问题

程序在运行后, 主模块要完成中断服务程序的安装, 即将中断服务程序的地址设置到中断矢量表的相应项目中。设置工作既可以用 MOV 指令直接写中断矢量表, 也可以用相关 DOS 功能调用, 最好是用后者, 因为这样更安全一些。

与中断矢量设置与读取有关的 DOS 功能调用如下:

(1) 设置中断矢量: (25H 号调用)

入口参数: AH=25H, AL=中断类型号, DS: DX=中断矢量

出口参数: 无

(2) 读取中断矢量: (35H 号调用)

入口参数: AH=35H, AL=中断类型号

出口参数: ES: BX=中断矢量

另外, 在 256 个中断类型号中, 60H~67H 是专为用户保留的中断, 用户自己开发的

中断一般应使用这些号。

【例 9.9】 中断服务程序的安装与调用。

本例演示了如何编写中断服务程序，如何安装(将中断服务程序的地址设置到中断矢量表中)，以及如何通过软中断指令调用中断服务程序。为了突出重点，本例中中断服务程序代码未驻留内存。

```
CODE SEGMENT
ASSUME CS: CODE
INT_START:                                以下是中断服务程序代码，它在屏幕上显示 1 个*号
PUSH    AX
PUSH    DX
MOV     AH, 2
MOV     DL, '*'
INT     21H
POP     DX
POP     AX
IRET                                         ; 中断服务程序到此结束
START:
MOV     AH, 25H                             ; 安装中断服务程序的 DOS 功能调用
MOV     AL, 67H                             ; 中断类型号
PUSH     CS
POP     DS                                 ; 中断服务程序的段基址(在 CS 内)赋给 DS
MOV     DX, OFFSET INT_START               ; 中断服务程序的偏移量赋给 DX
INT     21H                                 ; 进行中断矢量表设置
INT     67H                                 ; 调用 67H 号中断
MOV     AH, 4CH
INT     21H                                 ; 返回 DOS
CODE ENDS
END START
```

3. 中断服务程序的内存驻留问题

在上例中，当程序结束后，其所占内存被 DOS 收回，中断服务程序变为不可用。若想让程序退出后中断服务程序所占内存仍然保留，以便为后续程序提供中断服务，则必须让中断服务程序驻留内存。

DOS 的功能调用 31H 可实现终止并驻留内存。

入口参数：DX=驻留内存节数

AL=退出码(如果后续程序不用，可任意设置)

出口参数：无

DX 中要指出节数而不是字节数，一节等于 16 个字节。设需要驻留部分长度为 n 个字节，则计算公式为：

$$DX = (n \div 16) + 1 + 16$$

上式中加 1 是为了防止 n 不是 16 整数倍时将余数部分考虑上，再加 16 是因为 DOS 在启动应用程序时会在程序前加上一程序段前缀 PSP(DOS 管理程序用的数据结构)，它需要和程序一块驻留内存，PSP 占 256 字节，正好是 16 节。

【例 9.10】 驻留内存的中断服务程序。

以下程序运行后将自己驻留部分的地址登记在中断矢量表中 5 号中断位置处，从而用自己的中断服务程序替换了系统原先的 5 号中断服务程序。5 号中断为屏幕打印中断，当按键盘上的 Print Screen 键时，会触发这一中断。旧的中断服务程序(BIOS 提供)的功能是将屏幕内容拷贝到打印机上，而我们的程序将其替换后，按 Print Screen 键时，将不再打印屏幕，而是在屏幕上显示一个星号。

```
CODE SEGMENT
ASSUME CS: CODE
INT_START:                                ; 以下是中断服务程序代码，它在屏幕上显示一个*号
PUSH    AX
PUSH    DX
MOV AH, 2                                ; 显示字符功能调用
MOV DL, '*'                              ; 显示字符 ASCII 码
INT 21H
POP DX
POP AX
IRET                                     ; 中断服务程序到此结束
START:
PUSH    CS
POP DS                                  ; 中断服务程序的段基址(在 CS 内)赋给 DS
MOV DX, OFFSET INT_START                ; 中断服务程序的偏移量赋给 DX
MOV AH, 25H
MOV AL, 05H
INT 21H                                  ; 调用中断服务程序设置功能
MOV DX, START-INT_START                  ; START-INT_START 为需要驻留部分长度
MOV CL, 4
SHR DX, CL                              ; 右移 4 位，即除以 16
ADD DX, 11H ; 加上 17
MOV AH, 31H
INT 21H                                  ; 终止并驻留
CODE ENDS
END START
```

4. 硬件中断服务程序的编写

关于硬件中断的中断服务程序编写，有其特殊之处，那就是若 8259A 不工作在自动 EOI 方式时，必须在中断返回前向 8259A 发 EOI 命令。请看下例：

【例 9.11】 PC/XT 机内 8259A 的端口地址为 20H 和 21H，机内的 8259A 已被初始化成边沿触发、固定优先级、一般中断结束方式。8 个中断源中 7 个已被系统使用(见图 9.10)，IRQ2 保留给用户使用，其中断类型为 0AH。现有一外设，会定时通过 IRQ2 向 CPU 发中断，要求编写对应的中断服务程序。为简单起见，中断服务程序的功能为在屏幕上显示一串提示信息。

```
DATA SEGMENT
MESSDB 'THIS IS A IRQ2 INTERRUPT!', 0AH, 0DH, '$' ; 中断服务程序中的提示信息
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE, DS:DATA
```

```
INT_START:
    PUSH    DS
    PUSH    AX
    PUSH    DX
    MOV AX, DATA           ; 设置 DS 指向数据段
    MOV DS, AX
    MOV DX, OFFSET MESS     ; 显示发生中断的信息
    MOV AH, 09
    INT 21H
    MOV DX, 20H             ; PC/XT 系统中 8259A 的偶地址端口
    MOV AL, 20H            ; 普通 EOI 中断结束命令
    OUT DX, AL             ; 发中断结束命令
    POP DX
    POP AX
    POP DS
NEXT: IRET                 ; 中断返回

START:
    PUSH    CS
    POP DS                 ; 中断服务程序的段基址(在 CS 内)赋给 DS
    MOV DX, OFFSET INT_START ; 中断服务程序的偏移量赋给 DX
    MOV AH, 25H
    MOV AL, 0AH            ; IRQ2 对应的中断类型码
    INT 21H               ; 调用中断服务程序设置功能
    MOV DX, START-INT_START ; START-INT_START 为需要驻留部分长度
    MOV CL, 4
    SHR DX, CL             ; 右移 4 位, 即除以 16
    ADD DX, 11H            ; 加上 17
    MOV AH, 31H
    INT 21H               ; 终止并驻留
    CODE    ENDS
    END      START
```

9.5 小 结

本章首先介绍了中断技术的概念, 中断源, 中断类型, 中断矢量表, 中断优先级, 中断嵌套, 以便对中断技术有一个整体的把握; 然后介绍了 8086 CPU 的中断处理过程, 可编程中断控制器 8259A, 8086 微处理器的中断接口技术, 以便对中断技术的实际应用有一个全面的掌握。

9.6 习 题

1. 8086/8088 CPU 有哪几种中断?
2. 简要说明 8086/8088 中断的特点。
3. 简述 8086/8088 可屏蔽中断的响应过程。
4. 何为中断矢量表? 它有何作用? 位于内存的什么位置?

5. 30H 号中断的中断服务程序地址存放在中断矢量表的什么位置处?
6. 8259A 中断控制器的作用是什么?
7. 简述多个中断源、单一中断请求线的中断处理过程。
8. 8259A 的中断自动结束方式与非自动结束方式对中断服务程序的编写有何影响?
9. 某 8259A 初始化时, ICW1=1BH, ICW2=30H, ICW4=01H, 试说明 8259A 的工作情况。
10. 某系统中有三片 8259A 接成主/从方式, 二从片接在主片的 IR3、IR5 引脚上, 试画出硬件接线图, 并给出主片与二从片的初始化命令字 ICW3。
11. 当 8259A 需要级联使用时, 在缓冲方式与非缓冲方式下分别如何设置主/从片?
12. 简要说明 8259A 的 5 种中断优先权管理方式的特点。
13. 中断服务程序应包含哪几部分? 保存和恢复现场有何意义?
14. 如何安装中断服务程序?

第 10 章 微型计算机的 I/O 接口技术

教学提示：本章主要介绍 I/O 接口电路的基本概念、基本功能，可编程定时器/计数器 8253、可编程并行接口芯片 8255、可编程 DMA 控制器 8237 电路功能及其应用。

教学要求：通过本章学习，应该掌握 I/O 接口电路的基本概念、基本功能，以及可编程定时器/计数器 8253、可编程并行接口芯片 8255、可编程 DMA 控制器 8237 电路功能，并能根据不同应用、不同的外部设备，正确地选择接口电路，以组成特定的微机应用系统。

10.1 I/O 接口的概念与功能

组成微机最核心的硬件是 CPU 和存储器，最基本的语言是汇编语言。但是还必须配上各种外围设备进行人机交互，才能使微机进行工作。把外围设备同微机连接起来实现数据传送的电路称为 I/O 接口电路。

10.1.1 概述

计算机通过外围设备同外部世界通信或交换数据称为“输入/输出”。在微机系统中，常用的外围设备有键盘、显示器、软/硬盘驱动器、鼠标、打印机、扫描仪、绘图仪、调制解调器(MODEM)、网络适配器。随着计算机性能的不断提高，输入/输出设备也更加复杂多样，如影视、音频识别系统等。当计算机用于监测与过程控制中时，还需要模/数转换器(ADC)和数/模转换器(DAC)，以及 I/O 通道中一些专用设备。当要把这些外设与主机相连时，就需要配上相应的电路。通常把这种介于主机和外设之间的一种缓冲电路称为 I/O 接口电路(interface)，CPU 与外设之间交换数据的框图如图 10.1 所示，对于主机，接口提供外部设备的工作状态和数据；对于外部设备，接口电路寄存了主机发送给外部设备的命令和数据，使主机和外部设备之间协调一致地工作。

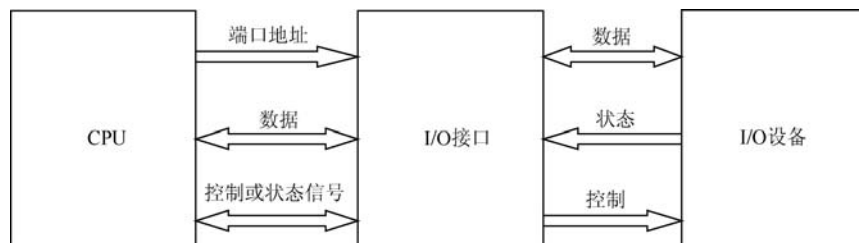


图 10.1 I/O 接口框图

10.1.2 I/O 接口电路的基本功能

主机与外围设备之间交换数据为什么需要通过接口电路？接口电路应具备哪些功能才能实现数据传送呢？下面进行具体介绍。

1. 对输入/输出数据进行缓冲、隔离和锁存

外设品种繁多，其工作原理、工作速度、信息格式、驱动方式都有差异。它不能直接和 CPU 总线相连，要借助于接口电路使外设与总线隔离，起缓冲、暂存数据的作用。在众多外设中，在某一时段仅允许被 CPU 选中的设备通过接口享用总线与 CPU 交换信息，而没有选中的设备由于接口的隔离作用不能享用总线。

对输入接口，其内部都有起缓冲和隔离作用的三态门电路，只有当 CPU 选中此接口，三态门选通时，才允许选定的输入设备将数据送至系统数据总线，而其他没有被选中的输入设备，此时相应的接口三态门“关闭”，从而达到与数据总线隔离的目的。

对于输出设备，由于 CPU 输出的数据仅在输出指令周期中的短暂时间内存在于数据总线上，故需在接口电路中设置数据锁存器，暂时锁存 CPU 送至外设的数据，以便使工作速度慢的外设有足够的时间准备接收数据及进行相应的数据处理，从而解决了主机的“快”和外设的“慢”之间的矛盾。

所以，根据输入/输出数据进行缓冲、隔离、锁存的要求，外设经接口与总线相连，其连接方法必须遵循“输入要三态、输出要锁存”的原则。

2. 对信号的形式和数据格式进行交换与匹配

CPU 只能处理数字信号，信号的电平一般在 0~5V 之间，而且提供的功率很小。而外部设备的信号形式多种多样，有数字量、模拟量(电压、电流、频率、相位)、开关量等。所以，在输入/输出时，必须将信号转变为适合对方需要的形式。例如，将电压信号变为电流信号，弱电信号变为强电信号，数字信号与模拟信号的相互转换，并行数据与串行数据的相互转换，配备校验位等。

3. 提供信息相互交换的应答联络信号

计算机执行指令时所完成的各种操作都是在规定的时钟信号下完成的，并有一定的时序。而外部设备也有自己的定时与逻辑控制，但通常与 CPU 的时序是不相同的。外设接口就需将外设的工作状态(如“忙”、“就绪”、“中断请求”)等信号及时通知 CPU，CPU 根据外设的工作状态经接口发出各种控制信号、命令及传递数据，接口不仅控制 CPU 送给外设的信息，也能缓存外设送给 CPU 的信息，以实现 CPU 与外设间信息符合时序的要求，协调工作。

4. 根据寻址信息选择相应的外设

一个计算机系统往往有多种外部设备，但 CPU 在某一段时间只能与一台外设交换信息，因此需要通过接口地址译码对外设进行寻址，以选定所需的外设，只有选中的设备才能与 CPU 交换信息；当同时有多个外设需要与 CPU 交换数据时，也需要通过外设接口来安排其优先顺序。

10.1.3 I/O 接口信号的分类

CPU 与 I/O 接口间通常需要下列接口信号。

1. 数据信息

数据通常为 8 位或 16 位, 可分为 3 种基本形式: 数字量、开关量和模拟量。

由键盘、光电输入机等提供的二进制形式的信息为数字量。

只有两个状态的量, 如电机的启停、开关的开合等, 只须用一位二进制数即可表示, 称为开关量。

由传感器等提供的信号往往是模拟量(连续变化的信号), 它需要先经过模/数(A/D)转换后, 才能输入到计算机中去, 如温度、电压等。

2. 状态信息

指 I/O 接口反映 I/O 设备工作状态的信息, 如表示输入装置是否已准备好的信息(READY 信号), 表示输出装置是否忙的信息(BUSY 信号)等。

3. 控制信息

指 CPU 向接口内部控制寄存器发出的各种控制命令, 用于改变接口的工作方式及功能, 如选通信号、启停信号等。

这三类信息的性质是不同的, 必须分别传送。通常是采用分配不同的端口地址的方法进行区别, 所以一个外设往往要占几个端口地址。一般状态信息和控制信息往往只有一位或二位, 故状态和控制信息常常共用一个端口地址。

10.1.4 I/O 端口的概念与编址方式

1. 端口地址的概念

CPU 既能够与内存交换数据, 也能与外设交换数据, 其工作原理是相似的。内存单元都进行了编址, 每一个字节的存储单元占一个地址, CPU 通过在地址线上发送地址信号来通知存储器要与哪一个存储单元交换数据; 同样, 计算机对外设接口也进行了编址, 叫做端口地址。在与 I/O 接口交换数据时, CPU 通过在地址线上发出要访问外设接口的端口地址来指出要与哪个 I/O 接口交换数据。

2. 两种编址方式

CPU 对外设的访问, 实质上是对外设接口电路中相应的端口进行访问。I/O 端口的编址方式有两种: 一种是 I/O 设备独立编址; 另一种是 I/O 设备与存储器统一编址。

1) I/O 设备独立编址

这种方式中存储器与 I/O 设备各有自己独立的地址空间, 各自单独编址, 互不相关。I/O 端口的读、写操作由 CPU 的引脚信号 $\overline{IOR}$ 和 $\overline{IOW}$ 来实现; 访问 I/O 端口用专用的 IN 指令和 OUT 指令。此方式的优点是 I/O 设备不占存储器地址空间; 缺点是需要专门的 I/O 指令。

2) I/O 设备与存储器统一编址

这种方式中存储器和 I/O 端口共用统一的地址空间。在这种编址方式下, CPU 将 I/O 设备与存储器同样看待, 因此不需要专门的 I/O 指令, CPU 对存储器的全部操作指令均可用于 I/O 操作, 故指令多, 系统编程比较灵活, I/O 端口的地址空间可大可小, 从而使外设的数目几乎不受限制。统一编址的缺点是 I/O 设备占用了部分存储器地址空间, 从而减少了存储器可用地址空间的大小, 影响了系统内存的容量。例如, 整个地址空间为 1MB, 地

址范围为 00000H~FFFFH, 如果 I/O 端口占有 00000H~0FFFFH 这 64KB 的地址, 那么存储器的地址空间只有从 10000H~FFFFH 的 960KB 个地址。

计算机的 I/O 设备采用哪种编址方式, 取决于 CPU 的硬件设计。IBM PC 系列机(Intel 系列 CPU)采用独立编址方式, 存储器用 20 位二进制数编址, 范围是: 00000H~FFFFH, 共 1MB; I/O 设备用 16 位二进制数编址, 范围是: 0000H~FFFFH, 共 64KB, 但实际系统只用了 0~3FFH 这 1024 个地址。

10.2 数据传送的控制方式

在计算机的操作过程中, 最基本和使用最多的操作是数据传送。在微机系统中, 数据主要在 CPU、存储器和 I/O 接口之间传送。在数据传送过程中, 关键问题是数据传送的控制方式, 微机系统中数据传送的控制方式主要有程序控制传送方式和 DMA(直接存储器存取)传送方式。

10.2.1 程序控制传送方式

程序控制的数据传送方式分为无条件传送方式、查询传送方式和中断传送方式。

1. 无条件传送方式

无条件传送方式又称为“同步传送方式”。主要用于外设的定时是固定的且已知的场合, 外设必须在微处理器限定的指令时间内把数据准备就绪, 并完成数据的接收或发送。通常采用的办法是: 把 I/O 指令插入到程序中, 当程序执行到该 I/O 指令时, 外设必须已为传送的数据做好准备, 在此指令时间内完成数据的传送任务。无条件传送是最简单的传送方式, 它所需要的硬件和软件都较少。

作为无条件输入的例子, 看一下图 10.2 所示的电路, 在此例中, 开关 S 的状态总是随时可读的。

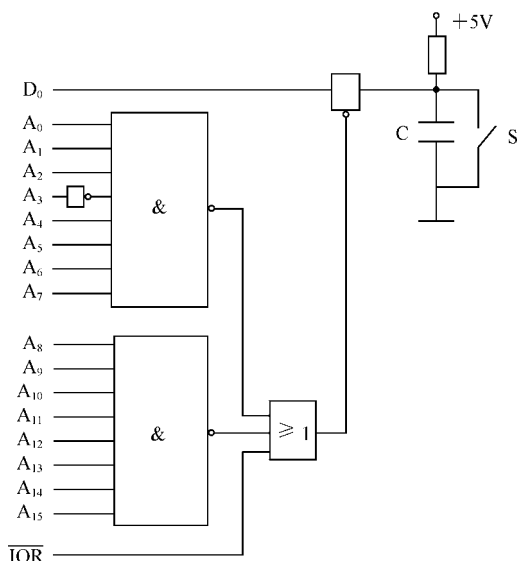


图 10.2 无条件传送方式

CPU 可随时用如下指令读取：

```
MOV DX, 0FFF7H
IN AL, DX
AND AL, 01H
JZ L1
; 以下是  $D_0=1$ ，即 S 打开时的处理
...
; 以下是  $D_0=0$ ，即 S 闭合时的处理
L1:
...
```

2. 查询传送方式

查询传送方式又称为“异步传送方式”。当 CPU 同外设工作不同步时，很难确保 CPU 在执行输入操作时，外设的数据一定是“准备好”的；而在执行输出操作时，外设寄存器一定是“空”的。为了确保数据传输的正确进行，便提出了查询传送方式。当采用这种传送方式时，CPU 必须先对外设进行状态测试。完成一次传送过程的步骤如下：

(1) 通过执行一条输入指令，读取所选外设当前的状态。

(2) 根据该设备的状态决定程序的去向，如果外设正处于“忙”或“未准备就绪”，则程序转回重复检测外设的状态；如果外设处于“空”或“准备就绪”，则发出一条输入/输出指令，进行一次数据传输。查询方式流程如图 10.3 所示。

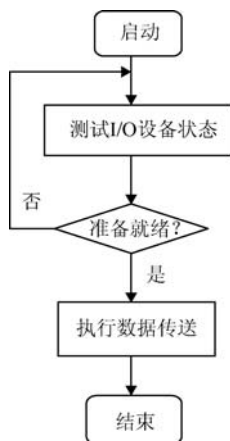


图 10.3 查询方式流程图

查询传送方式的优点：(1)安全可靠；(2)用于接口的硬件较省。缺点：CPU 必须循环等待外设准备就绪，导致效率不高。

【例 10.1】 假设外设的 $\overline{\text{BUSY}}$ 信号为低表示外设忙，不能接收数据， $\overline{\text{BUSY}}$ 为高表示外设不忙，可以接收一字节的数据。该外设与 8086 总线的接口如图 10.4 所示，图中 74LS273 为锁存芯片，其内部包含 8 个上升沿触发的 D 触发器。下面的一段程序将 BL 的内容输出到外设上去：

```
MOV DX, 00FFH
LOOP1: IN AL, DX
AND AL, 01H
```


复杂，一方面它要与外设连接，接受 DMA 请求和控制外设动作，另外它还要与 CPU 联系，请求取得总线控制权，最后它还必须与系统总线上各种总线相接，进行总线的控制。下面说明一种典型的 DMA 操作过程。

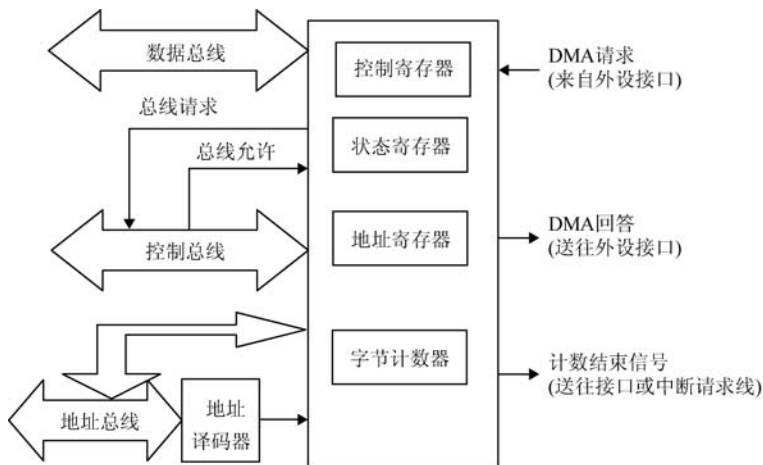


图 10.5 DMA 控制器编程结构和外部连线

1. 外设提出 DMA 传送请求

由外设或外设控制电路向 DMA 控制器发出 DMA 请求信号 DREQ，表示请求进行一次 DMA 传送。

2. DMA 控制器响应请求

DMA 控制器接到请求后，经控制电路向 CPU 提出保持请求信号 HOLD，并等待 CPU 的回答。如果控制器接有多个 DMA 设备，就要对各设备的请求进行排队，选择优先级别最高的请求输出，作为向 CPU 发出的保持请求。

3. CPU 响应

CPU 在每个时钟上升沿都检测有无 HOLD 请求，若有此请求，且自身正处在总线空闲周期中，CPU 就立即响应保持请求。如果 CPU 正在执行某个总线周期，那么要等到这个总线周期结束后再响应此保持请求。CPU 对保持请求有两个动作：第一个是从 HLDA 引脚端送出一个响应信号，告诉 DMA 控制器可以开始占用总线；第二个是将 CPU 与总线相连接的引脚置为高阻态，即释放总线。

4. DMA 控制器的动作

DMA 控制器在收到 HLDA 回答后，即开始对直接存储器存取的过程控制。它向外设送出 DACK 作为对 DMA 请求的响应，同时也作为外设的数据选通。还向系统总线送出控制信号和地址信号，以选择合适的存储单元。在一次 DMA 结束后，控制器撤除 HOLD 信号，CPU 也消除 HLDA，并重新开始对总线的使用。

现代计算机中 DMA 技术应用广泛，DMA 控制器也已经集成在一个可编程的大规模集

成电路上。PC 机中 DMA 控制器使用的是 Intel 公司的 8237A 芯片，在 10.5 节进行介绍。

10.3 8253 定时器/计数器

微机应用系统常常需要为处理机和外部设备提供实时时钟，以实现延时控制和定时，或对外部输入脉冲进行计数，实现这种功能的器件称为定时器/计数器。这种器件可用硬件电路实现计数/定时，但若改变计数或定时的要求，必须改变电路的参数，灵活性差。用软件实现计数和定时的方法通用性和灵活性好，但要占用 CPU 的时间。采用可编程定时器/计数器，其定时与计数功能由程序灵活地设定，设定后与 CPU 并行工作，不占用 CPU 的时间。Intel 公司生产的 Intel-8253 就是在微机系统中应用最广的定时器/计数器芯片，它是一种可编程芯片，工作方式、定时时间、输出信号形式等均可编程设置，使用十分方便。

10.3.1 8253 的功能与引脚

1. 8253 的基本功能

8253 的基本功能是对外部输入脉冲进行计数，若外部输入脉冲是连续而均匀的，则利用脉冲个数乘以脉冲周期可以计算出时间，从而实现了定时功能。8253 芯片内具有 3 个独立的 16 位减法计数器(或称为计数通道)，每个计数器性能如下：

- (1) 最高计数频率 2.6MHz。
 - (2) 可编程设定为按二进制计数或 BCD 码计数。
- 有 6 种工作方式，可编程确定工作在哪一种方式。

2. 8253 的引脚

8253 的引脚如图 10.6 所示。

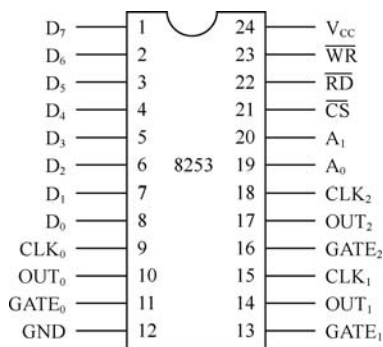


图 10.6 8253 引脚

8253 为 24 引脚双列直插式封装结构，其引脚按功能分为与 CPU 接口引脚和与外设接口引脚，分别如下：

1) 与 CPU 的接口引脚

D₇~D₀：三态双向数据线，与 CPU 数据总线直接相连。

$\overline{\text{WR}}$ ：写控制信号，输入，低电平有效。

$\overline{\text{RD}}$ ：读控制信号，输入，低电平有效。

A_1, A_0 ：地址线，输入，用于端口选择。8253 需要占用 4 个连续的端口地址，这 4 个端口地址分别对应 8253 内部的控制寄存器(接收并存放工作方式控制字)和三个计数通道的计数值寄存器(接收并存放计数初值)。具体如下：

$A_1A_0=11$ ，选中控制寄存器端口，可以向 8253 送控制字；

$A_1A_0=00、01、10$ ，分别选择计数器 0、1、2，可以对它们读/写计数值。

$\overline{\text{CS}}$ ：片选信号，输入，低电平有效。当 $\overline{\text{CS}}=0$ 时，8253 被选中，允许 CPU 对其进行读/写操作。

2) 与外设的接口引脚

$\text{CLK}_{0\sim 2}$ ：计数器 0、1、2 的外部计数时钟输入端。

$\text{GATE}_{0\sim 2}$ ：计数器 0、1、2 的门控信号输入端。门控信号用来禁止、允许或重新开始一个新计数过程，详见各工作方式的说明。

$\text{OUT}_{0\sim 2}$ ：计数器 0、1、2 的计数输出端。当定时/计数时间到时，该端输出标志信号。

3. 8253 的内部结构

8253 的内部结构如图 10.7 所示，主要由以下几部分组成。

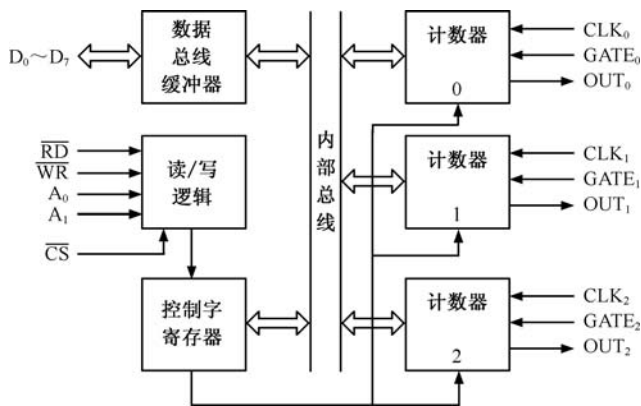


图 10.7 8253 内部结构图

1) 数据总线缓冲器

该缓冲器为 8 位双向三态，是 CPU 与 8253 内部之间的数据传输通道。

2) 读/写逻辑电路

接收 CPU 送来的读写、片选及地址信号，对 8253 内部各部件进行控制。

3) 控制字寄存器

每个计数通道有一个控制字寄存器，用来接收 CPU 写入的控制字。控制字是 8 位的，只能写不能读。三个计数通道的控制字寄存器共用一个控制端口，由写入的控制字的最高 2 位指明该控制字属于哪一个计数通道。

4) 计数器 0~2

8253 包含 3 个相互独立的、内部结构完全相同的 16 位减法计数器。每个计数器均包含一个 8 位的控制字寄存器，一个 16 位的计数初值寄存器 CR，它用来存放计数初值，可通过程序设定，以及计数执行单元 CE，它是一个 16 位的减 1 计数器，初值是计数初值寄存器的内容，它只对 CLK 脉冲计数，一旦计数器被启动后，每出现一个 CLK 脉冲，计数执行单元中的计数值减 1，当减为 0 时，通过 OUT 输出指示信号，当 CLK 是周期性的时钟信号时，计数器为定时功能，当 CLK 为非周期性事件计数信号时，呈现计数功能。一个 16 位的输出锁存器通常跟随计数执行单元内容的变化而变化，当接收到 CPU 发来的锁存命令时，就锁存当前的计数值而不跟随计数执行单元变化，直到 CPU 从中读取锁存值后，才恢复到跟随计数执行单元变化的状态，从而避免了 CPU 直接读取计数执行单元时干扰计数工作的可能。

10.3.2 8253 的工作方式

1. 方式 0 计数结束产生中断(Interrupt on Terminal Count)

采用方式 0 时，当写入控制字 CW 后，OUT 信号变为低电平。当将计数初值写入计数初值寄存器 CR 后，利用下一个 CLK 脉冲的下降沿将 CR 的内容装入计数执行单元 CE 中，再从下一个 CLK 脉冲的下降沿开始，CE 执行减 1 计数过程。在计数期间，输出 OUT 一直保持低电平，直到 CE 中的数值减到 0 时，OUT 变为高电平，以向 CPU 发出中断申请，其工作波形如图 10.8 所示。当写入控制字后，计数器的输出 OUT 变成低电平，若门控信号 GATE 为高电平，计数器开始减 1 计数并且维持 OUT 为低电平，当计数器减到 0 时，输出端 OUT 变成高电平，并且一直保持到重新装入初值或复位时为止。

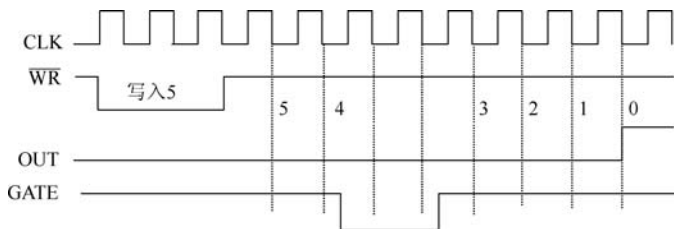


图 10.8 方式 0 的工作波形图

门控信号 GATE 可以暂停计数，当 GATE=0 时，计数停止；GATE 恢复为高电平后，继续计数。所以，如果在计数过程中，有一段时间 GATE 变为低电平，那么，输出端 OUT 的低电平持续时间会因此而延长相应的长度。

在计数过程中可以改变计数值，若是 8 位数，在写入新的计数值后立即按新值重新开始计数，若是 16 位数，写入第一个字节后计数停止，写入第二个字节后立即按新值重新计数(计数初值位数由控制字决定，见后述)。

2. 方式 1 可编程单稳态(Hardware Retriggerable one-shot)

方式 1 可以输出一个宽度可控的负脉冲。当 CPU 写入控制字后，OUT 即变为高电平，

同方式 2。图 10.11 为方式 3 的工作波形图。

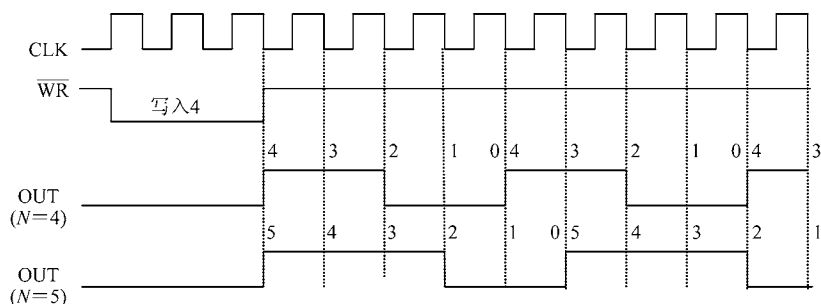


图 10.11 方式 3 的工作波形图

5. 方式 4 软件触发选通脉冲(Software Triggered Strobe)

当方式 4 写入控制字后, OUT 输出即变为高电平, 若门控信号 GATE 为高电平, 当写入计数初值到 CR 后, 在下一个 CLK 脉冲的下降沿将 CR 装入 CE 并启动计数器工作, 执行减 1 计数过程, 当计数到 0 时输出一个时钟周期的负脉冲, 计数器停止计数, 这种计数方式是一次性的。只有输入新的计数值才重新开始新的计数。计数期间, 如果写入新的计数值, 立即按新值重新计数(具体情况同方式 0)。当门控 GATE 为低电平时, 计数停止; GATE 为高电平时, 从初值开始重新计数。图 10.12 为方式 4 的工作波形图。

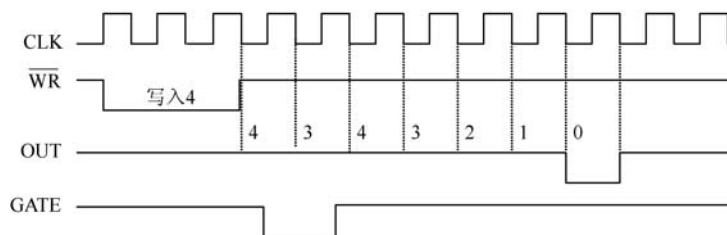


图 10.12 方式 4 的工作波形图

此方式中, 计数过程的启动是由输出指令对 CR 设置计数初值时被“触发”的, 并且只有再次将初值写入 CR 操作时才会启动另一个计数过程。

6. 方式 5 硬件触发选通脉冲(Hardware Triggered Strobe)

该方式在写入方式控制字后, 输出 OUT 保持高电平, 然后写入计数初值到 CR 后, OUT 仍然维持高电平, 但并不开始计数, 只有当门控信号 GATE 出现由低电平变高电平的上升沿后, 下一个 CLK 脉冲的下降沿才将 CR 装入 CE, 并启动计数器开始对 CLK 脉冲计数(相当于由硬件触发计数过程), 当计数到 0 时 OUT 上输出一个 CLK 周期的负脉冲, 然后计数器停止。计数过程在未结束之前, GATE 上重新出现上升沿时, 将使计数器从初值开始重新计数。图 10.13 为方式 5 的工作波形图。方式 5 是由 GATE 的上升沿触发计数器开始计数操作。

输出脉冲宽度在正常计数情况下, 如果写入的计数初值为 N , 输出端 OUT 维持 N 个时钟周期的高电平, 1 个时钟周期的低电平。

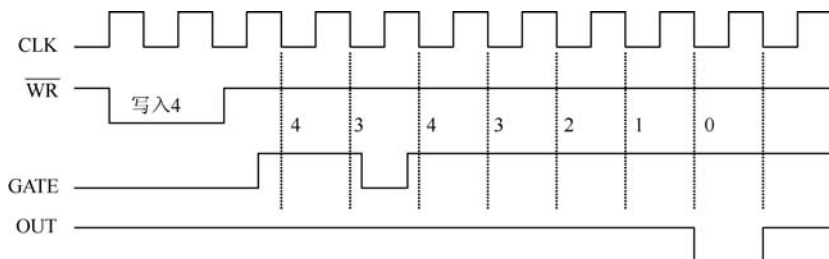


图 10.13 方式 5 的工作波形图

8253 的 6 种工作方式之特点总结见表 10-1。

表 10-1 8253 的工作方式

工作方式	开始方式	波形特征	是否自动循环	GATE 作用	中间改变计数值的有效性
0	立即 (在 GATE=1 时)	计数期间为低电平，结束后为高电平	否	低电平期间暂停计数	立即
1	GATE 上升沿	计数期间为低电平，其余为高电平	自动重置初值，但需 GATE 上升沿才能重新开始	上升沿重新计数	GATE 上升沿
2	立即 (在 GATE=1 时)	在最后一个计数期间为低电平，其余为高电平	是	恢复高电平重新计数	下一计数周期开始
3	立即 (在 GATE=1 时)	占空比 1: 1(或近似)的方波	是	恢复高电平重新计数	下一计数周期开始
4	立即 (在 GATE=1 时)	计数结束后输出一个 CLK 周期的低电平，其余为高电平	否	恢复高电平重新计数	立即
5	GATE 上升沿	同方式 4	自动重置初值，但需 GATE 上升沿才能重新开始	上升沿重新计数	GATE 上升沿

说明(适用于所有工作方式):

(1) 当控制字写入计数器后,所有控制逻辑电路立即处于复位状态,计数器输出端 OUT 进入规定的初始状态(高电平或低电平)。

(2) 计数初值写入计数初值寄存器后,要经过一个时钟周期,计数器才开始计数。

(3) 在时钟脉冲的上升沿对门控信号 GATE 进行采样。

(4) 计数器真正开始计数是在时钟脉冲的下降沿。

10.3.3 8253 的控制字与编程

1. 工作方式控制字

8253 的编程控制比较简单,只有一个 1 字节的控制字。该控制字用来设置各个计数通道的工作方式,以及计数值的锁存与读取(每个计数通道的计数值在计数过程中可随时读

取, 但控制字不能读取)。

8253 工作方式控制字的格式及含义如图 10.14 所示。

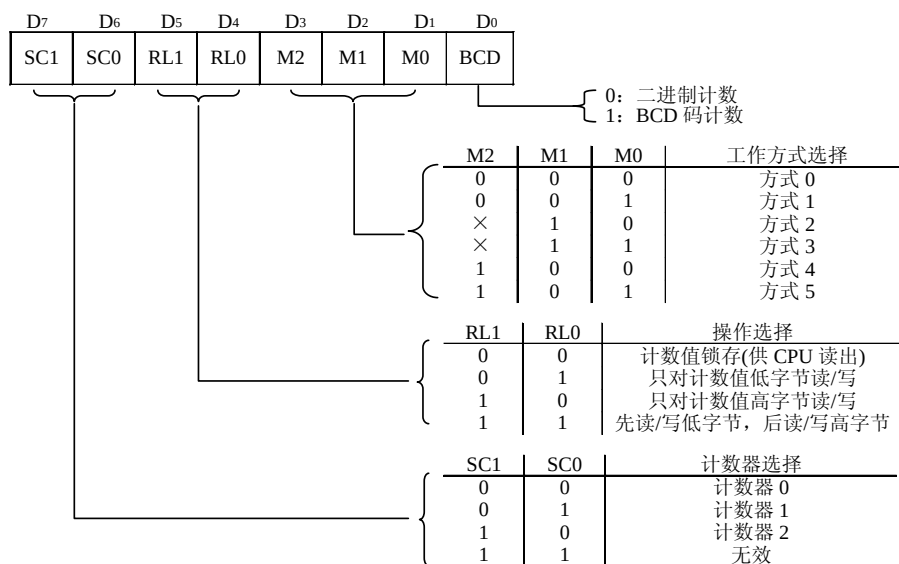
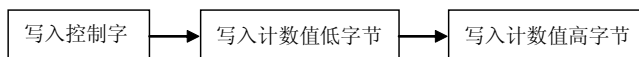


图 10.14 8253 的工作方式控制字

2. 初始化编程

8253 使用前, 必须首先对其进行初始化, 初始化包括写入控制字和计数初值。顺序如下:



关于计数初值, 需要说明以下几点:

(1) 使用任一计数通道, 首先要向该通道写入方式控制字, 以确定该通道的工作方式, 写入的控制字存入通道对应的控制寄存器中。当控制字 $D_0=0$ 时, 即二进制计数, 初值可在 $0000H \sim FFFFH$ 之间选择, 初值为 0 表示最大值 $65536(2^{16})$; 当控制字 $D_0=1$ 时, 装入初值应为 BCD 码格式, 其值可在 $0000 \sim 9999$ 之间选择, 初值为 0 表示最大值 $10000(10^4)$, 在写入指令中还必须写成十六进制数, 例如计数初值为 79, 采用 BCD 计数, 则指令中的 79 必须写成 79H。

(2) 控制字 D_5D_4 位决定控制字的位数(8 位或 16 位), 若为 “01”, 则只需写入计数值的低 8 位, 高 8 位自动置 0; 若为 “10”, 则只需写入计数值的高 8 位, 低 8 位自动置 0; 若为 “11”, 则计数值为 16 位, 分两次写入, 必须先写低 8 位, 后写高 8 位。

(3) 计数初值要写入各计数器对应的端口地址, 计数器 0 对应 $A_1A_0=00$ 的端口地址, 计数器 1 对应 $A_1A_0=01$ 的端口地址, 计数器 2 对应 $A_1A_0=10$ 的端口地址。

【例 10.2】 某系统中 8253 的端口地址为 $2F80H \sim 2F83H$, 要求通道 1 工作在方式 3, 以 BCD 方式计数, 计数初值为 1000, 试写出初始化程序。

此例中, 在写入计数值时, 可只写高 8 位, 也可写 16 位, 以写 16 位为例, 则控制字格式为:

```

01110111B=77H
MOV AL, 77H
MOV DX, 2F83H
OUT DX, AL
MOV DX, 2F81H
MOV AL, 0
OUT DX, AL
MOV AL, 10H
OUT DX, AL

```

【例 10.3】 某系统中 8253 的端口地址为 40H~43H, 要利用其通道 2 对 CLK₂ 上的外部输入脉冲进行计数, 计满 100 个后向 CPU 发中断请求, 试写出相应初始化程序。

本例中采用二进制计数, 则初值 100 为 64H, 写入时只写低 8 位即可。由于要向 CPU 发中断申请, 我们设置通道 2 工作在方式 0, 这样计数结束时的正跳变信号可作为中断请求信号。控制字的格式应为:

```

10010000B=90H
MOV AL, 90H
OUT 43H, AL
MOV AL, 64H
OUT 42H, AL

```

3. 8253 在 PC/XT 中的应用

PC/XT 中, 使用了一片 8253, 其地址范围为 40H~43H, 3 个 CLK 的输入均为 1.19MHz, GATE₀ 和 GATE₁ 接 +5V 电源, GATE₂ 由 8255 的 PB₀ 控制(参见 10.4 节中图 10.24)。3 个计数通道的作用及 BIOS 中初始化程序分别为:

(1) 计数器 0: 编程为方式 3, 每 55ms 向中断控制器的 IRQ₀ 引脚发送一次中断请求信号, 用于 CPU 计时和磁盘驱动器的定时。初始化程序如下:

```

MOV AL, 00110110B    ; 通道 0 方式控制字
OUT 43H, AL
MOV AL, 0
OUT 40H, AL           ; 计数初值为 65536
OUT 40H, AL

```

(2) 计数器 1: 工作于方式 2, OUT₁ 输出接至 DMA 请求触发器的 CP 端, 每隔 15.12μs 请求一次 DMA 操作, 进行动态 RAM 的刷新。初始化程序如下:

```

MOV AL, 01010100B
OUT 43H, AL
MOV AL, 12H
OUT 41H, AL

```

(3) 计数器 2 用于产生方波驱动扬声器发声。BIOS 中有一个发声子程序 BEEP, 它对计数通道 2 的初始化代码为:

```

MOV AL, 10110110B
OUT 43H, AL
MOV AX, 0533H
OUT 42H, AL

```

```
MOV AL, AH  
OUT 42H, AL
```

10.4 并行接口芯片 8255A

CPU 与外设间的数据传送都是通过接口来实现的。CPU 与接口的数据传输总是并行的，即一次传输 8 位或者 16 位，而接口与外设间的数据传输则可分为两种情况：串行传送与并行传送。串行传送是数据在一根传输线上一位一位地传输，而并行传输是数据在有多根传输线上一次以 8 位或 16 位为单位进行传输。与串行传输相比，并行传输需要较多的传输线，成本较高，但传输速度快，尤其适用于高速近距离的场合。

能实现并行传输的接口称为并行接口，并行接口分为不可编程并行接口与可编程并行接口。不可编程并行接口通常由三态缓冲器及数据锁存器等搭建而成，这种接口的控制比较简单，但要改变其功能必须改变硬件电路。可编程接口的最大特点是其功能可通过编程设置和改变，因而具有极大的灵活性。

10.4.1 8255A 概述

Intel-8255A 是一种通用的可编程并行 I/O 接口芯片，又称“可编程外设接口芯片”(Programmable Peripheral Interface, PPI)，可通过软件编程的方法分别设置它的 3 个 8 位 I/O 端口的工作方式，通用性强，使用灵活。通过 8255A，CPU 可以和大多数并行传输的外设直接连接，是应用最广的典型可编程并行接口芯片。

1. 8255A 的主要特性

8255A 是通用的 8 位并行输入/输出接口芯片，使用灵活，功能强大，具有如下特点：

(1) 8255A 具有 3 个 8 位的数据口(A 口、B 口和 C 口)，其中 C 口还可当作两个 4 位口来使用。三个数据口均可用来输入或输出。

(2) 8255A 具有 3 种工作方式：方式 0、方式 1 和方式 2。可适应 CPU 与外设间的多种数据传输方式，如无条件传送方式、查询传送方式、中断方式等。

(3) 8255A 的 C 口还具有按位置 0 与置 1 功能。

2. 8255A 的内部结构

8255A 内部结构如图 10.15 所示，它由以下几部分组成。

1) 数据总线缓冲器

该缓冲器为 8 位、双向、三态的缓冲器，直接与系统数据总线相连，是 CPU 与 8255A 间传送数据的必经之路。各种命令字的写入及状态字的读取，都是通过该数据总线缓冲器传送的。

2) 读/写控制逻辑

CPU 通过输入和输出指令，将地址信息和控制信息送至该部件，由该部件形成对端口的读/写控制，并通过 A 组控制和 B 组控制电路实现对数据、状态和控制信息的传输。

3) A 组和 B 组控制

A 口及 C 口的高 4 位构成 A 组, B 口及 C 口的低 4 位构成 B 组。A 组控制和 B 组控制接收 CPU 写入的控制字, 并据此决定 A 组端口及 B 组端口的工作方式。

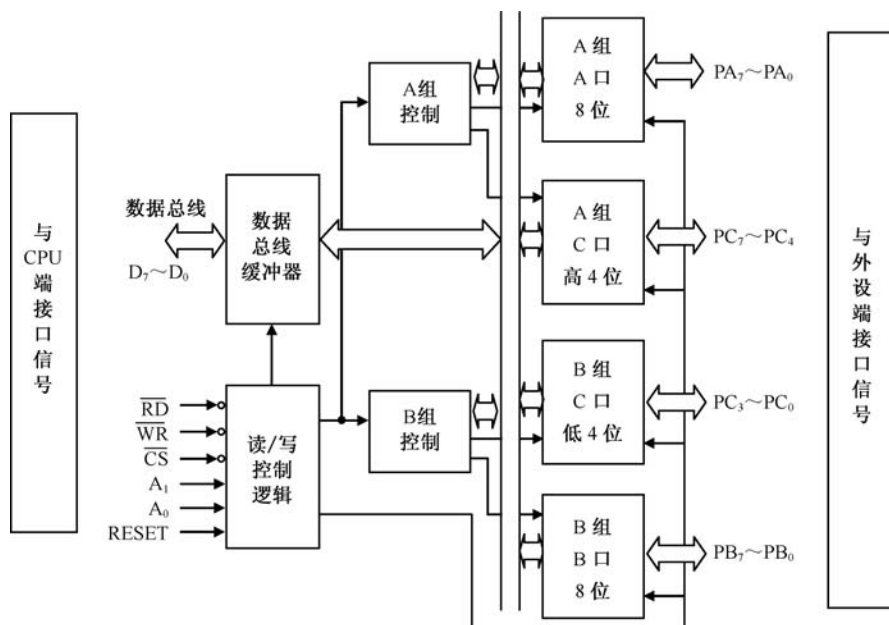


图 10.15 8255A 的内部结构

4) 数据端口 A、B、C

8255A 有 3 个 8 位数据端口, 分别具有如下特点:

(1) A 口具有一个 8 位数据输入锁存器和一个 8 位数据输出锁存器/缓冲器。端口 A 无论用作输入口还是输出口, 其数据均能被锁存。

(2) B 口具有输出锁存器/缓冲器和输入缓冲器。作为输入口时, 它不具备锁存能力, 因此外设输入的数据必须维持到被微处理器读取为止。

(3) C 口具有输出锁存器/缓冲器和输入缓冲器。C 口除用作输入和输出口外, 在方式 1 及方式 2 下, 其部分引脚要作为 A 口及 B 口的联络信号用, 具体情况在工作方式中介绍。

3. 8255A 的外部引脚

8255A 芯片采用 NMOS 工艺制造, 是一个 40 引脚双列直插式(DIP)封装组件, 其引脚排列如图 10.16 所示。

1) 与微处理器连接的信号线

$D_7 \sim D_0$: 数据线, 三态双向 8 位, 与系统的数据总线相连。

$\overline{CS}$: 片选信号, 低电平有效。为低电平时, 8255A 才能接受 CPU 的读/写。

$\overline{WR}$: 写信号, 低电平有效。为低电平时, CPU 可以向 8255A 写入数据或控制字。

$\overline{RD}$: 读信号, 低电平有效。为低电平时, 允许 CPU 从 8255A 读取各端口的数据和状态。

A_1, A_0 : 端口地址选择信号。用于选择 8255A 的 3 个数据端口和一个控制端口。 A_1A_0

=00 选择 A 口, $A_1A_0=01$ 选择 B 口, $A_1A_0=10$ 选择 C 口, $A_1A_0=11$ 选择控制口。

RESET: 复位信号, 高电平有效。为高电平时, 8255A 所有的寄存器清 0, 所有的输入/输出引脚均呈高阻态, 3 个数据端口置为方式 0 下的输入端口。

2) 8255A 与外部设备连接的信号线

$PA_7 \sim PA_0$: A 口与外部设备连接的 8 位数据线, 由 A 口的工作方式决定这些引脚用作输入/输出或双向。

$PB_7 \sim PB_0$: B 口与外部设备连接的 8 位数据线, 由 B 口的工作方式决定这些引脚用作输入/输出。

$PC_7 \sim PC_0$: C 口 8 位输入/输出数据线, 这些引脚的用途由 A 组、B 组的工作方式决定。

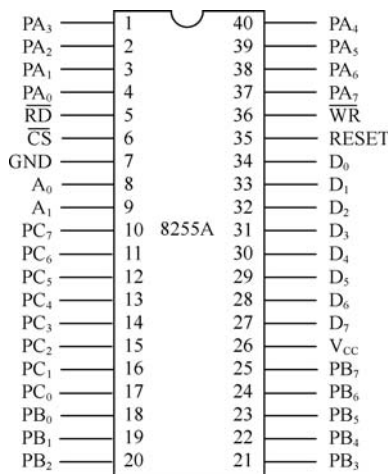


图 10.16 8255A 引脚

10.4.2 8255A 的控制字

控制字用来设置 8255A 的工作方式, 8255A 有二个控制字, 方式选择控制字和 C 口按位置位/复位控制字。这二个控制字写入同一端口地址($A_1A_0=11$), 为了进行区分, 控制字的 D_7 位作为标位, $D_7=1$ 表示是工作方式控制字; $D_7=0$ 表示是按位置位/复位控制字。

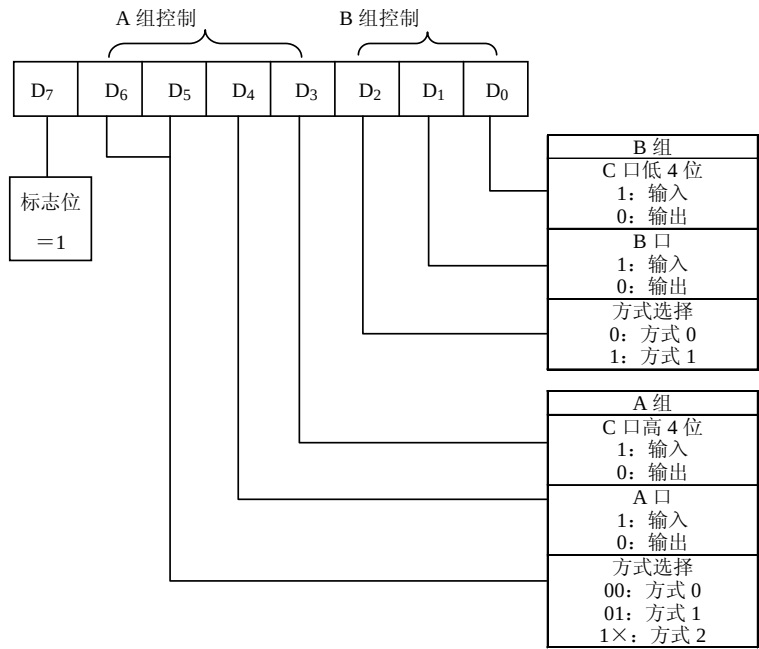
1. 工作方式控制字

8255A 有 3 种工作方式: 方式 0 基本输入/输出方式, 方式 1 选通输入/输出方式(应答方式)和方式 2 双向传送方式。8255A 各数据端口的工作方式由方式选择控制字进行设置。对 8255 进行初始化编程时, 通过向控制寄存器写入方式选择控制字, 可以让 3 个数据端口以需要的方式工作。工作方式控制字的格式如图 10.17 所示。

【例 10.4】 设 8255A 的端口地址为 60H~63H, 要求 A 组工作在方式 0, A 口输出, C 口高 4 位输入; B 组工作在方式 1, B 口输出, C 口低 4 位输入, 则对应的工作控制方式字为: 10001101B 或 8DH。

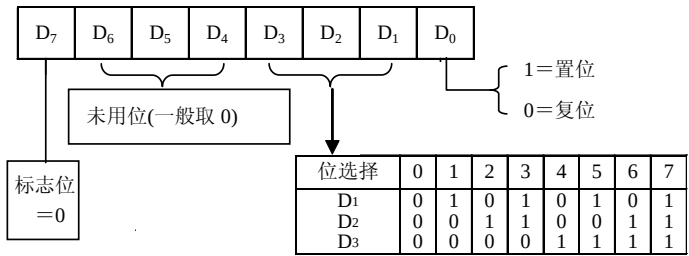
初始化程序如下:

```
MOV AL, 8DH      ; 设置方式字
OUT 63H, AL      ; 送到 8255A 控制字寄存器中
```



2. C 口按位置位/复位控制字

按位置位/复位控制字的作用是使 C 口的某一引脚输出特定的电平状态(高电平或低电平), 控制字的格式如图 10.18 所示。



说明: (1) 仅 C 口可按位置位/复位, 且只对 C 口的输出状态进行控制(对输入无作用)。

(2) 一次只能设置 C 口 1 位的状态。

(3) 这个控制字应写入控制口, 而不是 C 口。

【例 10.5】 要使 PC₇ 置 1, PC₃ 置 0, 设 8255A 的地址为 320H~323H, 则程序为:

```
MOV AL, 0FH      ; PC7 置 1 的控制字
MOV DX, 323H
OUT DX, AL
MOV AL, 06H      ; PC3 置 0 的控制字
OUT DX, AL
```


10.4.3 8255A 的工作方式

8255A 芯片有 3 种工作方式：方式 0、方式 1 和方式 2，下面介绍这 3 种方式的特点。

1. 方式 0：基本输入/输出

在方式 0 下，A、B、C 三个端口均作为输入/输出端口，这种输入/输出只是简单的输入/输出，无联络信号。如图 10.19 所示。

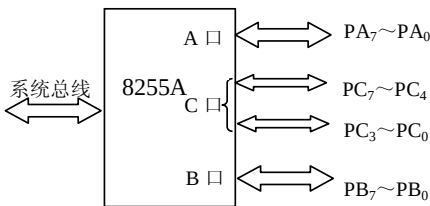


图 10.19 8255A 工作方式 0

8255A 工作于方式 0 时，它具有以下功能：

(1) 具有两个 8 位端口，即 A 口、B 口，两个 4 位端口，即 C 口的高 4 位、低 4 位。每个端口都可设定为输入或输出，共有 16 种组合，由方式控制字确定，但每个端口不能同时既输入又输出。

(2) 输出具有锁存能力，输入只有缓冲能力，而无锁存功能。

注意：在方式 0 下，C 口的高、低 4 位可分别设定为输入或输出，但 CPU 的 IN 或 OUT 指令必须至少以一个字节为单位进行读写，为此必须采取适当的屏蔽措施，见表 10-2。

表 10-2 C 口读/写时的屏蔽措施

CPU 操作	高 4 位(A 组)	低 4 位(B 组)	数据处理
IN	输入	输出	屏蔽掉低 4 位
IN	输出	输入	屏蔽掉高 4 位
IN	输入	输入	读入的 8 位均有效
OUT	输入	输出	送出的数据只设在低 4 位上
OUT	输出	输入	送出的数据只设在高 4 位上
OUT	输出	输出	送出的数据 8 位均有效

2. 方式 1：带选通的输入/输出

方式一的特点是：仅 A 口、B 口可工作在这种方式下，A 口或 B 口可以为输入，也可以为输出，但不能既输入又输出。不论输入还是输出，都要占用 C 口的某些引脚作为联络信号用，并且这种占用关系是固定的。C 口未被占用的位仍可用于输入或输出(控制字的 D₃ 位决定)。

方式 1 下的数据输入/输出均具有锁存能力。

1) 方式 1 的输入

A 口和 B 口都设置为方式 1 输入时的情况, 如图 10.20 所示。

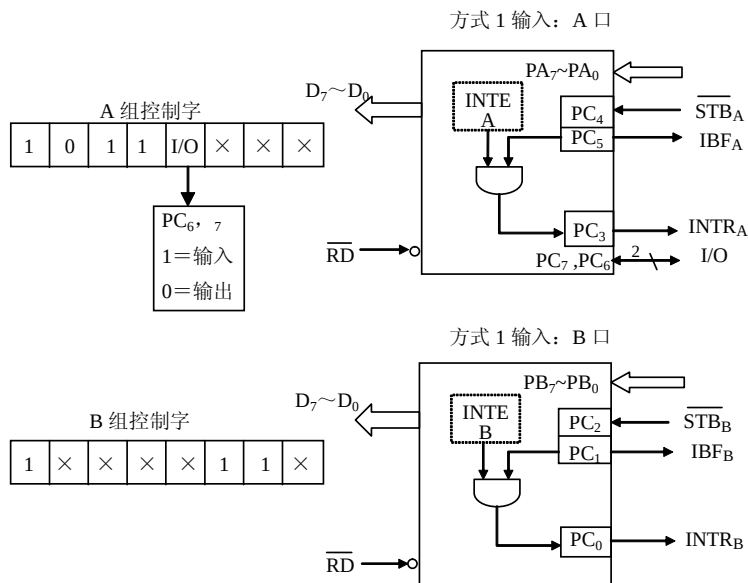


图 10.20 8255A 工作方式 1 输入

当 A 口设定为方式 1 输入时, A 口所用三条联络信号线是 C 口的 PC₃、PC₄、PC₅, B 口则用了 C 口的 PC₀、PC₁、PC₂ 作为联络信号。各联络线的定义如下:

STB: 外设送来的输入选通信号, 低电平有效。当有效时, 表示外设的数据已准备好, 同时将外设送来的数据锁存到 8255A 端口的输入数据缓冲器中。

IBF: 8255A 送外设的输入缓冲器满信号, 高电平有效。有效时, 说明外设数据已送到输入缓冲器中, 但尚未被 CPU 取走。该信号一方面可供微处理器查询用, 另一方面送给外设, 阻止外设发送新的数据。IBF 由 $\overline{\text{STB}}$ 信号置位, 由读信号的后沿将其复位。

INTR: 8255A 送到 CPU 或系统总线的中断请求信号, 高电平有效。当外设要向 CPU 传送数据或请求服务时, 8255A 用 INTR 端的高电平向 CPU 提出中断请求。INTR 变高的条件是: 当输入缓冲器满信号变为高即 IBF=1, 并且中断请求被允许即 INTE=1, 才能使 INTR 变高, 向 CPU 发出中断请求。

INTE: 中断允许信号。A 口用 PC₄ 的按位置位/复位控制, B 口用 PC₂ 的按位置位/复位控制。只有当 PC₄ 或 PC₂ 置 1 时, 才允许对应的端口送出中断请求。

下面以 A 口为例, 对方式 1 的工作过程描述如下:

当外设准备好数据, 并将数据送到数据线上后, 送出一个 $\overline{\text{STB}}$ 选通信号。8255A 的 A 口数据锁存器在 $\overline{\text{STB}}$ 的下降沿将数据锁存。8255A 向外设送出高电平 IBF 信号, 表示数据已锁存, 暂时不要再送数据。如果 PC₄=1, 这时就会使 INTR 变成高电平输出, 向 CPU 发出中断请求。CPU 响应中断, 执行 IN 命令时, $\overline{\text{RD}}$ 的下降沿清除中断请求, 而 $\overline{\text{RD}}$ 结束时的上升沿则使 IBF 复位到零。外设在检测到 IBF 为零后, 便开始发送下一数据。

2) 方式 1 的输出

A 口和 B 口都设置为方式 1 输出时的情况如图 10.21 所示。

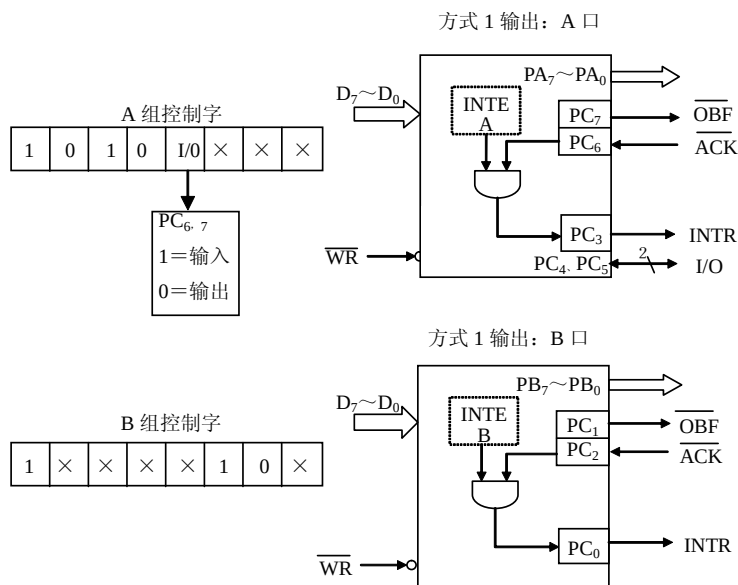


图 10.21 8255A 工作方式 1 输出

当 A 口与 B 口设为方式 1 输出时, 也分别指定 C 口的三条线为联络信号, A 口所用三条联络信号线是 C 口的 PC₃、PC₆、PC₇, B 口则用了 PC₀、PC₁、PC₂。各联络线的定义如下:

$\overline{\text{OBF}}$: 该信号是 8255A 输出给外设的输出缓冲器满信号, 低电平有效。当有效时, 表示 CPU 已将数据写到 8255A 的输出端口, 等待外设取走数据。

$\overline{\text{ACK}}$: 响应信号, 低电平有效, 由外设送来。当为低电平时, 表示 8255A 的数据已被外设取走, 是对 $\overline{\text{OBF}}$ 的回答信号。

INTR : 中断请求信号, 高电平有效。当外设取走数据后, 8255A 用 INTR 端向 CPU 发中断请求, 请求 CPU 输出后面的数据。INTR 引脚上只有当 $\overline{\text{ACK}}$ 、 $\overline{\text{OBF}}$ 和 INTE 都为高电平时, 才能发出中断请求, 该请求信号由 CPU 写操作时的 $\overline{\text{WR}}$ 上升沿复位。

INTE 中断允许触发器, A 口的 INTE 由对 PC₆ 的置位/复位设置, B 口的 INTE 由对 PC₂ 的置位/复位设置。

PC₄ 和 PC₅ 位可由控制字的 D₃ 位设置为输入或输出数据用。

方式 1 输出的信号交接过程说明如下:

CPU 通过执行 OUT 指令, 向 8255A 端口输出数据, 此时将产生 $\overline{\text{WR}}$ 有效信号。写操作完成后, $\overline{\text{WR}}$ 的上升沿使 $\overline{\text{OBF}}$ 变低, 表示输出缓冲器已满, 通知外设取走数据, 并且 $\overline{\text{WR}}$ 的上升沿使中断请求 INTR 变低, 即使之无效。外设取走数据后, 用一个有效的 $\overline{\text{ACK}}$ 信号回答 8255A, $\overline{\text{ACK}}$ 的下降沿使 $\overline{\text{OBF}}$ 变高(无效)。如果 $\text{INTE}=1$, 则 $\overline{\text{ACK}}$ 负脉冲的下降沿再使 INTR 变高(有效), 产生中断请求。CPU 可在中断服务程序中向 8255A 输出下一个数据。

3. 方式 2: 带选通的双向输入/输出

方式 2 是一种双向选通输入/输出方式, 只适用于 A 口。所谓双向输入/输出, 就是 A

口既可输入，又可输出。在方式 0 或方式 1 下，虽然 A 口、B 口也可以输入或输出，但一次只能设置为输入或输出一种状态，而不能既输入又输出，这是方式 2 与前二种方式的主要区别。方式 2 下 C 口的 5 条线(PC₃~PC₇)作为 A 口的联络线，B 口只能工作在方式 0 或方式 1 下。若 B 口工作在方式 1 下，C 口的 3 位(PC₀~PC₂)作为其联络线；若 B 口工作在方式 0 下，PC₀~PC₂可作为输入/输出线。方式 2 下的引脚定义如图 10.22 所示。

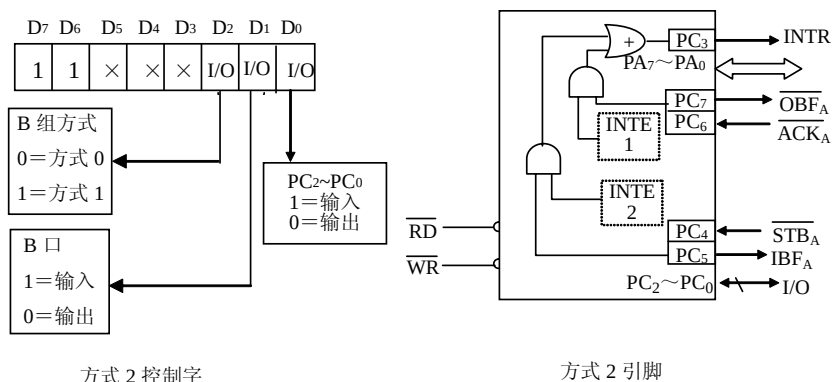


图 10.22 8255A 工作方式 2

在方式 2 下，各联络信号的含义如下：

INTR：中断请求信号，高电平有效。不管是输入还是输出，都由这个信号向 CPU 发中断申请。

OBF：输出缓冲器满，低电平有效。其作用等同于方式 1 输出时的 $\overline{\text{OBF}}$ 。

ACK：来自外设的响应信号，低电平有效。其作用等同于方式 1 输出时的 $\overline{\text{ACK}}$ 。

INTE1：A 口输出中断允许，由 PC₆ 置位/复位。当 INTE1 为 1 时，8255A 输出缓冲器空时通过 INTR 向微处理器发出输出中断请求信号；当 INTE1 为 0 时，屏蔽输出中断。

STB：来自外设的选通输入，低电平有效。其作用等同于方式 1 输入时的 $\overline{\text{STB}}$ 。

IBF：输入缓冲器满，高电平有效。其作用等同于方式 1 输入时的 IBF。

INTE2：A 口输入中断允许，由 PC₄ 置位/复位。当 INTE2 为 1 时，8255A 输入缓冲器满时通过 INTR 向微处理器发出输入中断请求信号；当 INTE2 为 0 时，屏蔽输入中断。

以上介绍了 8255A 的三种工作方式，它们分别应用于不同的场合。方式 0 可用于无条件输入或输出的场合，如读取开关量、控制 LED 显示等；方式 1 提供了联络信号，可用于查询或中断方式输入或输出的场合；方式 2 是一种双向工作方式，如果一个外设既是输入设备，又是输出设备，并且输入和输出是分时进行的，那么将此设备与 8255A 的 A 口相连，并使 A 口工作在方式 2 就非常方便，如磁盘就是一种这样的双向设备。微处理器既能对磁盘读，又能对磁盘写，并且读和写在时间上是不重合的。

10.4.4 8255A 编程

8255A 工作时首先要初始化，即要写入控制字来指定其工作方式。如果需要中断，还要用 C 口按位置位/复位控制字将中断标志 INTE 置 1 或置 0。初始化完成后，就可对 3 个数据端口进行读/写。

【例 10.6】如图 10.23 所示, 设 8255A 端口地址为 2F80~2F83H, 编程设置 8255A, A 组、B 组均工作于方式 0, A 口输出, B 口输出, C 口高 4 位输入, 低 4 位输出。然后, 读入开关 S 的状态, 若 S 打开, 则使发光二极管熄灭; 若 S 闭合, 则使发光二极管点亮。

```

MOV AL, 88H
MOV DX, 2F83H
OUT DX, AL
MOV DX, 2F82H
IN AL, DX
MOV DX, 2F81H
AND AL, 20H
JZ L1
; 条件成立时 PC5=0, S 闭合
MOV AL, 0
OUT DX, AL
JMP END1
L1:
MOV AL, 40H
OUT DX, AL
END1:
HLT

```

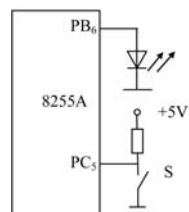


图 10.23 例 10.3 图

【例 10.7】PC 机的扬声器驱动系统如图 10.24 所示。扬声器的发声控制系统由 8255A PB 口的 D₀、D₁ 位与 8253 计数器的计数通道 2 共同控制。8255A 的端口地址为 60H~63H, 8253 的端口地址为 40H~43H。

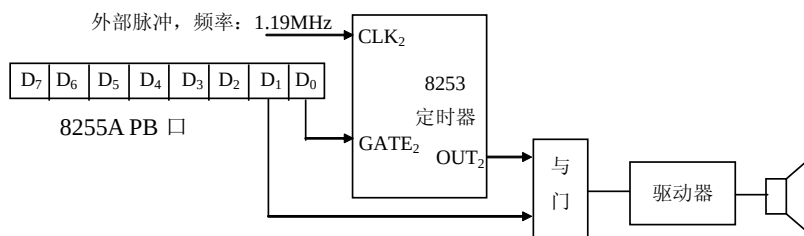


图 10.24 PC 机扬声器系统

扬声器发声有两种方式:

(1) 直接对 8255A 的 PB 口的 D₁ 位交替输出 0 和 1, 使扬声器交替地通与断, 推动扬声器发声。这种方式发声频率不好控制, 只能用于简单发声。

(2) 定时器控制发声: 让 8255A PB 口的 D₁、D₀ 位输出 1, 对 8253 编程, 使其在 OUT₂ 上输出指定频率的方波, 以驱动扬声器发声。这种方式好控制发声频率, 可满足较复杂的发声要求(但 PC 机中未提供控制音量的手段)。

由于扬声器总是随时可用的, 故 CPU 可用直接 I/O 方式对其操作。

采用第一种方法时 PC 机的发声程序如下:

```

CODE SEGMENT
ASSUME CS: CODE
START:
    MOV DX, 1000H    ; 开关次数
    IN AL, 61H       ; 取端口 61H 的内容
    PUSH AX          ; 入栈保存, 以便退出时恢复

```

```
        AND AL, 11111100B    ; 将第 0、1 位置 0
SOUND:
XOR AL, 2                    ; D1 位取反
OUT 61H, AL                  ; 输出到端口 61H
MOV CX, 2000H                ; 设置延时空循环的次数
DELAY:
LOOP    DELAY                ; 空循环, 延时一小会儿
DEC DX                       ; 共 1000H 次
JNZ SOUND
POP AX                       ; 从堆栈中弹出原 AX 内容
OUT 61H, AL                  ; 恢复原 61H 端口内容
MOV AH, 4CH
INT 21H                      ; 返回 DOS
CODE ENDS
END START
```

【例 10.8】 8255A 在并行打印机接口中的应用。

打印机内有一个以 8 位专用微处理器为核心的打印机控制器, 负责打印功能的处理, 以及打印机本身的管理, 并通过机内一个标准接口(Centronice 并行接口)与主机进行通信, 接收主机送来的打印数据和控制命令, 该接口位于打印机内, 采用多芯电缆与主机内的打印机接口电路(打印机适配器)相连。多芯电缆上的信号有数据信号、CPU 的命令信号和打印机状态信号等, 主要信号如表 10-3 所示。

表 10-3 打印机的主要接口信号

信 号	含 义	方 向	说 明
DATA ₈₋₁	数据 1-8	输入	主机送给打印的 8 位数据
$\overline{\text{STROBE}}$	选通脉冲	输入	负极性脉冲, 主机发出, 用于将 DATA ₁ ~DATA ₈ 上的数据置入打印机的缓冲器, 脉宽>0.5μs
$\overline{\text{SLCT IN}}$	选择输入	输入	只有该信号为低电平时, 打印机才能接受 DATA ₁ ~DATA ₈ 上的数据
$\overline{\text{AUTO FD XT}}$	自动走纸	输入	当该信号为低电平时, 每当打印完后(即遇到回车符), 打印机自动前进一步
$\overline{\text{INIT}}$	打印机初始化	输入	当该信号为低电平时, 打印控制器复位, 并清除打印缓冲器。在打印机处于接收数据和打印状态时, 该信号为高电平
$\overline{\text{ACK}}$	应答	输出	负脉冲, 宽度约为 5μs, 作为打印机已接收到一个数据的回答信号, 并准备好接收下一个数据
BUSY	忙	输出	若为高电平, 表示打印机当前忙, 不能接收数据。下列情况下该信号有效: 数据输入期间、打印操作期间、脱机状态、出错状态
PE	无打印纸	输出	高电平表示打印机缺纸
SLCT	选择状态	输出	高电平表示脱机状态, 低电平表示联机状态
ERROR	打印机出错	输出	当该信号为低电平时表示打印机出错或脱机或缺纸

打印机接口的时序关系如图 10.25 所示。从图中可以看出, 当主机需要打印一个数据时, 打印机接收主机传送数据的过程是:

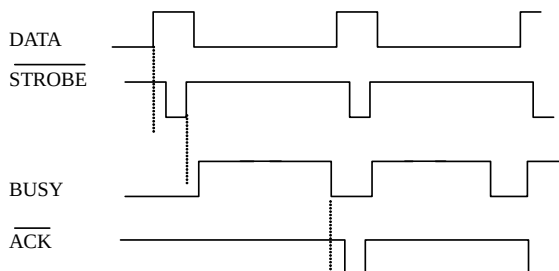


图 10.25 打印机接口时序

(1) 首先查询 **BUSY** 信号。若 **BUSY**=1(忙), 则等待; 当 **BUSY**=0(不忙)时, 才能送出数据。

(2) 将数据送到数据线上, 但此时数据并未自动进入打印机。

(3) 再送出一个数据选通信号 **STROBE** 给打印机, 此后数据线上的数据将进入打印机的内部缓冲器。

(4) 打印机发出“忙”信号, 即置 **BUSY**=1, 表明打印机正在处理输入的数据。等到输入的数据处理完毕(打印完 1 个字符或执行完 1 个功能操作), 打印机撤销“忙”信号, 即置 **BUSY**=0。

(5) 打印机送出一个回答信号 **ACK** 给主机, 表示上一个字符已经处理完毕。

以上是采用查询方式传送数据的过程。若采用中断方式传送数据时, 可利用 **ACK** 信号来产生中断请求, 在中断服务程序中送出下一个打印数据。如此重复工作, 就可以正确无误地把全部字符打印出来。

打印机接口电路如图 10.26 所示。

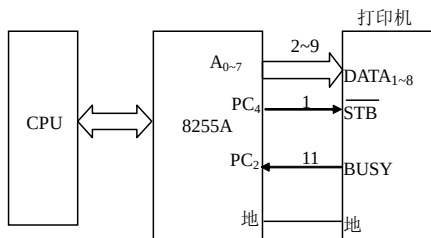


图 10.26 打印机接口电路原理图

该电路的设计思想是: 8255A 的 A 口、C 口工作于方式 0, A 口用来输出 8 位打印数据, C 口的 PC₄ 引脚用来产生 **STROBE** 信号, PC₂ 引脚用来接收 **BUSY** 信号。

设 8255A 的端口地址 80H~83H, 能打印一个字符的程序如下:

```

MOV AL, 81H                ; 8255A 工作方式控制字
OUT 83H, AL
MOV AL, 09H
OUT 83H, AL                ; 置 PC4 为高, 即使 STROBE = 1
Busy: IN AL, 82H           ; 读 C 口
AND AL, 04H                ; 查 PC2 = 0?
JNZ Busy                   ; 忙则等待, 不忙则向 A 口发送数据
MOV AL, 'A'                ; 被打印字符为 "A"
OUT 80H, AL                ; 送出打印数据

```

```
MOV AL, 08H          ; 置 PC4 为低, 以便产生负脉冲
OUT 83H, AL
NOP
NOP
MOV AL, 09H          ; 再使 PC4 变为高, 形成负脉冲
OUT 83H, AL
```

10.5 DMA 控制器 8237A

DMA 传送方式是计算机系统交换数据的一种重要方式, 其特点已在第 10.2 节中作了介绍。本节重点介绍实现 DMA 操作的关键器件 DMA 控制器。

10.5.1 概述

DMA 控制器一方面可以控制系统总线, 这时称它为总线主模块, 另一方面又可以和其他接口一样, 接受 CPU 对它的读/写操作, 这时 DMA 控制器就成了总线从模块。

作为总线主模块时, DMA 控制器必须完成以下功能:

- (1) 能够响应外设的 DMA 请求, 使 CPU 暂停工作, 接管总线控制权。
- (2) 能够按节拍提供外设和存储器工作的各种控制信号。
- (3) 提供存储器地址。

当 DMA 控制器在进行 DMA 过程的控制时, 称为处于 DMA 周期, 它就作为系统总线的主控设备。而在不进行 DMA 传输时, 称为空闲周期, DMA 控制器就作为 CPU 的一个外设。

8237A 是 Intel 公司生产的高性能可编程 DMA 控制器(DMAC), 与 Intel 公司的各种微处理器配合使用。可以实现外设直接访问系统的存储器, 能提供多种可编程控制特性, 实现动态控制, 增大了数据的吞吐量, 从而大大改善了微型计算机的性能。

8237A 工作时钟为 3MHz, 8237A-4 为 4MHz, 8237A-5 为 5MHz, 在使用上基本没有差别。

8237A 的主要技术特性:

- (1) 有 4 个完全独立的 DMA 通道, 可以分别编程控制 4 个不同的 DMA 操作对象。每个通道都允许开放或禁止 DMA 请求, 都可以独立控制自动预置方式。
- (2) 具有三种基本的传送方式: 单字节传送、数据块传送和请求传送。
- (3) 具有存储器到存储器的传送功能。
- (4) 可以用级联的方法无限扩展 DMA 通道数。
- (5) 具有正常时序和压缩时序两种基本时序。
- (6) 有两种优先级管理方式: 固定优先级和循环优先级。
- (7) 有较高的数据传输速率, 允许 DMA 传输速度高达 1.6MB/s。

10.5.2 8237A 引脚

1. 8237A 的引脚特点

8237A 采用 40 引脚双列直插封装(见图 10.27)。8237A 的引脚主要围绕其二种工作状态

而设置, 有一些引脚在两种状态下都有用, 而有些则是专用的。

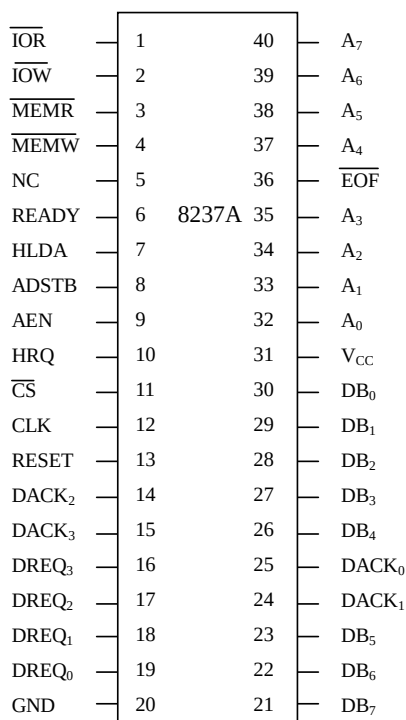


图 10.27 8237A 引脚

2. 与 DMA 周期有关的引脚

CLK: 时钟输入信号。

$\text{DREQ}_0 \sim \text{DREQ}_3$, $\text{DACK}_0 \sim \text{DACK}_3$: DMA 请求及响应信号。DMA 请求信号是由外设发出的信号, 它要求进行一次 DMA 传送。这个输入信号的有效极性是可编程的, 在芯片总清除后, 它以高电平为有效。DACK 则是 DMA 控制器通知外设可以开始 DMA 传送的信号, 信号有效的极性也是可以编程的, 清除后, 以低电平为有效。这是一对对应答信号, DREQ 必须保持到 DACK 有效值出现后才能撤销。

4 个通道的 DMA 请求可以安排不同的优先级判定法, 在固定方式下, DREQ_0 具有最高优先级。

HRQ: 总线请求信号, 输出。8237A 收到外设接口发来的 DREQ 信号后, 如果该通道的 DMA 请求没有被屏蔽, 则 DMA 控制器的 HRQ 端输出有效电平, 向微处理器发出总线请求。

HLDA: 总线响应信号, 输入。8237A 向 CPU 发总线请求信号后。CPU 给出 HLDA 作为应答, 当它有效时, 表明 CPU 已经让出了总线, 8237A 获得了总线控制权。

$\text{A}_7 \sim \text{A}_0$: 低 8 位地址。三态, 从 8237A 输出, 在 DMA 周期中用来对存储器寻址。其中 $\text{A}_3 \sim \text{A}_0$ 4 位是双向地址, 因为当 CPU 对 8237A 编程时, 这 4 个地址引脚又要作为对片内寄存器寻址的输入地址用。

$\text{DB}_7 \sim \text{DB}_0$: 数据总线。DMAC 作为主模块时, $\text{DB}_7 \sim \text{DB}_0$ 输出当前地址寄存器中的高 8 位地址, 并通过信号 ADSTB 打入外部锁存器, 这样, $\text{A}_7 \sim \text{A}_0$ 输出的低 8 位地址一起构

成 16 位地址。和 8086 CPU 的情况类似, 需要先把输出地址锁存起来, 作为对存储器寻址的高 8 位地址, 然后这组线才传送数据。

ADSTB: 地址选通信号。高电平有效的输出信号, 它只在 $DB_7 \sim DB_0$ 上出现地址的那段时间内有效。所以这个信号用来对 $A_{15} \sim A_8$ 地址信号进行锁存, 使它能送出并保持一段时间。

AEN: 地址使能信号。高电平有效的输出信号。AEN 使外部地址锁存器中锁存的高 8 位地址送到地址总线上, 与芯片直接输出的低 8 位地址共同构成了被访问存储器地址的偏移量。AEN 也使与 CPU 相连的地址锁存器无效, 这样, 就保证了地址总线的信号是来自 DMA 控制器而非来自 CPU。

顺便说一下, 8086 系统中使用的是 20 位地址信号, 而 8237A 只能产生 16 位地址信号, 被访问存储器的高 4 位地址信号须由 CPU 事先写入另外的页面寄存器中保存, 该页面寄存器的输出也由 AEN 控制。

MEMR: 存储器读信号, 低电平有效。此信号有效时, 所选中的存储器单元的内容被读到数据总线。

MEMW: 存储器写信号, 低电平有效。此信号有效时, 数据总线上的内容被写入选中的存储器单元。

IOR: DMAC 作为主模块时, $\overline{IOR}$ 输出外设接口的读控制信号, 此信号有效时, I/O 接口部件中的数据被读出送往数据总线。

IOW: DMAC 作为主模块时, $\overline{IOW}$ 输出外设接口的写控制信号, 此信号有效时, 存储器中读出的数据被写入 I/O 接口中。

READY: 就绪输入信号。其作用与 CPU 中的 READY 引脚一样, 在 DMA 操作中, 若外设速度太慢, 可在该端输入低电平, 8237A 将插入一些等待周期, 直到 READY 端恢复高电平后恢复正常节拍。

EOP: 过程结束信号。过程结束信号是个低电平有效的双向信号。8237A 允许用一个外部信号来终止一次有效的 DMA 服务, 这个外部信号就是从 $\overline{EOP}$ 端输入的一个低电平信号。当任何一个通道计数到 0, DMA 传送完成时, 8237A 也将从 $\overline{EOP}$ 端输出一个低电平信号。不管是接收外部的 $\overline{EOP}$ 还是内部产生的 $\overline{EOP}$, 都将终止 DMA 操作, 并清除请求。如果在编程时设置了自动启动功能, 这时就将把基本寄存器的内容自动写入该通道暂存器中。

3. 与 CPU 读、写有关的引脚

CS: 片选信号输入端, 低电平有效, 有 $A_4 \sim A_{15}$ 译码得到。 $\overline{CS}$ 为低电平时, 8237A 被选中, 微处理器可以对 8237A 进行读、写(进行预置或读取工作状态)。

$A_3 \sim A_0$: 最低的 4 位地址线, 它们是双向信号引脚。DMA 控制器作为从模块时, $A_3 \sim A_0$ 作为输入端, 用来选择 8237A 内部有关寄存器的 16 个端口地址, 为了方便, 常略去高位地址, 直接用 $0H \sim FH$ 作为寄存器地址。

$DB_7 \sim DB_0$: 8 位双向三态数据线。在 DMA 控制器作为从模块时, 微处理器通过 $DB_7 \sim DB_0$ 对 8237A 进行读、写。

IOR: 输入/输出设备读信号, 双向、三态、低电平有效。在 DMA 控制器作为从模块时, $\overline{IOR}$ 为输入/控制信号。此信号有效时, 微处理器读取 DMA 控制器中内部寄存器的值。

IOW: 输入/输出设备写信号。此信号有效时, 从系统总线送来, 实现 CPU 向 8237A

内部寄存器中写入信息。

RESET: 复位输入端, 高电平有效。通常与系统的清除信号联在一起, 一通电即自行清除。它把命令、状态、请求和暂存寄存器都清 0, 还清除先/后触发器, 置位屏蔽寄存器。清除后, 8237A 处于空闲周期。

10.5.3 8237A 内部结构与寄存器

1. 概述

8237A 内部结构框图如图 10.28 所示, 图中标明了有关寄存器和逻辑。8237A 寄存器数量众多, 理解这些寄存器的作用是正确使用 8237A 的关键。表 10-4 列出了 8237A 寄存器的名称、特性和地址, 这个地址是由前述 $A_3 \sim A_0$ 在空闲周期中由 CPU 提供的, 没有考虑 I/O 端口地址的高位编码。后面提到的各寄存器地址都指的是芯片内部地址。

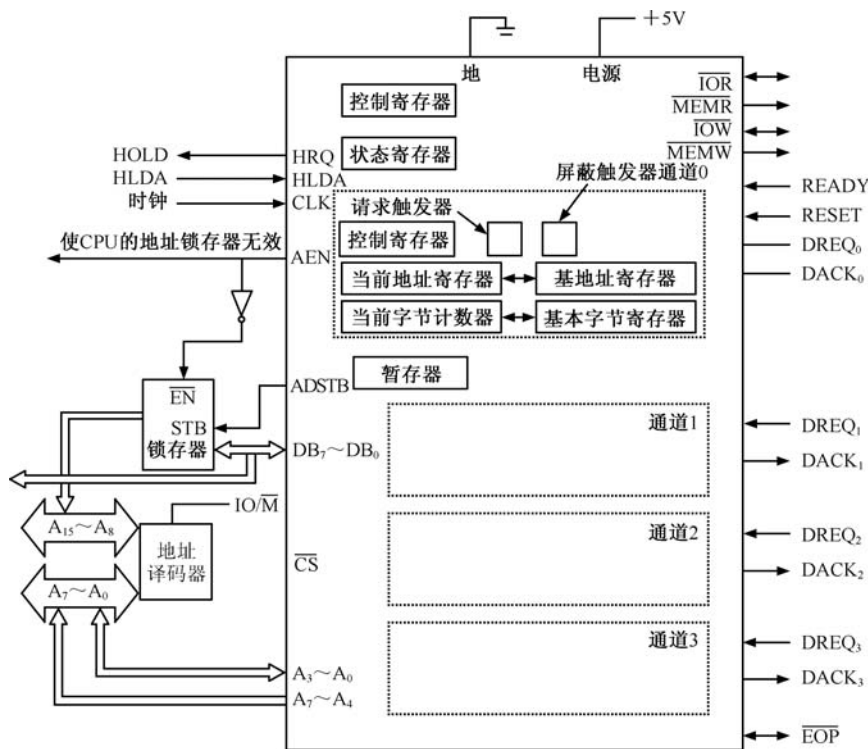


图 10.28 8237A 的内部编程结构和外部连接

2. 地址和字数寄存器

8237A 的地址和字数寄存器包括基地址寄存器、基本字节数寄存器、现行地址寄存器和现行字节数寄存器, 每个通道都必须有这 4 个寄存器, 所以总数达 16 个。其中两种基本寄存器只能写入, 而两种现行寄存器只能读(初始化时写入两种基本寄存器的内容自动写入到两种现行寄存器中, 故它们不必写入)。它们总是占用 8 个 I/O 端口地址, 具体地址见表 10-4, 地址自左至右排列分别对应通道 0~通道 3。

表 10-4 8237A 内部寄存器、命令与地址关系表

寄存器或命令	地 址
基地址寄存器(16 位, 共 4 个)	0H, 2H, 4H, 6H
基本字节数寄存器(16 位, 共 4 个)	1H, 3H, 5H, 7H
现行地址寄存器(16 位, 共 4 个)	0H, 2H, 4H, 6H
现行字节数寄存器(16 位, 共 4 个)	1H, 3H, 5H, 7H
暂存地址寄存器(16 位)	
暂存字数寄存器(16 位)	
状态寄存器(8 位)	8H(只读)
控制寄存器(8 位)	8H(只写)
暂存寄存器(8 位)	0DH(读操作)
工作方式寄存器(8 位)	0BH
屏蔽寄存器(4 位)	0AH
DMA 请求寄存器(4 位)	9H
清先/后触发器命令	0CH
主清除命令	0DH(写操作)
清屏蔽寄存器命令	0EH
综合屏蔽命令	0FH

在 DMA 传输时, 通常是以一个数据块作为一次传输的单位, 确定这个数据块的大小以及在内存中的存放地址, 以控制传送进行, 就是这 16 个寄存器的任务。

基地址寄存器是由 CPU 用程序控制写入的, 表示数据块在内存中的地址, 这个值一旦写入, 在整个传输进行过程中保持不变。

基本字节数寄存器中写入的是本次传输的数据块字节数, 这个数也不会随 DMA 传输的进行而变化。

两个基本寄存器写入内容时, 相应的现行地址寄存器也写入同样的内容。在 DMA 传送过程中, 每传送一个字节, 这两个现行寄存器的内容就变化一次。现行地址寄存器的变化方向由编程时的设置决定, 而现行字节数寄存器则恒作减量计数。当现行字数寄存器减至 0 时, 意味着此次传输已完成。

这一组寄存器的字长是 16 位, 但是供 CPU 读写的数据线却只有 8 位, 显然对一个寄存器的读写必须分两次进行。读、写时高、低字节的次序由一个触发器来控制, 详见后面的清先/后触发器命令。

3. 工作方式寄存器

8237A 的 4 个通道是完全独立的, 它们可以根据需要编程确定各自的工作方式, 所以工作方式寄存器应有 4 个, 每通道一个。图 10.29 是一个字节长的工作方式寄存器各位的含义。4 个方式寄存器只占用一个 I/O 端口地址, 不管哪个通道的方式字都用 0BH 地址写入, 利用方式字的最低两位 D₁D₀ 位的编码来指定该方式字属于哪个通道。

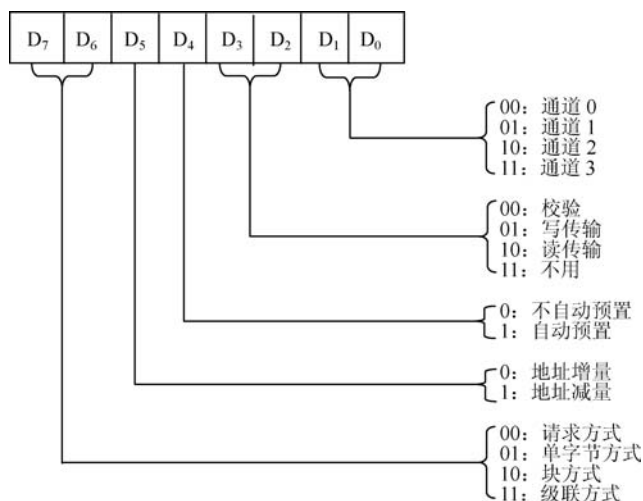


图 10.29 工作方式寄存器

D₇D₆: DMA 操作方式设定。共有 4 种方式，如下所述：

(1) 单字节方式：每次 DMA 传送只传输一个字节的数据，然后又把总线控制权还给 CPU。下一次传送必须重新向 CPU 提出请求才能进行，即使是连续地请求 DMA 操作，在两个字节传送中间至少也有一个总线周期是让 CPU 使用的。

(2) 成组传送方式：这种方法的特点是将一组数据的各字节连续传送，中间并不停顿，成组传送过程中，即使把 DMA 请求(外设来的)信号撤去，也能保证连续传送。当它计到 0 或外部输入有效的 $\overline{\text{EOP}}$ 信号时，此次传送结束，交回总线控制权。

(3) 请求方式：这也是一种成组传送方式，即要到一组数据全部传送完毕后才释放总线。但这种方式有一点与上面的成组传送方式不同，它要求在数据传送过程中外设送来的 DMA 请求信号 DREQ 一直保持有效。如果中间有一段时间这个信号变得无效，DMA 传送就被挂起，8237A 释放总线，CPU 可以继续操作，而 8237A 的地址和字节数的中间值，可以保存在相应的现行地址和字节数寄存器中。等到外设的 DREQ 请求信号变得有效时，DMA 传送将继续进行，直到一组传送结束。

(4) 级联方式：这种方式可以为系统提供更多的 DMA 通道。图 10.30 是级联的基本方法，这样在第二级上所接的外设提出的 DREQ ，要经两级 8237A 传播，并进行优先级判断，在得到 DACK 后才能开始传送。这时用于级联第一级的 8237A 就只用作优先级判断和 DREQ 请求的传送，本身不再向外输出信号去作 DMA 控制了。在级联使用时，前级的 8237A 是以通道为单位的，即可以使有些通道级联下级 8237A，有些通道不级联，仍作为单独的 DMA 控制器通道用。

D₅: 控制地址变化方向。如置“1”，每传送一个字节的数据，现行地址寄存器的内容减 1。反之，D₅=0，地址按增量方式变化。

D₄: 自动预置。如置“1”，将该通道设置成自动预置方式，即每当一次 DMA 传输结束后，2 个基本寄存器中的预置值将自动地再次写入 2 个现行寄存器中。要注意的是，如果一个通道被设置为自动预置方式，本通道对应的屏蔽位必须为 0。若 D₄=0，就不会自动产生这个动作。

D_3D_2 : 数据传送方向。写传输,是指数据从 I/O 设备写入到内存中去,读传输正好相反。校验传输是一种空操作,8237A 本身并不进行任何校验,而只是像 DMA 读或 DMA 写传输一样产生时序,产生地址信号,但是存储器和 I/O 控制线保持无效,所以并不进行传送,而外设可以利用这样的时序进行校验。

D_1D_0 : 通道选择。

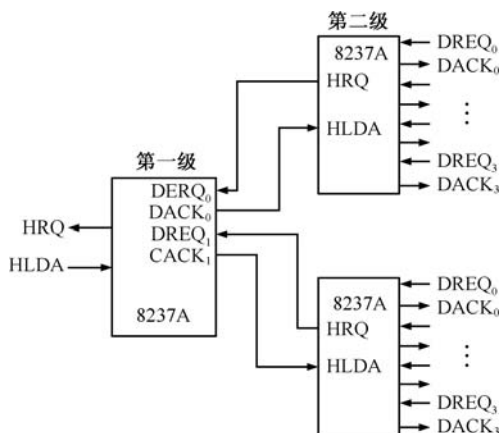


图 10.30 8237A 的级联使用

4. 控制寄存器

这是一个 8 位寄存器,用以控制 8237A 的一些工作信号,从而确定芯片的基本工作形态。CPU 可以用指令对它写入新的命令,而对原命令的清除也可以用总清或软件清除命令。寄存器中各位的定义如图 10.31 所示。

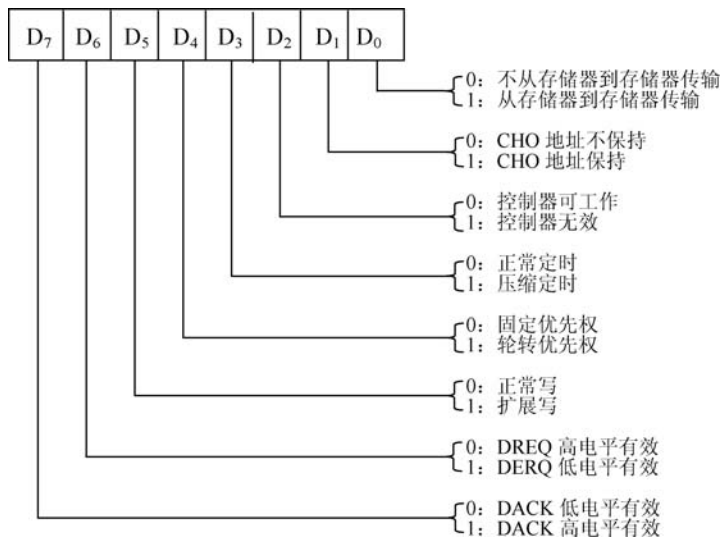


图 10.31 控制寄存器

D_6 、 D_7 : 分别控制 DREQ 和 DACK 有效的极性。 D_6 控制 DREQ, D_7 控制 DACK, 为“0”时设定有效电平为高电平, 为“1”时低电平才是有效的。一旦设定后, 4 个通道

的规定是一样的。

D₄: 选择不同的优先权。在固定优先权时, 通道 0 优先级最高, 通道 3 优先级最低。另一种优先权是循环式的, 刚被服务过的通道其优先权自动降为最低, 而其余各通道优先权依次升高一级, 从而使各通道的 DMA 被响应的可能性相同。

D₃ 和 D₅ 两位都与 8237A 的定时有关, 将在时序部分讨论。

D₂: 8237A 是否工作的控制位。如为“0”, 控制器可工作, 否则不工作。

D₁: 在存储器间传送数据时, 控制源地址变化与否。如果为“1”, 源地址在整个数据块传送中都保持不变。这种方式将使一个单元的内容送到规定长度的某一内存区域中去。如果 **D₀** 位为“0”, 即不作存储器之间的数据传输时, **D₁** 位就没有意义了。

D₀: 控制是否从存储器到存储器传输。若为 1, 是从存储器到存储器传输; 否则不是。

存储器间传送的操作只能用软件启动, 对请求寄存器(地址为 09H)写入 04H, 就可以用软件方法启动通道 0。当通道 0 输出 DACK 后, 就从存储器的源地址中取出一个字节数据, 存入 8237A 的暂存寄存器中。接着通道 1 从暂存寄存器中取出这个字节数据, 写入目的地址。地址的增、减量方式仍受各通道自己的方式字控制。传输字节数的控制使用通道 1 的计数器。传输结束时, 以 $\overline{\text{EOP}}$ 输出作为标志, 如果从 $\overline{\text{EOP}}$ 端输入一个负脉冲, 将终止正在进行的传输。

5. DMA 请求寄存器

8237A 的每个通道都配备了一个 DMA 请求触发器, 硬件 DREQ 线的有效电平会置位该触发器, 表示有 DMA 请求。也可以用软件设置 DMA 请求, 就是将请求字节写入请求寄存器。请求寄存器的格式如图 10.32 所示。

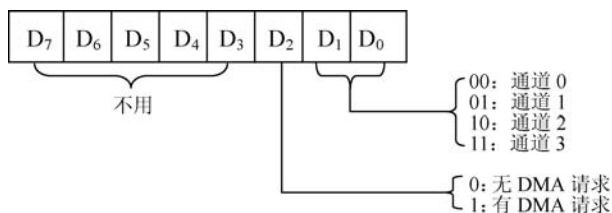


图 10.32 DMA 请求寄存器

D₁D₀: 选择通道。

D₂: 请求标志, “=1”表示有请求, “=0”表示无请求。

当 $\overline{\text{EOP}}$ 端有有效电平时, 将清除 DMA 请求标志, RESET 信号将清除所有寄存器。只有数据块方式允许使用软件请求, 若进行存储器到存储器的传输, 通道 0 必须使用软件请求, 以启动传输过程。

6. 屏蔽寄存器

8237A 的每个通道都配备了一个屏蔽触发器, 作为屏蔽标志位。当一个通道的屏蔽标志位为 1 时, 这个通道就不能接收 DMA 请求了, 不管是来自硬件的还是来自软件的。如果一个通道没有设置自动预置功能, 那么, EOP 信号有效时, 就会自动设置屏蔽标志位。DMA 的屏蔽标志位是通过往屏蔽寄存器写入屏蔽字节来设置的, 如图 10.33 所示。

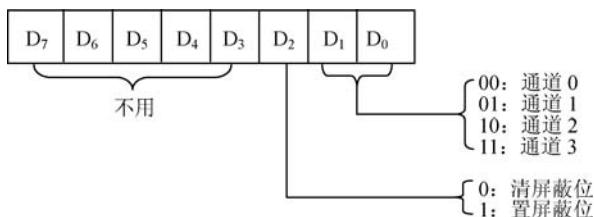


图 10.33 屏蔽寄存器

D₁D₀: 选择通道。

D₂: 屏蔽位设置, “=1” 设置屏蔽位, “=0” 清除屏蔽位。

此外, 8237A 还允许使用综合屏蔽命令来设置通道的屏蔽触发器, 如图 10.34 所示。

D₃~D₀ 中的某位为 1, 就使对应的通道设置屏蔽位。

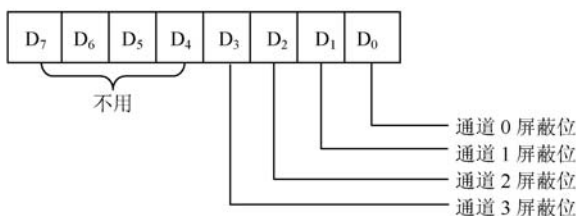


图 10.34 综合屏蔽命令字

若直接访问屏蔽寄存器(使用图 10.33 格式), 则地址为 0AH。若使用综合屏蔽命令字来访问屏蔽寄存器(使用图 10.34 格式), 则地址为 0FH。8237A 中还可使用清屏蔽寄存器命令来清除全部 4 个通道的屏蔽位, 见 10.5.4 小节的“清屏蔽寄存器命令”。

7. 状态寄存器

8237A 中有一个可供 CPU 读取的状态寄存器, 其格式如图 10.35 所示。其低 4 位反映在读命令这个瞬间各通道的字节计数器是否已减为 0。若其中某位为 1, 则表示相应通道字节计数器减至 0。高 4 位反映各通道的 DMA 请求情况, 1 为有请求。

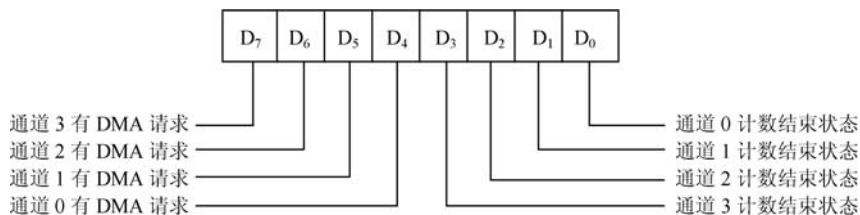


图 10.35 状态寄存器

8. 暂存寄存器

在存储器到存储器的传送方式时, 暂存寄存器保存从源单元读出的数据, 再由它写入目的单元。完成传送时, 其中保留传送的最后一个字节, 该字节可由 CPU 读出, RESET 之后使之复位。

10.5.4 8237A 的软件命令

8237A 还设计了专用的软件命令, 以实现对其编程控制。软件命令有三条: 主清除命令, 清除先/后触发器命令和清除屏蔽寄存器命令。

1. 主清除命令

主清除命令与硬件的 RESET 信号具有相同作用。执行这条命令后, 命令、状态、请求、暂存寄存器以及先/后触发器都被复位, 屏蔽寄存器被置位。然后, 8237A 处于空闲周期。

主清除命令的地址是 0DH。只须对该端口执行写操作即可发出主清除命令, 至于写入的数据, 8237A 并不关心, 可随意设置。

2. 清除先/后触发器命令

在上面地址和字节数寄存器中, 曾提到这些寄存器的字长都是 16 位, 但是供 CPU 读/写的数据通道却只有 8 位, 为此对一个寄存器的读/写必须分两次进行。由于一个寄存器只规定了一个地址, 所以在芯片内规定了一个读/写高/低字节的次序, 这个次序由一个先/后触发器来保证。当触发器为 0 时, 读/写低位字节, 为 1 时则读/写高位字节。触发器在每次读/写这组寄存器后都翻转一次。根据先/后触发器的这种工作特性, 每次读/写地址和字节数寄存器前要将其复位, 以确保操作状态。

要实现先/后触发器复位, 可以使用清除先/后触发器命令, 该命令是对 0CH 地址进行一次写操作即可, 而对于写入内容, 同主清除命令一样, 没有任何要求。

另外, 硬件 RESET 信号和软件主清除命令也会使先/后触发器复位。

3. 清屏蔽寄存器命令

清屏蔽寄存器命令的地址为 0EH。执行这个命令将清除全部 4 个通道的屏蔽寄存器, 使它们允许接收 DMA 请求。

10.5.5 8237A 的工作时序

8237A 有两种操作周期, 即空闲周期和有效(DMA)周期。所谓空闲周期是指 8237A 在复位后还没有编程, 或者已经编程但还没有接到 DMA 请求时的情况。在这个周期中, CPU 可对 8237A 进行编程, 也可从 8237A 中读出状态进行检查。有效(DMA)周期指当 DMA 控制器接到 CPU 的 HLDA 响应信号后, 8237A 进行 DMA 传输的过程。每个周期由若干个状态周期组成。每个状态周期都占一个时钟周期的时间, 但由于每个状态周期中它们完成的任务不相同, 所以又把它们分别叫做 SI、S₀、S₁、S₂、S₃、S₄ 和 SW 七种不同的周期。在空闲周期中只执行一种空闲状态 SI, 此种状态下, 每个时钟周期都检查 DREQ 状态, 以确定是否有通道请求 DMA 服务。同时, 也对 $\overline{CS}$ 端采样, 判断 CPU 是否要对 8237A 进行读/写操作。如 $\overline{CS}$ 为低电平, 且 HLDA 也是低电平, 就使芯片进入编程工作状态。

1. 正常 DMA 读/写操作

8237A 在 DMA 传送时的时序图如图 10.36 所示。当 8237A 编程后, 在接到外设的 DREQ 请求后, 就向 CPU 发出总线保持请求 HRQ, 并使芯片进入 S₀ 周期。S₀ 是个等待周期, 这

期间 8237A 等待 CPU 让出总线, 得到 HLDA 的响应。 S_0 周期中, 8237A 还是可以接受 CPU 的读/写操作控制。

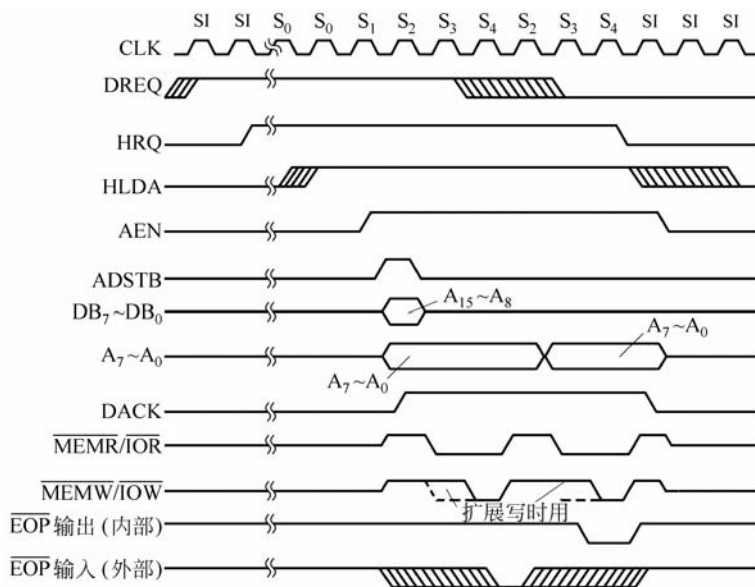


图 10.36 8237A 时序图

在接到 CPU 送来的 HLDA 信号后, 8237A 开始传送数据的操作。一个完整的传送周期应包括 $S_1 \sim S_4$ 四个工作状态。与 CPU 一样, 如果慢速存储器或外设不能在规定的 4 个周期中完成读/写工作, 可以利用 READY 端控制, 在 S_3 、 S_4 之间插入 SW 状态周期, 以求速度匹配。

当进行存储器到存储器传送的操作时, 每传送一个字节包括两个阶段, 即从存储器中读出(第 1 阶段), 以及写回存储器(第 2 阶段), 每个阶段都需要 4 个状态周期。

S_1 周期中, 地址线变得有效, 从 $A_7 \sim A_0$ 送出低 8 位地址, 从 $DB_7 \sim DB_0$ 送出高 8 位地址 $A_{15} \sim A_8$ 。同时 ADSTB 有效, 将高 8 位地址锁存(在 S_2 的下降沿)供输出。由于 S_1 是当 CPU 已经释放总线后进入的状态, 所以 8237A 还使 AEN 有效, 将地址信号接入系统总线, 形成了 16 位地址。在连续传送数据时, 存储器的字节地址都是相邻的, 即它们的地址中高 8 位地址(需锁存的那一部分)在大部分情况下是不变的。所以当地址变化没有更改高位地址数值时, S_1 就可以省略, 这时只要 S_2 、 S_3 和 S_4 三种状态周期就可以了。

S_2 状态周期开始时, 锁存了高位地址, 并开始数据传送。首先向外设送出 DACK, 通知外设可以开始数据传送。接着根据操作要求, 送出读/写控制信号。如 DMA 读操作, 就送出 MEMR 控制存储器输出数据; 如是 DMA 写操作就送 IOR 到外设, 使外设送出数据。

S_3 状态时, 8237A 送出写入数据相关的控制信号。DMA 读操作时就给外设送 IOW, 而 DMA 写操作则给存储器送 MEMW。

S_3 状态结束时, 在下降沿检测 8237A 的 READY 信号端。如果 READY 端为低电平, 即没有准备好, 就产生 SW 状态周期。在 SW 期间, 8237A 在 S_3 中送出的各种信号都维持不变, 即让外设和存储器有更多的时间读/写数据。如果在 S_3 或 SW 下降沿处检测到 READY

信号为高电平, 就进入 S_4 状态周期。

S_4 状态周期中结束本次一个字节数据传送。如果整个 DMA 传送结束, 就进入 SI 周期; 如果后面还有字节传送, 则进入下一个 $S_1 \sim S_4$ 周期。

最后简要说一下存储器间的传送时序。每传送一个字节要用 8 个周期, 前 4 个周期发出 $\overline{\text{MEMR}}$ 信号将源数据读入 8237A 的暂存寄存器。后 4 个周期送出 $\overline{\text{MEMW}}$, 将暂存寄存器内容写入目的地址。整个过程中 $\overline{\text{IOR}}$ 和 $\overline{\text{IOW}}$ 都不起作用。

2. 扩展写与压缩时序

所谓扩展写是当 8237A 输出写信号($\overline{\text{IOW}}$ 或 $\overline{\text{MEMW}}$)时, 使其有效的时间提前。正常情况下在 S_3 才送出的有效写控制信号, 提前到 S_2 就变得有效。这可以使得写入的设备有更多的写入时间。当命令寄存器的 $D_5=1$ 时, 就选择了扩展写方法。

压缩定时是当命令寄存器 $D_3=1$ 时采用的定时方式。在 $S_1 \sim S_4$ 四个周期中, S_1 是为锁存高 8 位地址用的, 而 S_3 则是一个延长周期, 给读写以充足的时间。在追求更高传输速度, 且器件的读/写速度又可跟得上, 的情况下, 就可以把 S_1 和 S_3 两个周期省去, 形成了时间压缩一半的时序。当然, 如果是一次 DMA 中的第一个字节传送, 或在传送中必须改变高位字节地址时, S_1 还是不能省略的。

10.5.6 8237A 的初始化及其在 PC 机中的应用

1. 8237A 的初始化

在进行 DMA 操作之前, 必须对 8237A 进行初始化编程。初始化编程有以下内容:

- (1) 输出复位命令, 使 8237A 处于复位状态, 以接收新的命令;
- (2) 写命令寄存器, 设置 8237A 的工作方式;
- (3) 写工作方式寄存器, 设置需使用的通道的工作方式和传送类型;
- (4) 根据所选通道, 把内存存储器的起始地址写入相应通道当前地址寄存器和基地址寄存器(先写低位, 后写高位);
- (5) 把传送的字节数-1, 写入当前字节计数器和基字节数寄存器;
- (6) 清除该通道屏蔽寄存器的屏蔽位;
- (7) 写入请求寄存器, 便可由软件启动 DMA 传送。否则, 经过以上 6 步编程后, 由通道 DREQ 启动 DMA 传送的过程。

在初始化编程时, 应注意的一个问题是, 要保证在初始化过程结束后才响应 DMA 请求, 否则可能会发生错误。为此, 可在编程开始时, 禁止 8237A 工作或屏蔽所选通道。当初始化编程结束后, 才允许 8237A 工作或清除该屏蔽位。

2. 8237A 在 PC 机中的应用

个人计算机的系统板上用了一片 8237A DMA 控制器, 它的 4 个通道中, 通道 0 用作动态存储器的刷新控制, 通道 1 为用户预留, 用户可以通过系统与之连接, 通道 2 作为软磁盘驱动器数据传输控制用, 通道 3 留给硬盘驱动器用。

PC/XT 机中 8237A 的端口地址为 0000H~000FH。为了能访问 20 位地址, 系统提供了

DMA 页面寄存器, 页面寄存器采用寄存器堆电路 74LS670, 存入 4 个 DMA 通道的高 4 位地址 $A_{19} \sim A_{16}$, 当 $\overline{\text{DMAAEN}}$ 为低电平时, 页面寄存器的输出允许端 READ 有效, 送出工作通道的高 4 位地址。它们与 8237A 输出的 16 位地址一起, 组成 20 位地址信息。通道 0、通道 1、通道 2、通道 3 对应的页面寄存器的 I/O 端口地址分别为 87H、83H、81H、82H。

通道 0 用于动态 RAM 刷新。8253 的通道 1 工作在定时方式, 且每 $15\mu\text{s}$ 就从 OUT_1 端输出一个脉冲, 这个脉冲被送到 8237A, 作为 DREQ_0 的 DMA 请求信号, 即每 $15\mu\text{s}$ 进入一次 DMA 操作。在 8237A 响应这个 DMA 请求时, 输出的 DACK_0 用来产生一个有效的 $\overline{\text{RAS}}$, 供存储器芯片刷新用。

按照刷新的要求, 每次刷新一行存储单元, 通道 0 应被置为单字节传送方式, 实现对 RAM 的读出操作, 地址从 0000H 开始, 传送字节数 FFFFH, 且应该要求它能自动预置。这样初始化程序段可以写成:

```
OUT 0DH, AL      ; 对 DMA 进行总清的软命令, DMA 停止工作
MOV AL, 0
OUT 0, AL        ; 置地址低字节
OUT 0, AL        ; 置地址高字节
MOV AL, 0FFH
OUT 1, AL        ; 置计数值低字节
OUT 1, AL        ; 置计数值低字节
MOV AL, 58H      ; 方式字的内容为单字节传送、读操作、地址增量且自动预置
OUT 0BH, AL
MOV AL, 0
OUT 08H, AL      ; 写命令寄存器, 启动 8237A
OUT 0AH, AL      ; 清屏蔽
```

通道 2 用来在软盘与存储器之间进行传送控制。软磁盘中的数据有写入、读出和校验等工作方式, 所以 DMA 通道 2 也要适合这种可能变化的工作方式。实际上这个通道被设置成单字节传送方式(因为软盘的读/写速度并不很高), 读/写方式, 且不要自动预置。这种情况使开始时将它预置成读/写方式, 在以后进入某种工作方式时, 要根据需要重新预置。它的读软盘时所作初始化程序段如下:

```
OUT 0CH, AL      ; 清字节指针
MOV AL, 46H
OUT 0BH, AL      ; 方式字, 单字节传送、写入, 非自动预置, 地址减量
MOV AX, XXXX
OUT 04H, AL      ; 置地址低 8 位
MOV AL, AH
OUT 04H, AL
MOV AL, XX       ; 给页面寄存器
AND AL, 0FH      ; 送  $A_{19} \sim A_{16}$  地址
OUT 81H, AL
MOV AX, XXXX
OUT 05H, AL      ; 置计数值低 8 位
MOV AL, AH
OUT 05H, AL      ; 置计数值高 8 位
MOV AL, 02H
OUT 0AH, AL      ; 清 2 通道屏蔽, 启动 8237A
```

10.6 小 结

本章首先介绍了 I/O 接口电路的基本功能、信号分类、端口编址方式、CPU 和 I/O 接口电路的数据传送控制方式,以便对 I/O 接口技术有一个整体把握。然后介绍了可编程定时器/计数器 8253、可编程并行接口芯片 8255、可编程 DMA 控制器 8237 电路功能及其应用。通过学习,能够根据不同的应用和不同的外部设备,正确地选择接口电路,以组成特定的微机应用系统。

10.7 习 题

1. 简述 I/O 接口电路的基本功能。
2. 简述 I/O 接口电路的信号分类。
3. 简述 I/O 接口电路的编址方式。
4. CPU 与接口电路之间数据传送的控制方式有几种?试比较它们各自的优缺点及适用场合。
5. 8253 有哪几种工作方式?各有何特点?
6. 说明 8253 在写入计数初值时,二进制计数和十进制计数有无区别?若有,有何区别?
7. 设 8253 的地址为 $F0H \sim F3H$, CLK_1 为 500kHz,欲让计数器 1 产生 50Hz 的方波输出,试对它进行初始化编程。
8. 用 8253 的通道 0 对外部事件进行计数,每计满 200 个脉冲时,产生一次中断请求信号,设 8253 的端口地址为 $20H \sim 23H$,试分析用何种工作方式较好,并写出初始化程序。
9. 某 PC 系统中,8253 的端口地址为 $40H \sim 43H$ 。当某一外部事件发生时(给出一高电平信号),2s 后向主机申请中断。若用 8253 实现此延时,试设计硬件连接图,并对 8253 进行初始化。(注:当一个计数通道不能满足计数要求时可考虑二个计数通道级联)
10. 8255A 有哪几种工作方式?每种工作方式有何特点?
11. 8255A 中,端口 C 有哪些独特的用法?
12. 假定 8255A 的地址为 $60H \sim 63H$,A 口工作在方式 2,B 口工作在方式 1 输入,请写出初始化程序。
13. 利用 8255A 模拟交通灯的控制:在十字路口的纵横二个方向上均有红、黄、绿三色交通灯(用三种颜色的发光二极管模拟),要求二个方向上的交通灯能按正常规律亮灭,画出硬件连线图并写出相应的控制程序。设 8255A 的端口地址为 $60H \sim 63H$ 。
14. 试用 8255A 设计一个并行接口,实现主机与打印机的连接,并给出以中断方式实现与打印机通信的程序。设 8255A 的端口地址为 $60H \sim 63H$ 。
15. 简述 8237A 的主要技术特性。
16. 简要说明 8237A 的三种基本传送类型的特点。
17. 在个人计算机中,8237A 的通道 2 为什么设置成单字节传送?如果用成块传送,会发生什么问题?(提示:通道 0 用于动态 RAM 的刷新,而刷新有时间要求)

第 11 章 串行通信接口及应用

教学提示：串行通信是指将数据在信号通路上一位一位地顺序传送，每一位数据占据一个固定的时间长度。而串行接口就是完成串行通信传输任务的接口。串行通信速度不如并行通信，但是串行通信具有抗干扰能力强、接口相对简单、成本低等诸多优点，所以在远程计算机通信系统中，采用串行通信方式特别适合。

教学要求：本章主要让学生了解串行通信的基本知识和各种通信接口标准。重点掌握可编程串行通信接口芯片 8250 的结构、功能及编程应用。

11.1 串行通信基础

在并行通信中，所用的数据传输线较多，因此不适合长距离传输。而串行通信所需的传输线较少，所以采用串行通信方式可以节省传输线。在数据位数较多，传输距离较长的情况下，这个优点更为突出，可以降低成本。

11.1.1 串行通信数据传送方式

在串行通信中，按照数据在通信线路上的传输方向，可以分为单工通信、半双工通信和全双工通信。

1. 单工通信

单工通信是指数据信号仅允许沿一个方向传输，即由一方发送数据，另一方接收数据，如图 11.1 所示。无线电广播就是单工通信方式。

2. 半双工通信

半双工通信是指通信双方都能接收或发送，但不能同时接收和发送的通信方式。在这种传送方式中，通信双方只能轮流地进行发送和接收，即 A 站发送，B 站接收；或 B 站发送，A 站接收，如图 11.2 所示。对讲机系统就是采用半双工通信方式的。

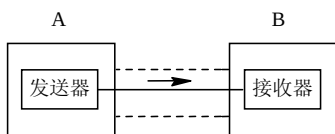


图 11.1 单工通信

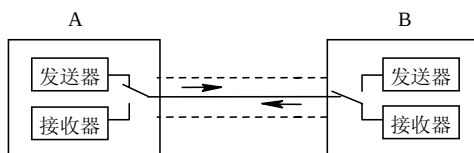


图 11.2 半双工通信

3. 全双工通信

全双工通信是指通信双方在同一时刻可以同时进行发送和接收数据，如图 11.3 所示。双工需要两条传输线。目前，在计算机网络通信系统中就是采用全双工通信的。

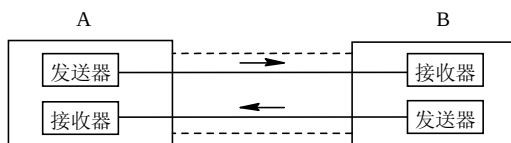


图 11.3 全双工通信

11.1.2 串行通信协议

在串行通信中，为了正确地传输每一个字符和字符中的每一位信息，必须要保证发送端和接收端同步工作。根据数据的收发方式，串行通信协议可分为异步通信协议和同步通信协议两种。

1. 异步通信协议

异步通信协议采用的数据格式是每个字符都按照一个独立的整体进行发送，字符的间隔时间可以任意变化，即每个字符作为独立的信息单位(帧)，可以随机地出现在数据流中。所谓“异步”，就是指通信时两个字符之间的间隔事先不能确定，也没有严格的定时要求。

异步通信协议规定的传输格式由 1 位起始位、5~8 位数据位、1 位奇偶校验位、1~2 位停止位和若干个空闲位等组成，如图 11.4 所示。其中，起始位是表示字符的开始，通知接收方开始接收数据；停止位是表示字符传输的结束。



图 11.4 异步串行通信数据格式

在串行通信中，传输速率是指单位时间内传送的二进制数据的位数，也称波特率，单位是位/秒(b/s，也称 bps)。常用的波特率有 110、300、600、1200、2400、4800、9600、19200、38400 等。字符速率是指每秒钟传送的字符数，它与波特率是两个相关但表达的意义不相同的概念。例如，若异步通信的数据格式由 1 位起始位、8 位数据位、1 位奇偶校验位、2 位停止位组成，波特率为 2400b/s，则每秒钟能够最多传送 $2400 \div (1+8+1+2) = 200$ 个字符。

采用异步通信格式的优点是，控制简单，不需收发双方时钟频率保持完全一致，可以有偏差，纠错方便。其缺点是一旦传输出错，则需要重发，传输效率低，信息冗余大。

2. 同步通信协议

在异步通信中，每传送一个字符，需要增加大约 20% 的附加信息位，即用起始位和停止位作为每个字符的开始和结束标志，占用了一些时间，降低了传输效率。因此在大批量数据传送时，为了提高速度，就要设法去掉这些标志，而采用同步传输。

在同步传输时，信息以数据块(由多个字符组成)为单位进行传送，不仅字符内部位与位之间是同步的，而且字符之间的传送也是同步的，因此，收发双方对时钟同步要求严格，所以设备也比较复杂，一般只用于需要高速大容量的通信场合。

同步通信协议主要分为两种：一种是面向字符的同步通信协议，它的特点是把每个帧看作由若干个字符组成的数据块，并规定了一些特殊字符作为同步字符以及传输过程的控制信息；另一种是面向比特的同步通信协议，它的特点是把数据及控制信息看作为比特流的组合，靠约定的比特组合模式来标志帧的开始和结束。常见的同步通信协议格式如图 11.5 所示，主要有单同步格式、双同步格式、SDLC 格式、HDLC 格式和外同步格式。

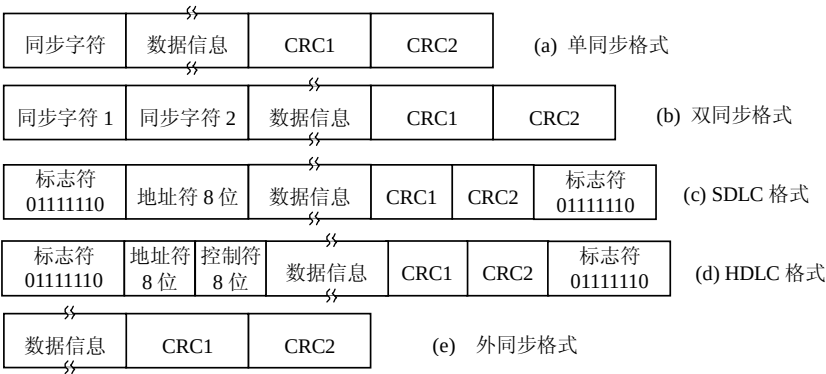


图 11.5 常见的几种同步通信格式

同步通信中需要注意的是，传输的数据都是成批连续的，两个字符之间不允许有空格，在没有字符要传输时要填上空字符，以此来保证数据传输的准确性。

采用同步通信协议的优点是传输效率高、传输速度快，适合要求快速、连续地传输大量数据的场合，但其技术比异步通信协议复杂、硬件开销大、成本高。因此，在一般应用中，采用异步通信的比较多。

11.2 串行通信的接口标准

为了实现不同厂商生产的计算机和各种外部设备之间进行串行通信，国际上制定了一些串行接口标准，常见的有 RS-232C 接口、RS-422A 接口、RS-485 接口。目前最普遍使用的是美国电子工业协会颁布的 RS-232C 接口标准。RS-232C 接口标准规定了机械、电气、功能等方面的参数。

11.2.1 RS-232C 的机械特性

RS-232C 是一种总线标准，可以保证硬件兼容。通常 RS-232C 接口有 9 针和 25 针等规格，各信号线的功能见表 11-1，其接口信号线排列如图 11.6 所示。

表 11-1 RS-232C 接口 25 针(9 针)信号线功能定义

25 针(9 针)引脚号	缩写名称	信号名称	功能特点
1	GND	保护地	作为设备接地端
2(3)	TXD	发送数据	将数据送调制解调器
3(2)	RXD	接收数据	从调制解调器接收数据
4(7)	RTS	请求发送	终端请求通信设备切换到发送方式

续表

25 针(9 针)引脚号	缩写名称	信号名称	功能特点
5(8)	CTS	允许发送	指出调制解调器准备好发送
6(6)	DSR	通信设备准备好	指出调制解调器进入工作状态
7(5)	SG	信号地	作为公用的信号地
8(1)	DCD	载波信号检测	指出调制解调器正在接收远程载波
9		空	
10		空	
11		空	
12		辅助信道接收信号检测	指出在第二通道上检测到信号
13		辅助信道允许发送	指出在第二通道准备好发送
14		辅助信道发送数据	往调制解调器以较低速率输出
15		发送器时钟(DCE)	为调制解调器提供发送器定时信号
16		辅助信道接收数据	从调制解调器以较低速率输入
17		接收器时钟	为接口和终端接收器提供信号定时
18		空	
19		辅助信道请求发送	闭合第二通道的发送器
20(4)	DTR	数据终端准备好	数据终端准备就绪，可以接收
21		信号质量检测	通信设备通知端，线路已经连通
22(9)	RI	振铃指示	指出在链路上检测到声音信号
23		数据率选择	可以选择数据信号速率
24		发送器时钟(DTE)	为接口和终端提供发送器定时信号
25		空	

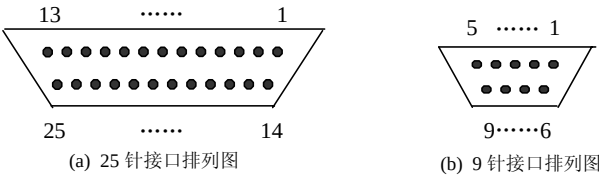


图 11.6 RS-232C 接口信号线排列图

11.2.2 RS-232C 的电气特性

RS-232C 中采用负逻辑规定逻辑电平，逻辑“1”电平规定为-3V~-15V，逻辑“0”电平规定为+3V~+15V。在实际使用中，常采用±12V 或±15V。由此可见，RS-232C 标准中的信号电平标准与计算机中广泛采用的 TTL 电平标准不相容，在使用时，必须有电平转换电路。

RS-232C 既是一种协议标准，又是一种电气标准，它采用单端、双极性电源供电电路。RS-232C 规定的传输速率有：50b/s、75b/s、110b/s、150b/s、300b/s、600b/s、1200b/s、2400b/s、4800b/s、9600b/s、19.2Kb/s、33.6Kb/s、56Kb/s 等，能够适应不同传输速率的设备。

在远距离通信时，一般要加 MODEM(调制解调器)，当计算机与 MODEM 连接时，只要将编号相同的引脚连接起来即可，如图 11.7 所示。

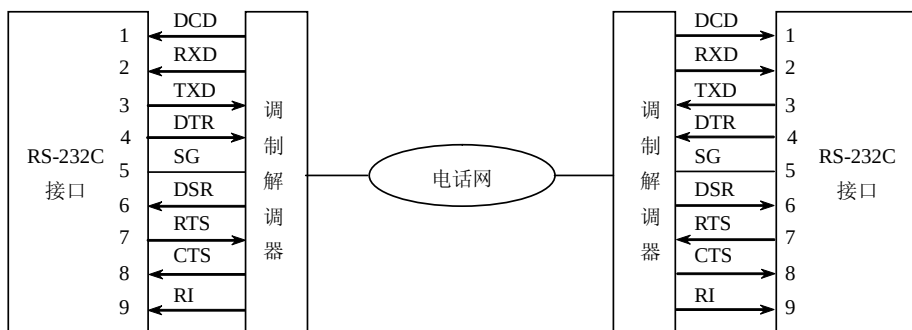
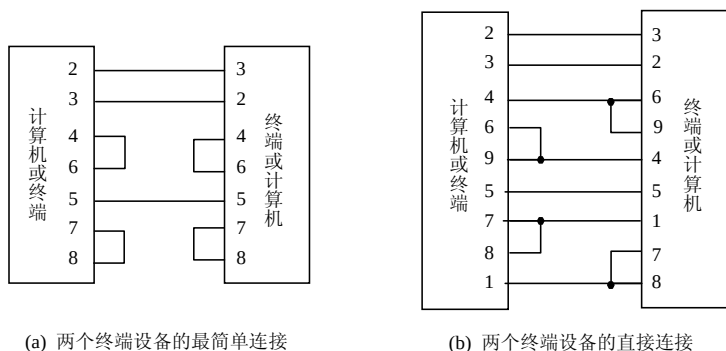


图 11.7 RS-232C(9 针)与调制解调器的连接图

在距离较近(小于 15m)的情况下进行通信时，不需要使用 MODEM，两个计算机的 RS-232C 接口可以直接互连，如图 11.8 所示。



(a) 两个终端设备的最简单连接

(b) 两个终端设备的直接连接

图 11.8 两个 RS-232C(9 针)终端设备的连接图

图 11.8(a)给出的是最简单的互连方式，只需 3 条线就可以实现相互之间的全双工通信，但是其许多功能(如流控)就没有了。

图 11.8(b)给出的是常用信号引脚的连接，为了交换信息，TXD 和 RXD 是交叉连接的，即一个发送数据，另一个接收数据；RTS 和 CTS 与 DCD 互连，即用请求发送 RTS 信号来产生清除发送 CTS 和载波检测 DCD 信号，以满足全双工通信的逻辑控制；用类似的方法可将 DTR、DSR 和 RI 互连，以满足 RS-232C 通信控制逻辑的要求。这种方法连线较多，但能够检测通信双方是否已准备就绪，故通信可靠性高。

11.3 可编程串行通信接口芯片 8250

在计算机进行的串行数据通信时，一方面需要实现并行到串行或串行到并行的转换，另一方面还需要按照传输协议发送和接收每个字符或数据块，这些工作可由软件实现，也可由硬件来实现。

能完成异步通信的硬件电路被称为 UART，典型的芯片有 INS8250、MC6850-ACIA、

MC6852-SSDA 等。既可以完成异步通信又可以完成同步通信的硬件电路被称为 UART，如 Intel 的 8251A、AMD9951 等。

1981 年，在 IBM PC 机主板上用 8250UART 与 MODEM 或串行打印机进行通信，由于 8250 的应用包含对 PC 机中 BIOS 的支持，因而确定了它的结构和特性。几年后，产生了 16450，它是 8250 的直接扩展，如增加了速率，可支持 115K 和 230K 的速率。在后来的 16550 芯片上，其功能比 8250 多 16 个字节的接收和发送 FIFO，而 16C650 则把 FIFO 增加到 32 个字节，现在的 16C750 则把 FIFO 增加到 64 个字节。大容量的 FIFO 减少了中断返回次数，提高了速度。这里，仅介绍 INS8250 的结构及应用。

11.3.1 概述

INS8250 是使用 +5V 电源的具有 40 个引脚的芯片，各个引脚的功能特点如表 11-2 所示，其内部结构及引脚信号如图 11.9 所示。其中，主复位信号有效时，对 8250 内部寄存器和部分外部信号线有影响，具体情况如表 11-3 所示。

表 11-2 8250 引脚功能

引脚名称	功能特点
$D_7 \sim D_0$	双向数据线，与系统数据总线相连，用于写入/读出状态字或者是发送/接收数据
$\overline{CS}_0$ 、 $\overline{CS}_1$ 、 $\overline{CS}_2$	片选信号线，有效时，选中 8250 芯片
A_2 、 A_1 、 A_0	内部寄存器地址信号线，用于选择 8250 芯片内部寄存器
$\overline{ADS}$	地址选通信号线，低电平有效
$\overline{DISTR}$ 、 $\overline{DISTR}$	数据输入选通信号线
$\overline{DOSTR}$ 、 $\overline{DOSTR}$	数据输出选通信号线
RCLK	接收时钟信号线
$\overline{CTS}$	清除发送信号线
$\overline{RTS}$	请求发送信号线
$\overline{DTR}$	数据终端准备好信号线，输出
$\overline{DSR}$	数据装置准备好信号线，输入
$\overline{RLSD}$	接收线路信号检测信号线，输入
$\overline{RI}$	振铃指示信号线，输入
$\overline{OUT}_1$	用户编程指定的输出端 1
$\overline{OUT}_2$	用户编程指定的输出端 2
$\overline{CS}_{OUT}$	片选输出信号线，输出
$\overline{DDIS}$	驱动器禁止信号线，输出
$\overline{BAUDOUT}$	波特率信号输出端
$\overline{INTR}$	中断请求输出信号线
S_{OUT}	串行数据输出信号线
S_{IN}	串行数据输入信号线
$XTAL_1$ 、 $XTAL_2$	外部时钟端，可接晶振或直接接外部时钟信号

续表

引脚名称	功能特点
MR	主复位输入信号线，高电平有效
V _{CC}	电源端，接+5V
GND	接地端
NC	空闲，未使用

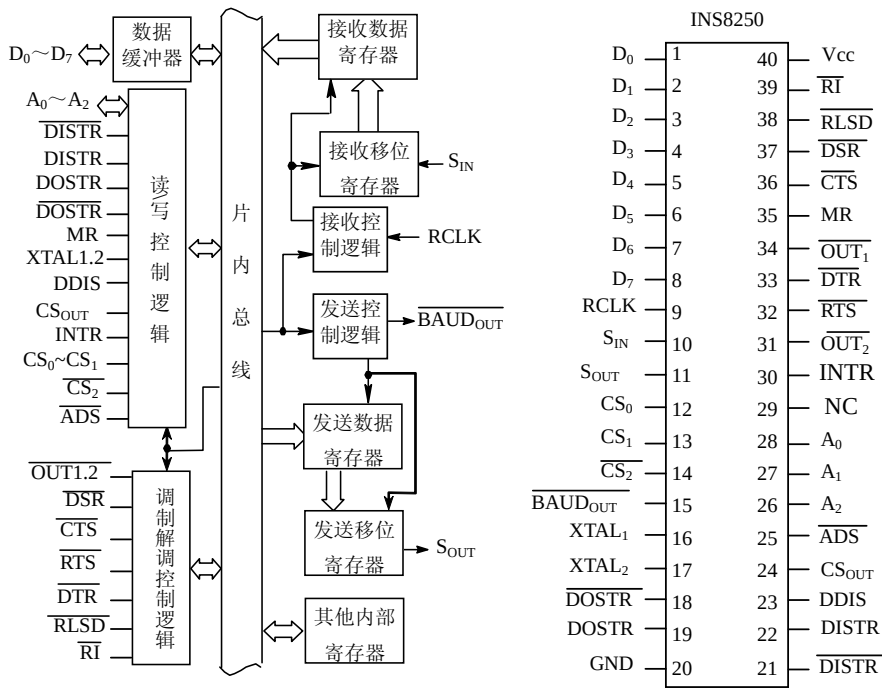


图 11.9 INS8250 内部结构及引脚图

表 11-3 MR 功能

寄存器或信号线	复位控制	复位后状态
通信控制寄存器、中断允许寄存器	MR	各位均为低电平
中断标志寄存器	MR	除第 0 位外、其余为低电平
MODEM 控制寄存器	MR	各位均为低电平
通信状态寄存器	MR	除 5、6 位外、其余为高电平
INTR(线路状态错)	读通信状态寄存器或 MR	低电平
INTR(发送寄存器空)	读中断标志寄存器，写发送数据寄存器或 MR	低电平
INTR(接收寄存器满)	读接收数据寄存器或 MR	低电平
INTR(MODEM 状态改变)	读 MODEM 状态寄存器或 MR	低电平
SOUT、OUT ₁ 、OUT ₂ 、RTS、DTR	MR	低电平

INS8250 的主要功能特点是：传输速率可在 50b/s~9600b/s 范围内编程选择；具有完整的 MODEM 控制功能；具有线路隔离、故障模拟等内部诊断功能；具有中断控制和优先权判决能力；可以支持半双工或全双工等。

当发送数据时，8250 接收从 CPU 送来的并行数据，并保存于发送数据寄存器中。只要发送移位寄存器没有正在发送数据，8250 就把发送数据寄存器中的数据送到发送移位寄存器中，并按照程序中所规定的格式和波特率，加入起始位、奇偶校验位和停止位，然后从串行数据输出端 S_{OUT} 逐位输出。

当接收数据时，8250 的接收移位寄存器对输入端 S_{IN} 输入的串行数据进行移位接收，并按照规定格式和波特率，自动删除起始位、奇偶校验位和停止位。然后将这些数据并行地传送到接收数据寄存器中，CPU 从接收数据寄存器中就可读出收到的数据了。

由此可见，发送移位寄存器和接收移位寄存器是 8250 的核心部件。发送移位寄存器完成“并一串转换”功能，即把计算机输出的并行数据转换成异步通信所需的串行码输出；接收移位寄存器完成“串一并转换”功能，即将收到的串行输入码转换成计算机所需的并行数据。

11.3.2 8250 的寄存器

8250 内部有 9 种可访问的寄存器，分别为：发送数据寄存器 THR，接收数据寄存器 RBR，通信控制寄存器 LCR，通信状态寄存器 LSR，波特率设置寄存器 DLR(也称除数锁存器)，MODEM 控制寄存器 MCR，MODEM 状态寄存器 MSR，中断允许寄存器 IER 和中断标志寄存器 IIR。这些寄存器主要用来设置其工作参数或者获得其工作状态。

1. 通信控制寄存器

这是一个可读/写寄存器，主要用于指定串行异步通信的数据格式。例如数据位数，奇偶校验的选择、停止位的多少。其各位意义如图 11.10 所示。

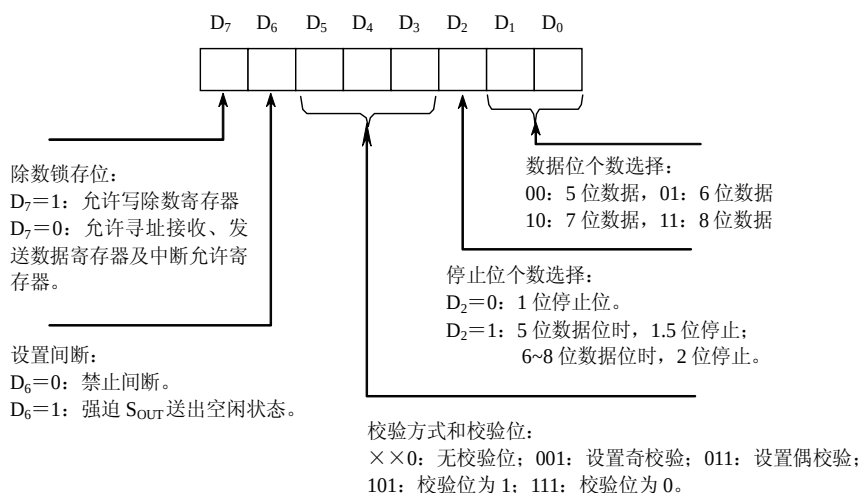


图 11.10 通信控制寄存器格式

值得注意的是，通信控制寄存器的 D₇，需要读/写除数锁存时，必须使 D₇=1，而在读

/写其他寄存器时，要使 $D_7=0$ 。

2. 通信状态寄存器

通信状态寄存器是一个 8 位寄存器，用于为 CPU 提供 8250 芯片的内部状态信息，主要是说明在通信过程中 8250 接收和发送数据的情况。其各位的功能含义如表 11-4 所示。

表 11-4 通信状态寄存器各位功能含义

位 状 态	意 义
$D_0=1$	表示 8250 已接收到一个有效字符，CPU 可以从 8250 接收数据寄存器中读取。一旦读取后，此位立即变为 0
$D_1=1$	表示 CPU 未及时把接收数据寄存器中的输入字符取走，8250 又接收新的数据而将前一个数据破坏
$D_2=1$	表示在 8250 接收到一个完整的数据进行奇偶校验时，发现被接收的数据有奇偶错
$D_3=1$	表示接收到的数据停止位不正确，也称格式错或帧出错
$D_4=1$	表示线路信号间断，即在超过一个完整的字符传输时间内接收的均为空闲状态
$D_5=1$	表示发送数据寄存器为空
$D_6=1$	表示发送移位寄存器为空
$D_7=0$	恒为 0，不用

在 8250 串行发送数据时，一旦发送移位寄存器已将其数据的各位串行移出，则发送数据寄存器的数据会自动传送给发送移位寄存器，然后数据从发送移位寄存器串行输出，同时，发送数据寄存器变为空状态。当发现发送数据寄存器为空后，可立即向它写入下一个要传送的数据。

3. 发送数据寄存器

发送数据寄存器是一个 8 位寄存器，发送数据时，CPU 将数据写入该发送数据寄存器。若发送移位寄存器为空，则该发送数据寄存器中的数据便会由 8250 的硬件并行送入到发送移位寄存器中，以便串行输出。

4. 接收数据寄存器

接收数据寄存器是一个 8 位寄存器，当 8250 接收到一个完整的字符时，会将该字符由接收移位寄存器传送到接收数据寄存器，CPU 可以直接到接收数据寄存器中读取数据。

5. MODEM 控制寄存器

MODEM 控制寄存器是一个 8 位的寄存器，控制芯片的 4 个引脚的输出和芯片的环路检测，以控制 MODEM 或其他数字设备。其各位的功能含义如表 11-5 所示。

6. MODEM 状态寄存器

MODEM 状态寄存器检测 MODEM 或其他设备加到 8250 上的 4 个控制引脚的输入状态以及这些控制线的状态变化。其各位的功能含义如表 11-6 所示。

表 11-5 MODEM 控制寄存器各位功能含义

位 状 态	意 义
$D_0=1$	控制 8250 的 $\overline{DTR}$ 为低电平, 表示数据终端准备好
$D_1=1$	控制 8250 的 $\overline{RTS}$ 为低电平
$D_2=1$	控制 8250 的 $\overline{OUT_1}$ 为低电平
$D_3=1$	控制 8250 的 $\overline{OUT_2}$ 为低电平
$D_4=1$	提供了用于 8250 诊断测试的局部环路返回特性, 实现 8250 自己对自己的自诊断测试
$D_5=0$	恒为 0, 不用
$D_6=0$	恒为 0, 不用
$D_7=0$	恒为 0, 不用

表 11-6 MODEM 状态寄存器各位功能含义

位 状 态	意 义
D_0 对应 $\overline{CTS}$	当某位为 1 时, 表示自上次读该寄存器之后, 相应的 MODEM 来的控制线输入信号已改变状态。当某位是 0 时, 则说明相应的输入信号状态无改变。在读这些寄存器时, 使这 4 位同时清零。中断允许寄存器的 $D_3=1$ 时, 则 MSR 的 $D_3 \sim D_0$ 的某一位为 1 时, 都会使芯片产生一个 MODEM 状态中断
D_1 对应 $\overline{DSR}$	
D_2 对应 $\overline{RI}$	
D_3 对应 $\overline{RLSD}$	
D_4 对应 $\overline{CTS}$ 反相	在自测试时, 为 MODEM 控制寄存器 $\overline{RTS}$ 位的状态
D_5 对应 $\overline{DSR}$ 反相	在自测试时, 为 MODEM 控制寄存器 $\overline{DTR}$ 位的状态
D_6 对应 $\overline{RI}$ 反相	在自测试时, 为 MODEM 控制寄存器 $\overline{OUT_1}$ 的状态
D_7 对应 $\overline{RLSD}$ 反相	在自测试时, 为 MODEM 控制寄存器 $\overline{OUT_2}$ 的状态

7. 除数锁存器

除数锁存器即波特率设置寄存器, 该寄存器为 16 位, 外部基准时钟被除数锁存器中的除数相除, 可以获得所需的波特率。如果外部基准时钟频率 f 已知, 而 8250 所要求的波特率 F 也已确定, 则就可以求出除数锁存器中应该锁存的除数为: 除数锁存器 $= f \div (16 \times F)$ 。

8. 中断允许寄存器

8250 共有 4 种中断源, 分别是接收器线路状态中断、接收数据寄存器“满”中断、发送数据寄存器“空”中断和 MODEM 状态中断。

中断允许寄存器的高 4 位不用, 只使用低 4 位, 当某位置“1”时, 表示允许相应的中断请求; 置“0”时, 表示禁止相应的中断请求。其各位的功能含义如表 11-7 所示。中断允许寄存器是可写可读的。

表 11-7 中断允许寄存器各位功能含义

位 状 态	意 义
$D_0=1$	接收数据寄存器“满”中断, 读接收数据寄存器可使此中断复位
$D_1=1$	发送数据寄存器“空”中断, 写发送数据寄存器可使此中断复位

续表

位 状 态	意 义
D ₂ =1	接收线路状态中断, 包括越限、奇偶错、格式错、间断等引起的中断, 读通信状态寄存器可使此中断复位
D ₃ =1	MODEM 状态中断, 包括发送结束、MODEM 准备好、振铃指示、接收线路信号检测等, 读 MODEM 状态寄存器可使此中断复位
D ₇ ~D ₄	恒为 0, 不用

9. 中断标志寄存器

8250 芯片中多个中断源共同使用同一个中断输出, 所以各个中断需要按照事件的紧迫性进行排序, 即优先权排队。8250 共有 4 级中断, 按优先权从高到低的中断源顺序是: 接收线路状态中断、接收数据寄存器“满”中断、发送数据寄存器“空”中断、MODEM 状态中断。

中断标志寄存器为 8 位, 高 5 位不用, 只使用低 3 位来实现 8250 的中断标志。中断标志寄存器可以指出有无待处理的中断发生及其类型, 其中第 0 位是用来表示是否有中断发生, 第 2 位和第 3 位用来表示发生中断的类型。当 D₀=1 时, 表示无中断发生; 当 D₀=0 时, 表示有中断发生。中断标志寄存器是只读的。其各位的功能含义如表 11-8 所示。

表 11-8 中断允许寄存器各位功能含义

位 状 态	意 义	位 状 态	意 义
D ₀ =1	无中断	D ₀ =0	有中断
D ₂ 、D ₁ =00	MODEM 中断	D ₂ 、D ₁ =01	发送数据寄存器“空”中断
D ₂ 、D ₁ =10	接收数据寄存器“满”中断	D ₂ 、D ₁ =11	接收线路状态中断
D ₇ ~D ₃	恒为 0, 不用		

11.3.3 8250 的编程及应用

8250 内部有许多与编程使用有关的寄存器, 在片选信号 CS₀、CS₁ 和 $\overline{\text{CS}}_2$ 有效的情况下, 先选中 8250, 再利用 A₂、A₁、A₀ 来确定被访问的寄存器。另外还可以采用两种方法进行寻址。

- (1) 利用通信控制寄存器的最高位(除数锁存位)置“1”, 来选中除数锁存寄存器。
- (2) 将发送数据寄存器和接收数据寄存器共享一个地址号, 由于发送数据寄存器是“只写”的, 而接收数据寄存器是“只读”的, 因此可以用“写入”访问发送数据寄存器, 用“读出”访问接收数据寄存器。具体地址分配如表 11-9 所示。

表 11-9 8250 的寻址(在片选信号有效的前提下)

除数锁存位	A ₂ A ₁ A ₀	$\overline{\text{RD}}$ $\overline{\text{WR}}$	所选定的寄存器	COM ₁
0	0 0 0	只读	接收数据寄存器	3F8H
0	0 0 0	只写	发送数据寄存器	3F8H

续表

除数锁存位	A ₂ A ₁ A ₀	$\overline{\text{RD}}$ $\overline{\text{WR}}$	所选定的寄存器	COM ₁
0	0 0 1	可读/写	中断允许寄存器	3F9H
×	0 1 0	只读	中断标志寄存器	3FAH
×	0 1 1	可读/写	通信控制寄存器	3FBH
×	1 0 0	可读/写	MODEM 控制寄存器	3FCH
×	1 0 1	只读	通信状态寄存器	3FDH
×	1 1 0	只读	MODEM 状态寄存器	3FEH
1	0 0 0	可读/写	除数(低 8 位)锁存器	3F8H
1	0 0 1	可读/写	除数(高 8 位)锁存器	3F9H
×	1 1 1		不用	3FFH

在 PC 机中, 串行通信接口大都由 8250 来实现。如图 11.11 所示, 在这种连接的情况下, 8250 的地址由 10 条地址线来决定, 其地址范围为 3F8H~3FFH。

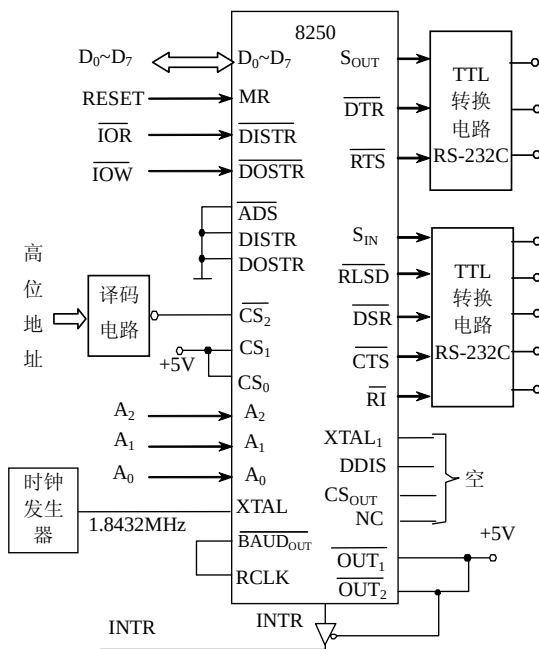


图 11.11 8250 的连接图

利用 8250 进行通信时, 首先要对其初始化, 即设置波特率、数据格式、是否中断、是否自测试操作等。初始化后, 则可采用程序查询或中断方式进行串行通信。实现串行通信的程序随着具体要求的不同, 可能很简单, 也可能十分复杂。

1. 8250 的初始化

对 8250 初始化时, 通常要对部分内部寄存器进行设置, 主要有通信控制寄存器、除数锁存器、MODEM 控制寄存器、中断允许寄存器等, 8250 初始化流程如图 11.12 所示。

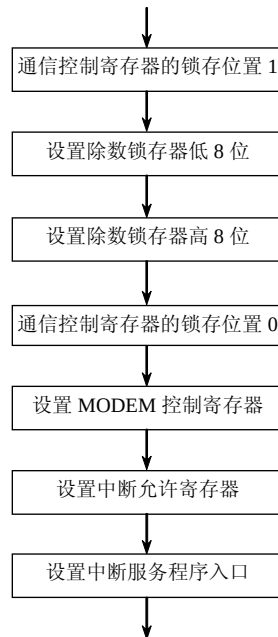


图 11.12 8250 初始化流程图

(1) 设置波特率。假设设置波特率的值为 1200b/s, 由于加在 XTAL₁ 上的时钟频率为 1.8432 MHz, 所以除数为 0060H。为了写除数锁存器, 应该先使通信控制寄存器的最高位(除数锁存位)的值为 1。该程序段如下:

```

MOV    AL, 10000000B    ; 置通信控制寄存器的除数锁存位的值为 1
MOV    DX, 3FBH
OUT    DX, AL
MOV    AL, 60H          ; 置 1200b/s 的除数低位
MOV    DX, 3F8H
OUT    DX, AL
MOV    AL, 00           ; 置 1200b/s 的除数高位
MOV    DX, 3F9H
OUT    DX, AL

```

(2) 设置字符数据格式。假设 7 个数据位, 1 个停止位, 设置奇校验。则该程序段如下:

```

MOV    AL, 00001010B    ; 设置数据格式
MOV    DX, 3F8H
OUT    DX, AL           ; 写入通信控制寄存器

```

(3) 设置 MODEM 控制字。MODEM 控制字有不允许中断输出、允许中断输出和自测试共计三种工作方式。

① 不允许中断输出。该方式的程序段如下:

```

MOV    AL, 03H
MOV    DX, 3FCH
OUT    DX, AL

```

- ② 允许中断输出。该方式的程序段如下：

```
MOV    DX, 3FCH
MOV    AL, 0BH
OUT    DX, AL
```

- ③ 自测试工作方式。该方式的程序段如下：

```
MOV    AL, 13H          ; 自测试工作方式若允许中断则应该为 1BH
MOV    DX, 3FCH
OUT    DX, AL
```

- (4) 设置中断允许寄存器。分为禁止全部中断和允许中断两种情况。

① 允许中断。若允许中断或仅允许四种类型中的某几类中断，则应写入相应的控制字，如允许除 MODEM 状态中断外的其余三种中断，则写入中断允许寄存器值为 07H，则该程序段如下：

```
MOV    AL, 07H
MOV    DX, 3F9H
OUT    DX, AL
```

- ② 禁止全部中断。如果禁止全部中断，则该程序段如下：

```
MOV    AL, 0
MOV    DX, 3F9H
OUT    DX, AL
```

2. 查询方式通信

在初始化程序之后，若采用查询方式发送数据，CPU 可以读取通信状态寄存器的内容来判断发送数据寄存器是否为空，并以此来发送数据；也可以采用查询方式接收数据，CPU 首先读取通信状态寄存器的内容来判断线路是否有错，若无错，再来判断是否已收到一个完整的数据，若是，则从接收数据寄存器中读取接收到的数据。

(1) 查询方式发送数据。假设要发送的字符存放在 SBUF 为首地址的内存中，字符个数存放在 BX 中，则发送这组字符的子程序段如下：

```
SEND_DATA: LEA    SI, SBUF
MOV    DX, 3FDH
TEST11: IN      AL, DX      ; 读取通信状态寄存器的内容
TEST   AL, 20H            ; 判断发送数据寄存器是否为空
JZ     TEST11
PUSH   DX
MOV    DX, 3F8H           ; 发送一个字节到发送数据寄存器
MOV    AL, [SI]
OUT    DX, AL
POP    DX
INC    SI
DEC    BX
JNZ    TEST11
RET
```

- (2) 查询方式接收数据。下面是 8250 接收一个数据的子程序段：

```
REVDATA: MOV    DX, 3FDH      ; 读取通信状态寄存器的内容
```

```

TEST12: IN      AL, DX
         TEST    AL, 1EH      ; 判断线路状态是否有错
         JNZ     ERROR
         TEST    AL, 01H      ; 判断是否接收到一个完整的数据
         JZ      TEST12
         MOV     DX, 3F8H
         IN      AL, DX
         RET

ERROR:   :
         RET

```

3. 中断方式通信

在串行通信中,可以采用查询方式来发送和接收数据,也可以利用 8250 的中断方式实现串行异步通信。

仍然以图 11.11 所示的连接形式为例,假设系统在工作过程中,以查询方式发送数据,而以中断方式接收数据。

(1) 8250 的初始化。该程序段如下:

```

INITIAL: MOV     DX, 3FBH
         MOV     AL, 80H
         OUT     DX, AL
         MOV     DX, 3F8H
         MOV     AL, 0CH      ; 置除数的低位
         OUT     DX, AL
         MOV     DX, 3F9H
         MOV     AL, 0        ; 置除数的高位
         OUT     DX, AL
         MOV     DX, 3FBH
         MOV     AL, 0AH
         OUT     DX, AL      ; 初始化通信控制寄存器
         MOV     DX, 3FCH
         MOV     AL, 0BH
         OUT     DX, AL      ; 初始化 MODEM 寄存器
         MOV     DX, 3F9H    ; 初始化中断允许寄存器
         MOV     AL, 01H
         OUT     DX, AL

STI

```

当 8250 从通信线路上接收了数据并将它存入接收数据寄存器后,就会发出接收数据寄存器满的中断请求信号,此中断请求经过中断控制器 8259 加到 CPU 上,CPU 响应中断后,便转入接收中断服务程序 REE 去执行。中断服务程序 REE 首先应进行断点现场保护,然后检查线路状态寄存器,看是否有差错,若有错,则转向错误处理;若无错,则读入接收的一个字符,并将其存入指定的存储单元 REVBUF 中,并发出中断结束命令给 8259,然后返回中断。

(2) 接收一个字符的中断服务程序段如下:

```

REE: PUSH     AX      ; 现场保护
     PUSH     BX
     PUSH     DX

```

```
PUSH    DS
MOV     DX, 3FDH
IN      AL, DX
TEST    AL, DX
TEST    AL, 1EH
JNZ     ERR                ; 有错, 则转向错误处理
MOV     DX, 3F8H
IN      AL, DX
AND     AL, 7FH
MOV     BX, OFFSET REVBUF
MOV     [BX], AL
MOV     DX, INTRR
MOV     AL, 20H            ; 发出中断命令给 8259
OUT     0A0H, AL
OUT     20H, AL
JMP     ENP
ERR:
:
ENP: POP    DS
POP     DX
POP     BX
POP     AX
STI
IRET
```

11.4 小 结

本章主要介绍了串行通信的基本概念、协议、串行接口标准及可编程串行通信接口芯片 8250 的应用。

由于串行通信方式是计算机中一种非常重要的、广泛应用的通信方式, 所以, 通过本章的学习, 读者应该了解串行通信的基本原理、通信协议、通信方式和各种接口标准, 掌握可编程串行通信接口芯片 8250 的结构、功能及编程应用。

异步通信在传送一个字符时, 由一位低电平的起始位开始, 接着传送数据位, 数据位的位数为 5~8 位。在传送时, 按照低位在前、高位在后的顺序传送。奇偶校验位用于校验数据传送的正确性, 可以没有, 也可以由程序来指定。最后传送的是高电平的停止位, 停止位可以是 1 位、1.5 位或 2 位, 两个字符之间空闲的位要由高电平来填充。

同步通信在传输信息时, 信息是以数据块为单位进行传送, 不仅字符内部位与位之间是同步的, 而且字符之间的传送也是同步的, 因此, 收发双方对时钟同步要求严格, 所以设备也比较复杂。同步通信主要有两种: 一种是面向字符的, 它的特点是把每个帧看作由若干个字符组成的数据块, 并规定了一些特殊字符作为同步字符以及传输过程的控制信息; 另一种是面向比特的, 它的特点是把数据及控制信息看作比特流的组合, 靠约定的比特组合模式来标志帧的开始和结束。

11.5 习 题

1. 选择题

- (1) 异步通信协议规定的字符格式中, 数据位一般是()位。
A. 1~2 B. 5~8 C. 3~4 D. 8
- (2) 在 RS-232C 接口中, CTS 是()。
A. 发送数据线 B. 接收数据线 C. 允许发送 D. 清除发送
- (3) 不属于串行异步通信协议规定的常用波特率是()。
A. 300b/s B. 400b/s C. 600b/s D. 1200b/s
- (4) INS8250 传输速率选择范围是()。
A. 50~9600b/s B. 100~10000b/s C. 20~2000b/s D. 30~3600b/s
- (5) 在串行通信中, 根据数据传送的方向不同, 可将串行通信分为三种方式, 以下哪种方式不属于串行通信的方式()。
A. 单工方式 B. 半双工方式 C. 全双工方式 D. 半单工方式
- (6) 能够完成串行异步通信的硬件电路称为()。
A. UART B. USART C. USRT D. UASRT
- (7) 既能完成异步通信, 又能完成同步通信的硬件电路称为()。
A. UART B. USART C. USRT D. UASRT
- (8) 既能完成异步通信, 又能完成同步通信的典型芯片是()。
A. 8251A B. INS8250 C. MC6852-SSDA D. MC6850-ACIA

2. 简答题

- (1) 什么是串行通信和串行接口?
(2) 串行异步通信的优、缺点是什么?
(3) 串行同步通信的优、缺点是什么?
(4) 8250 共有哪些中断源?
(5) 什么是 UART? 典型的 UART 有哪些?

3. 设计题

- (1) 使用 8250 编程。假设要发送的字符数量存放在 CX 中, 要发送的字符顺序地存放在 DATA 开始的内存区中, 请写出发送这组字符的程序段。
(2) 使用 8250 编程。用查询方式接收一个数据, 写出该子程序段。

第 12 章 系统扩展接口设计

教学提示：系统扩展接口是指计算机与外设之间建立联系、交互信息的接口电路。常见的外部设备主要有键盘、鼠标、显示器、打印机、扫描仪、数码相机等。学习这些外部设备接口的结构、工作原理及使用方法，对于计算机应用系统的开发是十分必要的。

教学要求：本章主要让学生了解各种输入/输出设备的工作原理及其接口电路。重点掌握键盘、CRT 显示器、打印机、鼠标的工作原理、接口及编程方法。

12.1 PC 机与键盘的接口

键盘是从英文打字机演变而来的，是计算机最常用的、也是最主要的标准输入设备。一个标准的通用键盘由按键、编码器和接口电路等主要部件组成，其基本功能就是及时发现被按下的键，并将该按键对应的代码送入计算机。

12.1.1 键盘的分类

PC 系列微机的键盘主要包括 26 个大小写字母键、0~9 数字键、功能键、组合控制键、光标控制键、标点符号及特殊符号键。根据按键本身的结构方式，键盘可分为接触式和电容式两种；根据键盘控制形态的不同，可把键盘分为编码式键盘和非编码式键盘两种。

(1) 接触式键盘。接触式键盘就是通常所说的机械式键盘，这类键盘的每个按键下有两个触点，平时这两个触点不接触，相当于断路，当该按键被按下时，两个触点直接接触，使电路闭合导通，产生信号。这类键盘手感差，易磨损，故障率较高，寿命短。

(2) 电容式键盘。电容式键盘触点之间并非直接接触，而是当按键被按下时，通过改变电容器电极之间的距离，在触点之间形成两个串联的平板电容，从而使脉冲信号通过。电容式键盘击键时无噪声，响应速度快，接触好，手感好，不易磨损，寿命长，灵敏度和稳定性也较好。为了避免电极间进入灰尘，按键采用密封组装，不可拆卸，不易维修。

(3) 编码式键盘。编码式键盘带有硬件电路，有专门的控制电路对键盘进行扫描，当某个按键被按下时，系统能自动检测到该按键所在的位置并能产生相应的代码。这种键盘响应速度快，但硬件电路复杂，而且按键代码固定，不易修改和扩充。

(4) 非编码式键盘。非编码式键盘没有独立的硬件电路，主要采用软件来识别被按下的按键并产生代码，即代码的生成是由键盘和 PC 机软件共同完成的。键盘本身只使用较为简单的硬件来识别被按下的按键的位置，向 PC 机提供按键的位置代码，然后由软件把这些位置代码转换为规定的编码。这种键盘的响应速度不如编码式键盘快，但它可以通过软件为按键重新定义编码，扩充键盘功能方便，应用广。

目前，使用的 PC 机键盘大多采用的是电容式非编码键盘，其按键排列为矩阵式。采用矩阵式结构的最大优点是键盘对外的引线少，如键盘有 $M \times N$ 个按键，则送往计算机的输入线为 $M+N$ 条，如图 12.1 所示。

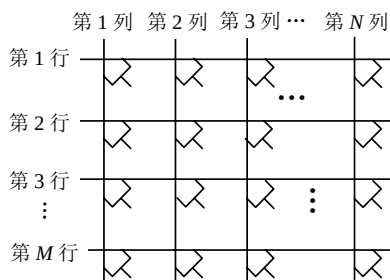


图 12.1 矩阵式结构键盘

12.1.2 键盘的工作原理

一般来说，对于非编码键盘主要考虑的问题是如何识别键盘矩阵中的闭合键，在键盘中通常用硬件或软硬件结合的方法来识别键盘中的闭合键，常用的按键识别方法有：扫描法、反转法。

1. 扫描法

把 PC 系列键盘视为二维矩阵的行列结构，如图 12.2 所示。以 8 行×8 列的键盘为例，当采用扫描法来识别按键时，键盘扫描程序周期性地对按键进行扫描，然后根据回收的信息来确定按键的行和列的位置码。

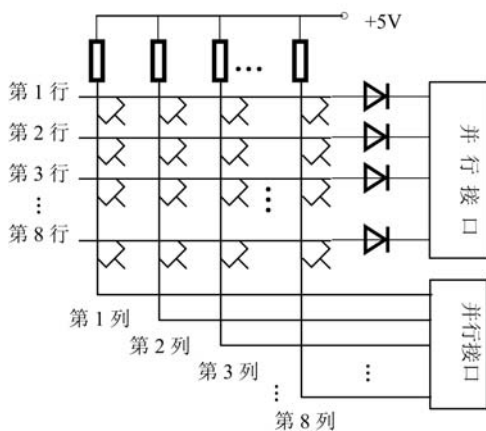


图 12.2 扫描法原理图

例如，先使第 1 行接地，其他行为高电平，然后通过各列来查看第 1 行是否有按键闭合。如果有按键闭合，则该按键所在的列就会变为低电平；如果所有的列都没有变为低电平，则表示第 1 行没有按键被按下。同理，照此方法，再将第 2 行，第 3 行，……逐个接地，然后检测各列是否有变为低电平的。直到最后一行，最后一列。在扫描过程中，若扫描到有按键闭合时，立即退出扫描，并将该按键的行、列的值进行处理，从而就确定了闭合按键所在的位置。

2. 反转法

反转法是利用可编程并行接口(如 8255A)来实现的，在硬件上要求键盘的行和列分别

连接到并行接口中。以 4×4 键盘为例, 如图 12.3, 12.4 所示, 先让列工作在输出方式, 而行工作在输入方式。首先进行列扫描、行检测, 即经过初始化编程, 使 CPU 通过输出端口向各列全部传送低电平, 随后输入端口读入行的值。若读入的数据全为“1”, 表示没有按键被按下, 如果有某个键被按下, 则必有一条行线为低电平。然后通过编程对两个并行端口重新进行方式设置, 使连接行的端口工作在输出方式, 而连接列的端口工作在输入方式, 并将刚才读到的行值通过所连接的并行口再输出到行线上, 然后读取列的值, 那么闭合键所对应的列必为低电平。这样当一个键被按下时, 就可以读到一对唯一的行值和列值。

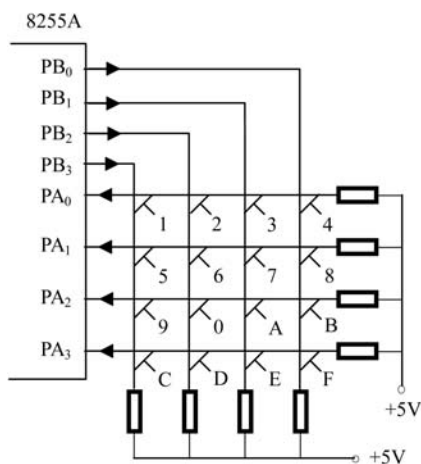


图 12.3 行反转法原理图(列线输出)

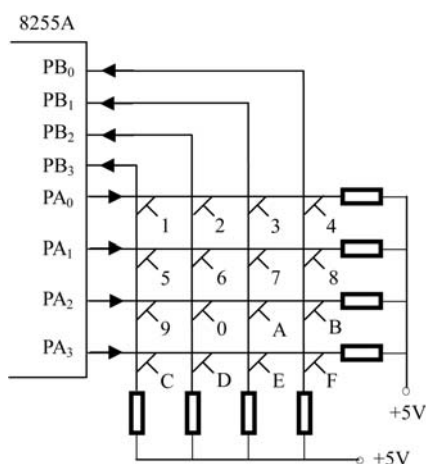


图 12.4 行反转法原理图(行线输出)

例如, 如果标号为“0”的按键被按下, 则第一次向列输出低电平后读入的行值为 1011B, 第二次向行输出 1011B 后, 会从列上读到 1011B, 这样行值和列值合并在一起为 10111011B, 即 BBH, 这个值对应着唯一的 0 号键。

在键盘设计时, 除了对按键码的识别外, 还需要解决抖动和重键问题。我们知道, 按键开关在开闭过程中不可避免地会出现瞬态抖动, 其时间长短约为 5~10ms。在抖动时检测键盘状态是不可靠的, 因此要进行除抖动处理, 除抖动可用硬件或软件实现。硬件除抖动电路通常由一个 R-S 触发器或单稳态电路构成。软件除抖动的方法是在检测到有按键按下时, 先延迟 10ms 再检测键是否仍保持闭合状态。

重键是指在同一时刻有两个或多个键同时按下的情况, 此时存在着是否给予识别或识别哪一个键的问题。对重键的处理, 通常情况下则是只承认先识别出来的键, 对同时按下的其他按键不作识别, 直到所有按键都释放以后, 才读入下一个按键。

12.1.3 键盘接口及应用

PC 系列微机使用编码式键盘, 它的内部由专门的 Intel 8048 芯片完成键盘开关矩阵的扫描、键盘扫描码的读取和发送等功能。键盘接口的作用主要有: 接收键盘送来的扫描码; 输出缓冲区满, 产生键盘中断; 接收并执行系统命令, 对键盘进行初始化、测试、复位等操作。目前 PC 常用的键盘接口有 3 种。

(1) 标准接口。一般用于早期的 AT 主板上, 所以也称作 AT 接口。标准接口为圆形, 比 PS/2 接口要大, 习惯上称之为大口。

(2) PS/2 接口。PS/2 接口为具有 6 针的圆形插座，目前 PC 上一般都具有连接键盘的 PS/2 接口。

(3) USB 接口。由于 USB 设备具有即插即用，支持热插拔等优点，很多设备都采用了 USB 接口，键盘也不例外。选择 USB 接口的键盘主要考虑主机上是否具有空余的 USB 接口。

当 PC 机的键盘接口收到一个字节数据后，通过机内 8259 向 CPU 请求中断，CPU 在中断允许的条件下，响应键盘中断，从而转入 BIOS 的键盘中断服务程序，主要处理的功能如下：

- (1) 从键盘接口读取键盘扫描码，判断是否合法；非法时结束中断，不予处理。
- (2) 如果是特殊键(如 CapsLock、Ins、Alt、Ctrl、Shift 等)，将状态存入 BIOS 数据区的键盘标志单元。
- (3) 将扫描码转换成 ASCII 码，判断转换后的 ASCII 码是否合法。
- (4) 判断键盘缓冲区是否已满，如果已经存满，则中断结束。键盘缓冲区未滿时，将 ASCII 码存入键盘缓冲区，并修改它的指针，结束中断，正常返回。
- (5) 对于系统复位组合键(Ctrl+Alt+Del)、中止组合键(Ctrl+C 或 Ctrl+Break)、暂停(Alt+NumLock)、打印屏幕(Shift+Print)等则直接执行，完成其对应的操作功能。

读取键盘缓冲区中的内容可通过 BIOS 中断或 DOS 中断功能调用来实现，下面分别讨论这两种键盘中断。

(1) BIOS 中断调用。BIOS 键盘中断(INT 16H)提供了基本的键盘操作，它的中断处理程序包括了 3 个不同的功能，分别根据 AH 寄存器中的内容来进行选择，其对应关系如表 12-1 所示。其中功能 0 是获取键盘的字符码和扫描码，功能 1 是判断当前是否有按键按下，功能 2 是查询一些特殊键的状态。

表 12-1 BIOS 键盘中断(INT 16H)

功 能 号	功 能	特 点
AH=00H	从键盘读入一个字符	AL 中的内容为字符码，AH 中的内容为扫描码
AH=01H	读键盘缓冲区的字符	ZF=0 时：AL 中的内容为字符码，AH 中的内容为扫描码； ZF=1 时：缓冲区空
AH=02H	读键盘状态字节	AL 中的内容为键盘状态字节

由于功能键不具有 ASCII 码值，但它们能改变其他按键所产生的代码，采用 AH=02H 的功能可以把表示这些按键状态的字节，即键盘状态字节内容回送到 AL 寄存器中。键盘状态字节内容如表 12-2 所示。

表 12-2 键盘状态字节内容

位 状 态	意 义	位 状 态	意 义
D0=1	按下右 Shift 键	D4=1	ScrollLock 键状态是 ON
D1=1	按下左 Shift 键	D5=1	NumLock 键状态是 ON
D2=1	按下 Ctrl 键	D6=1	CapsLock 键状态是 ON
D3=1	按下 Alt 键	D7=1	Insert 键状态是 ON

(2) DOS 功能调用。DOS 系统功能调用都是通过 INT 21H 号中断调用实现的，如表 12-3 所示，表示与键盘有关的功能调用。

表 12-3 DOS 中断调用(INT 21H)

功 能 号	功 能	特 点
AH=01H	键盘输入并回显	AL=输入字符
AH=06H	直接控制台 I/O	AL=输入字符；或输出一个字符到屏幕(DL=待输出字符)
AH=07H	键盘输入无回显	AL=输入字符
AH=08H	键盘输入无回显	AL=输入字符(检测 Ctrl+Break)
AH=0AH	键盘输入到缓冲区	DS: DX=缓冲区首地址

应用举例：用 INT 16H(AH=00H)中断调用，实现从键盘输入 20 个字符。该程序代码设计如下：

```

DATA    SEGMENT
BUF      DB  20  DUP ( ? )           ; 设置内存缓冲区
MESS     DB  'NO CHARACTER!', 0DH, 0AH, '$'
DATA     ENDS
CODE     SEGMENT
ASSUME   CS: CODE, DS: DATA
START:   MOV     AX, DATA
MOV      DS, AX
MOV      CX, 20                       ; 设置初值
MOV      BX, OFFSET BUF
LOP1:    MOV     AH, 1
PUSH     CX
MOV      CX, 0
MOV      DX, 0
INT      1AH
LOP2:    MOV     AH, 0
INT      1AH
CMP      DL, 100
JNZ      LOP2
MOV      AH, 1
INT      16H
JZ       DONE                          ; 无输入字符，则结束
MOV      AH, 0
INT      16H
MOV      [BX], AL                      ; 把字符存入内存缓冲区
INC      BX
POP      CX
LOOP     LOP1
JMP      EN
DONE:    MOV     DX, OFFSET MESS
MOV      AH, 09H
INT      21H                          ; 显示提示信息
EN:      MOV     AH, 4CH
INT      21H                          ; 返回 DOS
CODE     ENDS
END      START

```

12.2 PC 机与显示器的接口

在计算机系统中, 计算机通过显示设备向外部输出如字符、图形、图像和表格等计算机处理的各种信息。其中, 显示器是最常使用的显示设备, 它可以作为计算机内部信息的输出设备, 又可以与键盘配合作为输入设备。大多数计算机系统主要采用阴极射线管显示器(CRT)和液晶显示器(LCD)两种, 但在一些简单或专用的微机系统中, 往往只需要显示数字, 一般使用简单的数码管(LED)来构成系统的显示设备。

12.2.1 CRT 显示器

CRT 显示器是计算机系统的标准输出设备, 它的结构及原理与电视机相似, 由阴极射线管、视频放大电路和同步控制电路组成。随着大规模集成电路技术的发展, CRT 接口采用了单片机的智能控制, 并且有了大容量屏幕缓冲存储器。近年来, 在显示器的亮度、对比度、行场中心调整等方面均由原来的模拟量控制方法改为数字量控制方法, 使得彩色显示器性能有了很大提高。

CRT 显示器如果按照显像管的颜色来分, 可分为单色显示器和彩色显示器。彩色显示器通常是由红、绿、蓝三种颜色扫描叠加而成。在实际使用中, 人们习惯于以显示器所连接主机的显卡来区分, 如 MDA 单色显示器、CGA 彩色显示器、EGA 彩色显示器、VGA 彩色显示器、TVGA 彩色显示器等。

1. CRT 显示器性能指标

CRT 显示器是目前台式机中最常用的显示设备, 其性能指标主要有以下几方面:

(1) 显示尺寸。显示器的尺寸是指显示器屏幕对角线的长度。目前市场上的显示器主要有 14 英寸、15 英寸、17 英寸及以上的大屏幕显示器。

(2) 屏幕形状。显示器的屏幕形状取决于使用的显像管, 主要有球面、平面直角、柱面和纯平 4 种, 以平面直角和纯平最为常见。

(3) 像素和分辨率。像素是指组成图像的最小单位。比如 800×600 的分辨率是指在水平方向上有 800 个像素, 在垂直方向有 600 个像素。为了控制像素的亮度和彩色深度, 每个像素需要很多个二进制位来表示, 如果要显示 256 种颜色, 则每个像素至少需要 8 位(1 个字节)来表示。每帧画面的像素数决定了显示器画面的清晰度。分辨率是指像素点与点之间的距离, 像素数越多, 图像也就越细腻, 其分辨率就越高, 因此, 分辨率通常是以像素数来计量的。例如, 800×600 的分辨率, 其像素数为 480000。但分辨率受到点距和屏幕尺寸的限制。屏幕尺寸相同, 点距越小, 分辨率越高; 行扫描频率越高, 分辨率相应地也就越高。目前, 14 英寸显示器的分辨率为 1024×768 , 15 英寸显示器的分辨率为 1280×1024 。

(4) 扫描频率。扫描频率分为垂直扫描频率和水平扫描频率。垂直扫描频率是指显示器在某一显示方式下所能完成的每秒从上到下刷新的次数, 垂直扫描频率越高, 图像越稳定, 闪烁越小。水平扫描频率是指电子束每秒在屏幕上水平扫描的次数, 水平扫描的范围越宽, 可支持的分辨率就越高。

(5) 点距。点距是指显示屏上相邻两个色素之间的距离。点距越小, 屏幕上的像素点

就越多越密，则显示图像的效果越好，分辨率越高。

(6) 刷新频率。刷新频率就是每秒屏幕刷新的次数。刷新频率越低，图像闪烁就越明显。

(7) 带宽。带宽是指每秒钟电子枪扫描过的图像点的个数，以 MHz 为单位，它表明了显示器电路可以处理的频率范围。带宽越宽，表明显示控制能力越强，显示效果越好。

2. CRT 显示器的基本结构及工作原理

CRT 显示器主要由阴极射线管(电子枪)、视频放大驱动电路和同步扫描电路等 3 部分组成。彩色 CRT 显示器的组成结构框图如图 12.5 所示。

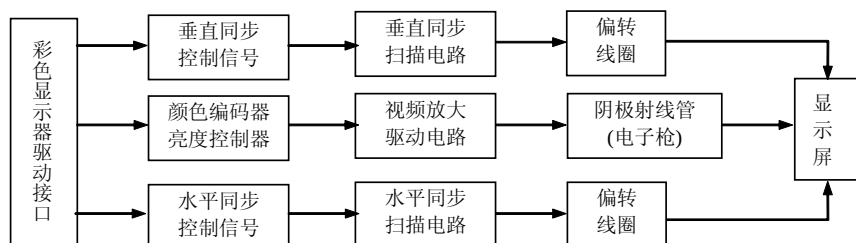


图 12.5 彩色 CRT 组成结构框图

(1) 视频放大驱动电路。接收颜色编码器和亮度控制器输出的红、绿、蓝和亮度控制信号，采用前置放大和末级功率推挽电路输出电流驱动阴极射线管。

(2) 阴极射线管。阴极射线管是 CRT 显示器的主要部分，是由阴极、栅极、加速极和聚焦极及荧光屏组成。当阴极射线管的灯丝加热后，由视频放大驱动电路输出的电流驱动阴极，使之发射电子束。

(3) 同步扫描电路。分为水平同步和垂直同步扫描电路，分别接收水平和垂直同步信号，并输出相应的扫描控制电流，驱动各自的偏转线圈，使电子束在偏转磁场的作用下进行有规律的扫描。

彩色 CRT 显示器的工作原理是：由阴极射线管发射的红、绿、蓝的三色电子束，经栅极、加速极(第一阳极)和聚焦极(第二阳极)，并在高能(第三阳极)的作用下，形成具有一定能量的电子束射向荧光屏。在垂直偏转线圈和水平偏转线圈经相应扫描电流驱动产生的磁场控制下，三色电子束被会聚到荧光屏内侧金属荫罩板上的某一小孔中，并轰击荧光屏的某一位置。此时，涂有荧光粉的屏幕被激发而出现红、绿、蓝三基色之一或由三基色组成的其他各种彩色亮点，从而达到显示的目的。

荧光屏的发光亮度随着加速级电压增加而增加。但通常是控制阴极驱动电流使亮度发生变化。另外，因为三基色阴极由外界 3 个相应的视频信号放大驱动电路所驱动，因此，改变三基色的组合状态可获得色彩的变化。

为了在整个屏幕上显示出字符或图形，必须采用光栅扫描方式。CRT 显示器中有水平和垂直偏转线圈，电子枪产生的电子束通过水平偏转线圈产生的磁场后从左到右做水平方向移动，到右端之后，又立刻回到左端；通过垂直偏转线圈产生的磁场后从上到下做垂直方向移动，到底部之后，又立刻回到上面。由于电子束从左到右、从上到下有规律地周期运动，在屏幕上会留下一条条扫描线，这些扫描线形成了光栅，这就是光栅扫描。如果电

子枪根据显示的内容产生电子束，就可以在荧光屏上显示出相应的图形或字符。

3. 显示器接口的种类及应用

显示器接口就是显示器的适配器，简称显卡。常见的显卡有以下几种：

(1) MDA 标准。MDA 是单色字符显示接口。它只支持 25 行×80 列单色字符显示，不支持图形方式，仅在早期的 PC 中使用。

(2) CGA 标准。CGA 是彩色图形适配器。与 MDA 相比，增加了彩色显示和图形显示两大功能，它支持字符、图形两种方式，但分辨率不高，颜色种类较少，是最早的彩色显卡产品，目前已不使用。

(3) EGA 标准。EGA 为增强型图形适配器，其字符、图形功能比 CGA 卡有较大提高，显示分辨率也较高，显示方式也比 CGA 卡丰富，有 11 种标准模式。

(4) VGA 标准。VGA 是视频图形阵列彩色显示接口，颜色可达 256 色。当分辨率为 800×600 时，可显示 16 色。VGA 卡兼容了上述各种显卡的显示模式，支持更高的分辨率和更多的颜色种类。

(5) SVGA 标准。SVGA 是超级 VGA。它是比 VGA 更强的显示标准。SVGA 的标准模式是 800×600，新型显示器分辨率可达 1280×1024、1600×1200 等。

(6) TVGA 标准。TVGA 是全功能视频图形阵列显示接口，它兼容 VGA 全部显示标准，并扩展了若干字符显示和图形显示的新标准，具有更高的分辨率和更多的色彩选择。当分辨率为 1024×768 时，可显示高彩色或真彩色。

通常，MDA 适配卡、CGA 适配卡适用 XT 机；EGA 适配卡适用于 AT、286、386 机；VGA、TVGA 适配卡适用于 386、486 和奔腾系列微机。

随着计算机技术的高速发展和普及，对视频显示系统的要求也越来越高。显示适配器从早期的文本显示方式，到现在第四代 3D 图形加速卡，在功能、显示速度等方面都有着极大的提高。显卡性能主要有以下几个方面：

(1) 显示分辨率。显示器上每个点的信号来自显示接口，显卡的分辨率不应低于显示器的分辨率。

(2) 刷新速度。显卡的刷新频率与显示器的扫描频率相同时，才能得到满意的效果。

(3) 颜色和灰度。除了可显示的点数，每个点的色彩数也是一个重要指标。色彩数量由显卡上每个像素使用的存储器位数决定。例如，每个点用 16 位存储，可以有 65536 种不同的色彩，也称为“16 位色”。彩色图形卡连接单色显示器时，用灰度等级代替颜色。

视频显示标准反映了各种视频显示图形卡的性能，或显示工作方式、屏幕显示规格、分辨率及显示色彩的种类。从 IBM 公司最早推出的视频显示标准 MDA 开始，陆续形成了一系列新的标准 CGA、EGA、VGA 和 TVGA 等，反映了显示技术的不断发展和人们对显示效果的要求不断提高。显然，在同样尺寸的显示器上，字符显示的列、行数越多，图形显示的分辨率越高，可显示的色彩种类越多，即表明相应的视频显示接口——图形显卡的性能越好。

此外，总线的性能对视频性能产生了很大的影响。采用 PCI 总线接口的显卡可满足一般应用的需要。对于处理大量图形的工作，如 CAD、动画制作等，AGP 接口的图形显卡性能更佳。

显示适配器与 CRT 的接口从其信号形式可分为两大类：数字信号接口和模拟信号接口。数字信号接口是指显示适配器送往 CRT 显示器的信号为数字信号。如前所述的彩色显示适配器，它用数字信号 R、G、B、I 来控制显示的色彩，只能显示 16 色。要增加显示的颜色数必须增加接口信号线，这使得接口信号不易规范。以前的 CGA 和 EGA 显示系统采用的就是这种 TTL 方式的 9 针数字信号接口。

目前，绝大部分显示系统采用的是模拟信号接口。在模拟信号接口中，显示适配器只需 3 路信号线向 CRT 显示器传送表征显示颜色的模拟信号，模拟信号的幅值范围为 0~5V，不同的幅值表示不同的颜色深度。由于模拟信号的连续性，这 3 个信号可以组合出无限的颜色。目前的 VGA 和 TVGA 显示系统采用的就是这种 15 针模拟信号接口。下面重点讨论彩色图形适配器接口电路的编程方法。

(1) 工作方式控制。彩色显示适配器中对工作方式的控制是采用工作方式选择寄存器来实现的，其端口地址为 3D8H，寄存器中各位含义如表 12-4 所示。在彩色图形适配器支持下，彩色图形显示器有以下两种工作方式：

表 12-4 彩色显示器工作方式选择寄存器

位 状 态	意 义	位 状 态	意 义
D ₀ =0	40×25 字符方式	D ₃ =0	不允许视频信号输出
D ₀ =1	80×25 字符方式	D ₃ =1	允许视频信号输出
D ₁ =0	字符方式	D ₄ =0	色彩选定
D ₁ =1	320×200 图形方式	D ₄ =1	640×200 黑白图形方式
D ₂ =0	彩色方式	D ₅ =0	字符模式下不许闪烁
D ₂ =1	黑白方式	D ₅ =1	字符模式下允许闪烁
D ₆	不使用	D ₇	不使用

字符方式。显示器以字符为单位在屏幕上显示字母和数字。在低分辨率显示方式下，屏幕可显示 40×25 个字符，在高分辨率显示方式下，屏幕可显示 80×25 个字符。

图形方式。该方式下通过控制各点的亮度或颜色显示出图形。IBM-PC 支持高分辨率和中分辨率两种图形显示方式。在高分辨率方式下，每屏显示 640(列)×200(行)点，每点可取黑白两种颜色；在中分辨率方式下，每屏显示 320(列)×200(行)点，每点可取两类 4 种颜色。

(2) 彩色选择控制。彩色显示适配器中对彩色选择的控制是采用彩色选择寄存器来实现的，这个寄存器的端口地址为 3D9H，其格式如图 12.6 所示。其中，彩色选择寄存器中的第 0~3 位分别表示蓝色(B)、绿色(G)、红色(R)和加亮(I)，它们可以组合成 16 种不同的颜色，如表 12-5 所示。

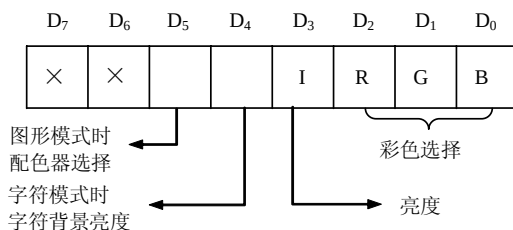


图 12.6 彩色选择寄存器

表 12-5 彩色编码表

状态(IRGB)	颜 色	状态(IRGB)	颜 色
IRGB=0000	黑	IRGB=1000	深灰
IRGB=0001	蓝	IRGB=1001	浅蓝
IRGB=0010	绿	IRGB=1010	浅绿
IRGB=0011	青	IRGB=1011	浅青
IRGB=0100	红	IRGB=1100	浅红
IRGB=0101	洋红	IRGB=1101	浅洋红
IRGB=0110	棕	IRGB=1110	黄
IRGB=0111	淡灰	IRGB=1111	白

在彩色字符方式下，显示缓冲区是按字节与显示内容对应，即字符的 ASCII 码保存于显示缓冲区的偶数地址单元中，彩色属性字节保存于显示缓冲区的奇数地址单元中。字符属性的定义如图 12.7 所示，其中，第 0~3 位用来控制字符颜色；第 3~6 位用来控制背景颜色。

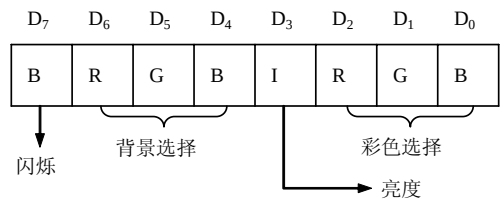


图 12.7 字符属性的定义

在图形方式下，显示缓冲区不是按字节与显示内容对应，而是一个字节对应多个像素点。在高分辨率方式下，每个字节对应 8 个像素点，即一个二进制位对应一个像素点。该二进制位的值决定像素点的亮与不亮。1 为亮，0 为不亮。在中分辨率下，每个字节对应 4 个像素点，即两个二进制位对应一个像素点。两位二进制数可确定 4 种颜色，这样在中分辨率下的每个像素点就可以具有彩色属性，见表 12-6 所示。

表 12-6 像素颜色定义

彩色选择器 D5	像素(C1C0)	像素颜色	彩色选择器 D5	像素(C1C0)	像素颜色
D5=0	C1C0=00	与底色相同	D5=1	C1C0=00	与底色相同
D5=0	C1C0=01	绿	D5=1	C1C0=01	青
D5=0	C1C0=10	红	D5=1	C1C0=10	洋红
D5=0	C1C0=11	黄	D5=1	C1C0=11	白

(3) 状态寄存器。状态寄存器用于标志彩色图形显示器的工作情况，这个寄存器的端口地址为 3DAH，其格式如图 12.8 所示。其中 D0 位用来指出 CRT 显示器是否处在水平扫描或垂直扫描的回扫期，若是，则 CPU 可以向缓冲区送入新的显示信息而不影响屏幕画面；D3 位是字符显示时视频输出信号的瞬间状态，目的是检查是否有信号输出，以便系统的故

障诊断与分析；D₁ 和 D₂ 位用于光笔标志。

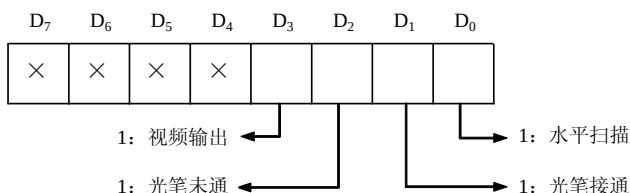


图 12.8 彩色状态寄存器

CPU 通过输入/输出指令访问这些寄存器，以实现对彩色图形显示器的控制。PC 系列机的 ROM BIOS 中有一组驱动 MDA 和 CGA 的显示 I/O 功能程序，显示器中断调用号为 10H，共有 20 个功能，如表 12-7 所示。

表 12-7 BIOS 中断调用(INT 10H)

功 能 号	功 能	功 能 号	功 能
AH=00H	设置显示方式	AH=0AH	在光标位置显示字符
AH=01H	置光标类型	AH=0BH	置彩色调色板
AH=02H	置光标位置	AH=0CH	写像素
AH=03H	读光标位置	AH=0DH	读像素
AH=04H	读光笔位置	AH=0EH	显示字符
AH=05H	置显示页	AH=0FH	取当前显示方式
AH=06H	屏幕初始化或上卷	AH=10H	置调色板寄存器
AH=07H	屏幕初始化或下卷	AH=11H	装入字符发生器
AH=08H	读光标位置的字符和属性	AH=12H	返回当前适配器设置的信息
AH=09H	光标位置显示字符及属性	AH=13H	显示字符串

应用举例：在屏幕上以红底蓝字显示“HELLO”，然后分别以红底绿字和红底蓝字相间显示“CHINA”。该程序代码设计如下：

```

DATA    SEGMENT
STR1    DB  'HELLO'
STR2    DB  'C', 42H, 'H', 41H, 'I', 42H, 'N', 41H, 'A', 42H
LEN     EQU  $-STR2
DATA    ENDS
CODE    SEGMENT
ASSUME  CS: CODE, DS: DATA, ES: DATA
START:  MOV  AX, DATA
        MOV  DS, AX
        MOV  ES, AX
        MOV  AH, 00H
        MOV  AL, 03H                ; 设置 80×25 彩色方式
        INT  10H
        MOV  BP, SEG  STR1
        MOV  ES, BP
        MOV  BP, OFFSET STR1
        MOV  CX, STR2-STR1

```

```

MOV  DX, 00H
MOV  BL, 41H
MOV  AL, 01H
MOV  AH, 13H                ; 显示字符串
INT  10H
MOV  AH, 03H                ; 读光标位置
INT  10H
MOV  BP, OFFSET  STR2
MOV  CX, LEN
MOV  AL, 03H
MOV  AH, 13H                ; 显示字符串
INT  10H
MOV  AH, 4CH                ; 返回 DOS
INT  21H
CODE  ENDS
END   START

```

12.2.2 LCD 显示器

LCD 显示器也称液晶显示器，采用的技术主要分为有源和无源两种。有源液晶显示器又称为薄膜晶体管液晶显示器，其每一个像素点都用一个薄膜晶体管来控制液晶的透光率。它的优点是色彩鲜艳，视角宽，图像质量高，响应速度快。但其成品率低，导致价格昂贵；无源液晶显示器是用电阻代替有源晶体管，成本低，制造较为容易。其缺点是色彩饱和度差，图像不够清晰，对比度也较低，视角窄，响应速度慢。

LCD 显示器具有低眩目的全平面屏幕。有源的 LCD 面板的色彩质量实际上超过了大多数 CRT 显示器。LCD 显示器提供比同尺寸 CRT 显示器更大的可视图像，有 4 种基本的 LCD 选择：无源单色、无源彩色、有源模拟彩色和最新的有源数字彩色。与 CRT 显示器相比，LCD 显示器的特点是体积小、外形薄、重量轻、功耗小、低发热、工作电压低、无污染、无辐射、无静电感应、显示信息量大、无闪烁，并能直接与 CMOS 集成电路相匹配，同时，它是真正的“平板式”显示设备，但价格高，分辨率稍低。液晶显示器，特别是点阵式液晶，已经成为现代仪器仪表用户界面的主要发展方向。

LCD 显示器的成像原理与 CRT 显示器完全不同。LCD 显示器不是用体积较大的显像管进行成像，而是利用液晶的物理特性成像，如图 12.9 所示。

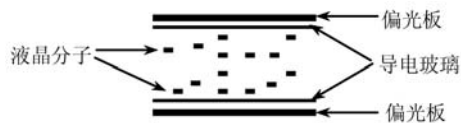


图 12.9 LCD 显示器结构图

LCD 显示器将液晶放置在两片可以导电的无钠玻璃之间，当导电玻璃加电时，中间的液晶分子会按照与导电玻璃垂直的方向顺序排列，使得光线不发生偏移和折射，而穿过液晶直射到对面的玻璃板上成像；当导电玻璃不加电时，中间的液晶分子是无规则分布，会使光线发生偏移和折射，不能直射到对面的玻璃板上，所以不能成像。

LCD 显示屏原是笔记本电脑的主要组件，但随着液晶显示技术的迅速发展，各家厂商竞相推出各种专用的控制和驱动大规模集成电路，使得液晶显示的控制和驱动极为方便，

而且可由 CPU 直接控制，以满足用户对液晶显示的多种要求。目前，许多 PC 机用户都把 CRT 显示器换成 LCD 显示器。

12.2.3 LED 显示器

在微机检测和控制系统的接口电路中，发光二极管(LED)常常作为重要的显示手段，它可以显示系统的状态、数字和字符。LED 显示器的驱动电路简单，价格低廉，因此，由 LED 组成的显示屏被广泛应用于各种场合。

LED 是一种由半导体组成的固态发光器件，在正向导电时能发出可见光，常用的 LED 有红色、绿色、黄色和蓝色。LED 的发光颜色与发光效率取决于制造材料与工艺，发光强度与其工作电流有关。它的工作寿命可长达十万小时以上，可靠性高。它具有类似于普通半导体二极管的特性，在正向导电时端电压近于恒定，通常约为 $1.6\text{V}\sim 2.4\text{V}$ ，工作电流一般约为 $10\text{mA}\sim 200\text{mA}$ 。它适合于与低电压的数字集成电路器件匹配工作。

LED 显示器通常用 7 段发光二极管组成，按“日”字形排列。这 7 段发光二极管称为 a、b、c、d、e、f、g，有的还带有小数点，用 h 表示，如图 12.10(a)所示。通过 7 段发光二极管的不同组合，可以显示 0~9 和 A~F 等 16 个字符，见表 12-8。

表 12-8 LED 显示器字符段码表

显示字符	共阴极段码	共阳极段码	显示字符	共阴极段码	共阳极段码
0	3FH	C0H	8	7FH	80H
1	06H	F9H	9	6FH	90H
2	5BH	A4H	A	77H	88H
3	4FH	B0H	B	7CH	83H
4	66H	99H	C	39H	C6H
5	6DH	92H	D	5EH	A1H
6	7DH	82H	E	79H	86H
7	07H	F8H	F	71H	8EH

各个 LED 可以按照共阴极和共阳极连接，共阴极 LED 的发光二极管阴极共地，当某个二极管的阳极接高电平时，该发光二极管点亮，如图 12.10(b)所示；共阳极 LED 的发光二极管的阳极并接于电源 +5V，如图 12.10(c)所示。

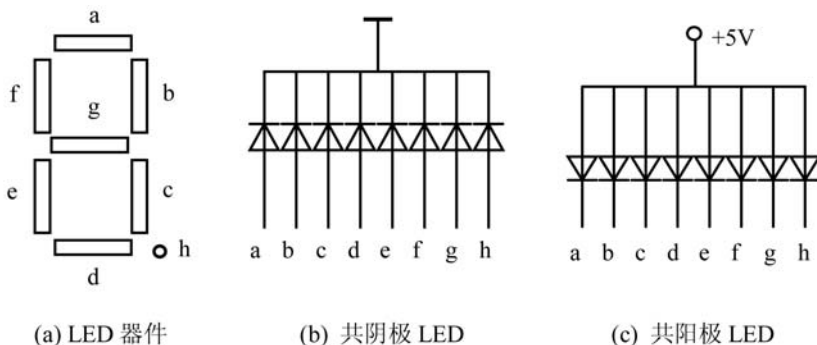


图 12.10 LED 显示器结构图

12.3 PC 机与打印机的接口

打印机是计算机系统中重要的输出设备，其主要功能是将计算机的运行结果及中间信息等打印在纸上，以便于长期保存。

12.3.1 打印机的工作原理

打印机的种类比较多，按照与微机接口的方式分类，有并行输出和串行输出打印机；按照打字方式分类，有击打式和非击打式打印机等。常见的打印机主要有以下几种：

(1) 针式打印机：针式打印机是最早的一种机械式打印机，它的主要工作是接收外部送来的数据或控制命令，然后根据控制命令的格式要求，将要打印的数据变为打印头的动作，利用针头撞击打印机色带将文字或图像数据记录在打印纸上。由于在打印时会产生相当大的噪声，故已逐渐被淘汰。但是它的价格便宜，经久耐用。

(2) 激光打印机：激光打印机属于一种高精度、高速度、低噪声的非击打式打印机，它是通过激光技术和电子照相技术完成印字功能的，主要由激光扫描系统、电子照相系统和控制系统 3 部分组成。

激光扫描系统的主要作用是使激光器产生的激光经调制后，变成载有字符或图形信息的激光束，经过在感光鼓上扫描，形成“静电潜像”。

电子照相系统是让感光鼓接触带有电荷的干墨粉，鼓面被激光照射的部位将吸附墨粉，然后将墨粉转印在纸上，纸上的墨粉经红外线热辐射定影后便形成永久性的字符或图像。

控制系统包括激光扫描控制、电子照相系统控制、缓冲存储器和接口控制等，主要完成接收和处理主机的各种命令和数据以及向主机发送状态。由于在打印时噪声小、速度快、打印质量高，所以激光打印机是目前市场的主流，但价格较高。

(3) 喷墨打印机：喷墨打印机也属于非击打式打印机，按其工作原理可以分为固体喷墨和液体喷墨两种，经常见到的大多是液体喷墨打印机。通过接口接收计算机发送的打印控制命令，经过处理后将图文信息转换为打印数据信号，送往打印头控制与驱动电路，驱动墨水喷出，通过喷头往复运动，形成一行行的字符，从而完成打印过程。这种打印机字迹清晰、美观、速度快，但打印成本高。

采用不同打印技术的打印机的性能有很大差别，一般而言，衡量打印机性能优劣的指标主要包括以下几个方面：

(1) 分辨率：分辨率是衡量打印机打印质量的主要参数之一，单位是 dpi，表示每英寸所打印的点数。例如，一台打印机的打印分辨率是 600dpi，表示每英寸打印 600 个点。分辨率越大，打印精确度越高，但是相应的输出时间越长。要达到好的印刷质量，分辨率应在 400dpi 以上。激光打印机打印质量好于喷墨打印机，喷墨打印机打印质量又好于针式打印机。打印机的价格会随着分辨率的升高而升高，目前市场主流的激光打印机和喷墨打印机的分辨率均达到 600dpi 以上。

(2) 打印速度：指打印机可以打印输出的速度。打印速度的大小关系到打印机的工作效率，也是衡量打印机性能优劣的重要指标之一。

(3) 打印成本：打印成本主要考虑打印所需的纸张、墨盒及打印机自身的购买价格。

(4) 打印幅面：不同打印机所能处理的幅面是不同的，常见的有 A3 和 A4 两种幅面。对于特殊用途，如广告设计、工程晒图等可以考虑更大幅面的打印机。

12.3.2 打印机接口及应用

计算机和打印机之间的数据传输可以用并行方式，也可以用串行方式。在串行接口中，早期的打印机采用的都是 RS-232 接口，现在的打印机通常都配有 USB 接口。而并行接口仍是目前打印机的首选接口。以并行接口为例，打印机与计算机之间通过一根电缆线连接，电缆线一端为 36 芯，与打印机相连，另一端为 25 芯，与计算机并行接口相连。目前，并行打印机的接口通常采用 Centronics 并行接口标准，该标准定义了 36 个引脚，部分引脚含义如表 12-9 所示。

表 12-9 Centronics 标准主要引脚功能

引 脚	方 向	功 能
1	输入	数据选通，有效时接收数据
2~9	输入	数据线
10	输入	响应信号，有效时准备接收数据
11	输出	忙信号，有效时不能接收数据
12	输出	纸用完
13	输出	选择联机，指出打印机不能工作
14	输入	自动换行
31	输入	打印机复位
32	输出	出错
36	输入	有效时，打印机不能工作

计算机的并行打印机接口的逻辑框图如图 12.11 所示，内部有 3 个寄存器，分别对应 3 个端口地址，即数据口、控制口和状态口，计算机可以分别对它们进行读/写操作。其中，控制寄存器的格式如图 12.12 所示，状态寄存器的格式如图 12.13 所示。当主机要向打印机写数据时，由命令译码器产生的控制信号将数据送至数据寄存器，等待写入打印机；主机向打印机发送命令时，欲写入的控制信号送至控制寄存器；反之，主机欲读取状态寄存器时，可以将状态寄存器的内容传送至主机。

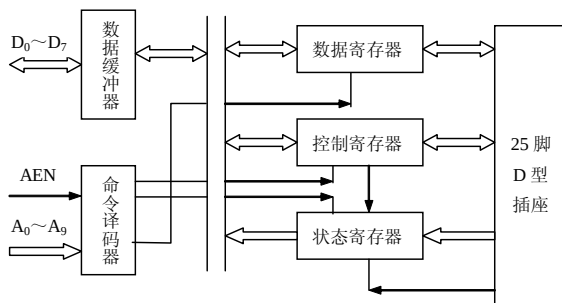


图 12.11 并行接口逻辑图

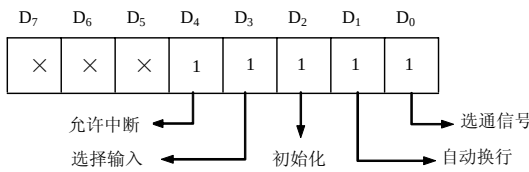


图 12.12 控制寄存器格式

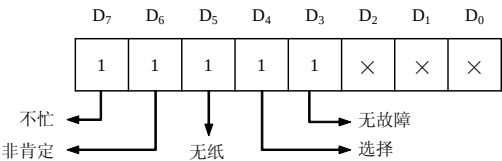


图 12.13 状态寄存器格式

要想让打印机打印字符，可以使用三种方法。一是采用 BIOS 功能调用，在 ROM BIOS 中固化有打印机 I/O 功能调用程序，可利用软件中断 INT 17H 来调用该功能程序，它共包括 3 个子功能，如表 12-10 所示。二是采用 DOS 功能调用，INT 21H 中的 5 号子功能的作用就是打印一个字符，只要是先把想打印的字符送入到 DL 中即可，值得注意的是 DOS 下没有初始化打印接口的功能。三是直接利用端口地址。

表 12-10 打印机 BIOS 中断调用(INT 17H)

AH 值	功 能	入 口 参 数	出 口 参 数
AH=00H	打印一个字符	AL=打印字符，DX=打印机号	AH=打印机状态
AH=01H	初始化打印机	DX=打印机号	AH=打印机状态
AH=02H	读打印机状态	DX=打印机号	AH=打印机状态

应用举例：直接使用端口地址编写程序，使打印机能打印信息。该程序主要由主程序、初始化子程序、打印子程序三部分组成。程序代码如下：

(1) 主程序。

```
DATA SEGMENT
BUF DB '123456', 0AH, 0DH, '$'
N=$-BUF
DATA ENDS
CODE SEGMENT
ASSUME CS: CODE, DS: DATA
START: MOV AX, DATA
MOV DS, AX
CALL INIT ; 调用初始化子程序
MOV CX, N
LEA SI, BUF
LOP: MOV AL, [SI]
CALL PRINT ; 调用打印子程序
LOOP LOP
MOV AH, 4CH
INT 21H ; 初始化子程序
INIT PROC
PUSH AX
PUSH DX
MOV DX, 37AH
MOV AL, 08H
OUT DX, AL
MOV AX, 1000
INIT_1: DEC AX ; 延时
```

```
JNZ    INIT_1
MOV     AL, 0CH
OUT     DX,  AL
POP     DX
POP     AX
RET                                           ; 打印子程序
PRINT   PROC
PUSH    AX
PUSH    DX
MOV     DX,  378H
OUT     DX,  AL
INC     DX
WAT:    IN     AL, DX
TEST    AL, 80H
JZ      WAT
MOV     DX, 37AH
MOV     AL, 0DH
OUT     DX,  AL
NOP
NOP
MOV     AL, 0CH
OUT     DX,  AL
POP     DX
POP     AX
RET
PRINT   ENDP
CODE    ENDS
END      START
```

12.4 PC 机与鼠标的接口

鼠标是一种快速定位器，其功能与键盘的光标键相似。鼠标能方便地将光标准确定位到要指定的屏幕位置，很方便地完成各种操作。因此，在 Windows 操作系统中，鼠标是计算机图形界面人机交互必不可少的输入设备。

12.4.1 鼠标的工作原理

鼠标通常通过鼠标接口与计算机连接，当在桌面上移动鼠标时，把鼠标移动的方向和距离转换成相应脉冲信号传送给计算机，在计算机中安装的鼠标驱动程序将脉冲信号转换为鼠标的水平方向、垂直方向的位移量，从而达到移动鼠标箭头的目的。

当鼠标在平面上移动时，随着移动方向和距离的变化，会产生在高低电平之间不断变化的脉冲信号，CPU 接收这些脉冲信号并对其计数。根据接收到的脉冲信号的情况，CPU 控制屏幕上的鼠标指针在横(X)轴、纵(Y)轴两个方向上移动距离的大小。

根据内部结构及测量位移部件的类型不同，鼠标一般分为机械式鼠标、光机式鼠标和光电式鼠标三种。

1. 机械式鼠标

机械式鼠标的构造最为简单,成本低廉,易于维修。将此类鼠标翻过来看时,会发现其底部有一个橡胶球,紧靠着橡胶球有两个相互垂直的转轴,在转轴的末端安装有译码轮,译码轮通过金属导电片与电刷直接接触。

机械式鼠标的工作原理是:当移动鼠标时,其底部的橡胶球会随之移动,再通过摩擦作用使两个转轴旋转,从而带动译码轮旋转,接触译码轮的电刷及相应电路可计算沿水平方向和垂直方向的偏移量,随即产生与二维空间位移相关的脉冲信号。

机械式鼠标在使用了一段时间以后,下部的橡胶球会因吸附桌面上的灰尘而变得转动不灵活,从而影响其定位精度和灵敏度。另外,由于电刷和译码轮直接接触,磨损也较为厉害,直接影响机械式鼠标的寿命。

2. 光机式鼠标

顾名思义,光机式鼠标就是光电和机械相结合的鼠标。其外形与机械式鼠标没什么区别,它只是在机械式鼠标的基础上,将磨损最厉害的接触式电刷和译码轮改进成为光栅轮,光栅轮上安装有发光二极管和光敏三极管,用于光电检测。

光机式鼠标的工作原理是:当移动鼠标时,其底部的橡胶球会带动两个转轴及光栅轮旋转,使发光二极管发出的光束有时会照射到光敏三极管上,有时光线被遮挡,照射不到。当光束照射到光敏三极管时,其阻值减少;光线被遮挡时,光敏三极管的阻值增大,经过信号处理后使光敏三极管产生与二维空间位移相关的高低电平,形成脉冲信号。

由于采用的是非接触部件,使磨损程度下降,从而大大提高了鼠标的使用寿命,同时在一定的范围内也提高了鼠标的精度。

3. 光电式鼠标

光电式鼠标的底部没有橡胶球及转轴,仅靠发光二极管与光敏三极管的组合,通过光学原理来测量鼠标的位移量,然后将光信号转变为电信号。

光电式鼠标最大的特点是传送速率快,灵敏度和准确度高,可靠性好。

12.4.2 鼠标接口及应用

鼠标接口主要有串行接口、PS/2 接口和 USB 接口 3 种类型,下面分别加以介绍。

(1) 串行接口。传统的鼠标是通过串行通信接口与计算机进行连接的,同其他通信设备一样,鼠标是使用一个 9 芯的连接器与计算机进行连接的。

(2) PS/2 接口。PS/2 鼠标最早由于用在 IBM PS/2 系列微机上而得名。它使用专用的 6 芯鼠标连接插座,安装灵活方便,不占用串口资源。现在的 PC 主板都有支持 PS/2 鼠标接口的插座。

(3) USB 接口。由于 USB 设备具有即插即用、支持热插拔等优点,鼠标也采用了 USB 接口。选择 USB 接口的鼠标要注意主机上是否具有空余的 USB 接口位置。

鼠标中断功能调用都是采用指令 1NT 33H 实现的,表 12-11 列出了鼠标所有的中断功能调用。

表 12-11 鼠标中断功能表(INT 33H)

编 号	作 用	编 号	作 用
AH=00H	初始化鼠标	AH=01H	显示鼠标指针
AH=02H	隐藏鼠标指针	AH=03H	读取按键状态和指针位置
AH=04H	设置指针位置	AH=05H	读取按键按下信息
AH=06H	读取按键释放信息	AH=07H	设置光标横向移动范围
AH=08H	设置光标纵向移动范围	AH=09H	设置图形指针类型
AH=0AH	设置文本指针类型	AH=0BH	读取鼠标移动计数器
AH=0CH	为鼠标事件安装中断处理程序	AH=0DH	打开光笔仿真
AH=0EH	关闭光笔仿真	AH=0FH	设置 mickey 与像素的比率
AH=10H	设置指针禁止区	AH=13H	设置倍速阈值
AH=14H	调用鼠标事件中断	AH=15H	获取鼠标驱动状态缓冲区大小
AH=16H	保存鼠标驱动器状态	AH=17H	恢复鼠标驱动器状态
AH=18H	安装鼠标事件的可选处理器	AH=19H	获取可选处理器的地址
AH=1AH	设置鼠标灵敏度	AH=1BH	获取鼠标灵敏度
AH=1CH	设置鼠标中断速率	AH=1DH	选择指针的显示页
AH=1EH	获取指针的显示页	AH=1FH	禁止鼠标驱动器
AH=20H	允许鼠标驱动器	AH=21H	重置鼠标驱动器
AH=22H	设置鼠标驱动器信息的语种	AH=23H	获取语种数
AH=24H	获取鼠标信息		

应用举例：移动鼠标，在屏幕上显示鼠标的坐标。该程序主要由初始化鼠标子程序、清屏子程序、鼠标位置子程序、将二进制的 X 和 Y 转化为 ASCII 码的子程序、显示 X 和 Y 坐标子程序等模块组成。程序代码如下：

```

STACK      SEGMENT
DB        64 DUP ( ? )
STACK      ENDS
DATA       SEGMENT
LEN_DATA   EQU    34                ; 显示长度
XCOORD     DW     0                ; 二进制 X 坐标
YCOORD     DW     0                ; 二进制 Y 坐标
ASCVAL     DW     ?
DISPDATA   LABEL  BYTE
BEGMESS    DB     'Mouse Positio: '
XMSG       DB     'X='              ; X 的信息
XASCII     DW     ?
           DB     ' '
YMSG       DB     'Y='              ; Y 的信息
YASCII     DW     ?
DISPEND    DB     ' '
DATA       ENDS
CODE       SEGMENT

```

```

ASSUME  CS: CODE, DS: DATA, ES: DATA, SS: STACK
AMAIN   PROC   FAR
START:  MOV    AX, DATA
MOV     DS, AX
MOV     ES, AX
CALL    QCLEAR                ; 清屏
CALL    BINITZ                ; 初始化鼠标
CMP     AX, 00
JE      A90
A20:    CALL    CPOINTR        ; 取鼠标指针
CMP     BX, 01
JE      A80
MOV     AX, XCOORD
CALL    DCONVERT
MOV     AX, ASCVAL
MOV     XASCII, AX
MOV     AX, YCOORD
CALL    DCONVRT
MOV     AX, ASCVAL
MOV     YASCII, AX
CALL    EDISPLY
JMP     A20
A80:    MOV     AX, 02H
INT     33H
A90:    CALL    QCLEAR
MOV     AX, 4C00H
INT     21H                    ; 返回 DOS
AMAIN   ENDP                  ; 清屏子程序
QCLEAR  PROC   NEAR
MOV     AX, 0600H              ; 请求清屏
MOV     BH, 30H
MOV     CX, 00
MOV     DX, 184FH
INT     10H
RET
QCLEAR  ENDP                  ; 初始化鼠标子程序
BINITZ  PROC   NEAR
MOV     AX, 00H
INT     33H                    ; 初始化鼠标
CMP     AX, 00
JE      B90
MOV     AX, 01H
INT     33H
B90:    RET
BINITZ  ENDP                  ; 将二进制的 X 和 Y 转化为 ASCII 码的子程序
DCONVRT PROC   NEAR
MOV     ASCVAL, 2020H
MOV     CX, 10
LEA     SI, ASCVAL+1
CMP     AX, CX
JB      D20
DIV     CL

```

```

OR    AH, 30H                      ; 转换成 ASCII
MOV   [SI], AH
DEC   SI
D20:  OR   AL, 30H
MOV   [SI], AL
RET
DCONVRT   ENDP                      ; 显示 X 和 Y 坐标子程序
EDISPLY   PROC   NEAR
MOV   AX, 1300H                    ; 请求显示
MOV   BX, 0031H
LEA   BP, DISPDATA
MOV   CX, LEN_DATA
MOV   DX, 0018H
INT   10H                          ; 显示 X、Y 位置
RET
EDISPLY   ENDP                      ; 鼠标位置子程序
CPOINTR   PROC   NEAR
MOV   AX, 03H
INT   33H                          ; 取指针位置
CMP   BX, 01H
JE     C90
SHR   CX, 03
PUSH  AX
MOV   AX, CX
MOV   CL, 3
SHR   AX, CL
MOV   CX, AX
SHR   DX, 03
PUSH  CX
MOV   AX, DX
MOV   CL, 3
SHR   AX, CL
MOV   DX, AX
POP   CX
POP   AX
CMP   CX, XCOORD
JNE   C30
CMP   DX, YCOORD
JE     C20
C30:   MOV  XCOORD, CX
MOV   YCOORD, DX
C90:   RET
CPOINTR   ENDP
CODE     ENDS
END      START

```

12.5 PC 机与扫描仪的接口

扫描仪是将各种图片、印刷品等输入到计算机进行处理的主要设备，它内部有一套光电转换系统，可以把各种图片信息转换成计算机图像数据，并传送给计算机，再由计算机

进行图像处理、编辑、存储、打印输出或传送给其他设备。

12.5.1 扫描仪的工作原理

目前，市场上的扫描仪种类很多，如果按色彩来分类，可分成单色和彩色两种；如果按用途来分类，可以分为通用型扫描仪、条码读入器和卡式阅读机；如果按扫描图稿的介质来分类，可以分为反射式(纸材料)、透视式(胶片)扫描仪及既可以扫反射稿又可以扫透视稿的多用途扫描仪；如果按感光器件来分类，可以分为平板式 CCD 和接触式 CIS 扫描仪。所谓的 CCD，又称电荷耦合器，是一种感光半导体芯片，用于捕捉图形，被广泛应用于扫描仪、复印机等设备。不同类型的扫描仪，其结构也不同，但工作原理大同小异，下面以应用最多的平板式 CCD 扫描仪为例进行分析。

平板式 CCD 扫描仪由顶盖、玻璃平台和底座等基本部件构成。玻璃平台用于放置被扫描的图稿，塑料上盖内侧有一个黑色(或白色)的胶垫，其作用是在顶盖放下时，能压紧被扫描文件，目前大多数扫描仪采用了浮动顶盖，以适应扫描不同厚度的对象。

透过扫描仪的玻璃平台，能看到安装在底座上的机械传动机构、扫描头及电路系统。机械传动机构的功能是带动扫描头沿着扫描仪纵向移动；扫描头的功能是将光信号转换为电信号；电路系统的功能是处理和传输图像。

扫描仪的基本原理是通过传动装置驱动扫描组件，将各类文档、相片、幻灯片、底片等稿件经过一系列的光/电转换，最终形成计算机能识别的数字信号，再由控制扫描仪操作的扫描软件读出这些数据，并重新组成数字化的图像文件，供计算机存储、显示、修改、完善，以满足人们各种形式的需要。图 12.14 给出了图像扫描的过程示意图。



图 12.14 扫描仪扫描过程示意图

当被扫描图稿正面向下放置在扫描仪玻璃平台上开始扫描时，机械传动机构带动扫描头沿扫描仪纵向移动，扫描头上光源发出的光线射向图稿，经图稿反射的光信号进入光/电转换器被转换为电信号，经电路系统处理后送入计算机。光/电转换机构沿扫描头横向放置，机械传动机构带动扫描头沿扫描仪纵向每移动一个单位距离，光电转换机构就采集扫描图稿上一条横线上的图形数据，当扫描头沿纵向扫过原稿以后，扫描仪就采集并传输了原稿上的全部图形信息。

12.5.2 扫描仪的主要性能指标及接口

衡量扫描仪性能好坏的指标主要有以下几种。

(1) 分辨率。分辨率是指每英寸图像上所能容纳的颜色点数量，用每英寸长度上扫描图像所含有像素点的个数来表示，单位为 dpi。如 1200dpi 表示的是在 1 英寸之内包含 1200 个颜色点。扫描仪的分辨率越高，能分析的图像画面就越精细，扫描效果也越好。此外，分辨率又有以下 3 种表现形式：

光学分辨率。指的是扫描仪上的感光组件每英寸能捕捉到的图像点数。实际上光学分辨率就是机械分辨率。因此，一台具有 $600 \times 1200\text{dpi}$ 分辨率的扫描仪表示其光学分辨率及

机械分辨率分别以 600dpi 为横轴, 1200dpi 为纵轴。

最大分辨率。指将感光组件所扫描到的图像资料再通过数学演算法所计算出的额外像素。因此, 会出现其光学分辨率可能仅有 600dpi 但最大分辨率却有 9600bpi 的情况。适度地利用数学演算手法将分辨率提高, 能增强扫描到的图像品质。

插值分辨率。指以数学平均值的方法将感光组件上每两个感光体所接收到的颜色平均化, 这些运算是通过特定的硬件芯片来完成的。如果一个 X 轴只有 300dpi 的 CCD, 配以 Y 轴 1200dpi, 再加上 CCD 辅助硬件所做的运算, 就可以制作出一个 600×1200 dpi 或 1200×1200 dpi 插值分辨率的图像。

(2) **色彩深度值。**色彩深度值是指扫描仪对色彩识别的能力大小。它决定了图像的细腻程度、层次和色彩动态范围, 具有和光学分辨率同样的重要性。比如一台拥有 30 位色彩深度的扫描仪, 能表现 2^{30} 种不同颜色, 而拥有 42 位、48 位色彩深度的扫描仪, 能表现的颜色数就更多。

(3) **灰度值。**灰度值是指进行灰度扫描时, 对图像由纯黑到纯白整个色彩区域进行划分的级数。灰度值越高, 扫描仪能够表现的层次就越细。

(4) **扫描幅面。**扫描幅面是表示可扫描图稿的最大尺寸。常见的扫描幅面有 A4、A3 等不同规格。大幅面的扫描仪其价位比较高, 对于一般的家庭及办公用户可以选择 A4 的扫描仪就可以满足需求了。

(5) **扫描速度。**扫描速度是表示在指定分辨率前提下, 在单位时间内所扫描的图像尺寸大小。它与每行的感光时间有关, 一般在 3ms~20ms 之间。彩色扫描仪的扫描速度低于单色扫描仪。

扫描仪接口分为 SCSI 接口、EPP 接口和 USB 接口三种。

(1) **SCSI 接口。**SCSI 接口稳定性高, 传输速度快, 最大传输速率可达 160M/S。但必须配备 SCSI 接口卡, 安装较麻烦, 并且容易产生硬件冲突, 所以现在使用的越来越少, 只在某些高端扫描仪上才配有这种接口。

(2) **EPP 接口。**EPP 接口又称增强模式接口, 使用并行线接在计算机的并行接口上, 安装较方便, 价格相对较低, 但比 SCSI 接口的速度低。

(3) **USB 接口。**USB 接口具有即插即用、支持热插拔等特点, 目前, 大多数扫描仪采用了 USB 接口。选择 USB 接口的鼠标要注意主机上是否具有空余的 USB 接口位置。

扫描仪作为计算机的重要输入设备, 已被广泛应用于报刊、出版印刷、广告设计、工程技术、金融业务等领域之中。它不仅能迅速实现大量的文字录入、计算机辅助设计、文档制作、图文数据库管理, 而且能逼真地录入各种图像, 特别是在网络 and 多媒体技术迅速发展的今天, 扫描仪更能有效地应用于传真、复印、电子邮件等工作。依靠特定软件的支持, 扫描仪还可用于制作电子相册、请柬、挂历等许多个性鲜明和充满乐趣的作品。

12.6 PC 机与数码相机的接口

随着信息技术的发展, 特别是微电子技术和光/电技术的高速发展, 导致了数码影像技术的形成和发展, 如数码相机、数码摄像机、MP3 等数码影像产品都先后在市场推出。

12.6.1 数码相机的工作原理

数码相机与传统相机从外观上看区别不是很大,其共同点都是由光学镜头、取景器、对焦系统、光圈、快门、闪光灯等组成。但数码相机有其特殊的结构,如 CCD 或 CMOS 图像传感器、A/D 转换器、LCD 显示屏、图像处理器、存储介质等部件单元。

(1) CCD(CMOS)图像传感器。CCD 是一种感光半导体芯片,用于捕捉图形,是数字相机的核心器件,它决定着相机的质量。与胶卷的原理相似,光线将穿过镜头,将图像信息投射到 CCD 上,CCD 会把图像信息转化为仿真的电信号,实现光/电转换功能。但和胶卷不同的是,CCD 既没有能力记录图形数据,也没有能力永久保存数据,甚至不具备“曝光”能力。CMOS 也是一种传感器,但它捕获的图像偏黑而且不够精细,在使用上不如 CCD 广泛。

(2) A/D 转换器。由于 CCD 产生的仿真电信号是属于模拟信号,数码相机利用 A/D 转换器将仿真电信号转换为数字信号,并传输给图像处理单元。

(3) 图像处理器。主要实现数字信号处理和图像压缩等功能,首先对 A/D 转换器输出的图像数字信号进行优化处理,然后对图像信号进行压缩,图像压缩的目的是为了节约存储器空间。

(4) 存储介质。在数码相机中,存储介质相当于普通相机中的胶卷,是用来保存图像数据的,所不同的是存储介质中的图像数据可以反复记录和删除。

数码相机的工作原理是:首先利用镜头把要拍摄图像的有关光信号汇聚到 CCD 图像传感器上,然后通过 CCD 将有关图像的光信号转换成仿真的电信号,实现光/电转换。由于该仿真信号是模拟信号,需用 A/D 转换器将仿真电信号转换为数字信号,再经图像处理器对数字信号进行优化和压缩后,存储到存储介质中。通过 I/O 接口,只要将存储器与计算机连接,即可在计算机的显示器上显示存储介质中所存放的图像,并可以进行加工处理或打印机输出。数码相机的工作原理如图 12.15 所示。

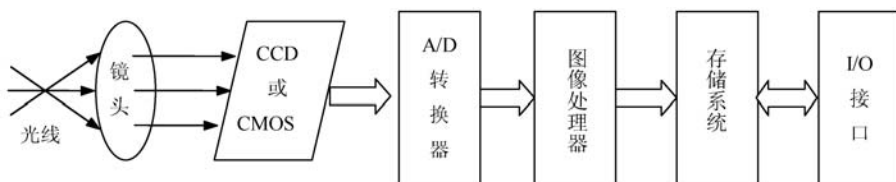


图 12.15 数码相机工作原理框图

12.6.2 数码相机主要技术指标及接口

数码相机的主要技术指标有像素数、分辨率、颜色深度、存储介质、液晶显示屏、快门、镜头、输出接口、电池、光学变焦和数字变焦等,下面仅介绍几种常见的技术指标。

(1) 像素数。像素是指在影像感应器上将光信号转换为电信号的基本工作单位,其决定了照片的分辨率。CCD 图像传感器是数码相机的成像器件,它由一种高感光度的半导体材料制成,能把光线转变为电荷,数字相机的 CCD 内含的晶体管数量越多,分辨率也越高。CCD 的像素数常被用作划分数码相机档次的主要依据。

(2) 分辨率。分辨率是指数码相机的最大分辨率，它的高低取决于相机中 CCD 芯片上像素的多少。像素如果越多，分辨率就越高，图像也越清晰，但生成的数据文件越大。

(3) 颜色深度。颜色深度是描述数码相机对色彩的分辨能力，它取决于 CCD 的光/电转换精度。目前，大多数数码相机的颜色深度都达到 24 位，甚至 36 位。

(4) 存储介质。数码相机的存储介质有内置式和可移动式两种。内置式存储介质是与数码相机固化在一起的，采用内置式存储介质的优点是一旦想拍就可以拍摄，而不需要另配存储介质。可移动式存储介质是随时可装入数码相机或从数码相机中取出的存储介质，存储满后可随时更换。

(5) 液晶显示屏。用来预览或回放拍摄的图像。当发现拍摄得不太好的图像时，就可以立即删除，直观性是其最大的特点。

(6) 光学变焦和数码变焦。光学变焦是真正的变焦，是依靠光学镜头结构来实现变焦的，达到了真正拉近的效果，而数码变焦实际上是一种画面放大，把原来 CCD 上的一部分图像放大到整幅画面，以复制相邻像素平均值的方法补进一个中间值，在视觉上给人一种画面被拉近了的错觉，实际意义不明显。

(7) 镜头。数码相机的成像面积和 CCD 的面积有关，而不同数码相机的 CCD 面积又不尽相等，所以其成像面积是不一样的。这 and 传统相机不同，传统相机的成像面积固定，如 135 相机的成像面积等于 135 胶卷的感光面积，即 $36\text{mm} \times 24\text{mm}$ 。在镜头焦距一定时，成像面积越小，镜头的视角也越小，因此一个镜头在一种数码相机中是广角镜头，而在另一些数码相机中可能就是一般镜头了。在数码相机的各项指标中，大多数的数码相机都有光学变焦镜头，但其变焦范围非常有限，很少有超过 10 倍的，所以这类相机一般都可以安装附加的远距照相镜头和过滤器。

(8) 快门。通常普通数码相机的快门大多在 $1/1000$ 秒之内，基本上可以应付大多数的日常拍摄。快门不单要看“快”还要看“慢”，就是快门的延时。如相机最长具有 15 秒的慢快门，用来拍摄夜景足够了。

数码相机的输出接口主要有串行接口、并行接口、USB 接口和 SCSI 接口等几种类型。一般的数码相机都采用 USB 接口输出。

由于数码相机可将图像数字化，操作简便，特别是能与计算机直接连接，而且在计算机上利用丰富且强大的图像处理软件对图像做各种平面处理，得到更好的艺术效果，因此数码相机被广泛应用于各个领域。由于数码相机具有高速数据传输、大容量存储和快捷方便等特点，近几年，在新闻摄影、网页制作、电子出版、广告设计等领域得到了广泛的应用。

12.7 小 结

本章介绍了几种常见的人机交互设备的结构、工作原理、性能特点、接口及应用等基本知识，其中键盘和鼠标、CRT 显示器、打印机的基本工作原理及编程方法作为重点内容来掌握。在此基础上，对扫描仪、数码相机和触摸屏等新型外设的基本工作原理、结构和工作过程作简要的了解，为今后的应用做好准备。

在计算机中，键盘、鼠标、CRT 显示器和打印机是必备的人机交互设备，它们能够完成各种常规信息的输入和输出。目前大部分计算机都配置通用键盘，如 PC 系列

微机就采用 104 个按键的键盘,从结构上看大多为电容式无触点键盘。键盘工作时要完成键开关状态的可靠输入、键的识别和将键值送给计算机等 3 项任务。鼠标是一种快速定位器,它可以方便地将光标准确定位到要指定的屏幕位置,是计算机图形界面人机交互必不可少的输入设备。显示器是计算机中用来显示各类信息以及图形和图像的输出设备,常用的有 CRT 显示器和 LCD 液晶显示器。CRT 显示器一般采用 15 芯 D 形插座作为与 CPU 联系的接口电路,再通过显卡与主机连接。打印机也是常用的输出设备,它将计算机中的各类信息打印到纸上,可以长期保存。目前常见的打印机有针式打印机、喷墨打印机和激光打印机 3 类,其中激光打印机的打印速度可达每分钟 2000 行,是目前打印机中速度最快的一种。扫描仪是采用光、机、电一体化的计算机外设产品,是除鼠标和键盘之外,计算机使用最广泛的输入设备,不仅能迅速实现大量的文字录入,而且能实时录入和处理各种图像信息,目前应用最多的是平台式 CCD 扫描仪。数码相机的特点是不需要胶卷,在拍摄时图像被聚焦到 CCD 元件上,然后通过 CCD 将图像转换成许多像素,以数字方式存储于相机的存储器中,将存储器与计算机连接后可在显示器上显示所拍摄的图像,并可以进行加工处理或打印输出。

12.8 习 题

1. 选择题

- (1) PC 机与各外部设备之间通过()相连。
A. 网卡 B. 接口电路 C. 控制器 D. 电缆
- (2) 属于输出设备的是()。
A. 鼠标 B. 打印机 C. 扫描仪 D. 数码相机
- (3) 下面()不属于打印机的性能指标。
A. 颜色深度 B. 打印速度 C. 分辨率 D. 打印成本
- (4) 下面()通常不用来当作鼠标的接口。
A. 并行接口 B. 串行接口 C. USB 接口 D. PS/2 接口

2. 简答题

- (1) 简述常用的人机交互设备有哪几类?
- (2) 键盘有哪些种类?
- (3) 鼠标有哪几种类型?简述光机式鼠标的工作原理。
- (4) CRT 显示器主要有哪几部分组成?
- (5) 常见的显卡主要有哪几种?
- (6) 常见的打印机有哪几种?评价打印机性能有哪些主要指标?
- (7) 简要论述扫描仪、数码相机的工作原理。

3. 设计题

- (1) 用 INT 16H(AH=00H)中断调用,设计一个键盘中断调用程序,实现从键盘输入 100 个字符。

(2) 设计显示器接口程序：要求在屏幕上以红底蓝字显示“HELLO”，然后分别以红底绿字和红底蓝字相间显示“WORLD”。

(3) 如何利用中断调用对鼠标进行初始化编程？

第 13 章 数/模和模/数转换器的接口设计

教学提示：数/模和模/数转换接口是计算机与控制对象的一个重要接口，特别是在检测和控制系统中，许多被检测对象是模拟量，不能直接被计算机处理，而必须先利用模/数转换器把模拟量转化为离散的数字信号，再输入计算机进行识别和处理，经计算机加工处理以后输出的控制命令，需利用数/模转换器把数字信号还原成模拟量。在控制系统中，若采集的信息有多种或控制的对象有多个，为了节省硬件，而用一个数/模和模/数转换器实现多个回路的参数采集或控制，则必须使用多路模拟开关。对于变化较快的模拟信号，为了使输入模/数转换器的模拟量在整个转换过程中保持不变，而转换结束后，又能跟随模拟量的变化而变化，则需使用采样保持电路。

教学要求：本章主要让学生掌握数/模和模/数转换器的工作原理及其应用，同时要求了解数/模和模/数转换器的基本概念、结构、典型芯片、主要性能指标以及多路模拟开关和采样保持电路的工作原理。

13.1 概 述

目前，随着数字技术和自动化技术的飞速发展，计算机已广泛地应用于自动控制和测量等领域。在实际控制系统和工业生产过程中，需要进行加工和处理的信号可以分为模拟信号和数字信号两种类型。通常，传感器所检测的信号如温度、压力、流量、速度、湿度等物理量都是随着时间连续变化的模拟信号，而现在广泛使用的微型计算机内部都是采用二进制表示的数字量进行信号的输入、存储、传输、加工与输出。为了能用计算机对模拟信号进行采集、加工和处理，就需要把采集到的模拟信号转换成数字信号送入到计算机中，同样经过计算机处理后的数字信号，要对外部设备实现控制必须将数字信号转换成模拟信号。

计算机控制系统的实现过程如图 13.1 所示，其中，模拟式测量仪表(如传感器)是测量生产或实验过程中的相关参数，即采样；模拟式执行部件是对生产或实验过程的控制，可以采用开关量控制，也可以采用模拟量控制。

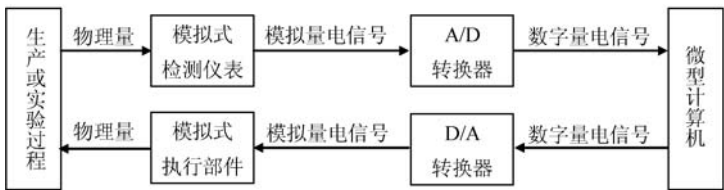


图 13.1 计算机控制系统框图

在计算机系统中，能够完成模拟信号转换成数字信号的过程称作模/数转换，简称 A/D 转换。完成 A/D 转换的装置叫 A/D 转换器(简称 ADC)；同理，能够完成数字信号转换成模

拟信号的过程称作数/模转换,简称 D/A 转换。完成 D/A 转换的装置叫 D/A 转换器(简称 DAC)。

由于 A/D 转换器的工作原理应用到 D/A 转换器中的部分工作原理,所以,先介绍 D/A 转换器。

13.2 D/A 转换器

在微机控制系统中, D/A 转换器是计算机与被控制设备之间传输信息时必不可少的桥梁,担负着把数字量转换成模拟量的任务。D/A 转换器按照可处理数据的位数不同,可分为 8 位、10 位和 12 位等转换器。

13.2.1 D/A 转换器工作原理

典型的 D/A 转换器芯片通常由模拟开关、电阻网络以及缓冲电路组成,其框图如图 13.2 所示。电阻网络是 D/A 转换器的核心部件,主要分为加权电阻网络和“T”形电阻网络两种结构形式。

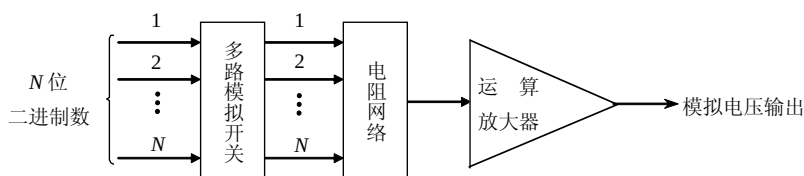


图 13.2 D/A 转换器框图

D/A 转换的基本原理是利用电阻网络,将 N 位二进制数逐位转换成模拟量并求和,从而实现将数字量转化为模拟量。

1. 加权电阻网络 D/A 转换器的工作原理

加权电阻网络 D/A 转换器的原理如图 13.3 所示,对于理想运算放大器而言,其中对于其它支路断开时任意一支开关控制电路在输出端对应的电压为: $V_0 = -V_{REF} \times R_F \times D_i / R_i$ 。式中, D_i 是数字量对应的控制位, $D_i = 0$ 时, S_i 打开, $D_i = 1$ 时, S_i 闭合。若输入端有 N 个支路,根据叠加原理,输出电压应该是各分支之和。

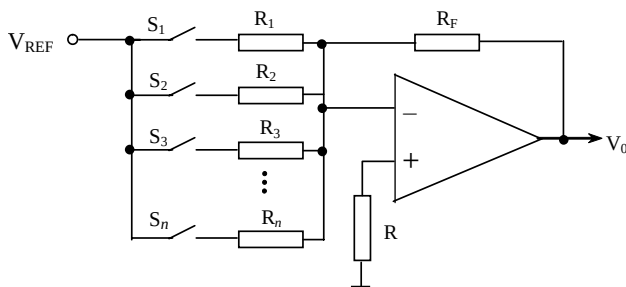


图 13.3 加权电阻网络 D/A 转换器原理图

假定在制造 D/A 转换器时,使 $R_F = R$, $R_1 = 2R$, $R_2 = 4R$, $R_3 = 8R$, $\dots$, $R_n = 2^n R$, 即

每一个电阻的加权为 2^i ，则输出电压可以表示为： $V_0 = -V_{REF} \times (2^{-1}D_1 + 2^{-2}D_2 + 2^{-3}D_3 + \dots + 2^{-n}D_n)$ 。以 8 位转换器为例，当 $D=0$ 时，开关都断开， $V_0=0$ ；当 $D=11111111B$ 时，开关都闭合， $V_0 = -(255/256) \times V_{REF}$ 。

从上面分析可以知道，D/A 转换器的转换精度与基准电压 V_{REF} 和加权电阻的精度以及数字量的位数 n 有关。显然，位数越多，加权电阻的精度越高，转换精度就越高，但同时所需要的加权电阻种类就越多，由于在集成电路中制造大量的高精度、高阻值的电阻比较困难，所以，通常使用“T”形电阻网络来代替加权电阻网络。

2. “T”形电阻网络 D/A 转换器的工作原理

“T”形电阻网络 D/A 转换器的原理如图 13.4 所示，它只有两种电阻 R 和 $2R$ ，用集成电路生产较容易，精度也容易保证，因此应用较广泛。图中的 S_i 同样由输入的数字量对应的控制位 D_i 控制，但是 $D_i=0$ 时， S_i 接地；只有 $D_i=1$ 时， S_i 接通运放输入端。其工作原理与加权电阻网络一样，输出电压为： $V_0 = -V_{REF} \times B/2^n$ 。式中 B ，表示待转换的十进制数字量。

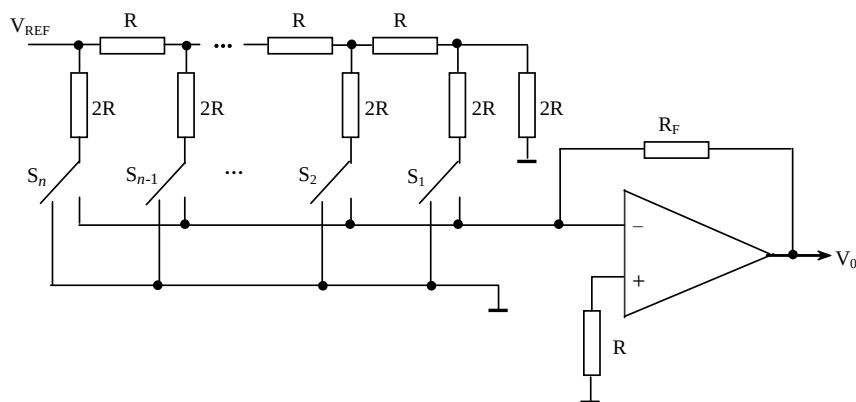


图 13.4 “T”形电阻网络 D/A 转换器原理图

D/A 转换器的输出形式有电压和电流两大类。电压输出型的 D/A 转换器的输出电压量程接近于 V_{REF} ，相当于一个电压源，内阻较小，可带动较大的负载；而电流型的 D/A 转换器相当于一个电流源，内阻较大，与之匹配的负载电阻不能太大。

13.2.2 D/A 转换器性能指标

D/A 转换器的主要性能指标有分辨率、转换精度、建立时间、温度系数、非线性误差等，它们都是衡量 D/A 转换器质量指标的重要参数。

1. 分辨率

分辨率是指 D/A 转换器对数字输入量变化的敏感程度的度量，它是 D/A 转换器所能分辨的最小的输入量，通常用数字量的位数来表示，如 8 位、12 位等。

假设输入信号的满量程电压为 V_{FS} ，那么分辨率为 N 位的 D/A 转换器，它可以分辨出最小电压量是 $V_{FS}/(2^N - 1)$ 。例如，对于 8 位的 D/A 转换器，其分辨率为 $1/(2^8 - 1) = 1/255$ 。若其满量程电压 V_{FS} 为 +5V，则 8 位的 D/A 转换器可分辨的最小电压为 $5/(2^8 - 1) = 1.96mV$ ，

这表明低于此值对应的数字量，转换器将不能进行分辨，这个值又称为最低有效位 LSB。由此可见，位数越多，分辨率越高。

2. 转换精度

转换精度反映了 D/A 转换器的精确程度。它与 D/A 转换器芯片结构、外部电路配置、电源等因素有关。若误差过大，则 D/A 转换就会出现错误。转换精度又分为绝对转换精度和相对转换精度。

绝对转换精度是以理想状态为参照，即 D/A 转换器的实际输出值与理论的理想值之间的差值，一般应低于 $1/2 \text{ LSB}$ 。

相对转换精度是对实际输出电压接近理想状态程度的描述，是指在满量程已校准的情况下，绝对转换精度相对于满刻度(FS)的百分比，或者用最低有效位的几分之几的形式来表示。如一个 8 位 D/A 转换器，相对转换精度可以用 $1/2 \text{ LSB}$ 表示，也可以表示成 0.195%。对于一个相对转换精度为 $1/2 \text{ LSB}$ 的 D/A 转换器，其最大相对误差为 $\text{FS}/2^{N+1}$ 。

3. 建立时间

D/A 转换器的建立时间，也称转换时间，是对 D/A 转换器转换速度快慢的敏感性能描述指标，即当输入数据发生变化后，输出模拟量达到稳定数值，也即进入规定的精度范围内所需要的时间。

在实际应用时，D/A 转换器的转换时间必须不大于数字量的输入信号发生变化的周期。电流型的 D/A 转换较快，电压型的 D/A 转换器响应时间较慢。

4. 温度系数

温度系数是 D/A 转换器受环境温度影响的特征。通常情况下，D/A 转换器的各项性能指标一般在环境温度为 25°C 下测定，当环境温度发生变化时，会对 D/A 转换精度产生影响。

5. 非线性误差

非线性误差也称为线性度，它是指实际转换特性曲线与理想转换特性曲线之间的最大偏差。

一般要求非线性误差的绝对值不大于 $1/2 \text{ LSB}$ 。非线性误差越小，说明线性度越好，D/A 转换器输出的模拟量与理想值的偏差就越小。

13.2.3 典型 D/A 转换器芯片

目前，使用的 D/A 转换器，多数是集成电路芯片。集成 D/A 转换芯片的类型很多，有不同的分类方法。若按位数分，可以分为 8 位、10 位、12 位、16 位等；若按输出方式分，有电流型和电压型两类；若按转换方式分，可分为串行和并行两种；若按工艺分，可分为 TTL 型和 MOS 型等。

典型的 D/A 转换器有 8 位通用型的 DAC0832，12 位的 DAC1208，电压输出型的 AD558 等。下面重点讨论 DAC0832。

DAC0832 是 8 位分辨率的 D/A 转换集成芯片，其明显特点是与微机连接简单、转换控制方便、价格低廉等，在微机系统中得到了广泛的应用。

由于 D/A 转换芯片的输入是数字量，输出为模拟量，模拟信号很容易受到电源和数字信

号的干扰而引起波动，为提高输出的稳定性和减少误差，模拟信号部分必须采用高精度基准电源 V_{REF} 和独立的地线，一般把数字地和模拟地分开。模拟地是模拟信号及基准电源的参考地，其余信号如工作电源地、数据、地址、控制等数字逻辑地的参考地都是数字地。D/A 转换器的输出一般都要接运算放大器，微小信号只有经放大后才能驱动执行机构的部件。

DAC0832 主要技术指标是：分辨率为 8 位，功耗为 20mW，单电源供电，电源范围为 $+5V \sim +15V$ ，建立时间为 $1\mu s$ ，电流型输出。

1. DAC0832 的内部结构

DAC0832 的内部结构框图如图 13.5 所示，它由 8 位输入寄存器、8 位 DAC 寄存器、8 位 DAC 转换器及转换控制电路构成，DAC 转换器采用“T”型电阻网络。

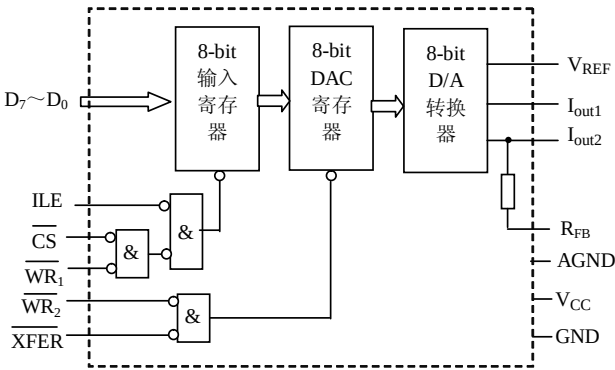


图 13.5 DAC0832 结构框图

2. DAC0832 的引脚功能

DAC0832 的外部封装为 20 脚双列直插式，见图 13.6。各引脚功能如表 13-1 所示。

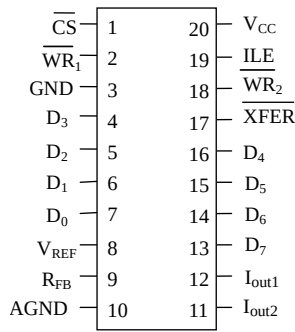


图 13.6 DAC0832 引脚图

表 13-1 DAC0832 引脚功能

引脚名称	功能特点
$D_0 \sim D_7$	8 位数据输入端
$\overline{CS}$	片选信号线，低电平有效
$\overline{ILE}$	数据锁存允许信号，高电平有效

续表

引脚名称	功能特点
$\overline{WR}_1$	输入寄存器的写入控制, 低电平有效
$\overline{WR}_2$	DAC 寄存器写入控制, 低电平有效
XFER	传送控制信号, 低电平有效
I_{out1}	模拟电流输出端口 1, 当 DAC 寄存器全为 1 时, 此电流最大, 当 DAC 寄存器全为 0 时, 此电流最小
I_{out2}	模拟电流输出端口 2, 在数值上, $I_{out1} + I_{out2} = \text{常数}$
R_{FB}	内部反馈电阻引出端, 接运算放大器的输出端
V_{REF}	基准电压输入端, 可在 $-10V \sim +10V$ 之间
V_{CC}	芯片的电源电压, 可在 $+5V$ 或 $+15V$ 选择
GND	数字信号地
AGND	模拟信号地

3. DAC0832 的工作方式

DAC0832 内部有两个寄存器, 即输入寄存器和 DAC 寄存器, 能实现 3 种工作方式: 单缓冲方式、双缓冲方式和直通方式。

(1) 单缓冲方式。单缓冲工作方式是使输入寄存器或 DAC 寄存器中的任意一个工作在直通状态, 另一个由 CPU 控制。通常 $\overline{WR}_2$ 和 $\overline{XFER}$ 连接数字地, 使 DAC 寄存器的输出能够跟随输入, 即第二级寄存器工作在直通状态, 输入寄存器的控制端 $\overline{ILE}$ 接 $+5V$, $\overline{CS}$ 接端口地址译码器输出, $\overline{WR}_1$ 连接系统总线的 $\overline{IOW}$ 信号, 电路连接如图 13.7 所示。

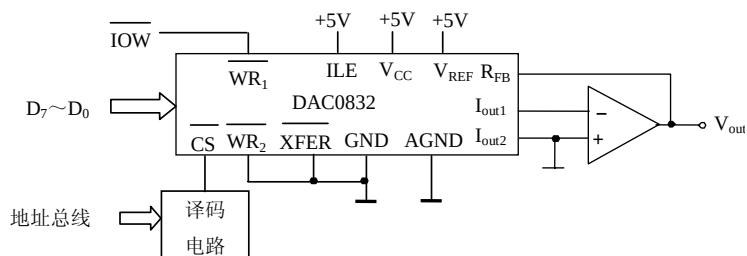


图 13.7 DAC0832 单缓冲方式的电路连接

由于不需检查 D/A 转换器的状态, 因此使用无条件输出方式完成 CPU 向 D/A 转换器进行数据传送。当 CPU 向端口地址发出写命令时, 要进行 D/A 转换的数据通过数据总线呈现在 DAC0832 的 $D7 \sim D0$ 上, 系统 IO 地址经译码电路产生片选信号, 系统总线的 $\overline{IOW}$ 信号为低电平后, 数据写入到输入寄存器中, 因为 DAC 寄存器为直通状态, 所以写入到输入寄存器的数据立刻进行 D/A 变换。

设需要转换的数据为 DATA, DAC0832 端口地址为 PORT, 工作在单缓冲方式下, 完成 D/A 转换的程序段如下:

```
MOV    AL, DATA    ; 转换数据送 AL
MOV    DX, PORT     ; 端口地址送 DX
```

OUT DX, AL ; 将数字量送到 D/A 转换器进行转换

(2) 双缓冲方式。双缓冲工作方式是指输入寄存器和 DAC 寄存器分别受到控制, 该方式电路连接如图 13.8 所示。

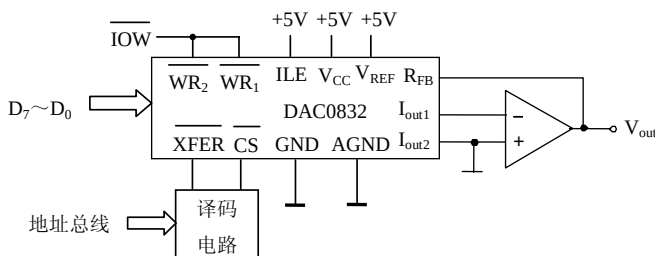


图 13.8 DAC0832 双缓冲方式的电路连接

在双缓冲方式下，CPU 要对 DAC0832 进行两步写操作，即首先将数据写入输入寄存器，然后将输入寄存器的内容写入 DAC 寄存器。设需要转换的数据为 DATA，输入寄存器的端口地址为 Y1，在双缓冲方式下，完成 D/A 转换的程序段如下：

MOV	AL, DATA	;	将转换数据送 AL
MOV	DX, Y1	;	输入寄存器端口地址 Y1 送 DX
OUT	DX, AL	;	数据送入寄存器
MOV	DX, Y2	;	DAC 寄存器端口地址 Y2 送 DX
OUT	DX, AL	;	启动 D/A 转换

(3) 直通方式。直通工作方式是指两个寄存器的有关控制信号都预先置为有效，两个寄存器都开通。只要数字量送到数据输入端，就立即进入 D/A 转换器进行转换。此时， $\overline{\text{CS}}$ 、 $\overline{\text{WR}}$ 、 $\overline{\text{WR}}$ 、 $\overline{\text{XFER}}$ 引脚都直接连接数字地， ILE 信号接高电平，这种方式一般应用较少。

4. DAC0832 的输出

DAC0832 是电流形式输出, 当需要电压形式输出时, 必须外接运算放大器。根据输出电压的极性不同, DAC0832 又可分为单极性输出和双极性输出两种输出方式。

(1) 单极性输出。DAC0832 的单极性输出电路如图 13.9 所示。 V_{REF} 可以接 $\pm 5V$ 或 $\pm 10V$ 参考电压, 当接 $+5V$ 时, 输出电压范围是 $0V \sim -5V$; 当接 $-5V$ 时, 输出电压范围是 $0V \sim +5V$; 当接 $+10V$ 时, 输出电压范围是 $0V \sim -10V$; 当接 $-10V$ 时, 输出电压范围是 $0V \sim +10V$ 。若输入数字为 $0 \sim 255$, 则输出为: $V_{out} = -V_{REF} \times B/256$ 。式中 B 为输入的十进制数, 因为转换结果 I_{out} 接运算放大器的反相端, 所以, 式中有一个负号。若 $V_{REF} = +5V$, 输入数字为: $0 \sim 255$ 时, $V_{out} = -(0 \sim 4.98)V$ 。

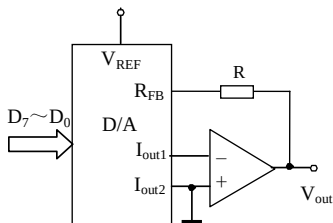


图 13.9 DAC0832 的单极性输出

(2) 双极性输出。即在单极性电压输出的基础上，在输出端再加一级运算放大器，就构成了双极性电压输出，DAC0832 的双极性输出电路如图 13.10 所示。

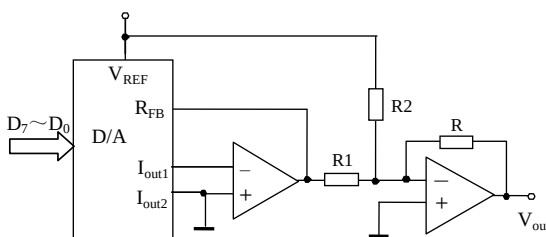


图 13.10 DAC0832 的双极性输出

13.2.4 典型 D/A 转换器的接口及应用

选择和使用 D/A 转换器时要注意以下几点：

(1) 合理选择 DAC 芯片，首先要考虑 D/A 转换器的分辨率和工作温度范围是否满足系统要求。然后根据 DAC 芯片的结构和应用特性，选择外围电路简单、使用方便的 D/A 转换器。

(2) 设计和连接接口时，具有三态输入数据寄存器的 DAC 芯片可直接与计算机 I/O 插槽上的数据总线相接，同时，要为 D/A 转换器配置一个端口地址。

(3) 若 D/A 芯片无参考电源时，则外接的参考电压应该稳定、可靠地工作。

DAC 芯片作为一个输出设备接口电路，与主机的连接比较简单，主要是处理好数据总线的连接。如果要求 DAC 有更高的分辨率时，应该采用 10 位、12 位甚至 16 位的 DAC 芯片。如果仍采用 8 位则被转换的数据必须分几次送出。

应用举例：利用 D/A 转换器来构造波形发生器，如图 13.11 所示。该电路由 DAC0832、双运算放大器 LM358 和地址译码部件组成，将 $\overline{WR}_2$ 、 $\overline{XFER}$ 信号接地，使 DAC0832 工作在单缓冲方式下。由于 DAC0832 是电流输出型的 D/A 转换器，该电路需要电压输出，所以采用双极性输出方式。

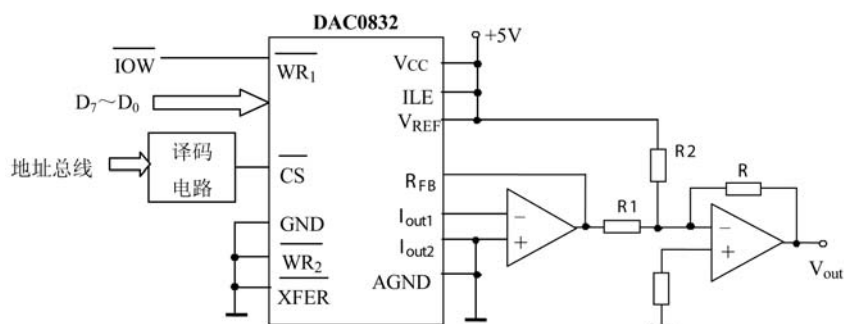


图 13.11 采用 DAC0832 构造的波形发生器

设定地址译码输出端口为 360H，该电路可输出 3 种波形，分别描述如下：

(1) 矩形波。给 DAC0832 持续 256 次送数据 0，然后 256 次送数据 FFH，依次重复处

理，DAC0832 就可输出一个矩形波。输出矩形波的程序段如下：

```

MOV    DX, 360H          ; 设定地址译码输出口
DD0:   MOV    CX, 0FFH
MOV    AL, 00
DD1:   OUT    DX, AL      ; 向 D/A 转换器送数据 0
LOOP   DD1               ; 循环 256 次，形成矩形波的低电平
MOV    CX, 0FFH
MOV    AL, 0FFH
DD2:   OUT    DX, AL      ; 向 D/A 转换器送数据 FFH
LOOP   DD2               ; 循环 256 次，形成矩形波的高电平
JMP    DD0               ; 重复上述的过程，形成多个矩形波

```

(2) 梯形波。给 DAC0832 持续 256 次送数据 0，然后逐次加 1 直到 255，然后持续 256 次，接着将 255 逐次减 1，依次重复处理，DAC0832 就可输出一个梯形波。输出梯形波的程序段如下：

```

MOV    DX, 360H          ; 设定地址译码输出口
MOV    CX, 0FFH
MOV    AL, 00
DD1:   OUT    DX, AL      ; 向 D/A 转换器送数据 0
LOOP   DD1               ; 循环 256 次，形成梯形波的下底
MOV    CX, 0FFH
DD2:   INC    AL          ; 循环加 1，以形成上升沿
OUT    DX, AL
LOOP   DD2
MOV    CX, 0FFH
DD3:   OUT    DX, AL      ; 输出上底
LOOP   DD3
MOV    CX, 0FFH
DD4:   DEC    AL
OUT    DX, AL            ; 输出下降沿
LOOP   DD4
JMP    DD1               ; 重复上述过程，形成多个梯形波

```

(3) 三角波。给 DAC0832 持续 256 次送数据 0，然后逐次加 1 直到 255，接着将 255 逐次减 1 到 0，依次重复，DAC0832 就可输出一个三角波。输出三角波的程序段如下：

```

MOV    DX, 360H          ; 设定地址译码输出口
DD0:   MOV    CX, 0FFH
MOV    AL, 00
DD1:   OUT    DX, AL      ; 向 D/A 转换器送数据 0
INC    AL
LOOP   DD1               ; 循环形成上升斜坡
MOV    CX, 0FFH
DD2:   DEC    AL
OUT    DX, AL
LOOP   DD2               ; 循环形成下降斜坡
JMP    DD0               ; 重复上述过程，形成多个三角波

```

13.3 A/D 转换器

在实际应用中,经常要把传感器从现场检测到的各种物理量,如温度、压力、流量、速度、角度等,转换成电信号,经过放大、滤波后,送到 A/D 转换器,由 A/D 转换器实现模拟量电压到数字量电压的转换,然后送到计算机中。

模拟量电压的大小随着时间不断地变化,为了通过转换得到确定的值,对连续变化的模拟量要按一定的规律和周期取出其中的某一瞬时值进行转换,这个值称为采样值。抽样定理指出:采样频率一般要高于或至少等于输入信号最高频率的 2 倍,实际应用中采样频率一般是信号最高频率的 3~5 倍。对于变化较快的输入模拟信号,A/D 转换前可设置采样保持电路(有关采样保持电路的知识在 13.4.2 小节中介绍),使得在转换期间保持固定的模拟信号值。

相邻两次采样的间隔时间称为采样周期。为了使输出量能充分反映输入量的变化情况,采样周期要根据输入量变化的快慢来决定,而一次 A/D 转换所需要的时间显然必须小于采样周期。

将模拟量表示为相应的数字量称作量化。与数字量的最小有效位 1LSB 相对应的模拟电压称为一个量化单位,如果模拟电压小于此值就不能转换为相应的数字量。量化得到的数值通常用二进制表示,有正负极性(双极性)的模拟量一般采用偏移码表示,数值为负数时符号位用 1 表示,为正数时符号位用 0 表示。例如,8 位二进制偏移码 00000000 代表 0 电压,01111111 代表正电压满量程,11111111 代表负电压满量程。

13.3.1 A/D 转换器的工作原理

实现 A/D 转换的方法很多,由于应用的特点和要求不同,需要采用不同工作原理的 A/D 转换器。A/D 转换器主要有计数式、并行式、双积分式、逐次逼近式等。这里主要介绍最常用的逐次逼近式 A/D 转换器的工作原理。

图 13.12 给出了 4 位的逐次逼近式 A/D 转换器的逻辑框图。其组成模块主要有数据发生器、数码寄存器、电阻开关网络和比较器。工作时,启动信号 START 使数据发生器开始工作,数据发生器在时钟脉冲的作用下,首先将数码寄存器的最高位置“1”(即 $D_3=1$, $D_2=0$, $D_1=0$, $D_0=0$),数码寄存器的输出经过电阻开关网络,转换成相应大小的反馈比较电压 V_R 。 V_R 与模拟量输入电压 V_{IN} 经过比较器比较,若 $V_{IN} > V_R$,说明数码还不够大,该位的“1”应该保留(即 $D_3=1$);反之,若 $V_{IN} < V_R$,说明数码过大,该位的“1”应该去掉(即 $D_3=0$)。按照这样方式,再使数码寄存器的次高位置“1”(即 $D_2=1$),并经电阻开关网络转换成相应大小的 V_R ,再次与 V_{IN} 经过比较器比较,同样,若 $V_{IN} > V_R$,该位的“1”应该保留(即 $D_2=1$);反之,若 $V_{IN} < V_R$,该位的“1”应该去掉(即 $D_2=0$)。如此这样,由最高位开始,逐位的比较下去,一直到最低位为止。数码寄存器中最终保留的数据就是转换结果。显然,转换结果的大小与输入的模拟量成正比,当输入模拟量取值为 $0 \sim V_{REF}$ 时,转换结果在 0000~1111B 范围内相应的取值,从而实现了 A/D 转换。转换结束时,A/D 转

换器给出转换结束信号 EOC。

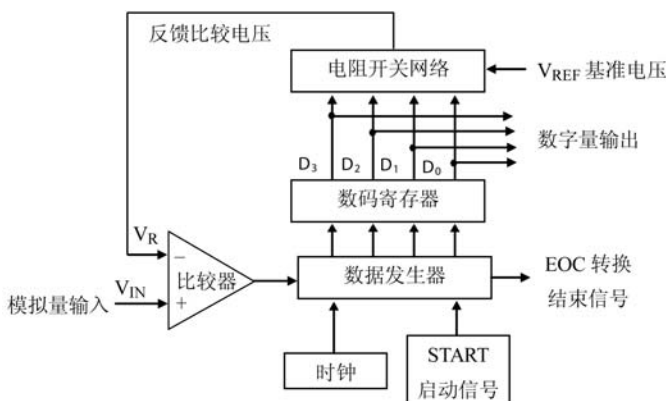


图 13.12 逐次逼近式 A/D 转换器逻辑框图

逐次逼近式 A/D 转换器的主要特点是：转换速度较快，转换时间固定，不随输入信号的大小而变化，分辨率可达 18 位，特别适用于高精度和高频信号的 A/D 转换。逐次逼近型 A/D 转换器是目前应用较多的 A/D 转换器芯片。

13.3.2 A/D 转换器的主要性能指标

A/D 转换器的主要性能指标有分辨率、转换精度、转换时间、温度系数、量程等。它们都是衡量 A/D 转换器质量指标的重要参数。

1. 分辨率

反映了 A/D 转换器对输入微小信号变化的响应能力，即能够分辨的最小模拟量。通常用输出的数字量的最低有效位(LSB)所对应的模拟输入电平值表示。例如，对于 8 位 A/D 转换器，输入电压满刻度为 5V 时，则对应输入的模拟电压的分辨率为 $5V/255=19.5mV$ 。由于分辨率直接与转换器的位数有关，所以也可简单地用数字量的位数来表示分辨率。例如，一个输出为 8 位二进制数的 A/D 转换器，称其分辨率为 8 位。

2. 转换精度

转换精度反映了 A/D 转换器实际输出与理想输出的接近程度，通常采用绝对误差和相对误差两种方法来表示。A/D 转换器的位数越多，相对误差和绝对误差就越小。

绝对误差等于实际转换结果与理论转换结果之差，通常以数字量的最小有效位的分数值来表示。例如： $\pm 1/2$ LSB、 $\pm 1/4$ LSB 等。

相对误差是指整个转换范围内的任一数字量所对应的模拟输入量的实际值与理论值之差，通常用绝对误差与模拟电压满量程的百分比来表示。例如，假设有一个 10 位的 A/D 芯片，其满量程为 10V，绝对精度为 $\pm 1/2$ LSB。则量化单位为 $10V/(2^{10}-1)=9.77mV$ ，绝对精度为 $9.77mV/2=4.88mV$ 。所以相对精度为 $4.88mV/10V=0.048\%$ 。

3. 转换时间

指完成一次 A/D 转换所需要的时间，即由发出转换命令信号到转换结束信号开始有效

的时间间隔。转换时间反映了 A/D 转换器的转换速度。

4. 温度系数

表示 A/D 转换器受环境温度影响的程度, 一般采用环境温度变化 1°C 所产生的相对转换误差来表示。

5. 量程

指 A/D 转换器能够转换的模拟输入电压的变化范围, 分单极性、双极性两种类型。例如, 单极性常见量程为 $0\sim 5\text{V}$, $0\sim 10\text{V}$, $0\sim 20\text{V}$; 双极性量程通常为 $-5\text{V}\sim +5\text{V}$, $-10\text{V}\sim +10\text{V}$ 等。

13.3.3 典型 A/D 转换器芯片

市场上 A/D 转换器芯片种类很多, 典型的 A/D 转换器芯片主要有 ADC0809、ADC1210、AD574、AD674、AD678 等, 下面重点讨论 ADC0809。

ADC0809 是 8 位、8 通道逐次逼近式 A/D 转换器, 片内有 8 路模拟开关, 可以同时连接 8 路模拟量, 单极性, 量程为 $0\sim 5\text{V}$ 。片内有三态输出锁存器, 可以直接与微机总线相连接。该芯片有较高的性能价格比, 适用于对精度和采样速度要求不高的场合或一般的工业控制领域。由于其价格低廉, 便于与微机连接, 因而应用十分广泛。

ADC0809 主要的技术指标是: 8 位分辨率, 增益温度系数为 0.02% , 功耗为 20mW , 单电源 $+5\text{V}$ 供电, 转换时间为 $100\mu\text{s}$, 能对 8 路模拟电压信号进行转换, 输出电平与 TTL 电平兼容。

1. ADC0809 的内部结构

ADC0809 的内部逻辑结构如图 13.13 所示, 主要由 8 路模拟选通开关、8 位 A/D 转换器、8 位三态输出锁存器和地址锁存器等组成。其中, 地址选择信号(ADDC $\sim$ ADDA)与 8 路模拟通道(IN7 $\sim$ IN0)之间的对应关系见表 13-2。

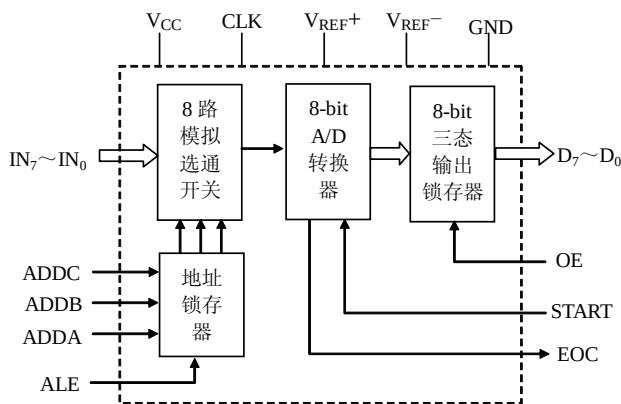


图 13.13 ADC0809 结构框图

表 13-2 模拟通道地址选择

地址选择信号状态			所选的通道	地址选择信号状态			所选的通道
ADDC	ADDB	ADDA		ADDC	ADDB	ADDA	
0	0	0	IN0	1	0	0	IN4
0	0	1	IN1	1	0	1	IN5
0	1	0	IN2	1	1	0	IN6
0	1	1	IN3	1	1	1	IN7

ADC0809 的转换过程是：

- (1) 在 ALE 信号的作用下，地址引脚 ADDC~ADDA 上的信号被地址锁存器锁存并选择相应的模拟信号，随后被选择的模拟信号进入 A/D 转换器。
- (2) 在启动脉冲 START 的作用下，A/D 转换器进行转换。
- (3) 转换完成后，EOC 由低电平变为高电平，该信号可以作为状态信号由 CPU 查询，也可以作为中断请求信号通知 CPU 本次 A/D 转换已经完成。
- (4) CPU 通过执行读 ADC0809 数据端口指令，使 OE 有效，打开三态输出锁存器，使转换结果通过系统数据总线进入 CPU。

2. ADC0809 的引脚功能

ADC0809 引脚图如图 13.14 所示，各引脚功能见表 13-3。

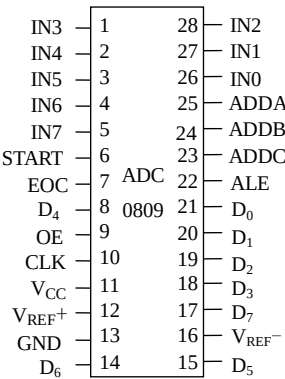


图 13.14 ADC0809 引脚图

表 13-3 ADC0809 引脚功能

引脚名称	功能特点
IN7~IN0	8 路模拟输入，可连接 8 路模拟量的输入
ADDC~ADDA	通道地址选择，用于选择 8 路模拟输入中的一路输入，具体情况见表 13-2 所示
D7~D0	8 位数据输出端
ALE	通道地址锁存信号，用来锁存 ADDC~ADDA 端的地址输入数据
OE	输出允许信号，三态输出锁存器将 A/D 转换结果输出
START	A/D 转换启动信号
EOC	转换结束状态信号

续表

引脚名称	功能特点
CLK	时钟输入端
V _{CC}	芯片的电源电压，+5V
V _{REF} ⁺ 和 V _{REF} ⁻	参考电压，要求电压相当稳定。其中 V _{REF} ⁺ 典型值为+5V，V _{REF} ⁻ 典型值为 0V
GND	地线

13.3.4 典型 A/D 转换器的接口及应用

通常使用的 A/D 转换芯片一般都具有数据输出、启动转换、转换结束、时钟和参考电平引脚。ADC 芯片与主机的连接主要就是处理这些引脚的连接问题。

(1) 数据输出线的连接。模拟信号经 A/D 转换后向主机送出数字量，所以 ADC 芯片就相当于给主机提供数据的输入设备。能够向主机提供数据的外设很多，它们的数据线都要连接到主机的数据总线上，为了防止总线冲突，任何时刻都只能有一个设备发送信息。因此，这些能够发送数据的外设，其数据输出端都必须通过三态输出锁存器连接到数据总线上。

此外，随着位数的不同，ADC 与微处理机数据总线的连接方式也不同。对于 8 位 ADC，其数字输出端可以与 8 位微处理机数据总线相连，然后用一条输入指令一次读出结果。但对于 8 位以上的 ADC 与 8 位微处理机连接就不那么简单了，此时必须增加读取控制逻辑，把 8 位以上的数据分两次或多次读取。

(2) A/D 转换的启动信号。开始 A/D 转换时必须加一个启动信号，芯片不同，所要求的启动信号也不同，一般有脉冲启动信号和电平控制信号。脉冲信号启动转换时，只要在启动引脚加一个脉冲即可，如 ADC0809、AD574 等芯片；电平信号启动转换是在启动引脚上加一个所要求的电平，且在转换过程中必须保持这一电平不变。

此外，采用软件也可以实现编程启动，通常是在要求启动 A/D 转换的时刻，用一个输出指令产生启动信号。还可以利用定时器产生信号，方便地实现定时启动，该方法适合于固定延迟时间的巡回检测等应用场合。

(3) 转换结束信号的处理方式。当 A/D 转换结束，ADC 输出一个转换结束信号，通知主机 A/D 转换已经结束，可以读取结果。主机检查判断 A/D 转换是否结束的方法主要有中断方式、查询方式、延时方式和 DMA 方式。

中断方式是把转换结束信号作为中断请求信号接到主机的中断请求线上。当转换结束时向 CPU 申请中断，CPU 响应中断后在中断服务程序中读取数据。该方式下 ADC 与 CPU 同时工作，适用于实时性较强或参数较多的数据采集系统。

查询方式是把转换结束信号作为状态信号经三态缓冲器送到主机系统数据总线的某一位上。主机在启动转换后开始查询是否转换结束，一旦查到结束信号便读取数据。该方式的程序设计比较简单，实时性也较强，是比较常用的一种方法。

延时方式是不使用转换结束信号的，主机启动 A/D 转换后，延时一段略大于 A/D 转换的时间即可读取数据。可以采用软件延时程序，此时无需硬件连线，但要占用主机大量时间，也可以用硬件完成延时。该方式多用于主机处理任务较少的系统中。

DMA 方式是把结束信号作为 DMA 请求信号，转换结束即启动 DMA 传送，通过 DMA

控制器直接将数据送入内存缓冲区。该方式适合于要求高速采集大量数据的场合。

(4) 时钟的提供。时钟是决定 A/D 转换速度的基准，整个转换过程都是在时钟作用下完成的。时钟信号可以由外部提供，用单独的振荡电路产生，或用主机时钟分频得到；也可以由芯片内部提供，一般采用启动信号来启动内部时钟电路。

(5) 参考电压的接法。当模拟信号为单极性时，参考电压 V_{REF-} 接地， V_{REF+} 接正极电源；当模拟信号为双极性时， V_{REF+} 和 V_{REF-} 分别接电源的正、负极性端。当然也可以把双极性信号转换为单极性信号再接入 ADC。根据 A/D 转换芯片的数字输出端是否带有三态锁存缓冲器，与主机的连接可以分为两种方式：第一种是直接相连，主要用于输出带有三态锁存缓冲器的 ADC 芯片，如 ADC0809、AD574 等。第二种是用三态锁存器，如 74LS373/374，或通用并行接口芯片，如 Intel 8255A，适用于不带三态锁存缓冲器的 ADC 芯片。

由于 ADC0809 的数据输出端为 8 位三态输出锁存器，所以数据线可直接与微机的数据线相接，但因为无片选信号线，所以需要相关的逻辑电路与之相匹配。下面讨论 ADC0809 与 CPU 连接的两种方式。

(1) ADC0809 与 CPU 直接连接。如图 13.15 所示，占用 3 个 I/O 端口：端口 1 用来向 ADC0809 输出模拟通道号并锁存；端口 2 用于启动转换；端口 3 读取转换后的数据结果。采用软件延时，则转换程序段如下：

```
MOV    AL, 07H
OUT    1FH, AL
CALL   DELAY100
IN     AL, 1FH
HLT
```

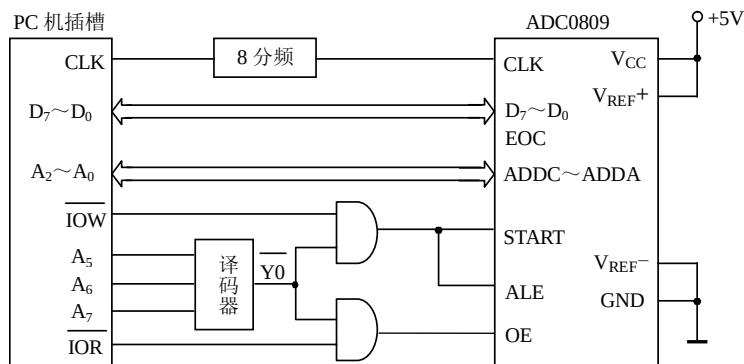


图 13.15 ADC0809 与 CPU 直接连接

(2) ADC0809 通过 8255A 与 CPU 连接。硬件电路的连接如图 13.16 所示，该系统可对 8 路模拟量分时进行数据采集。转换结果采用查询方式传送，除了一个传送转换结果的输入端口外，还需要传送 8 个模拟量的选择信号和 A/D 转换的状态信息。将 8255A 的 A 口输入方式设定为方式 0，B 口的 $PB_7 \sim PB_5$ 输出用来选择 8 路模拟量的地址选通信号， PC_1 输出 ADC0809 的控制信号， PB_0 作为启动信号。由于 ADC0809 需要脉冲启动，所以通过软件编程让 PB_0 输出一个正脉冲，EOC 信号接 PC_1 。

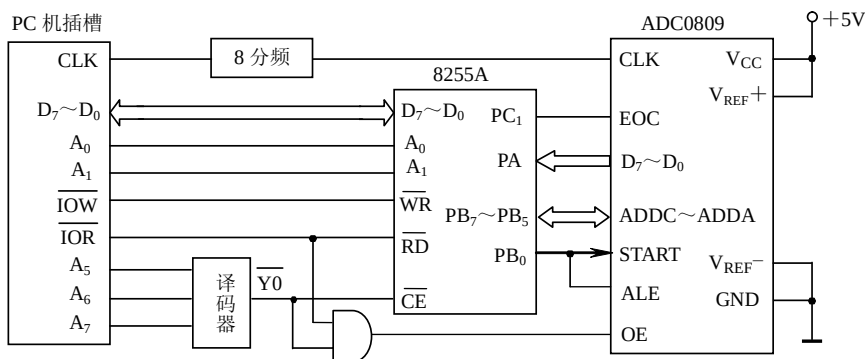


图 13.16 ADC0809 通过 8255A 与 CPU 连接

设定 8255A 的 A 口、B 口、C 口及控制口地址分别为 1CH、1DH、1EH 和 1FH，A/D 转换结果的存储区的首地址设为 40H，采样顺序从 IN0 到 IN7。译码器输出 $\overline{Y0}$ 选通 8255A，8255A 可设定 A 口为输入，B 口为输出，均为方式 0。以查询方式读取 A/D 转换后的结果，该程序段设计如下：

```

MOV    DX, 1FH                ; 8255 初始化
MOV    AL, 99H
OUT    DX, AL
MOV    SI, 40H                ; 保存数据存储地址
MOV    CX, 08H
    MOV    BH, 00H
LOOP1: MOV    BH, 08H
MOV    AL, 01H
MOV    DX, 1DH                ; 启动 0809
OUT    DX, AL
MOV    AL, 00H
OUT    DX, AL
LOOP2: IN     AL, DX
TEST   AL, 1EH                ; 查寻 EOC 信号
JZ     LOOP2
MOV    DX, 1CH                ; 读取数据
MOV    AL, DX
MOV    [SI], AL
INC    SI                    ; 存储单元加 1
INC    BH                    ; 循环寄存器加 1
LOOP   LOOP1                 ; 8 路转换没有结束则继续循环
HLT

```

13.4 多路模拟开关及采样保持电路

在计算机测量和控制系统中，很多时候需要对多路或多种参数进行采集和控制，为了节省硬件，共用同一个 A/D 或 D/A 转换器，实现多个回路的多参数的采集、转换和控制。能够解决多个回路和 A/D 或 D/A 转换器之间的切换问题的硬件设备称作多路模拟开关。

在有些应用场合, 由于 A/D 转换器完成一次转换过程需要一定的时间, 对于变化较快的模拟信号而言, 如果不采取一定的措施, 保证在 A/D 转换时间内, 使输入的模拟量维持不变, 就会引起转换误差, 影响精度。为了使输入到 A/D 转换器的模拟量在整个转换过程中保持不变, 而转换结束后, 又能跟随模拟量的变化而变化, 为此, 引入采样保持电路。

13.4.1 多路模拟开关

多路模拟开关的作用就是能够实现轮流切换各个被检测的物理量与 A/D 转换器(或者是 D/A 转换器与被控制对象)之间的通路。如图 13.17 所示, 是一个多回路分时共用 A/D、D/A 转换器的多路模拟开关框图。图中, 在模拟量输入端和模拟量输出端都分别用到了多路模拟开关, 但是两个多路模拟开关作用不同。在模拟量输入通道中实现的是“多选一”形式, 即多路模拟开关的输入是多路待转换的模拟量, 每次只能选其中的一路, 该通路经采样保持器, 送到 A/D 转换器进行转换后, 再通过 I/O 接口送入计算机进行处理。对于模拟量输出通道中实现的是“一到多”形式, 即 D/A 转换器输出的模拟控制信号, 经过多路模拟开关变换, 使之被送到多个控制对象的某一个上。

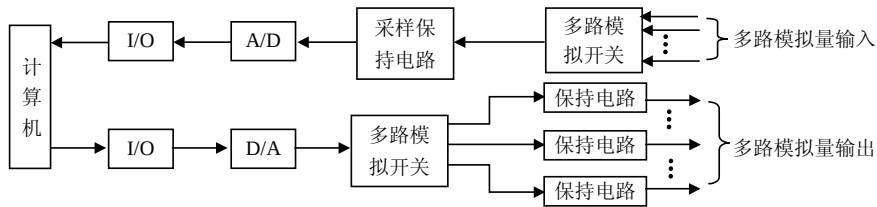


图 13.17 多回路分时共用 A/D、D/A 转换器框图

理想的多路模拟开关在断开时, 其开路阻抗无穷大, 接通时的导通电阻应为零。同时要求切换速度快、噪声小、寿命长、可靠性高。由于多路模拟开关用来切换模拟信号, 故它的性能直接影响输入模拟量和输出模拟量的精度和速度。使用时, 应根据实际需要和多路模拟开关的性能指标来进行选择。

随着大规模集成电路的发展, 厂家已推出各式各样的多路模拟开关, 从输入信号的连接方式来看, 有单端输入、双端输入或多路输入/多路输出; 从组成多路模拟开关的电路来看, 有 TTL、CMOS 和 HMOS 电路等。多路模拟开关芯片种类很多, 常见的有“八选一”多路模拟开关, 如 CD4051、AD7501 等; “十六选一”多路模拟开关, 如 CD4067、AD7506 等。使用时, 请查阅相关手册。目前, 已有不少厂家把多路模拟开关与 A/D 转换器做在同一芯片里, 如 ADC0809, 若采用这样的转换器, 不必再额外增加多路模拟开关就能实现多路模拟量的分时转换。

13.4.2 采样保持电路

采样保持电路可以实现的功能有: 保持采样信号不变, 以便完成 A/D 转换; 同时对几个模拟量进行采样, 以便进行数据处理和测量; 减少 D/A 转换器输出的不稳定性, 从而消除输出电压的峰值及缩短输出值的建立时间; 把一个 D/A 转换器的输出分配到几个输出点, 以保证输出的稳定性。

采样保持电路的基本组成如图 13.18 所示。采样保持电路一般由保持电容 C、输入输出缓冲器以及控制开关三部分组成。

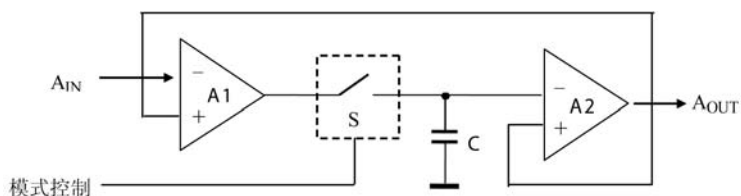


图 13.18 采样保持电路的基本组成

采样保持电路有两种工作模式，一种是采样模式，一种是保持模式。在采样模式期间，开关 S 闭合，输入缓冲器 A1 通过闭合的开关 S 给保持电容 C 快速充电，使采样保持电路的输出跟随模拟量输入电压变化。在开关 S 接通时，电容 C 充电时间越短越好，使其迅速达到输入电压。在保持模式期间，模式控制信号使开关 S 断开，输出缓冲器 A2 的输入阻抗高，电容将保持充电时的最高值，即保持命令发出时刻的模拟量输入值，直到保持命令撤销时为止。此时，采样保持电路重新跟踪输入信号的变化，直到下一个保持命令到来为止。

目前，大多数采样保持电路多集中在一个芯片里，但芯片内不含保持电容，用户可以根据需要进行选择并外接。电容的选择应综合考虑精度、采样频率、下降误差等参数。常见的采样保持电路芯片有：通用芯片，如 LF198、LF398、AD582K、AD583K 等；高速芯片，如 HTS-0025、THS-0060、HTC-0300 等；高分辨率芯片，如 SHA1144 等。使用时，请查阅相关手册。

13.5 小 结

本章主要介绍了 A/D、D/A 转换器的工作原理和典型芯片以及多路模拟开关和采样保持电路。DAC0832 是 CMOS 工艺制造的 8 位 D/A 转换器，由两级 8 位寄存器和一个 8 位 D/A 转换器组成。使用两级寄存器(输入寄存器和 DAC 寄存器)可以简化它的接口电路，使工作方式更加灵活。从输出信号来说，D/A 转换器的直接输出是电流，若片内有输出放大器，则能输出电压量，并能实现单极性或双极性电压输出。D/A 转换器的转换速度较快，一般其电流建立时间为 $1\mu\text{s}$ 。

A/D 转换器是模拟信号源与数字设备、数字计算机或其他数据系统之间联系的桥梁，它的任务是将连续变化的模拟信号转换为离散的数字信号以便于数字系统进行处理、存储、控制和显示。在工业控制系统和数据采集以及许多其他领域中，A/D 转换部件常常是不可缺少的重要部件。逐次逼近型是目前广泛应用的 A/D 转换器，其分辨率从 8 位到 16 位，转换时间从 $100\mu\text{s}$ 到几微秒，精度有不同等级，有的转换器内部还带有多路模拟开关。该类转换器具有较快的转换速度、与微机接口方便等主要优点，适用于对快速连续变化的物理量进行跟踪采集和记录等应用场合。ADC0809 是逐次逼近型 8 位 A/D 转换芯片，片内有 8 路模拟开关，可以同时连接 8 路模拟量，单极性，量程为 $0\sim 5\text{V}$ ，典型的转换速度为 $100\mu\text{s}$ ，片内有三态输出缓冲器，可直接与 CPU 总线

连接。该芯片有较高的性能价格比，适用于对精度和采样速度要求不高的场合或一般的工业控制领域。

在实际应用中，A/D、D/A 转换器的选择主要考虑转换精度和分辨率是否满足系统的需求；转换速率要根据被测对象的变化率及转换精度的要求来确定；为保证实时性和采样信号不失真的要求，转换时间必须满足由采样定理所确定的时间；要根据环境条件选择转换芯片的环境参数。在价格差别不大的情况下，最好选取既能保证性能，接口设计又简单的芯片。

在计算机测量和控制系统中，很多时候需要对多路或多种参数进行采集和控制，为了节省硬件，共用同一个 A/D 或 D/A 转换器，实现多个回路的多参数的采集、转换和控制，从而引入了多路模拟开关。对于变化较快的模拟信号而言，为了使输入到 A/D 转换器的模拟量在整个转换过程中保持不变，而转换结束后，又能跟随模拟量的变化而变化，则需引入采样保持电路。

13.6 习 题

1. 选择题

- (1) D/A 转换器的核心是()。
- A. 电阻网络 B. 输出锁存器 C. 多路模拟开关 D. 驱动放大电路
- (2) ADC 的转换精度主要取决于()。
- A. LSB B. 位数 C. 转换时间
- (3) 下面()不是 DAC 的主要性能指标。
- A. 分辨率 B. 转换精度 C. 建立时间 D. 制造工艺
- (4) 下面()芯片内部不具有采样保持电路。
- A. AD674 B. AD678 C. AD679 D. AD574

2. 简答题

- (1) 什么是 D/A 转换器？什么是 A/D 转换器？
- (2) 描述 D/A 转换器的性能指标主要有哪些？
- (3) DAC0832 转换器有哪些特点？其内部结构由哪几部分组成？
- (4) 简述 A/D 转换器的主要性能指标？
- (5) ADC0809 转换器有哪些特点？其内部结构由哪几部分组成？
- (6) 在实际应用中，选择和使用 D/A 转换器应注意哪几点？
- (7) 什么是多路模拟开关？采样保持电路能实现哪些功能？
- (8) 常见的采样保持电路芯片有哪些？
- (9) 常见的多路模拟开关芯片有哪些？

3. 计算题

- (1) 若 ADC 输入模拟电压信号的最高频率为 100kHz，采样频率的下限是多少？完成一次 A/D 转换时间的上限是多少？

(2) 如果一个 8 位 D/A 转换器的满量程(对应于数字量 255)为 10V, 分别确定模拟量为 2.0V 和 8.0V 所对应的数字量是多少?

(3) 某 12 位 D/A 转换器, 输出电压为 0~2.5V。当输入的数字量为 400H 时, 对应的输出电压是多少?

4. 设计题

假设有一个工作在单缓冲方式下的 DAC0832, 其输出接运算放大器后能够输出电压信号, 端口地址为 0A8CH。

(1) 写出产生矩形波信号的程序段。

(2) 写出产生三角波信号的程序段。

第 14 章 微型计算机应用系统的设计

教学提示：微机应用系统的设计，是利用学习与掌握的微机基本硬件与软件知识，将微机运用到监测监控、数据采集与处理和各类管理过程中。由于应用场合、形式和规模的不同，应用系统的构成、功能的强弱、外部接口与设备配备的要求也不同。从总体上来说，要设计和开发一个实用的微机应用系统，所面临的技术问题和设计步骤大致是相同的。

教学要求：了解应用系统的一般类型、基本要求和原则、分析设计过程和开发的一般方法，了解和掌握微机应用系统设计与开发的相关知识。

14.1 微型计算机应用系统概述

微型计算机问世以来，仅仅只有几十年的时间，但是发展的速度却是惊人的，它以其独有的优越性，体积小、重量轻、功耗省、价格低、灵活可靠和使用方便等特点，已经渗透到几乎所有的领域，对社会产生巨大的影响。微型计算机与小型计算机比较，微型计算机的功能已接近并在很多方面超过了小型机，特别是价格比小型机明显便宜，正是功能强、价格低这两大优势，成为微型机强盛生命力的基础，使它广泛用于各个领域，并且成了各行各业的有力助手。

14.1.1 微型计算机应用系统的一般类型

微机应用系统一般可分为以下三种类型。

1. 检测控制型

检测控制型微机应用系统用于对控制对象的状态作某种判断，然后再根据判断的结果去控制相应的执行机构。一般不需要做复杂的数学计算工作，但它必须有检测控制对象的某些现象的传感器。如果从传感器获得的信号是模拟量(连续的电压或电流值)，则还需将其转换为数字量以供计算机处理。对应的器件为 A/D 转换器。

作为一个计算机控制系统，如果把计算机比作人的“头脑”的话，则传感器就相当于“耳目”，执行机构则是行动的“手脚”，而 ADC 则起到“耳目”和“头脑”之间的匹配作用，或者说起到将模拟信号翻译成数字信号的作用，以适应计算机的工作需要。执行机构如果要求开关驱动，则计算机只要输出开关量(如继电器)即可；而如果执行机构是由模拟量驱动的，则计算机要将数字量再经过 DAC 转换才能产生用于控制的模拟量(如电压或电流值)。

2. 数据处理型

数据处理型计算机应用系统是指输入数据量比较大，数据输入后需要按照一定规则进行分类、排序(列表)、折算(如线性化)、换算(如求均值、方差等)等，然后送入有关的数学模型进行繁杂运算的计算机系统。这类系统为了便于人机对话，多数配有显示器和键盘，某些情

况下还可增设其他终端设备。这类系统多为键盘输入数据，然后在显示器上显示结果，这就是所谓的离线处理。如果系统中数据输入端随时输入数据，计算机连续运算，并随时或定时给出计算结果，该系统则为在线处理。当然在线处理系统要比离线处理系统复杂得多。

3. 混合型

混合型就是以上二者的复合型式。混合型计算机应用系统一方面既有很多数据输入，并进行相应的处理，另一方面又要根据处理的结果去控制相应的对象，以达到整个系统的预期目的。这类计算机广泛用于工业现场的实时控制，是微机应用系统设计的主流。

14.1.2 微型计算机应用系统的设计原则

在不同规模和要求系统中微机应用系统设计的基本原则大体是相同的。因此，这些共同的原则在设计前或设计过程中都必须予以很好的考虑。

1. 操作性能要好

微机应用系统的操作性能好，就是指系统的人机界面要友好，操作起来简单、方便，并且便于维护。为此，在设计整个系统的硬件和软件时，应处处为用户想到这一点。在设计系统软件时，就应该考虑配备什么样的软件和环境能降低操作人员对这些专业知识的要求。前面已经提到，系统中的某些用户程序是要由用户自己编写的。例如，在数控设备中，加工切削程序往往要由操作人员来编写或进行修改。如果这样的用户程序是用汇编语言编写的，那么对操作人员来说，就必须熟悉相应微机的汇编指令和程序设计的基本知识，否则是难以胜任的，这种软件设计方案必然会限制微机应用系统的推广和应用。

事实上，如果在系统上配上高级语言，特别是配上像工业控制中常用的数控语言，那么加工程序的编制就非常容易，一般操作人员就能很快掌握。这样就有利于微机应用系统的推广和应用。对硬件方面的要求也一样，例如，系统的控制开关不能太多、太复杂，操作顺序要尽量简单明了。另外，尽管微机应用系统的可靠性较高，但是不能理想地认为它不会发生故障。一旦出现故障，如何能尽快地排除，这也是系统设计时要考虑的问题。从软件角度来说，系统应配置自检或诊断程序，以便在故障发生时，能用程序来查找故障发生的部位，以缩短排除故障的时间；从硬件角度来说，零部件的配置应便于操作人员的检修。

当然，还有一些其他要求。例如，控制台要便于操作人员工作，显示器的颜色要和谐等，凡是涉及人机对话的问题都应逐一考虑。

2. 通用性好，便于扩展

通常一个微机应用系统在工作时都能同时控制几台设备。但是，在大多数情况下，各个设备的控制要求往往是有差别的。另外，所控制的设备也不是一成不变的，而是需要不断地进行更新。这样，就要求系统不仅能适应各种不同设备的要求，而且也要考虑在设备更新时，整个系统不需要做大的改动就能马上适应新的配置。因此，系统就需要有好的通用性，而且在必要时能灵活地进行扩展。微机应用系统要达到这样的要求，就必须尽可能地采用标准化、模块化设计。例如，尽可能采用通用的系统总线结构，如采用 STD 总线、AT 总线、MULTIBUS 总线等。在需要扩充时，只要增加一些相应的接口卡就能实现对所扩充的设备进行控制。另外，接口部件应尽量采用标准的通用大规模集成电路芯片。在考

考虑软件时，只要速度允许，就尽可能把接口硬件部分的操作功能用软件来替代，这样在改变被控设备时，就无需变动或较少变动硬件，只需要更改软件就行了。

系统的各项设计指标留有一定的裕量，也是可扩充的首要条件。例如，微机的工作速度如果在设计时不留有一定裕量，那么要想再进行系统扩充是完全不可能的。其他指标，如电源功率、内存容量、输入/输出通道、中断等，也应留有一定的余量。

3. 可靠性高

对任何微机应用系统来说，尽管各种各样的要求很多，但可靠性是最突出、最重要的一个基本要求。因为，一个系统能否长期安全可靠地正常工作，对一个工厂来说将要影响到整个装置、整个车间，乃至整个工厂的正常生产。一旦发生故障，就会造成整个生产过程的全面混乱甚至瘫痪，从而引起严重的后果，所以对可靠性有很高的要求。特别是作为控制核心的微机，其可靠性要求则更高。

在小型计算机时代，由于硬件价格较高，因此一个应用系统一般只配备一台计算机。这样，一旦计算机出现故障，整个系统就将瘫痪。随着价格低廉的微机的出现，人们通常采用多微机系统形式来构成一个应用系统，就是将应用系统分成若干个功能模块，每个功能模块由一个微机来进行控制，并将各个微机用网络连接起来，构成一个多机控制系统。这样，功能分散了，危险也相应地分散了。即使工作过程中某一台微机发生了故障，也不至于使整个系统瘫痪。当前常见的形式有以下两种。

1) 采用双机系统

在这种系统中，由两台微机作为系统的核心控制器。由于两台微机同时发生故障的概率很小，从而大大提高了系统的可靠性。双机系统中两台微机的工作方式，常见的有以下几种。

(1) 备份机工作方式。在这种方式中，一台微机投入系统运行，另一台虽然也同样处于运行状态，但是它是脱离系统的，只是作为系统的一台备份机。当投入系统运行的那台微机出现故障时，通过专门的程序和切换装置，自动地把备份机切入系统，以保持系统正常运行。被替换下来的微机经修复后，就变成系统的备份机，这样可使系统不因主机故障而影响系统的正常工作。

(2) 主-从工作方式。这种方式是两台微机同时投入系统运行。在正常情况下，这两台微机分别执行不同的任务。如一台微机可以承担系统的主要控制工作，而另一台可以执行诸如数据处理等一般性的工作。当其中一台发生故障时，故障机能自动地脱离系统，另一台微机就自动地承担起系统的所有任务，以保证系统的正常工作。

2) 采用多微机集散控制

这种系统结构是目前提高系统可靠性的一个重要发展趋势。在过去，如上面已提到的那样，计算机控制主要倾向于采用集中控制的方案，即根据系统的控制任务和要求，选择一台适当功能的计算机来承担系统的全部任务。这样做，一旦主机发生故障就会影响整个系统。随着机的出现，特别是它的价格不断下降，已有可能把系统的所有控制任务分散后由多台微机来承担。

有关微机应用系统可靠性的问题是一个十分重要而又复杂的课题。可靠性设计应包括硬件的、软件的、电源的、环境的及电磁兼容性的设计等，读者可参考相关资料作进一步的了解。

4. 设计周期短、价格便宜

在研制微机应用系统时,应考虑的一个重要因素是设计周期短、价格便宜。目前科学技术发展十分迅速,各种新的技术和产品不断出现,这就要求所设计的微机应用系统能跟上形势的发展。如果研制周期太长,会使产品失去竞争能力和实用价值。所以,微机应用系统不能搞大而全,应考虑实际情况来确定系统的设计规模。这样既可以缩短设计周期又可以降低系统的设计费用。在进行系统设计时,要仔细核算以降低整个系统的成本,在保证功能和性能的前提下,低的价格才有竞争力。在设计时可大量采用标准化产品(如 STD 总线、AT 总线的 A/D 转换卡等),然后进行二次开发,以缩短开发周期。

5. 对环境的适应性要好

在开发微机应用系统时,一定要考虑到其应用环境,保证在现场的环境下可靠地工作。例如,有的地方电源电压波动很大,有的地方环境温度变化剧烈,有的地方湿度很大,有的地方振动很厉害,而有的工作环境有粉尘、盐雾、腐蚀等。这些外界因素在系统设计中必须考虑,采用必要的措施保证微机应用系统安全可靠地工作。

14.1.3 微型计算机应用系统的设计过程和内容

微机应用系统的开发过程一般包括系统的总体设计、硬件设计、软件设计、系统调试等几个阶段。这几个开发阶段并非完全独立、各自进行的。在总体设计进行硬件和软件功能划分后,可以把硬件设计和软件设计分开同时进行,但要根据开发过程的需要不断协调一致,有时甚至需交叉进行。

硬件设计完成后将制造出样机。样机可以做在试验板上,也可以加工成 PCB 板的形式。由于微机应用系统一般都较为复杂,总线数量庞大,因此单元电路可以采用试验板的形式焊接和调试,而整个微机系统样机的主板一般都采用 PCB 板上加装集成电路插座的方式构成。当硬件电路完成后,可进行模块调试和整体联调,最后进行性能评价,即检查其功能和性能是否满足系统设计任务的需要。这时,可能又需要回过头来去修改整个设计,重新进行硬件和软件功能的划分。对于只有少数指标不够满意时,原则上尽量少做硬件上的变动,而用修改软件的方法改善其性能。

1. 总体设计

总体设计是指通过调研和分析,确定应用系统的功能、性能、使用的主要器件以及软硬件分工,并制定设计任务书。设计任务书包括:程序功能、技术指标、精度等级、实施方案、工程进度、所需设备、研制费用和人员分工等。总体设计是微机应用系统设计中十分重要的环节,它决定着应用系统的性能指标、生产成本、开发效率和生命周期。合理的总体设计在于对系统要求的全面分析和在此基础上对系统功能与实现方法的综合考虑。

1) 确定系统功能技术指标

设计一个应用系统的基础是完备的系统分析。大多数微机应用系统应用于测控领域,这一类系统要考虑的主要问题是开关量、模拟量的输入输出和过程控制算法。系统分析的目的是确定系统究竟要完成什么任务,并对控制对象及其控制要求进行文档化,定义应用系统所要达到的功能技术指标,包括:

(1) 系统需要检测的信号,对测试点、检测精度、检测元件及检测方法的要求;

- (2) 控制算法的精度和实时性要求, 可能出现的突发和平均数据率大小;
- (3) 系统输出信号的格式和电平, 使用的驱动机构和执行机构;
- (4) 系统的操作方式, 配备的外围设备, 对显示、打印、通信等的要求;
- (5) 系统的工作环境及可靠性等级, 与系统配合工作的其他电气设备的动态运行情况。

总之, 在开始设计前, 必须进行周密的调查研究, 明确工业现场的硬件资源和工作环境、应用系统要求实现的各项功能技术指标, 必要时可以进行现场勘察和小型试验, 对产品的实时性、可靠性、通用性、可维护性、先进性以及成本等进行综合考虑, 参考国内外同类产品的相关资料, 使确定的技术指标更加合理可行有竞争力。

2) 微处理器的选择

可供选择的微处理器很多, 须根据应用系统的设计目标、复杂程度、可靠性、精度和进度要求来选择性能价格比相对合理的微处理器。主要考虑以下几个方面:

(1) 市场货源。通常设计者只能从市场上可提供的微处理器中选择, 特别是将作为产品生产的系统, 微处理器需要使用性价比高的成熟产品, 所选机型必须有稳定、充足的货源。

(2) 机型性能。微处理器的性能包括位数、指令系统、片内资源、扩展能力、运算速度和可靠性等方面。要根据应用系统的实际需要, 选择最容易实现产品技术指标的微处理器, 并留有一定的扩展空间, 以便于维护和升级。

微处理器的选择一定要以够用为原则, 避免盲目追求高频 CPU, 因为随着频率的提高, 与之相关的配件(如 RAM)也要工作于较高的频率, 从而使系统发热量大幅增加, 可靠性下降, 整体制造成本提高。

(3) 研制周期。在研制任务重、时间紧的情况下, 要选择最熟悉的机型和器件, 也可直接采用商品化的配件(如研华工控系列板卡), 进行系统集成和二次开发。这种方法对于小批量的工程项目(非商品化的产品)特别有用。

当研制时间比较充裕、产品批量大的情况下, 则需要把应用系统的性能价格比核算作为选择处理器的重要依据。

3) 主要器件选择

除了微处理器外, 系统中还有传感器、模拟电路、输入/输出电路等部件和设备。这些部件在器件上应符合应用系统各方面要求, 在实现技术指标的同时, 再考虑性价比等因素。

在总体设计过程中, 不需要进行单元电路的设计。但可以对应用系统需要使用的主要器件进行查阅和建议。比如, 传感器往往是制约系统性能指标的瓶颈, 应根据工作环境和精度要求选择合适的传感器; A/D 转换器有多种形式, 应根据模拟信号的变化速度选用合适的 A/D 转换器, 如对缓变的温度输入宜采用慢速高精度的双积分型 A/D 转换器, 对频率范围较大的声音输入信号宜采用高速的逐次比较型 A/D 转换器; 如对视频信号进行采集和处理系统则应采用视频放大器 and 高速视频 A/D 转换器等。

4) 硬件和软件功能划分

系统的硬件配置和软件设计应紧密联系、相互结合。硬件和软件的划分确定哪些功能由硬件电路来实现, 哪些功能由软件来实现。这种划分和具体产品的要求有很大关系。

在某些场合, 硬件和软件具有一定的互换性。多用硬件可以提高工作速度, 减少软件开发的工作量, 但增加了硬件成本; 反之, 若用软件代替某些硬件的功能, 可以节省硬件开支, 增加了灵活性, 有利于系统升级, 但也增加了软件的复杂性。由于软件是一次性投

资,因此在研制产品批量比较大的情况下,能够用软件实现的功能尽量由软件来完成,以便简化硬件结构,降低生产成本,提高系统的可靠性。

总体设计时,需权衡利弊,划分好软硬件的功能,并按照各部分任务列出其清单。再根据总体方案对软、硬件任务的划分,画出由硬件框图和软件功能图组成的系统结构框图。

研制大型工业测控系统往往需要多方协作和联合攻关,因此,总体方案中应当大致规定接口电路地址、数据通信协议、监控程序结构、用户程序要求、系统软件的内存驻留区域以及采样信号的缓冲区域等公用数据格式。

5) 总体设计可行性验证

对于较复杂的微机应用系统或必须一次成功的系统,如巡航导弹伺服系统,在进行软硬件独立设计前需进行系统可行性验证,一般可通过人工论证、标准测试和软件仿真等三种技术途径检验设计的可行性。

2. 硬件设计

1) 单元电路设计

硬件设计的任务是在所选微处理器的基础上,根据总体设计要求,确定系统扩展所用的 I/O 接口、A/D 转换器、D/A 转换器及其外围电路,逐一设计调试电路,画出相应的电路原理图,最后整合成完整的硬件系统。硬件设计中应采用模块化设计,多数模块已在《数字电路》、《模拟电路》和本课程的前述章节中进行了介绍,在 14.2 节中将对常用的硬件模块的选用进行概括说明。

2) 综合设计

微机应用系统的各单元电路硬件设计完成后,对不是很确定的电路形式和电路参数一般都要通过软件仿真、搭接和测试单元电路等方式整定电路参数,补充形成完整的单元电路原理图。然后按照总体设计中的系统硬件结构,进行单元电路的硬件整合,形成一套硬件系统原理总图。在进行硬件整合设计时,系统的可靠性是一项最重要最基本的技术指标。可靠性通常是指在规定条件下、规定时间内完成规定功能的能力。规定的条件包括环境条件(如温度、湿度、振动等)、供电条件等;规定的时间一般指平均故障时间、平均无故障时间、连续正常运转时间等;规定的功能随微机应用系统的不同而不同。

在实际工作中,微机应用系统可能会受到各种外部和内部干扰,使系统工作产生错误或故障。为了减少这种错误和故障,就要采取各种提高可靠性的措施。因此在硬件整合中应认真考虑:整合后形成的系统总线负载大小和各部分负载分配;相互连接的各单元电路间信号电平是否兼容;系统中相互隔离部分的电路是否采用各自独立的电源和地;应尽量减少系统中电源电压的种类,合理安排不同类型电源和地线系统;双绞线抗共模干扰的能力较强,模块间需要进行长线连接的场合,是否选用了双绞线作为接口的信号线。

3) 安装和调试

完成用户系统样机的硬件设计和组装以后,便进入硬件安装调试阶段。其实,硬件调试和软件调试是分不开的,许多硬件故障是调试软件时才能发现的。通常是先排除系统中明显的硬件故障后才和软件结合起来调试。

常见的硬件故障有逻辑错误(包括错线、开路、短路、相位错等)、元器件失效(器件本身已损坏或性能不符合要求;由于组装错误造成的元器件失效,如电解电容、二极管的极性错误,集成块安装方向错误等)、可靠性差(如金属孔、接插件接触不良会造成系统时好

时坏, 经不起振动; 内部和外部的干扰、电源纹波系数过大、器件负载过大等造成逻辑电平不稳定; 走线和布局的不合理等)、电源故障(电压值不符合设计要求、电源引出线和插座不对应、电源功率不足和负载能力差)等。

硬件调试分为脱机调试和联机调试两个步骤。

(1) 脱机调试。脱机调试是在样机加电之前, 先用万用表等工具, 根据硬件电路原理图和装配图仔细检查样机线路的正确性, 并核对元器件的型号、规格和安装是否符合要求。应特别注意电源的走线, 防止电源之间的短路和极性错误, 并重点检查总线间是否存在相互间的短路或与其他信号线的短路。对于样机所用电源事先必须单独调试, 调试好后, 检查电压值、负载能力、极性等是否均符合要求, 才能加到系统的各个部件上。在不插芯片的情况下, 加电检查各插件上引线端的电位, 仔细测量各点电位是否正常, 尤其应注意 CPU 插座上的各点电位是否正常, 若有高压, 联机时会损坏系统。

(2) 联机调试。通过脱机调试可排除一些明显的硬件故障。但有些隐性硬件故障还是要通过联机调试才能发现和排除。

若开发的是带有 CPU 的小系统, 则应使用仿真器。在联机前先断电, 把开发系统的仿真插头插到样机上, 检查一下开发机与样机之间的电源、接地是否良好。一切正常, 即可打开电源。通电后执行开发系统的读/写命令, 对用户样机的存储器、I/O 端口进行读/写操作和逻辑检查。若有故障, 可用示波器观察有关波形(如选中的译码器输出波形和读/写控制信号、地址线、数据线的波形以及有关控制电平)。通过对波形的观察分析寻找故障原因, 并进一步排除故障。可能的故障有线路连接上有逻辑错误, 有断路或短路现象, 还有集成电路失效等。

若开发的是总线接口板, 则可将其插入到总线槽中, 通过 CPU 板对其进行调试。插入前应设置好接口地址、中断号和 DMA 通道号, 以免与 CPU 板的相关编号重叠。对于各接口板(如 A/D、D/A、数字 I/O、通信等)也可通过上述方法进行调试, 在调试过程中如发现用户系统工作不稳定, 一定要查出原因, 进行排除。

3. 软件设计

微机应用系统的软件设计是系统设计中最基本且工作量最大的任务, 一个较为复杂的应用系统软件, 如嵌入式操作系统, 其设计往往需要很多人进行多年的工作才能完成。事实上软件设计可包括接口控制程序设计和整体应用程序设计两大部分。前者一般采用汇编语言进行设计, 其目标是实现对各接口硬件电路的控制, 一般由硬件设计员以子程序、软中断、驱动程序等形式给出; 后者一般采用高级语言进行设计, 可交给软件设计员完成, 一般包括各种算法、人机对话、数据采集、输出控制等, 其中数据采集和输出控制是通过调用接口控制程序实现的。以下介绍的软件设计, 主要是指在硬件基础上的接口程序的设计, 其他程序的设计可参阅软件设计书籍。

接口程序的设计应根据硬件电路的结构和控制时序进行设计, 这些接口程序只负责某一独立模块的控制功能。如针对 A/D 转换电路, 编写一个采样一次模拟量的控制程序。其目的有两个, 一是检测硬件模块是否合理, 二是作为一个基本 I/O 程序供其他程序调用。对于有多个模块组成的微机系统, I/O 程序的编写和调试应逐个进行, 必要时可使某些暂时不用的模块脱开。所有模块均单独调试完毕后, 可进行整体联调。此时, 各 I/O 控制程序可供其他程序调用, 一般来说, I/O 程序是根据严格的控制时序来实现对硬件的控制的, 可使用汇编语言或 C 语言加以实现。

4. 软、硬件联调

在硬件系统和软件系统分别进行设计并调试的基础上,将硬件和软件放到同一系统中进行联调,又称集成测试。在联调中往往是逐步进行的,逐个硬件模块和软件模块进入系统,使它们进入正常工作。如果某一模块有问题,则可集中注意力加以解决,直至整个系统通过测试。

5. 实验室模拟运行

将整个联调好的系统在实验室中模拟现场的运行,此步骤称为离线仿真。这时,可由人工输入模拟信号(电压),用仪表(如万用表、示波器等)对输出进行指示,使系统连续运行。

在实验室模拟运行过程中,设计人员必须仔细观察运行过程中的各种状态,对任何不正常情况必须仔细分析其原因。必要时,可人为地制造一些干扰,以便观察系统的可靠性,亦可将电源拉偏,观察系统的适应能力等。

6. 现场调试、试运行

将所研制的系统放到用户现场,接上用户的常规及专用外设,对专用外设进行逐一调试,使它们进入正常状态。然后,执行用户程序,由用户使用,完成用户提出的功能,使系统进入试运行状态。在试运行过程中,开发者与使用者需要密切配合,仔细观察并记录系统运行的状态。如发现问题,要认真分析,务求尽快解决。在试运行过程中,系统的设计开发人员要认真编写大量的文件、资料。例如,研制项目的背景、研制报告、技术报告、使用维护手册、软件资料、硬件图纸、标准化规范、用户使用报告等。

7. 验收和鉴定

在用户使用半年或更长时间之后,若用户和设计者均对系统的性能感到满意,即可组织验收或鉴定,使设计工作完结。否则,依据需要修改设计,重新开发。

14.2 常用模块电路

微机系统的设计一般采用模块化设计,这些模块在《模拟电路》、《数字电路》和本课程的前述章节中进行了介绍,本节对各硬件模块的选用进行概括说明。

14.2.1 地址译码和总线驱动

处理器一般都提供大量的地址数据总线,用来访问充足的存储器空间和端口寄存器。如 8088/8086 微处理器有 20 位地址线,全译码方式可访问 1MB 存储器和 64K 的 I/O 地址,但应用系统有时不需要这么大的空间,为简化硬件设计逻辑,通常采用部分译码法和线选法相结合的办法。在进行标准总线(如 ISA、STD 等)系统设计时,各板卡的地址应选用 CPU 板空闲的端口地址,并可通过跳线或拨盘更改地址,以免冲突。常用的地址译码器可采用 74LS138、74LS139、74LS42 译码器和 74LS688 比较器等。

一般来说,微处理器外部总线的带负载能力是有限的,若扩展的电路负载超过了总线

负载能力，系统就无法可靠工作。这时总线上必须加驱动器，以提高其负载能力。总线驱动器不仅能提高端口的总线驱动能力，而且可提高系统的抗干扰能力，改善信号的边沿特征。常用的驱动器主要有：双向 8 位三态缓冲器 74LS245，一般用于驱动数据总线；单向 8 位三态缓冲器 74LS244，一般用于驱动地址总线。

14.2.2 存储器

对于功能相对简单的微机系统，可选用单片机外配小容量的 ROM(EPROM 或 EEPROM)和 SRAM 实现。对大多数 ISA、STD 总线的工业控制计算机系统，可选配工业级的 RAM(SRAM、DDR 等)，这些标准器件可根据需要进行扩充，但在工业系统中，应尽量避免使用硬盘、光驱等具有机械装置的外存储器，以防止恶劣的工业环境对其有影响，降低系统的可靠性。可使用大容量的 FLASH 盘进行替代，或将大量的现场数据传输给上位机而进行处理和保存。

14.2.3 数字量输入/输出接口

微机应用系统需要输入/输出的物理量主要有三种形式：

(1) 开关量：指的是 0/1 信号，代表了按钮开关、行程开关、继电器中触点的接通和断开等。

(2) 数字量：指的是幅度上离散的物理量，由多位开关量组成。输入数字量由键盘、拨码盘、数字式传感器或脉冲计数器产生，输出数字量送到 LED、LCD 显示器、打印机等。

(3) 模拟量：指的是时间和幅度上连续变化的物理量。模拟量可能是电信号，如电压、电流等，也可能是非电信号，如作为被测对象的压力、拉力、温度，作为控制对象的电机转速，加热炉温度等。

微机应用系统一般都需要扩展并行 I/O 接口，以进行数字量和开关量的输入/输出。输入/输出接口扩展的原则是：输入要缓冲(隔离)，输出要锁存。在选择 I/O 接口电路时，应从体积、价格、功能、负载和可靠性等几方面进行综合考虑，对常用 I/O 接口芯片的特性进行分析，根据系统总的输入、输出要求来决定 I/O 接口方式。

标准可编程接口芯片 8255 和 8155 的功能全面、接口简单、使用方便，对总线负载小，因而在测试控制系统中应用广泛，但缺点是不能直接驱动电流较大的外围器件、初始化比较复杂，受到干扰后可能处于非正常状态。因此在一些接口线要求较少或接口线的功能单一的应用系统中，可用通用逻辑器件实现，则显得比较灵活、接口线利用率高、成本较低、负载能力强、可直接驱动发光二极管等外围器件。但这类接口对总线负载大，接口比较复杂，不宜大量扩展。

常用的逻辑器件是：输入缓冲可采用 74LS244、74LS240、74LS245 等，输出锁存可采用 74LS373 或 74LS273 等。

外部数字信号的电平与微机系统内部电平标准可能不匹配，如一些继电器的控制电压需要 9V 或 12V，甚至更高。这些系统中，在数字输出接口的后级或数字输入接口的前级必须加电平转换模块，以满足微机系统的要求。

常见的电平转换模块有：CMOS 逻辑电路，如 CD4514 芯片；集电极开路输出(OC 门)，如 74LS07；光电耦合器件等。

14.2.4 模拟量输入/输出

工业测控要求微机应用系统不但能够接受和输出数字信号,而且能够采集工业现场的模拟信号,经过数据处理后,对工业生产过程进行反馈控制。当组成一个微机测控系统时,需根据测控对象的要求,设计相关的外围电路,构成模拟量输入/输出通道。一般来说,模拟输入/输出通道的核心是 A/D、D/A 转换器,其选择应根据系统的要求(如转换速率、转换精度、信号频率、接口形式、成本等)进行选择。此外,模拟通道还应包括信号调理、多路选择开关、采样保持、信号隔离、滤波等辅助电路。

常用的 A/D 转换器有 ADC0809、AD574、5G14433 等, D/A 转换器有 DAC0832、DAC1208 等。随着技术的发展,出现了许多高速、串行(SPI、I²C 接口)的 A/D、D/A 转换器,它们具有体积小、接口简单、功耗低、精度高等优点,在微机系统中得到了广泛应用。

14.2.5 信号隔离

对于医疗仪器等一些特殊应用系统,检测探头(包括传感器和放大电路)和治疗装置必须安全供电,应采用纽扣电池等。而整个仪器由于功耗很大,一般采用交流供电,两个部分之间要进行可靠的电源隔离和信号隔离。这不但可以有效防止工频干扰,最重要的是保证系统使用的安全性。

对于检测或控制对象是强电设备(如大功率电机的控制、强电的电量检测等),一般也需要将其与微机系统隔离,以保证微机系统的可靠性,防止各类干扰。

1. 数字信号的隔离

数字信号的隔离可采用光电耦合器,使用时必须将两边的电源和地线完全隔开,如图 14.1 所示。采用光电隔离时,两边的数字信号的传输是通过光传输的,没有电路上的连接,可有效阻止各类干扰,同时可实现电平转换。应当指出,信号从发光管到接收管有比较大的延迟时间,同时信号边沿特征可能变差,因此对于高频信号应选用高速光耦,同时可增加施密特整形电路(如 74LS14),以改善信号的边沿特征。

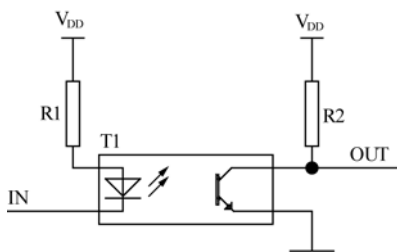


图 14.1 光电耦合器的应用

2. 模拟信号的隔离

对于中低频交流信号可采用变压器隔离。一般情况下采用变压器隔离,由于漏磁原因,精度不是很高,当频率提高时会产生较大的失真,一般应用在低频和精度要求不高的场合。

模拟量的光电隔离须采用以线性光耦为核心的隔离放大器,它具有线性度好、稳定性高、频率范围相对较宽,但成本较高,只能应用在高端设备中。

在低端设备中,可在 A/D 或 D/A 转换器与总线之间采用数字信号的隔离方法将其分开,而 A/D 或 D/A 芯片与模拟信号之间不采用隔离,其特点是成本低,可实现高速信号的处理,缺点是 A/D 或 D/A 芯片暴露在强电干扰之中,有可能使其工作不可靠。对于使用串行接口的 A/D、D/A 转换器使用该方法是一种不错的选择。另一种方法是利用双封装的两个普通光耦(如 T521-2)的非线性特征的相似性,设计电路使其非线性特征互相抵消,从而实现模拟量的隔离放大,这种方法可实现模拟量的隔离,成本较低,但精度不高,批量使用时离散性较大。

14.2.6 接口电路器件的选择

1. TTL 器件与 CMOS 器件

常见的 74LS 系列芯片是 TTL 器件,而 CD4000 系列则是 CMOS 器件,它们的内部结构不同,但逻辑功能是相似的,应根据各自的特点进行选用。其特点如下:

(1) 由于 CMOS 管的导通电阻比双极型三极管的导通电阻大,所以 CMOS 电路的工作速度比 TTL 电路低。一般情况下工作电压越高,CMOS 电路的工作频率越高。

(2) CMOS 电路的输入阻抗很高,在频率不高的情况下,电路的扇出能力较强,即带负载的能力比 TTL 电路强。

(3) CMOS 电路的电源电压允许范围较大,约在 3~18V,使电路的输出高、低电平的摆幅大,因此电路的抗干扰能力比 TTL 电路强。

(4) 由于 CMOS 电路工作时总是一管导通,另一管截止,而截止管的电阻很高,这就使在任何时候流过电路的电流都很小,因此 CMOS 电路的功耗比 TTL 电路小得多。门电路的功耗只有几个 μW ,中规模集成电路的功耗也不会超过 $100\mu\text{W}$ 。

(5) 因 CMOS 集成电路的功耗很小,使内部发热量小,因此 CMOS 集成电路的集成度比 TTL 电路高,可广泛使用在各类手提设备中。

(6) CMOS 集成电路的温度稳定性好,抗辐射能力强,因此 CMOS 电路适合于在特殊环境下工作。

(7) 由于 CMOS 电路的输入阻抗高,使其容易受静电感应而击穿,因此在使用和存放时,应注意静电屏蔽,焊接时电烙铁应接地良好,尤其是 CMOS 电路多余不用的输入端不能悬空,应根据需要接地或接高电平。

(8) 系统中如使用了 TTL 和 CMOS 器件,则在它们之间应注意电平匹配,必要时应加接电平转换模块。

2. 可编程逻辑器件的应用

目前,随着大规模可编程逻辑器件的大量出现,各种接口电路几乎都可由该器件实现,甚至整个微机应用系统都可以集成到一个大规模可编程逻辑器件中。使用时,一般需用硬件设计语言(如 VHDL、Verilog HDL 等)将设计要求描述给计算机,由计算机分析、综合、下载到可编程逻辑器件中,从而在芯片内部构成所需要的硬件逻辑电路。其优点是可实现硬件电路重构,不需修改线路板,即可实现硬件升级;可将一个复杂的系统用一片或几片实现,提高了系统的可靠性,降低了系统成本,缩短了开发周期。大规模可编程逻辑器件的广泛应用是电子技术发展的一个方向。

14.3 微机应用系统设计举例

大量的微机应用系统是以总线的方式设计的,而不以 CPU 的方式设计的。设计时,按不同功能设计相关的板卡,如 CPU 板、A/D 板、D/A 板、I/O 板、RAM 扩展板等。这些板卡均是基于相同的总线(如 STD、ISA、PCI 等)设计的,应用时只需将这些板卡插入总线槽中,即可构成应用系统。其优点是:首先,由于总线是标准化的,设计的接口板可应用于所有相同总线的微机系统,而不必局限于某种 CPU 的应用系统,便于大量生产各种通用的接口板卡,实现商品化,从而降低成本;其次,可通过更换相关的板卡,升级硬件系统,甚至更改 CPU;再次,模块化结构,方便了系统调试、提高了可靠性。

事实上,对于小批量使用的工程项目,可直接购买相关的接口板卡,然后进行二次开发,就可完成,既降低了成本,又缩短了系统的开发周期。对需大批量生产的产品,则可自行开发相关的接口板,有利于保护自主技术和进一步降低成本。下面介绍两个基于 STD 总线的接口板的设计。

14.3.1 STD 总线介绍

1. STD 总线简介

STD(Standard)总线是在 1978 年推出的用于工业控制微型计算机的标准系统总线。自从它问世以来,以其优越的性能和强大的生命力在工业控制领域中受到广泛的欢迎并得到迅速的发展,被定名为 IEEE961。STD 总线不仅是国外流行的工业控制机标准总线,而且也是我国工业控制机的标准总线。

STD 总线开始推出时,是针对当时的 8 位微型计算机的,随着科学技术的发展和计算机应用的需要,STD 总线经过修订和改进,利用复用技术,在原来定义的 56 个总线信号之下实现了 STD 总线支持 20 位地址,寻址 1MB 的直接寻址能力。在保证实现有 I/O 插件板兼容的条件下,提供全 16 位数据的传输能力。目前,国内所采用的大多是这种 8 位、16 位兼容的 STD 总线。20 世纪 80 年代末,STD 总线已由 56 个信号发展到 114 个、136 个信号。1989 年美国的 EAITECH 公司开发出了 32 位的 STD32,为高档的 STD 微型计算机系统的发展提供了有利的条件。

2. STD 总线的特点

(1) STD 总线具有较好的兼容性,可以向上或向下兼容。例如,8 位的 STD 产品可以与新标准的 16 位或 32 位 STD 产品一起工作。

(2) STD 总线的电路板采用小板结构,高度模块化,开发周期短,并且可以根据用户要求组成各种规模的微型计算机应用系统。

(3) STD 总线采用一整套高可靠性措施,使该总线构成的工业控制机可以长期可靠地工作在恶劣环境之下。

(4) STD 总线结构简单,并且能够支持多微处理器系统,是一种规模小并且性能好的系统总线。

(5) STD 总线系统模板品种齐全,价格低廉,性能良好,易于选用,为推广 STD 总线

产品创造了有利条件。

3. STD 总线信号定义

表 14-1 列出了 STD 总线信号的定义，总计 56 条信号，其中电源线 10 条，地址线 24 条，数据线 16 条，控制线 22 条，地址线与数据线采用分时复用工作方式。

表 14-1 STD 总线引脚功能

总线类别	元 件 面			焊 接 面		
	引脚	信号名称	功能	引脚	信号名称	功能
逻辑电源	1	V _{CC}	逻辑电源+5V	2	V _{CC}	逻辑电源+5V
	3	GND	逻辑地	4	GND	逻辑地
	5	VBB.1/VBAT	偏置 1/电池	6	VBB.2/VBAT	偏置 2/电池
数据总线	7	D ₃ /A ₁₉	数据总线/地址扩展总线	8	D ₇ /A ₂₃	数据总线/地址扩展总线
	9	D ₂ /A ₁₈		10	D ₆ /A ₂₂	
	11	D ₁ /A ₁₇		12	D ₅ /A ₂₁	
	13	D ₀ /A ₁₆		14	D ₄ /A ₂₀	
地址总线	15	A ₇	地址总线	16	A ₁₅ /D ₁₅	地址总线/数据扩展总线
	17	A ₆		18	A ₁₄ /D ₁₄	
	19	A ₅		20	A ₁₃ /D ₁₃	
	21	A ₄		22	A ₁₂ /D ₁₂	
	23	A ₃		24	A ₁₁ /D ₁₁	
	25	A ₂		26	A ₁₀ /D ₁₀	
	27	A ₁		28	A ₉ /D ₉	
	29	A ₀		30	A ₈ /D ₈	
控制总线	31	$\overline{\text{WR}}$	写命令	32	$\overline{\text{RD}}$	读命令
	33	$\overline{\text{IORQ}}$	I/O 地址选择	34	$\overline{\text{MEMRQ}}$	存储器地址选择
	35	$\overline{\text{IOEXP}}$	I/O 扩展	36	$\overline{\text{MEMEX}}$	存储器扩展
	37	$\overline{\text{REFRESH}}$	刷新定时	38	$\overline{\text{MCSYNC}}$	CPU 周期同步
	39	$\overline{\text{STATUS1}}$	CPU 状态	40	$\overline{\text{STATUS0}}$	CPU 状态
	41	$\overline{\text{BUSAK}}$	总线响应	42	$\overline{\text{BUSRQ}}$	总线请求
	43	$\overline{\text{INTAK}}$	中断响应	44	$\overline{\text{INTRQ}}$	中断请求
控制总线	45	$\overline{\text{WAITRQ}}$	等待请求	46	$\overline{\text{NMIEQ}}$	非屏蔽中断
	47	$\overline{\text{SYSRESET}}$	系统复位	48	$\overline{\text{PBRESET}}$	按钮复位
	49	$\overline{\text{CLOCK}}$	处理器时钟	50	$\overline{\text{CNTRL}}$	辅助定时
	51	PCO	优先级链输出	52	PCI	优先级链输入
辅助电源	53	AUXGND	辅助接地	54	AUXGND	辅助接地
	55	AUX+V	电源+12V	56	AUX-V	辅助电源-12V

14.3.2 光电隔离开关量输入接口板设计

1. 概述

该接口板提供 32 路开关量输入功能，具有光电隔离功能，输入端电源由外部提供。电路由 STD 总线、地址译码器、输入接口电路、光电隔离电路和外部输入插座等几部分组成。原理框图如图 14.2 所示，详细原理图如图 14.3 所示。

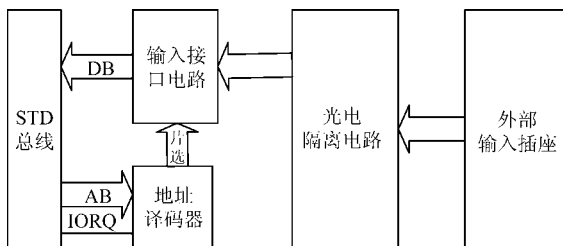


图 14.2 光电隔离开关量输入接口板原理框图

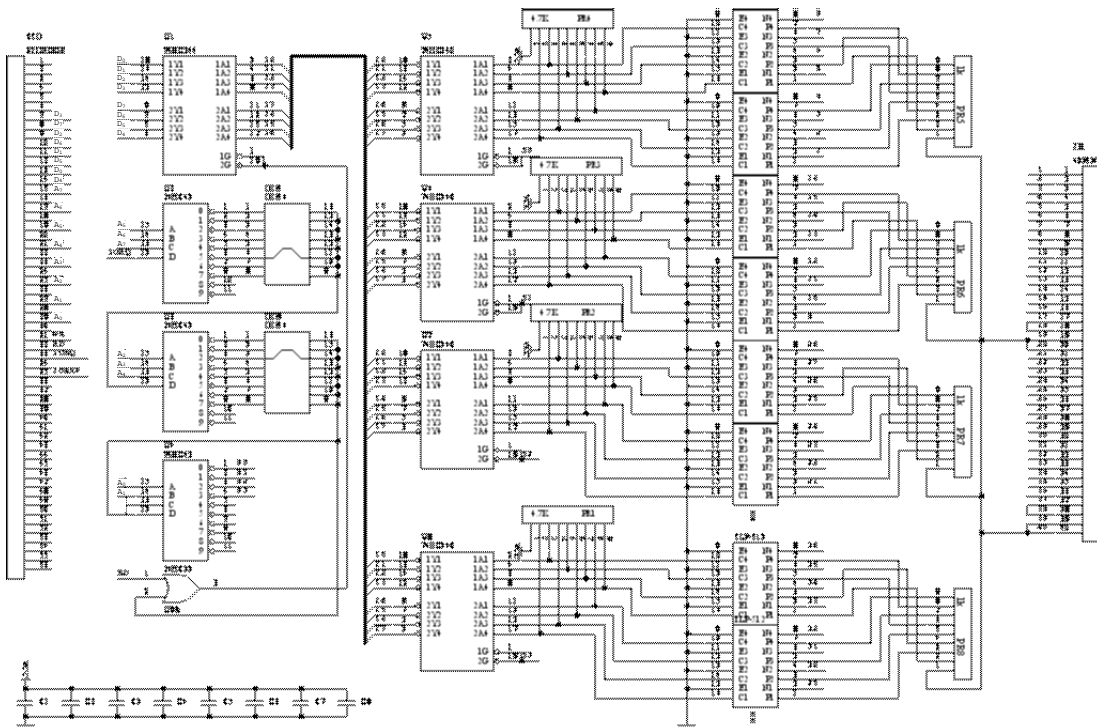


图 14.3 32 路光电隔离开关量输入接口板

2. STD 总线

STD 总线系统采用底板加接口板的结构。底板中有一系列插槽，同时提供电源，各插槽中相同位置的引脚均并联在一起，所以它们的地位是等同的，实际使用时可将接口板插入任意一个槽中。

接口板的一边设计 56 个金手指，其中元件面和焊接面各半，制板时应注意 STD 接口板的尺寸、厚薄、金手指的宽度、深度、间隔等都有一定的标准，绘制时应充分考虑。小批量试制时，56 个金手指应全部制作，以便于调试时使用；设计成熟后，不使用的金手指可空缺，以节省成本。

图 14.3 中 STDEdge 即为 STD 总线插头，主要使用了数据总线(D₀~D₇)，地址总线的低 8 位(A₀~A₇)和部分控制总线。

3. 地址译码器

地址译码器主要由三片 74HC42(U₂, U₃, U₄)和两个 8 位的拨盘组成(DIP1 和 DIP2)，该芯片为 BCD-十进制译码器，其功能表见表 14-2。

表 14-2 74HC42 的功能表

输 入				输 出									
D	C	B	A	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	1	1	1	1	1	1	1	1	1	0	1
0	0	1	0	1	1	1	1	1	1	1	0	1	1
0	0	1	1	1	1	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	1	1	0	1	1	1	1
0	1	0	1	1	1	1	1	0	1	1	1	1	1
0	1	1	0	1	1	1	0	1	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	1	1	1	1	1	1	1
1	0	0	1	0	1	1	1	1	1	1	1	1	1
1	0	1	0	1	1	1	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

拨盘的功能是控制本接口板的地址，当一个系统要使用多个本接口板时，通过拨盘左右连接的情况不同可设置成不同的地址，以防止地址冲突。以下分析时假设 DIP1 中的 6 和 11 连通，DIP2 中的 3 和 14 连通，如图 14.3 所示。应该指出 DIP1 和 DIP2 拨盘中都只能连通一个。

如图，译码电路最终输出为 S₀、S₁、S₂ 和 S₃，要使其中有一个有效必须保证 U₄ 的 D =0；而 U₄ 的 D 又由 U₃ 的 2 输出提供(DIP2 如图连接时)，要使 U₃ 的 2 输出为 0，则要求 U₃ 的 D、C、B、A 引脚输入为 0010(即地址总线中的 A₄、A₃、A₂=010)；同样 U₃ 的 D 又由 U₂ 的 5 输出提供，要使 U₂ 的 5 输出为 0，则要求 U₂ 的 D、C、B、A 引脚输入为 0101(即

地址总线中的 A_7 、 A_6 、 $A_5=101$)。综上,如图连接时 $A_7\sim A_2$ 必须为 101010,而 A_1 、 A_0 的不同选择可控制 $S_0\sim S_3$ 中何者有效,即 S_0 、 S_1 、 S_2 、 S_3 对应的地址为 $A8H(10101000H)$ 、 $A9H(10101001B)$ 、 $AAH(10101010B)$ 和 $ABH(10101011B)$,使用时还要保证 $\overline{IORQ}$ 和 $\overline{RD}$ 有效,即只有执行端口读操作时才有效,对应的指令为 IN 指令。

4. 输入接口电路

输入接口电路由 4 片 74HC240 和 1 片 74HC244 组成。其中 74HC240 是 8 三态反向缓冲器/线驱动器,1G 和 2G 为两组信号的三态控制端。当 $G=0$ 时,对应组信号由 A 反向后流向 Y;当 $G=1$ 时,Y 和 A 之间处于高阻态。使用中,G 由译码器输出 $S_0\sim S_3$ 控制,这样只有在相应端口地址读操作时数据才能有 A 流向 Y,其余时间处于高阻态。

74LS244 是 8 三态缓冲器/驱动器,其作用实现信号整形、驱动和隔离。当执行对 $S_0\sim S_3$ 端口读操作时,它的 G 端必为 0,使信号最终传输到数据总线中,此时再由正在执行的 IN 指令将一组 8 位信息读入到 AL 寄存器,供程序使用。

5. 光电隔离

光电隔离电路主要由 8 片 4 光耦 TLP-512-4 和电阻组成(PR1~PR8)组成。为简化电路布线,采用了排阻,单路的电路原理如图 14.4 所示。必须注意在光耦两边的电源和地不能有任何电路连接,信号是通过光进行传输的,否则就起不到隔离作用了。

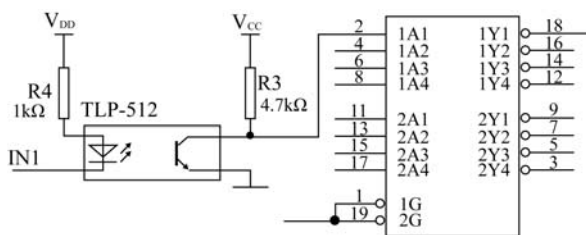


图 14.4 光电隔离原理图(1 路)

6. 外部输入

一般情况下,外部信号可由接线排输入。本电路中,因输入线较多,故采用排线将其引到接线排中,再与实际输入信号连接。接线方式应考虑方便性和可靠性。另外,考虑开关量的电平不同,光耦使用的输入端电源由外部输入(JP1 的 19、20、39、40 引脚),以满足不同用户要求。

7. 接口程序设计及其他

只要对相应的端口地址进行读操作,即可实现开关量的输入,例 14.1 程序段实现 32 位开关量的读操作,并将结果存放在 A, B, C, D 四个字节单元中(每个单元可存放 8 位开关状态,用 0 或 1 表示)。

【例 14.1】 32 位开关量读操作程序及其应用;端口地址和变量定义:

DATA	SEGMENT	
IO_PORT0	EQU	00A8H
IO_PORT1	EQU	00A9H

```

IO_PORT2    EQU        00AAH
IO_PORT3    EQU        00ABH
A           DB         0
B           DB         0
C           DB         0
D           DB         0
DATA        ENDS
; 程序
CODE        SEGMENT
ASSUME      DS: DATA, CS: CODE
START:      MOV         AX, DATA
            MOV         DS, AX
            CALL        PROC          ; 调用读接口子程序
            MOV         AH, 4CH
            INT         21H
PROC:       MOV         DX, IO_PORT0 ; DX 指向第一个端口, 前 8 位
            IN          AL, DX        ; 读入前 8 位的开关量状态
            MOV         A, AL         ; 存放
; 前 8 位处理完毕
            MOV         DX, IO_PORT1 ; DX 指向第二个端口
            IN          AL, DX        ; 读入 8 位的开关量状态
            MOV         B, AL         ; 存放
; 前 16 位处理完毕
            MOV         DX, IO_PORT2 ; DX 指向第三个端口
            IN          AL, DX        ; 读入 8 位的开关量状态
            MOV         C, AL         ; 存放
; 前 24 位处理完毕
            MOV         DX, IO_PORT3 ; DX 指向第四个端口
            IN          AL, DX        ; 读入 8 位的开关量状态
            MOV         D, AL         ; 存放
; 32 位处理完毕
            RET
CODE        ENDS
            END          START

```

调试时, 可在输入端接不同的逻辑状态, 执行上述程序, 查看 A、B、C、D 各单元的值是否与所接的状态相符合, 如不符合, 则应检查硬件电路。以上程序可作为该接口板的基本 I/O 模块, 所有通过本接口的操作, 均由本程序完成。当 DIP1、DIP2 拨盘连接的位置不同时(即选择其他端口地址时), 只需修改 EQU 定义参数即可, 程序不变。

为保证电源的稳定性, 滤除高频干扰, 各集成电路的电源和地之间应接去耦电容, 一般可选用 $0.01\mu\text{F} \sim 0.1\mu\text{F}$ 的独石电容, 原理图中用 C1~C8 表示。

14.3.3 8 路 12 位 A/D 转换接口板设计

1. 概述

该接口板提供 8 路 A/D 转换, 输入模拟电压幅度为 $-5\text{V} \sim +5\text{V}$ 。电路由 STD 总线、总线驱动、地址译码器、输出锁存器、输入接口电路、多路模拟切换电路和 AD574 等几部分组成。原理如图 14.5 所示, 详细原理图如图 14.6 所示。

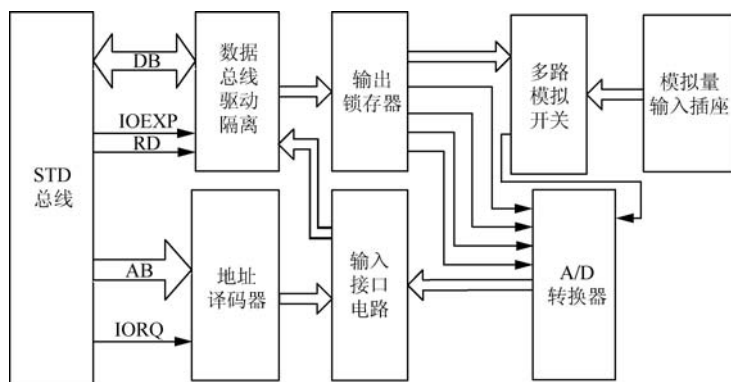


图 14.5 8 路 12 位 A/D 转换接口板原理框图

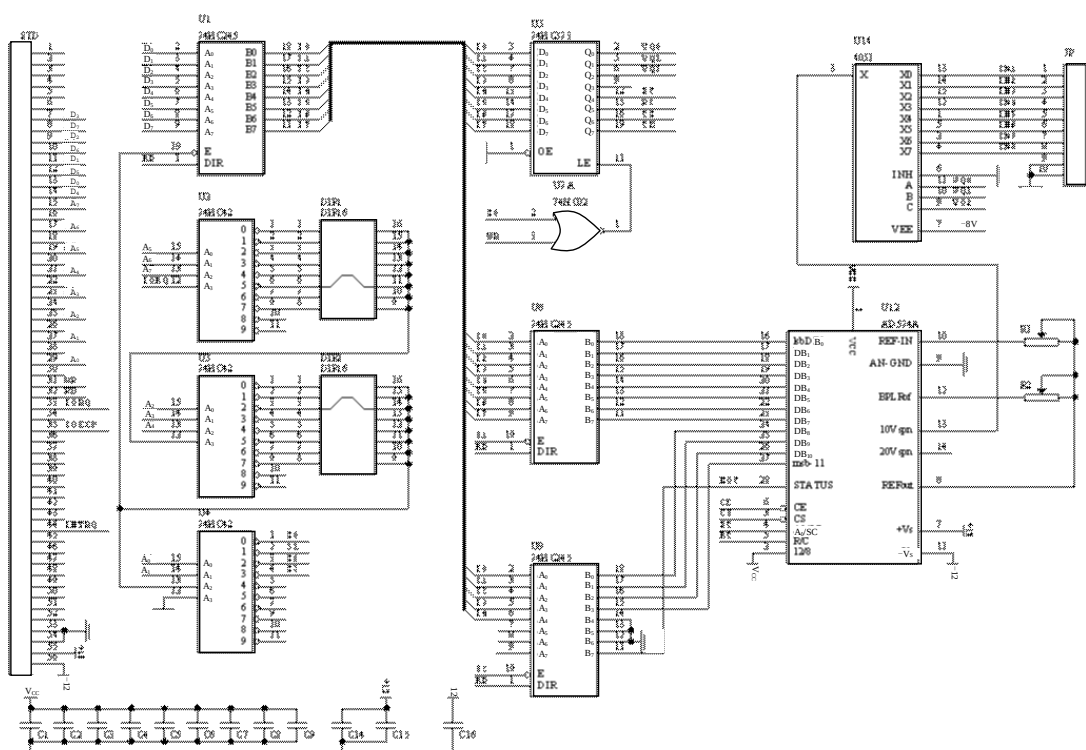


图 14.6 8 路 12 位 A/D 转换接口板原理图

2. 12 位 A/D 转换器 AD574A 简介

AD574A 是美国模拟器件公司(Analog Devices)生产的 12 位逐次逼近型快速 A/D 转换器。转换速度最大为 35 μ s，转换精度 $\leq 0.05\%$ ，是目前我国市场应用最广泛、价格适中的 A/D 转换器。AD574A 片内配有三态输出缓冲电路，因而可直接与各种典型的 8 位或 16 位的微处理器相连，而无须附加逻辑接口电路，且能与 CMOS 及 TTL 电平兼容。由于 AD574A 片内包含高精度的参考电压源和时钟电路，这使它在不需要任何外部电路和时钟信号的情况下，完成一切 A/D 转换功能，应用非常方便。

AD574A 的主要特性概述如下：

(1) 非线性误差：AD574AJ 为 ± 1 LSB；AD574AK 为 $\pm 1/2$ LSB。

(2) 转换速度：最大转换时间 $35\mu\text{S}$ ，属于中档速度。

(3) 输入模拟信号范围可为 $0\sim+10\text{V}$ ， $0\sim+20\text{V}$ ，也可为双极性 $\pm 5\text{V}$ 或 $\pm 10\text{V}$ 。

(4) AD574A 有两个模拟输入端，分别用于不同的电压范围： 10V_{IN} 适用于 $\pm 5\text{V}$ 的模拟输入， 20V_{IN} 适用于 $\pm 10\text{V}$ 的模拟输入。输出为 12 位，即 $\text{DB}_0\sim\text{DB}_{11}$ 。

(5) 利用不同的控制信号，既可实现高精度的 12 位变换，又可作快速的 8 位转换。转换后的数据有两种读出方式：12 位一次输出；8 位，4 位分两次输出。设有三态输出缓冲器，可直接与 8 位或 16 位的微处理器接口。

(6) 需三组电源： $+5\text{V}$ ， $V_{\text{CC}}(+12\sim+15\text{V})$ ， $V_{\text{EE}}(-12\text{V}\sim-15\text{V})$ 。由于转换精度高，所提供电源必须有良好的稳定性，并加以充分滤波，以防止高频噪声的干扰。

(7) 内设高精度的参考电压(10.00V)，只需外接一只适当阻值的电阻，便可向 DAC 部分的解码网络提供 I_{REF} ，转换操作所需的时钟信号由内部提供，不需外接任何元器件。

(8) 低功耗：典型功耗为 390mW 。

AD574A 为 28 引脚双列直插式封装，其引脚配置参见图 14.6 的电路原理图。各引脚功能见表 14-3。

表 14-3 AD574A 的引脚功能

引脚名称	引脚号	功 能
V_{CC}	1	数字逻辑部分的电源 $+5\text{V}$
$12/\bar{8}$	2	数据输出格式选择信号引脚。当该引脚为 1 时，双字节输出，即 12 条数据线同时有效输出，当该引脚为 0 时，为单字节输出，即只有高 8 位或低 4 位有效
$\overline{\text{CS}}$	3	片选信号端，低电平有效
A_0/SC	4	字节选择控制线。在转换期间： $A_0/\text{SC}=0$ ，AD574A 进行全 12 位转换，在读出期间：当 $A_0/\text{SC}=0$ 时，高 8 位数据有效； $A_0/\text{SC}=1$ 、低 4 位数据有效，中间 4 位为“0”，高 4 位为三态。因此当采用两次读出 12 位数据时，应遵循左对齐原则
$\text{R}/\overline{\text{C}}$	5	读数据/转换控制信号，当该引脚为 1 时，ADC 转换结果的数据允许被读取；当该引脚为 0，则允许启动 A/D 转换
CE	6	启动转换信号，高电平有效。可作为 A/D 转换启动或读数据的信号
V_{CC} 、 V_{EE}	7、11	模拟部分供电的正电源和负电源，为 $\pm 12\text{V}$ 或 $\pm 15\text{V}$
REF OUT	8	10V 内部参考电压输出端
REF IN	10	内部解码网络所需参考电压输入端
BPLROF	12	补偿调整。接至正负可调的分压网络，以调整 ADC 输出的零点
10V_{IN} 、 20V_{IN}	13、14	模拟量 10V 及 20V 量程的输入端口，信号的一端接至 AG 引脚
DG	15	数字公共端(数字地)
AG	9	模拟公共端(模拟地)。它是 AD574A 的内部参考点，必须与系统的模拟参考点相连。为了在高数字噪声含量的环境中从 AD574A 获得高精度的性能，AG 和 DG 在封装时已连接在一起，在某些情况下，AG 可在最方便的地方与参考点相连

续表

引脚名称	引脚号	功 能
DB ₀ ~DB ₁₁	16~27	数字量输出
STATUS	28	输出状态信号引脚。转换开始时，STATUS 达到高电平，转换过程中保持高电平。转换完成时返回到低电平。STS 可以作为状态信息被 CPU 查询，也可以用它的下沿向 CPU 发中断申请，通知 A/D 转换已完成，CPU 可以读取转换结果

3. 数据总线驱动/隔离电路

数据总线驱动/隔离电路由 74HC245(U1)构成，其主要作用是驱动数据总线，提高 STD 数据总线的带负载能力。有了该电路后，从 STD 总线上看，一个接口板只相当于一个负载。由于本接口板中数据的流向是双向的，因此用 RD 控制传输方向；使能端 E 由译码器输出控制，只有对本接口板的端口进行读写时 U1 才进入低阻态，其他操作均处于高阻态。

4. 地址译码器

原理同上节所述，拨盘 DIP1、DIP2 如图连接时，各选通线的地址和控制对象见表 14-4。

表 14-4 选通线的地址和控制对象

选通线名称	控制对象	地 址
S0	输出锁存器 74HC373(U5)	00A8H
S1	输入接口 74HC245(U8)	00A9H
S2	输入接口 74HC245(U9)	00AAH
S3	未使用	00ABH

5. 输出锁存器

输出锁存器由 74HC373 组成，其功能是在锁存信号有效时，锁存数据总线的状态。其输出由两部分组成，一是 VQ0~VQ2，作为多路模拟开关的控制信号；二是 CE、CS、SC 和 RC，作为 AD574 的控制信号。

6. 输入接口电路

输入接口电路由两片 74HC245 芯片组成，其中 U8 用于低 8 位数据输入，U9 的低 4 位用于高 4 位数据输入，最高位用于转换结束信号(EOC)位的读入。其原理同 14.3.1 节所述，请读者自行分析。

7. A/D 转换器

模拟输入采用了双极性接法，从 10V_{IN} 输入，则模拟电压的范围为-5V~+5V。使用时 R1，R2 应采用 100Ω 的多圈电位器，±12 电源输入端应加去耦电容(C15、C16)。

控制端输入信号 CS、CE、A0/SC、RC 由输出锁存器(U5)的输出端高 4 位控制，STS(EOC)通过输入口读入，以供 CPU 判断。控制时序如图 14.7 和 14.8 所示。

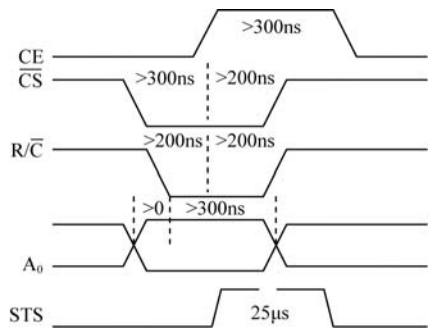


图 14.7 AD574 启动转换时序图

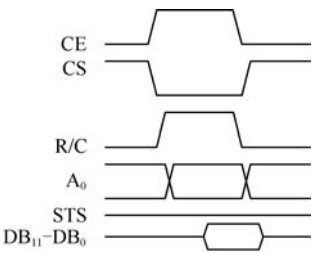


图 14.8 AD574 读周期时序图

8. 模拟多路开关

模拟多路开关采用 1 片 CD4051, CD4051 是单 8 通道数字控制模拟开关, 有三个二进制控制输入端 A、B、C 和 INH 输入, 具有低导通阻抗和很低的截止漏电流。幅值为 4.5~20V 的数字信号可控制峰—峰值至 20V 的模拟信号。例如, 若 $V_{DD}=+5V$, $V_{SS}=0$, $V_{EE}=-5V$, 则 0~5V 的数字信号可控制-5~+5V 的模拟信号。这些开关电路在整个 $V_{DD} \sim V_{SS}$ 和 $V_{DD} \sim V_{EE}$ 电源范围内具有极低的静态功耗, 与控制信号的逻辑状态无关。当 INH 输入端=“1”时, 所有的通道截止。三位二进制信号(A、B、C)选通 8 通道中的一通道, 可连接该输入端至输出。其功能表见表 14-5。

表 14-5 CD4051 功能表

控制输入				模拟输出
INH	C	B	A	X
0	0	0	0	X0
0	0	0	1	X1
0	0	1	0	X2
0	0	1	1	X3
0	1	0	0	X4
0	1	0	1	X5
0	1	1	0	X6
0	1	1	1	X7
1	×	×	×	无

各引脚功能如下:

C、B、A 地址控制端; X0~X7 模拟输入/输出端; INH 禁止端; X 模拟公共输出/输入端; V_{DD} 正电源; V_{EE} 模拟信号地; V_{SS} 数字信号地。

使用时, 由输出锁存器的输出(VQ2~VQ0)控制各 CD4051 的数字控制端, 如选择 IN7 通道时, VQ2~VQ0=110。

9. 程序设计

对于一路模拟信号的转换, 应包括三个阶段, 首先通过输出锁存器, 控制多路模拟开

关,选中该路模拟信号;其次,根据 A/D 控制时序,启动 A/D 转换;再次,通过输入接口读 STS(EOC)信号,等待 A/D 转换结束;最后,根据 A/D 控制时序,读入 12 位转换结果。

【例 14.2】对 IN1~IN8 模拟信号进行转换,结果存放在 12 个以 BUF 为首地址的字单元中。

; 端口地址和变量定义

DATA SEGMENT

DATA_OUT EQU 00A8H

DATA_INL EQU 00A9H

DATA_INH EQU 00AAH

START_AD EQU 10000000B ; 启动 AD 控制字,从最高位起分别对应
; CE、CS、R/C 和 SC

STADRT_ADE EQU 01000000B ; 启动 AD 结束

READ_AD EQU 10100000B ; 读 AD 数据控制字

BUF DW 12 DUP(0)

DATA ENDS

; 程序

CODE SEGMENT

ASSUME DS: DATA, CS: CODE

BEGIN: MOV AX, DATA

MOV DS, AX

CALL PROC ; 调用 AD 转换子程序

MOV AH, 4CH

INT 21H

PROC: MOV DX, DATA_OUT

MOV CX, 8 ; 共 8 路

MOV BL, 0 ; 指向首路

LEA DI, BUF ; DI 指向缓冲区首地址

NEXT: MOV AL, BL

ADD AL, START_AD ; 启动 AD 控制字与模拟通道选择控制字合并
; 为一个字节,以便在同一输出端口输出

MOV DX, DATA_OUT ; 指向数据锁存器端口地址

OUT DX, AL ; 输出到锁存器

NOP ; 延时,根据不同机器应保证大于 300ns

MOV AL, BL

ADD AL, STADRT_ADE ; 结束启动过程

OUT DX, AL

MOV DX, DATA_INH

AGAIN: IN AL, DX ; 读入 STS (EOC) 到 AL 的最高位

ANL AL, 10000000B

JNZ AGAIN ; 未完成 (EOC=1) 则等待

MOV DX, DATA_OUT

MOV AL, BL

ADD AL, READ_AD ; 形成读 AD 数据控制字

OUT DX, AL ; 输出控制字

MOV DX, DATA_INL ; 指向低位数据读入端口

IN AL, DX ; 读入低 8 位数据

MOV [DI], AL ; 存入缓冲区

INC DI ; 指针加 1

MOV DX, DATA_INH ; 指向高位数据读入端口

IN AL, DX ; 读入高 4 位数据

AND AL, 0FH ; 屏蔽高 4 位,结果在底 4 位中

```
MOV      [DI], AL      ; 存入缓冲区
INC      DI             ; 指向下一个字单元
MOV      AL, BL
ADD      AL, START_ADE ; 结束
OUT      DX, AL
INC      BL             ; 指向下一模拟通道
LOOP     NEXT
RET
CODE     ENDS
END      BEGIN
```

调试时,可在输入端接不同的电压值,执行上述程序,查看各单元的值是否与理论值接近,如不符合,应检查硬件电路。以上程序可作为该接口板的基本 I/O 模块,所有通过本接口的操作,均由本程序完成。当 DIP1、DIP2 拨盘连接的位置不同时(即选择其他端口地址时),只需修改 EQU 定义参数即可,程序不变。

14.4 小 结

学习本课程的最终目的是能把它应用到实际领域中去。由于微处理机的应用领域很广,技术要求各不相同,因此应用系统的软、硬件设计各种各样,但总体设计方法和研制步骤却具有共性。本章针对大多数应用场合,所使用的总线设计结构,简要介绍了微机应用系统的一般开发、研制方法,以及常用的模块电路的选用,并通过应用实例,说明了具体的硬件设计和软件设计,供读者参考。

14.5 习 题

1. 常见的微机应用系统可分为哪几种类型?各有何特点?
2. 微机应用系统设计原则主要有哪些?
3. 简述微机系统设计的过程和内容。
4. 在微机系统设计中,常用的模块电路有哪些?各有何作用?
5. 找几个微机应用系统设计的原理图,分析其设计思路。